Tcl/Tk 8.6

Reference manual pages

2026-10-27

Contents

Tcl **7**
Tcl 7
after 10
append 12
apply 13
array 14
bgerror 17
binary 18
break 32
case 32
catch 33
cd 35
chan 35
class 48
clock 50
close 64
concat 65
continue 66
copy 67
coroutine 68
dde 71
define 73
dict 80
encoding 87
eof 88
error 89
eval 90
exec 91
exit 98
expr 98
fblocked 105
fconfigure 106
fcopy 110
file 113
fileevent 120
filename 122
flush 124
for 125
foreach 126
format 127
gets 131
glob 132
global 135
history 136

http 137
if 150
incr 151
info 151
interp 161
join 174
lappend 175
lassign 175
library 176
lindex 182
linsert 183
list 184
llength 185
lmap 186
load 187
lrange 190
lrepeat 190
lreplace 191
lreverse 192
lsearch 193
lset 196
lsort 198
mathfunc 201
mathop 205
memory 209
msgcat 211
my 220
namespace 221
next 234
object 237
open 239
package 245
packagens 249
pid 250
pkgMkIndex 251
platform 254
platform_shell 255
prefix 256
proc 257
puts 259
pwd 260
re_syntax 261
read 271
refchan 272
regexp 279
registry 281

regsub 284
rename 286
return 287
safe 291
scan 296
seek 300
self 301
set 303
socket 304
source 308
split 309
string 310
subst 317
switch 318
tailcall 321
tclsh 322
tcltest 324
tclvars 340
tell 347
throw 347
time 348
timerate 349
tm 351
trace 355
transchan 361
try 363
unknown 364
unload 366
unset 368
update 369
uplevel 370
upvar 371
variable 373
vwait 374
while 379
zlib 380
Tk **386**
bell 386
bind 386
bindtags 397
bitmap 398
busy 400
button 403
canvas 406
checkbutton 432
chooseColor 436

chooseDirectory 437
clipboard 438
colors 440
console 460
cursors 462
destroy 465
dialog 466
entry 467
event 475
focus 483
focusNext 485
font 486
fontchooser 491
frame 494
getOpenFile 496
grab 499
grid 501
image 507
keysyms 508
label 544
labelframe 546
listbox 548
loadTk 556
lower 557
menu 557
menubar 568
menubutton 569
message 571
messageBox 573
option 575
optionMenu 577
options 577
pack-old 583
pack 585
palette 589
panedwindow 590
photo 595
place 602
popup 605
radiobutton 606
raise 610
scale 611
scrollbar 614
selection 619
send 622
spinbox 624

text 632
tk 665
tk_mac 667
tkerror 671
tkvars 672
tkwait 673
toplevel 674
winfo 676
wish 681
wm 684
Ttk 696
ttk_button 696
ttk_checkbutton 697
ttk_combobox 699
ttk_entry 701
ttk_frame 708
ttk_image 709
ttk_intro 710
ttk_label 713
ttk_labelframe 714
ttk_menubutton 715
ttk_notebook 716
ttk_panedwindow 721
ttk_progressbar 723
ttk_radiobutton 725
ttk_scale 726
ttk_scrollbar 728
ttk_separator 731
ttk_sizegrip 732
ttk_spinbox 733
ttk_style 735
ttk_treeview 737
ttk_vsapi 745
ttk_widget 746
License 752
Tcl license terms 752
Tk license terms 753

Tcl

Tcl

Keywords: Tcl, backslash, command, comment, script, substitution, variable

Name

Tcl - Tool Command Language

Synopsis

Summary of Tcl language syntax.

Description

The following rules define the syntax and semantics of the Tcl language:

[1] Commands.
A Tcl script is a string containing one or more commands. Semi-colons and newlines are command separators unless quoted as described below. Close brackets are command terminators during command substitution (see below) unless quoted.

[2] Evaluation.
A command is evaluated in two steps. First, the Tcl interpreter breaks the command into *words* and performs substitutions as described below. These substitutions are performed in the same way for all commands. Secondly, the first word is used to locate a command procedure to carry out the command, then all of the words of the command are passed to the command procedure. The command procedure is free to interpret each of its words in any way it likes, such as an integer, variable name, list, or Tcl script. Different commands interpret their words differently.

[3] Words.
Words of a command are separated by white space (except for newlines, which are command separators).

[4] Double quotes.
If the first character of a word is double-quote (""") then the word is terminated by the next double-quote character. If semi-colons, close brackets, or white space characters (including newlines) appear between the quotes then they are treated as ordinary characters and included in the word. Command substitution, variable substitution, and backslash substitution are performed on the characters between the quotes as described below. The double-quotes are not retained as part of the word.

[5] Argument expansion.
If a word starts with the string "{*}" followed by a non-whitespace character, then the leading "{*}" is removed and the rest of the word is parsed and substituted as any other word. After substitution, the word is parsed as a list (without command or variable substitutions; backslash substitutions are performed as is normal for a list and individual internal words may be surrounded by either braces or double-quote characters), and its words are added to the command being substituted. For instance, "cmd a {*}{b [c]} d {*}{$e f {g h}}" is equivalent to "cmd a b {[c]} d {$e} f {g h}".

[6] Braces.
If the first character of a word is an open brace ("{") and rule [5] does not apply, then the

word is terminated by the matching close brace ("}"). Braces nest within the word: for each additional open brace there must be an additional close brace (however, if an open brace or close brace within the word is quoted with a backslash then it is not counted in locating the matching close brace). No substitutions are performed on the characters between the braces except for backslash-newline substitutions described below, nor do semi-colons, newlines, close brackets, or white space receive any special interpretation. The word will consist of exactly the characters between the outer braces, not including the braces themselves.

[7] Command substitution.

If a word contains an open bracket ("[") then Tcl performs *command substitution*. To do this it invokes the Tcl interpreter recursively to process the characters following the open bracket as a Tcl script. The script may contain any number of commands and must be terminated by a close bracket ("]"). The result of the script (i.e. the result of its last command) is substituted into the word in place of the brackets and all of the characters between them. There may be any number of command substitutions in a single word. Command substitution is not performed on words enclosed in braces.

[8] Variable substitution.

If a word contains a dollar-sign ("$") followed by one of the forms described below, then Tcl performs *variable substitution*: the dollar-sign and the following characters are replaced in the word by the value of a variable. Variable substitution may take any of the following forms:

$*name*

Name is the name of a scalar variable; the name is a sequence of one or more characters that are a letter, digit, underscore, or namespace separators (two or more colons). Letters and digits are *only* the standard ASCII ones (**0**–**9**, **A**–**Z** and **a**–**z**).

$*name*(*index*)

Name gives the name of an array variable and *index* gives the name of an element within that array. *Name* must contain only letters, digits, underscores, and namespace separators, and may be an empty string. Letters and digits are *only* the standard ASCII ones (**0**–**9**, **A**–**Z** and **a**–**z**). Command substitutions, variable substitutions, and backslash substitutions are performed on the characters of *index*.

${*name*}

Name is the name of a scalar variable or array element. It may contain any characters whatsoever except for close braces. It indicates an array element if *name* is in the form "*arrayName*(*index*)" where *arrayName* does not contain any open parenthesis characters, "(", or close brace characters, "}", and *index* can be any sequence of characters except for close brace characters. No further substitutions are performed during the parsing of *name*.

There may be any number of variable substitutions in a single word. Variable substitution is not performed on words enclosed in braces.

Note that variables may contain character sequences other than those listed above, but in that case other mechanisms must be used to access them (e.g., via the set command's single-argument form).

[9] Backslash substitution.

If a backslash ("\") appears within a word then *backslash substitution* occurs. In all cases but those described below the backslash is dropped and the following character is treated as an ordinary character and included in the word. This allows characters such as double quotes,

close brackets, and dollar signs to be included in words without triggering special processing. The following table lists the backslash sequences that are handled specially, along with the value that replaces each sequence.

\a
Audible alert (bell) (Unicode U+000007).

\b
Backspace (Unicode U+000008).

\f
Form feed (Unicode U+00000C).

\n
Newline (Unicode U+00000A).

\r
Carriage-return (Unicode U+00000D).

\t
Tab (Unicode U+000009).

\v
Vertical tab (Unicode U+00000B).

whiteSpace
A single space character replaces the backslash, newline, and all spaces and tabs after the newline. This backslash sequence is unique in that it is replaced in a separate prepass before the command is actually parsed. This means that it will be replaced even when it occurs between braces, and the resulting space will be treated as a word separator if it is not in braces or quotes.

\\
Backslash ("\").

ooo
The digits *ooo* (one, two, or three of them) give a eight-bit octal value for the Unicode character that will be inserted, in the range *000–377* (i.e., the range U+000000–U+0000FF). The parser will stop just before this range overflows, or when the maximum of three digits is reached. The upper bits of the Unicode character will be 0.

\x*hh*
The hexadecimal digits *hh* (one or two of them) give an eight-bit hexadecimal value for the Unicode character that will be inserted. The upper bits of the Unicode character will be 0 (i.e., the character will be in the range U+000000–U+0000FF).

\u*hhhh*
The hexadecimal digits *hhhh* (one, two, three, or four of them) give a sixteen-bit hexadecimal value for the Unicode character that will be inserted. The upper bits of the Unicode character will be 0 (i.e., the character will be in the range U+000000–U+00FFFF).

\U*hhhhhhhh*
The hexadecimal digits *hhhhhhhh* (one up to eight of them) give a twenty-one-bit hexadecimal value for the Unicode character that will be inserted, in the range U+000000–U+10FFFF. The parser will stop just before this range overflows, or when the maximum of eight digits is reached. The upper bits of the Unicode character will be 0.

The range U+010000–U+10FFFD is reserved for the future.

Backslash substitution is not performed on words enclosed in braces, except for backslash-newline as described above.

[10] Comments.
If a hash character ("#") appears at a point where Tcl is expecting the first character of the first word of a command, then the hash character and the characters that follow it, up through the next newline, are treated as a comment and ignored. The comment character only has significance when it appears at the beginning of a command.

[11] Order of substitution.
Each character is processed exactly once by the Tcl interpreter as part of creating the words of a command. For example, if variable substitution occurs then no further substitutions are performed on the value of the variable; the value is inserted into the word verbatim. If command substitution occurs then the nested command is processed entirely by the recursive call to the Tcl interpreter; no substitutions are performed before making the recursive call and no additional substitutions are performed on the result of the nested script.

Substitutions take place from left to right, and each substitution is evaluated completely before attempting to evaluate the next. Thus, a sequence like

```
set y [set x 0][incr x][incr x]
```

will always set the variable *y* to the value, *012*.

[12] Substitution and word boundaries.
Substitutions do not affect the word boundaries of a command, except for argument expansion as specified in rule [5]. For example, during variable substitution the entire value of the variable becomes part of a single word, even if the variable's value contains spaces.

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

after

Keywords: after, cancel, delay, idle callback, sleep, time

Name

after - Execute a command after a time delay

Synopsis

after *ms*
after *ms* ?*script script script* ...?
after cancel *id*
after cancel *script script script ...*
after idle ?*script script script* ...?
after info ?*id*?

Description

This command is used to delay execution of the program or to execute a command in background sometime in the future. It has several forms, depending on the first argument to the command:

after *ms*
Ms must be an integer giving a time in milliseconds. A negative number is treated as 0. The command sleeps for *ms* milliseconds and then returns. While the command is sleeping the application does not respond to events.

after *ms* ?*script script script ...*?
In this form the command returns immediately, but it arranges for a Tcl command to be executed *ms* milliseconds later as an event handler. The command will be executed exactly once, at the given time. The delayed command is formed by concatenating all the *script* arguments in the same fashion as the concat command. The command will be executed at global level (outside the context of any Tcl procedure). If an error occurs while executing the delayed command then the background error will be reported by the command registered with interp bgerror. The **after** command returns an identifier that can be used to cancel the delayed command using **after cancel**. A *ms* value of 0 (or negative) queues the event immediately with priority over other event types (if not installed withn an event proc, which will wait for next round of events).

after cancel *id*
Cancels the execution of a delayed command that was previously scheduled. *Id* indicates which command should be canceled; it must have been the return value from a previous **after** command. If the command given by *id* has already been executed then the **after cancel** command has no effect.

after cancel *script script ...*
This command also cancels the execution of a delayed command. The *script* arguments are concatenated together with space separators (just as in the concat command). If there is a pending command that matches the string, it is canceled and will never be executed; if no such command is currently pending then the **after cancel** command has no effect.

after idle *script* ?*script script ...*?
Concatenates the *script* arguments together with space separators (just as in the concat command), and arranges for the resulting script to be evaluated later as an idle callback. The script will be run exactly once, the next time the event loop is entered and there are no events to process. The command returns an identifier that can be used to cancel the delayed command using **after cancel**. If an error occurs while executing the script then the background error will be reported by the command registered with interp bgerror.

after info ?*id*?
This command returns information about existing event handlers. If no *id* argument is supplied, the command returns a list of the identifiers for all existing event handlers created by the **after** command for this interpreter. If *id* is supplied, it specifies an existing handler; *id* must have been the return value from some previous call to **after** and it must not have triggered yet or been canceled. In this case the command returns a list with two elements. The first element of the list is the script associated with *id*, and the second element is either **idle** or **timer** to indicate what kind of event handler it is.

The **after** *ms* and **after idle** forms of the command assume that the application is event driven: the delayed commands will not be executed unless the application enters the event loop. In applications that are not normally event-driven, such as tclsh, the event loop can be entered with the vwait and update commands.

Examples
This defines a command to make Tcl do nothing at all for *N* seconds:

```
proc sleep {N} {
    after [expr {int($N * 1000)}]
}
```

This arranges for the command *wake_up* to be run in eight hours (providing the event loop is active at that time):

```
after [expr {1000 * 60 * 60 * 8}] wake_up
```

The following command can be used to do long-running calculations (as represented here by *::my_calc::one_step*, which is assumed to return a boolean indicating whether another step should be performed) in a step-by-step fashion, though the calculation itself needs to be arranged so it can work step-wise. This technique is extra careful to ensure that the event loop is not starved by the rescheduling of processing steps (arranging for the next step to be done using an already-triggered timer event only when the event queue has been drained) and is useful when you want to ensure that a Tk GUI remains responsive during a slow task.

```
proc doOneStep {} {
    if {[::my_calc::one_step]} {
        after idle [list after 0 doOneStep]
    }
}
doOneStep
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

append
Keywords: append, variable

Name
append - Append to variable

Synopsis
append *varName* ?*value value value* ...?

Description
Append all of the *value* arguments to the current value of variable *varName*. If *varName* does not exist, it is given a value equal to the concatenation of all the *value* arguments. The result of this command is the new value stored in variable *varName*. This command provides an efficient way to build up long variables incrementally. For example, "**append a $b**" is much more efficient than "**set a ab**" if **$a** is long.

Example
Building a string of comma-separated numbers piecemeal using a loop.

```
set var 0
for {set i 1} {$i<=10} {incr i} {
    append var "," $i
}
puts $var
# Prints 0,1,2,3,4,5,6,7,8,9,10
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

apply

Keywords: apply, anonymous function, argument, lambda, procedure

Name

apply - Apply an anonymous function

Synopsis

apply *func* ?*arg1 arg2 ...*?

Description

The command **apply** applies the function *func* to the arguments *arg1 arg2 ...* and returns the result.

The function *func* is a two element list *{args body}* or a three element list *{args body namespace}* (as if the list command had been used). The first element *args* specifies the formal arguments to *func*. The specification of the formal arguments *args* is shared with the proc command, and is described in detail in the corresponding manual page.

The contents of *body* are executed by the Tcl interpreter after the local variables corresponding to the formal arguments are given the values of the actual parameters *arg1 arg2 ...*. When *body* is being executed, variable names normally refer to local variables, which are created automatically when referenced and deleted when **apply** returns. One local variable is automatically created for each of the function's arguments. Global variables can only be accessed by invoking the global command or the upvar command. Namespace variables can only be accessed by invoking the variable command or the upvar command.

The invocation of **apply** adds a call frame to Tcl's evaluation stack (the stack of frames accessed via uplevel). The execution of *body* proceeds in this call frame, in the namespace given by *namespace* or in the global namespace if none was specified. If given, *namespace* is interpreted relative to the global namespace even if its name does not start with "::".

The semantics of **apply** can also be described by:

```
proc apply {fun args} {
    set len [llength $fun]
    if {($len < 2) || ($len > 3)} {
         error "can't interpret \"$fun\" as anonymous function"
    }
    lassign $fun argList body ns
    set name ::$ns::[getGloballyUniqueName]
```

```
    set body0 {
         rename [lindex [info level 0] 0] {}
    }
    proc $name $argList ${body0}$body
    set code [catch {uplevel 1 $name $args} res opt]
    return -options $opt $res
}
```

Examples

This shows how to make a simple general command that applies a transformation to each element of a list.

```
proc map {lambda list} {
    set result {}
    foreach item $list {
        lappend result [apply $lambda $item]
    }
    return $result
}
map {x {return [string length $x]:$x}} {a bb ccc dddd}
      → 1:a 2:bb 3:ccc 4:dddd
map {x {expr {$x**2 + 3*$x - 2}}} {-4 -3 -2 -1 0 1 2 3 4}
      → 2 -2 -4 -4 -2 2 8 16 26
```

The **apply** command is also useful for defining callbacks for use in the trace command:

```
set vbl "123abc"
trace add variable vbl write {apply {{v1 v2 op} {
    upvar 1 $v1 v
    puts "updated variable to \"$v\""
}}}
set vbl 123
set vbl abc
```

Copyright (c) 2006 Miguel Sofer
Copyright (c) 2006 Donal K. Fellows

array

Keywords: array, element names, search

Name

array - Manipulate array variables

Synopsis

array *option arrayName* ?*arg arg* ...?

Description

This command performs one of several operations on the variable given by *arrayName*. Unless

otherwise specified for individual commands below, *arrayName* must be the name of an existing array variable. The *option* argument determines what action is carried out by the command. The legal *options* (which may be abbreviated) are:

array anymore *arrayName searchId*
Returns 1 if there are any more elements left to be processed in an array search, 0 if all elements have already been returned. *SearchId* indicates which search on *arrayName* to check, and must have been the return value from a previous invocation of **array startsearch** . This option is particularly useful if an array has an element with an empty name, since the return value from **array nextelement** will not indicate whether the search has been completed.

array donesearch *arrayName searchId*
This command terminates an array search and destroys all the state associated with that search. *SearchId* indicates which search on *arrayName* to destroy, and must have been the return value from a previous invocation of **array startsearch**. Returns an empty string.

array exists *arrayName*
Returns 1 if *arrayName* is an array variable, 0 if there is no variable by that name or if it is a scalar variable.

array get *arrayName* ?*pattern*?
Returns a list containing pairs of elements. The first element in each pair is the name of an element in *arrayName* and the second element of each pair is the value of the array element. The order of the pairs is undefined. If *pattern* is not specified, then all of the elements of the array are included in the result. If *pattern* is specified, then only those elements whose names match *pattern* (using the matching rules of string match) are included. If *arrayName* is not the name of an array variable, or if the array contains no elements, then an empty list is returned. If traces on the array modify the list of elements, the elements returned are those that exist both before and after the call to **array get**.

array names *arrayName* ?*mode*? ?*pattern*?
Returns a list containing the names of all of the elements in the array that match *pattern*. *Mode* may be one of **-exact**, **-glob**, or **-regexp**. If specified, *mode* designates which matching rules to use to match *pattern* against the names of the elements in the array. If not specified, *mode* defaults to **-glob**. See the documentation for string match for information on glob style matching, and the documentation for regexp for information on regexp matching. If *pattern* is omitted then the command returns all of the element names in the array. If there are no (matching) elements in the array, or if *arrayName* is not the name of an array variable, then an empty string is returned.

array nextelement *arrayName searchId*
Returns the name of the next element in *arrayName*, or an empty string if all elements of *arrayName* have already been returned in this search. The *searchId* argument identifies the search, and must have been the return value of an **array startsearch** command. Warning: if elements are added to or deleted from the array, then all searches are automatically terminated just as if **array donesearch** had been invoked; this will cause **array nextelement** operations to fail for those searches.

array set *arrayName list*
Sets the values of one or more elements in *arrayName*. *list* must have a form like that returned by **array get**, consisting of an even number of elements. Each odd-numbered

element in *list* is treated as an element name within *arrayName*, and the following element in *list* is used as a new value for that array element. If the variable *arrayName* does not already exist and *list* is empty, *arrayName* is created with an empty array value.

array size *arrayName*
Returns a decimal string giving the number of elements in the array. If *arrayName* is not the name of an array then 0 is returned.

array startsearch *arrayName*
This command initializes an element-by-element search through the array given by *arrayName*, such that invocations of the **array nextelement** command will return the names of the individual elements in the array. When the search has been completed, the **array donesearch** command should be invoked. The return value is a search identifier that must be used in **array nextelement** and **array donesearch** commands; it allows multiple searches to be underway simultaneously for the same array. It is currently more efficient and easier to use either the **array get** or **array names**, together with foreach, to iterate over all but very large arrays. See the examples below for how to do this.

array statistics *arrayName*
Returns statistics about the distribution of data within the hashtable that represents the array. This information includes the number of entries in the table, the number of buckets, and the utilization of the buckets.

array unset *arrayName* ?*pattern*?
Unsets all of the elements in the array that match *pattern* (using the matching rules of string match). If *arrayName* is not the name of an array variable or there are no matching elements in the array, no error will be raised. If *pattern* is omitted and *arrayName* is an array variable, then the command unsets the entire array. The command always returns an empty string.

Examples

```
array set colorcount {
    red   1
    green 5
    blue  4
    white 9
}

foreach {color count} [array get colorcount] {
    puts "Color: $color Count: $count"
}
  → Color: blue Count: 4
    Color: white Count: 9
    Color: green Count: 5
    Color: red Count: 1

foreach color [array names colorcount] {
    puts "Color: $color Count: $colorcount($color)"
}
  → Color: blue Count: 4
```

```
    Color: white Count: 9
    Color: green Count: 5
    Color: red Count: 1

foreach color [lsort [array names colorcount]] {
    puts "Color: $color Count: $colorcount($color)"
}
  → Color: blue Count: 4
    Color: green Count: 5
    Color: red Count: 1
    Color: white Count: 9

array statistics colorcount
  → 4 entries in table, 4 buckets
    number of buckets with 0 entries: 1
    number of buckets with 1 entries: 2
    number of buckets with 2 entries: 1
    number of buckets with 3 entries: 0
    number of buckets with 4 entries: 0
    number of buckets with 5 entries: 0
    number of buckets with 6 entries: 0
    number of buckets with 7 entries: 0
    number of buckets with 8 entries: 0
    number of buckets with 9 entries: 0
    number of buckets with 10 or more entries: 0
    average search distance for entry: 1.2
```

Copyright (c) 1993-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

bgerror

Keywords: bgerror, background error, reporting

Name

bgerror - Command invoked to process background errors

Synopsis

bgerror *message*

Description

Release 8.5 of Tcl supports the interp bgerror command, which allows applications to register in an interpreter the command that will handle background errors in that interpreter. In older releases of Tcl, this level of control was not available, and applications could control the handling of background errors only by creating a command with the particular command name **bgerror** in the global namespace of an interpreter. The following documentation describes the interface requirements of the **bgerror** command an application might define to retain compatibility with pre-8.5 releases of Tcl. Applications intending to support only Tcl releases 8.5 and later should

simply make use of interp bgerror.

The **bgerror** command does not exist as built-in part of Tcl. Instead, individual applications or users can define a **bgerror** command (e.g. as a Tcl procedure) if they wish to handle background errors.

A background error is one that occurs in an event handler or some other command that did not originate with the application. For example, if an error occurs while executing a command specified with the after command, then it is a background error. For a non-background error, the error can simply be returned up through nested Tcl command evaluations until it reaches the top-level code in the application; then the application can report the error in whatever way it wishes. When a background error occurs, the unwinding ends in the Tcl library and there is no obvious way for Tcl to report the error.

When Tcl detects a background error, it saves information about the error and invokes a handler command registered by interp bgerror later as an idle event handler. The default handler command in turn calls the **bgerror** command. Before invoking **bgerror**, Tcl restores the errorInfo and errorCode variables to their values at the time the error occurred, then it invokes **bgerror** with the error message as its only argument. Tcl assumes that the application has implemented the **bgerror** command, and that the command will report the error in a way that makes sense for the application. Tcl will ignore any result returned by the **bgerror** command as long as no error is generated.

If another Tcl error occurs within the **bgerror** command (for example, because no **bgerror** command has been defined) then Tcl reports the error itself by writing a message to stderr.

If several background errors accumulate before **bgerror** is invoked to process them, **bgerror** will be invoked once for each error, in the order they occurred. However, if **bgerror** returns with a break exception, then any remaining errors are skipped without calling **bgerror**.

If you are writing code that will be used by others as part of a package or other kind of library, consider avoiding **bgerror**. The reason for this is that the application programmer may also want to define a **bgerror**, or use other code that does and thus will have trouble integrating your code.

Example

This **bgerror** procedure appends errors to a file, with a timestamp.

```
proc bgerror {message} {
    set timestamp [clock format [clock seconds]]
    set fl [open mylog.txt {WRONLY CREAT APPEND}]
    puts $fl "$timestamp: bgerror in $::argv '$message'"
    close $fl
}
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

binary

Keywords: binary, format, scan

Name
binary - Insert and extract fields from binary strings

Synopsis
binary decode *format* ?*-option value* ...? *data*
binary encode *format* ?*-option value* ...? *data*
binary format *formatString* ?*arg arg* ...?
binary scan *string formatString* ?*varName varName* ...?

Description
This command provides facilities for manipulating binary data. The subcommand **binary format** creates a binary string from normal Tcl values. For example, given the values 16 and 22, on a 32-bit architecture, it might produce an 8-byte binary string consisting of two 4-byte integers, one for each of the numbers. The subcommand **binary scan**, does the opposite: it extracts data from a binary string and returns it as ordinary Tcl string values. The binary encode and binary decode subcommands convert binary data to or from string encodings such as base64 (used in MIME messages for example).

Note that other operations on binary data, such as taking a subsequence of it, getting its length, or reinterpreting it as a string in some encoding, are done by other Tcl commands (respectively string range, string length and encoding convertfrom in the example cases). A binary string in Tcl is merely one where all the characters it contains are in the range \u0000-\u00FF.

Binary encode and decode
When encoding binary data as a readable string, the starting binary data is passed to the **binary encode** command, together with the name of the encoding to use and any encoding-specific options desired. Data which has been encoded can be converted back to binary form using **binary decode**. The following formats and options are supported.

base64

The **base64** binary encoding is commonly used in mail messages and XML documents, and uses mostly upper and lower case letters and digits. It has the distinction of being able to be rewrapped arbitrarily without losing information.

During encoding, the following options are supported:

-maxlen *length*
Indicates that the output should be split into lines of no more than *length* characters. By default, lines are not split.

-wrapchar *character*
Indicates that, when lines are split because of the **-maxlen** option, *character* should be used to separate lines. By default, this is a newline character, "\n".

During decoding, the following options are supported:

-strict
Instructs the decoder to throw an error if it encounters any characters that are not strictly part of the encoding itself. Otherwise it ignores them. RFC 2045 calls for base64 decoders to be non-strict.

hex

The **hex** binary encoding converts each byte to a pair of hexadecimal digits that represent the byte value as a hexadecimal integer. When encoding, lower characters are used. When decoding, upper and lower characters are accepted.

No options are supported during encoding. During decoding, the following options are supported:

-strict
Instructs the decoder to throw an error if it encounters whitespace characters. Otherwise it ignores them.

uuencode
The **uuencode** binary encoding used to be common for transfer of data between Unix systems and on USENET, but is less common these days, having been largely superseded by the **base64** binary encoding.

During encoding, the following options are supported (though changing them may produce files that other implementations of decoders cannot process):

-maxlen *length*
Indicates the maximum number of characters to produce for each encoded line. The valid range is 5 to 85. Line lengths outside that range cannot be accommodated by the encoding format. The default value is 61.

-wrapchar *character*
Indicates the character(s) to use to mark the end of each encoded line. Acceptable values are a sequence of zero or more characters from the set { \x09 (TAB), \x0B (VT), \x0C (FF), \x0D (CR) } followed by zero or one newline \x0A (LF). Any other values are rejected because they would generate encoded text that could not be decoded. The default value is a single newline.

During decoding, the following options are supported:

-strict
Instructs the decoder to throw an error if it encounters anything outside of the standard encoding format. Without this option, the decoder tolerates some deviations, mostly to forgive reflows of lines between the encoder and decoder.

Note that neither the encoder nor the decoder handle the header and footer of the uuencode format.

Binary format

The **binary format** command generates a binary string whose layout is specified by the *formatString* and whose contents come from the additional arguments. The resulting binary value is returned.

The *formatString* consists of a sequence of zero or more field specifiers separated by zero or more spaces. Each field specifier is a single type character followed by an optional flag character followed by an optional numeric *count*. Most field specifiers consume one argument to obtain the value to be formatted. The type character specifies how the value is to be formatted. The *count* typically indicates how many items of the specified type are taken from the value. If present, the *count* is a non-negative decimal integer or *, which normally indicates that all of the items in the value are to be used. If the number of arguments does not match the number of

fields in the format string that consume arguments, then an error is generated. The flag character is ignored for **binary format**.

Here is a small example to clarify the relation between the field specifiers and the arguments:

```
binary format d3d {1.0 2.0 3.0 4.0} 0.1
```

The first argument is a list of four numbers, but because of the count of 3 for the associated field specifier, only the first three will be used. The second argument is associated with the second field specifier. The resulting binary string contains the four numbers 1.0, 2.0, 3.0 and 0.1.

Each type-count pair moves an imaginary cursor through the binary data, storing bytes at the current position and advancing the cursor to just after the last byte stored. The cursor is initially at position 0 at the beginning of the data. The type may be any one of the following characters:

a

Stores a byte string of length *count* in the output string. Every character is taken as modulo 256 (i.e. the low byte of every character is used, and the high byte discarded) so when storing character strings not wholly expressible using the characters \u0000-\u00ff, the encoding convertto command should be used first to change the string into an external representation if this truncation is not desired (i.e. if the characters are not part of the ISO 8859-1 character set.) If *arg* has fewer than *count* bytes, then additional zero bytes are used to pad out the field. If *arg* is longer than the specified length, the extra characters will be ignored. If *count* is *****, then all of the bytes in *arg* will be formatted. If *count* is omitted, then one character will be formatted. For example,

```
binary format a7a*a alpha bravo charlie
```

will return a string equivalent to **alpha\000\000bravoc**,

```
binary format a* [encoding convertto utf-8 \u20ac]
```

will return a string equivalent to **\342\202\254** (which is the UTF-8 byte sequence for a Euro-currency character) and

```
binary format a* [encoding convertto iso8859-15 \u20ac]
```

will return a string equivalent to **\244** (which is the ISO 8859-15 byte sequence for a Euro-currency character). Contrast these last two with:

```
binary format a* \u20ac
```

which returns a string equivalent to **\254** (i.e. **\xac**) by truncating the high-bits of the character, and which is probably not what is desired.

A

This form is the same as **a** except that spaces are used for padding instead of nulls. For example,

```
binary format A6A*A alpha bravo charlie
```

will return **alpha bravoc**.

b

Stores a string of *count* binary digits in low-to-high order within each byte in the output string. *Arg* must contain a sequence of **1** and **0** characters. The resulting bytes are emitted in first to last order with the bits being formatted in low-to-high order within each byte. If *arg* has fewer than *count* digits, then zeros will be used for the remaining bits. If *arg* has more than the specified number of digits, the extra digits will be ignored. If *count* is *****, then all of the digits in *arg* will be formatted. If *count* is omitted, then one digit will be formatted. If the number of bits formatted does not end at a byte boundary, the remaining bits of the last byte will be zeros. For example,

```
binary format b5b* 11100 111000011010
```

will return a string equivalent to **\x07\x87\x05**.

B

This form is the same as **b** except that the bits are stored in high-to-low order within each byte. For example,

```
binary format B5B* 11100 111000011010
```

will return a string equivalent to **\xe0\xe1\xa0**.

H

Stores a string of *count* hexadecimal digits in high-to-low within each byte in the output string. *Arg* must contain a sequence of characters in the set "0123456789abcdefABCDEF". The resulting bytes are emitted in first to last order with the hex digits being formatted in high-to-low order within each byte. If *arg* has fewer than *count* digits, then zeros will be used for the remaining digits. If *arg* has more than the specified number of digits, the extra digits will be ignored. If *count* is *****, then all of the digits in *arg* will be formatted. If *count* is omitted, then one digit will be formatted. If the number of digits formatted does not end at a byte boundary, the remaining bits of the last byte will be zeros. For example,

```
binary format H3H*H2 ab DEF 987
```

will return a string equivalent to **\xab\x00\xde\xf0\x98**.

h

This form is the same as **H** except that the digits are stored in low-to-high order within each byte. This is seldom required. For example,

```
binary format h3h*h2 AB def 987
```

will return a string equivalent to **\xba\x00\xed\x0f\x89**.

c

Stores one or more 8-bit integer values in the output string. If no *count* is specified, then *arg* must consist of an integer value. If *count* is specified, *arg* must consist of a list containing at least that many integers. The low-order 8 bits of each integer are stored as a one-byte value at the cursor position. If *count* is *, then all of the integers in the list are formatted. If the number of elements in the list is greater than *count*, then the extra elements are ignored. For example,

```
binary format c3cc* {3 -3 128 1} 260 {2 5}
```

will return a string equivalent to **\x03\xfd\x80\x04\x02\x05**, whereas

```
binary format c {2 5}
```

will generate an error.

s

This form is the same as **c** except that it stores one or more 16-bit integers in little-endian byte order in the output string. The low-order 16-bits of each integer are stored as a two-byte value at the cursor position with the least significant byte stored first. For example,

```
binary format s3 {3 -3 258 1}
```

will return a string equivalent to **\x03\x00\xfd\xff\x02\x01**.

S

This form is the same as **s** except that it stores one or more 16-bit integers in big-endian byte order in the output string. For example,

```
binary format S3 {3 -3 258 1}
```

will return a string equivalent to **\x00\x03\xff\xfd\x01\x02**.

t

This form (mnemonically *tiny*) is the same as **s** and **S** except that it stores the 16-bit integers in the output string in the native byte order of the machine where the Tcl script is running. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the tcl_platform array.

i

This form is the same as **c** except that it stores one or more 32-bit integers in little-endian byte order in the output string. The low-order 32-bits of each integer are stored as a four-byte value at the cursor position with the least significant byte stored first. For example,

```
binary format i3 {3 -3 65536 1}
```

will return a string equivalent to **\x03\x00\x00\x00\xfd\xff\xff\xff\x00\x00\x01\x00**

I

This form is the same as **i** except that it stores one or more one or more 32-bit integers in big-endian byte order in the output string. For example,

```
binary format I3 {3 -3 65536 1}
```

will return a string equivalent to **\x00\x00\x00\x03\xff\xff\xff\xfd\x00\x01\x00\x00**

n

This form (mnemonically *number* or *normal*) is the same as **i** and **I** except that it stores the 32-bit integers in the output string in the native byte order of the machine where the Tcl script is running. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the tcl_platform array.

w

This form is the same as **c** except that it stores one or more 64-bit integers in little-endian byte order in the output string. The low-order 64-bits of each integer are stored as an eight-byte value at the cursor position with the least significant byte stored first. For example,

```
binary format w 7810179016327718216
```

will return the string **HelloTcl**

W

This form is the same as **w** except that it stores one or more one or more 64-bit integers in big-endian byte order in the output string. For example,

```
binary format Wc 4785469626960341345 110
```

will return the string **BigEndian**

m

This form (mnemonically the mirror of **w**) is the same as **w** and **W** except that it stores the 64-bit integers in the output string in the native byte order of the machine where the Tcl script is running. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the tcl_platform array.

f

This form is the same as **c** except that it stores one or more one or more single-precision floating point numbers in the machine's native representation in the output string. This representation is not portable across architectures, so it should not be used to communicate floating point numbers across the network. The size of a floating point number may vary across architectures, so the number of bytes that are generated may vary. If the value overflows the machine's native representation, then the value of FLT_MAX as defined by the system will be used instead. Because Tcl uses double-precision floating point numbers internally, there may be some loss of precision in the conversion to single-precision. For example, on a Windows system running on an Intel Pentium processor,

```
binary format f2 {1.6 3.4}
```

will return a string equivalent to **\xcd\xcc\xcc\x3f\x9a\x99\x59\x40**.

r

This form (mnemonically *real*) is the same as **f** except that it stores the single-precision floating point numbers in little-endian order. This conversion only produces meaningful output when used on machines which use the IEEE floating point representation (very common, but not universal.)

R

This form is the same as **r** except that it stores the single-precision floating point numbers in big-endian order.

d

This form is the same as **f** except that it stores one or more one or more double-precision floating point numbers in the machine's native representation in the output string. For example, on a Windows system running on an Intel Pentium processor,

```
binary format d1 {1.6}
```

will return a string equivalent to **\x9a\x99\x99\x99\x99\x99\xf9\x3f**.

q

This form (mnemonically the mirror of **d**) is the same as **d** except that it stores the double-precision floating point numbers in little-endian order. This conversion only produces meaningful output when used on machines which use the IEEE floating point representation (very common, but not universal.)

Q

This form is the same as **q** except that it stores the double-precision floating point numbers in big-endian order.

x

Stores *count* null bytes in the output string. If *count* is not specified, stores one null byte. If *count* is *, generates an error. This type does not consume an argument. For example,

```
binary format a3xa3x2a3 abc def ghi
```

will return a string equivalent to **abc\000def\000\000ghi**.

X

Moves the cursor back *count* bytes in the output string. If *count* is * or is larger than the current cursor position, then the cursor is positioned at location 0 so that the next byte stored will be the first byte in the result string. If *count* is omitted then the cursor is moved back one byte. This type does not consume an argument. For example,

```
binary format a3X*a3X2a3 abc def ghi
```

will return **dghi**.

@

Moves the cursor to the absolute location in the output string specified by *count*. Position 0 refers to the first byte in the output string. If *count* refers to a position beyond the last byte stored so far, then null bytes will be placed in the uninitialized locations and the cursor will be placed at the specified location. If *count* is *, then the cursor is moved to the current end of the output string. If *count* is omitted, then an error will be generated. This type does not consume an argument. For example,

```
binary format a5@2a1@*a3@10a1 abcde f ghi j
```

will return **abfdeghi\000\000j**.

Binary scan

The **binary scan** command parses fields from a binary string, returning the number of conversions performed. *String* gives the input bytes to be parsed (one byte per character, and characters not representable as a byte have their high bits chopped) and *formatString* indicates how to parse it. Each *varName* gives the name of a variable; when a field is scanned from *string* the result is assigned to the corresponding variable.

As with **binary format**, the *formatString* consists of a sequence of zero or more field specifiers

separated by zero or more spaces. Each field specifier is a single type character followed by an optional flag character followed by an optional numeric *count*. Most field specifiers consume one argument to obtain the variable into which the scanned values should be placed. The type character specifies how the binary data is to be interpreted. The *count* typically indicates how many items of the specified type are taken from the data. If present, the *count* is a non-negative decimal integer or *, which normally indicates that all of the remaining items in the data are to be used. If there are not enough bytes left after the current cursor position to satisfy the current field specifier, then the corresponding variable is left untouched and **binary scan** returns immediately with the number of variables that were set. If there are not enough arguments for all of the fields in the format string that consume arguments, then an error is generated. The flag character "u" may be given to cause some types to be read as unsigned values. The flag is accepted for all field types but is ignored for non-integer fields.

A similar example as with **binary format** should explain the relation between field specifiers and arguments in case of the binary scan subcommand:

```
binary scan $bytes s3s first second
```

This command (provided the binary string in the variable *bytes* is long enough) assigns a list of three integers to the variable *first* and assigns a single value to the variable *second*. If *bytes* contains fewer than 8 bytes (i.e. four 2-byte integers), no assignment to *second* will be made, and if *bytes* contains fewer than 6 bytes (i.e. three 2-byte integers), no assignment to *first* will be made. Hence:

```
puts [binary scan abcdefg s3s first second]
puts $first
puts $second
```

will print (assuming neither variable is set previously):

```
1
25185 25699 26213
can't read "second": no such variable
```

It is *important* to note that the **c**, **s**, and **S** (and **i** and **I** on 64bit systems) will be scanned into long data size values. In doing this, values that have their high bit set (0x80 for chars, 0x8000 for shorts, 0x80000000 for ints), will be sign extended. Thus the following will occur:

```
set signShort [binary format s1 0x8000]
binary scan $signShort s1 val; # val == 0xFFFF8000
```

If you require unsigned values you can include the "u" flag character following the field type. For example, to read an unsigned short value:

```
set signShort [binary format s1 0x8000]
binary scan $signShort su1 val; # val == 0x00008000
```

Each type-count pair moves an imaginary cursor through the binary data, reading bytes from the current position. The cursor is initially at position 0 at the beginning of the data. The type may be any one of the following characters:

a

The data is a byte string of length *count*. If *count* is *, then all of the remaining bytes in *string* will be scanned into the variable. If *count* is omitted, then one byte will be scanned. All bytes scanned will be interpreted as being characters in the range \u0000-\u00ff so the encoding convertfrom command will be needed if the string is not a binary string or a string encoded in ISO 8859-1. For example,

```
binary scan abcde\000fghi a6a10 var1 var2
```

will return **1** with the string equivalent to **abcde\000** stored in *var1* and *var2* left unmodified, and

```
binary scan \342\202\254 a* var1
set var2 [encoding convertfrom utf-8 $var1]
```

will store a Euro-currency character in *var2*.

A

This form is the same as **a**, except trailing blanks and nulls are stripped from the scanned value before it is stored in the variable. For example,

```
binary scan "abc efghi  \000" A* var1
```

will return **1** with **abc efghi** stored in *var1*.

b

The data is turned into a string of *count* binary digits in low-to-high order represented as a sequence of "1" and "0" characters. The data bytes are scanned in first to last order with the bits being taken in low-to-high order within each byte. Any extra bits in the last byte are ignored. If *count* is *, then all of the remaining bits in *string* will be scanned. If *count* is omitted, then one bit will be scanned. For example,

```
binary scan \x07\x87\x05 b5b* var1 var2
```

will return **2** with **11100** stored in *var1* and **1110000110100000** stored in *var2*.

B

This form is the same as **b**, except the bits are taken in high-to-low order within each byte. For example,

```
binary scan \x70\x87\x05 B5B* var1 var2
```

will return **2** with **01110** stored in *var1* and **1000011100000101** stored in *var2*.

H

The data is turned into a string of *count* hexadecimal digits in high-to-low order represented as a sequence of characters in the set "0123456789abcdef". The data bytes are scanned in first to last order with the hex digits being taken in high-to-low order within each byte. Any extra bits in the last byte are ignored. If *count* is *, then all of the remaining hex digits in *string* will be scanned. If *count* is omitted, then one hex digit will be scanned. For example,

```
binary scan \x07\xC6\x05\x1f\x34 H3H* var1 var2
```

will return **2** with **07c** stored in *var1* and **051f34** stored in *var2*.

h

This form is the same as **H**, except the digits are taken in reverse (low-to-high) order within each byte. For example,

```
binary scan \x07\x86\x05\x12\x34 h3h* var1 var2
```

will return **2** with **706** stored in *var1* and **502143** stored in *var2*.

Note that most code that wishes to parse the hexadecimal digits from multiple bytes in order should use the **H** format.

c

The data is turned into *count* 8-bit signed integers and stored in the corresponding variable as a list. If *count* is *****, then all of the remaining bytes in *string* will be scanned. If *count* is omitted, then one 8-bit integer will be scanned. For example,

```
binary scan \x07\x86\x05 c2c* var1 var2
```

will return **2** with **7 -122** stored in *var1* and **5** stored in *var2*. Note that the integers returned are signed, but they can be converted to unsigned 8-bit quantities using an expression like:

```
set num [expr { $num & 0xFF }]
```

s

The data is interpreted as *count* 16-bit signed integers represented in little-endian byte order. The integers are stored in the corresponding variable as a list. If *count* is *****, then all of the remaining bytes in *string* will be scanned. If *count* is omitted, then one 16-bit integer will be scanned. For example,

```
binary scan \x05\x00\x07\x00\xf0\xff s2s* var1 var2
```

will return **2** with **5 7** stored in *var1* and **-16** stored in *var2*. Note that the integers returned are signed, but they can be converted to unsigned 16-bit quantities using an expression like:

```
set num [expr { $num & 0xFFFF }]
```

S

This form is the same as **s** except that the data is interpreted as *count* 16-bit signed integers represented in big-endian byte order. For example,

```
binary scan \x00\x05\x00\x07\xff\xf0 S2S* var1 var2
```

will return **2** with **5 7** stored in *var1* and **-16** stored in *var2*.

t

The data is interpreted as *count* 16-bit signed integers represented in the native byte order of the machine running the Tcl script. It is otherwise identical to **s** and **S**. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the tcl_platform array.

i

The data is interpreted as *count* 32-bit signed integers represented in little-endian byte order.

The integers are stored in the corresponding variable as a list. If *count* is *****, then all of the remaining bytes in *string* will be scanned. If *count* is omitted, then one 32-bit integer will be scanned. For example,

```
set str \x05\x00\x00\x00\x07\x00\x00\x00\xf0\xff\xff\xff
binary scan $str i2i* var1 var2
```

will return **2** with **5 7** stored in *var1* and **-16** stored in *var2*. Note that the integers returned are signed, but they can be converted to unsigned 32-bit quantities using an expression like:

```
set num [expr { $num & 0xFFFFFFFF }]
```

I

This form is the same as **I** except that the data is interpreted as *count* 32-bit signed integers represented in big-endian byte order. For example,

```
set str \x00\x00\x00\x05\x00\x00\x00\x07\xff\xff\xff\xf0
binary scan $str I2I* var1 var2
```

will return **2** with **5 7** stored in *var1* and **-16** stored in *var2*.

n

The data is interpreted as *count* 32-bit signed integers represented in the native byte order of the machine running the Tcl script. It is otherwise identical to **i** and **I**. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the tcl_platform array.

w

The data is interpreted as *count* 64-bit signed integers represented in little-endian byte order. The integers are stored in the corresponding variable as a list. If *count* is *****, then all of the remaining bytes in *string* will be scanned. If *count* is omitted, then one 64-bit integer will be scanned. For example,

```
set str \x05\x00\x00\x00\x07\x00\x00\x00\xf0\xff\xff\xff
binary scan $str wi* var1 var2
```

will return **2** with **30064771077** stored in *var1* and **-16** stored in *var2*. Note that the integers returned are signed and cannot be represented by Tcl as unsigned values.

W

This form is the same as **w** except that the data is interpreted as *count* 64-bit signed integers represented in big-endian byte order. For example,

```
set str \x00\x00\x00\x05\x00\x00\x00\x07\xff\xff\xff\xf0
binary scan $str WI* var1 var2
```

will return **2** with **21474836487** stored in *var1* and **-16** stored in *var2*.

m

The data is interpreted as *count* 64-bit signed integers represented in the native byte order of the machine running the Tcl script. It is otherwise identical to **w** and **W**. To determine what the native byte order of the machine is, refer to the **byteOrder** element of the tcl_platform array.

f

The data is interpreted as *count* single-precision floating point numbers in the machine's native representation. The floating point numbers are stored in the corresponding variable as a list. If *count* is *, then all of the remaining bytes in *string* will be scanned. If *count* is omitted, then one single-precision floating point number will be scanned. The size of a floating point number may vary across architectures, so the number of bytes that are scanned may vary. If the data does not represent a valid floating point number, the resulting value is undefined and compiler dependent. For example, on a Windows system running on an Intel Pentium processor,

```
binary scan \x3f\xcc\xcc\xcd f var1
```

will return **1** with **1.6000000238418579** stored in *var1*.

r

This form is the same as **f** except that the data is interpreted as *count* single-precision floating point number in little-endian order. This conversion is not portable to the minority of systems not using IEEE floating point representations.

R

This form is the same as **f** except that the data is interpreted as *count* single-precision floating point number in big-endian order. This conversion is not portable to the minority of systems not using IEEE floating point representations.

d

This form is the same as **f** except that the data is interpreted as *count* double-precision floating point numbers in the machine's native representation. For example, on a Windows system running on an Intel Pentium processor,

```
binary scan \x9a\x99\x99\x99\x99\x99\xf9\x3f d var1
```

will return **1** with **1.6000000000000001** stored in *var1*.

q

This form is the same as **d** except that the data is interpreted as *count* double-precision floating point number in little-endian order. This conversion is not portable to the minority of systems not using IEEE floating point representations.

Q

This form is the same as **d** except that the data is interpreted as *count* double-precision floating point number in big-endian order. This conversion is not portable to the minority of systems not using IEEE floating point representations.

x

Moves the cursor forward *count* bytes in *string*. If *count* is * or is larger than the number of bytes after the current cursor position, then the cursor is positioned after the last byte in *string*. If *count* is omitted, then the cursor is moved forward one byte. Note that this type does not consume an argument. For example,

```
binary scan \x01\x02\x03\x04 x2H* var1
```

will return **1** with **0304** stored in *var1*.

X

Moves the cursor back *count* bytes in *string*. If *count* is ***** or is larger than the current cursor position, then the cursor is positioned at location 0 so that the next byte scanned will be the first byte in *string*. If *count* is omitted then the cursor is moved back one byte. Note that this type does not consume an argument. For example,

```
binary scan \x01\x02\x03\x04 c2XH* var1 var2
```

will return **2** with **1 2** stored in *var1* and **020304** stored in *var2*.

@

Moves the cursor to the absolute location in the data string specified by *count*. Note that position 0 refers to the first byte in *string*. If *count* refers to a position beyond the end of *string*, then the cursor is positioned after the last byte. If *count* is omitted, then an error will be generated. For example,

```
binary scan \x01\x02\x03\x04 c2@1H* var1 var2
```

will return **2** with **1 2** stored in *var1* and **020304** stored in *var2*.

Portability issues

The **r**, **R**, **q** and **Q** conversions will only work reliably for transferring data between computers which are all using IEEE floating point representations. This is very common, but not universal. To transfer floating-point numbers portably between all architectures, use their textual representation (as produced by format) instead.

Examples

This is a procedure to write a Tcl string to a binary-encoded channel as UTF-8 data preceded by a length word:

```
proc writeString {channel string} {
    set data [encoding convertto utf-8 $string]
    puts -nonewline [binary format Ia* \
            [string length $data] $data]
}
```

This procedure reads a string from a channel that was written by the previously presented *writeString* procedure:

```
proc readString {channel} {
    if {![binary scan [read $channel 4] I length]} {
        error "missing length"
    }
    set data [read $channel $length]
    return [encoding convertfrom utf-8 $data]
}
```

This converts the contents of a file (named in the variable *filename*) to base64 and prints them:

```
set f [open $filename rb]
set data [read $f]
close $f
puts [binary encode base64 -maxlen 64 $data]
```

Copyright (c) 1997 Sun Microsystems, Inc.
Copyright (c) 2008 Donal K. Fellows

break

Keywords: break, abort, loop

Name

break - Abort looping command

Synopsis

break

Description

This command is typically invoked inside the body of a looping command such as for or foreach or while. It returns a 3 (TCL_BREAK) result code, which causes a break exception to occur. The exception causes the current script to be aborted out to the innermost containing loop command, which then aborts its execution and returns normally. Break exceptions are also handled in a few other situations, such as the catch command, Tk event bindings, and the outermost scripts of procedure bodies.

Example

Print a line for each of the integers from 0 to 5:

```
for {set x 0} {$x<10} {incr x} {
    if {$x > 5} {
        break
    }
    puts "x is $x"
}
```

Copyright (c) 1993-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

case

Keywords: case, match, regular expression

Name

case - Evaluate one of several scripts, depending on a given value

Synopsis

case *string* ?**in**? *patList body ?patList body ...?*
case *string* ?**in**? *{patList body ?patList body ...?}*

Description

Note: the **case** *command is obsolete and is supported only for backward compatibility. At some point in the future it may be removed entirely. You should use the* switch *command instead.*

The **case** command matches *string* against each of the *patList* arguments in order. Each *patList* argument is a list of one or more patterns. If any of these patterns matches *string* then **case** evaluates the following *body* argument by passing it recursively to the Tcl interpreter and returns the result of that evaluation. Each *patList* argument consists of a single pattern or list of patterns. Each pattern may contain any of the wild-cards described under string match. If a *patList* argument is **default**, the corresponding body will be evaluated if no *patList* matches *string*. If no *patList* argument matches *string* and no default is given, then the **case** command returns an empty string.

Two syntaxes are provided for the *patList* and *body* arguments. The first uses a separate argument for each of the patterns and commands; this form is convenient if substitutions are desired on some of the patterns or commands. The second form places all of the patterns and commands together into a single argument; the argument must have proper list structure, with the elements of the list being the patterns and commands. The second form makes it easy to construct multi-line case commands, since the braces around the whole list make it unnecessary to include a backslash at the end of each line. Since the *patList* arguments are in braces in the second form, no command or variable substitutions are performed on them; this makes the behavior of the second form different than the first form in some cases.

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

catch

Keywords: catch, error, exception

Name

catch - Evaluate script and trap exceptional returns

Synopsis

catch *script* ?*resultVarName*? ?*optionsVarName*?

Description

The **catch** command may be used to prevent errors from aborting command interpretation. The **catch** command calls the Tcl interpreter recursively to execute *script*, and always returns without raising an error, regardless of any errors that might occur while executing *script*.

If *script* raises an error, **catch** will return a non-zero integer value corresponding to the exceptional return code returned by evaluation of *script*. Tcl defines the normal return code from script evaluation to be zero (0), or **TCL_OK**. Tcl also defines four exceptional return codes: 1 (**TCL_ERROR**), 2 (**TCL_RETURN**), 3 (**TCL_BREAK**), and 4 (**TCL_CONTINUE**). Errors during evaluation of a script are indicated by a return code of **TCL_ERROR**. The other exceptional return codes are returned by the return, break, and continue commands and in other special situations as documented. Tcl packages can define new commands that return other integer values as return codes as well, and scripts that make use of the return -code command can also have return codes other than the five defined by Tcl.

If the *resultVarName* argument is given, then the variable it names is set to the result of the script evaluation. When the return code from the script is 1 (**TCL_ERROR**), the value stored in *resultVarName* is an error message. When the return code from the script is 0 (**TCL_OK**), the value stored in *resultVarName* is the value returned from *script*.

If the *optionsVarName* argument is given, then the variable it names is set to a dictionary of return options returned by evaluation of *script*. Tcl specifies two entries that are always defined in the dictionary: **-code** and **-level**. When the return code from evaluation of *script* is not **TCL_RETURN**, the value of the **-level** entry will be 0, and the value of the **-code** entry will be the same as the return code. Only when the return code is **TCL_RETURN** will the values of the **-level** and **-code** entries be something else, as further described in the documentation for the return command.

When the return code from evaluation of *script* is **TCL_ERROR**, four additional entries are defined in the dictionary of return options stored in *optionsVarName*: **-errorinfo**, **-errorcode**, **-errorline**, and -errorstack. The value of the **-errorinfo** entry is a formatted stack trace containing more information about the context in which the error happened. The formatted stack trace is meant to be read by a person. The value of the **-errorcode** entry is additional information about the error stored as a list. The **-errorcode** value is meant to be further processed by programs, and may not be particularly readable by people. The value of the **-errorline** entry is an integer indicating which line of *script* was being evaluated when the error occurred. The value of the -errorstack entry is an even-sized list made of token-parameter pairs accumulated while unwinding the stack. The token may be "CALL", in which case the parameter is a list made of the proc name and arguments at the corresponding level; or it may be "UP", in which case the parameter is the relative level (as in uplevel) of the previous CALL. The salient differences with respect to -errorinfo are that:

1. it is a machine-readable form that is amenable to processing with [foreach {tok prm} ...],
2. it contains the true (substituted) values passed to the functions, instead of the static text of the calling sites, and
3. it is coarser-grained, with only one element per stack frame (like procs; no separate elements for foreach constructs for example).

The values of the **-errorinfo** and **-errorcode** entries of the most recent error are also available as values of the global variables **::errorInfo** and **::errorCode** respectively. The value of the -errorstack entry surfaces as info errorstack.

Tcl packages may provide commands that set other entries in the dictionary of return options, and the return command may be used by scripts to set return options in addition to those defined above.

Examples

The **catch** command may be used in an if to branch based on the success of a script.

```
if { [catch {open $someFile w} fid] } {
    puts stderr "Could not open $someFile for writing\n$fid"
    exit 1
}
```

There are more complex examples of **catch** usage in the documentation for the return command.

Copyright (c) 1993-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

cd

Keywords: cd, working directory

Name

cd - Change working directory

Synopsis

cd ?*dirName*?

Description

Change the current working directory to *dirName*, or to the home directory (as specified in the HOME environment variable) if *dirName* is not given. Returns an empty string. Note that the current working directory is a per-process resource; the **cd** command changes the working directory for all interpreters and (in a threaded environment) all threads.

Examples

Change to the home directory of the user **fred**:

```
cd ~fred
```

Change to the directory **lib** that is a sibling directory of the current one:

```
cd ../lib
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

chan

Keywords: chan, blocking, channel, end of file, events, input, non-blocking, offset, output, readable, seek, stdio, tell, writable

Name

chan - Read, write and manipulate channels

Synopsis

chan *option* ?*arg arg* ...?

Description

This command provides several operations for reading from, writing to and otherwise manipulating open channels (such as have been created with the open and socket commands, or the default named channels **stdin**, **stdout** or **stderr** which correspond to the process's standard input, output and error streams respectively). *Option* indicates what to do with the channel; any unique abbreviation for *option* is acceptable. Valid options are:

chan blocked *channelId*

This tests whether the last input operation on the channel called *channelId* failed because it would have otherwise caused the process to block, and returns 1 if that was the case. It

returns 0 otherwise. Note that this only ever returns 1 when the channel has been configured to be non-blocking; all Tcl channels have blocking turned on by default.

chan close *channelId* ?*direction*?

Close and destroy the channel called *channelId*. Note that this deletes all existing file-events registered on the channel. If the direction argument (which must be read or write or any unique abbreviation of them) is present, the channel will only be half-closed, so that it can go from being read-write to write-only or read-only respectively. If a read-only channel is closed for reading, it is the same as if the channel is fully closed, and respectively similar for write-only channels. Without the direction argument, the channel is closed for both reading and writing (but only if those directions are currently open). It is an error to close a read-only channel for writing, or a write-only channel for reading.

As part of closing the channel, all buffered output is flushed to the channel's output device (only if the channel is ceasing to be writable), any buffered input is discarded (only if the channel is ceasing to be readable), the underlying operating system resource is closed and *channelId* becomes unavailable for future use (both only if the channel is being completely closed).

If the channel is blocking and the channel is ceasing to be writable, the command does not return until all output is flushed. If the channel is non-blocking and there is unflushed output, the channel remains open and the command returns immediately; output will be flushed in the background and the channel will be closed when all the flushing is complete.

If *channelId* is a blocking channel for a command pipeline then **chan close** waits for the child processes to complete.

If the channel is shared between interpreters, then **chan close** makes *channelId* unavailable in the invoking interpreter but has no other effect until all of the sharing interpreters have closed the channel. When the last interpreter in which the channel is registered invokes **chan close** (or close), the cleanup actions described above occur. With half-closing, the half-close of the channel only applies to the current interpreter's view of the channel until all channels have closed it in that direction (or completely). See the interp command for a description of channel sharing.

Channels are automatically fully closed when an interpreter is destroyed and when the process exits. Channels are switched to blocking mode, to ensure that all output is correctly flushed before the process exits.

The command returns an empty string, and may generate an error if an error occurs while flushing output. If a command in a command pipeline created with open returns an error, **chan close** generates an error (similar to the exec command.)

Note that half-closes of sockets and command pipelines can have important side effects because they result in a shutdown() or close() of the underlying system resource, which can change how other processes or systems respond to the Tcl program.

chan configure *channelId* ?*optionName*? ?*value*? ?*optionName value* ...?

Query or set the configuration options of the channel named *channelId*.

If no *optionName* or *value* arguments are supplied, the command returns a list containing alternating option names and values for the channel. If *optionName* is supplied but no *value* then the command returns the current value of the given option. If one or more pairs of *optionName* and *value* are supplied, the command sets each of the named options to the corresponding *value*; in this case the return value is an empty string.

The options described below are supported for all channels. In addition, each channel type may add options that only it supports. See the manual entry for the command that creates each type of channel for the options supported by that specific type of channel. For example, see the manual entry for the socket command for additional options for sockets, and the open command for additional options for serial devices.

-blocking *boolean*
The **-blocking** option determines whether I/O operations on the channel can cause the process to block indefinitely. The value of the option must be a proper boolean value. Channels are normally in blocking mode; if a channel is placed into non-blocking mode it will affect the operation of the **chan gets**, **chan read**, **chan puts**, **chan flush**, and **chan close** commands; see the documentation for those commands for details. For non-blocking mode to work correctly, the application must be using the Tcl event loop (e.g. by calling Tcl_DoOneEvent or invoking the vwait command).

-buffering *newValue*
If *newValue* is **full** then the I/O system will buffer output until its internal buffer is full or until the **chan flush** command is invoked. If *newValue* is **line**, then the I/O system will automatically flush output for the channel whenever a newline character is output. If *newValue* is **none**, the I/O system will flush automatically after every output operation. The default is for **-buffering** to be set to **full** except for channels that connect to terminal-like devices; for these channels the initial setting is **line**. Additionally, **stdin** and **stdout** are initially set to **line**, and **stderr** is set to **none**.

-buffersize *newSize*
newSize must be an integer; its value is used to set the size of buffers, in bytes, subsequently allocated for this channel to store input or output. *newSize* must be a number of no more than one million, allowing buffers of up to one million bytes in size.

-encoding *name*
This option is used to specify the encoding of the channel as one of the named encodings returned by encoding names or the special value binary, so that the data can be converted to and from Unicode for use in Tcl. For instance, in order for Tcl to read characters from a Japanese file in **shiftjis** and properly process and display the contents, the encoding would be set to **shiftjis**. Thereafter, when reading from the channel, the bytes in the Japanese file would be converted to Unicode as they are read. Writing is also supported - as Tcl strings are written to the channel they will automatically be converted to the specified encoding on output.

If a file contains pure binary data (for instance, a JPEG image), the encoding for the channel should be configured to be binary. Tcl will then assign no interpretation to the data in the file and simply read or write raw bytes. The Tcl binary command can be used to manipulate this

byte-oriented data. It is usually better to set the **-translation** option to binary when you want to transfer binary data, as this turns off the other automatic interpretations of the bytes in the stream as well.

The default encoding for newly opened channels is the same platform- and locale-dependent system encoding used for interfacing with the operating system, as returned by encoding system.

-eofchar *char*
see below ...

-eofchar *{inChar outChar}*
This option supports DOS file systems that use Control-z (\x1A) as an end of file marker. If *char* is not an empty string, then this character signals end-of-file when it is encountered during input. For output, the end-of-file character is output when the channel is closed. If *char* is the empty string, then there is no special end of file character marker. For read-write channels, a two-element list specifies the end of file marker for input and output, respectively. As a convenience, when setting the end-of-file character for a read-write channel you can specify a single value that will apply to both reading and writing. When querying the end-of-file character of a read-write channel, a two-element list will always be returned. The default value for **-eofchar** is the empty string in all cases except for files under Windows. In that case the **-eofchar** is Control-z (\x1A) for reading and the empty string for writing. The acceptable range for **-eofchar** values is \x01 - \x7f; attempting to set **-eofchar** to a value outside of this range will generate an error.

-translation *translation*
see below ...

-translation *{inTranslation outTranslation}*
In Tcl scripts the end of a line is always represented using a single newline character (\n). However, in actual files and devices the end of a line may be represented differently on different platforms, or even for different devices on the same platform. For example, under UNIX newlines are used in files, whereas carriage-return-linefeed sequences are normally used in network connections. On input (i.e., with **chan gets** and **chan read**) the Tcl I/O system automatically translates the external end-of-line representation into newline characters. Upon output (i.e., with **chan puts**), the I/O system translates newlines to the external end-of-line representation. The default translation mode, **auto**, handles all the common cases automatically, but the **-translation** option provides explicit control over the end of line translations.

The value associated with **-translation** is a single item for read-only and write-only channels. The value is a two-element list for read-write channels; the read translation mode is the first element of the list, and the write translation mode is the second element. As a convenience, when setting the translation mode for a read-write channel you can specify a single value that will apply to both reading and writing. When querying the translation mode of a read-write channel, a two-element list will always be returned. The following values are currently supported:

auto
As the input translation mode, **auto** treats any of newline (**lf**), carriage return (**cr**), or carriage return followed by a newline (**crlf**) as the end of line representation. The end of

line representation can even change from line-to-line, and all cases are translated to a newline. As the output translation mode, **auto** chooses a platform specific representation; for sockets on all platforms Tcl chooses **crlf**, for all Unix flavors, it chooses **lf**, and for the various flavors of Windows it chooses **crlf**. The default setting for **-translation** is **auto** for both input and output.

binary
: Like **lf**, no end-of-line translation is performed, but in addition, sets **-eofchar** to the empty string to disable it, and sets **-encoding** to **iso8859-1**. With this one setting, a channel is fully configured for binary input and output: Each byte read from the channel becomes the Unicode character having the same value as that byte, and each character written to the channel becomes a single byte in the output. This makes it possible to work seamlessly with binary data as long as each character in the data remains in the range of 0 to 255 so that there is no distinction between binary data and text. For example, A JPEG image can be read from a such a channel, manipulated, and then written back to such a channel.

cr
: The end of a line in the underlying file or device is represented by a single carriage return character. As the input translation mode, **cr** mode converts carriage returns to newline characters. As the output translation mode, **cr** mode translates newline characters to carriage returns.

crlf
: The end of a line in the underlying file or device is represented by a carriage return character followed by a linefeed character. As the input translation mode, **crlf** mode converts carriage-return-linefeed sequences to newline characters. As the output translation mode, **crlf** mode translates newline characters to carriage-return-linefeed sequences. This mode is typically used on Windows platforms and for network connections.

lf
: The end of a line in the underlying file or device is represented by a single newline (linefeed) character. In this mode no translations occur during either input or output. This mode is typically used on UNIX platforms.

chan copy *inputChan outputChan* ?*-size size*? ?*-command callback*?

Copy data from the channel *inputChan*, which must have been opened for reading, to the channel *outputChan*, which must have been opened for writing. The **chan copy** command leverages the buffering in the Tcl I/O system to avoid extra copies and to avoid buffering too much data in main memory when copying large files to slow destinations like network sockets.

The **chan copy** command transfers data from *inputChan* until end of file or *size* bytes or characters have been transferred; *size* is in bytes if the two channels are using the same encoding, and is in characters otherwise. If no **-size** argument is given, then the copy goes until end of file. All the data read from *inputChan* is copied to *outputChan*. Without the **-command** option, **chan copy** blocks until the copy is complete and returns the number of bytes or characters (using the same rules as for the **-size** option) written to *outputChan*.

The **-command** argument makes **chan copy** work in the background. In this case it returns immediately and the *callback* is invoked later when the copy completes. The *callback* is called

with one or two additional arguments that indicates how many bytes were written to *outputChan*. If an error occurred during the background copy, the second argument is the error string associated with the error. With a background copy, it is not necessary to put *inputChan* or *outputChan* into non-blocking mode; the **chan copy** command takes care of that automatically. However, it is necessary to enter the event loop by using the vwait command or by using Tk.

You are not allowed to do other I/O operations with *inputChan* or *outputChan* during a background **chan copy**. If either *inputChan* or *outputChan* get closed while the copy is in progress, the current copy is stopped and the command callback is *not* made. If *inputChan* is closed, then all data already queued for *outputChan* is written out.

Note that *inputChan* can become readable during a background copy. You should turn off any **chan event** or fileevent handlers during a background copy so those handlers do not interfere with the copy. Any I/O attempted by a **chan event** or fileevent handler will get a "channel busy" error.

Chan copy translates end-of-line sequences in *inputChan* and *outputChan* according to the **-translation** option for these channels (see **chan configure** above). The translations mean that the number of bytes read from *inputChan* can be different than the number of bytes written to *outputChan*. Only the number of bytes written to *outputChan* is reported, either as the return value of a synchronous **chan copy** or as the argument to the callback for an asynchronous **chan copy**.

Chan copy obeys the encodings and character translations configured for the channels. This means that the incoming characters are converted internally first UTF-8 and then into the encoding of the channel **chan copy** writes to (see **chan configure** above for details on the **-encoding** and **-translation** options). No conversion is done if both channels are set to encoding binary and have matching translations. If only the output channel is set to encoding binary the system will write the internal UTF-8 representation of the incoming characters. If only the input channel is set to encoding binary the system will assume that the incoming bytes are valid UTF-8 characters and convert them according to the output encoding. The behaviour of the system for bytes which are not valid UTF-8 characters is undefined in this case.

chan create *mode cmdPrefix*

This subcommand creates a new script level channel using the command prefix *cmdPrefix* as its handler. Any such channel is called a **reflected** channel. The specified command prefix, **cmdPrefix**, must be a non-empty list, and should provide the API described in the refchan manual page. The handle of the new channel is returned as the result of the **chan create** command, and the channel is open. Use either close or **chan close** to remove the channel.

The argument *mode* specifies if the new channel is opened for reading, writing, or both. It has to be a list containing any of the strings "**read**" or "**write**", The list must have at least one element, as a channel you can neither write to nor read from makes no sense. The handler command for the new channel must support the chosen mode, or an error is thrown.

The command prefix is executed in the global namespace, at the top of call stack, following the appending of arguments as described in the refchan manual page. Command resolution happens at the time of the call. Renaming the command, or destroying it means that the next call of a handler method may fail, causing the channel command invoking the handler to fail as well. Depending on the subcommand being invoked, the error message may not be able to explain the reason for that failure.

Every channel created with this subcommand knows which interpreter it was created in, and only ever executes its handler command in that interpreter, even if the channel was shared with and/or was moved into a different interpreter. Each reflected channel also knows the thread it was created in, and executes its handler command only in that thread, even if the channel was moved into a different thread. To this end all invocations of the handler are forwarded to the original thread by posting special events to it. This means that the original thread (i.e. the thread that executed the **chan create** command) must have an active event loop, i.e. it must be able to process such events. Otherwise the thread sending them will *block indefinitely*. Deadlock may occur.

Note that this permits the creation of a channel whose two endpoints live in two different threads, providing a stream-oriented bridge between these threads. In other words, we can provide a way for regular stream communication between threads instead of having to send commands.

When a thread or interpreter is deleted, all channels created with this subcommand and using this thread/interpreter as their computing base are deleted as well, in all interpreters they have been shared with or moved into, and in whatever thread they have been transferred to. While this pulls the rug out under the other thread(s) and/or interpreter(s), this cannot be avoided. Trying to use such a channel will cause the generation of a regular error about unknown channel handles.

This subcommand is **safe** and made accessible to safe interpreters. While it arranges for the execution of arbitrary Tcl code the system also makes sure that the code is always executed within the safe interpreter.

chan eof *channelId*

Test whether the last input operation on the channel called *channelId* failed because the end of the data stream was reached, returning 1 if end-of-file was reached, and 0 otherwise.

chan event *channelId event* ?*script*?

Arrange for the Tcl script *script* to be installed as a *file event handler* to be called whenever the channel called *channelId* enters the state described by *event* (which must be either **readable** or **writable**); only one such handler may be installed per event per channel at a time. If *script* is the empty string, the current handler is deleted (this also happens if the channel is closed or the interpreter deleted). If *script* is omitted, the currently installed script is returned (or an empty string if no such handler is installed). The callback is only performed if the event loop is being serviced (e.g. via vwait or update).

A file event handler is a binding between a channel and a script, such that the script is evaluated whenever the channel becomes readable or writable. File event handlers are most

commonly used to allow data to be received from another process on an event-driven basis, so that the receiver can continue to interact with the user or with other channels while waiting for the data to arrive. If an application invokes **chan gets** or **chan read** on a blocking channel when there is no input data available, the process will block; until the input data arrives, it will not be able to service other events, so it will appear to the user to "freeze up". With **chan event**, the process can tell when data is present and only invoke **chan gets** or **chan read** when they will not block.

A channel is considered to be readable if there is unread data available on the underlying device. A channel is also considered to be readable if there is unread data in an input buffer, except in the special case where the most recent attempt to read from the channel was a **chan gets** call that could not find a complete line in the input buffer. This feature allows a file to be read a line at a time in non-blocking mode using events. A channel is also considered to be readable if an end of file or error condition is present on the underlying file or device. It is important for *script* to check for these conditions and handle them appropriately; for example, if there is no special check for end of file, an infinite loop may occur where *script* reads no data, returns, and is immediately invoked again.

A channel is considered to be writable if at least one byte of data can be written to the underlying file or device without blocking, or if an error condition is present on the underlying file or device. Note that client sockets opened in asynchronous mode become writable when they become connected or if the connection fails.

Event-driven I/O works best for channels that have been placed into non-blocking mode with the **chan configure** command. In blocking mode, a **chan puts** command may block if you give it more data than the underlying file or device can accept, and a **chan gets** or **chan read** command will block if you attempt to read more data than is ready; no events will be processed while the commands block. In non-blocking mode **chan puts**, **chan read**, and **chan gets** never block.

The script for a file event is executed at global level (outside the context of any Tcl procedure) in the interpreter in which the **chan event** command was invoked. If an error occurs while executing the script then the command registered with interp bgerror is used to report the error. In addition, the file event handler is deleted if it ever returns an error; this is done in order to prevent infinite loops due to buggy handlers.

chan flush *channelId*

Ensures that all pending output for the channel called *channelId* is written.

If the channel is in blocking mode the command does not return until all the buffered output has been flushed to the channel. If the channel is in non-blocking mode, the command may return before all buffered output has been flushed; the remainder will be flushed in the background as fast as the underlying file or device is able to absorb it.

chan gets *channelId* ?*varName*?

Reads the next line from the channel called *channelId*. If *varName* is not specified, the result of the command will be the line that has been read (without a trailing newline character) or an empty string upon end-of-file or, in non-blocking mode, if the data available is exhausted.

If *varName* is specified, the line that has been read will be written to the variable called *varName* and result will be the number of characters that have been read or -1 if end-of-file was reached or, in non-blocking mode, if the data available is exhausted.

If an end-of-file occurs while part way through reading a line, the partial line will be returned (or written into *varName*). When *varName* is not specified, the end-of-file case can be distinguished from an empty line using the **chan eof** command, and the partial-line-but-non-blocking case can be distinguished with the **chan blocked** command.

chan names ?*pattern*?
Produces a list of all channel names. If *pattern* is specified, only those channel names that match it (according to the rules of string match) will be returned.

chan pending *mode channelId*
Depending on whether *mode* is **input** or **output**, returns the number of bytes of input or output (respectively) currently buffered internally for *channelId* (especially useful in a readable event callback to impose application-specific limits on input line lengths to avoid a potential denial-of-service attack where a hostile user crafts an extremely long line that exceeds the available memory to buffer it). Returns -1 if the channel was not opened for the mode in question.

chan pipe
Creates a standalone pipe whose read- and write-side channels are returned as a 2-element list, the first element being the read side and the second the write side. Can be useful e.g. to redirect separately stderr and stdout from a subprocess. To do this, spawn with "2>@" or ">@" redirection operators onto the write side of a pipe, and then immediately close it in the parent. This is necessary to get an EOF on the read side once the child has exited or otherwise closed its output.

Note that the pipe buffering semantics can vary at the operating system level substantially; it is not safe to assume that a write performed on the output side of the pipe will appear instantly to the input side. This is a fundamental difference and Tcl cannot conceal it. The overall stream semantics *are* compatible, so blocking reads and writes will not see most of the differences, but the details of what exactly gets written when are not. This is most likely to show up when using pipelines for testing; care should be taken to ensure that deadlocks do not occur and that potential short reads are allowed for.

chan pop *channelId*
Removes the topmost transformation from the channel channelId, if there is any. If there are no transformations added to channelId, this is equivalent to chan close of that channel. The result is normally the empty string, but can be an error in some situations (i.e. where the underlying system stream is closed and that results in an error).

chan postevent *channelId eventSpec*
This subcommand is used by command handlers specified with **chan create**. It notifies the channel represented by the handle *channelId* that the event(s) listed in the *eventSpec* have occurred. The argument has to be a list containing any of the strings **read** and **write**. The list must contain at least one element as it does not make sense to invoke the command if there are no events to post.

Note that this subcommand can only be used with channel handles that were created/opened

by **chan create**. All other channels will cause this subcommand to report an error.

As only the Tcl level of a channel, i.e. its command handler, should post events to it we also restrict the usage of this command to the interpreter that created the channel. In other words, posting events to a reflected channel from an interpreter that does not contain it's implementation is not allowed. Attempting to post an event from any other interpreter will cause this subcommand to report an error.

Another restriction is that it is not possible to post events that the I/O core has not registered an interest in. Trying to do so will cause the method to throw an error. See the command handler method **watch** described in **refchan**, the document specifying the API of command handlers for reflected channels.

This command is **safe** and made accessible to safe interpreters. It can trigger the execution of **chan event** handlers, whether in the current interpreter or in other interpreters or other threads, even where the event is posted from a safe interpreter and listened for by a trusted interpreter. **Chan event** handlers are *always* executed in the interpreter that set them up.

chan push *channelId cmdPrefix*

Adds a new transformation on top of the channel channelId. The cmdPrefix argument describes a list of one or more words which represent a handler that will be used to implement the transformation. The command prefix must provide the API described in the transchan manual page. The result of this subcommand is a handle to the transformation. Note that it is important to make sure that the transformation is capable of supporting the channel mode that it is used with or this can make the channel neither readable nor writable.

chan puts ?-nonewline? ?*channelId*? *string*

Writes *string* to the channel named *channelId* followed by a newline character. A trailing newline character is written unless the optional flag **-nonewline** is given. If *channelId* is omitted, the string is written to the standard output channel, **stdout**.

Newline characters in the output are translated by **chan puts** to platform-specific end-of-line sequences according to the currently configured value of the **-translation** option for the channel (for example, on PCs newlines are normally replaced with carriage-return-linefeed sequences; see **chan configure** above for details).

Tcl buffers output internally, so characters written with **chan puts** may not appear immediately on the output file or device; Tcl will normally delay output until the buffer is full or the channel is closed. You can force output to appear immediately with the **chan flush** command.

When the output buffer fills up, the **chan puts** command will normally block until all the buffered data has been accepted for output by the operating system. If *channelId* is in non-blocking mode then the **chan puts** command will not block even if the operating system cannot accept the data. Instead, Tcl continues to buffer the data and writes it in the background as fast as the underlying file or device can accept it. The application must use the Tcl event loop for non-blocking output to work; otherwise Tcl never finds out that the file or device is ready for more output data. It is possible for an arbitrarily large amount of data to

be buffered for a channel in non-blocking mode, which could consume a large amount of memory. To avoid wasting memory, non-blocking I/O should normally be used in an event-driven fashion with the **chan event** command (do not invoke **chan puts** unless you have recently been notified via a file event that the channel is ready for more output data).

chan read *channelId* ?*numChars*?

see below ...

chan read ?-nonewline? *channelId*

In the first form, the result will be the next *numChars* characters read from the channel named *channelId*; if *numChars* is omitted, all characters up to the point when the channel would signal a failure (whether an end-of-file, blocked or other error condition) are read. In the second form (i.e. when *numChars* has been omitted) the flag **-nonewline** may be given to indicate that any trailing newline in the string that has been read should be trimmed.

If *channelId* is in non-blocking mode, **chan read** may not read as many characters as requested: once all available input has been read, the command will return the data that is available rather than blocking for more input. If the channel is configured to use a multi-byte encoding, then there may actually be some bytes remaining in the internal buffers that do not form a complete character. These bytes will not be returned until a complete character is available or end-of-file is reached. The **-nonewline** switch is ignored if the command returns before reaching the end of the file.

Chan read translates end-of-line sequences in the input into newline characters according to the **-translation** option for the channel (see **chan configure** above for a discussion on the ways in which **chan configure** will alter input).

When reading from a serial port, most applications should configure the serial port channel to be non-blocking, like this:

```
chan configure channelId -blocking 0.
```

Then **chan read** behaves much like described above. Note that most serial ports are comparatively slow; it is entirely possible to get a **readable** event for each character read from them. Care must be taken when using **chan read** on blocking serial ports:

chan read *channelId numChars*

In this form **chan read** blocks until *numChars* have been received from the serial port.

chan read *channelId*

In this form **chan read** blocks until the reception of the end-of-file character, see **chan configure -eofchar**. If there no end-of-file character has been configured for the channel, then **chan read** will block forever.

chan seek *channelId offset* ?*origin*?

Sets the current access position within the underlying data stream for the channel named *channelId* to be *offset* bytes relative to *origin*. *Offset* must be an integer (which may be negative) and *origin* must be one of the following:

start

The new access position will be *offset* bytes from the start of the underlying file or device.

current

The new access position will be *offset* bytes from the current access position; a negative

offset moves the access position backwards in the underlying file or device.

end
The new access position will be *offset* bytes from the end of the file or device. A negative *offset* places the access position before the end of file, and a positive *offset* places the access position after the end of file.

The *origin* argument defaults to **start**.

Chan seek flushes all buffered output for the channel before the command returns, even if the channel is in non-blocking mode. It also discards any buffered and unread input. This command returns an empty string. An error occurs if this command is applied to channels whose underlying file or device does not support seeking.

Note that *offset* values are byte offsets, not character offsets. Both **chan seek** and **chan tell** operate in terms of bytes, not characters, unlike **chan read**.

chan tell *channelId*
Returns a number giving the current access position within the underlying data stream for the channel named *channelId*. This value returned is a byte offset that can be passed to **chan seek** in order to set the channel to a particular position. Note that this value is in terms of bytes, not characters like **chan read**. The value returned is -1 for channels that do not support seeking.

chan truncate *channelId* ?*length*?
Sets the byte length of the underlying data stream for the channel named *channelId* to be *length* (or to the current byte offset within the underlying data stream if *length* is omitted). The channel is flushed before truncation.

Examples

Simple channel operation examples

Instruct Tcl to always send output to **stdout** immediately, whether or not it is to a terminal:

```
chan configure stdout -buffering none
```

In the following example a file is opened using the encoding CP1252, which is common on Windows, searches for a string, rewrites that part, and truncates the file two lines later.

```
set f [open somefile.txt r+]
chan configure $f -encoding cp1252
set offset 0

# Search for string "FOOBAR" in the file
while {[chan gets $f line] >= 0} {
    set idx [string first FOOBAR $line]
    if {$idx >= 0} {
        # Found it; rewrite line

        chan seek $f [expr {$offset + $idx}]
        chan puts -nonewline $f BARFOO
```

```
        # Skip to end of following line, and truncate
        chan gets $f
        chan gets $f
        chan truncate $f

        # Stop searching the file now
        break
    }

    # Save offset of start of next line for later
    set offset [chan tell $f]
}
chan close $f
```

A network server that echoes its input line-by-line without preventing servicing of other connections at the same time:

```
# This is a very simple logger...
proc log {message} {
    chan puts stdout $message
}

# This is called whenever a new client connects to the server
proc connect {chan host port} {
    set clientName [format <%s:%d> $host $port]
    log "connection from $clientName"
    chan configure $chan -blocking 0 -buffering line
    chan event $chan readable [list echoLine $chan $clientName]
}

# This is called whenever either at least one byte of input
# data is available, or the channel was closed by the client.
proc echoLine {chan clientName} {
    chan gets $chan line
    if {[chan eof $chan]} {
        log "finishing connection from $clientName"
        chan close $chan
    } elseif {![chan blocked $chan]} {
        # Didn't block waiting for end-of-line
        log "$clientName - $line"
        chan puts $chan $line
    }
}

# Create the server socket and enter the event-loop to wait
# for incoming connections...
socket -server connect 12345
vwait forever
```

Copyright (c) 2005-2006 Donal K. Fellows

class

Keywords: class, metaclass, object

Name

oo::class - class of all classes

Synopsis

package require TclOO

oo::class *method* ?*arg* ...?

Class hierarchy

```
oo::object
   → oo::class
```

Description

Classes are objects that can manufacture other objects according to a pattern stored in the factory object (the class). An instance of the class is created by calling one of the class's factory methods, typically either **create** if an explicit name is being given, or **new** if an arbitrary unique name is to be automatically selected.

The **oo::class** class is the class of all classes; every class is an instance of this class, which is consequently an instance of itself. This class is a subclass of oo::object, so every class is also an object. Additional metaclasses (i.e., classes of classes) can be defined if necessary by subclassing **oo::class**. Note that the **oo::class** object hides the **new** method on itself, so new classes should always be made using the **create** method.

Constructor

The constructor of the **oo::class** class takes an optional argument which, if present, is sent to the oo::define command (along with the name of the newly-created class) to allow the class to be conveniently configured at creation time.

Destructor

The **oo::class** class does not define an explicit destructor. However, when a class is destroyed, all its subclasses and instances are also destroyed, along with all objects that it has been mixed into.

Exported methods

***cls* create *name* ?*arg ...*?**

This creates a new instance of the class *cls* called *name* (which is resolved within the calling context's namespace if not fully qualified), passing the arguments, *arg ...*, to the constructor, and (if that returns a successful result) returning the fully qualified name of the created object (the result of the constructor is ignored). If the constructor fails (i.e. returns a non-OK result) then the object is destroyed and the error message is the result of this method call.

***cls* new ?*arg ...*?**

This creates a new instance of the class *cls* with a new unique name, passing the arguments, *arg ...*, to the constructor, and (if that returns a successful result) returning the fully qualified name of the created object (the result of the constructor is ignored). If the constructor fails (i.e., returns a non-OK result) then the object is destroyed and the error message is the result of this method call.

Note that this method is not exported by the **oo::class** object itself, so classes should not be created using this method.

Non-exported methods

The **oo::class** class supports the following non-exported methods:

***cls* createWithNamespace *name nsName* ?*arg ...*?**

This creates a new instance of the class *cls* called *name* (which is resolved within the calling context's namespace if not fully qualified), passing the arguments, *arg ...*, to the constructor,

and (if that returns a successful result) returning the fully qualified name of the created object (the result of the constructor is ignored). The name of the instance's internal namespace will be *nsName*; it is an error if that namespace cannot be created. If the constructor fails (i.e., returns a non-OK result) then the object is destroyed and the error message is the result of this method call.

Examples

This example defines a simple class hierarchy and creates a new instance of it. It then invokes a method of the object before destroying the hierarchy and showing that the destruction is transitive.

```
oo::class create fruit {
    method eat {} {
        puts "yummy!"
    }
}
oo::class create banana {
    superclass fruit
    constructor {} {
        my variable peeled
        set peeled 0
    }
    method peel {} {
        my variable peeled
        set peeled 1
        puts "skin now off"
    }
    method edible? {} {
        my variable peeled
        return $peeled
    }
    method eat {} {
        if {![my edible?]} {
            my peel
        }
        next
    }
}
set b [banana new]
$b eat               → prints "skin now off" and "yummy!"
fruit destroy
$b eat               → error "unknown command"
```

Copyright (c) 2007 Donal K. Fellows

clock

Keywords: clock, date, time

Name

clock - Obtain and manipulate dates and times

Synopsis

package require Tcl 8.5
clock add *timeVal* ?*count unit...*? ?*-option value*?
clock clicks ?*-option*?
clock format *timeVal* ?*-option value...*?
clock microseconds
clock milliseconds
clock scan *inputString* ?*-option value...*?
clock seconds

Description

The **clock** command performs several operations that obtain and manipulate values that represent times. The command supports several subcommands that determine what action is carried out by the command.

clock add *timeVal ?count unit...? ?-option value?*
Adds a (possibly negative) offset to a time that is expressed as an integer number of seconds. See Clock arithmetic for a full description.

clock clicks ?*-option*?
If no *-option* argument is supplied, returns a high-resolution time value as a system-dependent integer value. The unit of the value is system-dependent but should be the highest resolution clock available on the system such as a CPU cycle counter. See High resolution timers for a full description.

If the *-option* argument is **-milliseconds**, then the command is synonymous with **clock milliseconds** (see below). This usage is obsolete, and **clock milliseconds** is to be considered the preferred way of obtaining a count of milliseconds.

If the *-option* argument is **-microseconds**, then the command is synonymous with **clock microseconds** (see below). This usage is obsolete, and **clock microseconds** is to be considered the preferred way of obtaining a count of microseconds.

clock format *timeVal ?-option value...?*
Formats a time that is expressed as an integer number of seconds into a format intended for consumption by users or external programs. See Formatting times for a full description.

clock microseconds
Returns the current time as an integer number of microseconds. See High resolution timers for a full description.

clock milliseconds
Returns the current time as an integer number of milliseconds. See High resolution timers for a full description.

clock scan *inputString ?-option value...?*
Scans a time that is expressed as a character string and produces an integer number of seconds. See Scanning times for a full description.

clock seconds
Returns the current time as an integer number of seconds.

Parameters

count
An integer representing a count of some unit of time. See Clock arithmetic for the details.

timeVal
An integer value passed to the **clock** command that represents an absolute time as a number of seconds from the *epoch time* of 1 January 1970, 00:00 UTC. Note that the count of seconds does not include any leap seconds; seconds are counted as if each UTC day has exactly 86400 seconds. Tcl responds to leap seconds by speeding or slowing its clock by a tiny fraction for some minutes until it is back in sync with UTC; its data model does not represent minutes that have 59 or 61 seconds.

unit
One of the words, **seconds**, **minutes**, **hours**, **days**, **weeks**, **months**, or **years**, or any unique prefix of such a word. Used in conjunction with *count* to identify an interval of time, for example, *3 seconds* or *1 year*.

Options

-base time
Specifies that any relative times present in a **clock scan** command are to be given relative to *time*. *time* must be expressed as a count of nominal seconds from the epoch time of 1 January 1970, 00:00 UTC.

-format format
Specifies the desired output format for **clock format** or the expected input format for **clock scan**. The *format* string consists of any number of characters other than the per-cent sign ("**%**") interspersed with any number of *format groups*, which are two-character sequences beginning with the per-cent sign. The permissible format groups, and their interpretation, are described under Format groups.

On **clock format**, the default format is

```
%a %b %d %H:%M:%S %Z %Y
```

On **clock scan**, the lack of a **-format** option indicates that a "free format scan" is requested; see Free form scan for a description of what happens.

-gmt boolean
If *boolean* is true, specifies that a time specified to **clock add**, **clock format** or **clock scan** should be processed in UTC. If *boolean* is false, the processing defaults to the local time zone. This usage is obsolete; the correct current usage is to specify the UTC time zone with "**-timezone** *:UTC*" or any of the equivalent ways to specify it.

-locale localeName
Specifies that locale-dependent scanning and formatting (and date arithmetic for dates preceding the adoption of the Gregorian calendar) is to be done in the locale identified by *localeName*. The locale name may be any of the locales acceptable to the msgcat package, or it may be the special name *system*, which represents the current locale of the process, or the null string, which represents Tcl's default locale.

The effect of locale on scanning and formatting is discussed in the descriptions of the individual format groups under Format groups. The effect of locale on clock arithmetic is discussed under Clock arithmetic.

-timezone zoneName
Specifies that clock arithmetic, formatting, and scanning are to be done according to the rules for the time zone specified by *zoneName*. The permissible values, and their interpretation, are discussed under Time zones. On subcommands that expect a **-timezone** argument, the default is to use the *current time zone*. The current time zone is determined, in order of preference, by:
1. the environment variable **TCL_TZ**.
2. the environment variable **TZ**.
3. on Windows systems, the time zone settings from the Control Panel.

If none of these is present, the C **localtime** and **mktime** functions are used to attempt to convert times between local and Greenwich. On 32-bit systems, this approach is likely to have bugs, particularly for times that lie outside the window (approximately the years 1902 to 2037) that can be represented in a 32-bit integer.

Clock arithmetic

The **clock add** command performs clock arithmetic on a value (expressed as nominal seconds from the epoch time of 1 January 1970, 00:00 UTC) given as its first argument. The remaining arguments (other than the possible **-timezone**, **-locale** and **-gmt** options) are integers and keywords in alternation, where the keywords are chosen from **seconds**, **minutes**, **hours**, **days**, **weeks**, **months**, or **years**, or any unique prefix of such a word.

Addition of seconds, minutes and hours is fairly straightforward; the given time increment (times sixty for minutes, or 3600 for hours) is simply added to the *timeVal* given to the **clock add** command. The result is interpreted as a nominal number of seconds from the Epoch.

Surprising results may be obtained when crossing a point at which a leap second is inserted or removed; the **clock add** command simply ignores leap seconds and therefore assumes that times come in sequence, 23:59:58, 23:59:59, 00:00:00. This assumption is handled by the fact that Tcl's model of time reacts to leap seconds by speeding or slowing the clock by a miniscule amount until Tcl's time is back in step with the world.

The fact that adding and subtracting hours is defined in terms of absolute time means that it will add fixed amounts of time in time zones that observe summer time (Daylight Saving Time). For example, the following code sets the value of **x** to **04:00:00** because the clock has changed in the interval in question.

```
set s [clock scan {2004-10-30 05:00:00} \
           -format {%Y-%m-%d %H:%M:%S} \
           -timezone :America/New_York]
set a [clock add $s 24 hours -timezone :America/New_York]
set x [clock format $a \
           -format {%H:%M:%S} -timezone :America/New_York]
```

Adding and subtracting days and weeks is accomplished by converting the given time to a calendar day and time of day in the appropriate time zone and locale. The requisite number of

days (weeks are converted to days by multiplying by seven) is added to the calendar day, and the date and time are then converted back to a count of seconds from the epoch time.

Adding and subtracting a given number of days across the point that the time changes at the start or end of summer time (Daylight Saving Time) results in the *same local time* on the day in question. For instance, the following code sets the value of **x** to **05:00:00**.

```
set s [clock scan {2004-10-30 05:00:00} \
           -format {%Y-%m-%d %H:%M:%S} \
           -timezone :America/New_York]
set a [clock add $s 1 day -timezone :America/New_York]
set x [clock format $a \
           -format {%H:%M:%S} -timezone :America/New_York]
```

In cases of ambiguity, where the same local time happens twice on the same day, the earlier time is used. In cases where the conversion yields an impossible time (for instance, 02:30 during the Spring Daylight Saving Time change using US rules), the time is converted as if the clock had not changed. Thus, the following code will set the value of **x** to **03:30:00**.

```
set s [clock scan {2004-04-03 02:30:00} \
           -format {%Y-%m-%d %H:%M:%S} \
           -timezone :America/New_York]
set a [clock add $s 1 day -timezone :America/New_York]
set x [clock format $a \
           -format {%H:%M:%S} -timezone :America/New_York]
```

Adding a given number of days or weeks works correctly across the conversion between the Julian and Gregorian calendars; the omitted days are skipped. The following code sets **z** to **1752-09-14**.

```
set x [clock scan 1752-09-02 -format %Y-%m-%d -locale en_US]
set y [clock add $x 1 day -locale en_US]
set z [clock format $y -format %Y-%m-%d -locale en_US]
```

In the bizarre case that adding the given number of days yields a date that does not exist because it falls within the dropped days of the Julian-to-Gregorian conversion, the date is converted as if it was on the Julian calendar.

Adding a number of months, or a number of years, is similar; it converts the given time to a calendar date and time of day. It then adds the requisite number of months or years, and reconverts the resulting date and time of day to an absolute time.

If the resulting date is impossible because the month has too few days (for example, when adding 1 month to 31 January), the last day of the month is substituted. Thus, adding 1 month to 31 January will result in 28 February in a common year or 29 February in a leap year.

The rules for handling anomalies relating to summer time and to the Gregorian calendar are the same when adding/subtracting months and years as they are when adding/subtracting days and weeks.

If multiple *count unit* pairs are present on the command, they are evaluated consecutively, from left to right.

High resolution timers

Most of the subcommands supported by the **clock** command deal with times represented as a count of seconds from the epoch time, and this is the representation that **clock seconds** returns. There are three exceptions, which are all intended for use where higher-resolution times are required. **clock milliseconds** returns the count of milliseconds from the epoch time, and **clock microseconds** returns the count of microseconds from the epoch time. In addition, there is a **clock clicks** command that returns a platform-dependent high-resolution timer. Unlike **clock seconds** and **clock milliseconds**, the value of **clock clicks** is not guaranteed to be tied to any fixed epoch; it is simply intended to be the most precise interval timer available, and is intended only for relative timing studies such as benchmarks.

Formatting times

The **clock format** command produces times for display to a user or writing to an external medium. The command accepts times that are expressed in seconds from the epoch time of 1 January 1970, 00:00 UTC, as returned by **clock seconds**, **clock scan**, **clock add**, file atime or file mtime.

If a **-format** option is present, the following argument is a string that specifies how the date and time are to be formatted. The string consists of any number of characters other than the per-cent sign ("**%**") interspersed with any number of *format groups*, which are two-character sequences beginning with the per-cent sign. The permissible format groups, and their interpretation, are described under Format groups.

If a **-timezone** option is present, the following argument is a string that specifies the time zone in which the date and time are to be formatted. As an alternative to "**-timezone** *:UTC*", the obsolete usage "**-gmt** *true*" may be used. See Time zones for the permissible variants for the time zone.

If a **-locale** option is present, the following argument is a string that specifies the locale in which the time is to be formatted, in the same format that is used for the msgcat package. Note that the default, if **-locale** is not specified, is the root locale **{}** rather than the current locale. The current locale may be obtained by using **-locale current**. In addition, some platforms support a **system** locale that reflects the user's current choices. For instance, on Windows, the format that the user has selected from dates and times in the Control Panel can be obtained by using the **system** locale. On platforms that do not define a user selection of date and time formats separate from **LC_TIME**, **-locale system** is synonymous with **-locale current**.

Scanning times

The **clock scan** command accepts times that are formatted as strings and converts them to counts of seconds from the epoch time of 1 January 1970, 00:00 UTC. It normally takes a **-format** option that is followed by a string describing the expected format of the input. (See Free form scan for the effect of **clock scan** without such an argument.) The string consists of any number of characters other than the per-cent sign ("**%**"), interspersed with any number of *format groups*, which are two-character sequences beginning with the per-cent sign. The permissible format groups, and their interpretation, are described under Format groups.

If a **-timezone** option is present, the following argument is a string that specifies the time zone in which the date and time are to be interpreted. As an alternative to **-timezone** *:UTC*, the obsolete usage **-gmt** *true* may be used. See Time zones for the permissible variants for the time

zone.

If a **-locale** option is present, the following argument is a string that specifies the locale in which the time is to be interpreted, in the same format that is used for the msgcat package. Note that the default, if **-locale** is not specified, is the root locale **{}** rather than the current locale. The current locale may be obtained by using **-locale current**. In addition, some platforms support a **system** locale that reflects the user's current choices. For instance, on Windows, the format that the user has selected from dates and times in the Control Panel can be obtained by using the **system** locale. On platforms that do not define a user selection of date and time formats separate from **LC_TIME**, **-locale system** is synonymous with **-locale current**.

If a **-base** option is present, the following argument is a time (expressed in seconds from the epoch time) that is used as a *base time* for interpreting relative times. If no **-base** option is present, the base time is the current time.

Scanning of times in fixed format works by determining three things: the date, the time of day, and the time zone. These three are then combined into a point in time, which is returned as the number of seconds from the epoch.

Before scanning begins, the format string is preprocessed to replace **%c**, **%Ec**, **%x**, **%Ex**, **%X**. **%Ex**, **%r**, **%R**, **%T**, **%D**, **%EY** and **%+** format groups with counterparts that are appropriate to the current locale and contain none of the above groups. For instance, **%D** will (in the **en_US** locale) be replaced with **%m/%d/%Y**.

The date is determined according to the fields that are present in the preprocessed format string. In order of preference:

1. If the string contains a **%s** format group, representing seconds from the epoch, that group is used to determine the date.
2. If the string contains a **%J** format group, representing the Julian Day Number, that group is used to determine the date.
3. If the string contains a complete set of format groups specifying century, year, month, and day of month; century, year, and day of year; or ISO8601 fiscal year, week of year, and day of week; those groups are combined and used to determine the date. If more than one complete set is present, the one at the rightmost position in the string is used.
4. If the string lacks a century but contains a set of format groups specifying year of century, month and day of month; year of century and day of year; or two-digit ISO8601 fiscal year, week of year, and day of week; those groups are combined and used to determine the date. If more than one complete set is present, the one at the rightmost position in the string is used. The year is presumed to lie in the range 1938 to 2037 inclusive.
5. If the string entirely lacks any specification for the year (or contains the year only on the locale's alternative calendar) and contains a set of format groups specifying month and day of month, day of year, or week of year and day of week, those groups are combined and used to determine the date. If more than one complete set is present, the one at the rightmost position in the string is used. The year is determined by interpreting the base time in the given time zone.
6. If the string contains none of the above sets, but has a day of the month or day of the week, the day of the month or day of the week are used to determine the date by interpreting the base time in the given time zone and returning the given day of the current week or month. (The week runs from Monday to Sunday, ISO8601-fashion.) If both day of month and day of week are present, the day of the month takes priority.

7. If none of the above rules results in a usable date, the date of the base time in the given time zone is used.

The time is also determined according to the fields that are present in the preprocessed format string. In order of preference:

1. If the string contains a **%s** format group, representing seconds from the epoch, that group determines the time of day.
2. If the string contains either an hour on the 24-hour clock or an hour on the 12-hour clock plus an AM/PM indicator, that hour determines the hour of the day. If the string further contains a group specifying the minute of the hour, that group combines with the hour. If the string further contains a group specifying the second of the minute, that group combines with the hour and minute.
3. If the string contains neither a **%s** format group nor a group specifying the hour of the day, then midnight (**00:00**, the start of the given date) is used. The time zone is determined by either the **-timezone** or **-gmt** options, or by using the current time zone.

If a format string lacks a **%z** or **%Z** format group, it is possible for the time to be ambiguous because it appears twice in the same day, once without and once with Daylight Saving Time. If this situation occurs, the first occurrence of the time is chosen. (For this reason, it is wise to have the input string contain the time zone when converting local times. This caveat does not apply to UTC times.)

If the interpretation of the groups yields an impossible time because a field is out of range, enough of that field's unit will be added to or subtracted from the time to bring it in range. Thus, if attempting to scan or format day 0 of the month, one day will be subtracted from day 1 of the month, yielding the last day of the previous month.

If the interpretation of the groups yields an impossible time because a Daylight Saving Time change skips over that time, or an ambiguous time because a Daylight Saving Time change skips back so that the clock observes the given time twice, and no time zone specifier (**%z** or **%Z**) is present in the format, the time is interpreted as if the clock had not changed.

Format groups

The following format groups are recognized by the **clock scan** and **clock format** commands.

%a
: On output, produces an abbreviation (*e.g.,* **Mon**) for the day of the week in the given locale. On input, matches the name of the day of the week in the given locale (in either abbreviated or full form, or any unique prefix of either form).

%A
: On output, produces the full name (*e.g.,* **Monday**) of the day of the week in the given locale. On input, matches the name of the day of the week in the given locale (in either abbreviated or full form, or any unique prefix of either form).

%b
: On output, produces an abbreviation (*e.g.,* **Jan**) for the name of the month in the given locale. On input, matches the name of the month in the given locale (in either abbreviated or full form, or any unique prefix of either form).

%B
: On output, produces the full name (*e.g.,* **January**) of the month in the given locale. On input,

matches the name of the month in the given locale (in either abbreviated or full form, or any unique prefix of either form).

%c
On output, produces a localized representation of date and time of day; the localized representation is expected to use the Gregorian calendar. On input, matches whatever **%c** produces.

%C
On output, produces the number of the century in Indo-Arabic numerals. On input, matches one or two digits, possibly with leading whitespace, that are expected to be the number of the century.

%d
On output, produces the number of the day of the month, as two decimal digits. On input, matches one or two digits, possibly with leading whitespace, that are expected to be the number of the day of the month.

%D
This format group is synonymous with **%m/%d/%Y**. It should be used only in exchanging data within the **en_US** locale, since other locales typically do not use this order for the fields of the date.

%e
On output, produces the number of the day of the month, as one or two decimal digits (with a leading blank for one-digit dates). On input, matches one or two digits, possibly with leading whitespace, that are expected to be the number of the day of the month.

%Ec
On output, produces a locale-dependent representation of the date and time of day in the locale's alternative calendar. On input, matches whatever **%Ec** produces. The locale's alternative calendar need not be the Gregorian calendar.

%EC
On output, produces a locale-dependent name of an era in the locale's alternative calendar. On input, matches the name of the era or any unique prefix.

%EE
On output, produces the string **B.C.E.** or **C.E.**, or a string of the same meaning in the locale, to indicate whether **%Y** refers to years before or after Year 1 of the Common Era. On input, accepts the string **B.C.E.**, **B.C.**, **C.E.**, **A.D.**, or the abbreviation appropriate to the current locale, and uses it to fix whether **%Y** refers to years before or after Year 1 of the Common Era.

%Ex
On output, produces a locale-dependent representation of the date in the locale's alternative calendar. On input, matches whatever **%Ex** produces. The locale's alternative calendar need not be the Gregorian calendar.

%EX
On output, produces a locale-dependent representation of the time of day in the locale's alternative numerals. On input, matches whatever **%EX** produces.

%Ey
On output, produces a locale-dependent number of the year of the era in the locale's alternative calendar and numerals. On input, matches such a number.

%EY

On output, produces a representation of the year in the locale's alternative calendar and numerals. On input, matches what **%EY** produces. Often synonymous with **%EC%Ey**.

%g
On output, produces a two-digit year number suitable for use with the week-based ISO8601 calendar; that is, the year number corresponds to the week number produced by **%V**. On input, accepts such a two-digit year number, possibly with leading whitespace.

%G
On output, produces a four-digit year number suitable for use with the week-based ISO8601 calendar; that is, the year number corresponds to the week number produced by **%V**. On input, accepts such a four-digit year number, possibly with leading whitespace.

%h
This format group is synonymous with **%b**.

%H
On output, produces a two-digit number giving the hour of the day (00-23) on a 24-hour clock. On input, accepts such a number.

%I
On output, produces a two-digit number giving the hour of the day (12-11) on a 12-hour clock. On input, accepts such a number.

%j
On output, produces a three-digit number giving the day of the year (001-366). On input, accepts such a number.

%J
On output, produces a string of digits giving the Julian Day Number. On input, accepts a string of digits and interprets it as a Julian Day Number. The Julian Day Number is a count of the number of calendar days that have elapsed since 1 January, 4713 BCE of the proleptic Julian calendar. The epoch time of 1 January 1970 corresponds to Julian Day Number 2440588.

%k
On output, produces a one- or two-digit number giving the hour of the day (0-23) on a 24-hour clock. On input, accepts such a number.

%l
On output, produces a one- or two-digit number giving the hour of the day (12-11) on a 12-hour clock. On input, accepts such a number.

%m
On output, produces the number of the month (01-12) with exactly two digits. On input, accepts two digits and interprets them as the number of the month.

%M
On output, produces the number of the minute of the hour (00-59) with exactly two digits. On input, accepts two digits and interprets them as the number of the minute of the hour.

%N
On output, produces the number of the month (1-12) with one or two digits, and a leading blank for one-digit dates. On input, accepts one or two digits, possibly with leading whitespace, and interprets them as the number of the month.

%Od, %Oe, %OH, %OI, %Ok, %Ol, %Om, %OM, %OS, %Ou, %Ow, %Oy
All of these format groups are synonymous with their counterparts without the "**O**", except

that the string is produced and parsed in the locale-dependent alternative numerals.

%p

On output, produces an indicator for the part of the day, **AM** or **PM**, appropriate to the given locale. If the script of the given locale supports multiple letterforms, lowercase is preferred. On input, matches the representation **AM** or **PM** in the given locale, in either case.

%P

On output, produces an indicator for the part of the day, **am** or **pm**, appropriate to the given locale. If the script of the given locale supports multiple letterforms, uppercase is preferred. On input, matches the representation **AM** or **PM** in the given locale, in either case.

%Q

This format group is reserved for internal use within the Tcl library.

%r

On output, produces a locale-dependent time of day representation on a 12-hour clock. On input, accepts whatever **%r** produces.

%R

On output, the time in 24-hour notation (%H:%M). For a version including the seconds, see **%T** below. On input, accepts whatever **%R** produces.

%s

On output, simply formats the *timeVal* argument as a decimal integer and inserts it into the output string. On input, accepts a decimal integer and uses is as the time value without any further processing. Since **%s** uniquely determines a point in time, it overrides all other input formats.

%S

On output, produces a two-digit number of the second of the minute (00-59). On input, accepts two digits and uses them as the second of the minute.

%t

On output, produces a TAB character. On input, matches a TAB character.

%T

Synonymous with **%H:%M:%S**.

%u

On output, produces the number of the day of the week (**1**→Monday, **7**→Sunday). On input, accepts a single digit and interprets it as the day of the week. Sunday may be either **0** or **7**.

%U

On output, produces the ordinal number of the week of the year (00-53). The first Sunday of the year is the first day of week 01. On input accepts two digits which are otherwise ignored. This format group is never used in determining an input date. This interpretation of the week of the year was once common in US banking but is now largely obsolete. See **%V** for the ISO8601 week number.

%V

On output, produces the number of the ISO8601 week as a two digit number (01-53). Week 01 is the week containing January 4; or the first week of the year containing at least 4 days; or the week containing the first Thursday of the year (the three statements are equivalent). Each week begins on a Monday. On input, accepts the ISO8601 week number.

%w

On output, produces the ordinal number of the day of the week (Sunday==0; Saturday==6). On input, accepts a single digit and interprets it as the day of the week; Sunday may be represented as either 0 or 7. Note that **%w** is not the ISO8601 weekday number, which is produced and accepted by **%u**.

%W
On output, produces a week number (00-53) within the year; week 01 begins on the first Monday of the year. On input, accepts two digits, which are otherwise ignored. This format group is never used in determining an input date. It is not the ISO8601 week number; that week is produced and accepted by **%V**.

%x
On output, produces the date in a locale-dependent representation. On input, accepts whatever **%x** produces and is used to determine calendar date.

%X
On output, produces the time of day in a locale-dependent representation. On input, accepts whatever **%X** produces and is used to determine time of day.

%y
On output, produces the two-digit year of the century. On input, accepts two digits, and is used to determine calendar date. The date is presumed to lie between 1938 and 2037 inclusive. Note that **%y** does not yield a year appropriate for use with the ISO8601 week number **%V**; programs should use **%g** for that purpose.

%Y
On output, produces the four-digit calendar year. On input, accepts four digits and may be used to determine calendar date. Note that **%Y** does not yield a year appropriate for use with the ISO8601 week number **%V**; programs should use **%G** for that purpose.

%z
On output, produces the current time zone, expressed in hours and minutes east (+hhmm) or west (-hhmm) of Greenwich. On input, accepts a time zone specifier (see Time zones below) that will be used to determine the time zone.

%Z
On output, produces the current time zone's name, possibly translated to the given locale. On input, accepts a time zone specifier (see Time zones below) that will be used to determine the time zone. This option should, in general, be used on input only when parsing RFC822 dates. Other uses are fraught with ambiguity; for instance, the string **BST** may represent British Summer Time or Brazilian Standard Time. It is recommended that date/time strings for use by computers use numeric time zones instead.

%%
On output, produces a literal "**%**" character. On input, matches a literal "**%**" character.

%+
Synonymous with "**%a %b %e %H:%M:%S %Z %Y**".

Time zones

When the **clock** command is processing a local time, it has several possible sources for the time zone to use. In order of preference, they are:

1. A time zone specified inside a string being parsed and matched by a **%z** or **%Z** format group.
2. A time zone specified with the **-timezone** option to the **clock** command (or, equivalently, by

-gmt 1).
3. A time zone specified in an environment variable **TCL_TZ**.
4. A time zone specified in an environment variable **TZ**.
5. The local time zone from the Control Panel on Windows systems.
6. The C library's idea of the local time zone, as defined by the **mktime** and **localtime** functions.

In case [1] *only,* the string is tested to see if it is one of the strings:

```
gmt     ut      utc     bst     wet     wat     at
nft     nst     ndt     ast     adt     est     edt
cst     cdt     mst     mdt     pst     pdt     yst
ydt     hst     hdt     cat     ahst    nt      idlw
cet     cest    met     mewt    mest    swt     sst
eet     eest    bt      it      zp4     zp5     ist
zp6     wast    wadt    jt      cct     jst     cast
cadt    east    eadt    gst     nzt     nzst    nzdt
idle
```

If it is a string in the above list, it designates a known time zone, and is interpreted as such.

For time zones in case [1] that do not match any of the above strings, and always for cases [2]-[6], the following rules apply.

If the time zone begins with a colon, it is one of a standardized list of names like **:America/New_York** that give the rules for various locales. A complete list of the location names is too lengthy to be listed here. On most Tcl installations, the definitions of the locations are to be found in named files in the directory "*/no_backup/tools/lib/tcl8.5/clock/tzdata*". On some Unix systems, these files are omitted, and the definitions are instead obtained from system files in "*/usr/share/zoneinfo*", "*/usr/share/lib/zoneinfo*" or "*/usr/local/etc/zoneinfo*". As a special case, the name **:localtime** refers to the local time zone as defined by the C library.

A time zone string consisting of a plus or minus sign followed by four or six decimal digits is interpreted as an offset in hours, minutes, and seconds (if six digits are present) from UTC. The plus sign denotes a sign east of Greenwich; the minus sign one west of Greenwich.

A time zone string conforming to the Posix specification of the **TZ** environment variable will be recognized. The specification may be found at https://pubs.opengroup.org/onlinepubs/009695399/basedefs/xbd_chap08.html.

If the Posix time zone string contains a DST (Daylight Savings Time) part, but doesn't contain a rule stating when DST starts or ends, then default rules are used. For Timezones with an offset between 0 and +12, the current European/Russian rules are used, otherwise the current US rules are used. In Europe (offset +0 to +2) the switch to summertime is done each last Sunday in March at 1:00 GMT, and the switch back is each last Sunday in October at 2:00 GMT. In Russia (offset +3 to +12), the switch dates are the same, only the switch to summertime is at 2:00 local time, and the switch back is at 3:00 local time in all time zones. The US switch to summertime takes place each second Sunday in March at 2:00 local time, and the switch back is each first Sunday in November at 3:00 local time. These default rules mean that in all European, Russian and US (or compatible) time zones, DST calculations will be correct for dates in 2007 and later, unless in the future the rules change again.

Any other time zone string is processed by prefixing a colon and attempting to use it as a location name, as above.

Localization

Developers wishing to localize the date and time formatting and parsing are referred to https://tip.tcl-lang.org/173 for a specification.

Free form scan

If the **clock scan** command is invoked without a **-format** option, then it requests a *free-form scan. This form of scan is deprecated.* The reason for the deprecation is that there are too many ambiguities. (Does the string "2000" represent a year, a time of day, or a quantity?) No set of rules for interpreting free-form dates and times has been found to give unsurprising results in all cases.

If free-form scan is used, only the **-base** and **-gmt** options are accepted. The **-timezone** and **-locale** options will result in an error if **-format** is not supplied.

For the benefit of users who need to understand legacy code that uses free-form scan, the documentation for how free-form scan interprets a string is included here:

If only a time is specified, the current date is assumed. If the *inputString* does not contain a time zone mnemonic, the local time zone is assumed, unless the **-gmt** argument is true, in which case the clock value is calculated assuming that the specified time is relative to Greenwich Mean Time. **-gmt**, if specified, affects only the computed time value; it does not impact the interpretation of **-base**.

If the **-base** flag is specified, the next argument should contain an integer clock value. Only the date in this value is used, not the time. This is useful for determining the time on a specific day or doing other date-relative conversions.

The *inputString* argument consists of zero or more specifications of the following form:

time
: A time of day, which is of the form: **hh?:mm?:ss?? ?meridian? ?zone?** or **hhmm ?meridian? ?zone?** If no meridian is specified, **hh** is interpreted on a 24-hour clock.

date
: A specific month and day with optional year. The acceptable formats are "**mm/dd**?**/yy**?", "**monthname dd**?**, yy**?", "**day, dd monthname** ?**yy**?", "**dd monthname yy**", "?**CC**?**yymmdd**", and "**dd-monthname-**?**CC**?**yy**". The default year is the current year. If the year is less than 100, we treat the years 00-68 as 2000-2068 and the years 69-99 as 1969-1999. Not all platforms can represent the years 38-70, so an error may result if these years are used.

ISO 8601 point-in-time
: An ISO 8601 point-in-time specification, such as "*CCyymmdd***T***hhmmss*", where **T** is the literal "T", "*CCyymmdd hhmmss*", "*CCyymmdd***T***hh:mm:ss*", or "*CCyy-mm-dd***T***hh:mm:ss*". Note that only these four formats are accepted. The command does *not* accept the full range of point-in-time specifications specified in ISO8601. Other formats can be recognized by giving an explicit **-format** option to the **clock scan** command.

relative time
: A specification relative to the current time. The format is **number unit**. Acceptable units are **year**, **fortnight**, **month**, **week**, **day**, **hour**, **minute** (or **min**), and **second** (or **sec**). The unit

can be specified as a singular or plural, as in **3 weeks**. These modifiers may also be specified: **tomorrow**, **yesterday**, **today**, **now**, **last**, **this**, **next**, **ago**.

The actual date is calculated according to the following steps.

First, any absolute date and/or time is processed and converted. Using that time as the base, day-of-week specifications are added. Next, relative specifications are used. If a date or day is specified, and no absolute or relative time is given, midnight is used. Finally, a correction is applied so that the correct hour of the day is produced after allowing for daylight savings time differences and the correct date is given when going from the end of a long month to a short month.

Copyright

Copyright © 2004 Kevin B. Kenny kennykb@acm.org. All rights reserved.

Copyright (c) 2004 Kevin B. Kenny kennykb@acm.org. All rights reserved.

close

Keywords: close, blocking, channel, nonblocking, half-close

Name

close - Close an open channel

Synopsis

close *channelId* ?*r(ead)|w(rite)*?

Description

Closes or half-closes the channel given by *channelId*.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension.

The single-argument form is a simple "full-close": all buffered output is flushed to the channel's output device, any buffered input is discarded, the underlying file or device is closed, and *channelId* becomes unavailable for use.

If the channel is blocking, the command does not return until all output is flushed. If the channel is nonblocking and there is unflushed output, the channel remains open and the command returns immediately; output will be flushed in the background and the channel will be closed when all the flushing is complete.

If *channelId* is a blocking channel for a command pipeline then **close** waits for the child processes to complete.

If the channel is shared between interpreters, then **close** makes *channelId* unavailable in the invoking interpreter but has no other effect until all of the sharing interpreters have closed the channel. When the last interpreter in which the channel is registered invokes **close**, the cleanup actions described above occur. See the interp command for a description of channel sharing.

Channels are automatically closed when an interpreter is destroyed and when the process exits. From 8.6 on (TIP#398), nonblocking channels are no longer switched to blocking mode when exiting; this guarantees a timely exit even when the peer or a communication channel is stalled.

To ensure proper flushing of stalled nonblocking channels on exit, one must now either (a) actively switch them back to blocking or (b) use the environment variable TCL_FLUSH_NONBLOCKING_ON_EXIT, which when set and not equal to "0" restores the previous behavior.

The command returns an empty string, and may generate an error if an error occurs while flushing output. If a command in a command pipeline created with open returns an error, **close** generates an error (similar to the exec command.)

The two-argument form is a "half-close": given a bidirectional channel like a socket or command pipeline and a (possibly abbreviated) direction, it closes only the sub-stream going in that direction. This means a shutdown() on a socket, and a close() of one end of a pipe for a command pipeline. Then, the Tcl-level channel data structure is either kept or freed depending on whether the other direction is still open.

A single-argument close on an already half-closed bidirectional channel is defined to just "finish the job". A half-close on an already closed half, or on a wrong-sided unidirectional channel, raises an error.

In the case of a command pipeline, the child-reaping duty falls upon the shoulders of the last close or half-close, which is thus allowed to report an abnormal exit error.

Currently only sockets and command pipelines support half-close. A future extension will allow reflected and stacked channels to do so.

Example

This illustrates how you can use Tcl to ensure that files get closed even when errors happen by combining catch, **close** and return:

```
proc withOpenFile {filename channelVar script} {
    upvar 1 $channelVar chan
    set chan [open $filename]
    catch {
        uplevel 1 $script
    } result options
    close $chan
    return -options $options $result
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

concat

Keywords: concat, concatenate, join, list

Name

concat - Join lists together

Synopsis

concat ?*arg arg* ...?

Description
This command joins each of its arguments together with spaces after trimming leading and trailing white-space from each of them. If all of the arguments are lists, this has the same effect as concatenating them into a single list. Arguments that are empty (after trimming) are ignored entirely. It permits any number of arguments; if no *arg*s are supplied, the result is an empty string.

Examples
Although **concat** will concatenate lists, flattening them in the process (so giving the following interactive session):

```
% concat a b {c d e} {f {g h}}
a b c d e f {g h}
```

it will also concatenate things that are not lists, as can be seen from this session:

```
% concat " a b {c   " d "  e} f"
a b {c d e} f
```

Note also that the concatenation does not remove spaces from the middle of values, as can be seen here:

```
% concat "a   b   c" { d e f }
a   b   c d e f
```

(i.e., there are three spaces between each of the **a**, the **b** and the **c**).

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

continue
Keywords: continue, iteration, loop

Name
continue - Skip to the next iteration of a loop

Synopsis
continue

Description
This command is typically invoked inside the body of a looping command such as for or foreach or while. It returns a 4 (TCL_CONTINUE) result code, which causes a continue exception to occur. The exception causes the current script to be aborted out to the innermost containing loop command, which then continues with the next iteration of the loop. Catch exceptions are also handled in a few other situations, such as the catch command and the outermost scripts of procedure bodies.

Example
Print a line for each of the integers from 0 to 10 *except* 5:

```
for {set x 0} {$x<10} {incr x} {
    if {$x == 5} {
        continue
    }
    puts "x is $x"
}
```

Copyright (c) 1993-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

copy

Keywords: copy, clone, duplication, object

Name

oo::copy - create copies of objects and classes

Synopsis

package require TclOO

oo::copy *sourceObject* ?*targetObject*? ?*targetNamespace*?

Description

The **oo::copy** command creates a copy of an object or class. It takes the name of the object or class to be copied, *sourceObject*, and optionally the name of the object or class to create, *targetObject*, which will be resolved relative to the current namespace if not an absolute qualified name and targetNamespace which is the name of the namespace that will hold the internal state of the object (my command, etc.); it must not refer to an existing namespace. If either targetObject or targetNamespace is omitted or is given as the empty string, a new name is chosen. Names, unless specified, are chosen with the same algorithm used by the new method of oo::class. The copied object will be of the same class as the source object, and will have all its per-object methods copied. If it is a class, it will also have all the class methods in the class copied, but it will not have any of its instances copied.

After the targetObject has been created and all definitions of its configuration (e.g., methods, filters, mixins) copied, the method of targetObject will be invoked, to allow for customization of the created object such as installing related variable traces. The only argument given will be sourceObject. The default implementation of this method (in oo::object) just copies the procedures and variables in the namespace of sourceObject to the namespace of targetObject. If this method call does not return a result that is successful (i.e., an error or other kind of exception) then the targetObject will be deleted and an error returned.

The result of the **oo::copy** command will be the fully-qualified name of the new object or class.

Examples

This example creates an object, copies it, modifies the source object, and then demonstrates that the copied object is indeed a copy.

```
oo::object create src
oo::objdefine src method msg {} {puts foo}
oo::copy src dst
oo::objdefine src method msg {} {puts bar}
src msg             → prints "bar"
dst msg             → prints "foo"
```

Copyright (c) 2007 Donal K. Fellows

coroutine

Keywords: coroutine, generator

Name

coroutine, yield, yieldto - Create and produce values from coroutines

Synopsis

coroutine *name command* ?*arg...*?
yield ?*value*?
yieldto *command* ?*arg...*?
name ?*value...*?

Description

The **coroutine** command creates a new coroutine context (with associated command) named *name* and executes that context by calling *command*, passing in the other remaining arguments without further interpretation. Once *command* returns normally or with an exception (e.g., an error) the coroutine context *name* is deleted.

Within the context, values may be generated as results by using the **yield** command; if no *value* is supplied, the empty string is used. When that is called, the context will suspend execution and the **coroutine** command will return the argument to **yield**. The execution of the context can then be resumed by calling the context command, optionally passing in the *single* value to use as the result of the **yield** call that caused the context to be suspended. If the coroutine context never yields and instead returns conventionally, the result of the **coroutine** command will be the result of the evaluation of the context.

The coroutine may also suspend its execution by use of the yieldto command, which instead of returning, cedes execution to some command called command (resolved in the context of the coroutine) and to which any number of arguments may be passed. Since every coroutine has a context command, yieldto can be used to transfer control directly from one coroutine to another (this is only advisable if the two coroutines are expecting this to happen) but any command may be the target. If a coroutine is suspended by this mechanism, the coroutine processing can be resumed by calling the context command optionally passing in an arbitrary number of arguments. The return value of the yieldto call will be the list of arguments passed to the context command; it is up to the caller to decide what to do with those values.

The recommended way of writing a version of **yield** that allows resumption with multiple arguments is by using **yieldto** and the return command, like this:

```
proc yieldm {value} {
    yieldto return -level 0 $value
}
```

The coroutine can also be deleted by destroying the command *name*, and the name of the current coroutine can be retrieved by using info coroutine. If there are deletion traces on variables in the coroutine's implementation, they will fire at the point when the coroutine is explicitly deleted (or, naturally, if the command returns conventionally).

At the point when *command* is called, the current namespace will be the global namespace and there will be no stack frames above it (in the sense of upvar and uplevel). However, which command to call will be determined in the namespace that the **coroutine** command was called from.

Examples

This example shows a coroutine that will produce an infinite sequence of even values, and a loop that consumes the first ten of them.

```
proc allNumbers {} {
    yield
    set i 0
    while 1 {
        yield $i
        incr i 2
    }
}
coroutine nextNumber allNumbers
for {set i 0} {$i < 10} {incr i} {
    puts "received [nextNumber]"
}
rename nextNumber {}
```

In this example, the coroutine acts to add up the arguments passed to it.

```
coroutine accumulator apply {{} {
    set x 0
    while 1 {
        incr x [yield $x]
    }
}}
for {set i 0} {$i < 10} {incr i} {
    puts "$i -> [accumulator $i]"
}
```

This example demonstrates the use of coroutines to implement the classic Sieve of Eratosthenes algorithm for finding prime numbers. Note the creation of coroutines inside a coroutine.

```
proc filterByFactor {source n} {
    yield [info coroutine]
    while 1 {
        set x [$source]
        if {$x % $n} {
            yield $x
        }
    }
}
coroutine allNumbers apply {{} {while 1 {yield [incr x]}}}
coroutine eratosthenes apply {c {
    yield
    while 1 {
        set n [$c]
        yield $n
        set c [coroutine prime$n filterByFactor $c $n]
    }
}} allNumbers
for {set i 1} {$i <= 20} {incr i} {
    puts "prime#$i = [eratosthenes]"
}
```

This example shows how a value can be passed around a group of three coroutines that yield to each other:

```
proc juggler {name target {value ""}} {
    if {$value eq ""} {
        set value [yield [info coroutine]]
    }
    while {$value ne ""} {
        puts "$name : $value"
        set value [string range $value 0 end-1]
        lassign [yieldto $target $value] value
    }
}
coroutine j1 juggler Larry [
    coroutine j2 juggler Curly [
        coroutine j3 juggler Moe j1]] "Nyuck!Nyuck!Nyuck!"
```

Detailed semantics

This example demonstrates that coroutines start from the global namespace, and that *command* resolution happens before the coroutine stack is created.

```
proc report {where level} {
    # Where was the caller called from?
    set ns [uplevel 2 {namespace current}]
    yield "made $where $level context=$ns name=[info coroutine]"
```

```
}
proc example {} {
    report outer [info level]
}
namespace eval demo {
    proc example {} {
        report inner [info level]
    }
    proc makeExample {} {
        puts "making from [info level]"
        puts [coroutine coroEg example]
    }
    makeExample
}
```

Which produces the output below. In particular, we can see that stack manipulation has occurred (comparing the levels from the first and second line) and that the parent level in the coroutine is the global namespace. We can also see that coroutine names are local to the current namespace if not qualified, and that coroutines may yield at depth (e.g., in called procedures).

```
making from 2
made inner 1 context=:: name=::demo::coroEg
```

Copyright (c) 2009 Donal K. Fellows.

dde

Keywords: dde, application, name, remote execution

Name

dde - Execute a Dynamic Data Exchange command

Synopsis

package require dde 1.4
dde servername ?**-force**? ?*-handler proc*? ?**--**? ?*topic*?
dde execute ?**-async**? ?**-binary**? *service topic data*
dde poke ?**-binary**? *service topic item data*
dde request ?**-binary**? *service topic item*
dde services *service topic*
dde eval ?**-async**? *topic cmd* ?*arg arg* ...?

Description

This command allows an application to send Dynamic Data Exchange (DDE) command when running under Microsoft Windows. Dynamic Data Exchange is a mechanism where applications can exchange raw data. Each DDE transaction needs a *service name* and a *topic*. Both the *service name* and *topic* are application defined; Tcl uses the service name **TclEval**, while the topic name is the name of the interpreter given by **dde servername**. Other applications have their own *service names* and *topics*. For instance, Microsoft Excel has the service name **Excel**.

Dde commands

The following commands are a subset of the full Dynamic Data Exchange set of commands.

dde servername ?-force? ?-handler *proc*? ?--? ?*topic*?

dde servername registers the interpreter as a DDE server with the service name **TclEval** and the topic name specified by *topic*. If no *topic* is given, **dde servername** returns the name of the current topic or the empty string if it is not registered as a service. If the given *topic* name is already in use, then a suffix of the form " #2" or " #3" is appended to the name to make it unique. The command's result will be the name actually used. The **-force** option is used to force registration of precisely the given *topic* name.

The **-handler** option specifies a Tcl procedure that will be called to process calls to the dde server. If the package has been loaded into a safe interpreter then a **-handler** procedure must be defined. The procedure is called with all the arguments provided by the remote call.

dde execute ?-async? ?-binary? *service topic data*

dde execute takes the *data* and sends it to the server indicated by *service* with the topic indicated by *topic*. Typically, *service* is the name of an application, and *topic* is a file to work on. The *data* field is given to the remote application. Typically, the application treats the *data* field as a script, and the script is run in the application. The **-async** option requests asynchronous invocation. The command returns an error message if the script did not run, unless the **-async** flag was used, in which case the command returns immediately with no error. Without the -binary option all data will be sent in unicode. For dde clients which don't implement the CF_UNICODE clipboard format, this will automatically be translated to the system encoding. You can use the -binary option in combination with the result of encoding convertto to send data in any other encoding.

dde poke ?-binary? *service topic item data*

dde poke passes the *data* to the server indicated by *service* using the *topic* and *item* specified. Typically, *service* is the name of an application. *topic* is application specific but can be a command to the server or the name of a file to work on. The *item* is also application specific and is often not used, but it must always be non-null. The *data* field is given to the remote application. Without the -binary option all data will be sent in unicode. For dde clients which don't implement the CF_UNICODE clipboard format, this will automatically be translated to the system encoding. You can use the -binary option in combination with the result of encoding convertto to send data in any other encoding.

dde request ?-binary? *service topic item*

dde request is typically used to get the value of something; the value of a cell in Microsoft Excel or the text of a selection in Microsoft Word. *service* is typically the name of an application, *topic* is typically the name of the file, and *item* is application-specific. The command returns the value of *item* as defined in the application. Normally this is interpreted to be a string with terminating null. If **-binary** is specified, the result is returned as a byte array.

dde services *service topic*

dde services returns a list of service-topic pairs that currently exist on the machine. If *service* and *topic* are both empty strings ({}), then all service-topic pairs currently available on the system are returned. If *service* is empty and *topic* is not, then all services with the specified topic are returned. If *service* is non-empty and *topic* is, all topics for a given service

are returned. If both are non-empty, if that service-topic pair currently exists, it is returned; otherwise, an empty string is returned.

dde eval ?-async? *topic cmd* ?*arg arg ...*?

dde eval evaluates a command and its arguments using the interpreter specified by *topic*. The DDE service must be the **TclEval** service. The **-async** option requests asynchronous invocation. The command returns an error message if the script did not run, unless the **-async** flag was used, in which case the command returns immediately with no error. This command can be used to replace send on Windows.

Dde and Tcl

A Tcl interpreter always has a service name of **TclEval**. Each different interpreter of all running Tcl applications must be given a unique name specified by **dde servername**. Each interp is available as a DDE topic only if the **dde servername** command was used to set the name of the topic for each interp. So a **dde services TclEval {}** command will return a list of service-topic pairs, where each of the currently running interps will be a topic.

When Tcl processes a **dde execute** command, the data for the execute is run as a script in the interp named by the topic of the **dde execute** command.

When Tcl processes a **dde request** command, it returns the value of the variable given in the dde command in the context of the interp named by the dde topic. Tcl reserves the variable **$TCLEVAL$EXECUTE$RESULT** for internal use, and **dde request** commands for that variable will give unpredictable results.

An external application which wishes to run a script in Tcl should have that script store its result in a variable, run the **dde execute** command, and then run **dde request** to get the value of the variable.

When using DDE, be careful to ensure that the event queue is flushed using either update or vwait. This happens by default when using wish unless a blocking command is called (such as exec without adding the **&** to place the process in the background). If for any reason the event queue is not flushed, DDE commands may hang until the event queue is flushed. This can create a deadlock situation.

Example

This asks Internet Explorer (which must already be running) to go to a particularly important website:

```
package require dde
dde execute -async iexplore WWW_OpenURL http://www.tcl-lang.org/
```

Copyright (c) 1997 Sun Microsystems, Inc.
Copyright (c) 2001 ActiveState Corporation.

define

Keywords: define, class, definition, method, object, slot

Name

oo::define, oo::objdefine, oo::Slot - define and configure classes and objects

Synopsis

package require TclOO

oo::define *class defScript*
oo::define *class subcommand arg* ?*arg* ...?
oo::objdefine *object defScript*
oo::objdefine *object subcommand arg* ?*arg* ...?

oo::Slot *arg...*

Class hierarchy

```
oo::object
   → oo::Slot
```

Description

The **oo::define** command is used to control the configuration of classes, and the **oo::objdefine** command is used to control the configuration of objects (including classes as instance objects), with the configuration being applied to the entity named in the *class* or the *object* argument. Configuring a class also updates the configuration of all subclasses of the class and all objects that are instances of that class or which mix it in (as modified by any per-instance configuration). The way in which the configuration is done is controlled by either the *defScript* argument or by the *subcommand* and following *arg* arguments; when the second is present, it is exactly as if all the arguments from *subcommand* onwards are made into a list and that list is used as the *defScript* argument.

Configuring classes

The following commands are supported in the *defScript* for **oo::define**, each of which may also be used in the *subcommand* form:

constructor *argList bodyScript*
: This creates or updates the constructor for a class. The formal arguments to the constructor (defined using the same format as for the Tcl proc command) will be *argList*, and the body of the constructor will be *bodyScript*. When the body of the constructor is evaluated, the current namespace of the constructor will be a namespace that is unique to the object being constructed. Within the constructor, the next command should be used to call the superclasses' constructors. If *bodyScript* is the empty string, the constructor will be deleted.

deletemethod *name* ?*name ...*?
: This deletes each of the methods called *name* from a class. The methods must have previously existed in that class. Does not affect the superclasses of the class, nor does it affect the subclasses or instances of the class (except when they have a call chain through the class being modified) or the class object itself.

destructor *bodyScript*
: This creates or updates the destructor for a class. Destructors take no arguments, and the body of the destructor will be *bodyScript*. The destructor is called when objects of the class are deleted, and when called will have the object's unique namespace as the current namespace. Destructors should use the next command to call the superclasses' destructors. Note that destructors are not called in all situations (e.g. if the interpreter is destroyed). If *bodyScript* is the empty string, the destructor will be deleted. Note that errors during the

evaluation of a destructor *are not returned* to the code that causes the destruction of an object. Instead, they are passed to the currently-defined bgerror handler.

export *name ?name ...?*
This arranges for each of the named methods, *name*, to be exported (i.e. usable outside an instance through the instance object's command) by the class being defined. Note that the methods themselves may be actually defined by a superclass; subclass exports override superclass visibility, and may in turn be overridden by instances.

filter ?*-slotOperation*? ?*methodName ...*?
This slot (see Slotted definitions below) sets or updates the list of method names that are used to guard whether method call to instances of the class may be called and what the method's results are. Each *methodName* names a single filtering method (which may be exposed or not exposed); it is not an error for a non-existent method to be named since they may be defined by subclasses. By default, this slot works by appending.

forward *name cmdName ?arg ...?*
This creates or updates a forwarded method called *name*. The method is defined be forwarded to the command called *cmdName*, with additional arguments, *arg* etc., added before those arguments specified by the caller of the method. The *cmdName* will always be resolved using the rules of the invoking objects' namespaces, i.e., when *cmdName* is not fully-qualified, the command will be searched for in each object's namespace, using the instances' namespace's path, or by looking in the global namespace. The method will be exported if *name* starts with a lower-case letter, and non-exported otherwise.

method *name argList bodyScript*
This creates or updates a method that is implemented as a procedure-like script. The name of the method is *name*, the formal arguments to the method (defined using the same format as for the Tcl proc command) will be *argList*, and the body of the method will be *bodyScript*. When the body of the method is evaluated, the current namespace of the method will be a namespace that is unique to the current object. The method will be exported if *name* starts with a lower-case letter, and non-exported otherwise; this behavior can be overridden via **export** and **unexport**.

mixin ?*-slotOperation*? ?*className ...*?
This slot (see Slotted definitions below) sets or updates the list of additional classes that are to be mixed into all the instances of the class being defined. Each *className* argument names a single class that is to be mixed in. By default, this slot works by replacement.

renamemethod *fromName toName*
This renames the method called *fromName* in a class to *toName*. The method must have previously existed in the class, and *toName* must not previously refer to a method in that class. Does not affect the superclasses of the class, nor does it affect the subclasses or instances of the class (except when they have a call chain through the class being modified), or the class object itself. Does not change the export status of the method; if it was exported before, it will be afterwards.

self ***subcommand arg ...***
see below ...

self ***script***
This command is equivalent to calling **oo::objdefine** on the class being defined (see Configuring objects below for a description of the supported values of *subcommand*). It

follows the same general pattern of argument handling as the **oo::define** and **oo::objdefine** commands, and "**oo::define** *cls* self *subcommand ...*" operates identically to "**oo::objdefine** *cls subcommand ...*".

superclass ?*-slotOperation*? ?*className ...*?
This slot (see Slotted definitions below) allows the alteration of the superclasses of the class being defined. Each *className* argument names one class that is to be a superclass of the defined class. Note that objects must not be changed from being classes to being non-classes or vice-versa, that an empty parent class is equivalent to oo::object, and that the parent classes of oo::object and oo::class may not be modified. By default, this slot works by replacement.

unexport *name* ?*name ...*?
This arranges for each of the named methods, *name*, to be not exported (i.e. not usable outside the instance through the instance object's command, but instead just through the my command visible in each object's context) by the class being defined. Note that the methods themselves may be actually defined by a superclass; subclass unexports override superclass visibility, and may be overridden by instance unexports.

variable ?*-slotOperation*? ?*name ...*?
This slot (see Slotted definitions below) arranges for each of the named variables to be automatically made available in the methods, constructor and destructor declared by the class being defined. Each variable name must not have any namespace separators and must not look like an array access. All variables will be actually present in the instance object on which the method is executed. Note that the variable lists declared by a superclass or subclass are completely disjoint, as are variable lists declared by instances; the list of variable names is just for methods (and constructors and destructors) declared by this class. By default, this slot works by appending.

Configuring objects

The following commands are supported in the *defScript* for **oo::objdefine**, each of which may also be used in the *subcommand* form:

class *className*
This allows the class of an object to be changed after creation. Note that the class's constructors are not called when this is done, and so the object may well be in an inconsistent state unless additional configuration work is done.

deletemethod *name* ?*name ...*
This deletes each of the methods called *name* from an object. The methods must have previously existed in that object (e.g., because it was created through **oo::objdefine method**). Does not affect the classes that the object is an instance of, or remove the exposure of those class-provided methods in the instance of that class.

export *name* ?*name ...*?
This arranges for each of the named methods, *name*, to be exported (i.e. usable outside the object through the object's command) by the object being defined. Note that the methods themselves may be actually defined by a class or superclass; object exports override class visibility.

filter ?*-slotOperation*? ?*methodName ...*?
This slot (see Slotted definitions below) sets or updates the list of method names that are

used to guard whether a method call to the object may be called and what the method's results are. Each *methodName* names a single filtering method (which may be exposed or not exposed); it is not an error for a non-existent method to be named. Note that the actual list of filters also depends on the filters set upon any classes that the object is an instance of. By default, this slot works by appending.

forward *name cmdName* ?*arg ...*?
This creates or updates a forwarded object method called *name*. The method is defined be forwarded to the command called *cmdName*, with additional arguments, *arg* etc., added before those arguments specified by the caller of the method. Forwarded methods should be deleted using the **method** subcommand. The method will be exported if *name* starts with a lower-case letter, and non-exported otherwise.

method *name argList bodyScript*
This creates, updates or deletes an object method. The name of the method is *name*, the formal arguments to the method (defined using the same format as for the Tcl proc command) will be *argList*, and the body of the method will be *bodyScript*. When the body of the method is evaluated, the current namespace of the method will be a namespace that is unique to the object. The method will be exported if *name* starts with a lower-case letter, and non-exported otherwise.

mixin ?*-slotOperation*? ?*className ...*?
This slot (see Slotted definitions below) sets or updates a per-object list of additional classes that are to be mixed into the object. Each argument, *className*, names a single class that is to be mixed in. By default, this slot works by replacement.

renamemethod *fromName toName*
This renames the method called *fromName* in an object to *toName*. The method must have previously existed in the object, and *toName* must not previously refer to a method in that object. Does not affect the classes that the object is an instance of and cannot rename in an instance object the methods provided by those classes (though a **oo::objdefine forward**ed method may provide an equivalent capability). Does not change the export status of the method; if it was exported before, it will be afterwards.

unexport *name* ?*name ...*?
This arranges for each of the named methods, *name*, to be not exported (i.e. not usable outside the object through the object's command, but instead just through the my command visible in the object's context) by the object being defined. Note that the methods themselves may be actually defined by a class; instance unexports override class visibility.

variable ?*-slotOperation*? ?*name ...*?
This slot (see Slotted definitions below) arranges for each of the named variables to be automatically made available in the methods declared by the object being defined. Each variable name must not have any namespace separators and must not look like an array access. All variables will be actually present in the object on which the method is executed. Note that the variable lists declared by the classes and mixins of which the object is an instance are completely disjoint; the list of variable names is just for methods declared by this object. By default, this slot works by appending.

Slotted definitions

Some of the configurable definitions of a class or object are *slotted definitions*. This means that

the configuration is implemented by a slot object, that is an instance of the class **oo::Slot**, which manages a list of values (class names, variable names, etc.) that comprises the contents of the slot.

The **oo::Slot** class defines three operations (as methods) that may be done on the slot:

***slot* -append ?*member ...*?**
: This appends the given member elements to the slot definition.

***slot* -clear**
: This sets the slot definition to the empty list.

***slot* -set ?*member ...*?**
: This replaces the slot definition with the given member elements.

A consequence of this is that any use of a slot's default operation where the first member argument begins with a hyphen will be an error. One of the above operations should be used explicitly in those circumstances.

You only need to make an instance of **oo::Slot** if you are defininining your own slot that behaves like a standard slot.

Slot implementation

Internally, slot objects also define a method **--default-operation** which is forwarded to the default operation of the slot (thus, for the class "**variable**" slot, this is forwarded to "my -append "), and these methods which provide the implementation interface:

***slot* Get**
: Returns a list that is the current contents of the slot. This method must always be called from a stack frame created by a call to oo::define or oo::objdefine.

slot* Resolve *element
: This converts an element of the slotted collection into its resolved form; for a simple value, it could just return the value, but for a slot that contains references to commands or classes it should convert those into their fully-qualified forms (so they can be compared with string equals): that could be done by forwarding to namespace which or similar.

slot* Set *elementList
: Sets the contents of the slot to the list elementList and returns the empty string. This method must always be called from a stack frame created by a call to oo::define or oo::objdefine.

The implementation of these methods is slot-dependent (and responsible for accessing the correct part of the class or object definition). Slots also have an unknown method handler to tie all these pieces together, and they hide their **destroy** method so that it is not invoked inadvertently. It is *recommended* that any user changes to the slot mechanism itself be restricted to defining new operations whose names start with a hyphen.

Note that slot instances are not expected to contain the storage for the slot they manage; that will be in or attached to the class or object that they manage. Those instances should provide their own implementations of the **Get** and **Set** methods (and optionally **Resolve**; that defaults to a do-nothing pass-through).

Examples

This example demonstrates how to use both forms of the **oo::define** and **oo::objdefine** commands (they work in the same way), as well as illustrating four of the subcommands of

them.

```
oo::class create c
c create o
oo::define c method foo {} {
    puts "world"
}
oo::objdefine o {
    method bar {} {
        my Foo "hello "
        my foo
    }
    forward Foo ::puts -nonewline
    unexport foo
}
o bar                → prints "hello world"
o foo                → error "unknown method foo"
o Foo Bar            → error "unknown method Foo"
oo::objdefine o renamemethod bar lollipop
o lollipop           → prints "hello world"
```

This example shows how additional classes can be mixed into an object. It also shows how **mixin** is a slot that supports appending:

```
oo::object create inst
inst m1             → error "unknown method m1"
inst m2             → error "unknown method m2"

oo::class create A {
    method m1 {} {
        puts "red brick"
    }
}
oo::objdefine inst {
    mixin A
}
inst m1             → prints "red brick"
inst m2             → error "unknown method m2"

oo::class create B {
    method m2 {} {
        puts "blue brick"
    }
}
oo::objdefine inst {
    mixin -append B
}
inst m1             → prints "red brick"
inst m2             → prints "blue brick"
```

Copyright (c) 2007 Donal K. Fellows

dict

Keywords: dict, dictionary, create, update, lookup, iterate, filter, map

Name

dict - Manipulate dictionaries

Synopsis

dict *option arg* ?*arg* ...?

Description

Performs one of several operations on dictionary values or variables containing dictionary values (see the Dictionary values section below for a description), depending on *option*. The legal *option*s (which may be abbreviated) are:

dict append *dictionaryVariable key* ?*string ...*?

This appends the given string (or strings) to the value that the given key maps to in the dictionary value contained in the given variable, writing the resulting dictionary value back to that variable. Non-existent keys are treated as if they map to an empty string. The updated dictionary value is returned.

dict create ?*key value ...*?
Return a new dictionary that contains each of the key/value mappings listed as arguments (keys and values alternating, with each key being followed by its associated value.)

dict exists *dictionaryValue key* ?*key ...*?
This returns a boolean value indicating whether the given key (or path of keys through a set of nested dictionaries) exists in the given dictionary value. This returns a true value exactly when **dict get** on that path will succeed.

dict filter *dictionaryValue filterType arg* ?*arg ...*?
This takes a dictionary value and returns a new dictionary that contains just those key/value pairs that match the specified filter type (which may be abbreviated.) Supported filter types are:

dict filter *dictionaryValue* key ?*globPattern ...*?
The key rule only matches those key/value pairs whose keys match any of the given patterns (in the style of string match.)

dict filter *dictionaryValue* script {*keyVariable valueVariable*} *script*
The script rule tests for matching by assigning the key to the *keyVariable* and the value to the *valueVariable*, and then evaluating the given script which should result in a boolean value (with the key/value pair only being included in the result of the **dict filter** when a true value is returned.) Note that the first argument after the rule selection word is a two-element list. If the *script* returns with a condition of TCL_BREAK, no further key/value pairs are considered for inclusion in the resulting dictionary, and a condition of TCL_CONTINUE is equivalent to a false result. The key/value pairs are tested in the order in which the keys were inserted into the dictionary.

dict filter *dictionaryValue* value ?*globPattern ...*?
The value rule only matches those key/value pairs whose values match any of the given patterns (in the style of string match.)

dict for {*keyVariable valueVariable*} *dictionaryValue body*
This command takes three arguments, the first a two-element list of variable names (for the key and value respectively of each mapping in the dictionary), the second the dictionary value to iterate across, and the third a script to be evaluated for each mapping with the key and value variables set appropriately (in the manner of foreach.) The result of the command is an empty string. If any evaluation of the body generates a TCL_BREAK result, no further pairs from the dictionary will be iterated over and the **dict for** command will terminate successfully immediately. If any evaluation of the body generates a TCL_CONTINUE result, this shall be treated exactly like a normal TCL_OK result. The order of iteration is the order in which the keys were inserted into the dictionary.

dict get *dictionaryValue* ?*key ...*?
Given a dictionary value (first argument) and a key (second argument), this will retrieve the value for that key. Where several keys are supplied, the behaviour of the command shall be as if the result of **dict get $dictVal $key** was passed as the first argument to **dict get** with the remaining arguments as second (and possibly subsequent) arguments. This facilitates lookups in nested dictionaries. For example, the following two commands are equivalent:

```
dict get $dict foo bar spong
dict get [dict get [dict get $dict foo] bar] spong
```

If no keys are provided, **dict get** will return a list containing pairs of elements in a manner similar to array get. That is, the first element of each pair would be the key and the second element would be the value for that key.

It is an error to attempt to retrieve a value for a key that is not present in the dictionary.

dict incr *dictionaryVariable key* ?*increment*?

This adds the given increment value (an integer that defaults to 1 if not specified) to the value that the given key maps to in the dictionary value contained in the given variable, writing the resulting dictionary value back to that variable. Non-existent keys are treated as if they map to 0. It is an error to increment a value for an existing key if that value is not an integer. The updated dictionary value is returned.

dict info *dictionaryValue*

This returns information (intended for display to people) about the given dictionary though the format of this data is dependent on the implementation of the dictionary. For dictionaries that are implemented by hash tables, it is expected that this will return the string produced by Tcl_HashStats, similar to array statistics.

dict keys *dictionaryValue* ?*globPattern*?

Return a list of all keys in the given dictionary value. If a pattern is supplied, only those keys that match it (according to the rules of string match) will be returned. The returned keys will be in the order that they were inserted into the dictionary.

dict lappend *dictionaryVariable key* ?*value ...*?

This appends the given items to the list value that the given key maps to in the dictionary value contained in the given variable, writing the resulting dictionary value back to that variable. Non-existent keys are treated as if they map to an empty list, and it is legal for there to be no items to append to the list. It is an error for the value that the key maps to to not be representable as a list. The updated dictionary value is returned.

dict map {*keyVariable valueVariable*} *dictionaryValue body*

This command applies a transformation to each element of a dictionary, returning a new dictionary. It takes three arguments: the first is a two-element list of variable names (for the key and value respectively of each mapping in the dictionary), the second the dictionary value to iterate across, and the third a script to be evaluated for each mapping with the key and value variables set appropriately (in the manner of lmap). In an iteration where the evaluated script completes normally (TCL_OK, as opposed to an error, etc.) the result of the script is put into an accumulator dictionary using the key that is the current contents of the *keyVariable* variable at that point. The result of the **dict map** command is the accumulator dictionary after all keys have been iterated over.

If the evaluation of the body for any particular step generates a break, no further pairs from the dictionary will be iterated over and the **dict map** command will terminate successfully immediately. If the evaluation of the body for a particular step generates a continue result, the current iteration is aborted and the accumulator dictionary is not modified. The order of iteration is the natural order of the dictionary (typically the order in which the keys were added to the dictionary; the order is the same as that used in **dict for**).

dict merge ?*dictionaryValue ...*?

Return a dictionary that contains the contents of each of the *dictionaryValue* arguments. Where two (or more) dictionaries contain a mapping for the same key, the resulting dictionary

maps that key to the value according to the last dictionary on the command line containing a mapping for that key.

dict remove *dictionaryValue* ?*key ...*?
Return a new dictionary that is a copy of an old one passed in as first argument except without mappings for each of the keys listed. It is legal for there to be no keys to remove, and it also legal for any of the keys to be removed to not be present in the input dictionary in the first place.

dict replace *dictionaryValue* ?*key value ...*?
Return a new dictionary that is a copy of an old one passed in as first argument except with some values different or some extra key/value pairs added. It is legal for this command to be called with no key/value pairs, but illegal for this command to be called with a key but no value.

dict set *dictionaryVariable key* ?*key ...*? *value*
This operation takes the name of a variable containing a dictionary value and places an updated dictionary value in that variable containing a mapping from the given key to the given value. When multiple keys are present, this operation creates or updates a chain of nested dictionaries. The updated dictionary value is returned.

dict size *dictionaryValue*
Return the number of key/value mappings in the given dictionary value.

dict unset *dictionaryVariable key* ?*key ...*?
This operation (the companion to **dict set**) takes the name of a variable containing a dictionary value and places an updated dictionary value in that variable that does not contain a mapping for the given key. Where multiple keys are present, this describes a path through nested dictionaries to the mapping to remove. At least one key must be specified, but the last key on the key-path need not exist. All other components on the path must exist. The updated dictionary value is returned.

dict update *dictionaryVariable key varName* ?*key varName ...*? *body*
Execute the Tcl script in *body* with the value for each *key* (as found by reading the dictionary value in *dictionaryVariable*) mapped to the variable *varName*. There may be multiple *key*/*varName* pairs. If a *key* does not have a mapping, that corresponds to an unset *varName*. When *body* terminates, any changes made to the *varName*s is reflected back to the dictionary within *dictionaryVariable* (unless *dictionaryVariable* itself becomes unreadable, when all updates are silently discarded), even if the result of *body* is an error or some other kind of exceptional exit. The result of **dict update** is (unless some kind of error occurs) the result of the evaluation of *body*.

Each *varName* is mapped in the scope enclosing the **dict update**; it is recommended that this command only be used in a local scope (procedure, lambda term for apply, or method). Because of this, the variables set by **dict update** will continue to exist after the command finishes (unless explicitly unset). Note that the mapping of values to variables does not use traces; changes to the *dictionaryVariable*'s contents only happen when *body* terminates.

dict values *dictionaryValue* ?*globPattern*?
Return a list of all values in the given dictionary value. If a pattern is supplied, only those values that match it (according to the rules of string match) will be returned. The returned values will be in the order of that the keys associated with those values were inserted into the

dictionary.

dict with *dictionaryVariable* ?*key ...*? *body*

Execute the Tcl script in *body* with the value for each key in *dictionaryVariable* mapped (in a manner similarly to **dict update**) to a variable with the same name. Where one or more *key*s are available, these indicate a chain of nested dictionaries, with the innermost dictionary being the one opened out for the execution of *body*. As with **dict update**, making *dictionaryVariable* unreadable will make the updates to the dictionary be discarded, and this also happens if the contents of *dictionaryVariable* are adjusted so that the chain of dictionaries no longer exists. The result of **dict with** is (unless some kind of error occurs) the result of the evaluation of *body*.

The variables are mapped in the scope enclosing the **dict with**; it is recommended that this command only be used in a local scope (procedure, lambda term for apply, or method). Because of this, the variables set by **dict with** will continue to exist after the command finishes (unless explicitly unset). Note that the mapping of values to variables does not use traces; changes to the *dictionaryVariable*'s contents only happen when *body* terminates.

If the *dictionaryVariable* contains a value that is not a dictionary at the point when the *body* terminates (which can easily happen if the name is the same as any of the keys in dictionary) then an error occurs at that point. This command is thus not recommended for use when the keys in the dictionary are expected to clash with the *dictionaryVariable* name itself. Where the contained key does map to a dictionary, the net effect is to combine that inner dictionary into the outer dictionary; see the Examples below for an illustration of this.

Dictionary values

Dictionaries are values that contain an efficient, order-preserving mapping from arbitrary keys to arbitrary values. Each key in the dictionary maps to a single value. They have a textual format that is exactly that of any list with an even number of elements, with each mapping in the dictionary being represented as two items in the list. When a command takes a dictionary and produces a new dictionary based on it (either returning it or writing it back into the variable that the starting dictionary was read from) the new dictionary will have the same order of keys, modulo any deleted keys and with new keys added on to the end. When a string is interpreted as a dictionary and it would otherwise have duplicate keys, only the last value for a particular key is used; the others are ignored, meaning that, "apple banana" and "apple carrot apple banana" are equivalent dictionaries (with different string representations).

Operations that derive a new dictionary from an old one (e.g., updates like **dict set** and **dict unset**) preserve the order of keys in the dictionary. The exceptions to this are for any new keys they add, which are appended to the sequence, and any keys that are removed, which are excised from the order.

Examples

Basic dictionary usage:

```
# Make a dictionary to map extensions to descriptions
set filetypes [dict create .txt "Text File" .tcl "Tcl File"]

# Add/update the dictionary
dict set filetypes .tcl "Tcl Script"
dict set filetypes .tm  "Tcl Module"
dict set filetypes .gif "GIF Image"
dict set filetypes .png "PNG Image"

# Simple read from the dictionary
set ext ".tcl"
set desc [dict get $filetypes $ext]
puts "$ext is for a $desc"

# Somewhat more complex, with existence test
foreach filename [glob *] {
    set ext [file extension $filename]
    if {[dict exists $filetypes $ext]} {
        puts "$filename is a [dict get $filetypes $ext]"
    }
}
```

Constructing and using nested dictionaries:

```
# Data for one employee
dict set employeeInfo 12345-A forenames "Joe"
dict set employeeInfo 12345-A surname   "Schmoe"
dict set employeeInfo 12345-A street "147 Short Street"
dict set employeeInfo 12345-A city   "Springfield"
dict set employeeInfo 12345-A phone  "555-1234"
# Data for another employee
dict set employeeInfo 98372-J forenames "Anne"
dict set employeeInfo 98372-J surname   "Other"
dict set employeeInfo 98372-J street "32995 Oakdale Way"
dict set employeeInfo 98372-J city   "Springfield"
dict set employeeInfo 98372-J phone  "555-8765"
# The above data probably ought to come from a database...

# Print out some employee info
set i 0
puts "There are [dict size $employeeInfo] employees"
dict for {id info} $employeeInfo {
    puts "Employee #[incr i]: $id"
    dict with info {
        puts "   Name: $forenames $surname"
        puts "   Address: $street, $city"
        puts "   Telephone: $phone"
```

```
    }
}
# Another way to iterate and pick out names...
foreach id [dict keys $employeeInfo] {
    puts "Hello, [dict get $employeeInfo $id forenames]!"
}
```

A localizable version of string toupper:

```
# Set up the basic C locale
set capital [dict create C [dict create]]
foreach c [split {abcdefghijklmnopqrstuvwxyz} ""] {
    dict set capital C $c [string toupper $c]
}

# English locales can luckily share the "C" locale
dict set capital en [dict get $capital C]
dict set capital en_US [dict get $capital C]
dict set capital en_GB [dict get $capital C]

# ... and so on for other supported languages ...

# Now get the mapping for the current locale and use it.
set upperCaseMap [dict get $capital $env(LANG)]
set upperCase [string map $upperCaseMap $string]
```

Showing the detail of **dict with**:

```
proc sumDictionary {varName} {
    upvar 1 $varName vbl
    foreach key [dict keys $vbl] {
        # Manufacture an entry in the subdictionary
        dict set vbl $key total 0
        # Add the values and remove the old
        dict with vbl $key {
            set total [expr {$x + $y + $z}]
            unset x y z
        }
    }
    puts "last total was $total, for key $key"
}

set myDict {
    a {x 1 y 2 z 3}
    b {x 6 y 5 z 4}
}

sumDictionary myDict
```

```
#    prints: last total was 15, for key b

puts "dictionary is now \"$myDict\""
#    prints: dictionary is now "a {total 6} b {total 15}"
```

When **dict with** is used with a key that clashes with the name of the dictionary variable:

```
set foo {foo {a b} bar 2 baz 3}
dict with foo {}
puts $foo
#    prints: a b foo {a b} bar 2 baz 3
```

Copyright (c) 2003 Donal K. Fellows

encoding

Keywords: encoding, unicode

Name

encoding - Manipulate encodings

Synopsis

encoding *option* ?*arg arg* ...?

Introduction

Strings in Tcl are logically a sequence of 16-bit Unicode characters. These strings are represented in memory as a sequence of bytes that may be in one of several encodings: modified UTF-8 (which uses 1 to 3 bytes per character), 16-bit "Unicode" (which uses 2 bytes per character, with an endianness that is dependent on the host architecture), and binary (which uses a single byte per character but only handles a restricted range of characters). Tcl does not guarantee to always use the same encoding for the same string.

Different operating system interfaces or applications may generate strings in other encodings such as Shift-JIS. The **encoding** command helps to bridge the gap between Unicode and these other formats.

Description

Performs one of several encoding related operations, depending on *option*. The legal *option*s are:

encoding convertfrom ?*encoding*? *data*
: Convert *data* to Unicode from the specified *encoding*. The characters in *data* are treated as binary data where the lower 8-bits of each character is taken as a single byte. The resulting sequence of bytes is treated as a string in the specified *encoding*. If *encoding* is not specified, the current system encoding is used.

encoding convertto ?*encoding*? *string*
: Convert *string* from Unicode to the specified *encoding*. The result is a sequence of bytes that represents the converted string. Each byte is stored in the lower 8-bits of a Unicode character (indeed, the resulting string is a binary string as far as Tcl is concerned, at least initially). If *encoding* is not specified, the current system encoding is used.

encoding dirs ?*directoryList*?

Tcl can load encoding data files from the file system that describe additional encodings for it to work with. This command sets the search path for ***.enc** encoding data files to the list of directories *directoryList*. If *directoryList* is omitted then the command returns the current list of directories that make up the search path. It is an error for *directoryList* to not be a valid list. If, when a search for an encoding data file is happening, an element in *directoryList* does not refer to a readable, searchable directory, that element is ignored.

encoding names

Returns a list containing the names of all of the encodings that are currently available. The encodings "utf-8" and "iso8859-1" are guaranteed to be present in the list.

encoding system ?*encoding*?

Set the system encoding to *encoding*. If *encoding* is omitted then the command returns the current system encoding. The system encoding is used whenever Tcl passes strings to system calls.

Example

The following example converts a byte sequence in Japanese euc-jp encoding to a TCL string:

```
set s [encoding convertfrom euc-jp "\xA4\xCF"]
```

The result is the unicode codepoint: "\u306F", which is the Hiragana letter HA.

Copyright (c) 1998 Scriptics Corporation.

eof

Keywords: eof, channel, end of file

Name

eof - Check for end of file condition on channel

Synopsis

eof *channelId*

Description

Returns 1 if an end of file condition occurred during the most recent input operation on *channelId* (such as gets), 0 otherwise.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension.

Examples

Read and print out the contents of a file line-by-line:

```
set f [open somefile.txt]
while {1} {
    set line [gets $f]
    if {[eof $f]} {
        close $f
        break
```

```
    }
    puts "Read line: $line"
}
```

Read and print out the contents of a file by fixed-size records:

```
set f [open somefile.dat]
fconfigure $f -translation binary
set recordSize 40
while {1} {
    set record [read $f $recordSize]
    if {[eof $f]} {
        close $f
        break
    }
    puts "Read record: $record"
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

error

Keywords: error, exception

Name

error - Generate an error

Synopsis

error *message* ?*info*? ?*code*?

Description

Returns a TCL_ERROR code, which causes command interpretation to be unwound. *Message* is a string that is returned to the application to indicate what went wrong.

The **-errorinfo** return option of an interpreter is used to accumulate a stack trace of what was in progress when an error occurred; as nested commands unwind, the Tcl interpreter adds information to the **-errorinfo** return option. If the *info* argument is present, it is used to initialize the **-errorinfo** return options and the first increment of unwind information will not be added by the Tcl interpreter. In other words, the command containing the **error** command will not appear in the stack trace; in its place will be *info*. Historically, this feature had been most useful in conjunction with the catch command: if a caught error cannot be handled successfully, *info* can be used to return a stack trace reflecting the original point of occurrence of the error:

```
catch {...} errMsg
set savedInfo $::errorInfo
...
error $errMsg $savedInfo
```

When working with Tcl 8.5 or later, the following code should be used instead:

```
catch {...} errMsg options
...
return -options $options $errMsg
```

If the *code* argument is present, then its value is stored in the **-errorcode** return option. The **-errorcode** return option is intended to hold a machine-readable description of the error in cases where such information is available; see the return manual page for information on the proper format for this option's value.

Example

Generate an error if a basic mathematical operation fails:

```
if {1+2 != 3} {
    error "something is very wrong with addition"
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

eval

Keywords: eval, concatenate, evaluate, script

Name

eval - Evaluate a Tcl script

Synopsis

eval *arg* ?*arg* ...?

Description

Eval takes one or more arguments, which together comprise a Tcl script containing one or more commands. **Eval** concatenates all its arguments in the same fashion as the concat command, passes the concatenated string to the Tcl interpreter recursively, and returns the result of that evaluation (or any error generated by it). Note that the list command quotes sequences of words in such a way that they are not further expanded by the **eval** command.

Examples

Often, it is useful to store a fragment of a script in a variable and execute it later on with extra values appended. This technique is used in a number of places throughout the Tcl core (e.g. in fcopy, lsort and trace command callbacks). This example shows how to do this using core Tcl commands:

```
set script {
    puts "logging now"
    lappend $myCurrentLogVar
}
set myCurrentLogVar log1
# Set up a switch of logging variable part way through!
after 20000 set myCurrentLogVar log2
```

```
for {set i 0} {$i<10} {incr i} {
    # Introduce a random delay
    after [expr {int(5000 * rand())}]
    update    ;# Check for the asynch log switch
    eval $script $i [clock clicks]
}
```

Note that in the most common case (where the script fragment is actually just a list of words forming a command prefix), it is better to use **{*}$script** when doing this sort of invocation pattern. It is less general than the **eval** command, and hence easier to make robust in practice. The following procedure acts in a way that is analogous to the lappend command, except it inserts the argument values at the start of the list in the variable:

```
proc lprepend {varName args} {
    upvar 1 $varName var
    # Ensure that the variable exists and contains a list
    lappend var
    # Now we insert all the arguments in one go
    set var [eval [list linsert $var 0] $args]
}
```

However, the last line would now normally be written without **eval**, like this:

```
set var [linsert $var 0 {*}$args]
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

exec

Keywords: exec, execute, pipeline, redirection, subprocess

Name

exec - Invoke subprocesses

Synopsis

exec ?*switches*? *arg* ?*arg* ...? ?**&**?

Description

This command treats its arguments as the specification of one or more subprocesses to execute. The arguments take the form of a standard shell pipeline where each *arg* becomes one word of a command, and each distinct command becomes a subprocess. The result of the command is the standard output of the final subprocess in the pipeline, interpreted using the system encoding; to use any other encoding (especially including binary data), the pipeline must be opened, configured and read explicitly.

If the initial arguments to **exec** start with **-** then they are treated as command-line switches and are not part of the pipeline specification. The following switches are currently supported:

-ignorestderr

Stops the **exec** command from treating the output of messages to the pipeline's standard error channel as an error case.

-keepnewline
Retains a trailing newline in the pipeline's output. Normally a trailing newline will be deleted.

--
Marks the end of switches. The argument following this one will be treated as the first *arg* even if it starts with a **-**.

If an *arg* (or pair of *arg*s) has one of the forms described below then it is used by **exec** to control the flow of input and output among the subprocess(es). Such arguments will not be passed to the subprocess(es). In forms such as "**<** *fileName*", *fileName* may either be in a separate argument from "**<**" or in the same argument with no intervening space (i.e. "**<***fileName*").

|
Separates distinct commands in the pipeline. The standard output of the preceding command will be piped into the standard input of the next command.

|&
Separates distinct commands in the pipeline. Both standard output and standard error of the preceding command will be piped into the standard input of the next command. This form of redirection overrides forms such as 2> and >&.

< fileName
The file named by *fileName* is opened and used as the standard input for the first command in the pipeline.

<@ fileId
FileId must be the identifier for an open file, such as the return value from a previous call to open. It is used as the standard input for the first command in the pipeline. *FileId* must have been opened for reading.

<< value
Value is passed to the first command as its standard input.

> fileName
Standard output from the last command is redirected to the file named *fileName*, overwriting its previous contents.

2> fileName
Standard error from all commands in the pipeline is redirected to the file named *fileName*, overwriting its previous contents.

>& fileName
Both standard output from the last command and standard error from all commands are redirected to the file named *fileName*, overwriting its previous contents.

>> fileName
Standard output from the last command is redirected to the file named *fileName*, appending to it rather than overwriting it.

2>> fileName
Standard error from all commands in the pipeline is redirected to the file named *fileName*, appending to it rather than overwriting it.

>>& fileName
Both standard output from the last command and standard error from all commands are

redirected to the file named *fileName*, appending to it rather than overwriting it.

>@ *fileId*
FileId must be the identifier for an open file, such as the return value from a previous call to open. Standard output from the last command is redirected to *fileId*'s file, which must have been opened for writing.

2>@ *fileId*
FileId must be the identifier for an open file, such as the return value from a previous call to open. Standard error from all commands in the pipeline is redirected to *fileId*'s file. The file must have been opened for writing.

2>@1
Standard error from all commands in the pipeline is redirected to the command result. This operator is only valid at the end of the command pipeline.

>&@ *fileId*
FileId must be the identifier for an open file, such as the return value from a previous call to open. Both standard output from the last command and standard error from all commands are redirected to *fileId*'s file. The file must have been opened for writing.

If standard output has not been redirected then the **exec** command returns the standard output from the last command in the pipeline, unless "2>@1" was specified, in which case standard error is included as well. If any of the commands in the pipeline exit abnormally or are killed or suspended, then **exec** will return an error and the error message will include the pipeline's output followed by error messages describing the abnormal terminations; the **-errorcode** return option will contain additional information about the last abnormal termination encountered. If any of the commands writes to its standard error file and that standard error is not redirected and **-ignorestderr** is not specified, then **exec** will return an error; the error message will include the pipeline's standard output, followed by messages about abnormal terminations (if any), followed by the standard error output.

If the last character of the result or error message is a newline then that character is normally deleted from the result or error message. This is consistent with other Tcl return values, which do not normally end with newlines. However, if **-keepnewline** is specified then the trailing newline is retained.

If standard input is not redirected with "<", "<<" or "<@" then the standard input for the first command in the pipeline is taken from the application's current standard input.

If the last *arg* is "&" then the pipeline will be executed in background. In this case the **exec** command will return a list whose elements are the process identifiers for all of the subprocesses in the pipeline. The standard output from the last command in the pipeline will go to the application's standard output if it has not been redirected, and error output from all of the commands in the pipeline will go to the application's standard error file unless redirected.

The first word in each command is taken as the command name; tilde-substitution is performed on it, and if the result contains no slashes then the directories in the PATH environment variable are searched for an executable by the given name. If the name contains a slash then it must refer to an executable reachable from the current directory. No "glob" expansion or other shell-like substitutions are performed on the arguments to commands.

Portability issues

Windows (all versions)

Reading from or writing to a socket, using the "**@** *fileId*" notation, does not work. When reading from a socket, a 16-bit DOS application will hang and a 32-bit application will return immediately with end-of-file. When either type of application writes to a socket, the information is instead sent to the console, if one is present, or is discarded.

Note that the current escape resp. quoting of arguments for windows works only with executables using CommandLineToArgv, CRT-library or similar, as well as with the windows batch files (excepting the newline, see below). Although it is the common escape algorithm, but, in fact, the way how the executable parses the command-line (resp. splits it into single arguments) is decisive.

Unfortunately, there is currently no way to supply newline character within an argument to the batch files (**.cmd** or **.bat**) or to the command processor (**cmd.exe /c**), because this causes truncation of command-line (also the argument chain) on the first newline character. But it works properly with an executable (using CommandLineToArgv, etc).

Argument quoting

The arguments of the **exec** command are mapped to the arguments of the called program. Additional quote characters (**"**) are automatically added around arguments if expected. Special characters are escaped by inserting backslash characters.

The MS-Windows environment does execute programs mentioned in the arguments and called batch files (conspec) replace environment variables, which may have side effects (vulnerabilities) or break any already existing quoting (for example, if the environment variable contains a special character like a **"**). Examples are:

```
% exec my-echo.cmd {test&whoami}
  test
  mylogin
% exec my-echo.cmd "ENV X:%X%"
  ENV X: CONTENT OF X
```

The following formatting is automatically performed on any argument item to avoid subprogram execution: Any special character argument containing a special character (**&**, **|**, **^**, **<**, **>**,**!**, **(**,**)**, **(**, **%**) is automatically enclosed in quotes (**"**). Any data quote is escaped by insertion of backslash characters.

The automatic resolving of environment variables using "**%var%**" is critical, but has more use than danger and is not escaped.

TCL 8.6.10 refined this quoting by adding quoting for data quotes and individual quoting of "**%** ". This may break present scripts which rely on the replacement functionality of environment variables. Thus, the individual quoting of "**%**" was removed in TCL 8.6.14, as environment variables are seen more helpful than a problem. A solution with command parameters is envisaged for a future release of TCL.

The Tk console text widget does not provide real standard IO capabilities. Under Tk, when redirecting from standard input, all applications will see an immediate end-of-file; information redirected to standard output or standard error will be discarded.

Either forward or backward slashes are accepted as path separators for arguments to Tcl commands. When executing an application, the path name specified for the application may also contain forward or backward slashes as path separators. Bear in mind, however, that most Windows applications accept arguments with forward slashes only as option delimiters and backslashes only in paths. Any arguments to an application that specify a path name with forward slashes will not automatically be converted to use the backslash character. If an argument contains forward slashes as the path separator, it may or may not be recognized as a path name, depending on the program.

Two or more forward or backward slashes in a row in a path refer to a network path. For example, a simple concatenation of the root directory **c:/** with a subdirectory **/windows/system** will yield **c://windows/system** (two slashes together), which refers to the mount point called **system** on the machine called **windows** (and the **c:/** is ignored), and is not equivalent to **c:/windows/system**, which describes a directory on the current computer. The file join command should be used to concatenate path components.

Note that there are two general types of Win32 console applications:
1. CLI — CommandLine Interface, simple stdio exchange. **netstat.exe** for example.
2. TUI — Textmode User Interface, any application that accesses the console API for doing such things as cursor movement, setting text color, detecting key presses and mouse movement, etc. An example would be **telnet.exe** from Windows 2000. These types of applications are not common in a windows environment, but do exist.

exec will not work well with TUI applications when a console is not present, as is done when launching applications under wish. It is desirable to have console applications hidden and detached. This is a designed-in limitation as **exec** wants to communicate over pipes. The Expect extension addresses this issue when communicating with a TUI application.

When attempting to execute an application, **exec** first searches for the name as it was specified. Then, in order, **.com**, **.exe**, **.bat** and **.cmd** are appended to the end of the specified name and it searches for the longer name. If a directory name was not specified as part of the application name, the following directories are automatically searched in order when attempting to locate the application:
- The directory from which the Tcl executable was loaded.
- The current directory.
- The Windows 32-bit system directory.
- The Windows home directory.
- The directories listed in the path.

In order to execute shell built-in commands like **dir** and **copy**, the caller must prepend the desired command with "**cmd.exe /c** " because built-in commands are not implemented using executables.

Unix (including macOS)

The **exec** command is fully functional and works as described.

Unix examples

Here are some examples of the use of the **exec** command on Unix. To execute a simple program and get its result:

```
exec uname -a
```

Working with non-zero results

To execute a program that can return a non-zero result, you should wrap the call to **exec** in catch and check the contents of the **-errorcode** return option if you have an error:

```
set status 0
if {[catch {exec grep foo bar.txt} results options]} {
    set details [dict get $options -errorcode]
    if {[lindex $details 0] eq "CHILDSTATUS"} {
        set status [lindex $details 2]
    } else {
        # Some other error; regenerate it to let caller handle
        return -options $options -level 0 $results
    }
}
```

This is more easily written using the try command, as that makes it simpler to trap specific types of errors. This is done using code like this:

```
try {
    set results [exec grep foo bar.txt]
    set status 0
} trap CHILDSTATUS {results options} {
    set status [lindex [dict get $options -errorcode] 2]
}
```

Working with quoted arguments

When translating a command from a Unix shell invocation, care should be taken over the fact that single quote characters have no special significance to Tcl. Thus:

```
awk '{sum += $1} END {print sum}' numbers.list
```

would be translated into something like:

```
exec awk {{sum += $1} END {print sum}} numbers.list
```

Working with globbing

If you are converting invocations involving shell globbing, you should remember that Tcl does not handle globbing or expand things into multiple arguments by default. Instead you should write things like this:

```
exec ls -l {*}[glob *.tcl]
```

Working with user-supplied shell script fragments

One useful technique can be to expose to users of a script the ability to specify a fragment of shell script to execute that will have some data passed in on standard input that was produced by the Tcl program. This is a common technique for using the *lpr* program for printing. By far the simplest way of doing this is to pass the user's script to the user's shell for processing, as this

avoids a lot of complexity with parsing other languages.

```
set lprScript [get from user...]
set postscriptData [generate somehow...]

exec $env(SHELL) -c $lprScript << $postscriptData
```

Windows examples

Here are some examples of the use of the **exec** command on Windows. To start an instance of *notepad* editing a file without waiting for the user to finish editing the file:

```
exec notepad myfile.txt &
```

To print a text file using *notepad*:

```
exec notepad /p myfile.txt
```

Working with console programs

If a program calls other programs, such as is common with compilers, then you may need to resort to batch files to hide the console windows that sometimes pop up:

```
exec cmp.bat somefile.c -o somefile
```

With the file *cmp.bat* looking something like:

```
@gcc %*
```

or like another variant using single parameters:

```
@gcc %1 %2 %3 %4 %5 %6 %7 %8 %9
```

Working with command built-ins

Sometimes you need to be careful, as different programs may have the same name and be in the path. It can then happen that typing a command at the DOS prompt finds *a different program* than the same command run via **exec**. This is because of the (documented) differences in behaviour between **exec** and DOS batch files.

When in doubt, use the command auto_execok: it will return the complete path to the program as seen by the **exec** command. This applies especially when you want to run "internal" commands like *dir* from a Tcl script (if you just want to list filenames, use the glob command.) To do that, use this:

```
exec {*}[auto_execok dir] *.tcl
```

Working with native filenames

Many programs on Windows require filename arguments to be passed in with backslashes as pathname separators. This is done with the help of the file nativename command. For example, to make a directory (on NTFS) encrypted so that only the current user can access it requires use of the *CIPHER* command, like this:

```
set secureDir "~/Desktop/Secure Directory"
file mkdir $secureDir
exec CIPHER /e /s:[file nativename $secureDir]
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2006 Donal K. Fellows.

exit

Keywords: exit, abort, process

Name

exit - End the application

Synopsis

exit ?*returnCode*?

Description

Terminate the process, returning *returnCode* to the system as the exit status. If *returnCode* is not specified then it defaults to 0.

Example

Since non-zero exit codes are usually interpreted as error cases by the calling process, the **exit** command is an important part of signaling that something fatal has gone wrong. This code fragment is useful in scripts to act as a general problem trap:

```
proc main {} {
    # ... put the real main code in here ...
}

if {[catch {main} msg options]} {
    puts stderr "unexpected script error: $msg"
    if {[info exists env(DEBUG)]} {
        puts stderr "---- BEGIN TRACE ----"
        puts stderr [dict get $options -errorinfo]
        puts stderr "---- END TRACE ----"
    }

    # Reserve code 1 for "expected" error exits...
    exit 2
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

expr

Keywords: expr, arithmetic, boolean, compare, expression, fuzzy comparison

Name
expr - Evaluate an expression

Synopsis
expr *arg* ?*arg arg* ...?

Description
Concatenates *arg*s (adding separator spaces between them), evaluates the result as a Tcl expression, and returns the value. The operators permitted in Tcl expressions include a subset of the operators permitted in C expressions. For those operators common to both Tcl and C, Tcl applies the same meaning and precedence as the corresponding C operators. Expressions almost always yield numeric results (integer or floating-point values). For example, the expression

```
expr 8.2 + 6
```

evaluates to 14.2. Tcl expressions differ from C expressions in the way that operands are specified. Also, Tcl expressions support non-numeric operands and string comparisons, as well as some additional operators not found in C.

Operands
A Tcl expression consists of a combination of operands, operators, parentheses and commas. White space may be used between the operands and operators and parentheses (or commas); it is ignored by the expression's instructions. Where possible, operands are interpreted as integer values. Integer values may be specified in decimal (the normal case), in binary (if the first two characters of the operand are **0b**), in octal (if the first two characters of the operand are **0o**), or in hexadecimal (if the first two characters of the operand are **0x**). For compatibility with older Tcl releases, an octal integer value is also indicated simply when the first character of the operand is **0**, whether or not the second character is also **o**. If an operand does not have one of the integer formats given above, then it is treated as a floating-point number if that is possible. Floating-point numbers may be specified in any of several common formats making use of the decimal digits, the decimal point**.**, the characters **e** or **E** indicating scientific notation, and the sign characters **+** or **-**. For example, all of the following are valid floating-point numbers: 2.1, 3., 6e4, 7.91e+16. Also recognized as floating point values are the strings **Inf** and **NaN** making use of any case for each character. If no numeric interpretation is possible (note that all literal operands that are not numeric or boolean must be quoted with either braces or with double quotes), then an operand is left as a string (and only a limited set of operators may be applied to it).

Operands may be specified in any of the following ways:

1. As a numeric value, either integer or floating-point.
2. As a boolean value, using any form understood by string is **boolean**.
3. As a Tcl variable, using standard **$** notation. The variable's value will be used as the operand.
4. As a string enclosed in double-quotes. The expression parser will perform backslash, variable, and command substitutions on the information between the quotes, and use the resulting value as the operand
5. As a string enclosed in braces. The characters between the open brace and matching close brace will be used as the operand without any substitutions.

6. As a Tcl command enclosed in brackets. The command will be executed and its result will be used as the operand.
7. As a mathematical function whose arguments have any of the above forms for operands, such as **sin($x)**. See Math functions below for a discussion of how mathematical functions are handled.

Where the above substitutions occur (e.g. inside quoted strings), they are performed by the expression's instructions. However, the command parser may already have performed one round of substitution before the expression processor was called. As discussed below, it is usually best to enclose expressions in braces to prevent the command parser from performing substitutions on the contents.

For some examples of simple expressions, suppose the variable **a** has the value 3 and the variable **b** has the value 6. Then the command on the left side of each of the lines below will produce the value on the right side of the line:

```
expr {3.1 + $a} 6.1
expr {2 + "$a.$b"}  5.6
expr {4*[llength "6 2"]}    8
expr {{word one} < "word $a"}   0
```

Operators

The valid operators (most of which are also available as commands in the **tcl::mathop** namespace; see the mathop(n) manual page for details) are listed below, grouped in decreasing order of precedence:

- + ~!
Unary minus, unary plus, bit-wise NOT, logical NOT. None of these operators may be applied to string operands, and bit-wise NOT may be applied only to integers.

******
Exponentiation. Valid for any numeric operands. The maximum exponent value that Tcl can handle if the first number is an integer > 1 is 268435455.

*** / %**
Multiply, divide, remainder. None of these operators may be applied to string operands, and remainder may be applied only to integers. The remainder will always have the same sign as the divisor and an absolute value smaller than the absolute value of the divisor.

When applied to integers, the division and remainder operators can be considered to partition the number line into a sequence of equal-sized adjacent non-overlapping pieces where each piece is the size of the divisor; the division result identifies which piece the divisor lay within, and the remainder result identifies where within that piece the divisor lay. A consequence of this is that the result of "-57 **/** 10" is always -6, and the result of "-57 **%** 10" is always 3.

+ -
Add and subtract. Valid for any numeric operands.

<< >>
Left and right shift. Valid for integer operands only. A right shift always propagates the sign bit.

< > <= >=

Boolean less, greater, less than or equal, and greater than or equal. Each operator produces 1 if the condition is true, 0 otherwise. These operators may be applied to strings as well as numeric operands, in which case string comparison is used.

==!=
Boolean equal and not equal. Each operator produces a zero/one result. Valid for all operand types.

eq ne
Boolean string equal and string not equal. Each operator produces a zero/one result. The operand types are interpreted only as strings.

in ni
List containment and negated list containment. Each operator produces a zero/one result and treats its first argument as a string and its second argument as a Tcl list. The **in** operator indicates whether the first argument is a member of the second argument list; the **ni** operator inverts the sense of the result.

&
Bit-wise AND. Valid for integer operands only.

^
Bit-wise exclusive OR. Valid for integer operands only.

|
Bit-wise OR. Valid for integer operands only.

&&
Logical AND. Produces a 1 result if both operands are non-zero, 0 otherwise. Valid for boolean and numeric (integers or floating-point) operands only. This operator evaluates lazily; it only evaluates its second operand if it must in order to determine its result.

||
Logical OR. Produces a 0 result if both operands are zero, 1 otherwise. Valid for boolean and numeric (integers or floating-point) operands only. This operator evaluates lazily; it only evaluates its second operand if it must in order to determine its result.

x*? *y*: *z
If-then-else, as in C. If *x* evaluates to non-zero, then the result is the value of *y*. Otherwise the result is the value of *z*. The *x* operand must have a boolean or numeric value. This operator evaluates lazily; it evaluates only one of *y* or *z*.

See the C manual for more details on the results produced by each operator. The exponentiation operator promotes types like the multiply and divide operators, and produces a result that is the same as the output of the **pow** function (after any type conversions.) All of the binary operators but exponentiation group left-to-right within the same precedence level; exponentiation groups right-to-left. For example, the command

```
expr {4*2 < 7}
```

returns 0, while

```
expr {2**3**2}
```

returns 512.

The **&&**, **||**, and**?:** operators have "lazy evaluation", just as in C, which means that operands are

not evaluated if they are not needed to determine the outcome. For example, in the command

```
expr {$v?[a]:[b]}
```

only one of "**[a]**" or "**[b]**" will actually be evaluated, depending on the value of **$v**. Note, however, that this is only true if the entire expression is enclosed in braces; otherwise the Tcl parser will evaluate both "**[a]**" and "**[b]**" before invoking the **expr** command.

Math functions

When the expression parser encounters a mathematical function such as **sin($x)**, it replaces it with a call to an ordinary Tcl command in the **tcl::mathfunc** namespace. The processing of an expression such as:

```
expr {sin($x+$y)}
```

is the same in every way as the processing of:

```
expr {[tcl::mathfunc::sin [expr {$x+$y}]]}
```

which in turn is the same as the processing of:

```
tcl::mathfunc::sin [expr {$x+$y}]
```

The executor will search for **tcl::mathfunc::sin** using the usual rules for resolving functions in namespaces. Either ::tcl::mathfunc::sin or **[namespace current]::tcl::mathfunc::sin** will satisfy the request, and others may as well (depending on the current namespace path setting).

Some mathematical functions have several arguments, separated by commas like in C. Thus:

```
expr {hypot($x,$y)}
```

ends up as

```
tcl::mathfunc::hypot $x $y
```

See the mathfunc(n) manual page for the math functions that are available by default.

Types, overflow, and precision

All internal computations involving integers are done calling on the LibTomMath multiple precision integer library as required so that all integer calculations are performed exactly. Note that in Tcl releases prior to 8.5, integer calculations were performed with one of the C types *long int* or *Tcl_WideInt*, causing implicit range truncation in those calculations where values overflowed the range of those types. Any code that relied on these implicit truncations will need to explicitly add **int()** or **wide()** function calls to expressions at the points where such truncation is required to take place.

All internal computations involving floating-point are done with the C type *double*. When converting a string to floating-point, exponent overflow is detected and results in the *double* value of **Inf** or **-Inf** as appropriate. Floating-point overflow and underflow are detected to the degree supported by the hardware, which is generally pretty reliable.

Conversion among internal representations for integer, floating-point, and string operands is done automatically as needed. For arithmetic computations, integers are used until some floating-point number is introduced, after which floating-point is used. For example,

```
expr {5 / 4}
```

returns 1, while

```
expr {5 / 4.0}
expr {5 / ( [string length "abcd"] + 0.0 )}
```

both return 1.25. Floating-point values are always returned with a "**.**" or an "**e**" so that they will not look like integer values. For example,

```
expr {20.0/5.0}
```

returns **4.0**, not **4**.

String operations

String values may be used as operands of the comparison operators, although the expression evaluator tries to do comparisons as integer or floating-point when it can, i.e., when all arguments to the operator allow numeric interpretations, except in the case of the **eq** and **ne** operators. If one of the operands of a comparison is a string and the other has a numeric value, a canonical string representation of the numeric operand value is generated to compare with the string operand. Canonical string representation for integer values is a decimal string format. Canonical string representation for floating-point values is that produced by the **%g** format specifier of Tcl's format command. For example, the commands

```
expr {"0x03" > "2"}
expr {"0y" > "0x12"}
```

both return 1. The first comparison is done using integer comparison, and the second is done using string comparison. Because of Tcl's tendency to treat values as numbers whenever possible, it is not generally a good idea to use operators like **==** when you really want string comparison and the values of the operands could be arbitrary; it is better in these cases to use the **eq** or **ne** operators, or the string command instead.

Performance considerations

Enclose expressions in braces for the best speed and the smallest storage requirements. This allows the Tcl bytecode compiler to generate the best code.

As mentioned above, expressions are substituted twice: once by the Tcl parser and once by the **expr** command. For example, the commands

```
set a 3
set b {$a + 2}
expr $b*4
```

return 11, not a multiple of 4. This is because the Tcl parser will first substitute "**$a + 2**" for the variable **b**, then the **expr** command will evaluate the expression "**$a + 2*4**".

Most expressions do not require a second round of substitutions. Either they are enclosed in braces or, if not, their variable and command substitutions yield numbers or strings that do not themselves require substitutions. However, because a few unbraced expressions need two rounds of substitutions, the bytecode compiler must emit additional instructions to handle this situation. The most expensive code is required for unbraced expressions that contain command

substitutions. These expressions must be implemented by generating new code each time the expression is executed.

If it is necessary to include a non-constant expression string within the wider context of an otherwise-constant expression, the most efficient technique is to put the varying part inside a recursive **expr**, as this at least allows for the compilation of the outer part, though it does mean that the varying part must itself be evaluated as a separate expression. Thus, in this example the result is 20 and the outer expression benefits from fully cached bytecode compilation.

```
set a 3
set b {$a + 2}
expr {[expr $b] * 4}
```

When the expression is unbraced to allow the substitution of a function or operator, consider using the commands documented in the mathfunc(n) or mathop(n) manual pages directly instead.

Examples

Define a procedure that computes an "interesting" mathematical function:

```
proc tcl::mathfunc::calc {x y} {
    expr { ($x**2 - $y**2) / exp($x**2 + $y**2) }
}
```

Convert polar coordinates into cartesian coordinates:

```
# convert from ($radius,$angle)
set x [expr { $radius * cos($angle) }]
set y [expr { $radius * sin($angle) }]
```

Convert cartesian coordinates into polar coordinates:

```
# convert from ($x,$y)
set radius [expr { hypot($y, $x) }]
set angle  [expr { atan2($y, $x) }]
```

Print a message describing the relationship of two string values to each other:

```
puts "a and b are [expr {$a eq $b ? {equal} : {different}}]"
```

Set a variable to whether an environment variable is both defined at all and also set to a true boolean value:

```
set isTrue [expr {
    [info exists ::env(SOME_ENV_VAR)] &&
    [string is true -strict $::env(SOME_ENV_VAR)]
}]
```

Generate a random integer in the range 0..99 inclusive:

```
set randNum [expr { int(100 * rand()) }]
```

Copyright

```
Copyright © 1993 The Regents of the University of California.
Copyright © 1994-2000 Sun Microsystems Incorporated.
Copyright © 2005 Kevin B. Kenny <kennykb@acm.org>. All rights reserved.
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-2000 Sun Microsystems, Inc.
Copyright (c) 2005 Kevin B. Kenny kennykb@acm.org*. All rights reserved*

fblocked

Keywords: fblocked, blocking, nonblocking

Name

fblocked - Test whether the last input operation exhausted all available input

Synopsis

fblocked *channelId*

Description

The **fblocked** command returns 1 if the most recent input operation on *channelId* returned less information than requested because all available input was exhausted. For example, if gets is invoked when there are only three characters available for input and no end-of-line sequence, gets returns an empty string and a subsequent call to **fblocked** will return 1.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension.

Example

The **fblocked** command is particularly useful when writing network servers, as it allows you to write your code in a line-by-line style without preventing the servicing of other connections. This can be seen in this simple echo-service:

```
# This is called whenever a new client connects to the server
proc connect {chan host port} {
    set clientName [format <%s:%d> $host $port]
    puts "connection from $clientName"
    fconfigure $chan -blocking 0 -buffering line
    fileevent $chan readable [list echoLine $chan $clientName]
}

# This is called whenever either at least one byte of input
# data is available, or the channel was closed by the client.
proc echoLine {chan clientName} {
    gets $chan line
    if {[eof $chan]} {
        puts "finishing connection from $clientName"
        close $chan
    } elseif {![fblocked $chan]} {
```

```
        # Didn't block waiting for end-of-line
        puts "$clientName - $line"
        puts $chan $line
    }
}

# Create the server socket and enter the event-loop to wait
# for incoming connections...
socket -server connect 12345
vwait forever
```

Copyright (c) 1996 Sun Microsystems, Inc.

fconfigure

Keywords: fconfigure, blocking, buffering, carriage return, end of line, flushing, linemode, newline, nonblocking, platform, translation, encoding, filter, byte array, binary

Name

fconfigure - Set and get options on a channel

Synopsis

fconfigure *channelId*
fconfigure *channelId name*
fconfigure *channelId name value* ?*name value* ...?

Description

The **fconfigure** command sets and retrieves options for channels.

ChannelId identifies the channel for which to set or query an option and must refer to an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension.

If no *name* or *value* arguments are supplied, the command returns a list containing alternating option names and values for the channel. If *name* is supplied but no *value* then the command returns the current value of the given option. If one or more pairs of *name* and *value* are supplied, the command sets each of the named options to the corresponding *value*; in this case the return value is an empty string.

The options described below are supported for all channels. In addition, each channel type may add options that only it supports. See the manual entry for the command that creates each type of channels for the options that that specific type of channel supports. For example, see the manual entry for the socket command for additional options for sockets, and the open command for additional options for serial devices.

-blocking *boolean*

The **-blocking** option determines whether I/O operations on the channel can cause the process to block indefinitely. The value of the option must be a proper boolean value. Channels are normally in blocking mode; if a channel is placed into nonblocking mode it will affect the operation of the gets, read, puts, flush, and close commands by allowing them to

operate asynchronously; see the documentation for those commands for details. For nonblocking mode to work correctly, the application must be using the Tcl event loop (e.g. by calling Tcl_DoOneEvent or invoking the vwait command).

-buffering *newValue*

If *newValue* is **full** then the I/O system will buffer output until its internal buffer is full or until the flush command is invoked. If *newValue* is **line**, then the I/O system will automatically flush output for the channel whenever a newline character is output. If *newValue* is **none**, the I/O system will flush automatically after every output operation. The default is for **-buffering** to be set to **full** except for channels that connect to terminal-like devices; for these channels the initial setting is **line**. Additionally, **stdin** and **stdout** are initially set to **line**, and **stderr** is set to **none**.

-buffersize *newSize*

Newvalue must be an integer; its value is used to set the size of buffers, in bytes, subsequently allocated for this channel to store input or output. *Newvalue* must be between one and one million, allowing buffers of one to one million bytes in size.

-encoding *name*

This option is used to specify the encoding of the channel, so that the data can be converted to and from Unicode for use in Tcl. For instance, in order for Tcl to read characters from a Japanese file in **shiftjis** and properly process and display the contents, the encoding would be set to **shiftjis**. Thereafter, when reading from the channel, the bytes in the Japanese file would be converted to Unicode as they are read. Writing is also supported - as Tcl strings are written to the channel they will automatically be converted to the specified encoding on output.

If a file contains pure binary data (for instance, a JPEG image), the encoding for the channel should be configured to be binary. Tcl will then assign no interpretation to the data in the file and simply read or write raw bytes. The Tcl binary command can be used to manipulate this byte-oriented data. It is usually better to set the **-translation** option to binary when you want to transfer binary data, as this turns off the other automatic interpretations of the bytes in the stream as well.

The default encoding for newly opened channels is the same platform- and locale-dependent system encoding used for interfacing with the operating system, as returned by encoding system.

-eofchar *char*

see below ...

-eofchar {*inChar outChar*}

This option supports DOS file systems that use Control-z (\x1A) as an end of file marker. If *char* is not an empty string, then this character signals end-of-file when it is encountered during input. For output, the end-of-file character is output when the channel is closed. If *char* is the empty string, then there is no special end of file character marker. For read-write channels, a two-element list specifies the end of file marker for input and output, respectively. As a convenience, when setting the end-of-file character for a read-write channel you can specify a single value that will apply to both reading and writing. When querying the end-of-file character of a read-write channel, a two-element list will always be returned. The default value for **-eofchar** is the empty string in all cases except for files under Windows. In

that case the **-eofchar** is Control-z (\x1A) for reading and the empty string for writing. The acceptable range for **-eofchar** values is \x01 - \x7F; attempting to set **-eofchar** to a value outside of this range will generate an error.

-translation *mode*
see below ...

-translation {*inMode outMode*}
In Tcl scripts the end of a line is always represented using a single newline character (\n). However, in actual files and devices the end of a line may be represented differently on different platforms, or even for different devices on the same platform. For example, under UNIX newlines are used in files, whereas carriage-return-linefeed sequences are normally used in network connections. On input (i.e., with gets and read) the Tcl I/O system automatically translates the external end-of-line representation into newline characters. Upon output (i.e., with puts), the I/O system translates newlines to the external end-of-line representation. The default translation mode, **auto**, handles all the common cases automatically, but the **-translation** option provides explicit control over the end of line translations.

The value associated with **-translation** is a single item for read-only and write-only channels. The value is a two-element list for read-write channels; the read translation mode is the first element of the list, and the write translation mode is the second element. As a convenience, when setting the translation mode for a read-write channel you can specify a single value that will apply to both reading and writing. When querying the translation mode of a read-write channel, a two-element list will always be returned. The following values are currently supported:

auto
As the input translation mode, **auto** treats any of newline (**lf**), carriage return (**cr**), or carriage return followed by a newline (**crlf**) as the end of line representation. The end of line representation can even change from line-to-line, and all cases are translated to a newline. As the output translation mode, **auto** chooses a platform specific representation; for sockets on all platforms Tcl chooses **crlf**, for all Unix flavors, it chooses **lf**, and for the various flavors of Windows it chooses **crlf**. The default setting for **-translation** is **auto** for both input and output.

binary
No end-of-line translations are performed. This is nearly identical to **lf** mode, except that in addition binary mode also sets the end-of-file character to the empty string (which disables it) and sets the encoding to binary (which disables encoding filtering). See the description of **-eofchar** and **-encoding** for more information.

Internally, i.e. when it comes to the actual behaviour of the translator this value **is** identical to **lf** and is therefore reported as such when queried. Even if binary was used to set the translation.

cr
The end of a line in the underlying file or device is represented by a single carriage return character. As the input translation mode, **cr** mode converts carriage returns to newline characters. As the output translation mode, **cr** mode translates newline characters to carriage returns.

crlf
The end of a line in the underlying file or device is represented by a carriage return character followed by a linefeed character. As the input translation mode, **crlf** mode converts carriage-return-linefeed sequences to newline characters. As the output translation mode, **crlf** mode translates newline characters to carriage-return-linefeed sequences. This mode is typically used on Windows platforms and for network connections.

lf
The end of a line in the underlying file or device is represented by a single newline (linefeed) character. In this mode no translations occur during either input or output. This mode is typically used on UNIX platforms.

Standard channels

The Tcl standard channels (**stdin**, **stdout**, and **stderr**) can be configured through this command like every other channel opened by the Tcl library. Beyond the standard options described above they will also support any special option according to their current type. If, for example, a Tcl application is started by the **inet** super-server common on Unix system its Tcl standard channels will be sockets and thus support the socket options.

Examples

Instruct Tcl to always send output to **stdout** immediately, whether or not it is to a terminal:

```
fconfigure stdout -buffering none
```

Open a socket and read lines from it without ever blocking the processing of other events:

```
set s [socket some.where.com 12345]
fconfigure $s -blocking 0
fileevent $s readable "readMe $s"
proc readMe chan {
    if {[gets $chan line] < 0} {
        if {[eof $chan]} {
            close $chan
            return
        }
        # Could not read a complete line this time; Tcl's
        # internal buffering will hold the partial line for us
        # until some more data is available over the socket.
    } else {
        puts stdout $line
    }
}
```

Read a PPM-format image from a file:

```
# Open the file and put it into Unix ASCII mode
set f [open teapot.ppm]
fconfigure $f -encoding ascii -translation lf

```

```
# Get the header
if {[gets $f] ne "P6"} {
    error "not a raw-bits PPM"
}

# Read lines until we have got non-comment lines
# that supply us with three decimal values.
set words {}
while {[llength $words] < 3} {
    gets $f line
    if {[string match "#*" $line]} continue
    lappend words {*}[join [scan $line %d%d%d]]
}

# Those words supply the size of the image and its
# overall depth per channel. Assign to variables.
lassign $words xSize ySize depth

# Now switch to binary mode to pull in the data,
# one byte per channel (red,green,blue) per pixel.
fconfigure $f -translation binary
set numDataBytes [expr {3 * $xSize * $ySize}]
set data [read $f $numDataBytes]

close $f
```

Copyright (c) 1995-1996 Sun Microsystems, Inc.

fcopy

Keywords: fcopy, blocking, channel, end of line, end of file, nonblocking, read, translation

Name

fcopy - Copy data from one channel to another

Synopsis

fcopy *inchan outchan* ?*-size size*? ?*-command callback*?

Description

The **fcopy** command copies data from one I/O channel, *inchan* to another I/O channel, *outchan*. The **fcopy** command leverages the buffering in the Tcl I/O system to avoid extra copies and to avoid buffering too much data in main memory when copying large files to slow destinations like network sockets.

The **fcopy** command transfers data from *inchan* until end of file or *size* bytes or characters have been transferred; *size* is in bytes if the two channels are using the same encoding, and is in characters otherwise. If no **-size** argument is given, then the copy goes until end of file. All the data read from *inchan* is copied to *outchan*. Without the **-command** option, **fcopy** blocks until

the copy is complete and returns the number of bytes or characters (using the same rules as for the **-size** option) written to *outchan*.

The **-command** argument makes **fcopy** work in the background. In this case it returns immediately and the *callback* is invoked later when the copy completes. The *callback* is called with one or two additional arguments that indicates how many bytes were written to *outchan*. If an error occurred during the background copy, the second argument is the error string associated with the error. With a background copy, it is not necessary to put *inchan* or *outchan* into non-blocking mode; the **fcopy** command takes care of that automatically. However, it is necessary to enter the event loop by using the vwait command or by using Tk.

You are not allowed to do other input operations with *inchan*, or output operations with *outchan*, during a background **fcopy**. The converse is entirely legitimate, as exhibited by the bidirectional fcopy example below.

If either *inchan* or *outchan* get closed while the copy is in progress, the current copy is stopped and the command callback is *not* made. If *inchan* is closed, then all data already queued for *outchan* is written out.

Note that *inchan* can become readable during a background copy. You should turn off any fileevent handlers during a background copy so those handlers do not interfere with the copy. Any wrong-sided I/O attempted (by a fileevent handler or otherwise) will get a "channel busy" error.

Fcopy translates end-of-line sequences in *inchan* and *outchan* according to the **-translation** option for these channels. See the manual entry for fconfigure for details on the **-translation** option. The translations mean that the number of bytes read from *inchan* can be different than the number of bytes written to *outchan*. Only the number of bytes written to *outchan* is reported, either as the return value of a synchronous **fcopy** or as the argument to the callback for an asynchronous **fcopy**.

Fcopy obeys the encodings and character translations configured for the channels. This means that the incoming characters are converted internally first UTF-8 and then into the encoding of the channel **fcopy** writes to. See the manual entry for fconfigure for details on the **-encoding** and **-translation** options. No conversion is done if both channels are set to encoding "binary" and have matching translations. If only the output channel is set to encoding "binary" the system will write the internal UTF-8 representation of the incoming characters. If only the input channel is set to encoding "binary" the system will assume that the incoming bytes are valid UTF-8 characters and convert them according to the output encoding. The behaviour of the system for bytes which are not valid UTF-8 characters is undefined in this case.

Examples

The first example transfers the contents of one channel exactly to another. Note that when copying one file to another, it is better to use file copy which also copies file metadata (e.g. the file access permissions) where possible.

```
fconfigure $in -translation binary
fconfigure $out -translation binary
fcopy $in $out
```

This second example shows how the callback gets passed the number of bytes transferred. It also uses vwait to put the application into the event loop. Of course, this simplified example

could be done without the command callback.

```
proc Cleanup {in out bytes {error {}}} {
    global total
    set total $bytes
    close $in
    close $out
    if {[string length $error] != 0} {
        # error occurred during the copy
    }
}
set in [open $file1]
set out [socket $server $port]
fcopy $in $out -command [list Cleanup $in $out]
vwait total
```

The third example copies in chunks and tests for end of file in the command callback.

```
proc CopyMore {in out chunk bytes {error {}}} {
    global total done
    incr total $bytes
    if {([string length $error] != 0) || [eof $in]} {
        set done $total
        close $in
        close $out
    } else {
        fcopy $in $out -size $chunk \
                -command [list CopyMore $in $out $chunk]
    }
}
set in [open $file1]
set out [socket $server $port]
set chunk 1024
set total 0
fcopy $in $out -size $chunk \
        -command [list CopyMore $in $out $chunk]
vwait done
```

The fourth example starts an asynchronous, bidirectional fcopy between two sockets. Those could also be pipes from two [open "|hal 9000" r+] (though their conversation would remain secret to the script, since all four fileevent slots are busy).

```
set flows 2
proc Done {dir args} {
     global flows done
     puts "$dir is over."
     incr flows -1
     if {$flows<=0} {set done 1}
```

```
}
fcopy $sok1 $sok2 -command [list Done UP]
fcopy $sok2 $sok1 -command [list Done DOWN]
vwait done
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

file

Keywords: file, attributes, copy files, delete files, directory, move files, name, rename files, stat, user

Name

file - Manipulate file names and attributes

Synopsis

file *option name* ?*arg arg* ...?

Description

This command provides several operations on a file's name or attributes. *Name* is the name of a file; if it starts with a tilde, then tilde substitution is done before executing the command (see the manual entry for filename for details). *Option* indicates what to do with the file name. Any unique abbreviation for *option* is acceptable. The valid options are:

file atime *name* ?*time*?
: Returns a decimal string giving the time at which file *name* was last accessed. If *time* is specified, it is an access time to set for the file. The time is measured in the standard POSIX fashion as seconds from a fixed starting time (often January 1, 1970). If the file does not exist or its access time cannot be queried or set then an error is generated. On Windows, FAT file systems do not support access time.

file attributes *name*
: see below ...

file attributes *name* ?*option*?
: see below ...

file attributes *name* ?*option value option value...*?
: This subcommand returns or sets platform-specific values associated with a file. The first form returns a list of the platform-specific options and their values. The second form returns the value for the given option. The third form sets one or more of the values. The values are as follows:

  On Unix, **-group** gets or sets the group name for the file. A group id can be given to the command, but it returns a group name. **-owner** gets or sets the user name of the owner of the file. The command returns the owner name, but the numerical id can be passed when setting the owner. **-permissions** retrieves or sets a file's access permissions, using octal notation by default. This option also provides limited support for setting permissions using the symbolic notation accepted by the **chmod** command, following the form [**ugo**]?[[**+-=**][**rwxst**]**,**[...]]. Multiple permission specifications may be given, separated by commas. E.g.,

u+s,go-rw would set the setuid bit for a file's owner as well as remove read and write permission for the file's group and other users. An **ls**-style string of the form **rwxrwxrwx** is also accepted but must always be 9 characters long. E.g., **rwxr-xr-t** is equivalent to **01755**. On versions of Unix supporting file flags, **-readonly** returns the value of, or sets, or clears the readonly attribute of a file, i.e., the user immutable flag (**uchg**) to the **chflags** command.

On Windows, **-archive** gives the value or sets or clears the archive attribute of the file. **-hidden** gives the value or sets or clears the hidden attribute of the file. **-longname** will expand each path element to its long version. This attribute cannot be set. **-readonly** gives the value or sets or clears the readonly attribute of the file. **-shortname** gives a string where every path element is replaced with its short (8.3) version of the name if possible. For path elements that cannot be mapped to short names, the long name is retained. This attribute cannot be set. **-system** gives or sets or clears the value of the system attribute of the file.

On macOS and Darwin, **-creator** gives or sets the Finder creator type of the file. **-hidden** gives or sets or clears the hidden attribute of the file. **-readonly** gives or sets or clears the readonly attribute of the file. **-rsrclength** gives the length of the resource fork of the file, this attribute can only be set to the value 0, which results in the resource fork being stripped off the file.

file channels ?*pattern*?

If *pattern* is not specified, returns a list of names of all registered open channels in this interpreter. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for string match.

file copy ?-force? ?--? *source target*

see below ...

file copy ?-force? ?--? *source* ?*source* ...? *targetDir*

The first form makes a copy of the file or directory *source* under the pathname *target*. If *target* is an existing directory, then the second form is used. The second form makes a copy inside *targetDir* of each *source* file listed. If a directory is specified as a *source*, then the contents of the directory will be recursively copied into *targetDir*. Existing files will not be overwritten unless the **-force** option is specified (when Tcl will also attempt to adjust permissions on the destination file or directory if that is necessary to allow the copy to proceed). When copying within a single filesystem, *file copy* will copy soft links (i.e. the links themselves are copied, not the things they point to). Trying to overwrite a non-empty directory, overwrite a directory with a file, or overwrite a file with a directory will all result in errors even if **-force** was specified. Arguments are processed in the order specified, halting at the first error, if any. A **--** marks the end of switches; the argument following the **--** will be treated as a *source* even if it starts with a **-**.

file delete ?-force? ?--? ?*pathname* ... ?

Removes the file or directory specified by each *pathname* argument. Non-empty directories will be removed only if the **-force** option is specified. When operating on symbolic links, the links themselves will be deleted, not the objects they point to. Trying to delete a non-existent file is not considered an error. Trying to delete a read-only file will cause the file to be deleted, even if the **-force** flags is not specified. If the **-force** option is specified on a directory, Tcl will attempt both to change permissions and move the current directory "pwd" out of the given path if that is necessary to allow the deletion to proceed. Arguments are processed in the

order specified, halting at the first error, if any. A **--** marks the end of switches; the argument following the **--** will be treated as a *pathname* even if it starts with a **-**.

file dirname *name*

Returns a name comprised of all of the path components in *name* excluding the last element. If *name* is a relative file name and only contains one path element, then returns "**.**". If *name* refers to a root directory, then the root directory is returned. For example,

```
file dirname c:/
```

returns **c:/**.

Note that tilde substitution will only be performed if it is necessary to complete the command. For example,

```
file dirname ~/src/foo.c
```

returns **~/src**, whereas

```
file dirname ~
```

returns /**home** (or something similar).

file executable *name*

Returns **1** if file *name* is executable by the current user, **0** otherwise. On Windows, which does not have an executable attribute, the command treats all directories and any files with extensions **exe**, **com**, **cmd** or **bat** as executable.

file exists *name*

Returns **1** if file *name* exists and the current user has search privileges for the directories leading to it, **0** otherwise.

file extension *name*

Returns all of the characters in *name* after and including the last dot in the last element of *name*. If there is no dot in the last element of *name* then returns the empty string.

file isdirectory *name*

Returns **1** if file *name* is a directory, **0** otherwise.

file isfile *name*

Returns **1** if file *name* is a regular file, **0** otherwise.

file join *name* ?*name ...*?

Takes one or more file names and combines them, using the correct path separator for the current platform. If a particular *name* is relative, then it will be joined to the previous file name argument. Otherwise, any earlier arguments will be discarded, and joining will proceed from the current argument. For example,

```
file join a b /foo bar
```

returns /**foo**/**bar**.

Note that any of the names can contain separators, and that the result is always canonical for the current platform: / for Unix and Windows.

file link ?*-linktype*? *linkName* ?*target*?

If only one argument is given, that argument is assumed to be *linkName*, and this command returns the value of the link given by *linkName* (i.e. the name of the file it points to). If *linkName* is not a link or its value cannot be read (as, for example, seems to be the case with hard links, which look just like ordinary files), then an error is returned.

If 2 arguments are given, then these are assumed to be *linkName* and *target*. If *linkName* already exists, or if *target* does not exist, an error will be returned. Otherwise, Tcl creates a new link called *linkName* which points to the existing filesystem object at *target* (which is also the returned value), where the type of the link is platform-specific (on Unix a symbolic link will be the default). This is useful for the case where the user wishes to create a link in a cross-platform way, and does not care what type of link is created.

If the user wishes to make a link of a specific type only, (and signal an error if for some reason that is not possible), then the optional *-linktype* argument should be given. Accepted values for *-linktype* are "**-symbolic**" and "**-hard**".

On Unix, symbolic links can be made to relative paths, and those paths must be relative to the actual *linkName*'s location (not to the cwd), but on all other platforms where relative links are not supported, target paths will always be converted to absolute, normalized form before the link is created (and therefore relative paths are interpreted as relative to the cwd). Furthermore, "~user" paths are always expanded to absolute form. When creating links on filesystems that either do not support any links, or do not support the specific type requested, an error message will be returned. Most Unix platforms support both symbolic and hard links (the latter for files only). Windows supports symbolic directory links and hard file links on NTFS drives.

file lstat *name varName*

Same as **stat** option (see below) except uses the *lstat* kernel call instead of *stat*. This means that if *name* refers to a symbolic link the information returned in *varName* is for the link rather than the file it refers to. On systems that do not support symbolic links this option behaves exactly the same as the **stat** option.

file mkdir ?*dir* ...?

Creates each directory specified. For each pathname *dir* specified, this command will create all non-existing parent directories as well as *dir* itself. If an existing directory is specified, then no action is taken and no error is returned. Trying to overwrite an existing file with a directory will result in an error. Arguments are processed in the order specified, halting at the first error, if any.

file mtime *name* ?*time*?

Returns a decimal string giving the time at which file *name* was last modified. If *time* is specified, it is a modification time to set for the file (equivalent to Unix **touch**). The time is measured in the standard POSIX fashion as seconds from a fixed starting time (often January 1, 1970). If the file does not exist or its modified time cannot be queried or set then an error is generated.

file nativename *name*

Returns the platform-specific name of the file. This is useful if the filename is needed to pass to a platform-specific call, such as to a subprocess via exec under Windows (see Examples below).

file normalize *name*
Returns a unique normalized path representation for the file-system object (file, directory, link, etc), whose string value can be used as a unique identifier for it. A normalized path is an absolute path which has all "../" and "./" removed. Also it is one which is in the "standard" format for the native platform. On Unix, this means the segments leading up to the path must be free of symbolic links/aliases (but the very last path component may be a symbolic link), and on Windows it also means we want the long form with that form's case-dependence (which gives us a unique, case-dependent path). The one exception concerning the last link in the path is necessary, because Tcl or the user may wish to operate on the actual symbolic link itself (for example **file delete**, **file rename**, **file copy** are defined to operate on symbolic links, not on the things that they point to).

file owned *name*
Returns **1** if file *name* is owned by the current user, **0** otherwise.

file pathtype *name*
Returns one of **absolute**, **relative**, **volumerelative**. If *name* refers to a specific file on a specific volume, the path type will be **absolute**. If *name* refers to a file relative to the current working directory, then the path type will be **relative**. If *name* refers to a file relative to the current working directory on a specified volume, or to a specific file on the current working volume, then the path type is **volumerelative**.

file readable *name*
Returns **1** if file *name* is readable by the current user, **0** otherwise.

file readlink *name*
Returns the value of the symbolic link given by *name* (i.e. the name of the file it points to). If *name* is not a symbolic link or its value cannot be read, then an error is returned. On systems that do not support symbolic links this option is undefined.

file rename ?-force? ?--? *source target*
see below ...

file rename ?-force? ?--? *source* ?*source ...*? *targetDir*
The first form takes the file or directory specified by pathname *source* and renames it to *target*, moving the file if the pathname *target* specifies a name in a different directory. If *target* is an existing directory, then the second form is used. The second form moves each *source* file or directory into the directory *targetDir*. Existing files will not be overwritten unless the **-force** option is specified. When operating inside a single filesystem, Tcl will rename symbolic links rather than the things that they point to. Trying to overwrite a non-empty directory, overwrite a directory with a file, or a file with a directory will all result in errors. Arguments are processed in the order specified, halting at the first error, if any. A **--** marks the end of switches; the argument following the **--** will be treated as a *source* even if it starts with a **-**.

file rootname *name*
Returns all of the characters in *name* up to but not including the last "." character in the last component of name. If the last component of *name* does not contain a dot, then returns *name* .

file separator ?*name*?
If no argument is given, returns the character which is used to separate path segments for native files on this platform. If a path is given, the filesystem responsible for that path is

asked to return its separator character. If no file system accepts *name*, an error is generated.

file size *name*

Returns a decimal string giving the size of file *name* in bytes. If the file does not exist or its size cannot be queried then an error is generated.

file split *name*

Returns a list whose elements are the path components in *name*. The first element of the list will have the same path type as *name*. All other elements will be relative. Path separators will be discarded unless they are needed to ensure that an element is unambiguously relative. For example, under Unix

```
file split /foo/~bar/baz
```

returns "**/ foo ./~bar baz**" to ensure that later commands that use the third component do not attempt to perform tilde substitution.

file stat *name varName*

Invokes the **stat** kernel call on *name*, and uses the variable given by *varName* to hold information returned from the kernel call. *VarName* is treated as an array variable, and the following elements of that variable are set: **atime**, **ctime**, **dev**, **gid**, **ino**, **mode**, **mtime**, **nlink**, **size**, **type**, **uid**. Each element except **type** is a decimal string with the value of the corresponding field from the **stat** return structure; see the manual entry for **stat** for details on the meanings of the values. The **type** element gives the type of the file in the same form returned by the command **file type**. This command returns an empty string.

file system *name*

Returns a list of one or two elements, the first of which is the name of the filesystem to use for the file, and the second, if given, an arbitrary string representing the filesystem-specific nature or type of the location within that filesystem. If a filesystem only supports one type of file, the second element may not be supplied. For example the native files have a first element "native", and a second element which when given is a platform-specific type name for the file's system (e.g. "NTFS", "FAT", on Windows). A generic virtual file system might return the list "vfs ftp" to represent a file on a remote ftp site mounted as a virtual filesystem through an extension called "vfs". If the file does not belong to any filesystem, an error is generated.

file tail *name*

Returns all of the characters in the last filesystem component of *name*. Any trailing directory separator in *name* is ignored. If *name* contains no separators then returns *name*. So, **file tail a/b**, **file tail a/b/** and **file tail b** all return **b**.

file tempfile ?*nameVar*? ?*template*?

Creates a temporary file and returns a read-write channel opened on that file. If the nameVar is given, it specifies a variable that the name of the temporary file will be written into; if absent, Tcl will attempt to arrange for the temporary file to be deleted once it is no longer required. If the template is present, it specifies parts of the template of the filename to use when creating it (such as the directory, base-name or extension) though some platforms may ignore some or all of these parts and use a built-in default instead.

Note that temporary files are *only* ever created on the native filesystem. As such, they can be relied upon to be used with operating-system native APIs and external programs that require

a filename.

file type *name*
Returns a string giving the type of file *name*, which will be one of **file**, **directory**, **characterSpecial**, **blockSpecial**, **fifo**, **link**, or **socket**.

file volumes
Returns the absolute paths to the volumes mounted on the system, as a proper Tcl list. Without any virtual filesystems mounted as root volumes, on UNIX, the command will always return "/", since all filesystems are locally mounted. On Windows, it will return a list of the available local drives (e.g. "a:/ c:/"). If any virtual filesystem has mounted additional volumes, they will be in the returned list.

file writable *name*
Returns **1** if file *name* is writable by the current user, **0** otherwise.

Portability issues

Unix
These commands always operate using the real user and group identifiers, not the effective ones.

Windows
The **file owned** subcommand uses the user identifier (SID) of the process token, not the thread token which may be impersonating some other user.

Examples

This procedure shows how to search for C files in a given directory that have a correspondingly-named object file in the current directory:

```
proc findMatchingCFiles {dir} {
    set files {}
    switch $::tcl_platform(platform) {
        windows {
            set ext .obj
        }
        unix {
           set ext .o
        }
    }
    foreach file [glob -nocomplain -directory $dir *.c] {
        set objectFile [file tail [file rootname $file]]$ext
        if {[file exists $objectFile]} {
            lappend files $file
        }
    }
    return $files
}
```

Rename a file and leave a symbolic link pointing from the old location to the new place:

```
set oldName foobar.txt
set newName foo/bar.txt
# Make sure that where we're going to move to exists...
if {![file isdirectory [file dirname $newName]]} {
    file mkdir [file dirname $newName]
}
file rename $oldName $newName
file link -symbolic $oldName $newName
```

On Windows, a file can be "started" easily enough (equivalent to double-clicking on it in the Explorer interface) but the name passed to the operating system must be in native format:

```
exec {*}[auto_execok start] {} [file nativename ~/example.txt]
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

fileevent

Keywords: fileevent, asynchronous I/O, blocking, channel, event handler, nonblocking, readable, script, writable.

Name

fileevent - Execute a script when a channel becomes readable or writable

Synopsis

fileevent *channelId* **readable** ?*script*?
fileevent *channelId* **writable** ?*script*?

Description

This command is used to create *file event handlers*. A file event handler is a binding between a channel and a script, such that the script is evaluated whenever the channel becomes readable or writable. File event handlers are most commonly used to allow data to be received from another process on an event-driven basis, so that the receiver can continue to interact with the user while waiting for the data to arrive. If an application invokes gets or read on a blocking channel when there is no input data available, the process will block; until the input data arrives, it will not be able to service other events, so it will appear to the user to "freeze up". With **fileevent**, the process can tell when data is present and only invoke gets or read when they will not block.

The *channelId* argument to **fileevent** refers to an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension.

If the *script* argument is specified, then **fileevent** creates a new event handler: *script* will be evaluated whenever the channel becomes readable or writable (depending on the second argument to **fileevent**). In this case **fileevent** returns an empty string. The **readable** and **writable** event handlers for a file are independent, and may be created and deleted separately. However, there may be at most one **readable** and one **writable** handler for a file at a given time in a given interpreter. If **fileevent** is called when the specified handler already exists in the

invoking interpreter, the new script replaces the old one.

If the *script* argument is not specified, **fileevent** returns the current script for *channelId*, or an empty string if there is none. If the *script* argument is specified as an empty string then the event handler is deleted, so that no script will be invoked. A file event handler is also deleted automatically whenever its channel is closed or its interpreter is deleted.

A channel is considered to be readable if there is unread data available on the underlying device. A channel is also considered to be readable if there is unread data in an input buffer, except in the special case where the most recent attempt to read from the channel was a gets call that could not find a complete line in the input buffer. This feature allows a file to be read a line at a time in nonblocking mode using events. A channel is also considered to be readable if an end of file or error condition is present on the underlying file or device. It is important for *script* to check for these conditions and handle them appropriately; for example, if there is no special check for end of file, an infinite loop may occur where *script* reads no data, returns, and is immediately invoked again.

A channel is considered to be writable if at least one byte of data can be written to the underlying file or device without blocking, or if an error condition is present on the underlying file or device.

Event-driven I/O works best for channels that have been placed into nonblocking mode with the fconfigure command. In blocking mode, a puts command may block if you give it more data than the underlying file or device can accept, and a gets or read command will block if you attempt to read more data than is ready; a readable underlying file or device may not even guarantee that a blocking [read 1] will succeed (counter-examples being multi-byte encodings, compression or encryption transforms). In all such cases, no events will be processed while the commands block.

In nonblocking mode puts, read, and gets never block. See the documentation for the individual commands for information on how they handle blocking and nonblocking channels.

Testing for the end of file condition should be done after any attempts read the channel data. The eof flag is set once an attempt to read the end of data has occurred and testing before this read will require an additional event to be fired.

The script for a file event is executed at global level (outside the context of any Tcl procedure) in the interpreter in which the **fileevent** command was invoked. If an error occurs while executing the script then the command registered with interp bgerror is used to report the error. In addition, the file event handler is deleted if it ever returns an error; this is done in order to prevent infinite loops due to buggy handlers.

Example

In this setup **GetData** will be called with the channel as an argument whenever $chan becomes readable. The read call will read whatever binary data is currently available without blocking. Here the channel has the fileevent removed when an end of file occurs to avoid being continually called (see above). Alternatively the channel may be closed on this condition.

```
proc GetData {chan} {
    set data [read $chan]
    puts "[string length $data] $data"
    if {[eof $chan]} {
```

```
        fileevent $chan readable {}
    }
}

fconfigure $chan -blocking 0 -translation binary
fileevent $chan readable [list GetData $chan]
```

The next example demonstrates use of gets to read line-oriented data.

```
proc GetData {chan} {
    if {[gets $chan line] >= 0} {
        puts $line
    }
    if {[eof $chan]} {
        close $chan
    }
}

fconfigure $chan -blocking 0 -buffering line -translation crlf
fileevent $chan readable [list GetData $chan]
```

Credits

fileevent is based on the **addinput** command created by Mark Diekhans.

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2008 Pat Thoyts

filename

Keywords: filename, current directory, absolute file name, relative file name, volume-relative file name, portability

Name

filename - File name conventions supported by Tcl commands

Introduction

All Tcl commands and C procedures that take file names as arguments expect the file names to be in one of three forms, depending on the current platform. On each platform, Tcl supports file names in the standard forms(s) for that platform. In addition, on all platforms, Tcl supports a Unix-like syntax intended to provide a convenient way of constructing simple file names. However, scripts that are intended to be portable should not assume a particular form for file names. Instead, portable scripts must use the file split and file join commands to manipulate file names (see the file manual entry for more details).

Path types

File names are grouped into three general types based on the starting point for the path used to specify the file: absolute, relative, and volume-relative. Absolute names are completely qualified, giving a path to the file relative to a particular volume and the root directory on that volume.

Relative names are unqualified, giving a path to the file relative to the current working directory. Volume-relative names are partially qualified, either giving the path relative to the root directory on the current volume, or relative to the current directory of the specified volume. The file pathtype command can be used to determine the type of a given path.

Path syntax

The rules for native names depend on the value reported in the Tcl **platform** element of the tcl_platform array:

Unix

On Unix and Apple macOS platforms, Tcl uses path names where the components are separated by slashes. Path names may be relative or absolute, and file names may contain any character other than slash. The file names**.** and**..** are special and refer to the current directory and the parent of the current directory respectively. Multiple adjacent slash characters are interpreted as a single separator, except for the first double slash **//** in absolute paths. Any number of trailing slash characters at the end of a path are simply ignored, so the paths **foo**, **foo/** and **foo//** are all identical, and in particular **foo/** does not necessarily mean a directory is being referred.

The following examples illustrate various forms of path names:

/
: Absolute path to the root directory.

/etc/passwd
: Absolute path to the file named **passwd** in the directory **etc** in the root directory.

.
: Relative path to the current directory.

foo
: Relative path to the file **foo** in the current directory.

foo/bar
: Relative path to the file **bar** in the directory **foo** in the current directory.

../foo
: Relative path to the file **foo** in the directory above the current directory.

Windows

On Microsoft Windows platforms, Tcl supports both drive-relative and UNC style names. Both **/** and **** may be used as directory separators in either type of name. Drive-relative names consist of an optional drive specifier followed by an absolute or relative path. UNC paths follow the general form **\\servername\sharename\path\file**, but must at the very least contain the server and share components, i.e. **\\servername\sharename**. In both forms, the file names**.** and**..** are special and refer to the current directory and the parent of the current directory respectively. The following examples illustrate various forms of path names:

\\Host\share/file
: Absolute UNC path to a file called file in the root directory of the export point **share** on the host **Host**. Note that repeated use of file dirname on this path will give **//Host/share**, and will never give just **//Host**.

c:foo
: Volume-relative path to a file **foo** in the current directory on drive **c**.

c:/foo
Absolute path to a file **foo** in the root directory of drive **c**.

foo\bar
Relative path to a file **bar** in the **foo** directory in the current directory on the current volume.

\foo
Volume-relative path to a file **foo** in the root directory of the current volume.

\\foo
Volume-relative path to a file **foo** in the root directory of the current volume. This is not a valid UNC path, so the assumption is that the extra backslashes are superfluous.

Tilde substitution

In addition to the file name rules described above, Tcl also supports *csh*-style tilde substitution. If a file name starts with a tilde, then the file name will be interpreted as if the first element is replaced with the location of the home directory for the given user. If the tilde is followed immediately by a separator, then the **$HOME** environment variable is substituted. Otherwise the characters between the tilde and the next separator are taken as a user name, which is used to retrieve the user's home directory for substitution. This works on Unix, macOS and Windows (except very old releases).

Old Windows platforms do not support tilde substitution when a user name follows the tilde. On these platforms, attempts to use a tilde followed by a user name will generate an error that the user does not exist when Tcl attempts to interpret that part of the path or otherwise access the file. The behaviour of these paths when not trying to interpret them is the same as on Unix. File names that have a tilde without a user name will be correctly substituted using the **$HOME** environment variable, just like for Unix.

Portability issues

Not all file systems are case sensitive, so scripts should avoid code that depends on the case of characters in a file name. In addition, the character sets allowed on different devices may differ, so scripts should choose file names that do not contain special characters like: **<>:?"/\|**. The safest approach is to use names consisting of alphanumeric characters only. Care should be taken with filenames which contain spaces (common on Windows systems) and filenames where the backslash is the directory separator (Windows native path names).

On Windows platforms there are file and path length restrictions. Complete paths or filenames longer than about 260 characters will lead to errors in most file operations.

Another Windows peculiarity is that any number of trailing dots "." in filenames are totally ignored, so, for example, attempts to create a file or directory with a name "foo." will result in the creation of a file/directory with name "foo". This fact is reflected in the results of file normalize. Furthermore, a file name consisting only of dots "....." or dots with trailing characters ".....abc" is illegal.

Copyright (c) 1995-1996 Sun Microsystems, Inc.

flush

Keywords: flush, blocking, buffer, channel, nonblocking, output

Name
flush - Flush buffered output for a channel

Synopsis
flush *channelId*

Description
Flushes any output that has been buffered for *channelId*.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdout** or **stderr**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension. The channel must have been opened for writing.

If the channel is in blocking mode the command does not return until all the buffered output has been flushed to the channel. If the channel is in nonblocking mode, the command may return before all buffered output has been flushed; the remainder will be flushed in the background as fast as the underlying file or device is able to absorb it.

Example
Prompt for the user to type some information in on the console:

```
puts -nonewline "Please type your name: "
flush stdout
gets stdin name
puts "Hello there, $name!"
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

for
Keywords: for, boolean, iteration, loop

Name
for - 'For' loop

Synopsis
for *start test next body*

Description
For is a looping command, similar in structure to the C **for** statement. The *start*, *next*, and *body* arguments must be Tcl command strings, and *test* is an expression string. The **for** command first invokes the Tcl interpreter to execute *start*. Then it repeatedly evaluates *test* as an expression; if the result is non-zero it invokes the Tcl interpreter on *body*, then invokes the Tcl interpreter on *next*, then repeats the loop. The command terminates when *test* evaluates to 0. If a continue command is invoked within *body* then any remaining commands in the current execution of *body* are skipped; processing continues by invoking the Tcl interpreter on *next*, then evaluating *test*, and so on. If a break command is invoked within *body* or *next*, then the **for** command will return immediately. The operation of break and continue are similar to the corresponding statements in C. **For** returns an empty string.

Note: *test* should almost always be enclosed in braces. If not, variable substitutions will be made before the **for** command starts executing, which means that variable changes made by the loop body will not be considered in the expression. This is likely to result in an infinite loop. If *test* is enclosed in braces, variable substitutions are delayed until the expression is evaluated (before each loop iteration), so changes in the variables will be visible. See below for an example:

Examples

Print a line for each of the integers from 0 to 9:

```
for {set x 0} {$x<10} {incr x} {
    puts "x is $x"
}
```

Either loop infinitely or not at all because the expression being evaluated is actually the constant, or even generate an error! The actual behaviour will depend on whether the variable *x* exists before the **for** command is run and whether its value is a value that is less than or greater than/equal to ten, and this is because the expression will be substituted before the **for** command is executed.

```
for {set x 0} $x<10 {incr x} {
    puts "x is $x"
}
```

Print out the powers of two from 1 to 1024:

```
for {set x 1} {$x<=1024} {set x [expr {$x * 2}]} {
    puts "x is $x"
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

foreach

Keywords: foreach, iteration, list, loop

Name

foreach - Iterate over all elements in one or more lists

Synopsis

foreach *varname list body*
foreach *varlist1 list1 ?varlist2 list2 ...? body*

Description

The **foreach** command implements a loop where the loop variable(s) take on values from one or more lists. In the simplest case there is one loop variable, *varname*, and one list, *list*, that is a list of values to assign to *varname*. The *body* argument is a Tcl script. For each element of *list* (in order from first to last), **foreach** assigns the contents of the element to *varname* as if the lindex command had been used to extract the element, then calls the Tcl interpreter to execute *body*.

In the general case there can be more than one value list (e.g., *list1* and *list2*), and each value

list can be associated with a list of loop variables (e.g., *varlist1* and *varlist2*). During each iteration of the loop the variables of each *varlist* are assigned consecutive values from the corresponding *list*. Values in each *list* are used in order from first to last, and each value is used exactly once. The total number of loop iterations is large enough to use up all the values from all the value lists. If a value list does not contain enough elements for each of its loop variables in each iteration, empty values are used for the missing elements.

The break and continue statements may be invoked inside *body*, with the same effect as in the for command. **Foreach** returns an empty string.

Examples

This loop prints every value in a list together with the square and cube of the value:

```
set values {1 3 5 7 2 4 6 8}    ;# Odd numbers first, for fun!
puts "Value\tSquare\tCube"  ;# Neat-looking header
foreach x $values { ;# Now loop and print...
    puts " $x\t [expr {$x**2}]\t [expr {$x**3}]"
}
```

The following loop uses i and j as loop variables to iterate over pairs of elements of a single list.

```
set x {}
foreach {i j} {a b c d e f} {
    lappend x $j $i
}
# The value of x is "b a d c f e"
# There are 3 iterations of the loop.
```

The next loop uses i and j to iterate over two lists in parallel.

```
set x {}
foreach i {a b c} j {d e f g} {
    lappend x $i $j
}
# The value of x is "a d b e c f {} g"
# There are 4 iterations of the loop.
```

The two forms are combined in the following example.

```
set x {}
foreach i {a b c} {j k} {d e f g} {
    lappend x $i $j $k
}
# The value of x is "a d e b f g c {} {}"
# There are 3 iterations of the loop.
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

format

Keywords: format, conversion specifier, sprintf, string, substitution

Name
format - Format a string in the style of sprintf

Synopsis
format *formatString* ?*arg arg* ...?

Introduction
This command generates a formatted string in a fashion similar to the ANSI C **sprintf** procedure. *FormatString* indicates how to format the result, using **%** conversion specifiers as in **sprintf**, and the additional arguments, if any, provide values to be substituted into the result. The return value from **format** is the formatted string.

Details on formatting
The command operates by scanning *formatString* from left to right. Each character from the format string is appended to the result string unless it is a percent sign. If the character is a **%** then it is not copied to the result string. Instead, the characters following the **%** character are treated as a conversion specifier. The conversion specifier controls the conversion of the next successive *arg* to a particular format and the result is appended to the result string in place of the conversion specifier. If there are multiple conversion specifiers in the format string, then each one controls the conversion of one additional *arg*. The **format** command must be given enough *arg*s to meet the needs of all of the conversion specifiers in *formatString*.

Each conversion specifier may contain up to six different parts: an XPG3 position specifier, a set of flags, a minimum field width, a precision, a size modifier, and a conversion character. Any of these fields may be omitted except for the conversion character. The fields that are present must appear in the order given above. The paragraphs below discuss each of these fields in turn.

Optional positional specifier
If the **%** is followed by a decimal number and a **$**, as in "**%2$d**", then the value to convert is not taken from the next sequential argument. Instead, it is taken from the argument indicated by the number, where 1 corresponds to the first *arg*. If the conversion specifier requires multiple arguments because of * characters in the specifier then successive arguments are used, starting with the argument given by the number. This follows the XPG3 conventions for positional specifiers. If there are any positional specifiers in *formatString* then all of the specifiers must be positional.

Optional flags
The second portion of a conversion specifier may contain any of the following flag characters, in any order:

-
: Specifies that the converted argument should be left-justified in its field (numbers are normally right-justified with leading spaces if needed).

+
: Specifies that a number should always be printed with a sign, even if positive.

space
: Specifies that a space should be added to the beginning of the number if the first character is not a sign.

0
Specifies that the number should be padded on the left with zeroes instead of spaces.

#
Requests an alternate output form. For **o** conversions it guarantees that the first digit is always **0**. For **x** or **X** conversions, **0x** or **0X** (respectively) will be added to the beginning of the result unless it is zero. For **b** conversions, **0b** will be added to the beginning of the result unless it is zero. For all floating-point conversions (**e**, **E**, **f**, **g**, and **G**) it guarantees that the result always has a decimal point. For **g** and **G** conversions it specifies that trailing zeroes should not be removed.

Optional field width

The third portion of a conversion specifier is a decimal number giving a minimum field width for this conversion. It is typically used to make columns line up in tabular printouts. If the converted argument contains fewer characters than the minimum field width then it will be padded so that it is as wide as the minimum field width. Padding normally occurs by adding extra spaces on the left of the converted argument, but the **0** and **-** flags may be used to specify padding with zeroes on the left or with spaces on the right, respectively. If the minimum field width is specified as * rather than a number, then the next argument to the **format** command determines the minimum field width; it must be an integer value.

Optional precision/bound

The fourth portion of a conversion specifier is a precision, which consists of a period followed by a number. The number is used in different ways for different conversions. For **e**, **E**, and **f** conversions it specifies the number of digits to appear to the right of the decimal point. For **g** and **G** conversions it specifies the total number of digits to appear, including those on both sides of the decimal point (however, trailing zeroes after the decimal point will still be omitted unless the **#** flag has been specified). For integer conversions, it specifies a minimum number of digits to print (leading zeroes will be added if necessary). For **s** conversions it specifies the maximum number of characters to be printed; if the string is longer than this then the trailing characters will be dropped. If the precision is specified with * rather than a number then the next argument to the **format** command determines the precision; it must be a numeric string.

Optional size modifier

The fifth part of a conversion specifier is a size modifier, which must be **ll**, **h**, or **l**. If it is **ll** it specifies that an integer value is taken without truncation for conversion to a formatted substring. If it is **h** it specifies that an integer value is truncated to a 16-bit range before converting. This option is rarely useful. If it is **l** it specifies that the integer value is truncated to the same range as that produced by the **wide()** function of the expr command (at least a 64-bit range). If neither **h** nor **l** are present, the integer value is truncated to the same range as that produced by the **int()** function of the expr command (at least a 32-bit range, but determined by the value of the **wordSize** element of the tcl_platform array).

Mandatory conversion type

The last thing in a conversion specifier is an alphabetic character that determines what kind of conversion to perform. The following conversion characters are currently supported:

d
Convert integer to signed decimal string.

u
Convert integer to unsigned decimal string. The conversion makes no sense without reference to a truncation range, so the size modifier **ll** is not permitted in combination with conversion character **u**.

i
Convert integer to signed decimal string (equivalent to **d**).

o
Convert integer to unsigned octal string.

x or X
Convert integer to unsigned hexadecimal string, using digits "0123456789abcdef" for **x** and "0123456789ABCDEF" for **X**).

b
Convert integer to unsigned binary string, using digits 0 and 1.

c
Convert integer to the Unicode character it represents.

s
No conversion; just insert string.

f
Convert number to signed decimal string of the form *xx.yyy*, where the number of *y*'s is determined by the precision (default: 6). If the precision is 0 then no decimal point is output.

e or E
Convert number to scientific notation in the form *x.yyy***e±***zz*, where the number of *y*'s is determined by the precision (default: 6). If the precision is 0 then no decimal point is output. If the **E** form is used then **E** is printed instead of **e**.

g or G
If the exponent is less than -4 or greater than or equal to the precision, then convert number as for **%e** or **%E**. Otherwise convert as for **%f**. Trailing zeroes and a trailing decimal point are omitted.

%
No conversion: just insert **%**.

Differences from ANSI sprintf

The behavior of the format command is the same as the ANSI C **sprintf** procedure except for the following differences:

1. Tcl guarantees that it will be working with UNICODE characters.
2. **%p** and **%n** specifiers are not supported.
3. For **%c** conversions the argument must be an integer value, which will then be converted to the corresponding character value.
4. The size modifiers are ignored when formatting floating-point values. The **ll** modifier has no **sprintf** counterpart. The **b** specifier has no **sprintf** counterpart.

Examples

Convert the numeric value of a UNICODE character to the character itself:

```
set value 120
set char [format %c $value]
```

Convert the output of time into seconds to an accuracy of hundredths of a second:

```
set us [lindex [time $someTclCode] 0]
puts [format "%.2f seconds to execute" [expr {$us / 1e6}]]
```

Create a packed X11 literal color specification:

```
# Each color-component should be in range (0..255)
set color [format "#%02x%02x%02x" $r $g $b]
```

Use XPG3 format codes to allow reordering of fields (a technique that is often used in localized message catalogs; see msgcat) without reordering the data values passed to **format**:

```
set fmt1 "Today, %d shares in %s were bought at $%.2f each"
puts [format $fmt1 123 "Global BigCorp" 19.37]

set fmt2 "Bought %2\$s equity ($%3$.2f x %1\$d) today"
puts [format $fmt2 123 "Global BigCorp" 19.37]
```

Print a small table of powers of three:

```
# Set up the column widths
set w1 5
set w2 10

# Make a nice header (with separator) for the table first
set sep +-[string repeat - $w1]-+-[string repeat - $w2]-+
puts $sep
puts [format "| %-*s | %-*s |" $w1 "Index" $w2 "Power"]
puts $sep

# Print the contents of the table
set p 1
for {set i 0} {$i<=20} {incr i} {
    puts [format "| %*d | %*ld |" $w1 $i $w2 $p]
    set p [expr {wide($p) * 3}]
}

# Finish off by printing the separator again
puts $sep
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

gets

Keywords: gets, blocking, channel, end of file, end of line, line, non-blocking, read

Name
gets - Read a line from a channel

Synopsis
gets *channelId* ?*varName*?

Description
This command reads the next line from *channelId*, returns everything in the line up to (but not including) the end-of-line character(s), and discards the end-of-line character(s).

ChannelId must be an identifier for an open channel such as the Tcl standard input channel (**stdin**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension. The channel must have been opened for input.

If *varName* is omitted the line is returned as the result of the command. If *varName* is specified then the line is placed in the variable by that name and the return value is a count of the number of characters returned.

If end of file occurs while scanning for an end of line, the command returns whatever input is available up to the end of file. If *channelId* is in non-blocking mode and there is not a full line of input available, the command returns an empty string and does not consume any input. If *varName* is specified and an empty string is returned in *varName* because of end-of-file or because of insufficient data in non-blocking mode, then the return count is -1. Note that if *varName* is not specified then the end-of-file and no-full-line-available cases can produce the same results as if there were an input line consisting only of the end-of-line character(s). The eof and fblocked commands can be used to distinguish these three cases.

Example
This example reads a file one line at a time and prints it out with the current line number attached to the start of each line.

```
set chan [open "some.file.txt"]
set lineNumber 0
while {[gets $chan line] >= 0} {
    puts "[incr lineNumber]: $line"
}
close $chan
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

glob
Keywords: glob, exist, file, pattern

Name
glob - Return names of files that match patterns

Synopsis
glob ?*switches*? ?*pattern* ...?

Description

This command performs file name "globbing" in a fashion similar to the csh shell or bash shell. It returns a list of the files whose names match any of the *pattern* arguments. No particular order is guaranteed in the list, so if a sorted list is required the caller should use lsort.

Options

If the initial arguments to **glob** start with **-** then they are treated as switches. The following switches are currently supported:

-directory *directory*
: Search for files which match the given patterns starting in the given *directory*. This allows searching of directories whose name contains glob-sensitive characters without the need to quote such characters explicitly. This option may not be used in conjunction with **-path**, which is used to allow searching for complete file paths whose names may contain glob-sensitive characters.

-join
: The remaining pattern arguments, after option processing, are treated as a single pattern obtained by joining the arguments with directory separators.

-nocomplain
: Allows an empty list to be returned without error; without this switch an error is returned if the result list would be empty.

-path *pathPrefix*
: Search for files with the given *pathPrefix* where the rest of the name matches the given patterns. This allows searching for files with names similar to a given file (as opposed to a directory) even when the names contain glob-sensitive characters. This option may not be used in conjunction with **-directory**. For example, to find all files with the same root name as $path, but differing extensions, you should use "**glob -path [file rootname $path].***" which will work even if **$path** contains numerous glob-sensitive characters.

-tails
: Only return the part of each file found which follows the last directory named in any **-directory** or **-path** path specification. Thus "**glob -tails -directory $dir ***" is equivalent to "**set pwd [pwd]; cd $dir; glob *; cd $pwd**". For **-path** specifications, the returned names will include the last path segment, so "**glob -tails -path [file rootname ~/foo.tex].***" will return paths like **foo.aux foo.bib foo.tex** etc.

-types *typeList*
: Only list files or directories which match *typeList*, where the items in the list have two forms. The first form is like the -type option of the Unix find command: *b* (block special file), *c* (character special file), *d* (directory), *f* (plain file), *l* (symbolic link), *p* (named pipe), or *s* (socket), where multiple types may be specified in the list. **Glob** will return all files which match at least one of the types given. Note that symbolic links will be returned both if **-types l** is given, or if the target of a link matches the requested type. So, a link to a directory will be returned if **-types d** was specified.

  The second form specifies types where all the types given must match. These are *r*, *w*, *x* as file permissions, and *readonly*, *hidden* as special permission cases. On the Macintosh, macOS types and creators are also supported, where any item which is four characters long is assumed to be a macOS type (e.g. **TEXT**). Items which are of the form *{macintosh type*

XXXX} or *{macintosh creator XXXX}* will match types or creators respectively. Unrecognized types, or specifications of multiple macOS types/creators will signal an error.

The two forms may be mixed, so **-types {d f r w}** will find all regular files OR directories that have both read AND write permissions. The following are equivalent:

```
glob -type d *
glob */
```

except that the first case doesn't return the trailing "/" and is more platform independent.

--

Marks the end of switches. The argument following this one will be treated as a *pattern* even if it starts with a **-**.

Globbing patterns

The *pattern* arguments may contain any of the following special characters, which are a superset of those supported by string match:

?

Matches any single character.

*****

Matches any sequence of zero or more characters.

[*chars*]

Matches any single character in *chars*. If *chars* contains a sequence of the form *a*-*b* then any character between *a* and *b* (inclusive) will match.

x

Matches the character *x*.

{*a*,*b*,...}

Matches any of the sub-patterns *a*, *b*, etc.

On Unix, as with csh, a "." at the beginning of a file's name or just after a "/" must be matched explicitly or with a {} construct, unless the **-types hidden** flag is given (since "." at the beginning of a file's name indicates that it is hidden). On other platforms, files beginning with a "." are handled no differently to any others, except the special directories "." and ".." which must be matched explicitly (this is to avoid a recursive pattern like "glob -join * * * *" from recursing up the directory hierarchy as well as down). In addition, all "/" characters must be matched explicitly.

If the first character in a *pattern* is "~" then it refers to the home directory for the user whose name follows the "~". If the "~" is followed immediately by "/" then the value of the HOME environment variable is used.

The **glob** command differs from csh globbing in two ways. First, it does not sort its result list (use the lsort command if you want the list sorted). Second, **glob** only returns the names of files that actually exist; in csh no check for existence is made unless a pattern contains a ?, *, or [] construct.

When the **glob** command returns relative paths whose filenames start with a tilde "~" (for example through **glob *** or **glob -tails**, the returned list will not quote the tilde with "./". This means care must be taken if those names are later to be used with file join, to avoid them being

interpreted as absolute paths pointing to a given user's home directory.

Windows portability issues

For Windows UNC names, the servername and sharename components of the path may not contain ?, *, or [] constructs. On Windows NT, if *pattern* is of the form "*~username@domain*", it refers to the home directory of the user whose account information resides on the specified NT domain server. Otherwise, user account information is obtained from the local computer.

Since the backslash character has a special meaning to the glob command, glob patterns containing Windows style path separators need special care. The pattern "*C:\\foo**" is interpreted as "*C:\foo**" where "*\f*" will match the single character "*f*" and "*** will match the single character "*" and will not be interpreted as a wildcard character. One solution to this problem is to use the Unix style forward slash as a path separator. Windows style paths can be converted to Unix style paths with the command "file join **$path**" or "file normalize **$path**".

Examples

Find all the Tcl files in the current directory:

```
glob *.tcl
```

Find all the Tcl files in the user's home directory, irrespective of what the current directory is:

```
glob -directory ~ *.tcl
```

Find all subdirectories of the current directory:

```
glob -type d *
```

Find all files whose name contains an "a", a "b" or the sequence "cde":

```
glob -type f *{a,b,cde}*
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

global

Keywords: global, namespace, procedure, variable

Name

global - Access global variables

Synopsis

global ?*varname* ...?

Description

This command has no effect unless executed in the context of a proc body. If the **global** command is executed in the context of a proc body, it creates local variables linked to the corresponding global variables (though these linked variables, like those created by upvar, are not included in the list returned by info locals).

If *varname* contains namespace qualifiers, the local variable's name is the unqualified name of the global variable, as determined by the namespace tail command.

varname is always treated as the name of a variable, not an array element. An error is returned if the name looks like an array element, such as **a(b)**.

Examples

This procedure sets the namespace variable *::a::x*

```
proc reset {} {
    global a::x
    set x 0
}
```

This procedure accumulates the strings passed to it in a global buffer, separated by newlines. It is useful for situations when you want to build a message piece-by-piece (as if with puts) but send that full message in a single piece (e.g. over a connection opened with socket or as part of a counted HTTP response).

```
proc accum {string} {
    global accumulator
    append accumulator $string \n
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

history

Keywords: history, event, record

Name

history - Manipulate the history list

Synopsis

history ?*option*? ?*arg arg* ...?

Description

The **history** command performs one of several operations related to recently-executed commands recorded in a history list. Each of these recorded commands is referred to as an "event". When specifying an event to the **history** command, the following forms may be used:

1. A number: if positive, it refers to the event with that number (all events are numbered starting at 1). If the number is negative, it selects an event relative to the current event (**-1** refers to the previous event, **-2** to the one before that, and so on). Event **0** refers to the current event.
2. A string: selects the most recent event that matches the string. An event is considered to match the string either if the string is the same as the first characters of the event, or if the string matches the event in the sense of the string match command.

The **history** command can take any of the following forms:

history
Same as **history info**, described below.

history add *command* ?exec?
Adds the *command* argument to the history list as a new event. If **exec** is specified (or

abbreviated) then the command is also executed and its result is returned. If **exec** is not specified then an empty string is returned as result.

history change *newValue* ?*event*?
Replaces the value recorded for an event with *newValue*. *Event* specifies the event to replace, and defaults to the *current* event (not event **-1**). This command is intended for use in commands that implement new forms of history substitution and wish to replace the current event (which invokes the substitution) with the command created through substitution. The return value is an empty string.

history clear
Erase the history list. The current keep limit is retained. The history event numbers are reset.

history event ?*event*?
Returns the value of the event given by *event*. *Event* defaults to **-1**.

history info ?*count*?
Returns a formatted string (intended for humans to read) giving the event number and contents for each of the events in the history list except the current event. If *count* is specified then only the most recent *count* events are returned.

history keep ?*count*?
This command may be used to change the size of the history list to *count* events. Initially, 20 events are retained in the history list. If *count* is not specified, the current keep limit is returned.

history nextid
Returns the number of the next event to be recorded in the history list. It is useful for things like printing the event number in command-line prompts.

history redo ?*event*?
Re-executes the command indicated by *event* and returns its result. *Event* defaults to **-1**. This command results in history revision: see below for details.

History revision

Pre-8.0 Tcl had a complex history revision mechanism. The current mechanism is more limited, and the old history operations **substitute** and **words** have been removed. (As a consolation, the **clear** operation was added.)

The history option **redo** results in much simpler "history revision". When this option is invoked then the most recent event is modified to eliminate the history command and replace it with the result of the history command. If you want to redo an event without modifying history, then use the **event** operation to retrieve some event, and the **add** operation to add it to history and execute it.

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

http

Keywords: http, internet, security policy, socket, www

Name

http - Client-side implementation of the HTTP/1.1 protocol

Synopsis

package require http ?**2.9**?
::http::config ?*-option value ...*?
::http::geturl *url* ?*-option value ...*?
::http::formatQuery *key value* ?*key value ...*?
::http::quoteString *value*
::http::reset *token* ?*why*?
::http::wait *token*
::http::status *token*
::http::size *token*
::http::code *token*
::http::ncode *token*
::http::meta *token*
::http::data *token*
::http::error *token*
::http::cleanup *token*
::http::register *proto port command*
::http::registerError *port* ?*message*?
::http::unregister *proto*

Exported commands

Namespace **http** exports the commands **config**, **formatQuery**, **geturl**, **quoteString**, **register**, **registerError**, **reset**, **unregister**, and **wait**.

It does not export the commands **cleanup**, **code**, **data**, **error**, **meta**, **ncode**, **size**, or **status**.

Description

The **http** package provides the client side of the HTTP/1.1 protocol, as defined in RFC 7230 to RFC 7235, which supersede RFC 2616. The package implements the GET, POST, and HEAD operations of HTTP/1.1. It allows configuration of a proxy host to get through firewalls. The package is compatible with the **Safesock** security policy, so it can be used by untrusted applets to do URL fetching from a restricted set of hosts. This package can be extended to support additional HTTP transport protocols, such as HTTPS, by providing a custom socket command, via **::http::register**.

The **::http::geturl** procedure does a HTTP transaction. Its *options* determine whether a GET, POST, or HEAD transaction is performed. The return value of **::http::geturl** is a token for the transaction. The value is also the name of an array in the ::http namespace that contains state information about the transaction. The elements of this array are described in the State array section.

If the **-command** option is specified, then the HTTP operation is done in the background. **::http::geturl** returns immediately after generating the HTTP request and the callback is invoked when the transaction completes. For this to work, the Tcl event loop must be active. In Tk applications this is always true. For pure-Tcl applications, the caller can use **::http::wait** after calling **::http::geturl** to start the event loop.

Note: The event queue is even used without the **-command** option. As a side effect, arbitrary commands may be processed while **http::geturl** is running.

Commands

::http::config ?*options*?

The **::http::config** command is used to set and query the name of the proxy server and port, and the User-Agent name used in the HTTP requests. If no options are specified, then the current configuration is returned. If a single argument is specified, then it should be one of the flags described below. In this case the current value of that setting is returned. Otherwise, the options should be a set of flags and values that define the configuration:

-accept *mimetypes*

The Accept header of the request. The default is */*, which means that all types of documents are accepted. Otherwise you can supply a comma-separated list of mime type patterns that you are willing to receive. For example, "image/gif, image/jpeg, text/*".

-pipeline *boolean*

Specifies whether HTTP/1.1 transactions on a persistent socket will be pipelined. See the **PERSISTENT SOCKETS** section for details. The default is 1.

-postfresh *boolean*

Specifies whether requests that use the **POST** method will always use a fresh socket, overriding the **-keepalive** option of command **http::geturl**. See the **PERSISTENT SOCKETS** section for details. The default is 0.

-proxyhost *hostname*

The name of the proxy host, if any. If this value is the empty string, the URL host is contacted directly.

-proxyport *number*

The proxy port number.

-proxyfilter *command*

The command is a callback that is made during **::http::geturl** to determine if a proxy is required for a given host. One argument, a host name, is added to *command* when it is invoked. If a proxy is required, the callback should return a two-element list containing the proxy server and proxy port. Otherwise the filter should return an empty list. The default filter returns the values of the **-proxyhost** and **-proxyport** settings if they are non-empty.

The **::http::geturl** command runs the **-proxyfilter** callback inside a catch command. Therefore an error in the callback command does not call the bgerror handler. See the Errors section for details.

-repost *boolean*

Specifies what to do if a POST request over a persistent connection fails because the server has half-closed the connection. If boolean **true**, the request will be automatically retried; if boolean **false** it will not, and the application that uses **http::geturl** is expected to seek user confirmation before retrying the POST. The value **true** should be used only under certain conditions. See the **PERSISTENT SOCKETS** section for details. The default is 0.

-urlencoding *encoding*

The *encoding* used for creating the x-url-encoded URLs with **::http::formatQuery** and **::http::quoteString**. The default is **utf-8**, as specified by RFC 2718. Prior to http 2.5 this was unspecified, and that behavior can be returned by specifying the empty string (**{}**), although *iso8859-1* is recommended to restore similar behavior but without the **::http::formatQuery** or **::http::quoteString** throwing an error processing non-latin-1

characters.

-useragent *string*
The value of the User-Agent header in the HTTP request. In an unsafe interpreter, the default value depends upon the operating system, and the version numbers of **http** and **Tcl**, and is (for example) "**Mozilla/5.0 (Windows; U; Windows NT 10.0) http/2.9.0 Tcl/8.6.9**". A safe interpreter cannot determine its operating system, and so the default in a safe interpreter is to use a Windows 10 value with the current version numbers of **http** and **Tcl**.

-zip *boolean*
If the value is boolean **true**, then by default requests will send a header "**Accept-Encoding: gzip,deflate,compress**". If the value is boolean **false**, then by default this header will not be sent. In either case the default can be overridden for an individual request by supplying a custom **Accept-Encoding** header in the **-headers** option of **http::geturl**. The default is 1.

::http::geturl *url* ?*options*?
The **::http::geturl** command is the main procedure in the package. The **-query** option causes a POST operation and the **-validate** option causes a HEAD operation; otherwise, a GET operation is performed. The **::http::geturl** command returns a *token* value that can be used to get information about the transaction. See the State array and Errors section for details. The **::http::geturl** command blocks until the operation completes, unless the **-command** option specifies a callback that is invoked when the HTTP transaction completes. **::http::geturl** takes several options:

-binary *boolean*
Specifies whether to force interpreting the URL data as binary. Normally this is auto-detected (anything not beginning with a **text** content type or whose content encoding is **gzip** or **compress** is considered binary data).

-blocksize *size*
The block size used when reading the URL. At most *size* bytes are read at once. After each block, a call to the **-progress** callback is made (if that option is specified).

-channel *name*
Copy the URL contents to channel *name* instead of saving it in **state(body)**.

-command *callback*
Invoke *callback* after the HTTP transaction completes. This option causes **::http::geturl** to return immediately. The *callback* gets an additional argument that is the *token* returned from **::http::geturl**. This token is the name of an array that is described in the State array section. Here is a template for the callback:

```
proc httpCallback {token} {
    upvar #0 $token state
    # Access state as a Tcl array
}
```

The **::http::geturl** command runs the **-command** callback inside a catch command. Therefore an error in the callback command does not call the bgerror handler. See the Errors section for details.

-handler *callback*
Invoke *callback* whenever HTTP data is available; if present, nothing else will be done with the HTTP data. This procedure gets two additional arguments: the socket for the HTTP data and the *token* returned from **::http::geturl**. The token is the name of a global array that is described in the State array section. The procedure is expected to return the number of bytes read from the socket. Here is a template for the callback:

```
proc httpHandlerCallback {socket token} {
    upvar #0 $token state
    # Access socket, and state as a Tcl array
    # For example...
    ...
    set data [read $socket 1000]
    set nbytes [string length $data]
    ...
    return $nbytes
}
```

The **http::geturl** code for the **-handler** option is not compatible with either compression or chunked transfer-encoding. If **-handler** is specified, then to work around these issues **http::geturl** will reduce the HTTP protocol to 1.0, and override the **-zip** option (i.e. it will not send the header "**Accept-Encoding: gzip,deflate,compress**").

If options **-handler** and **-channel** are used together, the handler is responsible for copying the data from the HTTP socket to the specified channel. The name of the channel is available to the handler as element **-channel** of the token array.

The **::http::geturl** command runs the **-handler** callback inside a catch command. Therefore an error in the callback command does not call the bgerror handler. See the Errors section for details.

-headers *keyvaluelist*
This option is used to add headers not already specified by **::http::config** to the HTTP request. The *keyvaluelist* argument must be a list with an even number of elements that alternate between keys and values. The keys become header field names. Newlines are stripped from the values so the header cannot be corrupted. For example, if *keyvaluelist* is **Pragma no-cache** then the following header is included in the HTTP request:

```
Pragma: no-cache
```

-keepalive *boolean*
If boolean **true**, attempt to keep the connection open for servicing multiple requests. Default is 0.

-method *type*
Force the HTTP request method to *type*. **::http::geturl** will auto-select GET, POST or HEAD based on other options, but this option overrides that selection and enables choices like PUT and DELETE for WebDAV support.

It is the caller's responsibility to ensure that the headers and request body (if any) conform

to the requirements of the request method. For example, if using **-method** *POST* to send a POST with an empty request body, the caller must also supply the option "-headers {Content-Length 0}".

-myaddr *address*
Pass an specific local address to the underlying socket call in case multiple interfaces are available.

-progress *callback*
The *callback* is made after each transfer of data from the URL. The callback gets three additional arguments: the *token* from **::http::geturl**, the expected total size of the contents from the **Content-Length** meta-data, and the current number of bytes transferred so far. The expected total size may be unknown, in which case zero is passed to the callback. Here is a template for the progress callback:

```
proc httpProgress {token total current} {
    upvar #0 $token state
}
```

-protocol *version*
Select the HTTP protocol version to use. This should be 1.0 or 1.1 (the default). Should only be necessary for servers that do not understand or otherwise complain about HTTP/1.1.

-query *query*
This flag (if the value is non-empty) causes **::http::geturl** to do a POST request that passes the string *query* verbatim to the server as the request payload. The content format (and encoding) of *query* is announced by the request header **Content-Type** which is set by the option **-type**. Any value of **-type** is permitted, and it is the responsibility of the caller to supply *query* in the correct format.

If **-type** is not specified, it defaults to *application/x-www-form-urlencoded*, which requires *query* to be an x-url-encoding formatted query-string (this **-type** and query format are used in a POST submitted from an html form). The **::http::formatQuery** procedure can be used to do the formatting.

-queryblocksize *size*
The block size used when posting query data to the URL. At most *size* bytes are written at once. After each block, a call to the **-queryprogress** callback is made (if that option is specified).

-querychannel *channelID*
This flag causes **::http::geturl** to do a POST request that passes the data contained in *channelID* to the server. The data contained in *channelID* must be an x-url-encoding formatted query unless the **-type** option below is used. If a Content-Length header is not specified via the **-headers** options, **::http::geturl** attempts to determine the size of the post data in order to create that header. If it is unable to determine the size, it returns an error.

-queryprogress *callback*
The *callback* is made after each transfer of data to the URL (i.e. POST) and acts exactly like the **-progress** option (the callback format is the same).

-strict *boolean*

Whether to enforce RFC 3986 URL validation on the request. Default is 1.

-timeout *milliseconds*
If *milliseconds* is non-zero, then **::http::geturl** sets up a timeout to occur after the specified number of milliseconds. A timeout results in a call to **::http::reset** and to the **-command** callback, if specified. The return value of **::http::status** is **timeout** after a timeout has occurred.

-type *mime-type*
Use *mime-type* as the **Content-Type** value, instead of the default value (**application/x-www-form-urlencoded**) during a POST operation.

-validate *boolean*
If *boolean* is non-zero, then **::http::geturl** does an HTTP HEAD request. This request returns meta information about the URL, but the contents are not returned. The meta information is available in the **state(meta)** variable after the transaction. See the State array section for details.

::http::formatQuery *key value* ?*key value* ...?
This procedure does x-url-encoding of query data. It takes an even number of arguments that are the keys and values of the query. It encodes the keys and values, and generates one string that has the proper & and = separators. The result is suitable for the **-query** value passed to **::http::geturl**.

::http::quoteString *value*
This procedure does x-url-encoding of string. It takes a single argument and encodes it.

::http::reset *token* ?*why*?
This command resets the HTTP transaction identified by *token*, if any. This sets the **state(status)** value to *why*, which defaults to **reset**, and then calls the registered **-command** callback.

::http::wait *token*
This is a convenience procedure that blocks and waits for the transaction to complete. This only works in trusted code because it uses vwait. Also, it is not useful for the case where **::http::geturl** is called *without* the **-command** option because in this case the **::http::geturl** call does not return until the HTTP transaction is complete, and thus there is nothing to wait for.

::http::data *token*
This is a convenience procedure that returns the **body** element (i.e., the URL data) of the state array.

::http::error *token*
This is a convenience procedure that returns the **error** element of the state array.

::http::status *token*
This is a convenience procedure that returns the **status** element of the state array.

::http::code *token*
This is a convenience procedure that returns the **http** element of the state array.

::http::ncode *token*
This is a convenience procedure that returns just the numeric return code (200, 404, etc.) from the **http** element of the state array.

::http::size *token*

This is a convenience procedure that returns the **currentsize** element of the state array, which represents the number of bytes received from the URL in the **::http::geturl** call.

::http::meta *token*
This is a convenience procedure that returns the **meta** element of the state array which contains the HTTP response headers. See below for an explanation of this element.

::http::cleanup *token*
This procedure cleans up the state associated with the connection identified by *token*. After this call, the procedures like **::http::data** cannot be used to get information about the operation. It is *strongly* recommended that you call this function after you are done with a given HTTP request. Not doing so will result in memory not being freed, and if your app calls **::http::geturl** enough times, the memory leak could cause a performance hit...or worse.

::http::register *proto port command*
This procedure allows one to provide custom HTTP transport types such as HTTPS, by registering a prefix, the default port, and the command to execute to create the Tcl **channel**. E.g.:

```
package require http
package require tls

::http::register https 443 ::tls::socket

set token [::http::geturl https://my.secure.site/]
```

::http::registerError *port* ?*message*?
This procedure allows a registered protocol handler to deliver an error message for use by **http**. Calling this command does not raise an error. The command is useful when a registered protocol detects an problem (for example, an invalid TLS certificate) that will cause an error to propagate to **http**. The command allows **http** to provide a precise error message rather than a general one. The command returns the value provided by the last call with argument *message*, or the empty string if no such call has been made.

::http::unregister *proto*
This procedure unregisters a protocol handler that was previously registered via **::http::register**, returning a two-item list of the default port and handler command that was previously installed (via **::http::register**) if there was such a handler, and an error if there was no such handler.

Errors

The **::http::geturl** procedure will raise errors in the following cases: invalid command line options, an invalid URL, a URL on a non-existent host, or a URL at a bad port on an existing host. These errors mean that it cannot even start the network transaction. It will also raise an error if it gets an I/O error while writing out the HTTP request header. For synchronous **::http::geturl** calls (where **-command** is not specified), it will raise an error if it gets an I/O error while reading the HTTP reply headers or data. Because **::http::geturl** does not return a token in these cases, it does all the required cleanup and there is no issue of your app having to call **::http::cleanup**.

For asynchronous **::http::geturl** calls, all of the above error situations apply, except that if there is any error while reading the HTTP reply headers or data, no exception is thrown. This is

because after writing the HTTP headers, **::http::geturl** returns, and the rest of the HTTP transaction occurs in the background. The command callback can check if any error occurred during the read by calling **::http::status** to check the status and if its *error*, calling **::http::error** to get the error message.

Alternatively, if the main program flow reaches a point where it needs to know the result of the asynchronous HTTP request, it can call **::http::wait** and then check status and error, just as the callback does.

The **::http::geturl** command runs the **-command**, **-handler**, and **-proxyfilter** callbacks inside a catch command. Therefore an error in the callback command does not call the bgerror handler. When debugging one of these callbacks, it may be convenient to report errors by using a catch command within the callback command itself, e.g. to write an error message to stdout.

In any case, you must still call **::http::cleanup** to delete the state array when you are done.

There are other possible results of the HTTP transaction determined by examining the status from **::http::status**. These are described below.

ok
: If the HTTP transaction completes entirely, then status will be **ok**. However, you should still check the **::http::code** value to get the HTTP status. The **::http::ncode** procedure provides just the numeric error (e.g., 200, 404 or 500) while the **::http::code** procedure returns a value like "HTTP 404 File not found".

eof
: If the server closes the socket without replying, then no error is raised, but the status of the transaction will be eof.

error
: The error message will also be stored in the **error** status array element, accessible via **::http::error**.

timeout
: A timeout occurred before the transaction could complete

reset
: user-reset

Another error possibility is that **::http::geturl** is unable to write all the post query data to the server before the server responds and closes the socket. The error message is saved in the **posterror** status array element and then **::http::geturl** attempts to complete the transaction. If it can read the server's response it will end up with an **ok** status, otherwise it will have an eof status.

State array

The **::http::geturl** procedure returns a *token* that can be used to get to the state of the HTTP transaction in the form of a Tcl array. Use this construct to create an easy-to-use array variable:

```
upvar #0 $token state
```

Once the data associated with the URL is no longer needed, the state array should be unset to free up storage. The **::http::cleanup** procedure is provided for that purpose. The following elements of the array are supported:

binary

This is boolean **true** if (after decoding any compression specified by the "Content-Encoding" response header) the HTTP response is binary. It is boolean **false** if the HTTP response is text.

body
The contents of the URL. This will be empty if the **-channel** option has been specified. This value is returned by the **::http::data** command.

charset
The value of the charset attribute from the **Content-Type** meta-data value. If none was specified, this defaults to the RFC standard **iso8859-1**, or the value of **$::http::defaultCharset**. Incoming text data will be automatically converted from this charset to utf-8.

coding
A copy of the **Content-Encoding** meta-data value.

currentsize
The current number of bytes fetched from the URL. This value is returned by the **::http::size** command.

error
If defined, this is the error string seen when the HTTP transaction was aborted.

http
The HTTP status reply from the server. This value is returned by the **::http::code** command. The format of this value is:

```
HTTP/1.1 code string
```

The *code* is a three-digit number defined in the HTTP standard. A code of 200 is OK. Codes beginning with 4 or 5 indicate errors. Codes beginning with 3 are redirection errors. In this case the **Location** meta-data specifies a new URL that contains the requested information.

meta
The HTTP protocol returns meta-data that describes the URL contents. The **meta** element of the state array is a list of the keys and values of the meta-data. This is in a format useful for initializing an array that just contains the meta-data:

```
array set meta $state(meta)
```

Some of the meta-data keys are listed below, but the HTTP standard defines more, and servers are free to add their own.

Content-Type
The type of the URL contents. Examples include **text/html**, **image/gif, application/postscript** and **application/x-tcl**.

Content-Length
The advertised size of the contents. The actual size obtained by **::http::geturl** is available as **state(currentsize)**.

Location
An alternate URL that contains the requested data.

posterror
The error, if any, that occurred while writing the post query data to the server.

status
See description in the chapter Errors above for a list and description of **status**. During the transaction this value is the empty string.

totalsize
A copy of the **Content-Length** meta-data value.

type
A copy of the **Content-Type** meta-data value.

url
The requested URL.

Persistent connections

Basics

See RFC 7230 Sec 6, which supersedes RFC 2616 Sec 8.1.

A persistent connection allows multiple HTTP/1.1 transactions to be carried over the same TCP connection. Pipelining allows a client to make multiple requests over a persistent connection without waiting for each response. The server sends responses in the same order that the requests were received.

If a POST request fails to complete, typically user confirmation is needed before sending the request again. The user may wish to verify whether the server was modified by the failed POST request, before sending the same request again.

A HTTP request will use a persistent socket if the call to **http::geturl** has the option **-keepalive true**. It will use pipelining where permitted if the **http::config** option **-pipeline** is boolean **true** (its default value).

The http package maintains no more than one persistent connection to each server (i.e. each value of "domain:port"). If **http::geturl** is called to make a request over a persistent connection while the connection is busy with another request, the new request will be held in a queue until the connection is free.

The http package does not support HTTP/1.0 persistent connections controlled by the **Keep-Alive** header.

Special cases

This subsection discusses issues related to closure of the persistent connection by the server, automatic retry of failed requests, the special treatment necessary for POST requests, and the options for dealing with these cases.

In accordance with RFC 7230, **http::geturl** does not pipeline requests that use the POST method. If a POST uses a persistent connection and is not the first request on that connection, **http::geturl** waits until it has received the response for the previous request; or (if **http::config** option **-postfresh** is boolean **true**) it uses a new connection for each POST.

If the server is processing a number of pipelined requests, and sends a response header "**Connection: close**" with one of the responses (other than the last), then subsequent responses are unfulfilled. **http::geturl** will send the unfulfilled requests again over a new connection.

A difficulty arises when a HTTP client sends a request over a persistent connection that has been idle for a while. The HTTP server may half-close an apparently idle connection while the client is

sending a request, but before the request arrives at the server: in this case (an "asynchronous close event") the request will fail. The difficulty arises because the client cannot be certain whether the POST modified the state of the server. For HEAD or GET requests, **http::geturl** opens another connection and retransmits the failed request. However, if the request was a POST, RFC 7230 forbids automatic retry by default, suggesting either user confirmation, or confirmation by user-agent software that has semantic understanding of the application. The **http::config** option **-repost** allows for either possibility.

Asynchronous close events can occur only in a short interval of time. The **http** package monitors each persistent connection for closure by the server. Upon detection, the connection is also closed at the client end, and subsequent requests will use a fresh connection.

If the **http::geturl** command is called with option **-keepalive true**, then it will both try to use an existing persistent connection (if one is available), and it will send the server a "**Connection: keep-alive**" request header asking to keep the connection open for future requests.

The **http::config** options **-pipeline**, **-postfresh**, and **-repost** relate to persistent connections.

Option **-pipeline**, if boolean **true**, will pipeline GET and HEAD requests made over a persistent connection. POST requests will not be pipelined - if the POST is not the first transaction on the connection, its request will not be sent until the previous response has finished. GET and HEAD requests made after a POST will not be sent until the POST response has been delivered, and will not be sent if the POST fails.

Option **-postfresh**, if boolean **true**, will override the **http::geturl** option **-keepalive**, and always open a fresh connection for a POST request.

Option **-repost**, if **true**, permits automatic retry of a POST request that fails because it uses a persistent connection that the server has half-closed (an "asynchronous close event"). Subsequent GET and HEAD requests in a failed pipeline will also be retried. *The -repost option should be used only if the application understands that the retry is appropriate* - specifically, the application must know that if the failed POST successfully modified the state of the server, a repeat POST would have no adverse effect.

Protocol upgrades

The HTTP/1.1 **Connection** and **Upgrade** client headers inform the server that the client wishes to change the protocol used over the existing connection (RFC 7230). This mechanism can be used to request a WebSocket (RFC 6455), a higher version of the HTTP protocol (HTTP 2), or TLS encryption. If the server accepts the upgrade request, its response code will be 101.

To request a protocol upgrade when calling **http::geturl**, the **-headers** option must supply appropriate values for **Connection** and **Upgrade**, and the **-command** option must supply a command that implements the requested protocol and can also handle the server response if the server refuses the protocol upgrade. For upgrade requests **http::geturl** ignores the value of option **-keepalive**, and always uses the value **0** so that the upgrade request is not made over a connection that is intended for multiple HTTP requests.

The Tcllib library **websocket** implements WebSockets, and makes the necessary calls to commands in the **http** package.

There is currently no native Tcl client library for HTTP/2.

The **Upgrade** mechanism is not used to request TLS in web browsers, because **http** and **https** are served over different ports. It is used by protocols such as Internet Printing Protocol (IPP) that

are built on top of **http(s)** and use the same TCP port number for both secure and insecure traffic.

In browsers, opportunistic encryption is instead implemented by the **Upgrade-Insecure-Requests** client header. If a secure service is available, the server response code is a 307 redirect, and the response header **Location** specifies the target URL. The browser must call **http::geturl** again in order to fetch this URL. See https://w3c.github.io/webappsec-upgrade-insecure-requests/

Example

This example creates a procedure to copy a URL to a file while printing a progress meter, and prints the meta-data associated with the URL.

```
proc httpcopy { url file {chunk 4096} } {
    set out [open $file w]
    set token [::http::geturl $url -channel $out \
            -progress httpCopyProgress -blocksize $chunk]
    close $out

    # This ends the line started by httpCopyProgress
    puts stderr ""

    upvar #0 $token state
    set max 0
    foreach {name value} $state(meta) {
        if {[string length $name] > $max} {
            set max [string length $name]
        }
        if {[regexp -nocase ^location$ $name]} {
            # Handle URL redirects
            puts stderr "Location:$value"
            return [httpcopy [string trim $value] $file $chunk]
        }
    }
    incr max
    foreach {name value} $state(meta) {
        puts [format "%-*s %s" $max $name: $value]
    }

    return $token
}
proc httpCopyProgress {args} {
    puts -nonewline stderr .
    flush stderr
}
```

Copyright (c) 1995-1997 Sun Microsystems, Inc.
Copyright (c) 1998-2000 Ajuba Solutions.

Copyright (c) 2004 ActiveState Corporation.

if

Keywords: if, boolean, conditional, else, false, true

Name

if - Execute scripts conditionally

Synopsis

if *expr1* ?**then**? *body1* **elseif** *expr2* ?**then**? *body2* **elseif** ... ?**else**? ?*bodyN*?

Description

The *if* command evaluates *expr1* as an expression (in the same way that expr evaluates its argument). The value of the expression must be a boolean (a numeric value, where 0 is false and anything is true, or a string value such as **true** or **yes** for true and **false** or **no** for false); if it is true then *body1* is executed by passing it to the Tcl interpreter. Otherwise *expr2* is evaluated as an expression and if it is true then **body2** is executed, and so on. If none of the expressions evaluates to true then *bodyN* is executed. The **then** and **else** arguments are optional "noise words" to make the command easier to read. There may be any number of **elseif** clauses, including zero. *BodyN* may also be omitted as long as **else** is omitted too. The return value from the command is the result of the body script that was executed, or an empty string if none of the expressions was non-zero and there was no *bodyN*.

Examples

A simple conditional:

```
if {$vbl == 1} { puts "vbl is one" }
```

With an **else**-clause:

```
if {$vbl == 1} {
    puts "vbl is one"
} else {
    puts "vbl is not one"
}
```

With an **elseif**-clause too:

```
if {$vbl == 1} {
    puts "vbl is one"
} elseif {$vbl == 2} {
    puts "vbl is two"
} else {
    puts "vbl is not one or two"
}
```

Remember, expressions can be multi-line, but in that case it can be a good idea to use the optional **then** keyword for clarity:

```
if {
    $vbl == 1
    || $vbl == 2
    || $vbl == 3
} then {
    puts "vbl is one, two or three"
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

incr

Keywords: incr, add, increment, variable, value

Name

incr - Increment the value of a variable

Synopsis

incr *varName* ?*increment*?

Description

Increments the value stored in the variable whose name is *varName*. The value of the variable must be an integer. If *increment* is supplied then its value (which must be an integer) is added to the value of variable *varName*; otherwise 1 is added to *varName*. The new value is stored as a decimal string in variable *varName* and also returned as result.

Starting with the Tcl 8.5 release, the variable *varName* passed to **incr** may be unset, and in that case, it will be set to the value *increment* or to the default increment value of **1**.

Examples

Add one to the contents of the variable *x*:

```
incr x
```

Add 42 to the contents of the variable *x*:

```
incr x 42
```

Add the contents of the variable *y* to the contents of the variable *x*:

```
incr x $y
```

Add nothing at all to the variable *x* (often useful for checking whether an argument to a procedure is actually integral and generating an error if it is not):

```
incr x 0
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

info

Keywords: info, command, information, interpreter, introspection, level, namespace, object, procedure, variable

Name

info - Return information about the state of the Tcl interpreter

Synopsis

info *option* ?*arg arg* ...?

Description

This command provides information about various internals of the Tcl interpreter. The legal *option*s (which may be abbreviated) are:

info args *procname*
: Returns a list containing the names of the arguments to procedure *procname*, in order. *Procname* must be the name of a Tcl command procedure.

info body *procname*
: Returns the body of procedure *procname*. *Procname* must be the name of a Tcl command procedure.

info class *subcommand class ?arg ...*
: Returns information about the class, class. The subcommands are described in Class introspection below.

info cmdcount
: Returns a count of the total number of commands that have been invoked in this interpreter.

info commands ?*pattern*?
: If *pattern* is not specified, returns a list of names of all the Tcl commands visible (i.e. executable without using a qualified name) to the current namespace, including both the built-in commands written in C and the command procedures defined using the **proc** command. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for string match. *pattern* can be a qualified name like **Foo::print***. That is, it may specify a particular namespace using a sequence of namespace names separated by double colons (**::**), and may have pattern matching special characters at the end to specify a set of commands in that namespace. If *pattern* is a qualified name, the resulting list of command names has each one qualified with the name of the specified namespace, and only the commands defined in the named namespace are returned.

info complete *command*
: Returns 1 if *command* is a complete Tcl command in the sense of having no unclosed quotes, braces, brackets or array element names. If the command does not appear to be complete then 0 is returned. This command is typically used in line-oriented input environments to allow users to type in commands that span multiple lines; if the command is not complete, the script can delay evaluating it until additional lines have been typed to complete the command.

info coroutine
: Returns the name of the currently executing coroutine, or the empty string if either no coroutine is currently executing, or the current coroutine has been deleted (but has not yet returned or yielded since deletion).

info default *procname arg varname*
Procname must be the name of a Tcl command procedure and *arg* must be the name of an argument to that procedure. If *arg* does not have a default value then the command returns **0** . Otherwise it returns **1** and places the default value of *arg* into variable *varname*.

info errorstack ?*interp*?
Returns, in a form that is programmatically easy to parse, the function names and arguments at each level from the call stack of the last error in the given interp, or in the current one if not specified.

This form is an even-sized list alternating tokens and parameters. Tokens are currently either **CALL**, **UP**, or **INNER**, but other values may be introduced in the future. **CALL** indicates a procedure call, and its parameter is the corresponding **info level 0**. **UP** indicates a shift in variable frames generated by uplevel or similar, and applies to the previous **CALL** item. Its parameter is the level offset. **INNER** identifies the "inner context", which is the innermost atomic command or bytecode instruction that raised the error, along with its arguments when available. While **CALL** and **UP** allow to follow complex call paths, **INNER** homes in on the offending operation in the innermost procedure call, even going to sub-expression granularity.

This information is also present in the **-errorstack** entry of the options dictionary returned by 3-argument catch; **info errorstack** is a convenient way of retrieving it for uncaught errors at top-level in an interactive tclsh.

info exists *varName*
Returns **1** if the variable named *varName* exists in the current context (either as a global or local variable) and has been defined by being given a value, returns **0** otherwise.

info frame ?*number*?
This command provides access to all frames on the stack, even those hidden from **info level**. If *number* is not specified, this command returns a number giving the frame level of the command. This is 1 if the command is invoked at top-level. If *number* is specified, then the result is a dictionary containing the location information for the command at the *number*ed level on the stack.

If *number* is positive (> 0) then it selects a particular stack level (1 refers to the outer-most active command, 2 to the command it called, and so on, up to the current frame level which refers to **info frame** itself); otherwise it gives a level relative to the current command (0 refers to the current command, i.e., **info frame** itself, -1 to its caller, and so on).

This is similar to how **info level** works, except that this subcommand reports all frames, like sourced scripts, evals, uplevels, etc.

Note that for nested commands, like "foo [bar [x]]", only "x" will be seen by an **info frame** invoked within "x". This is the same as for **info level** and error stack traces.

The result dictionary may contain the keys listed below, with the specified meanings for their values:

type
This entry is always present and describes the nature of the location for the command. The recognized values are source, **proc**, **eval**, and **precompiled**.
source

means that the command is found in a script loaded by the source command.

proc
means that the command is found in dynamically created procedure body.

eval
means that the command is executed by eval or uplevel.

precompiled
means that the command is found in a precompiled script (loadable by the package **tbcload**), and no further information will be available.

line
This entry provides the number of the line the command is at inside of the script it is a part of. This information is not present for type **precompiled**. For type source this information is counted relative to the beginning of the file, whereas for the last two types the line is counted relative to the start of the script.

file
This entry is present only for type source. It provides the normalized path of the file the command is in.

cmd
This entry provides the string representation of the command. This is usually the unsubstituted form, however for commands which are a canonically-constructed list (e.g., as produced by the list command) executed by eval it is the substituted form as they have no other string representation. Care is taken that the canonicality property of the latter is not spoiled.

proc
This entry is present only if the command is found in the body of a regular Tcl procedure. It then provides the name of that procedure.

lambda
This entry is present only if the command is found in the body of an anonymous Tcl procedure, i.e. a lambda. It then provides the entire definition of the lambda in question.

level
This entry is present only if the queried frame has a corresponding frame returned by **info level**. It provides the index of this frame, relative to the current level (0 and negative numbers).

A thing of note is that for procedures statically defined in files the locations of commands in their bodies will be reported with type source and absolute line numbers, and not as type **proc**. The same is true for procedures nested in statically defined procedures, and literal eval scripts in files or statically defined procedures.

In contrast, procedure definitions and eval within a dynamically evaluated environment count line numbers relative to the start of their script, even if they would be able to count relative to the start of the outer dynamic script. That type of number usually makes more sense.

A different way of describing this behaviour is that file based locations are tracked as deeply as possible, and where this is not possible the lines are counted based on the smallest possible eval or procedure body, as that scope is usually easier to find than any dynamic outer scope.

The syntactic form **{*}** is handled like eval. I.e. if it is given a literal list argument the system tracks the line number within the list words as well, and otherwise all line numbers are counted relative to the start of each word (smallest scope)

info functions ?*pattern*?
If *pattern* is not specified, returns a list of all the math functions currently defined. If *pattern* is specified, only those functions whose name matches *pattern* are returned. Matching is determined using the same rules as for string match.

info globals ?*pattern*?
If *pattern* is not specified, returns a list of all the names of currently-defined global variables. Global variables are variables in the global namespace. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for string match.

info hostname
Returns the name of the computer on which this invocation is being executed. Note that this name is not guaranteed to be the fully qualified domain name of the host. Where machines have several different names (as is common on systems with both TCP/IP (DNS) and NetBIOS-based networking installed,) it is the name that is suitable for TCP/IP networking that is returned.

info level ?*number*?
If *number* is not specified, this command returns a number giving the stack level of the invoking procedure, or 0 if the command is invoked at top-level. If *number* is specified, then the result is a list consisting of the name and arguments for the procedure call at level *number* on the stack. If *number* is positive then it selects a particular stack level (1 refers to the top-most active procedure, 2 to the procedure it called, and so on); otherwise it gives a level relative to the current level (0 refers to the current procedure, -1 to its caller, and so on). See the uplevel command for more information on what stack levels mean.

info library
Returns the name of the library directory in which standard Tcl scripts are stored. This is actually the value of the tcl_library variable and may be changed by setting tcl_library.

info loaded ?*interp*?
Returns a list describing all of the packages that have been loaded into *interp* with the load command. Each list element is a sub-list with two elements consisting of the name of the file from which the package was loaded and the name of the package. For statically-loaded packages the file name will be an empty string. If *interp* is omitted then information is returned for all packages loaded in any interpreter in the process. To get a list of just the packages in the current interpreter, specify an empty string for the *interp* argument.

info locals ?*pattern*?
If *pattern* is not specified, returns a list of all the names of currently-defined local variables, including arguments to the current procedure, if any. Variables defined with the global, upvar and variable commands will not be returned. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for string match.

info nameofexecutable
Returns the full path name of the binary file from which the application was invoked. If Tcl was unable to identify the file, then an empty string is returned.

info object *subcommand object* ?*arg ...*
Returns information about the object, object. The subcommands are described in Object introspection below.

info patchlevel
Returns the value of the global variable tcl_patchLevel, which holds the exact version of the Tcl library by default.

info procs ?*pattern*?
If *pattern* is not specified, returns a list of all the names of Tcl command procedures in the current namespace. If *pattern* is specified, only those procedure names in the current namespace matching *pattern* are returned. Matching is determined using the same rules as for string match. If *pattern* contains any namespace separators, they are used to select a namespace relative to the current namespace (or relative to the global namespace if *pattern* starts with **::**) to match within; the matching pattern is taken to be the part after the last namespace separator.

info script ?*filename*?
If a Tcl script file is currently being evaluated (i.e. there is a call to Tcl_EvalFile active or there is an active invocation of the source command), then this command returns the name of the innermost file being processed. If *filename* is specified, then the return value of this command will be modified for the duration of the active invocation to return that name. This is useful in virtual file system applications. Otherwise the command returns an empty string.

info sharedlibextension
Returns the extension used on this platform for the names of files containing shared libraries (for example, **.so** under Solaris). If shared libraries are not supported on this platform then an empty string is returned.

info tclversion
Returns the value of the global variable tcl_version, which holds the major and minor version of the Tcl library by default.

info vars ?*pattern*?
If *pattern* is not specified, returns a list of all the names of currently-visible variables. This includes locals and currently-visible globals. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for string match. *pattern* can be a qualified name like **Foo::option***. That is, it may specify a particular namespace using a sequence of namespace names separated by double colons (**::**), and may have pattern matching special characters at the end to specify a set of variables in that namespace. If *pattern* is a qualified name, the resulting list of variable names has each matching namespace variable qualified with the name of its namespace. Note that a currently-visible variable may not yet "exist" if it has not been set (e.g. a variable declared but not set by variable).

Class introspection

The following *subcommand* values are supported by **info class**:

info class call *class method*
Returns a description of the method implementations that are used to provide a stereotypical instance of class's implementation of method (stereotypical instances being objects instantiated by a class without having any object-specific definitions added). This consists of a

list of lists of four elements, where each sublist consists of a word that describes the general type of method implementation (being one of method for an ordinary method, filter for an applied filter, and unknown for a method that is invoked as part of unknown method handling), a word giving the name of the particular method invoked (which is always the same as method for the method type, and "unknown" for the unknown type), a word giving the fully qualified name of the class that defined the method, and a word describing the type of method implementation (see info class methodtype).

Note that there is no inspection of whether the method implementations actually use next to transfer control along the call chain.

info class constructor *class*
This subcommand returns a description of the definition of the constructor of class class. The definition is described as a two element list; the first element is the list of arguments to the constructor in a form suitable for passing to another call to proc or a method definition, and the second element is the body of the constructor. If no constructor is present, this returns the empty list.

info class definition *class method*
This subcommand returns a description of the definition of the method named method of class class. The definition is described as a two element list; the first element is the list of arguments to the method in a form suitable for passing to another call to proc or a method definition, and the second element is the body of the method.

info class destructor *class*
This subcommand returns the body of the destructor of class class. If no destructor is present, this returns the empty string.

info class filters *class*
This subcommand returns the list of filter methods set on the class.

info class forward *class method*
This subcommand returns the argument list for the method forwarding called method that is set on the class called class.

info class instances *class* ?*pattern*?
This subcommand returns a list of instances of class class. If the optional pattern argument is present, it constrains the list of returned instances to those that match it according to the rules of string match.

info class methods *class* ?*options...*?
This subcommand returns a list of all public (i.e. exported) methods of the class called class. Any of the following options may be specified, controlling exactly which method names are returned:

-all
If the -all flag is given, the list of methods will include those methods defined not just by the class, but also by the class's superclasses and mixins.

-private
If the -private flag is given, the list of methods will also include the private (i.e. non-exported) methods of the class (and superclasses and mixins, if -all is also given).

info class methodtype *class method*

This subcommand returns a description of the type of implementation used for the method named method of class class. When the result is method, further information can be discovered with info class definition, and when the result is forward, further information can be discovered with info class forward.

info class mixins *class*
This subcommand returns a list of all classes that have been mixed into the class named class.

info class subclasses *class* ?*pattern*?
This subcommand returns a list of direct subclasses of class class. If the optional pattern argument is present, it constrains the list of returned classes to those that match it according to the rules of string match.

info class superclasses *class*
This subcommand returns a list of direct superclasses of class class in inheritance precedence order.

info class variables *class*
This subcommand returns a list of all variables that have been declared for the class named class (i.e. that are automatically present in the class's methods, constructor and destructor).

Object introspection

The following *subcommand* values are supported by **info object**:

info object call *object method*
Returns a description of the method implementations that are used to provide object's implementation of method. This consists of a list of lists of four elements, where each sublist consists of a word that describes the general type of method implementation (being one of method for an ordinary method, filter for an applied filter, and unknown for a method that is invoked as part of unknown method handling), a word giving the name of the particular method invoked (which is always the same as method for the method type, and "unknown" for the unknown type), a word giving what defined the method (the fully qualified name of the class, or the literal string object if the method implementation is on an instance), and a word describing the type of method implementation (see info object methodtype).

Note that there is no inspection of whether the method implementations actually use next to transfer control along the call chain.

info object class *object* ?*className*?
If className is unspecified, this subcommand returns class of the object object. If className is present, this subcommand returns a boolean value indicating whether the object is of that class.

info object definition *object method*
This subcommand returns a description of the definition of the method named method of object object. The definition is described as a two element list; the first element is the list of arguments to the method in a form suitable for passing to another call to proc or a method definition, and the second element is the body of the method.

info object filters *object*
This subcommand returns the list of filter methods set on the object.

info object forward *object method*

This subcommand returns the argument list for the method forwarding called method that is set on the object called object.

info object isa *category object* ?*arg*?

This subcommand tests whether an object belongs to a particular category, returning a boolean value that indicates whether the object argument meets the criteria for the category. The supported categories are:

info object isa class *object*

This returns whether object is a class (i.e. an instance of oo::class or one of its subclasses).

info object isa metaclass *object*

This returns whether object is a class that can manufacture classes (i.e. is oo::class or a subclass of it).

info object isa mixin *object class*

This returns whether class is directly mixed into object.

info object isa object *object*

This returns whether object really is an object.

info object isa typeof *object class*

This returns whether class is the type of object (i.e. whether object is an instance of class or one of its subclasses, whether direct or indirect).

info object methods *object* ?*option...*?

This subcommand returns a list of all public (i.e. exported) methods of the object called object. Any of the following options may be specified, controlling exactly which method names are returned:

-all

If the -all flag is given, the list of methods will include those methods defined not just by the object, but also by the object's class and mixins, plus the superclasses of those classes.

-private

If the -private flag is given, the list of methods will also include the private (i.e. non-exported) methods of the object (and classes, if -all is also given).

info object methodtype *object method*

This subcommand returns a description of the type of implementation used for the method named method of object object. When the result is method, further information can be discovered with info object definition, and when the result is forward, further information can be discovered with info object forward.

info object mixins *object*

This subcommand returns a list of all classes that have been mixed into the object named object.

info object namespace *object*

This subcommand returns the name of the internal namespace of the object named object.

info object variables *object*

This subcommand returns a list of all variables that have been declared for the object named object (i.e. that are automatically present in the object's methods).

info object vars *object* ?*pattern*?

This subcommand returns a list of all variables in the private namespace of the object named

object. If the optional pattern argument is given, it is a filter (in the syntax of a string match glob pattern) that constrains the list of variables returned. Note that this is different from the list returned by info object variables; that can include variables that are currently unset, whereas this can include variables that are not automatically included by any of object's methods (or those of its class, superclasses or mixins).

Examples

This command prints out a procedure suitable for saving in a Tcl script:

```
proc printProc {procName} {
    set result [list proc $procName]
    set formals {}
    foreach var [info args $procName] {
        if {[info default $procName $var def]} {
            lappend formals [list $var $def]
        } else {
            # Still need the list-quoting because variable
            # names may properly contain spaces.
            lappend formals [list $var]
        }
    }
    puts [lappend result $formals [info body $procName]]
}
```

Examples with objects

Every object necessarily knows what its class is; this information is trivially extractable through introspection:

```
oo::class create c
c create o
puts [info object class o]
                    → prints "::c"
puts [info object class c]
                    → prints "::oo::class"
```

The introspection capabilities can be used to discover what class implements a method and get how it is defined. This procedure illustrates how:

```
proc getDef {obj method} {
    foreach inf [info object call $obj $method] {
        lassign $inf calltype name locus methodtype
        # Assume no forwards or filters, and hence no $calltype
        # or $methodtype checks...
        if {$locus eq "object"} {
            return [info object definition $obj $name]
        } else {
            return [info class definition $locus $name]
        }
```

```
    }
    error "no definition for $method"
}
```

This is an alternate way of looking up the definition; it is implemented by manually scanning the list of methods up the inheritance tree. This code assumes that only single inheritance is in use, and that there is no complex use of mixed-in classes (in such cases, using **info object call** as above is the simplest way of doing this by far):

```
proc getDef {obj method} {
    if {$method in [info object methods $obj]} {
        # Assume no forwards
        return [info object definition $obj $method]
    }
    set cls [info object class $obj]
    while {$method ni [info class methods $cls]} {
        # Assume the simple case
        set cls [lindex [info class superclass $cls] 0]
        if {$cls eq ""} {
            error "no definition for $method"
        }
    }
    # Assume no forwards
    return [info class definition $cls $method]
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.
Copyright (c) 1993-1997 Bell Labs Innovations for Lucent Technologies
Copyright (c) 1998-2000 Ajuba Solutions
Copyright (c) 2007-2012 Donal K. Fellows

interp

Keywords: interp, alias, parent interpreter, safe interpreter, child interpreter

Name

interp - Create and manipulate Tcl interpreters

Synopsis

interp *subcommand* ?*arg arg* ...?

Description

This command makes it possible to create one or more new Tcl interpreters that co-exist with the creating interpreter in the same application. The creating interpreter is called the *parent* and the new interpreter is called a *child*. A parent can create any number of children, and each child can itself create additional children for which it is parent, resulting in a hierarchy of interpreters.

Each interpreter is independent from the others: it has its own name space for commands,

procedures, and global variables. A parent interpreter may create connections between its children and itself using a mechanism called an *alias*. An *alias* is a command in a child interpreter which, when invoked, causes a command to be invoked in its parent interpreter or in another child interpreter. The only other connections between interpreters are through environment variables (the env variable), which are normally shared among all interpreters in the application, and by resource limit exceeded callbacks. Note that the name space for files (such as the names returned by the open command) is no longer shared between interpreters. Explicit commands are provided to share files and to transfer references to open files from one interpreter to another.

The **interp** command also provides support for *safe* interpreters. A safe interpreter is a child whose functions have been greatly restricted, so that it is safe to execute untrusted scripts without fear of them damaging other interpreters or the application's environment. For example, all IO channel creation commands and subprocess creation commands are made inaccessible to safe interpreters. See Safe interpreters below for more information on what features are present in a safe interpreter. The dangerous functionality is not removed from the safe interpreter; instead, it is *hidden*, so that only trusted interpreters can obtain access to it. For a detailed explanation of hidden commands, see Hidden commands, below. The alias mechanism can be used for protected communication (analogous to a kernel call) between a child interpreter and its parent. See Alias invocation, below, for more details on how the alias mechanism works.

A qualified interpreter name is a proper Tcl list containing a subset of its ancestors in the interpreter hierarchy, terminated by the string naming the interpreter in its immediate parent. Interpreter names are relative to the interpreter in which they are used. For example, if "**a**" is a child of the current interpreter and it has a child "**a1**", which in turn has a child "**a11**", the qualified name of "**a11**" in "**a**" is the list "**a1 a11**".

The **interp** command, described below, accepts qualified interpreter names as arguments; the interpreter in which the command is being evaluated can always be referred to as **{}** (the empty list or string). Note that it is impossible to refer to a parent (ancestor) interpreter by name in a child interpreter except through aliases. Also, there is no global name by which one can refer to the first interpreter created in an application. Both restrictions are motivated by safety concerns.

The interp command

The **interp** command is used to create, delete, and manipulate child interpreters, and to share or transfer channels between interpreters. It can have any of several forms, depending on the *subcommand* argument:

interp alias *srcPath srcToken*
: Returns a Tcl list whose elements are the *targetCmd* and *arg*s associated with the alias represented by *srcToken* (this is the value returned when the alias was created; it is possible that the name of the source command in the child is different from *srcToken*).

interp alias *srcPath srcToken* {}
: Deletes the alias for *srcToken* in the child interpreter identified by *srcPath*. *srcToken* refers to the value returned when the alias was created; if the source command has been renamed, the renamed command will be deleted.

interp alias *srcPath srcCmd targetPath targetCmd* ?*arg arg ...*?
: This command creates an alias between one child and another (see the **alias** child command

below for creating aliases between a child and its parent). In this command, either of the child interpreters may be anywhere in the hierarchy of interpreters under the interpreter invoking the command. *SrcPath* and *srcCmd* identify the source of the alias. *SrcPath* is a Tcl list whose elements select a particular interpreter. For example, "**a b**" identifies an interpreter "**b**", which is a child of interpreter "**a**", which is a child of the invoking interpreter. An empty list specifies the interpreter invoking the command. *srcCmd* gives the name of a new command, which will be created in the source interpreter. *TargetPath* and *targetCmd* specify a target interpreter and command, and the *arg* arguments, if any, specify additional arguments to *targetCmd* which are prepended to any arguments specified in the invocation of *srcCmd*. *TargetCmd* may be undefined at the time of this call, or it may already exist; it is not created by this command. The alias arranges for the given target command to be invoked in the target interpreter whenever the given source command is invoked in the source interpreter. See Alias invocation below for more details. The command returns a token that uniquely identifies the command created *srcCmd*, even if the command is renamed afterwards. The token may but does not have to be equal to *srcCmd*.

interp aliases ?*path*?

This command returns a Tcl list of the tokens of all the source commands for aliases defined in the interpreter identified by *path*. The tokens correspond to the values returned when the aliases were created (which may not be the same as the current names of the commands).

interp bgerror *path* ?*cmdPrefix*?

This command either gets or sets the current background exception handler for the interpreter identified by *path*. If *cmdPrefix* is absent, the current background exception handler is returned, and if it is present, it is a list of words (of minimum length one) that describes what to set the interpreter's background exception handler to. See the Background exception handling section for more details.

interp cancel ?-unwind? ?--? ?*path*? ?*result*?

Cancels the script being evaluated in the interpreter identified by path. Without the -unwind switch the evaluation stack for the interpreter is unwound until an enclosing catch command is found or there are no further invocations of the interpreter left on the call stack. With the -unwind switch the evaluation stack for the interpreter is unwound without regard to any intervening catch command until there are no further invocations of the interpreter left on the call stack. The -- switch can be used to mark the end of switches; it may be needed if path is an unusual value such as -safe. If result is present, it will be used as the error message string; otherwise, a default error message string will be used.

interp create ?-safe? ?--? ?*path*?

Creates a child interpreter identified by *path* and a new command, called a *child command*. The name of the child command is the last component of *path*. The new child interpreter and the child command are created in the interpreter identified by the path obtained by removing the last component from *path*. For example, if *path* is **a b c** then a new child interpreter and child command named **c** are created in the interpreter identified by the path **a b**. The child command may be used to manipulate the new interpreter as described below. If *path* is omitted, Tcl creates a unique name of the form **interp***x*, where *x* is an integer, and uses it for the interpreter and the child command. If the **-safe** switch is specified (or if the parent interpreter is a safe interpreter), the new child interpreter will be created as a safe interpreter with limited functionality; otherwise the child will include the full set of Tcl built-in commands and variables. The **--** switch can be used to mark the end of switches; it may be needed if

path is an unusual value such as **-safe**. The result of the command is the name of the new interpreter. The name of a child interpreter must be unique among all the children for its parent; an error occurs if a child interpreter by the given name already exists in this parent. The initial recursion limit of the child interpreter is set to the current recursion limit of its parent interpreter.

interp debug *path* ?-frame ?*bool*??

Controls whether frame-level stack information is captured in the child interpreter identified by *path*. If no arguments are given, option and current setting are returned. If **-frame** is given, the debug setting is set to the given boolean if provided and the current setting is returned. This only affects the output of info frame, in that exact frame-level information for command invocation at the bytecode level is only captured with this setting on.

For example, with code like

```
proc mycontrol {... script} {
  ...
  uplevel 1 $script
  ...
}

proc dosomething {...} {
  ...
  mycontrol {
    somecode
  }
}
```

the standard setting will provide a relative line number for the command **somecode** and the relevant frame will be of type eval. With frame-debug active on the other hand the tracking extends so far that the system will be able to determine the file and absolute line number of this command, and return a frame of type source. This more exact information is paid for with slower execution of all commands.

Note that once it is on, this flag cannot be switched back off: such attempts are silently ignored. This is needed to maintain the consistency of the underlying interpreter's state.

interp delete ?*path ...?*

Deletes zero or more interpreters given by the optional *path* arguments, and for each interpreter, it also deletes its children. The command also deletes the child command for each interpreter deleted. For each *path* argument, if no interpreter by that name exists, the command raises an error.

interp eval ***path arg ?arg ...?***

This command concatenates all of the *arg* arguments in the same fashion as the concat command, then evaluates the resulting string as a Tcl script in the child interpreter identified by *path*. The result of this evaluation (including all return options, such as **-errorinfo** and **-errorcode** information, if an error occurs) is returned to the invoking interpreter. Note that the script will be executed in the current context stack frame of the *path* interpreter; this is so that the implementations (in a parent interpreter) of aliases in a child interpreter can execute

scripts in the child that find out information about the child's current state and stack frame.

interp exists *path*
Returns **1** if a child interpreter by the specified *path* exists in this parent, **0** otherwise. If *path* is omitted, the invoking interpreter is used.

interp expose *path hiddenName ?exposedCmdName?*
Makes the hidden command *hiddenName* exposed, eventually bringing it back under a new *exposedCmdName* name (this name is currently accepted only if it is a valid global name space name without any ::), in the interpreter denoted by *path*. If an exposed command with the targeted name already exists, this command fails. Hidden commands are explained in more detail in Hidden commands, below.

interp hide *path exposedCmdName ?hiddenCmdName?*
Makes the exposed command *exposedCmdName* hidden, renaming it to the hidden command *hiddenCmdName*, or keeping the same name if *hiddenCmdName* is not given, in the interpreter denoted by *path*. If a hidden command with the targeted name already exists, this command fails. Currently both *exposedCmdName* and *hiddenCmdName* can not contain namespace qualifiers, or an error is raised. Commands to be hidden by **interp hide** are looked up in the global namespace even if the current namespace is not the global one. This prevents children from fooling a parent interpreter into hiding the wrong command, by making the current namespace be different from the global one. Hidden commands are explained in more detail in Hidden commands, below.

interp hidden *path*
Returns a list of the names of all hidden commands in the interpreter identified by *path*.

interp invokehidden *path ?-option ...? hiddenCmdName ?arg ...?*
Invokes the hidden command *hiddenCmdName* with the arguments supplied in the interpreter denoted by *path*. No substitutions or evaluation are applied to the arguments. Three *-option*s are supported, all of which start with **-**: **-namespace** (which takes a single argument afterwards, *nsName*), **-global**, and **--**. If the **-namespace** flag is present, the hidden command is invoked in the namespace called *nsName* in the target interpreter. If the **-global** flag is present, the hidden command is invoked at the global level in the target interpreter; otherwise it is invoked at the current call frame and can access local variables in that and outer call frames. The **--** flag allows the *hiddenCmdName* argument to start with a "-" character, and is otherwise unnecessary. If both the **-namespace** and **-global** flags are present, the **-namespace** flag is ignored. Note that the hidden command will be executed (by default) in the current context stack frame of the *path* interpreter. Hidden commands are explained in more detail in Hidden commands, below.

interp issafe ?*path*?
Returns **1** if the interpreter identified by the specified *path* is safe, **0** otherwise.

interp limit *path limitType ?-option? ?value...?*
Sets up, manipulates and queries the configuration of the resource limit *limitType* for the interpreter denoted by *path*. If no *-option* is specified, return the current configuration of the limit. If *-option* is the sole argument, return the value of that option. Otherwise, a list of *-option*/*value* argument pairs must supplied. See Resource limits below for a more detailed explanation of what limits and options are supported.

interp marktrusted *path*
Marks the interpreter identified by *path* as trusted. Does not expose the hidden commands.

This command can only be invoked from a trusted interpreter. The command has no effect if the interpreter identified by *path* is already trusted.

interp recursionlimit *path* ?*newlimit*?
Returns the maximum allowable nesting depth for the interpreter specified by *path*. If *newlimit* is specified, the interpreter recursion limit will be set so that nesting of more than *newlimit* calls to Tcl_Eval and related procedures in that interpreter will return an error. The *newlimit* value is also returned. The *newlimit* value must be a positive integer between 1 and the maximum value of a non-long integer on the platform.

The command sets the maximum size of the Tcl call stack only. It cannot by itself prevent stack overflows on the C stack being used by the application. If your machine has a limit on the size of the C stack, you may get stack overflows before reaching the limit set by the command. If this happens, see if there is a mechanism in your system for increasing the maximum size of the C stack.

interp share *srcPath channelId destPath*
Causes the IO channel identified by *channelId* to become shared between the interpreter identified by *srcPath* and the interpreter identified by *destPath*. Both interpreters have the same permissions on the IO channel. Both interpreters must close it to close the underlying IO channel; IO channels accessible in an interpreter are automatically closed when an interpreter is destroyed.

interp slaves ?*path*?
Returns a Tcl list of the names of all the child interpreters associated with the interpreter identified by *path*. If *path* is omitted, the invoking interpreter is used.

interp children ?path?
Synonym for. interp slaves ?path?

interp target *path alias*
Returns a Tcl list describing the target interpreter for an alias. The alias is specified with an interpreter path and source command name, just as in **interp alias** above. The name of the target interpreter is returned as an interpreter path, relative to the invoking interpreter. If the target interpreter for the alias is the invoking interpreter then an empty list is returned. If the target interpreter for the alias is not the invoking interpreter or one of its descendants then an error is generated. The target command does not have to be defined at the time of this invocation.

interp transfer *srcPath channelId destPath*
Causes the IO channel identified by *channelId* to become available in the interpreter identified by *destPath* and unavailable in the interpreter identified by *srcPath*.

Child command

For each child interpreter created with the **interp** command, a new Tcl command is created in the parent interpreter with the same name as the new interpreter. This command may be used to invoke various operations on the interpreter. It has the following general form:

```
child command ?arg arg ...?
```

child is the name of the interpreter, and *command* and the *arg*s determine the exact behavior of the command. The valid forms of this command are:

***child* aliases**
Returns a Tcl list whose elements are the tokens of all the aliases in *child*. The tokens correspond to the values returned when the aliases were created (which may not be the same as the current names of the commands).

child* alias *srcToken
Returns a Tcl list whose elements are the *targetCmd* and *arg*s associated with the alias represented by *srcToken* (this is the value returned when the alias was created; it is possible that the actual source command in the child is different from *srcToken*).

***child* alias *srcToken* {}**
Deletes the alias for *srcToken* in the child interpreter. *srcToken* refers to the value returned when the alias was created; if the source command has been renamed, the renamed command will be deleted.

child* alias *srcCmd targetCmd ?arg..?
Creates an alias such that whenever *srcCmd* is invoked in *child*, *targetCmd* is invoked in the parent. The *arg* arguments will be passed to *targetCmd* as additional arguments, prepended before any arguments passed in the invocation of *srcCmd*. See Alias invocation below for details. The command returns a token that uniquely identifies the command created *srcCmd*, even if the command is renamed afterwards. The token may but does not have to be equal to *srcCmd*.

***child* bgerror ?*cmdPrefix*?**
This command either gets or sets the current background exception handler for the *child* interpreter. If *cmdPrefix* is absent, the current background exception handler is returned, and if it is present, it is a list of words (of minimum length one) that describes what to set the interpreter's background exception handler to. See the Background exception handling section for more details.

child* eval *arg ?arg..?
This command concatenates all of the *arg* arguments in the same fashion as the concat command, then evaluates the resulting string as a Tcl script in *child*. The result of this evaluation (including all return options, such as **-errorinfo** and **-errorcode** information, if an error occurs) is returned to the invoking interpreter. Note that the script will be executed in the current context stack frame of *child*; this is so that the implementations (in a parent interpreter) of aliases in a child interpreter can execute scripts in the child that find out information about the child's current state and stack frame.

***child* expose *hiddenName* ?*exposedCmdName*?**
This command exposes the hidden command *hiddenName*, eventually bringing it back under a new *exposedCmdName* name (this name is currently accepted only if it is a valid global name space name without any ::), in *child*. If an exposed command with the targeted name already exists, this command fails. For more details on hidden commands, see Hidden commands, below.

***child* hide *exposedCmdName* ?*hiddenCmdName*?**
This command hides the exposed command *exposedCmdName*, renaming it to the hidden command *hiddenCmdName*, or keeping the same name if the argument is not given, in the *child* interpreter. If a hidden command with the targeted name already exists, this command fails. Currently both *exposedCmdName* and *hiddenCmdName* can not contain namespace qualifiers, or an error is raised. Commands to be hidden are looked up in the global

namespace even if the current namespace is not the global one. This prevents children from fooling a parent interpreter into hiding the wrong command, by making the current namespace be different from the global one. For more details on hidden commands, see Hidden commands, below.

***child* hidden**
Returns a list of the names of all hidden commands in *child*.

***child* invokehidden ?*-option ...*? *hiddenName* ?*arg..*?**
This command invokes the hidden command *hiddenName* with the supplied arguments, in *child*. No substitutions or evaluations are applied to the arguments. Three *-option*s are supported, all of which start with **-**: **-namespace** (which takes a single argument afterwards, *nsName*), **-global**, and **--**. If the **-namespace** flag is given, the hidden command is invoked in the specified namespace in the child. If the **-global** flag is given, the command is invoked at the global level in the child; otherwise it is invoked at the current call frame and can access local variables in that or outer call frames. The **--** flag allows the *hiddenCmdName* argument to start with a "-" character, and is otherwise unnecessary. If both the **-namespace** and **-global** flags are given, the **-namespace** flag is ignored. Note that the hidden command will be executed (by default) in the current context stack frame of *child*. For more details on hidden commands, see Hidden commands, below.

***child* issafe**
Returns **1** if the child interpreter is safe, **0** otherwise.

***child* limit *limitType* ?*-option*? ?*value...*?**
Sets up, manipulates and queries the configuration of the resource limit *limitType* for the child interpreter. If no *-option* is specified, return the current configuration of the limit. If *-option* is the sole argument, return the value of that option. Otherwise, a list of *-option*/*value* argument pairs must supplied. See Resource limits below for a more detailed explanation of what limits and options are supported.

***child* marktrusted**
Marks the child interpreter as trusted. Can only be invoked by a trusted interpreter. This command does not expose any hidden commands in the child interpreter. The command has no effect if the child is already trusted.

***child* recursionlimit ?*newlimit*?**
Returns the maximum allowable nesting depth for the *child* interpreter. If *newlimit* is specified, the recursion limit in *child* will be set so that nesting of more than *newlimit* calls to Tcl_Eval**()** and related procedures in *child* will return an error. The *newlimit* value is also returned. The *newlimit* value must be a positive integer between 1 and the maximum value of a non-long integer on the platform.

The command sets the maximum size of the Tcl call stack only. It cannot by itself prevent stack overflows on the C stack being used by the application. If your machine has a limit on the size of the C stack, you may get stack overflows before reaching the limit set by the command. If this happens, see if there is a mechanism in your system for increasing the maximum size of the C stack.

Safe interpreters

A safe interpreter is one with restricted functionality, so that is safe to execute an arbitrary script

from your worst enemy without fear of that script damaging the enclosing application or the rest of your computing environment. In order to make an interpreter safe, certain commands and variables are removed from the interpreter. For example, commands to create files on disk are removed, and the exec command is removed, since it could be used to cause damage through subprocesses. Limited access to these facilities can be provided, by creating aliases to the parent interpreter which check their arguments carefully and provide restricted access to a safe subset of facilities. For example, file creation might be allowed in a particular subdirectory and subprocess invocation might be allowed for a carefully selected and fixed set of programs.

A safe interpreter is created by specifying the **-safe** switch to the **interp create** command. Furthermore, any child created by a safe interpreter will also be safe.

A safe interpreter is created with exactly the following set of built-in commands:

after append apply array
binary break catch chan
clock close concat continue
dict eof error eval
expr fblocked fcopy fileevent
flush for foreach format
gets global if incr
info **interp** join lappend
lassign lindex linsert list
llength lrange lrepeat lreplace
lsearch lset lsort namespace
package pid proc puts
read regexp regsub rename
return scan seek set
split string subst switch
tell **time** trace unset
update uplevel upvar variable
vwait while
zlib

The following commands are hidden by **interp create** when it creates a safe interpreter:

cd encoding exec exit
fconfigure file glob load
open pwd socket source
unload

These commands can be recreated later as Tcl procedures or aliases, or re-exposed by **interp expose**.

The following commands from Tcl's library of support procedures are not present in a safe interpreter:

auto_exec_ok auto_import auto_load
auto_load_index auto_qualify unknown

Note in particular that safe interpreters have no default unknown command, so Tcl's default autoloading facilities are not available. Autoload access to Tcl's commands that are normally autoloaded:

auto_mkindex **auto_mkindex_old**
auto_reset history
parray pkg_mkIndex
::pkg::create ::safe::interpAddToAccessPath
::safe::interpCreate ::safe::interpConfigure
::safe::interpDelete ::safe::interpFindInAccessPath
::safe::interpInit ::safe::setLogCmd
tcl_endOfWord tcl_findLibrary
tcl_startOfNextWord tcl_startOfPreviousWord
tcl_wordBreakAfter tcl_wordBreakBefore

can only be provided by explicit definition of an unknown command in the safe interpreter. This will involve exposing the source command. This is most easily accomplished by creating the safe interpreter with Tcl's **Safe-Tcl** mechanism. **Safe-Tcl** provides safe versions of source, load, and other Tcl commands needed to support autoloading of commands and the loading of packages.

In addition, the env variable is not present in a safe interpreter, so it cannot share environment variables with other interpreters. The env variable poses a security risk, because users can store sensitive information in an environment variable. For example, the PGP manual recommends storing the PGP private key protection password in the environment variable *PGPPASS*. Making this variable available to untrusted code executing in a safe interpreter would incur a security risk.

If extensions are loaded into a safe interpreter, they may also restrict their own functionality to eliminate unsafe commands. For a discussion of management of extensions for safety see the manual entries for **Safe-Tcl** and the load Tcl command.

A safe interpreter may not alter the recursion limit of any interpreter, including itself.

Alias invocation

The alias mechanism has been carefully designed so that it can be used safely in an untrusted script which is being executed in a safe interpreter even if the target of the alias is not a safe interpreter. The most important thing in guaranteeing safety is to ensure that information passed from the child to the parent is never evaluated or substituted in the parent; if this were to occur, it would enable an evil script in the child to invoke arbitrary functions in the parent, which would compromise security.

When the source for an alias is invoked in the child interpreter, the usual Tcl substitutions are performed when parsing that command. These substitutions are carried out in the source interpreter just as they would be for any other command invoked in that interpreter. The command procedure for the source command takes its arguments and merges them with the *targetCmd* and *arg*s for the alias to create a new array of arguments. If the words of *srcCmd* were "*srcCmd arg1 arg2 ... argN*", the new set of words will be "*targetCmd arg arg ... arg arg1 arg2 ... argN*", where *targetCmd* and *arg*s are the values supplied when the alias was created. *TargetCmd* is then used to locate a command procedure in the target interpreter, and that command procedure is invoked with the new set of arguments. An error occurs if there is no command named *targetCmd* in the target interpreter. No additional substitutions are performed on the words: the target command procedure is invoked directly, without going through the normal Tcl evaluation mechanism. Substitutions are thus performed on each word exactly once: *targetCmd* and *args* were substituted when parsing the command that created the alias, and

arg1 - *argN* are substituted when the alias's source command is parsed in the source interpreter.

When writing the *targetCmd*s for aliases in safe interpreters, it is very important that the arguments to that command never be evaluated or substituted, since this would provide an escape mechanism whereby the child interpreter could execute arbitrary code in the parent. This in turn would compromise the security of the system.

Hidden commands

Safe interpreters greatly restrict the functionality available to Tcl programs executing within them. Allowing the untrusted Tcl program to have direct access to this functionality is unsafe, because it can be used for a variety of attacks on the environment. However, there are times when there is a legitimate need to use the dangerous functionality in the context of the safe interpreter. For example, sometimes a program must be sourced into the interpreter. Another example is Tk, where windows are bound to the hierarchy of windows for a specific interpreter; some potentially dangerous functions, e.g. window management, must be performed on these windows within the interpreter context.

The **interp** command provides a solution to this problem in the form of *hidden commands*. Instead of removing the dangerous commands entirely from a safe interpreter, these commands are hidden so they become unavailable to Tcl scripts executing in the interpreter. However, such hidden commands can be invoked by any trusted ancestor of the safe interpreter, in the context of the safe interpreter, using **interp invoke**. Hidden commands and exposed commands reside in separate name spaces. It is possible to define a hidden command and an exposed command by the same name within one interpreter.

Hidden commands in a child interpreter can be invoked in the body of procedures called in the parent during alias invocation. For example, an alias for source could be created in a child interpreter. When it is invoked in the child interpreter, a procedure is called in the parent interpreter to check that the operation is allowable (e.g. it asks to source a file that the child interpreter is allowed to access). The procedure then it invokes the hidden source command in the child interpreter to actually source in the contents of the file. Note that two commands named source exist in the child interpreter: the alias, and the hidden command.

Because a parent interpreter may invoke a hidden command as part of handling an alias invocation, great care must be taken to avoid evaluating any arguments passed in through the alias invocation. Otherwise, malicious child interpreters could cause a trusted parent interpreter to execute dangerous commands on their behalf. See the section on Alias invocation for a more complete discussion of this topic. To help avoid this problem, no substitutions or evaluations are applied to arguments of **interp invokehidden**.

Safe interpreters are not allowed to invoke hidden commands in themselves or in their descendants. This prevents them from gaining access to hidden functionality in themselves or their descendants.

The set of hidden commands in an interpreter can be manipulated by a trusted interpreter using **interp expose** and **interp hide**. The **interp expose** command moves a hidden command to the set of exposed commands in the interpreter identified by *path*, potentially renaming the command in the process. If an exposed command by the targeted name already exists, the operation fails. Similarly, **interp hide** moves an exposed command to the set of hidden commands in that interpreter. Safe interpreters are not allowed to move commands between the set of hidden and exposed commands, in either themselves or their descendants.

Currently, the names of hidden commands cannot contain namespace qualifiers, and you must first rename a command in a namespace to the global namespace before you can hide it. Commands to be hidden by **interp hide** are looked up in the global namespace even if the current namespace is not the global one. This prevents children from fooling a parent interpreter into hiding the wrong command, by making the current namespace be different from the global one.

Resource limits

Every interpreter has two kinds of resource limits that may be imposed by any parent interpreter upon its children. Command limits (of type **command**) restrict the total number of Tcl commands that may be executed by an interpreter (as can be inspected via the info cmdcount command), and time limits (of type **time**) place a limit by which execution within the interpreter must complete. Note that time limits are expressed as *absolute* times (as in clock seconds) and not relative times (as in after) because they may be modified after creation.

When a limit is exceeded for an interpreter, first any handler callbacks defined by parent interpreters are called. If those callbacks increase or remove the limit, execution within the (previously) limited interpreter continues. If the limit is still in force, an error is generated at that point and normal processing of errors within the interpreter (by the catch command) is disabled, so the error propagates outwards (building a stack-trace as it goes) to the point where the limited interpreter was invoked (e.g. by **interp eval**) where it becomes the responsibility of the calling code to catch and handle.

Limit options

Every limit has a number of options associated with it, some of which are common across all kinds of limits, and others of which are particular to the kind of limit.

-command
: This option (common for all limit types) specifies (if non-empty) a Tcl script to be executed in the global namespace of the interpreter reading and writing the option when the particular limit in the limited interpreter is exceeded. The callback may modify the limit on the interpreter if it wishes the limited interpreter to continue executing. If the callback generates an exception, it is reported through the background exception mechanism (see Background exception handling). Note that the callbacks defined by one interpreter are completely isolated from the callbacks defined by another, and that the order in which those callbacks are called is undefined.

-granularity
: This option (common for all limit types) specifies how frequently (out of the points when the Tcl interpreter is in a consistent state where limit checking is possible) that the limit is actually checked. This allows the tuning of how frequently a limit is checked, and hence how often the limit-checking overhead (which may be substantial in the case of time limits) is incurred.

-milliseconds
: This option specifies the number of milliseconds after the moment defined in the **-seconds** option that the time limit will fire. It should only ever be specified in conjunction with the **-seconds** option (whether it was set previously or is being set this invocation.)

-seconds
: This option specifies the number of seconds after the epoch (see clock seconds) that the time limit for the interpreter will be triggered. The limit will be triggered at the start of the second

unless specified at a sub-second level using the **-milliseconds** option. This option may be the empty string, which indicates that a time limit is not set for the interpreter.

-value
This option specifies the number of commands that the interpreter may execute before triggering the command limit. This option may be the empty string, which indicates that a command limit is not set for the interpreter.

Where an interpreter with a resource limit set on it creates a child interpreter, that child interpreter will have resource limits imposed on it that are at least as restrictive as the limits on the creating parent interpreter. If the parent interpreter of the limited parent wishes to relax these conditions, it should hide the **interp** command in the child and then use aliases and the **interp invokehidden** subcommand to provide such access as it chooses to the **interp** command to the limited parent as necessary.

Background exception handling

When an exception happens in a situation where it cannot be reported directly up the stack (e.g. when processing events in an update or vwait call) the exception is instead reported through the background exception handling mechanism. Every interpreter has a background exception handler registered; the default exception handler arranges for the bgerror command in the interpreter's global namespace to be called, but other exception handlers may be installed and process background exceptions in substantially different ways.

A background exception handler consists of a non-empty list of words to which will be appended two further words at invocation time. The first word will be the interpreter result at time of the exception, typically an error message, and the second will be the dictionary of return options at the time of the exception. These are the same values that catch can capture when it controls script evaluation in a non-background situation. The resulting list will then be executed in the interpreter's global namespace without further substitutions being performed.

Credits

The safe interpreter mechanism is based on the Safe-Tcl prototype implemented by Nathaniel Borenstein and Marshall Rose.

Examples

Creating and using an alias for a command in the current interpreter:

```
interp alias {} getIndex {} lsearch {alpha beta gamma delta}
set idx [getIndex delta]
```

Executing an arbitrary command in a safe interpreter where every invocation of lappend is logged:

```
set i [interp create -safe]
interp hide $i lappend
interp alias $i lappend {} loggedLappend $i
proc loggedLappend {i args} {
    puts "logged invocation of lappend $args"
    interp invokehidden $i lappend {*}$args
}
interp eval $i $someUntrustedScript
```

Setting a resource limit on an interpreter so that an infinite loop terminates.

```
set i [interp create]
interp limit $i command -value 1000
interp eval $i {
    set x 0
    while {1} {
        puts "Counting up... [incr x]"
    }
}
```

Copyright (c) 1995-1996 Sun Microsystems, Inc.
Copyright (c) 2004 Donal K. Fellows
Copyright (c) 2006-2008 Joe Mistachkin.

join

Keywords: join, element, list, separator

Name

join - Create a string by joining together list elements

Synopsis

join *list* ?*joinString*?

Description

The *list* argument must be a valid Tcl list. This command returns the string formed by joining all of the elements of *list* together with *joinString* separating each adjacent pair of elements. The *joinString* argument defaults to a space character.

Examples

Making a comma-separated list:

```
set data {1 2 3 4 5}
join $data ", "
     → 1, 2, 3, 4, 5
```

Using **join** to flatten a list by a single level:

```
set data {1 {2 3} 4 {5 {6 7} 8}}
join $data
     → 1 2 3 4 5 {6 7} 8
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

lappend

Keywords: lappend, append, element, list, variable

Name

lappend - Append list elements onto a variable

Synopsis

lappend *varName* ?*value value value* ...?

Description

This command treats the variable given by *varName* as a list and appends each of the *value* arguments to that list as a separate element, with spaces between elements. If *varName* does not exist, it is created as a list with elements given by the *value* arguments. **Lappend** is similar to append except that the *value*s are appended as list elements rather than raw text. This command provides a relatively efficient way to build up large lists. For example, "**lappend a $b**" is much more efficient than "**set a [concat $a [list $b]]**" when **$a** is long.

Example

Using **lappend** to build up a list of numbers.

```
% set var 1
1
% lappend var 2
1 2
% lappend var 3 4 5
1 2 3 4 5
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org*. All rights reserved.*

lassign

Keywords: lassign, assign, element, list, multiple, set, variable

Name

lassign - Assign list elements to variables

Synopsis

lassign *list* ?*varName* ...?

Description

This command treats the value *list* as a list and assigns successive elements from that list to the variables given by the *varName* arguments in order. If there are more variable names than list elements, the remaining variables are set to the empty string. If there are more list elements than variables, a list of unassigned elements is returned.

Examples

An illustration of how multiple assignment works, and what happens when there are either too few or too many elements.

```
lassign {a b c} x y z       ;# Empty return
puts $x                     ;# Prints "a"
puts $y                     ;# Prints "b"
puts $z                     ;# Prints "c"

lassign {d e} x y z         ;# Empty return
puts $x                     ;# Prints "d"
puts $y                     ;# Prints "e"
puts $z                     ;# Prints ""

lassign {f g h i} x y       ;# Returns "h i"
puts $x                     ;# Prints "f"
puts $y                     ;# Prints "g"
```

The **lassign** command has other uses. It can be used to create the analogue of the "shift" command in many shell languages like this:

```
set ::argv [lassign $::argv argumentToReadOff]
```

Copyright (c) 1992-1999 Karl Lehenbauer & Mark Diekhans
Copyright (c) 2004 Donal K. Fellows

library

Keywords: library, auto-exec, auto-load, unknown, word, whitespace

Name

auto_execok, auto_import, auto_load, auto_mkindex, auto_qualify, auto_reset, parray, tcl_findLibrary, tcl_endOfWord, tcl_nonwordchars, tcl_startOfNextWord, tcl_startOfPreviousWord, tcl_wordBreakAfter, tcl_wordBreakBefore, tcl_wordchars - standard library of Tcl procedures

Synopsis

auto_execok *cmd*
auto_import *pattern*
auto_load *cmd*
auto_mkindex *dir pattern pattern ...*
auto_qualify *command namespace*
auto_reset
tcl_findLibrary *basename version patch initScript enVarName varName*
parray *arrayName* ?*pattern*?

tcl_endOfWord *str start*
tcl_startOfNextWord *str start*
tcl_startOfPreviousWord *str start*
tcl_wordBreakAfter *str start*
tcl_wordBreakBefore *str start*

Introduction

Tcl includes a library of Tcl procedures for commonly-needed functions. The procedures defined in the Tcl library are generic ones suitable for use by many different applications. The location of the Tcl library is returned by the info library command. In addition to the Tcl library, each application will normally have its own library of support procedures as well; the location of this library is normally given by the value of the **$*app*_library** global variable, where *app* is the name of the application. For example, the location of the Tk library is kept in the variable tk_library.

To access the procedures in the Tcl library, an application should source the file **init.tcl** in the library, for example with the Tcl command

```
source [file join [info library] init.tcl]
```

If the library procedure Tcl_Init is invoked from an application's Tcl_AppInit procedure, this happens automatically. The code in **init.tcl** will define the unknown procedure and arrange for the other procedures to be loaded on-demand using the auto-load mechanism defined below.

Command procedures

The following procedures are provided in the Tcl library:

auto_execok *cmd*

Determines whether there is an executable file or shell builtin by the name *cmd*. If so, it returns a list of arguments to be passed to exec to execute the executable file or shell builtin named by *cmd*. If not, it returns an empty string. This command examines the directories in the current search path (given by the PATH environment variable) in its search for an executable file named *cmd*. On Windows platforms, the search is expanded with the same directories and file extensions as used by exec. **Auto_execok** remembers information about previous searches in an array named **auto_execs**; this avoids the path search in future calls for the same *cmd*. The command **auto_reset** may be used to force **auto_execok** to forget its cached information.

For example, to run the *umask* shell builtin on Linux, you would do:

```
exec {*}[auto_execok umask]
```

To run the *DIR* shell builtin on Windows, you would do:

```
exec {*}[auto_execok dir]
```

To discover if there is a *frobnicate* binary on the user's PATH, you would do:

```
set mayFrob [expr {[llength [auto_execok frobnicate]] > 0}]
```

auto_import *pattern*

Auto_import is invoked during namespace import to see if the imported commands specified by *pattern* reside in an autoloaded library. If so, the commands are loaded so that they will be available to the interpreter for creating the import links. If the commands do not reside in an autoloaded library, **auto_import** does nothing. The pattern matching is performed according to the matching rules of namespace import.

It is not normally necessary to call this command directly.

auto_load *cmd*

This command attempts to load the definition for a Tcl command named *cmd*. To do this, it searches an *auto-load path*, which is a list of one or more directories. The auto-load path is given by the global variable auto_path if it exists. If there is no auto_path variable, then the **TCLLIBPATH** environment variable is used, if it exists. Otherwise the auto-load path consists of just the Tcl library directory. Within each directory in the auto-load path there must be a file **tclIndex** that describes one or more commands defined in that directory and a script to evaluate to load each of the commands. The **tclIndex** file should be generated with the **auto_mkindex** command. If *cmd* is found in an index file, then the appropriate script is evaluated to create the command. The **auto_load** command returns 1 if *cmd* was successfully created. The command returns 0 if there was no index entry for *cmd* or if the script did not actually define *cmd* (e.g. because index information is out of date). If an error occurs while processing the script, then that error is returned. **Auto_load** only reads the index information once and saves it in the array **auto_index**; future calls to **auto_load** check for *cmd* in the array rather than re-reading the index files. The cached index information may be deleted with the command **auto_reset**. This will force the next **auto_load** command to reload the index database from disk.

It is not normally necessary to call this command directly; the default unknown handler will do so.

auto_mkindex *dir pattern pattern ...*

Generates an index suitable for use by **auto_load**. The command searches *dir* for all files whose names match any of the *pattern* arguments (matching is done with the glob command), generates an index of all the Tcl command procedures defined in all the matching files, and stores the index information in a file named **tclIndex** in *dir*. If no pattern is given a pattern of ***.tcl** will be assumed. For example, the command

```
auto_mkindex foo *.tcl
```

will read all the **.tcl** files in subdirectory **foo** and generate a new index file **foo/tclIndex**.

Auto_mkindex parses the Tcl scripts by sourcing them into a child interpreter and monitoring the proc and namespace commands that are executed. Extensions can use the (undocumented) auto_mkindex_parser package to register other commands that can contribute to the auto_load index. You will have to read through auto.tcl to see how this works.

Auto_mkindex_old (which has the same syntax as **auto_mkindex**) parses the Tcl scripts in a relatively unsophisticated way: if any line contains the word "proc" as its first characters then it is assumed to be a procedure definition and the next word of the line is taken as the procedure's name. Procedure definitions that do not appear in this way (e.g. they have spaces

before the proc) will not be indexed. If your script contains "dangerous" code, such as global initialization code or procedure names with special characters like **$**, *, **[** or **]**, you are safer using **auto_mkindex_old**.

auto_reset
Destroys all the information cached by **auto_execok** and **auto_load**. This information will be re-read from disk the next time it is needed. **Auto_reset** also deletes any procedures listed in the auto-load index, so that fresh copies of them will be loaded the next time that they are used.

auto_qualify *command namespace*
Computes a list of fully qualified names for *command*. This list mirrors the path a standard Tcl interpreter follows for command lookups: first it looks for the command in the current namespace, and then in the global namespace. Accordingly, if *command* is relative and *namespace* is not **::**, the list returned has two elements: *command* scoped by *namespace*, as if it were a command in the *namespace* namespace; and *command* as if it were a command in the global namespace. Otherwise, if either *command* is absolute (it begins with **::**), or *namespace* is **::**, the list contains only *command* as if it were a command in the global namespace.

Auto_qualify is used by the auto-loading facilities in Tcl, both for producing auto-loading indexes such as *pkgIndex.tcl*, and for performing the actual auto-loading of functions at runtime.

tcl_findLibrary *basename version patch initScript enVarName varName*
This is a standard search procedure for use by extensions during their initialization. They call this procedure to look for their script library in several standard directories. The last component of the name of the library directory is normally *basenameversion* (e.g., tk8.0), but it might be "library" when in the build hierarchies. The *patch* argument is not used. The *initScript* file will be sourced into the interpreter once it is found. The directory in which this file is found is stored into the global variable *varName*. If this variable is already defined (e.g., by C code during application initialization) then no searching is done. Otherwise the search looks in these directories: the directory named by the environment variable *enVarName*; relative to the Tcl library directory; relative to the executable file in the standard installation bin or bin/*arch* directory; relative to the executable file in the current build tree; relative to the executable file in a parallel build tree.

parray *arrayName* ?*pattern*?
Prints on standard output the names and values of all the elements in the array *arrayName*, or just the names that match *pattern* (using the matching rules of string match) and their values if *pattern* is given. *ArrayName* must be an array accessible to the caller of **parray**. It may be either local or global.

For example, to print the contents of the tcl_platform array, do:

```
parray tcl_platform
```

Word boundary helpers

These procedures are mainly used internally by Tk.

tcl_endOfWord *str start*

Returns the index of the first end-of-word location that occurs after a starting index *start* in the string *str*. An end-of-word location is defined to be the first non-word character following the first word character after the starting point. Returns -1 if there are no more end-of-word locations after the starting point. See the description of **tcl_wordchars** and **tcl_nonwordchars** below for more details on how Tcl determines which characters are word characters.

tcl_startOfNextWord *str start*

Returns the index of the first start-of-word location that occurs after a starting index *start* in the string *str*. A start-of-word location is defined to be the first word character following a non-word character. Returns -1 if there are no more start-of-word locations after the starting point.

For example, to print the indices of the starts of each word in a string according to platform rules:

```
set theString "The quick brown fox"
for {set idx 0} {$idx >= 0} {
        set idx [tcl_startOfNextWord $theString $idx]} {
    puts "Word start index: $idx"
}
```

tcl_startOfPreviousWord *str start*

Returns the index of the first start-of-word location that occurs before a starting index *start* in the string *str*. Returns -1 if there are no more start-of-word locations before the starting point.

tcl_wordBreakAfter *str start*

Returns the index of the first word boundary after the starting index *start* in the string *str*. Returns -1 if there are no more boundaries after the starting point in the given string. The index returned refers to the second character of the pair that comprises a boundary.

tcl_wordBreakBefore *str start*

Returns the index of the first word boundary before the starting index *start* in the string *str*. Returns -1 if there are no more boundaries before the starting point in the given string. The index returned refers to the second character of the pair that comprises a boundary.

Variables

The following global variables are defined or used by the procedures in the Tcl library. They fall into two broad classes, handling unknown commands and packages, and determining what are words.

Autoloading and package management variables

auto_execs

Used by **auto_execok** to record information about whether particular commands exist as executable files.

Not normally usefully accessed directly by user code.

auto_index

Used by **auto_load** to save the index information read from disk.

Not normally usefully accessed directly by user code.

auto_noexec
If set to any value, then unknown will not attempt to auto-exec any commands.

auto_noload
If set to any value, then unknown will not attempt to auto-load any commands.

auto_path
If set, then it must contain a valid Tcl list giving directories to search during auto-load operations (including for package index files when using the default package unknown handler). This variable is initialized during startup to contain, in order: the directories listed in the **TCLLIBPATH** environment variable, the directory named by the tcl_library global variable, the parent directory of tcl_library, the directories listed in the tcl_pkgPath variable. Additional locations to look for files and package indices should normally be added to this variable using lappend.

For example, to add the *lib* directory next to the running script, you would do:

```
lappend auto_path [file dirname [info script]]/lib
```

Note that if the script uses cd, it is advisable to ensure that entries on the auto_path are file normalized.

env(TCL_LIBRARY)
If set, then it specifies the location of the directory containing library scripts (the value of this variable will be assigned to the tcl_library variable and therefore returned by the command info library). If this variable is not set then a default value is used.

Use of this environment variable is not recommended outside of testing. Tcl installations should already know where to find their own script files, as the value is baked in during the build or installation.

env(TCLLIBPATH)
If set, then it must contain a valid Tcl list giving directories to search during auto-load operations. Directories must be specified in Tcl format, using "/" as the path separator, regardless of platform. This variable is only used when initializing the auto_path variable.

A key consequence of this variable is that it gives a way to let the user of a script specify the list of places where that script may use package require to read packages from. It is not normally usefully settable within a Tcl script itself *except* to influence where other interpreters load from (whether made with interp create or launched as their own threads or subprocesses).

Word boundary determination variables

These variables are only used in the **tcl_endOfWord**, **tcl_startOfNextWord**, **tcl_startOfPreviousWord**, **tcl_wordBreakAfter**, and **tcl_wordBreakBefore** commands.

tcl_nonwordchars
This variable contains a regular expression that is used by routines like **tcl_endOfWord** to identify whether a character is part of a word or not. If the pattern matches a character, the character is considered to be a non-word character. On Windows platforms, spaces, tabs, and

newlines are considered non-word characters. Under Unix, everything but numbers, letters and underscores are considered non-word characters.

tcl_wordchars
This variable contains a regular expression that is used by routines like **tcl_endOfWord** to identify whether a character is part of a word or not. If the pattern matches a character, the character is considered to be a word character. On Windows platforms, words are comprised of any character that is not a space, tab, or newline. Under Unix, words are comprised of numbers, letters or underscores.

Copyright (c) 1991-1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

lindex

Keywords: lindex, element, index, list

Name

lindex - Retrieve an element from a list

Synopsis

lindex *list* ?*index* ...?

Description

The **lindex** command accepts a parameter, *list*, which it treats as a Tcl list. It also accepts zero or more *indices* into the list. The indices may be presented either consecutively on the command line, or grouped in a Tcl list and presented as a single argument.

If no indices are presented, the command takes the form:

```
lindex list
```

or

```
lindex list {}
```

In this case, the return value of **lindex** is simply the value of the *list* parameter.

When presented with a single index, the **lindex** command treats *list* as a Tcl list and returns the *index*'th element from it (0 refers to the first element of the list). In extracting the element, **lindex** observes the same rules concerning braces and quotes and backslashes as the Tcl command interpreter; however, variable substitution and command substitution do not occur. If *index* is negative or greater than or equal to the number of elements in *value*, then an empty string is returned. The interpretation of each simple *index* value is the same as for the command string index, supporting simple index arithmetic and indices relative to the end of the list.

If additional *index* arguments are supplied, then each argument is used in turn to select an element from the previous indexing operation, allowing the script to select elements from sublists. The command,

```
lindex $a 1 2 3
```

or

```
lindex $a {1 2 3}
```

is synonymous with

```
lindex [lindex [lindex $a 1] 2] 3
```

Examples

Lists can be indexed into from either end:

```
lindex {a b c} 0
      → a
lindex {a b c} 2
      → c
lindex {a b c} end
      → c
lindex {a b c} end-1
      → b
```

Lists or sequences of indices allow selection into lists of lists:

```
lindex {a b c}
      → a b c
lindex {a b c} {}
      → a b c
lindex {{a b c} {d e f} {g h i}} 2 1
      → h
lindex {{a b c} {d e f} {g h i}} {2 1}
      → h
lindex {{{a b} {c d}} {{e f} {g h}}} 1 1 0
      → g
lindex {{{a b} {c d}} {{e f} {g h}}} {1 1 0}
      → g
```

List indices may also perform limited computation, adding or subtracting fixed amounts from other indices:

```
set idx 1
lindex {a b c d e f} $idx+2
      → d
set idx 3
lindex {a b c d e f} $idx+2
      → f
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org*. All rights reserved.*

linsert

Keywords: linsert, element, insert, list

Name
linsert - Insert elements into a list

Synopsis
linsert *list index* ?*element element* ...?

Description
This command produces a new list from *list* by inserting all of the *element* arguments just before the *index*'th element of *list*. Each *element* argument will become a separate element of the new list. If *index* is less than or equal to zero, then the new elements are inserted at the beginning of the list, and if *index* is greater or equal to the length of *list*, it is as if it was **end**. As with string index, the *index* value supports both simple index arithmetic and end-relative indexing.

Subject to the restrictions that indices must refer to locations inside the list and that the *element* s will always be inserted in order, insertions are done so that when *index* is start-relative, the first *element* will be at that index in the resulting list, and when *index* is end-relative, the last *element* will be at that index in the resulting list.

Example
Putting some values into a list, first indexing from the start and then indexing from the end, and then chaining them together:

```
set oldList {the fox jumps over the dog}
set midList [linsert $oldList 1 quick]
set newList [linsert $midList end-1 lazy]
# The old lists still exist though...
set newerList [linsert [linsert $oldList end-1 quick] 1 lazy]
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org*. All rights reserved.*

list
Keywords: list, element, quoting

Name
list - Create a list

Synopsis
list ?*arg arg* ...?

Description
This command returns a list comprised of all the *arg*s, or an empty string if no *arg*s are specified. Braces and backslashes get added as necessary, so that the lindex command may be used on the result to re-extract the original arguments, and also so that eval may be used to execute the resulting list, with *arg1* comprising the command's name and the other *arg*s comprising its arguments. **List** produces slightly different results than concat: concat removes one level of grouping before forming the list, while **list** works directly from the original arguments.

Example
The command

```
list a b "c d e  " "  f {g h}"
```

will return

```
a b {c d e  } {  f {g h}}
```

while concat with the same arguments will return

```
a b c d e f {g h}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org. All rights reserved.

llength
Keywords: llength, element, list, length

Name
llength - Count the number of elements in a list

Synopsis
llength *list*

Description
Treats *list* as a list and returns a decimal string giving the number of elements in it.

Examples
The result is the number of elements:

```
% llength {a b c d e}
5
% llength {a b c}
3
% llength {}
0
```

Elements are not guaranteed to be exactly words in a dictionary sense of course, especially when quoting is used:

```
% llength {a b {c d} e}
4
% llength {a b { } c d e}
6
```

An empty list is not necessarily an empty string:

```
% set var { }; puts "[string length $var],[llength $var]"
1,0
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org*. All rights reserved.*

lmap

Keywords: lmap, foreach, iteration, list, loop, map

Name

lmap - Iterate over all elements in one or more lists and collect results

Synopsis

lmap *varname list body*
lmap *varlist1 list1* ?*varlist2 list2* ...? *body*

Description

The **lmap** command implements a loop where the loop variable(s) take on values from one or more lists, and the loop returns a list of results collected from each iteration.

In the simplest case there is one loop variable, *varname*, and one list, *list*, that is a list of values to assign to *varname*. The *body* argument is a Tcl script. For each element of *list* (in order from first to last), **lmap** assigns the contents of the element to *varname* as if the lindex command had been used to extract the element, then calls the Tcl interpreter to execute *body*. If execution of the body completes normally then the result of the body is appended to an accumulator list. **lmap** returns the accumulator list.

In the general case there can be more than one value list (e.g., *list1* and *list2*), and each value list can be associated with a list of loop variables (e.g., *varlist1* and *varlist2*). During each iteration of the loop the variables of each *varlist* are assigned consecutive values from the corresponding *list*. Values in each *list* are used in order from first to last, and each value is used exactly once. The total number of loop iterations is large enough to use up all the values from all the value lists. If a value list does not contain enough elements for each of its loop variables in each iteration, empty values are used for the missing elements.

The break and continue statements may be invoked inside *body*, with the same effect as in the for and foreach commands. In these cases the body does not complete normally and the result is not appended to the accumulator list.

Examples

Zip lists together:

```
set list1 {a b c d}
set list2 {1 2 3 4}
set zipped [lmap a $list1 b $list2 {list $a $b}]
# The value of zipped is "{a 1} {b 2} {c 3} {d 4}"
```

Filter a list to remove odd values:

```
set values {1 2 3 4 5 6 7 8}
proc isEven {n} {expr {($n % 2) == 0}}
set goodOnes [lmap x $values {expr {
    [isEven $x] ? $x : [continue]
}}]
# The value of goodOnes is "2 4 6 8"
```

Take a prefix from a list based on the contents of the list:

```
set values {8 7 6 5 4 3 2 1}
proc isGood {counter} {expr {$n > 3}}
set prefix [lmap x $values {expr {
    [isGood $x] ? $x : [break]
}}]
# The value of prefix is "8 7 6 5 4"
```

Copyright (c) 2012 Trevor Davel

load

Keywords: load, binary code, dynamic library, safe interpreter, shared library

Name

load - Load machine code and initialize new commands

Synopsis

load ?**-global**? ?**-lazy**? ?**--**? *fileName*
load ?**-global**? ?**-lazy**? ?**--**? *fileName prefix*
load ?**-global**? ?**-lazy**? ?**--**? *fileName prefix interp*

Description

This command loads binary code from a file into the application's address space and calls an initialization procedure in the library to incorporate it into an interpreter. *fileName* is the name of the file containing the code; its exact form varies from system to system but on most systems it is a shared library, such as a **.so** file under Solaris or a DLL under Windows. *prefix* is used to compute the name of an initialization procedure. *interp* is the path name of the interpreter into which to load the library (see the interp manual entry for details); if *interp* is omitted, it defaults to the interpreter in which the **load** command was invoked.

Once the file has been loaded into the application's address space, one of two initialization procedures will be invoked in the new code. Typically the initialization procedure will add new commands to a Tcl interpreter. The name of the initialization procedure is determined by *prefix* and whether or not the target interpreter is a safe one. For normal interpreters the name of the initialization procedure will have the form *pfx*_**Init**, where *pfx* is the same as *prefix* except that the first letter is converted to upper case and all other letters are converted to lower case. For example, if *prefix* is **foo** or **FOo**, the initialization procedure's name will be **Foo_Init**.

If the target interpreter is a safe interpreter, then the name of the initialization procedure will be *pfx*_**SafeInit** instead of *pfx*_**Init**. The *pfx*_**SafeInit** function should be written carefully, so that it initializes the safe interpreter only with partial functionality provided by the library that is safe

for use by untrusted code. For more information on Safe-Tcl, see the safe manual entry.

The initialization procedure must match the following prototype:

```
typedef int Tcl_PackageInitProc(
        Tcl_Interp *interp);
```

The *interp* argument identifies the interpreter in which the library is to be loaded. The initialization procedure must return TCL_OK or TCL_ERROR to indicate whether or not it completed successfully; in the event of an error it should set the interpreter's result to point to an error message. The result of the **load** command will be the result returned by the initialization procedure.

The actual loading of a file will only be done once for each *fileName* in an application. If a given *fileName* is loaded into multiple interpreters, then the first **load** will load the code and call the initialization procedure; subsequent **load**s will call the initialization procedure without loading the code again. For Tcl versions lower than 8.5, it is not possible to unload or reload a library. From version 8.5 however, the unload command allows the unloading of libraries loaded with **load**, for libraries that are aware of the Tcl's unloading mechanism.

The **load** command also supports libraries that are statically linked with the application, if those libraries have been registered by calling the Tcl_StaticPackage procedure. If *fileName* is an empty string, then *prefix* must be specified.

If *prefix* is omitted or specified as an empty string, Tcl tries to guess the prefix. This may be done differently on different platforms. The default guess, which is used on most UNIX platforms, is to take the last element of *fileName*, strip off the first three characters if they are **lib**, and use any following alphabetic and underline characters, converted to titlecase as the prefix. For example, the command **load libxyz4.2.so** uses the prefix **Xyz** and the command **load bin/last.so {}** uses the prefix **Last**.

If *fileName* is an empty string, then *prefix* must be specified. The **load** command first searches for a statically loaded library (one that has been registered by calling the Tcl_StaticPackage procedure) by that name; if one is found, it is used. Otherwise, the **load** command searches for a dynamically loaded library by that name, and uses it if it is found. If several different files have been **load**ed with different versions of the library, Tcl picks the file that was loaded first.

If **-global** is specified preceding the filename, all symbols found in the shared library are exported for global use by other libraries. The option **-lazy** delays the actual loading of symbols until their first actual use. The options may be abbreviated. The option **--** indicates the end of the options, and should be used if you wish to use a filename which starts with **-** and you provide a prefix to the **load** command.

On platforms which do not support the **-global** or **-lazy** options, the options still exist but have no effect. Note that use of the **-global** or **-lazy** option may lead to crashes in your application later (in case of symbol conflicts resp. missing symbols), which cannot be detected during the **load**. So, only use this when you know what you are doing, you will not get a nice error message when something is wrong with the loaded library.

Portability issues

Windows

When a load fails with "library not found" error, it is also possible that a dependent library was

not found. To see the dependent libraries, type "dumpbin -imports " in a DOS console to see what the library must import. When loading a DLL in the current directory, Windows will ignore "./" as a path specifier and use a search heuristic to find the DLL instead. To avoid this, load the DLL with:

```
load [file join [pwd] mylib.DLL]
```

Bugs

If the same file is **load**ed by different *fileName*s, it will be loaded into the process's address space multiple times. The behavior of this varies from system to system (some systems may detect the redundant loads, others may not).

Example

The following is a minimal extension:

```
#include <tcl.h>
#include <stdio.h>
static int fooCmd(void *clientData,
        Tcl_Interp *interp, int objc, Tcl_Obj *const objv[]) {
    printf("called with %d arguments\n", objc);
    return TCL_OK;
}
int Foo_Init(Tcl_Interp *interp) {
    if (Tcl_InitStubs(interp, "8.1", 0) == NULL) {
    return TCL_ERROR;
    }
    printf("creating foo command");
    Tcl_CreateObjCommand(interp, "foo", fooCmd, NULL, NULL);
    return TCL_OK;
}
```

When built into a shared/dynamic library with a suitable name (e.g. **foo.dll** on Windows, **libfoo.so** on Solaris and Linux) it can then be loaded into Tcl with the following:

```
# Load the extension
switch $tcl_platform(platform) {
    windows {
        load [file join [pwd] foo.dll]
    }
    unix {
        load [file join [pwd] libfoo[info sharedlibextension]]
    }
}

# Now execute the command defined by the extension
foo
```

Copyright (c) 1995-1996 Sun Microsystems, Inc.

lrange

Keywords: lrange, element, list, range, sublist

Name

lrange - Return one or more adjacent elements from a list

Synopsis

lrange *list first last*

Description

List must be a valid Tcl list. This command will return a new list consisting of elements *first* through *last*, inclusive. The index values *first* and *last* are interpreted the same as index values for the command string index, supporting simple index arithmetic and indices relative to the end of the list. If *first* is less than zero, it is treated as if it were zero. If *last* is greater than or equal to the number of elements in the list, then it is treated as if it were **end**. If *first* is greater than *last* then an empty string is returned. Note: "**lrange** *list first first*" does not always produce the same result as "lindex *list first*" (although it often does for simple fields that are not enclosed in braces); it does, however, produce exactly the same results as "list [lindex *list first*]"

Examples

Selecting the first two elements:

```
% lrange {a b c d e} 0 1
a b
```

Selecting the last three elements:

```
% lrange {a b c d e} end-2 end
c d e
```

Selecting everything except the first and last element:

```
% lrange {a b c d e} 1 end-1
b c d
```

Selecting a single element with **lrange** is not the same as doing so with lindex:

```
% set var {some {elements to} select}
some {elements to} select
% lindex $var 1
elements to
% lrange $var 1 1
{elements to}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org. All rights reserved.

lrepeat

Keywords: lrepeat, element, index, list

Name
lrepeat - Build a list by repeating elements

Synopsis
lrepeat *count* ?*element* ...?

Description
The **lrepeat** command creates a list of size *count* * *number of elements* by repeating *count* times the sequence of elements *element ...*. *count* must be a non-negative integer, *element* can be any Tcl value. Note that **lrepeat 1 element ...** is identical to **list element ...**.

Examples
```
lrepeat 3 a
      → a a a
lrepeat 3 [lrepeat 3 0]
      → {0 0 0} {0 0 0} {0 0 0}
lrepeat 3 a b c
      → a b c a b c a b c
lrepeat 3 [lrepeat 2 a] b c
      → {a a} b c {a a} b c {a a} b c
```

Copyright (c) 2003 Simon Geard. All rights reserved.

lreplace
Keywords: lreplace, element, list, replace

Name
lreplace - Replace elements in a list with new elements

Synopsis
lreplace *list first last* ?*element element* ...?

Description
lreplace returns a new list formed by replacing zero or more elements of *list* with the *element* arguments. *first* and *last* are index values specifying the first and last elements of the range to replace. The index values *first* and *last* are interpreted the same as index values for the command string index, supporting simple index arithmetic and indices relative to the end of the list. 0 refers to the first element of the list, and **end** refers to the last element of the list.

If either *first* or *last* is less than zero, it is considered to refer to before the first element of the list. This allows **lreplace** to prepend elements to *list*. If either first or last indicates a position greater than the index of the last element of the list, it is treated as if it is an index one greater than the last element. This allows lreplace to append elements to list.

If *last* is less than *first*, then any specified elements will be inserted into the list before the element specified by *first* with no elements being deleted.

The *element* arguments specify zero or more new elements to be added to the list in place of those that were deleted. Each *element* argument will become a separate element of the list. If

no *element* arguments are specified, then the elements between *first* and *last* are simply deleted.

Examples

Replacing an element of a list with another:

```
% lreplace {a b c d e} 1 1 foo
a foo c d e
```

Replacing two elements of a list with three:

```
% lreplace {a b c d e} 1 2 three more elements
a three more elements d e
```

Deleting the last element from a list in a variable:

```
% set var {a b c d e}
a b c d e
% set var [lreplace $var end end]
a b c d
```

A procedure to delete a given element from a list:

```
proc lremove {listVariable value} {
    upvar 1 $listVariable var
    set idx [lsearch -exact $var $value]
    set var [lreplace $var $idx $idx]
}
```

Appending elements to the list; note that end+2 will initially be treated as if it is 6 here, but both that and 12345 are greater than the index of the final item so they behave identically:

```
% set var {a b c d e}
a b c d e
% set var [lreplace $var 12345 end+2 f g h i]
a b c d e f g h i
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org*. All rights reserved.*

lreverse

Keywords: lreverse, element, list, reverse

Name

lreverse - Reverse the order of a list

Synopsis

lreverse *list*

Description

The **lreverse** command returns a list that has the same elements as its input list, *list*, except with the elements in the reverse order.

Examples

```
lreverse {a a b c}
      → c b a a
lreverse {a b {c d} e f}
      → f e {c d} b a
```

Copyright (c) 2006 Donal K. Fellows. All rights reserved.

lsearch

Keywords: lsearch, binary search, linear search, list, match, pattern, regular expression, search, string

Name

lsearch - See if a list contains a particular element

Synopsis

lsearch ?*options*? *list pattern*

Description

This command searches the elements of *list* to see if one of them matches *pattern*. If so, the command returns the index of the first matching element (unless the options **-all** or **-inline** are specified.) If not, the command returns **-1** or (if options **-all** or **-inline** are specified) the empty string. The *option* arguments indicates how the elements of the list are to be matched against *pattern* and must have one of the values below:

Matching style options

If all matching style options are omitted, the default matching style is **-glob**. If more than one matching style is specified, the last matching style given takes precedence.

-exact
: *Pattern* is a literal string that is compared for exact equality against each list element.

-glob
: *Pattern* is a glob-style pattern which is matched against each list element using the same rules as the string match command.

-regexp
: *Pattern* is treated as a regular expression and matched against each list element using the rules described in the re_syntax reference page.

-sorted
: The list elements are in sorted order. If this option is specified, **lsearch** will use a more efficient searching algorithm to search *list*. If no other options are specified, *list* is assumed to be sorted in increasing order, and to contain ASCII strings. This option is mutually exclusive with **-glob** and **-regexp**, and is treated exactly like **-exact** when either **-all** or **-not** are specified.

General modifier options

These options may be given with all matching styles.

-all
Changes the result to be the list of all matching indices (or all matching values if **-inline** is specified as well.) If indices are returned, the indices will be in ascending numeric order. If values are returned, the order of the values will be the order of those values within the input *list*.

-inline
The matching value is returned instead of its index (or an empty string if no value matches.) If **-all** is also specified, then the result of the command is the list of all values that matched.

-not
This negates the sense of the match, returning the index of the first non-matching value in the list.

-start *index*
The list is searched starting at position *index*. The interpretation of the *index* value is the same as for the command string index, supporting simple index arithmetic and indices relative to the end of the list.

Contents description options

These options describe how to interpret the items in the list being searched. They are only meaningful when used with the **-exact** and **-sorted** options. If more than one is specified, the last one takes precedence. The default is **-ascii**.

-ascii
The list elements are to be examined as Unicode strings (the name is for backward-compatibility reasons.)

-dictionary
The list elements are to be compared using dictionary-style comparisons (see lsort for a fuller description). Note that this only makes a meaningful difference from the **-ascii** option when the **-sorted** option is given, because values are only dictionary-equal when exactly equal.

-integer
The list elements are to be compared as integers.

-nocase
Causes comparisons to be handled in a case-insensitive manner. Has no effect if combined with the **-dictionary**, **-integer**, or **-real** options.

-real
The list elements are to be compared as floating-point values.

Sorted list options

These options (only meaningful with the **-sorted** option) specify how the list is sorted. If more than one is given, the last one takes precedence. The default option is **-increasing**.

-decreasing
The list elements are sorted in decreasing order. This option is only meaningful when used with **-sorted**.

-increasing
The list elements are sorted in increasing order. This option is only meaningful when used

with **-sorted**.

-bisect
Inexact search when the list elements are in sorted order. For an increasing list the last index where the element is less than or equal to the pattern is returned. For a decreasing list the last index where the element is greater than or equal to the pattern is returned. If the pattern is before the first element or the list is empty, -1 is returned. This option implies -sorted and cannot be used with either -all or -not.

Nested list options

These options are used to search lists of lists. They may be used with any other options.

-index *indexList*
This option is designed for use when searching within nested lists. The *indexList* argument gives a path of indices (much as might be used with the lindex or lset commands) within each element to allow the location of the term being matched against.

-subindices
If this option is given, the index result from this command (or every index result when **-all** is also specified) will be a complete path (suitable for use with lindex or lset) within the overall list to the term found. This option has no effect unless the **-index** is also specified, and is just a convenience short-cut.

Examples

Basic searching:

```
lsearch {a b c d e} c
      → 2
lsearch -all {a b c a b c} c
      → 2 5
```

Using **lsearch** to filter lists:

```
lsearch -inline {a20 b35 c47} b*
      → b35
lsearch -inline -not {a20 b35 c47} b*
      → a20
lsearch -all -inline -not {a20 b35 c47} b*
      → a20 c47
lsearch -all -not {a20 b35 c47} b*
      → 0 2
```

This can even do a "set-like" removal operation:

```
lsearch -all -inline -not -exact {a b c a d e a f g a} a
      → b c d e f g
```

Searching may start part-way through the list:

```
lsearch -start 3 {a b c a b c} c
      → 5
```

It is also possible to search inside elements:

```
lsearch -index 1 -all -inline {{abc abc} {abc bcd} {abc cde}} *bc*
      → {abc abc} {abc bcd}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org*. All rights reserved.*
Copyright (c) 2003-2004 Donal K. Fellows.

lset

Keywords: lset, element, index, list, replace, set

Name

lset - Change an element in a list

Synopsis

lset *varName* ?*index* ...? newValue

Description

The **lset** command accepts a parameter, *varName*, which it interprets as the name of a variable containing a Tcl list. It also accepts zero or more *indices* into the list. The indices may be presented either consecutively on the command line, or grouped in a Tcl list and presented as a single argument. Finally, it accepts a new value for an element of *varName*.

If no indices are presented, the command takes the form:

```
lset varName newValue
```

or

```
lset varName {} newValue
```

In this case, *newValue* replaces the old value of the variable *varName*.

When presented with a single index, the **lset** command treats the content of the *varName* variable as a Tcl list. It addresses the *index*'th element in it (0 refers to the first element of the list). When interpreting the list, **lset** observes the same rules concerning braces and quotes and backslashes as the Tcl command interpreter; however, variable substitution and command substitution do not occur. The command constructs a new list in which the designated element is replaced with *newValue*. This new list is stored in the variable *varName*, and is also the return value from the **lset** command.

If *index* is negative or greater than the number of elements in *$varName*, then an error occurs.

If *index* is equal to the number of elements in *$varName*, then the given element is appended to the list.

The interpretation of each simple *index* value is the same as for the command string index, supporting simple index arithmetic and indices relative to the end of the list.

If additional *index* arguments are supplied, then each argument is used in turn to address an element within a sublist designated by the previous indexing operation, allowing the script to alter elements in sublists (or append elements to sublists). The command,

```
lset a 1 2 newValue
```

or

```
lset a {1 2} newValue
```

replaces element 2 of sublist 1 with *newValue*.

The integer appearing in each *index* argument must be greater than or equal to zero. The integer appearing in each *index* argument must be less than or equal to the length of the corresponding list. In other words, the **lset** command can change the size of a list only by appending an element (setting the one after the current end). If an index is outside the permitted range, an error is reported.

Examples

In each of these examples, the initial value of *x* is:

```
set x [list [list a b c] [list d e f] [list g h i]]
      → {a b c} {d e f} {g h i}
```

The indicated return value also becomes the new value of *x* (except in the last case, which is an error which leaves the value of *x* unchanged.)

```
lset x {j k l}
      → j k l
lset x {} {j k l}
      → j k l
lset x 0 j
      → j {d e f} {g h i}
lset x 2 j
      → {a b c} {d e f} j
lset x end j
      → {a b c} {d e f} j
lset x end-1 j
      → {a b c} j {g h i}
lset x 2 1 j
      → {a b c} {d e f} {g j i}
lset x {2 1} j
      → {a b c} {d e f} {g j i}
lset x {2 3} j
      → list index out of range
```

In the following examples, the initial value of *x* is:

```
set x [list [list [list a b] [list c d]] \
            [list [list e f] [list g h]]]
      → {{a b} {c d}} {{e f} {g h}}
```

The indicated return value also becomes the new value of *x*.

```
lset x 1 1 0 j
      → {{a b} {c d}} {{e f} {j h}}
lset x {1 1 0} j
      → {{a b} {c d}} {{e f} {j h}}
```

Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org*. All rights reserved.*

lsort

Keywords: lsort, element, list, order, sort

Name

lsort - Sort the elements of a list

Synopsis

lsort ?*options*? *list*

Description

This command sorts the elements of *list*, returning a new list in sorted order. The implementation of the **lsort** command uses the merge-sort algorithm which is a stable sort that has O(n log n) performance characteristics.

By default ASCII sorting is used with the result returned in increasing order. However, any of the following options may be specified before *list* to control the sorting process (unique abbreviations are accepted):

-ascii
: Use string comparison with Unicode code-point collation order (the name is for backward-compatibility reasons.) This is the default.

-dictionary
: Use dictionary-style comparison. This is the same as **-ascii** except (a) case is ignored except as a tie-breaker and (b) if two strings contain embedded numbers, the numbers compare as integers, not characters. For example, in **-dictionary** mode, **bigBoy** sorts between **bigbang** and **bigboy**, and **x10y** sorts between **x9y** and **x11y**. Overrides the **-nocase** option.

-integer
: Convert list elements to integers and use integer comparison.

-real
: Convert list elements to floating-point values and use floating comparison.

-command *command*
: Use *command* as a comparison command. To compare two elements, evaluate a Tcl script consisting of *command* with the two elements appended as additional arguments. The script should return an integer less than, equal to, or greater than zero if the first element is to be considered less than, equal to, or greater than the second, respectively.

-increasing
: Sort the list in increasing order ("smallest"items first). This is the default.

-decreasing
: Sort the list in decreasing order ("largest"items first).

-indices

Return a list of indices into *list* in sorted order instead of the values themselves.

-index *indexList*

If this option is specified, each of the elements of *list* must itself be a proper Tcl sublist (unless **-stride** is used). Instead of sorting based on whole sublists, **lsort** will extract the *indexList*'th element from each sublist (as if the overall element and the *indexList* were passed to lindex) and sort based on the given element. For example,

```
lsort -integer -index 1 \
      {{First 24} {Second 18} {Third 30}}
```

returns **{Second 18} {First 24} {Third 30}**,

```
lsort -index end-1 \
        {{a 1 e i} {b 2 3 f g} {c 4 5 6 d h}}
```

returns **{c 4 5 6 d h} {a 1 e i} {b 2 3 f g}**, and

```
lsort -index {0 1} {
    {{b i g} 12345}
    {{d e m o} 34512}
    {{c o d e} 54321}
}
```

returns **{{d e m o} 34512} {{b i g} 12345} {{c o d e} 54321}** (because **e** sorts before **i** which sorts before **o**.) This option is much more efficient than using **-command** to achieve the same effect.

-stride *strideLength*

If this option is specified, the list is treated as consisting of groups of *strideLength* elements and the groups are sorted by either their first element or, if the **-index** option is used, by the element within each group given by the first index passed to **-index** (which is then ignored by **-index**). Elements always remain in the same position within their group.

The list length must be an integer multiple of *strideLength*, which in turn must be at least 2.

For example,

```
lsort -stride 2 {carrot 10 apple 50 banana 25}
```

returns "apple 50 banana 25 carrot 10", and

```
lsort -stride 2 -index 1 -integer {carrot 10 apple 50 banana 25}
```

returns "carrot 10 banana 25 apple 50".

-nocase

Causes comparisons to be handled in a case-insensitive manner. Has no effect if combined with the **-dictionary**, **-integer**, or **-real** options.

-unique

If this option is specified, then only the last set of duplicate elements found in the list will be retained. Note that duplicates are determined relative to the comparison used in the sort.

Thus if **-index 0** is used, **{1 a}** and **{1 b}** would be considered duplicates and only the second element, **{1 b}**, would be retained.

Notes

The options to **lsort** only control what sort of comparison is used, and do not necessarily constrain what the values themselves actually are. This distinction is only noticeable when the list to be sorted has fewer than two elements.

The **lsort** command is reentrant, meaning it is safe to use as part of the implementation of a command used in the **-command** option.

Examples

Sorting a list using ASCII sorting:

```
% lsort {a10 B2 b1 a1 a2}
B2 a1 a10 a2 b1
```

Sorting a list using Dictionary sorting:

```
% lsort -dictionary {a10 B2 b1 a1 a2}
a1 a2 a10 b1 B2
```

Sorting lists of integers:

```
% lsort -integer {5 3 1 2 11 4}
1 2 3 4 5 11
% lsort -integer {1 2 0x5 7 0 4 -1}
-1 0 1 2 4 0x5 7
```

Sorting lists of floating-point numbers:

```
% lsort -real {5 3 1 2 11 4}
1 2 3 4 5 11
% lsort -real {.5 0.07e1 0.4 6e-1}
0.4 .5 6e-1 0.07e1
```

Sorting using indices:

```
% # Note the space character before the c
% lsort {{a 5} { c 3} {b 4} {e 1} {d 2}}
{ c 3} {a 5} {b 4} {d 2} {e 1}
% lsort -index 0 {{a 5} { c 3} {b 4} {e 1} {d 2}}
{a 5} {b 4} { c 3} {d 2} {e 1}
% lsort -index 1 {{a 5} { c 3} {b 4} {e 1} {d 2}}
{e 1} {d 2} { c 3} {b 4} {a 5}
```

Sorting a dictionary:

```
% set d [dict create c d a b h i f g c e]
c e a b h i f g
% lsort -stride 2 $d
a b c e f g h i
```

Sorting using striding and multiple indices:

```
% # Note the first index value is relative to the group
% lsort -stride 3 -index {0 1} \
     {{Bob Smith} 25 Audi {Jane Doe} 40 Ford}
{{Jane Doe} 40 Ford {Bob Smith} 25 Audi}
```

Stripping duplicate values using sorting:

```
% lsort -unique {a b c a b c a b c}
a b c
```

More complex sorting using a comparison function:

```
% proc compare {a b} {
    set a0 [lindex $a 0]
    set b0 [lindex $b 0]
    if {$a0 < $b0} {
        return -1
    } elseif {$a0 > $b0} {
        return 1
    }
    return [string compare [lindex $a 1] [lindex $b 1]]
}
% lsort -command compare \
        {{3 apple} {0x2 carrot} {1 dingo} {2 banana}}
{1 dingo} {2 banana} {0x2 carrot} {3 apple}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 1999 Scriptics Corporation
Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org*. All rights reserved.*

mathfunc

Keywords: mathfunc

Name

mathfunc - Mathematical functions for Tcl expressions

Synopsis

package require Tcl 8.5
::tcl::mathfunc::abs *arg*
::tcl::mathfunc::acos *arg*
::tcl::mathfunc::asin *arg*
::tcl::mathfunc::atan *arg*
::tcl::mathfunc::atan2 *y x*
::tcl::mathfunc::bool *arg*
::tcl::mathfunc::ceil *arg*
::tcl::mathfunc::cos *arg*

::tcl::mathfunc::cosh *arg*
::tcl::mathfunc::double *arg*
::tcl::mathfunc::entier *arg*
::tcl::mathfunc::exp *arg*
::tcl::mathfunc::floor *arg*
::tcl::mathfunc::fmod *x y*
::tcl::mathfunc::hypot *x y*
::tcl::mathfunc::int *arg*
::tcl::mathfunc::isqrt *arg*
::tcl::mathfunc::log *arg*
::tcl::mathfunc::log10 *arg*
::tcl::mathfunc::max *arg* ?*arg* ...?
::tcl::mathfunc::min *arg* ?*arg* ...?
::tcl::mathfunc::pow *x y*
::tcl::mathfunc::rand
::tcl::mathfunc::round *arg*
::tcl::mathfunc::sin *arg*
::tcl::mathfunc::sinh *arg*
::tcl::mathfunc::sqrt *arg*
::tcl::mathfunc::srand *arg*
::tcl::mathfunc::tan *arg*
::tcl::mathfunc::tanh *arg*
::tcl::mathfunc::wide *arg*

Description

The expr command handles mathematical functions of the form **sin($x)** or **atan2($y,$x)** by converting them to calls of the form **[tcl::mathfunc::sin [expr {$x}]]** or **[tcl::mathfunc::atan2 [expr {$y}] [expr {$x}]]**. A number of math functions are available by default within the namespace **::tcl::mathfunc**; these functions are also available for code apart from expr, by invoking the given commands directly.

Tcl supports the following mathematical functions in expressions, all of which work solely with floating-point numbers unless otherwise noted:

abs acos asin atan
atan2 bool ceil cos
cosh double entier exp
floor fmod hypot int
isqrt log log10 max
min pow rand round
sin sinh sqrt srand
tan tanh wide

In addition to these predefined functions, applications may define additional functions by using proc (or any other method, such as interp alias or Tcl_CreateObjCommand) to define new commands in the **tcl::mathfunc** namespace. In addition, an obsolete interface named Tcl_CreateMathFunc() is available to extensions that are written in C. The latter interface is not recommended for new implementations.

Detailed definitions

abs *arg*
Returns the absolute value of *arg*. *Arg* may be either integer or floating-point, and the result is returned in the same form.

acos *arg*
Returns the arc cosine of *arg*, in the range [*0*,*pi*] radians. *Arg* should be in the range [*-1*,*1*].

asin *arg*
Returns the arc sine of *arg*, in the range [*-pi/2*,*pi/2*] radians. *Arg* should be in the range [*-1*,*1*].

atan *arg*
Returns the arc tangent of *arg*, in the range [*-pi/2*,*pi/2*] radians.

atan2 *y x*
Returns the arc tangent of *y*/*x*, in the range [*-pi*,*pi*] radians. *x* and *y* cannot both be 0. If *x* is greater than *0*, this is equivalent to "**atan** [expr {*y*/*x*}]".

bool *arg*
Accepts any numeric value, or any string acceptable to **string is boolean**, and returns the corresponding boolean value **0** or **1**. Non-zero numbers are true. Other numbers are false. Non-numeric strings produce boolean value in agreement with **string is true** and **string is false**.

ceil *arg*
Returns the smallest integral floating-point value (i.e. with a zero fractional part) not less than *arg*. The argument may be any numeric value.

cos *arg*
Returns the cosine of *arg*, measured in radians.

cosh *arg*
Returns the hyperbolic cosine of *arg*. If the result would cause an overflow, an error is returned.

double *arg*
The argument may be any numeric value, If *arg* is a floating-point value, returns *arg*, otherwise converts *arg* to floating-point and returns the converted value. May return **Inf** or **-Inf** when the argument is a numeric value that exceeds the floating-point range.

entier *arg*
The argument may be any numeric value. The integer part of *arg* is determined and returned. The integer range returned by this function is unlimited, unlike **int** and **wide** which truncate their range to fit in particular storage widths.

exp *arg*
Returns the exponential of *arg*, defined as *e****arg*. If the result would cause an overflow, an error is returned.

floor *arg*
Returns the largest integral floating-point value (i.e. with a zero fractional part) not greater than *arg*. The argument may be any numeric value.

fmod *x y*
Returns the floating-point remainder of the division of *x* by *y*. If *y* is 0, an error is returned.

hypot *x y*
Computes the length of the hypotenuse of a right-angled triangle, approximately "**sqrt** [expr

{*x***x*+*y***y*}]" except for being more numerically stable when the two arguments have substantially different magnitudes.

int *arg*
The argument may be any numeric value. The integer part of *arg* is determined, and then the low order bits of that integer value up to the machine word size are returned as an integer value. For reference, the number of bytes in the machine word are stored in the **wordSize** element of the tcl_platform array.

isqrt *arg*
Computes the integer part of the square root of *arg*. *Arg* must be a positive value, either an integer or a floating point number. Unlike **sqrt**, which is limited to the precision of a floating point number, *isqrt* will return a result of arbitrary precision.

log *arg*
Returns the natural logarithm of *arg*. *Arg* must be a positive value.

log10 *arg*
Returns the base 10 logarithm of *arg*. *Arg* must be a positive value.

max *arg...*
Accepts one or more numeric arguments. Returns the one argument with the greatest value.

min *arg...*
Accepts one or more numeric arguments. Returns the one argument with the least value.

pow *x y*
Computes the value of *x* raised to the power *y*. If *x* is negative, *y* must be an integer value.

rand
Returns a pseudo-random floating-point value in the range (*0*,*1*). The generator algorithm is a simple linear congruential generator that is not cryptographically secure. Each result from **rand** completely determines all future results from subsequent calls to **rand**, so **rand** should not be used to generate a sequence of secrets, such as one-time passwords. The seed of the generator is initialized from the internal clock of the machine or may be set with the **srand** function.

round *arg*
If *arg* is an integer value, returns *arg*, otherwise converts *arg* to integer by rounding and returns the converted value.

sin *arg*
Returns the sine of *arg*, measured in radians.

sinh *arg*
Returns the hyperbolic sine of *arg*. If the result would cause an overflow, an error is returned.

sqrt *arg*
The argument may be any non-negative numeric value. Returns a floating-point value that is the square root of *arg*. May return **Inf** when the argument is a numeric value that exceeds the square of the maximum value of the floating-point range.

srand *arg*
The *arg*, which must be an integer, is used to reset the seed for the random number generator of **rand**. Returns the first random number (see **rand**) from that seed. Each interpreter has its own seed.

tan *arg*

Returns the tangent of *arg*, measured in radians.

tanh *arg*

Returns the hyperbolic tangent of *arg*.

wide *arg*

The argument may be any numeric value. The integer part of *arg* is determined, and then the low order 64 bits of that integer value are returned as an integer value.

Copyright

```
Copyright © 1993 The Regents of the University of California.
Copyright © 1994-2000 Sun Microsystems Incorporated.
Copyright © 2005, 2006 Kevin B. Kenny <kennykb@acm.org>.
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-2000 Sun Microsystems, Inc.
Copyright (c) 2005 Kevin B. Kenny kennykb@acm.org*. All rights reserved*

mathop

Keywords: mathop, command, expression, operator

Name

mathop - Mathematical operators as Tcl commands

Synopsis

package require Tcl 8.5
::tcl::mathop::! *number*
::tcl::mathop::~ *number*
::tcl::mathop::+ ?*number* ...?
::tcl::mathop::- *number* ?*number* ...?
::tcl::mathop::* ?*number* ...?
::tcl::mathop::/ *number* ?*number* ...?
::tcl::mathop::% *number number*
::tcl::mathop::** ?*number* ...?
::tcl::mathop::& ?*number* ...?
::tcl::mathop::| ?*number* ...?
::tcl::mathop::^ ?*number* ...?
::tcl::mathop::<< *number number*
::tcl::mathop::>> *number number*
::tcl::mathop::== ?*arg* ...?
::tcl::mathop::!= *arg arg*
::tcl::mathop::< ?*arg* ...?
::tcl::mathop::<= ?*arg* ...?
::tcl::mathop::>= ?*arg* ...?
::tcl::mathop::> ?*arg* ...?
::tcl::mathop::eq ?*arg* ...?
::tcl::mathop::ne *arg arg*
::tcl::mathop::in *arg list*

::tcl::mathop::ni *arg list*

Description

The commands in the **::tcl::mathop** namespace implement the same set of operations as supported by the expr command. All are exported from the namespace, but are not imported into any other namespace by default. Note that renaming, reimplementing or deleting any of the commands in the namespace does *not* alter the way that the expr command behaves, and nor does defining any new commands in the **::tcl::mathop** namespace.

The following operator commands are supported:

~! + - *
/ % ** & |
^ >> << == eq
!= ne < <= >
>= in ni

Mathematical operators

The behaviors of the mathematical operator commands are as follows:

! *boolean*
Returns the boolean negation of *boolean*, where *boolean* may be any numeric value or any other form of boolean value (i.e. it returns truth if the argument is falsity or zero, and falsity if the argument is truth or non-zero).

+ ?*number ...*?
Returns the sum of arbitrarily many arguments. Each *number* argument may be any numeric value. If no arguments are given, the result will be zero (the summation identity).

- *number* ?*number ...*?
If only a single *number* argument is given, returns the negation of that numeric value. Otherwise returns the number that results when all subsequent numeric values are subtracted from the first one. All *number* arguments must be numeric values. At least one argument must be given.

*** ?*number ...*?**
Returns the product of arbitrarily many arguments. Each *number* may be any numeric value. If no arguments are given, the result will be one (the multiplicative identity).

/ *number* ?*number ...*?
If only a single *number* argument is given, returns the reciprocal of that numeric value (i.e. the value obtained by dividing 1.0 by that value). Otherwise returns the number that results when the first numeric argument is divided by all subsequent numeric arguments. All *number* arguments must be numeric values. At least one argument must be given.

Note that when the leading values in the list of arguments are integers, integer division will be used for those initial steps (i.e. the intermediate results will be as if the functions *floor* and *int* are applied to them, in that order). If all values in the operation are integers, the result will be an integer.

% *number number*
Returns the integral modulus (i.e., remainder) of the first argument with respect to the second. Each *number* must have an integral value. Also, the sign of the result will be the

same as the sign of the second *number*, which must not be zero.

Note that Tcl defines this operation exactly even for negative numbers, so that the following command returns a true value (omitting the namespace for clarity):

```
== [* [/ x y] y] [- x [% x y]]
```

**** ?*number* ...?**
Returns the result of raising each value to the power of the result of recursively operating on the result of processing the following arguments, so "**** 2 3 4**" is the same as "**** 2 [** 3 4]**". Each *number* may be any numeric value, though the second number must not be fractional if the first is negative. The maximum exponent value that Tcl can handle if the first number is an integer > 1 is 268435455. If no arguments are given, the result will be one, and if only one argument is given, the result will be that argument. The result will have an integral value only when all arguments are integral values.

Comparison operators

The behaviors of the comparison operator commands (most of which operate preferentially on numeric arguments) are as follows:

== ?*arg* ...?
Returns whether each argument is equal to the arguments on each side of it in the sense of the expr == operator (*i.e.*, numeric comparison if possible, exact string comparison otherwise). If fewer than two arguments are given, this operation always returns a true value.

eq ?*arg* ...?
Returns whether each argument is equal to the arguments on each side of it using exact string comparison. If fewer than two arguments are given, this operation always returns a true value.

!= *arg arg*
Returns whether the two arguments are not equal to each other, in the sense of the expr!= operator (*i.e.*, numeric comparison if possible, exact string comparison otherwise).

ne *arg arg*
Returns whether the two arguments are not equal to each other using exact string comparison.

< ?*arg* ...?
Returns whether the arbitrarily-many arguments are ordered, with each argument after the first having to be strictly more than the one preceding it. Comparisons are performed preferentially on the numeric values, and are otherwise performed using UNICODE string comparison. If fewer than two arguments are present, this operation always returns a true value. When the arguments are numeric but should be compared as strings, the string compare command should be used instead.

<= ?*arg* ...?
Returns whether the arbitrarily-many arguments are ordered, with each argument after the first having to be equal to or more than the one preceding it. Comparisons are performed preferentially on the numeric values, and are otherwise performed using UNICODE string comparison. If fewer than two arguments are present, this operation always returns a true value. When the arguments are numeric but should be compared as strings, the string compare command should be used instead.

> ?*arg* ...?
Returns whether the arbitrarily-many arguments are ordered, with each argument after the first having to be strictly less than the one preceding it. Comparisons are performed preferentially on the numeric values, and are otherwise performed using UNICODE string comparison. If fewer than two arguments are present, this operation always returns a true value. When the arguments are numeric but should be compared as strings, the string compare command should be used instead.

>= ?*arg* ...?
Returns whether the arbitrarily-many arguments are ordered, with each argument after the first having to be equal to or less than the one preceding it. Comparisons are performed preferentially on the numeric values, and are otherwise performed using UNICODE string comparison. If fewer than two arguments are present, this operation always returns a true value. When the arguments are numeric but should be compared as strings, the string compare command should be used instead.

Bit-wise operators

The behaviors of the bit-wise operator commands (all of which only operate on integral arguments) are as follows:

~ *number*
Returns the bit-wise negation of *number*. *Number* may be an integer of any size. Note that the result of this operation will always have the opposite sign to the input *number*.

& ?*number* ...?
Returns the bit-wise AND of each of the arbitrarily many arguments. Each *number* must have an integral value. If no arguments are given, the result will be minus one.

| ?*number* ...?
Returns the bit-wise OR of each of the arbitrarily many arguments. Each *number* must have an integral value. If no arguments are given, the result will be zero.

^ ?*number* ...?
Returns the bit-wise XOR of each of the arbitrarily many arguments. Each *number* must have an integral value. If no arguments are given, the result will be zero.

<< *number number*
Returns the result of bit-wise shifting the first argument left by the number of bits specified in the second argument. Each *number* must have an integral value.

>> *number number*
Returns the result of bit-wise shifting the first argument right by the number of bits specified in the second argument. Each *number* must have an integral value.

List operators

The behaviors of the list-oriented operator commands are as follows:

in *arg list*
Returns whether the value *arg* is present in the list *list* (according to exact string comparison of elements).

ni *arg list*
Returns whether the value *arg* is not present in the list *list* (according to exact string comparison of elements).

Examples

The simplest way to use the operators is often by using namespace path to make the commands available. This has the advantage of not affecting the set of commands defined by the current namespace.

```
namespace path {::tcl::mathop ::tcl::mathfunc}

# Compute the sum of some numbers
set sum [+ 1 2 3]

# Compute the average of a list
set list {1 2 3 4 5 6}
set mean [/ [+ {*}$list] [double [llength $list]]]

# Test for list membership
set gotIt [in 3 $list]

# Test to see if a value is within some defined range
set inRange [<= 1 $x 5]

# Test to see if a list is sorted
set sorted [<= {*}$list]
```

Copyright (c) 2006-2007 Donal K. Fellows.

memory

Keywords: memory, debug

Name

memory - Control Tcl memory debugging capabilities

Synopsis

memory *option* ?*arg arg* ...?

Description

The **memory** command gives the Tcl developer control of Tcl's memory debugging capabilities. The memory command has several suboptions, which are described below. It is only available when Tcl has been compiled with memory debugging enabled (when TCL_MEM_DEBUG is defined at compile time), and after Tcl_InitMemory has been called.

memory active *file*
: Write a list of all currently allocated memory to the specified *file*.

memory break_on_malloc *count*
: After the *count* allocations have been performed, ckalloc outputs a message to this effect and that it is now attempting to enter the C debugger. Tcl will then issue a *SIGINT* signal against itself. If you are running Tcl under a C debugger, it should then enter the debugger command mode.

memory info

Returns a report containing the total allocations and frees since Tcl began, the current packets allocated (the current number of calls to ckalloc not met by a corresponding call to ckfree), the current bytes allocated, and the maximum number of packets and bytes allocated.

memory init [on|off]
Turn on or off the preinitialization of all allocated memory with bogus bytes. Useful for detecting the use of uninitialized values.

memory objs *file*
Causes a list of all allocated Tcl_Obj values to be written to the specified *file* immediately, together with where they were allocated. Useful for checking for leaks of values.

memory onexit *file*
Causes a list of all allocated memory to be written to the specified *file* during the finalization of Tcl's memory subsystem. Useful for checking that memory is properly cleaned up during process exit.

memory tag *string*
Each packet of memory allocated by ckalloc can have associated with it a string-valued tag. In the lists of allocated memory generated by **memory active** and **memory onexit**, the tag for each packet is printed along with other information about the packet. The **memory tag** command sets the tag value for subsequent calls to ckalloc to be *string*.

memory trace [on|off]
Turns memory tracing on or off. When memory tracing is on, every call to ckalloc causes a line of trace information to be written to *stderr*, consisting of the word *ckalloc*, followed by the address returned, the amount of memory allocated, and the C filename and line number of the code performing the allocation. For example:

```
ckalloc 40e478 98 tclProc.c 1406
```

Calls to ckfree are traced in the same manner.

memory trace_on_at_malloc *count*
Enable memory tracing after *count* ckallocs have been performed. For example, if you enter **memory trace_on_at_malloc 100**, after the 100th call to ckalloc, memory trace information will begin being displayed for all allocations and frees. Since there can be a lot of memory activity before a problem occurs, judicious use of this option can reduce the slowdown caused by tracing (and the amount of trace information produced), if you can identify a number of allocations that occur before the problem sets in. The current number of memory allocations that have occurred since Tcl started is printed on a guard zone failure.

memory validate [on|off]
Turns memory validation on or off. When memory validation is enabled, on every call to ckalloc or ckfree, the guard zones are checked for every piece of memory currently in existence that was allocated by ckalloc. This has a large performance impact and should only be used when overwrite problems are strongly suspected. The advantage of enabling memory validation is that a guard zone overwrite can be detected on the first call to ckalloc or ckfree after the overwrite occurred, rather than when the specific memory with the overwritten guard zone(s) is freed, which may occur long after the overwrite occurred.

Copyright (c) 1992-1999 Karl Lehenbauer & Mark Diekhans
Copyright (c) 2000 Scriptics Corporation.

msgcat

Keywords: msgcat, internationalization, i18n, localization, l10n, message, text, translation

Name

msgcat - Tcl message catalog

Synopsis

package require Tcl 8.5
package require msgcat 1.6

::msgcat::mc *src-string* ?*arg arg* ...?
::msgcat::mcmax ?*src-string src-string* ...?
::msgcat::mcexists ?**-exactnamespace**? ?**-exactlocale**? *src-string*
::msgcat::mclocale ?*newLocale*?
::msgcat::mcpreferences
::msgcat::mcloadedlocales subcommand ?*locale*?
::msgcat::mcload *dirname*
::msgcat::mcset *locale src-string* ?*translate-string*?
::msgcat::mcmset *locale src-trans-list*
::msgcat::mcflset *src-string* ?*translate-string*?
::msgcat::mcflmset *src-trans-list*
::msgcat::mcunknown *locale src-string* ?*arg arg* ...?
::msgcat::mcpackagelocale subcommand ?*locale*?
::msgcat::mcpackageconfig subcommand *option* ?*value*?
::msgcat::mcforgetpackage

Description

The **msgcat** package provides a set of functions that can be used to manage multi-lingual user interfaces. Text strings are defined in a "message catalog" which is independent from the application, and which can be edited or localized without modifying the application source code. New languages or locales may be provided by adding a new file to the message catalog.

msgcat distinguishes packages by its namespace. Each package has its own message catalog and configuration settings in **msgcat**.

A *locale* is a specification string describing a user language like **de_ch** for Swiss German. In **msgcat**, there is a global locale initialized by the system locale of the current system. Each package may decide to use the global locale or to use a package specific locale.

The global locale may be changed on demand, for example by a user initiated language change or within a multi user application like a web server.

Commands

::msgcat::mc *src-string* ?*arg arg* ...?

Returns a translation of *src-string* according to the current locale. If additional arguments past *src-string* are given, the format command is used to substitute the additional arguments in the translation of *src-string*.

::msgcat::mc will search the messages defined in the current namespace for a translation of

src-string; if none is found, it will search in the parent of the current namespace, and so on until it reaches the global namespace. If no translation string exists, **::msgcat::mcunknown** is called and the string returned from **::msgcat::mcunknown** is returned.

::msgcat::mc is the main function used to localize an application. Instead of using an English string directly, an application can pass the English string through **::msgcat::mc** and use the result. If an application is written for a single language in this fashion, then it is easy to add support for additional languages later simply by defining new message catalog entries.

::msgcat::mcmax ?*src-string src-string* ...?

Given several source strings, **::msgcat::mcmax** returns the length of the longest translated string. This is useful when designing localized GUIs, which may require that all buttons, for example, be a fixed width (which will be the width of the widest button).

::msgcat::mcexists ?-exactnamespace? ?-exactlocale? *src-string*

Return true, if there is a translation for the given src-string.

The search may be limited by the option **-exactnamespace** to only check the current namespace and not any parent namespaces.

It may also be limited by the option **-exactlocale** to only check the first prefered locale (e.g. first element returned by **::msgcat::mcpreferences** if global locale is used).

::msgcat::mclocale ?*newLocale*?

This function sets the locale to *newLocale*. If *newLocale* is omitted, the current locale is returned, otherwise the current locale is set to *newLocale*. msgcat stores and compares the locale in a case-insensitive manner, and returns locales in lowercase. The initial locale is determined by the locale specified in the user's environment. See Locale specification below for a description of the locale string format.

If the locale is set, the preference list of locales is evaluated. Locales in this list are loaded now, if not jet loaded.

::msgcat::mcpreferences

Returns an ordered list of the locales preferred by the user, based on the user's language specification. The list is ordered from most specific to least preference. The list is derived from the current locale set in msgcat by **::msgcat::mclocale**, and cannot be set independently. For example, if the current locale is en_US_funky, then **::msgcat::mcpreferences** returns **{en_us_funky en_us en {}}**.

::msgcat::mcloadedlocales subcommand ?*locale*?

This group of commands manage the list of loaded locales for packages not setting a package locale.

The subcommand **get** returns the list of currently loaded locales.

The subcommand **present** requires the argument *locale* and returns true, if this locale is loaded.

The subcommand **clear** removes all locales and their data, which are not in the current preference list.

::msgcat::mcload *dirname*
Searches the specified directory for files that match the language specifications returned by ::msgcat::mcloadedlocales get (or msgcat::mcpackagelocale preferences if a package locale is set) (note that these are all lowercase), extended by the file extension ".msg". Each matching file is read in order, assuming a UTF-8 encoding. The file contents are then evaluated as a Tcl script. This means that Unicode characters may be present in the message file either directly in their UTF-8 encoded form, or by use of the backslash-u quoting recognized by Tcl evaluation. The number of message files which matched the specification and were loaded is returned.

In addition, the given folder is stored in the **msgcat** package configuration option *mcfolder* to eventually load message catalog files required by a locale change.

::msgcat::mcset *locale src-string* ?*translate-string*?
Sets the translation for *src-string* to *translate-string* in the specified *locale* and the current namespace. If *translate-string* is not specified, *src-string* is used for both. The function returns *translate-string*.

::msgcat::mcmset *locale src-trans-list*
Sets the translation for multiple source strings in *src-trans-list* in the specified *locale* and the current namespace. *src-trans-list* must have an even number of elements and is in the form { *src-string translate-string* ?*src-string translate-string ...*?} **::msgcat::mcmset** can be significantly faster than multiple invocations of **::msgcat::mcset**. The function returns the number of translations set.

::msgcat::mcflset *src-string* ?*translate-string*?
Sets the translation for *src-string* to *translate-string* in the current namespace for the locale implied by the name of the message catalog being loaded via **::msgcat::mcload**. If *translate-string* is not specified, *src-string* is used for both. The function returns *translate-string*.

::msgcat::mcflmset *src-trans-list*
Sets the translation for multiple source strings in *src-trans-list* in the current namespace for the locale implied by the name of the message catalog being loaded via **::msgcat::mcload**. *src-trans-list* must have an even number of elements and is in the form {*src-string translate-string* ?*src-string translate-string ...*?} **::msgcat::mcflmset** can be significantly faster than multiple invocations of **::msgcat::mcflset**. The function returns the number of translations set.

::msgcat::mcunknown *locale src-string* ?*arg arg* ...?
This routine is called by **::msgcat::mc** in the case when a translation for *src-string* is not defined in the current locale. The default action is to return *src-string* passed by format if there are any arguments. This procedure can be redefined by the application, for example to log error messages for each unknown string. The **::msgcat::mcunknown** procedure is invoked at the same stack context as the call to **::msgcat::mc**. The return value of **::msgcat::mcunknown** is used as the return value for the call to **::msgcat::mc**.

Note that this routine is only called if the concerned package did not set a package locale unknown command name.

::msgcat::mcforgetpackage

The calling package clears all its state within the msgcat package including all settings and translations.

Locale specification

The locale is specified to **msgcat** by a locale string passed to **::msgcat::mclocale**. The locale string consists of a language code, an optional country code, and an optional system-specific code, each separated by "_". The country and language codes are specified in standards ISO-639 and ISO-3166. For example, the locale "en" specifies English and "en_US" specifies U.S. English.

When the msgcat package is first loaded, the locale is initialized according to the user's environment. The variables **env(LC_ALL)**, **env(LC_MESSAGES)**, and **env(LANG)** are examined in order. The first of them to have a non-empty value is used to determine the initial locale. The value is parsed according to the XPG4 pattern

```
language[_country][.codeset][@modifier]
```

to extract its parts. The initial locale is then set by calling **::msgcat::mclocale** with the argument

```
language[_country][_modifier]
```

On Windows and Cygwin, if none of those environment variables is set, msgcat will attempt to extract locale information from the registry. From Windows Vista on, the RFC4747 locale name "lang-script-country-options" is transformed to the locale as "lang_country_script" (Example: sr-Latn-CS -> sr_cs_latin). For Windows XP, the language id is transformed analoguously (Example: 0c1a -> sr_yu_cyrillic). If all these attempts to discover an initial locale from the user's environment fail, msgcat defaults to an initial locale of "C".

When a locale is specified by the user, a "best match" search is performed during string translation. For example, if a user specifies en_GB_Funky, the locales "en_gb_funky", "en_gb", "en" and "" (the empty string) are searched in order until a matching translation string is found. If no translation string is available, then the unknown handler is called.

Namespaces and message catalogs

Strings stored in the message catalog are stored relative to the namespace from which they were added. This allows multiple packages to use the same strings without fear of collisions with other packages. It also allows the source string to be shorter and less prone to typographical error.

For example, executing the code

```
::msgcat::mcset en hello "hello from ::"
namespace eval foo {
    ::msgcat::mcset en hello "hello from ::foo"
}
puts [::msgcat::mc hello]
namespace eval foo {puts [::msgcat::mc hello]}
```

will print

```
hello from ::
hello from ::foo
```

When searching for a translation of a message, the message catalog will search first the current namespace, then the parent of the current namespace, and so on until the global namespace is reached. This allows child namespaces to "inherit" messages from their parent namespace.

For example, executing (in the "en" locale) the code

```
::msgcat::mcset en m1 ":: message1"
::msgcat::mcset en m2 ":: message2"
::msgcat::mcset en m3 ":: message3"
namespace eval ::foo {
    ::msgcat::mcset en m2 "::foo message2"
    ::msgcat::mcset en m3 "::foo message3"
}
namespace eval ::foo::bar {
    ::msgcat::mcset en m3 "::foo::bar message3"
}
namespace import ::msgcat::mc
puts "[mc m1]; [mc m2]; [mc m3]"
namespace eval ::foo {puts "[mc m1]; [mc m2]; [mc m3]"}
namespace eval ::foo::bar {puts "[mc m1]; [mc m2]; [mc m3]"}
```

will print

```
:: message1; :: message2; :: message3
:: message1; ::foo message2; ::foo message3
:: message1; ::foo message2; ::foo::bar message3
```

Location and format of message files

Message files can be located in any directory, subject to the following conditions:

1. All message files for a package are in the same directory.
2. The message file name is a msgcat locale specifier (all lowercase) followed by ".msg". For example:

```
es.msg    – spanish
en_gb.msg – United Kingdom English
```

Exception: The message file for the root locale "" is called "**ROOT.msg**". This exception is made so as not to cause peculiar behavior, such as marking the message file as "hidden" on Unix file systems.

1. The file contains a series of calls to **mcflset** and **mcflmset**, setting the necessary translation strings for the language, likely enclosed in a namespace eval so that all source strings are tied to the namespace of the package. For example, a short **es.msg** might contain:

```
namespace eval ::mypackage {
    ::msgcat::mcflset "Free Beer" "Cerveza Gratis"
}
```

Recommended message setup for packages

If a package is installed into a subdirectory of the tcl_pkgPath and loaded via package require,

the following procedure is recommended.

1. During package installation, create a subdirectory **msgs** under your package directory.
2. Copy your *.msg files into that directory.
3. Add the following command to your package initialization script:

```
# load language files, stored in msgs subdirectory
::msgcat::mcload [file join [file dirname [info script]] msgs]
```

Positional codes for format and scan commands

It is possible that a message string used as an argument to format might have positionally dependent parameters that might need to be repositioned. For example, it might be syntactically desirable to rearrange the sentence structure while translating.

```
format "We produced %d units in location %s" $num $city
format "In location %s we produced %d units" $city $num
```

This can be handled by using the positional parameters:

```
format "We produced %1\$d units in location %2\$s" $num $city
format "In location %2\$s we produced %1\$d units" $num $city
```

Similarly, positional parameters can be used with scan to extract values from internationalized strings. Note that it is not necessary to pass the output of **::msgcat::mc** to format directly; by passing the values to substitute in as arguments, the formatting substitution is done directly.

```
msgcat::mc {Produced %1$d at %2$s} $num $city
# ... where that key is mapped to one of the
# human-oriented versions by msgcat::mcset
```

Package private locale

A package using **msgcat** may choose to use its own package private locale and its own set of loaded locales, independent to the global locale set by **::msgcat::mclocale**.

This allows a package to change its locale without causing any locales load or removal in other packages and not to invoke the global locale change callback (see below).

This action is controled by the following ensemble:

::msgcat::mcpackagelocale set ?*locale*?
: Set or change a package private locale. The package private locale is set to the given *locale* if the *locale* is given. If the option *locale* is not given, the package is set to package private locale mode, but no locale is changed (e.g. if the global locale was valid for the package before, it is copied to the package private locale).

  This command may cause the load of locales.

::msgcat::mcpackagelocale get
: Return the package private locale or the global locale, if no package private locale is set.

::msgcat::mcpackagelocale preferences
: Return the package private preferences or the global preferences, if no package private locale is set.

::msgcat::mcpackagelocale loaded
Return the list of locales loaded for this package.

::msgcat::mcpackagelocale isset
Returns true, if a package private locale is set.

::msgcat::mcpackagelocale unset
Unset the package private locale and use the global locale. Load and remove locales to adjust the list of loaded locales for the package to the global loaded locales list.

::msgcat::mcpackagelocale present *locale*
Returns true, if the given locale is loaded for the package.

::msgcat::mcpackagelocale clear
Clear any loaded locales of the package not present in the package preferences.

Changing package options

Each package using msgcat has a set of options within **msgcat**. The package options are described in the next sectionPackage options. Each package option may be set or unset individually using the following ensemble:

::msgcat::mcpackageconfig get *option*
Return the current value of the given *option*. This call returns an error if the option is not set for the package.

::msgcat::mcpackageconfig isset *option*
Returns 1, if the given *option* is set for the package, 0 otherwise.

::msgcat::mcpackageconfig set *option value*
Set the given *option* to the given *value*. This may invoke additional actions in dependency of the *option*. The return value is 0 or the number of loaded packages for the option **mcfolder**.

::msgcat::mcpackageconfig unset *option*
Unsets the given *option* for the package. No action is taken if the *option* is not set for the package. The empty string is returned.

Package options

The following package options are available for each package:

mcfolder
This is the message folder of the package. This option is set by mcload and by the subcommand set. Both are identical and both return the number of loaded message catalog files.

Setting or changing this value will load all locales contained in the preferences valid for the package. This implies also to invoke any set loadcmd (see below).

Unsetting this value will disable message file load for the package.

loadcmd
This callback is invoked before a set of message catalog files are loaded for the package which has this property set.

This callback may be used to do any preparation work for message file load or to get the message data from another source like a data base. In this case, no message files are used

(mcfolder is unset).

See section Callback invocation below. The parameter list appended to this callback is the list of locales to load.

If this callback is changed, it is called with the preferences valid for the package.

changecmd

This callback is invoked when a default local change was performed. Its purpose is to allow a package to update any dependency on the default locale like showing the GUI in another language.

See the callback invocation section below. The parameter list appended to this callback is **mcpreferences**. The registered callbacks are invoked in no particular order.

unknowncmd

Use a package locale mcunknown procedure instead of the standard version supplied by the msgcat package (**msgcat::mcunknown**).

The called procedure must return the formatted message which will finally be returned by **msgcat::mc**.

A generic unknown handler is used if set to the empty string. This consists of returning the key if no arguments are given. With given arguments, the format command is used to process the arguments.

See section Callback invocation below. The appended arguments are identical to **msgcat::mcunknown**.

Callback invocation

A package may decide to register one or multiple callbacks, as described above.

Callbacks are invoked, if:

1. the callback command is set,
2. the command is not the empty string,
3. the registering namespace exists.

If a called routine fails with an error, the bgerror routine for the interpreter is invoked after command completion. Only exception is the callback **unknowncmd**, where an error causes the invoking **mc**-command to fail with that error.

Examples

Packages which display a GUI may update their widgets when the global locale changes. To register to a callback, use:

```
namespace eval gui {
    msgcat::mcpackageconfig changecmd updateGUI

    proc updateGUI args {
```

```
        puts "New locale is '[lindex $args 0]'."
    }
}
% msgcat::mclocale fr
fr
% New locale is 'fr'.
```

If locales (or additional locales) are contained in another source like a database, a package may use the load callback and not **mcload**:

```
namespace eval db {
    msgcat::mcpackageconfig loadcmd loadMessages

    proc loadMessages args {
        foreach locale $args {
            if {[LocaleInDB $locale]} {
                msgcat::mcmset $locale [GetLocaleList $locale]
            }
        }
    }
}
```

The clock command implementation uses **msgcat** with a package locale to implement the command line parameter **-locale**. Here are some sketches of the implementation:

First, a package locale is initialized and the generic unknown function is deactivated:

```
msgcat::mcpackagelocale set
msgcat::mcpackageconfig unknowncmd ""
```

As an example, the user requires the week day in a certain locale as follows:

```
clock format [clock seconds] -format %A -locale fr
```

clock sets the package locale to **fr** and looks for the day name as follows:

```
msgcat::mcpackagelocale set $locale
return [lindex [msgcat::mc DAYS_OF_WEEK_FULL] $day]
### Returns "mercredi"
```

Within clock, some message-catalog items are heavy in computation and thus are dynamically cached using:

```
proc ::tcl::clock::LocalizeFormat { locale format } {
    set key FORMAT_$format
    if { [::msgcat::mcexists -exactlocale -exactnamespace $key] } {
        return [mc $key]
    }
    #...expensive computation of format clipped...
    mcset $locale $key $format
    return $format
}
```

Credits

The message catalog code was developed by Mark Harrison.

Copyright (c) 1998 Mark Harrison.

my

Keywords: my, method, method visibility, object, private method, public method

Name

my - invoke any method of current object

Synopsis

package require TclOO

my *methodName* ?*arg* ...?

Description

The **my** command is used to allow methods of objects to invoke any method of the object (or its class). In particular, the set of valid values for *methodName* is the set of all methods supported by an object and its superclasses, including those that are not exported. The object upon which the method is invoked is always the one that is the current context of the method (i.e. the object that is returned by self object) from which the **my** command is invoked.

Each object has its own **my** command, contained in its instance namespace.

Examples

This example shows basic use of **my** to use the **variables** method of the oo::object class, which is not publicly visible by default:

```
oo::class create c {
    method count {} {
        my variable counter
        puts [incr counter]
    }
}
c create o
o count             → prints "1"
o count             → prints "2"
o count             → prints "3"
```

Copyright (c) 2007 Donal K. Fellows

namespace

Keywords: namespace, command, ensemble, exported, internal, variable

Name

namespace - create and manipulate contexts for commands and variables

Synopsis

namespace *subcommand* ?*arg* ...?

Description

The **namespace** command lets you create, access, and destroy separate contexts for commands and variables. See the section What is a namespace? below for a brief overview of namespaces. The legal values of *subcommand* are listed below. Note that you can abbreviate the *subcommand*s.

namespace children ?*namespace*? ?*pattern*?
: Returns a list of all child namespaces that belong to the namespace *namespace*. If *namespace* is not specified, then the children are returned for the current namespace. This command returns fully-qualified names, which start with a double colon (**::**). If the optional *pattern* is given, then this command returns only the names that match the glob-style pattern. The actual pattern used is determined as follows: a pattern that starts with double colon (**::**) is used directly, otherwise the namespace *namespace* (or the fully-qualified name of the current namespace) is prepended onto the pattern.

namespace code *script*
: Captures the current namespace context for later execution of the script *script*. It returns a new script in which *script* has been wrapped in a **namespace inscope** command. The new script has two important properties. First, it can be evaluated in any namespace and will cause *script* to be evaluated in the current namespace (the one where the **namespace code** command was invoked). Second, additional arguments can be appended to the resulting script and they will be passed to *script* as additional arguments. For example, suppose the command **set script [namespace code {foo bar}]** is invoked in namespace **::a::b**. Then **eval $script [list x y]** can be executed in any namespace (assuming the value of **script** has been passed in properly) and will have the same effect as the command **::namespace eval ::a::b {foo bar x y}**. This command is needed because extensions like Tk normally execute callback scripts in the global namespace. A scoped command captures a command together with its namespace context in a way that allows it to be executed properly later. See the section Scoped scripts for some examples of how this is used to create callback scripts.

namespace current
: Returns the fully-qualified name for the current namespace. The actual name of the global namespace is "" (i.e., an empty string), but this command returns **::** for the global namespace as a convenience to programmers.

namespace delete ?*namespace namespace ...*?
: Each namespace *namespace* is deleted and all variables, procedures, and child namespaces contained in the namespace are deleted. If a procedure is currently executing inside the namespace, the namespace will be kept alive until the procedure returns; however, the

namespace is marked to prevent other code from looking it up by name. If a namespace does not exist, this command returns an error. If no namespace names are given, this command does nothing.

namespace ensemble *subcommand* ?*arg ...*?
Creates and manipulates a command that is formed out of an ensemble of subcommands. See the section Ensembles below for further details.

namespace eval *namespace arg* ?*arg ...*?
Activates a namespace called *namespace* and evaluates some code in that context. If the namespace does not already exist, it is created. If more than one *arg* argument is specified, the arguments are concatenated together with a space between each one in the same fashion as the eval command, and the result is evaluated.

If *namespace* has leading namespace qualifiers and any leading namespaces do not exist, they are automatically created.

namespace exists *namespace*
Returns **1** if *namespace* is a valid namespace in the current context, returns **0** otherwise.

namespace export ?-clear? ?*pattern pattern ...*?
Specifies which commands are exported from a namespace. The exported commands are those that can be later imported into another namespace using a **namespace import** command. Both commands defined in a namespace and commands the namespace has previously imported can be exported by a namespace. The commands do not have to be defined at the time the **namespace export** command is executed. Each *pattern* may contain glob-style special characters, but it may not include any namespace qualifiers. That is, the pattern can only specify commands in the current (exporting) namespace. Each *pattern* is appended onto the namespace's list of export patterns. If the **-clear** flag is given, the namespace's export pattern list is reset to empty before any *pattern* arguments are appended. If no *pattern*s are given and the **-clear** flag is not given, this command returns the namespace's current export list.

namespace forget ?*pattern pattern ...*?
Removes previously imported commands from a namespace. Each *pattern* is a simple or qualified name such as **x**, **foo::x** or **a::b::p***. Qualified names contain double colons (**::**) and qualify a name with the name of one or more namespaces. Each "qualified pattern" is qualified with the name of an exporting namespace and may have glob-style special characters in the command name at the end of the qualified name. Glob characters may not appear in a namespace name. For each "simple pattern" this command deletes the matching commands of the current namespace that were imported from a different namespace. For "qualified patterns", this command first finds the matching exported commands. It then checks whether any of those commands were previously imported by the current namespace. If so, this command deletes the corresponding imported commands. In effect, this undoes the action of a **namespace import** command.

namespace import ?-force? ?*pattern pattern ...*?
Imports commands into a namespace, or queries the set of imported commands in a namespace. When no arguments are present, **namespace import** returns the list of commands in the current namespace that have been imported from other namespaces. The commands in the returned list are in the format of simple names, with no namespace

qualifiers at all. This format is suitable for composition with **namespace forget** (see Examples below).

When *pattern* arguments are present, each *pattern* is a qualified name like **foo::x** or **a::p***. That is, it includes the name of an exporting namespace and may have glob-style special characters in the command name at the end of the qualified name. Glob characters may not appear in a namespace name. When the namespace name is not fully qualified (i.e., does not start with a namespace separator) it is resolved as a namespace name in the way described in the Name resolution section; it is an error if no namespace with that name can be found.

All the commands that match a *pattern* string and which are currently exported from their namespace are added to the current namespace. This is done by creating a new command in the current namespace that points to the exported command in its original namespace; when the new imported command is called, it invokes the exported command. This command normally returns an error if an imported command conflicts with an existing command. However, if the **-force** option is given, imported commands will silently replace existing commands. The **namespace import** command has snapshot semantics: that is, only requested commands that are currently defined in the exporting namespace are imported. In other words, you can import only the commands that are in a namespace at the time when the **namespace import** command is executed. If another command is defined and exported in this namespace later on, it will not be imported.

namespace inscope *namespace script* ?*arg ...*?

Executes a script in the context of the specified *namespace*. This command is not expected to be used directly by programmers; calls to it are generated implicitly when applications use **namespace code** commands to create callback scripts that the applications then register with, e.g., Tk widgets. The **namespace inscope** command is much like the **namespace eval** command except that the *namespace* must already exist, and **namespace inscope** appends additional *arg*s as proper list elements.

```
namespace inscope ::foo $script $x $y $z
```

is equivalent to

```
namespace eval ::foo [concat $script [list $x $y $z]]
```

thus additional arguments will not undergo a second round of substitution, as is the case with **namespace eval**.

namespace origin *command*

Returns the fully-qualified name of the original command to which the imported command *command* refers. When a command is imported into a namespace, a new command is created in that namespace that points to the actual command in the exporting namespace. If a command is imported into a sequence of namespaces *a, b,...,n* where each successive namespace just imports the command from the previous namespace, this command returns the fully-qualified name of the original command in the first namespace, *a*. If *command* does not refer to an imported command, the command's own fully-qualified name is returned.

namespace parent ?*namespace*?

Returns the fully-qualified name of the parent namespace for namespace *namespace*. If

namespace is not specified, the fully-qualified name of the current namespace's parent is returned.

namespace path ?*namespaceList*?
Returns the command resolution path of the current namespace. If *namespaceList* is specified as a list of named namespaces, the current namespace's command resolution path is set to those namespaces and returns the empty list. The default command resolution path is always empty. See the section Name resolution below for an explanation of the rules regarding name resolution.

namespace qualifiers *string*
Returns any leading namespace qualifiers for *string*. Qualifiers are namespace names separated by double colons (**::**). For the *string* **::foo::bar::x**, this command returns **::foo::bar**, and for **::** it returns an empty string. This command is the complement of the **namespace tail** command. Note that it does not check whether the namespace names are, in fact, the names of currently defined namespaces.

namespace tail *string*
Returns the simple name at the end of a qualified string. Qualifiers are namespace names separated by double colons (**::**). For the *string* **::foo::bar::x**, this command returns **x**, and for **::** it returns an empty string. This command is the complement of the **namespace qualifiers** command. It does not check whether the namespace names are, in fact, the names of currently defined namespaces.

namespace upvar *namespace ?otherVar myVar ...?*
This command arranges for zero or more local variables in the current procedure to refer to variables in *namespace*. The namespace name is resolved as described in section Name resolution. The command **namespace upvar $ns a b** has the same behaviour as **upvar 0 ${ns}::a b**, with the sole exception of the resolution rules used for qualified namespace or variable names. **namespace upvar** returns an empty string.

namespace unknown ?*script*?
Sets or returns the unknown command handler for the current namespace. The handler is invoked when a command called from within the namespace cannot be found in the current namespace, the namespace's path nor in the global namespace. The *script* argument, if given, should be a well formed list representing a command name and optional arguments. When the handler is invoked, the full invocation line will be appended to the script and the result evaluated in the context of the namespace. The default handler for all namespaces is **::unknown**. If no argument is given, it returns the handler for the current namespace.

namespace which ?-command? ?-variable? *name*
Looks up *name* as either a command or variable and returns its fully-qualified name. For example, if *name* does not exist in the current namespace but does exist in the global namespace, this command returns a fully-qualified name in the global namespace. If the command or variable does not exist, this command returns an empty string. If the variable has been created but not defined, such as with the variable command or through a trace on the variable, this command will return the fully-qualified name of the variable. If no flag is given, *name* is treated as a command name. See the section Name resolution below for an explanation of the rules regarding name resolution.

What is a namespace?

A namespace is a collection of commands and variables. It encapsulates the commands and variables to ensure that they will not interfere with the commands and variables of other namespaces. Tcl has always had one such collection, which we refer to as the *global namespace*. The global namespace holds all global variables and commands. The **namespace eval** command lets you create new namespaces. For example,

```
namespace eval Counter {
    namespace export bump
    variable num 0

    proc bump {} {
        variable num
        incr num
    }
}
```

creates a new namespace containing the variable **num** and the procedure **bump**. The commands and variables in this namespace are separate from other commands and variables in the same program. If there is a command named **bump** in the global namespace, for example, it will be different from the command **bump** in the **Counter** namespace.

Namespace variables resemble global variables in Tcl. They exist outside of the procedures in a namespace but can be accessed in a procedure via the variable command, as shown in the example above.

Namespaces are dynamic. You can add and delete commands and variables at any time, so you can build up the contents of a namespace over time using a series of **namespace eval** commands. For example, the following series of commands has the same effect as the namespace definition shown above:

```
namespace eval Counter {
    variable num 0
    proc bump {} {
        variable num
        return [incr num]
    }
}
namespace eval Counter {
    proc test {args} {
        return $args
    }
}
namespace eval Counter {
     rename test ""
}
```

Note that the **test** procedure is added to the **Counter** namespace, and later removed via the rename command.

Namespaces can have other namespaces within them, so they nest hierarchically. A nested

namespace is encapsulated inside its parent namespace and can not interfere with other namespaces.

Qualified names

Each namespace has a textual name such as history or **::safe::interp**. Since namespaces may nest, qualified names are used to refer to commands, variables, and child namespaces contained inside namespaces. Qualified names are similar to the hierarchical path names for Unix files or Tk widgets, except that **::** is used as the separator instead of **/** or**.**. The topmost or global namespace has the name "" (i.e., an empty string), although **::** is a synonym. As an example, the name **::safe::interp::create** refers to the command **create** in the namespace interp that is a child of namespace **::safe**, which in turn is a child of the global namespace, **::**.

If you want to access commands and variables from another namespace, you must use some extra syntax. Names must be qualified by the namespace that contains them. From the global namespace, we might access the **Counter** procedures like this:

```
Counter::bump 5
Counter::Reset
```

We could access the current count like this:

```
puts "count = $Counter::num"
```

When one namespace contains another, you may need more than one qualifier to reach its elements. If we had a namespace **Foo** that contained the namespace **Counter**, you could invoke its **bump** procedure from the global namespace like this:

```
Foo::Counter::bump 3
```

You can also use qualified names when you create and rename commands. For example, you could add a procedure to the **Foo** namespace like this:

```
proc Foo::Test {args} {return $args}
```

And you could move the same procedure to another namespace like this:

```
rename Foo::Test Bar::Test
```

There are a few remaining points about qualified names that we should cover. Namespaces have nonempty names except for the global namespace. **::** is disallowed in simple command, variable, and namespace names except as a namespace separator. Extra colons in any separator part of a qualified name are ignored; i.e. two or more colons are treated as a namespace separator. A trailing **::** in a qualified variable or command name refers to the variable or command named {}. However, a trailing **::** in a qualified namespace name is ignored.

Name resolution

In general, all Tcl commands that take variable and command names support qualified names. This means you can give qualified names to such commands as set, proc, rename, and interp alias. If you provide a fully-qualified name that starts with a **::**, there is no question about what command, variable, or namespace you mean. However, if the name does not start with a **::** (i.e., is *relative*), Tcl follows basic rules for looking it up:

- **Variable names** are always resolved by looking first in the current namespace, and then in the global namespace.
- **Command names** are always resolved by looking in the current namespace first. If not found there, they are searched for in every namespace on the current namespace's command path (which is empty by default). If not found there, command names are looked up in the global namespace (or, failing that, are processed by the appropriate **namespace unknown** handler.)
- **Namespace names** are always resolved by looking in only the current namespace.

In the following example,

```
set traceLevel 0
namespace eval Debug {
    printTrace $traceLevel
}
```

Tcl looks for **traceLevel** in the namespace **Debug** and then in the global namespace. It looks up the command **printTrace** in the same way. If a variable or command name is not found in either context, the name is undefined. To make this point absolutely clear, consider the following example:

```
set traceLevel 0
namespace eval Foo {
    variable traceLevel 3

    namespace eval Debug {
        printTrace $traceLevel
    }
}
```

Here Tcl looks for **traceLevel** first in the namespace **Foo::Debug**. Since it is not found there, Tcl then looks for it in the global namespace. The variable **Foo::traceLevel** is completely ignored during the name resolution process.

You can use the **namespace which** command to clear up any question about name resolution. For example, the command:

```
namespace eval Foo::Debug {namespace which -variable traceLevel}
```

returns **::traceLevel**. On the other hand, the command,

```
namespace eval Foo {namespace which -variable traceLevel}
```

returns **::Foo::traceLevel**.

As mentioned above, namespace names are looked up differently than the names of variables and commands. Namespace names are always resolved in the current namespace. This means, for example, that a **namespace eval** command that creates a new namespace always creates a child of the current namespace unless the new namespace name begins with **::**.

Tcl has no access control to limit what variables, commands, or namespaces you can reference. If you provide a qualified name that resolves to an element by the name resolution rule above, you can access the element.

You can access a namespace variable from a procedure in the same namespace by using the

variable command. Much like the global command, this creates a local link to the namespace variable. If necessary, it also creates the variable in the current namespace and initializes it. Note that the global command only creates links to variables in the global namespace. It is not necessary to use a variable command if you always refer to the namespace variable using an appropriate qualified name.

Importing commands

Namespaces are often used to represent libraries. Some library commands are used so frequently that it is a nuisance to type their qualified names. For example, suppose that all of the commands in a package like BLT are contained in a namespace called **Blt**. Then you might access these commands like this:

```
Blt::graph .g -background red
Blt::table . .g 0,0
```

If you use the **graph** and **table** commands frequently, you may want to access them without the **Blt::** prefix. You can do this by importing the commands into the current namespace, like this:

```
namespace import Blt::*
```

This adds all exported commands from the **Blt** namespace into the current namespace context, so you can write code like this:

```
graph .g -background red
table . .g 0,0
```

The **namespace import** command only imports commands from a namespace that that namespace exported with a **namespace export** command.

Importing *every* command from a namespace is generally a bad idea since you do not know what you will get. It is better to import just the specific commands you need. For example, the command

```
namespace import Blt::graph Blt::table
```

imports only the **graph** and **table** commands into the current context.

If you try to import a command that already exists, you will get an error. This prevents you from importing the same command from two different packages. But from time to time (perhaps when debugging), you may want to get around this restriction. You may want to reissue the **namespace import** command to pick up new commands that have appeared in a namespace. In that case, you can use the **-force** option, and existing commands will be silently overwritten:

```
namespace import -force Blt::graph Blt::table
```

If for some reason, you want to stop using the imported commands, you can remove them with a **namespace forget** command, like this:

```
namespace forget Blt::*
```

This searches the current namespace for any commands imported from **Blt**. If it finds any, it removes them. Otherwise, it does nothing. After this, the **Blt** commands must be accessed with

the **Blt::** prefix.

When you delete a command from the exporting namespace like this:

```
rename Blt::graph ""
```

the command is automatically removed from all namespaces that import it.

Exporting commands

You can export commands from a namespace like this:

```
namespace eval Counter {
    namespace export bump reset
    variable Num 0
    variable Max 100

    proc bump {{by 1}} {
        variable Num
        incr Num $by
        Check
        return $Num
    }
    proc reset {} {
        variable Num
        set Num 0
    }
    proc Check {} {
        variable Num
        variable Max
        if {$Num > $Max} {
            error "too high!"
        }
    }
}
```

The procedures **bump** and **reset** are exported, so they are included when you import from the **Counter** namespace, like this:

```
namespace import Counter::*
```

However, the **Check** procedure is not exported, so it is ignored by the import operation.

The **namespace import** command only imports commands that were declared as exported by their namespace. The **namespace export** command specifies what commands may be imported by other namespaces. If a **namespace import** command specifies a command that is not exported, the command is not imported.

Scoped scripts

The **namespace code** command is the means by which a script may be packaged for evaluation in a namespace other than the one in which it was created. It is used most often to create event handlers, Tk bindings, and traces for evaluation in the global context. For instance,

the following code indicates how to direct a variable trace callback into the current namespace:

```
namespace eval a {
    variable b
    proc theTraceCallback { n1 n2 op } {
        upvar 1 $n1 var
        puts "the value of $n1 has changed to $var"
        return
    }
    trace add variable b write [namespace code theTraceCallback]
}
set a::b c
```

When executed, it prints the message:

```
the value of a::b has changed to c
```

Ensembles

The **namespace ensemble** is used to create and manipulate ensemble commands, which are commands formed by grouping subcommands together. The commands typically come from the current namespace when the ensemble was created, though this is configurable. Note that there may be any number of ensembles associated with any namespace (including none, which is true of all namespaces by default), though all the ensembles associated with a namespace are deleted when that namespace is deleted. The link between an ensemble command and its namespace is maintained however the ensemble is renamed.

Three subcommands of the **namespace ensemble** command are defined:

namespace ensemble create ?*option value ...*?
: Creates a new ensemble command linked to the current namespace, returning the fully qualified name of the command created. The arguments to **namespace ensemble create** allow the configuration of the command as if with the **namespace ensemble configure** command. If not overridden with the **-command** option, this command creates an ensemble with exactly the same name as the linked namespace. See the section Ensemble options below for a full list of options supported and their effects.

namespace ensemble configure *command* ?*option*? ?*value ...*?
: Retrieves the value of an option associated with the ensemble command named *command*, or updates some options associated with that ensemble command. See the section Ensemble options below for a full list of options supported and their effects.

namespace ensemble exists *command*
: Returns a boolean value that describes whether the command *command* exists and is an ensemble command. This command only ever returns an error if the number of arguments to the command is wrong.

When called, an ensemble command takes its first argument and looks it up (according to the rules described below) to discover a list of words to replace the ensemble command and subcommand with. The resulting list of words is then evaluated (with no further substitutions) as if that was what was typed originally (i.e. by passing the list of words through Tcl_EvalObjv) and returning the result of the command. Note that it is legal to make the target of an ensemble

rewrite be another (or even the same) ensemble command. The ensemble command will not be visible through the use of the uplevel or info level commands.

Ensemble options

The following options, supported by the **namespace ensemble create** and **namespace ensemble configure** commands, control how an ensemble command behaves:

-map
: When non-empty, this option supplies a dictionary that provides a mapping from subcommand names to a list of prefix words to substitute in place of the ensemble command and subcommand words (in a manner similar to an alias created with interp alias; the words are not reparsed after substitution); if the first word of any target is not fully qualified when set, it is assumed to be relative to the *current* namespace and changed to be exactly that (that is, it is always fully qualified when read). When this option is empty, the mapping will be from the local name of the subcommand to its fully-qualified name. Note that when this option is non-empty and the **-subcommands** option is empty, the ensemble subcommand names will be exactly those words that have mappings in the dictionary.

-parameters
: This option gives a list of named arguments (the names being used during generation of error messages) that are passed by the caller of the ensemble between the name of the ensemble and the subcommand argument. By default, it is the empty list.

-prefixes
: This option (which is enabled by default) controls whether the ensemble command recognizes unambiguous prefixes of its subcommands. When turned off, the ensemble command requires exact matching of subcommand names.

-subcommands
: When non-empty, this option lists exactly what subcommands are in the ensemble. The mapping for each of those commands will be either whatever is defined in the **-map** option, or to the command with the same name in the namespace linked to the ensemble. If this option is empty, the subcommands of the namespace will either be the keys of the dictionary listed in the **-map** option or the exported commands of the linked namespace at the time of the invocation of the ensemble command.

-unknown
: When non-empty, this option provides a partial command (to which all the words that are arguments to the ensemble command, including the fully-qualified name of the ensemble, are appended) to handle the case where an ensemble subcommand is not recognized and would otherwise generate an error. When empty (the default) an error (in the style of Tcl_GetIndexFromObj) is generated whenever the ensemble is unable to determine how to implement a particular subcommand. See Unknown handler behaviour for more details.

The following extra option is allowed by **namespace ensemble create**:

-command
: This write-only option allows the name of the ensemble created by **namespace ensemble create** to be anything in any existing namespace. The default value for this option is the fully-qualified name of the namespace in which the **namespace ensemble create** command is invoked.

The following extra option is allowed by **namespace ensemble configure**:

-namespace
This read-only option allows the retrieval of the fully-qualified name of the namespace which the ensemble was created within.

Unknown handler behaviour

If an unknown handler is specified for an ensemble, that handler is called when the ensemble command would otherwise return an error due to it being unable to decide which subcommand to invoke. The exact conditions under which that occurs are controlled by the **-subcommands**, **-map** and **-prefixes** options as described above.

To execute the unknown handler, the ensemble mechanism takes the specified **-unknown** option and appends each argument of the attempted ensemble command invocation (including the ensemble command itself, expressed as a fully qualified name). It invokes the resulting command in the scope of the attempted call. If the execution of the unknown handler terminates normally, the ensemble engine reparses the subcommand (as described below) and tries to dispatch it again, which is ideal for when the ensemble's configuration has been updated by the unknown subcommand handler. Any other kind of termination of the unknown handler is treated as an error.

The result of the unknown handler is expected to be a list (it is an error if it is not). If the list is an empty list, the ensemble command attempts to look up the original subcommand again and, if it is not found this time, an error will be generated just as if the **-unknown** handler was not there (i.e. for any particular invocation of an ensemble, its unknown handler will be called at most once.) This makes it easy for the unknown handler to update the ensemble or its backing namespace so as to provide an implementation of the desired subcommand and reparse.

When the result is a non-empty list, the words of that list are used to replace the ensemble command and subcommand, just as if they had been looked up in the **-map**. It is up to the unknown handler to supply all namespace qualifiers if the implementing subcommand is not in the namespace of the caller of the ensemble command. Also note that when ensemble commands are chained (e.g. if you make one of the commands that implement an ensemble subcommand into an ensemble, in a manner similar to the **text** widget's tag and mark subcommands) then the rewrite happens in the context of the caller of the outermost ensemble. That is to say that ensembles do not in themselves place any namespace contexts on the Tcl call stack.

Where an empty **-unknown** handler is given (the default), the ensemble command will generate an error message based on the list of commands that the ensemble has defined (formatted similarly to the error message from Tcl_GetIndexFromObj). This is the error that will be thrown when the subcommand is still not recognized during reparsing. It is also an error for an **-unknown** handler to delete its namespace.

Examples

Create a namespace containing a variable and an exported command:

```
namespace eval foo {
    variable bar 0
    proc grill {} {
        variable bar
        puts "called [incr bar] times"
```

```
    }
    namespace export grill
}
```

Call the command defined in the previous example in various ways.

```
# Direct call
::foo::grill

# Use the command resolution path to find the name
namespace eval boo {
    namespace path ::foo
    grill
}

# Import into current namespace, then call local alias
namespace import foo::grill
grill

# Create two ensembles, one with the default name and one with a
# specified name.  Then call through the ensembles.
namespace eval foo {
    namespace ensemble create
    namespace ensemble create -command ::foobar
}
foo grill
foobar grill
```

Look up where the command imported in the previous example came from:

```
puts "grill came from [namespace origin grill]"
```

Remove all imported commands from the current namespace:

```
namespace forget {*}[namespace import]
```

Create an ensemble for simple working with numbers, using the -parameters option to allow the operator to be put between the first and second arguments.

```
namespace eval do {
    namespace export *
    namespace ensemble create -parameters x
    proc plus  {x y} {expr { $x + $y }}
    proc minus {x y} {expr { $x - $y }}
}

# In use, the ensemble works like this:
puts [do 1 plus [do 9 minus 7]]
```

Copyright (c) 1993-1997 Bell Labs Innovations for Lucent Technologies

Copyright (c) 1997 Sun Microsystems, Inc.
Copyright (c) 2000 Scriptics Corporation.
Copyright (c) 2004-2005 Donal K. Fellows.

next

Keywords: next, call, method, method chain

Name

next, nextto - invoke superclass method implementations

Synopsis

package require TclOO

next ?*arg* ...?
nextto *class* ?*arg* ...?

Description

The **next** command is used to call implementations of a method by a class, superclass or mixin that are overridden by the current method. It can only be used from within a method. It is also used within filters to indicate the point where a filter calls the actual implementation (the filter may decide to not go along the chain, and may process the results of going along the chain of methods as it chooses). The result of the **next** command is the result of the next method in the method chain; if there are no further methods in the method chain, the result of **next** will be an error. The arguments, *arg*, to **next** are the arguments to pass to the next method in the chain.

The **nextto** command is the same as the **next** command, except that it takes an additional *class* argument that identifies a class whose implementation of the current method chain (see info object **call**) should be used; the method implementation selected will be the one provided by the given class, and it must refer to an existing non-filter invocation that lies further along the chain than the current implementation.

The method chain

When a method of an object is invoked, things happen in several stages:

1. The structure of the object, its class, superclasses, filters, and mixins, are examined to build a *method chain*, which contains a list of method implementations to invoke.
2. The first method implementation on the chain is invoked.
3. If that method implementation invokes the **next** command, the next method implementation is invoked (with its arguments being those that were passed to **next**).
4. The result from the overall method call is the result from the outermost method implementation; inner method implementations return their results through **next**.
5. The method chain is cached for future use.

Method search order

When constructing the method chain, method implementations are searched for in the following order:

1. In the classes mixed into the object, in class traversal order. The list of mixins is checked in natural order.
2. In the classes mixed into the classes of the object, with sources of mixing in being searched in

class traversal order. Within each class, the list of mixins is processed in natural order.
3. In the object itself.
4. In the object's class.
5. In the superclasses of the class, following each superclass in a depth-first fashion in the natural order of the superclass list.

Any particular method implementation always comes as *late* in the resulting list of implementations as possible; this means that if some class, A, is both mixed into a class, B, and is also a superclass of B, the instances of B will always treat A as a superclass from the perspective of inheritance. This is true even when the multiple inheritance is processed indirectly.

Filters

When an object has a list of filter names set upon it, or is an instance of a class (or has mixed in a class) that has a list of filter names set upon it, before every invocation of any method the filters are processed. Filter implementations are found in class traversal order, as are the lists of filter names (each of which is traversed in natural list order). Explicitly invoking a method used as a filter will cause that method to be invoked twice, once as a filter and once as a normal method.

Each filter should decide for itself whether to permit the execution to go forward to the proper implementation of the method (which it does by invoking the **next** command as filters are inserted into the front of the method call chain) and is responsible for returning the result of **next**.

Filters are invoked when processing an invocation of the **unknown** method because of a failure to locate a method implementation, but *not* when invoking either constructors or destructors. (Note however that the **destroy** method is a conventional method, and filters are invoked as normal when it is called.)

Examples

This example demonstrates how to use the **next** command to call the (super)class's implementation of a method. The script:

```
oo::class create theSuperclass {
    method example {args} {
        puts "in the superclass, args = $args"
    }
}
oo::class create theSubclass {
    superclass theSuperclass
    method example {args} {
        puts "before chaining from subclass, args = $args"
        next a {*}$args b
        next pureSynthesis
        puts "after chaining from subclass"
    }
}
theSubclass create obj
```

```
oo::objdefine obj method example args {
    puts "per-object method, args = $args"
    next x {*}$args y
    next
}
obj example 1 2 3
```

prints the following:

```
per-object method, args = 1 2 3
before chaining from subclass, args = x 1 2 3 y
in the superclass, args = a x 1 2 3 y b
in the superclass, args = pureSynthesis
after chaining from subclass
before chaining from subclass, args =
in the superclass, args = a b
in the superclass, args = pureSynthesis
after chaining from subclass
```

This example demonstrates how to build a simple cache class that applies memoization to all the method calls of the objects it is mixed into, and shows how it can make a difference to computation times:

```
oo::class create cache {
    filter Memoize
    method Memoize args {
        # Do not filter the core method implementations
        if {[lindex [self target] 0] eq "::oo::object"} {
            return [next {*}$args]
        }

        # Check if the value is already in the cache
        my variable ValueCache
        set key [self target],$args
        if {[info exist ValueCache($key)]} {
            return $ValueCache($key)
        }

        # Compute value, insert into cache, and return it
        return [set ValueCache($key) [next {*}$args]]
    }
    method flushCache {} {
        my variable ValueCache
        unset ValueCache
        # Skip the caching
        return -level 2 ""
    }
}
```

```
oo::object create demo
oo::objdefine demo {
    mixin cache
    method compute {a b c} {
        after 3000 ;# Simulate deep thought
        return [expr {$a + $b * $c}]
    }
    method compute2 {a b c} {
        after 3000 ;# Simulate deep thought
        return [expr {$a * $b + $c}]
    }
}

puts [demo compute  1 2 3]      → prints "7" after delay
puts [demo compute2 4 5 6]      → prints "26" after delay
puts [demo compute  1 2 3]      → prints "7" instantly
puts [demo compute2 4 5 6]      → prints "26" instantly
puts [demo compute  4 5 6]      → prints "34" after delay
puts [demo compute  4 5 6]      → prints "34" instantly
puts [demo compute  1 2 3]      → prints "7" instantly
demo flushCache
puts [demo compute  1 2 3]      → prints "7" after delay
```

Copyright (c) 2007 Donal K. Fellows

object

Keywords: object, base class, class, root class

Name

oo::object - root class of the class hierarchy

Synopsis

package require TclOO

oo::object *method* ?*arg* ...?

Class hierarchy

```
oo::object
```

Description

The **oo::object** class is the root class of the object hierarchy; every object is an instance of this class. Since classes are themselves objects, they are instances of this class too. Objects are always referred to by their name, and may be renamed while maintaining their identity.

Instances of objects may be made with either the **create** or **new** methods of the **oo::object** object itself, or by invoking those methods on any of the subclass objects; see oo::class for more details. The configuration of individual objects (i.e., instance-specific methods, mixed-in classes,

etc.) may be controlled with the oo::objdefine command.

Each object has a unique namespace associated with it, the instance namespace. This namespace holds all the instance variables of the object, and will be the current namespace whenever a method of the object is invoked (including a method of the class of the object). When the object is destroyed, its instance namespace is deleted. The instance namespace contains the object's my command, which may be used to invoke non-exported methods of the object or to create a reference to the object for the purpose of invocation which persists across renamings of the object.

Constructor

The **oo::object** class does not define an explicit constructor.

Destructor

The **oo::object** class does not define an explicit destructor.

Exported methods

The **oo::object** class supports the following exported methods:

***obj* destroy**

This method destroys the object, *obj*, that it is invoked upon, invoking any destructors on the object's class in the process. It is equivalent to using rename to delete the object command. The result of this method is always the empty string.

Non-exported methods

The **oo::object** class supports the following non-exported methods:

***obj* eval ?*arg ...*?**

This method concatenates the arguments, *arg*, as if with concat, and then evaluates the resulting script in the namespace that is uniquely associated with *obj*, returning the result of the evaluation.

Note that object-internal commands such as my and self can be invoked in this context.

***obj* unknown ?*methodName*? ?*arg ...*?**

This method is called when an attempt to invoke the method *methodName* on object *obj* fails. The arguments that the user supplied to the method are given as *arg* arguments. If methodName is absent, the object was invoked with no method name at all (or any other arguments). The default implementation (i.e., the one defined by the **oo::object** class) generates a suitable error, detailing what methods the object supports given whether the object was invoked by its public name or through the my command.

***obj* variable ?*varName ...*?**

This method arranges for each variable called *varName* to be linked from the object *obj*'s unique namespace into the caller's context. Thus, if it is invoked from inside a procedure then the namespace variable in the object is linked to the local variable in the procedure. Each *varName* argument must not have any namespace separators in it. The result is the empty string.

obj* varname *varName

This method returns the globally qualified name of the variable *varName* in the unique namespace for the object *obj*.

obj sourceObjectName
: This method is used by the oo::object command to copy the state of one object to another. It is responsible for copying the procedures and variables of the namespace of the source object (sourceObjectName) to the current object. It does not copy any other types of commands or any traces on the variables; that can be added if desired by overriding this method in a subclass.

Examples

This example demonstrates basic use of an object.

```
set obj [oo::object new]
$obj foo             → error "unknown method foo"
oo::objdefine $obj method foo {} {
    my variable count
    puts "bar[incr count]"
}
$obj foo             → prints "bar1"
$obj foo             → prints "bar2"
$obj variable count  → error "unknown method variable"
$obj destroy
$obj foo             → error "unknown command obj"
```

Copyright (c) 2007-2008 Donal K. Fellows

open

Keywords: open, access mode, append, create, file, non-blocking, permissions, pipeline, process, serial

Name

open - Open a file-based or command pipeline channel

Synopsis

open *fileName*
open *fileName access*
open *fileName access permissions*

Description

This command opens a file, serial port, or command pipeline and returns a channel identifier that may be used in future invocations of commands like read, puts, and close. If the first character of *fileName* is not **|** then the command opens a file: *fileName* gives the name of the file to open, and it must conform to the conventions described in the filename manual entry.

The *access* argument, if present, indicates the way in which the file (or command pipeline) is to be accessed. In the first form *access* may have any of the following values:

r
: Open the file for reading only; the file must already exist. This is the default value if *access* is not specified.

r+

Open the file for both reading and writing; the file must already exist.

w
Open the file for writing only. Truncate it if it exists. If it does not exist, create a new file.

w+
Open the file for reading and writing. Truncate it if it exists. If it does not exist, create a new file.

a
Open the file for writing only. If the file does not exist, create a new empty file. Set the file pointer to the end of the file prior to each write.

a+
Open the file for reading and writing. If the file does not exist, create a new empty file. Set the initial access position to the end of the file.

All of the legal *access* values above may have the character **b** added as the second or third character in the value to indicate that the opened channel should be configured as if with the fconfigure **-translation binary** option, making the channel suitable for reading or writing of binary data.

In the second form, *access* consists of a list of any of the following flags, most of which have the standard POSIX meanings. One of the flags must be either **RDONLY**, **WRONLY** or **RDWR**.

RDONLY
Open the file for reading only.

WRONLY
Open the file for writing only.

RDWR
Open the file for both reading and writing.

APPEND
Set the file pointer to the end of the file prior to each write.

BINARY
Configure the opened channel with the **-translation binary** option.

CREAT
Create the file if it does not already exist (without this flag it is an error for the file not to exist).

EXCL
If **CREAT** is also specified, an error is returned if the file already exists.

NOCTTY
If the file is a terminal device, this flag prevents the file from becoming the controlling terminal of the process.

NONBLOCK
Prevents the process from blocking while opening the file, and possibly in subsequent I/O operations. The exact behavior of this flag is system- and device-dependent; its use is discouraged (it is better to use the fconfigure command to put a file in nonblocking mode). For details refer to your system documentation on the **open** system call's **O_NONBLOCK** flag.

TRUNC
If the file exists it is truncated to zero length.

If a new file is created as part of opening it, *permissions* (an integer) is used to set the permissions for the new file in conjunction with the process's file mode creation mask. *Permissions* defaults to 0666.

Command pipelines

If the first character of *fileName* is "**|**" then the remaining characters of *fileName* are treated as a list of arguments that describe a command pipeline to invoke, in the same style as the arguments for exec. In this case, the channel identifier returned by **open** may be used to write to the command's input pipe or read from its output pipe, depending on the value of *access*. If write-only access is used (e.g. *access* is "**w**"), then standard output for the pipeline is directed to the current standard output unless overridden by the command. If read-only access is used (e.g. *access* is "**r**"), standard input for the pipeline is taken from the current standard input unless overridden by the command. The id of the spawned process is accessible through the pid command, using the channel id returned by **open** as argument.

If the command (or one of the commands) executed in the command pipeline returns an error (according to the definition in exec), a Tcl error is generated when close is called on the channel unless the pipeline is in non-blocking mode then no exit status is returned (a silent close with -blocking 0).

It is often useful to use the fileevent command with pipelines so other processing may happen at the same time as running the command in the background.

Serial communications

If *fileName* refers to a serial port, then the specified serial port is opened and initialized in a platform-dependent manner. Acceptable values for the *fileName* to use to open a serial port are described in the PORTABILITY ISSUES section.

The fconfigure command can be used to query and set additional configuration options specific to serial ports (where supported):

-mode *baud*,*parity*,*data*,*stop*
: This option is a set of 4 comma-separated values: the baud rate, parity, number of data bits, and number of stop bits for this serial port. The *baud* rate is a simple integer that specifies the connection speed. *Parity* is one of the following letters: **n**, **o**, **e**, **m**, **s**; respectively signifying the parity options of "none", "odd", "even", "mark", or "space". *Data* is the number of data bits and should be an integer from 5 to 8, while *stop* is the number of stop bits and should be the integer 1 or 2.

-handshake *type*
: (Windows and Unix). This option is used to setup automatic handshake control. Note that not all handshake types maybe supported by your operating system. The *type* parameter is case-independent.

  If *type* is **none** then any handshake is switched off. **rtscts** activates hardware handshake. Hardware handshake signals are described below. For software handshake **xonxoff** the handshake characters can be redefined with **-xchar**. An additional hardware handshake **dtrdsr** is available only under Windows. There is no default handshake configuration, the initial value depends on your operating system settings. The **-handshake** option cannot be queried.

-queue
(Windows and Unix). The **-queue** option can only be queried. It returns a list of two integers representing the current number of bytes in the input and output queue respectively.

-timeout *msec*
(Windows and Unix). This option is used to set the timeout for blocking read operations. It specifies the maximum interval between the reception of two bytes in milliseconds. For Unix systems the granularity is 100 milliseconds. The **-timeout** option does not affect write operations or nonblocking reads. This option cannot be queried.

-ttycontrol *{signal boolean signal boolean ...}*
(Windows and Unix). This option is used to setup the handshake output lines (see below) permanently or to send a BREAK over the serial line. The *signal* names are case-independent. **{RTS 1 DTR 0}** sets the RTS output to high and the DTR output to low. The BREAK condition (see below) is enabled and disabled with **{BREAK 1}** and **{BREAK 0}** respectively. It is not a good idea to change the **RTS** (or **DTR**) signal with active hardware handshake **rtscts** (or **dtrdsr**). The result is unpredictable. The **-ttycontrol** option cannot be queried.

-ttystatus
(Windows and Unix). The **-ttystatus** option can only be queried. It returns the current modem status and handshake input signals (see below). The result is a list of signal,value pairs with a fixed order, e.g. **{CTS 1 DSR 0 RING 1 DCD 0}**. The *signal* names are returned upper case.

-xchar *{xonChar xoffChar}*
(Windows and Unix). This option is used to query or change the software handshake characters. Normally the operating system default should be DC1 (0x11) and DC3 (0x13) representing the ASCII standard XON and XOFF characters.

-pollinterval *msec*
(Windows only). This option is used to set the maximum time between polling for fileevents. This affects the time interval between checking for events throughout the Tcl interpreter (the smallest value always wins). Use this option only if you want to poll the serial port more or less often than 10 msec (the default).

-sysbuffer *inSize*
see below ...

-sysbuffer *{inSize outSize}*
(Windows only). This option is used to change the size of Windows system buffers for a serial channel. Especially at higher communication rates the default input buffer size of 4096 bytes can overrun for latent systems. The first form specifies the input buffer size, in the second form both input and output buffers are defined.

-lasterror
(Windows only). This option is query only. In case of a serial communication error, read or puts returns a general Tcl file I/O error. fconfigure **-lasterror** can be called to get a list of error details. See below for an explanation of the various error codes.

Serial port signals

RS-232 is the most commonly used standard electrical interface for serial communications. A negative voltage (-3V..-12V) define a mark (on=1) bit and a positive voltage (+3..+12V) define a space (off=0) bit (RS-232C). The following signals are specified for incoming and outgoing data, status lines and handshaking. Here we are using the terms *workstation* for your computer and

modem for the external device, because some signal names (DCD, RI) come from modems. Of course your external device may use these signal lines for other purposes.

TXD(output)
Transmitted Data: Outgoing serial data.

RXD(input)
Received Data: Incoming serial data.

RTS(output)
Request To Send: This hardware handshake line informs the modem that your workstation is ready to receive data. Your workstation may automatically reset this signal to indicate that the input buffer is full.

CTS(input)
Clear To Send: The complement to RTS. Indicates that the modem is ready to receive data.

DTR(output)
Data Terminal Ready: This signal tells the modem that the workstation is ready to establish a link. DTR is often enabled automatically whenever a serial port is opened.

DSR(input)
Data Set Ready: The complement to DTR. Tells the workstation that the modem is ready to establish a link.

DCD(input)
Data Carrier Detect: This line becomes active when a modem detects a "Carrier" signal.

RI(input)
Ring Indicator: Goes active when the modem detects an incoming call.

BREAK
A BREAK condition is not a hardware signal line, but a logical zero on the TXD or RXD lines for a long period of time, usually 250 to 500 milliseconds. Normally a receive or transmit data signal stays at the mark (on=1) voltage until the next character is transferred. A BREAK is sometimes used to reset the communications line or change the operating mode of communications hardware.

Error codes (Windows only)

A lot of different errors may occur during serial read operations or during event polling in background. The external device may have been switched off, the data lines may be noisy, system buffers may overrun or your mode settings may be wrong. That is why a reliable software should always catch serial read operations. In cases of an error Tcl returns a general file I/O error. Then fconfigure **-lasterror** may help to locate the problem. The following error codes may be returned.

RXOVER
Windows input buffer overrun. The data comes faster than your scripts reads it or your system is overloaded. Use fconfigure **-sysbuffer** to avoid a temporary bottleneck and/or make your script faster.

TXFULL
Windows output buffer overrun. Complement to RXOVER. This error should practically not happen, because Tcl cares about the output buffer status.

OVERRUN

UART buffer overrun (hardware) with data lost. The data comes faster than the system driver receives it. Check your advanced serial port settings to enable the FIFO (16550) buffer and/or setup a lower(1) interrupt threshold value.

RXPARITY
A parity error has been detected by your UART. Wrong parity settings with fconfigure **-mode** or a noisy data line (RXD) may cause this error.

FRAME
A stop-bit error has been detected by your UART. Wrong mode settings with fconfigure **-mode** or a noisy data line (RXD) may cause this error.

BREAK
A BREAK condition has been detected by your UART (see above).

Portability issues

Windows

Valid values for *fileName* to open a serial port are of the form **com***X*, where *X* is a number, generally from 1 to 9. A legacy form accepted as well is **com***X***:**. This notation only works for serial ports from 1 to 9. An attempt to open a serial port that does not exist or has a number greater than 9 will fail. An alternate form of opening serial ports is to use the filename **//./comX**, where X is any number that corresponds to a serial port.

When running Tcl interactively, there may be some strange interactions between the real console, if one is present, and a command pipeline that uses standard input or output. If a command pipeline is opened for reading, some of the lines entered at the console will be sent to the command pipeline and some will be sent to the Tcl evaluator. If a command pipeline is opened for writing, keystrokes entered into the console are not visible until the pipe is closed. These problems only occur because both Tcl and the child application are competing for the console at the same time. If the command pipeline is started from a script, so that Tcl is not accessing the console, or if the command pipeline does not use standard input or output, but is redirected from or to a file, then the above problems do not occur.

Files opened in the "**a**" mode or with the **APPEND** flag set are implemented by seeking immediately before each write, which is not an atomic operation and does not carry the guarantee of strict appending that is present on POSIX platforms.

Unix

Valid values for *fileName* to open a serial port are generally of the form **/dev/tty***X*, where *X* is **a** or **b**, but the name of any pseudo-file that maps to a serial port may be used. Advanced configuration options are only supported for serial ports when Tcl is built to use the POSIX serial interface.

When running Tcl interactively, there may be some strange interactions between the console, if one is present, and a command pipeline that uses standard input. If a command pipeline is opened for reading, some of the lines entered at the console will be sent to the command pipeline and some will be sent to the Tcl evaluator. This problem only occurs because both Tcl and the child application are competing for the console at the same time. If the command pipeline is started from a script, so that Tcl is not accessing the console, or if the command pipeline does not use standard input, but is redirected from a file, then the above problem

does not occur.

See the **PORTABILITY ISSUES** section of the exec command for additional information not specific to command pipelines about executing applications on the various platforms

Examples

Open a file for writing, forcing it to be created and raising an error if it already exists.

```
set myNewFile [open filename.txt {WRONLY CREAT EXCL}]
```

Open a file for writing as a log file.

```
set myLogFile [open filename.log "a"]
fconfigure $myLogFile -buffering line
```

Open a command pipeline and catch any errors:

```
set fl [open "| ls this_file_does_not_exist"]
set data [read $fl]
if {[catch {close $fl} err]} {
    puts "ls command failed: $err"
}
```

Open a command pipeline and read binary data from it. Note the unusual form with "|[list" that handles non-trivial edge cases with arguments that potentially have spaces in.

```
set fl [open |[list create_image_data $input] "rb"]
set binData [read $fl]
close $fl
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

package

Keywords: package, version

Name

package - Facilities for package loading and version control

Synopsis

package forget ?*package package* ...?
package ifneeded *package version* ?*script*?
package names
package present *package* ?*requirement...*?
package present -exact *package version*
package provide *package* ?*version*?
package require *package* ?*requirement...*?
package require -exact *package version*
package unknown ?*command*?
package vcompare *version1 version2*
package versions *package*

package vsatisfies *version requirement...*
package prefer ?**latest**|**stable**?

Description

This command keeps a simple database of the packages available for use by the current interpreter and how to load them into the interpreter. It supports multiple versions of each package and arranges for the correct version of a package to be loaded based on what is needed by the application. This command also detects and reports version clashes. Typically, only the **package require** and **package provide** commands are invoked in normal Tcl scripts; the other commands are used primarily by system scripts that maintain the package database.

The behavior of the **package** command is determined by its first argument. The following forms are permitted:

package forget ?*package package ...*?
: Removes all information about each specified package from this interpreter, including information provided by both **package ifneeded** and **package provide**.

package ifneeded *package version* ?*script*?
: This command typically appears only in system configuration scripts to set up the package database. It indicates that a particular version of a particular package is available if needed, and that the package can be added to the interpreter by executing *script*. The script is saved in a database for use by subsequent **package require** commands; typically, *script* sets up auto-loading for the commands in the package (or calls load and/or source directly), then invokes **package provide** to indicate that the package is present. There may be information in the database for several different versions of a single package. If the database already contains information for *package* and *version*, the new *script* replaces the existing one. If the *script* argument is omitted, the current script for version *version* of package *package* is returned, or an empty string if no **package ifneeded** command has been invoked for this *package* and *version*.

package names
: Returns a list of the names of all packages in the interpreter for which a version has been provided (via **package provide**) or for which a **package ifneeded** script is available. The order of elements in the list is arbitrary.

package present ?-exact? *package* ?*requirement...*?
: This command is equivalent to **package require** except that it does not try and load the package if it is not already loaded.

package provide *package* ?*version*?
: This command is invoked to indicate that version *version* of package *package* is now present in the interpreter. It is typically invoked once as part of an **ifneeded** script, and again by the package itself when it is finally loaded. An error occurs if a different version of *package* has been provided by a previous **package provide** command. If the *version* argument is omitted, then the command returns the version number that is currently provided, or an empty string if no **package provide** command has been invoked for *package* in this interpreter.

package require *package* ?*requirement...*?
: This command is typically invoked by Tcl code that wishes to use a particular version of a particular package. The arguments indicate which package is wanted, and the command ensures that a suitable version of the package is loaded into the interpreter. If the command

succeeds, it returns the version number that is loaded; otherwise it generates an error.

A suitable version of the package is any version which satisfies at least one of the requirements as defined in the section Requirement below. If multiple versions are suitable the implementation with the highest version is chosen. This last part is additionally influenced by the selection mode set with **package prefer**.

In the "stable" selection mode the command will select the highest stable version satisfying the requirements, if any. If no stable version satisfies the requirements, the highest unstable version satisfying the requirements will be selected. In the "latest" selection mode the command will accept the highest version satisfying all the requirements, regardless of its stableness.

If a version of *package* has already been provided (by invoking the **package provide** command), then its version number must satisfy the *requirement*s and the command returns immediately. Otherwise, the command searches the database of information provided by previous **package ifneeded** commands to see if an acceptable version of the package is available. If so, the script for the highest acceptable version number is evaluated in the global namespace; it must do whatever is necessary to load the package, including calling **package provide** for the package. If the **package ifneeded** database does not contain an acceptable version of the package and a **package unknown** command has been specified for the interpreter then that command is evaluated in the global namespace; when it completes, Tcl checks again to see if the package is now provided or if there is a **package ifneeded** script for it. If all of these steps fail to provide an acceptable version of the package, then the command returns an error.

package require -exact *package version*
This form of the command is used when only the given *version* of *package* is acceptable to the caller. This command is equivalent to **package require** *package version-version*.

package unknown ?*command*?
This command supplies a "last resort" command to invoke during **package require** if no suitable version of a package can be found in the **package ifneeded** database. If the *command* argument is supplied, it contains the first part of a command; when the command is invoked during a **package require** command, Tcl appends one or more additional arguments giving the desired package name and requirements. For example, if *command* is **foo bar** and later the command **package require test 2.4** is invoked, then Tcl will execute the command **foo bar test 2.4** to load the package. If no requirements are supplied to the **package require** command, then only the name will be added to invoked command. If the **package unknown** command is invoked without a *command* argument, then the current **package unknown** script is returned, or an empty string if there is none. If *command* is specified as an empty string, then the current **package unknown** script is removed, if there is one.

package vcompare *version1 version2*
Compares the two version numbers given by *version1* and *version2*. Returns -1 if *version1* is an earlier version than *version2*, 0 if they are equal, and 1 if *version1* is later than *version2*.

package versions *package*
Returns a list of all the version numbers of *package* for which information has been provided

by **package ifneeded** commands.

package vsatisfies *version requirement...*
Returns 1 if the *version* satisfies at least one of the given requirements, and 0 otherwise. *requirements* are defined in the Requirement section below.

package prefer ?latest|stable?
With no arguments, the commands returns either "latest" or "stable", whichever describes the current mode of selection logic used by **package require**.

When passed the argument "latest", it sets the selection logic mode to "latest".

When passed the argument "stable", if the mode is already "stable", that value is kept. If the mode is already "latest", then the attempt to set it back to "stable" is ineffective and the mode value remains "latest".

When passed any other value as an argument, raise an invalid argument error.

When an interpreter is created, its initial selection mode value is set to "stable" unless the environment variable **TCL_PKG_PREFER_LATEST** is set (to any value). Otherwise the initial (and permanent) selection mode value is set to "latest".

Version numbers

Version numbers consist of one or more decimal numbers separated by dots, such as 2 or 1.162 or 3.1.13.1. The first number is called the major version number. Larger numbers correspond to later versions of a package, with leftmost numbers having greater significance. For example, version 2.1 is later than 1.3 and version 3.4.6 is later than 3.3.5. Missing fields are equivalent to zeroes: version 1.3 is the same as version 1.3.0 and 1.3.0.0, so it is earlier than 1.3.1 or 1.3.0.2. In addition, the letters "a" (alpha) and/or "b" (beta) may appear exactly once to replace a dot for separation. These letters semantically add a negative specifier into the version, where "a" is -2, and "b" is -1. Each may be specified only once, and "a" or "b" are mutually exclusive in a specifier. Thus 1.3a1 becomes (semantically) 1.3.-2.1, 1.3b1 is 1.3.-1.1. Negative numbers are not directly allowed in version specifiers. A version number not containing the letters "a" or "b" as specified above is called a **stable** version, whereas presence of the letters causes the version to be called is **unstable**. A later version number is assumed to be upwards compatible with an earlier version number as long as both versions have the same major version number. For example, Tcl scripts written for version 2.3 of a package should work unchanged under versions 2.3.2, 2.4, and 2.5.1. Changes in the major version number signify incompatible changes: if code is written to use version 2.1 of a package, it is not guaranteed to work unmodified with either version 1.7.3 or version 3.1.

Package indices

The recommended way to use packages in Tcl is to invoke **package require** and **package provide** commands in scripts, and use the procedure pkg_mkIndex to create package index files. Once you have done this, packages will be loaded automatically in response to **package require** commands. See the documentation for pkg_mkIndex for details.

Requirement

A *requirement* string checks, if a compatible version number of a package is present. Most commands accept a list of requirement strings where the highest suitable version is matched.

Each *requirement* is allowed to have any of the forms:

min
This form is called "min-bounded".

min-
This form is called "min-unbound".

min-max
This form is called "bounded".

where "*min*" and "*max*" are valid version numbers. The legacy syntax is a special case of the extended syntax, keeping backward compatibility. Regarding satisfaction the rules are:

1. The *version* has to pass at least one of the listed *requirement*s to be satisfactory.
2. A version satisfies a "bounded" requirement when
   [a]
   For *min* equal to the *max* if, and only if the *version* is equal to the *min*.
   [b]
   Otherwise if, and only if the *version* is greater than or equal to the *min*, and less than the *max*, where both *min* and *max* have been padded internally with "a0". Note that while the comparison to *min* is inclusive, the comparison to *max* is exclusive.

1. A "min-bounded" requirement is a "bounded" requirement in disguise, with the *max* part implicitly specified as the next higher major version number of the *min* part. A version satisfies it per the rules above.
2. A *version* satisfies a "min-unbound" requirement if, and only if it is greater than or equal to the *min*, where the *min* has been padded internally with "a0". There is no constraint to a maximum.

Examples

To state that a Tcl script requires the Tk and http packages, put this at the top of the script:

```
package require Tk
package require http
```

To test to see if the Snack package is available and load if it is (often useful for optional enhancements to programs where the loss of the functionality is not critical) do this:

```
if {[catch {package require Snack}]} {
    # Error thrown - package not found.
    # Set up a dummy interface to work around the absence
} else {
    # We have the package, configure the app to use it
}
```

Copyright (c) 1996 Sun Microsystems, Inc.

packagens

Keywords: packagens, auto-load, index, package, version

Name
pkg::create - Construct an appropriate 'package ifneeded' command for a given package specification

Synopsis
::pkg::create -name *packageName* **-version** *packageVersion* ?*-load filespec*? ... ?*-source filespec*? ...

Description
::pkg::create is a utility procedure that is part of the standard Tcl library. It is used to create an appropriate package ifneeded command for a given package specification. It can be used to construct a **pkgIndex.tcl** file for use with the package mechanism.

Options
The parameters supported are:

-name *packageName*
: This parameter specifies the name of the package. It is required.

-version *packageVersion*
: This parameter specifies the version of the package. It is required.

-load *filespec*
: This parameter specifies a binary library that must be loaded with the load command. *filespec* is a list with two elements. The first element is the name of the file to load. The second, optional element is a list of commands supplied by loading that file. If the list of procedures is empty or omitted, **::pkg::create** will set up the library for direct loading (see pkg_mkIndex). Any number of **-load** parameters may be specified.

-source *filespec*
: This parameter is similar to the **-load** parameter, except that it specifies a Tcl library that must be loaded with the source command. Any number of **-source** parameters may be specified.

At least one **-load** or **-source** parameter must be given.

Copyright (c) 1998-2000 Scriptics Corporation.

pid
Keywords: pid, file, pipeline, process identifier

Name
pid - Retrieve process identifiers

Synopsis
pid ?*fileId*?

Description
If the *fileId* argument is given then it should normally refer to a process pipeline created with the open command. In this case the **pid** command will return a list whose elements are the process identifiers of all the processes in the pipeline, in order. The list will be empty if *fileId* refers to an

open file that is not a process pipeline. If no *fileId* argument is given then **pid** returns the process identifier of the current process. All process identifiers are returned as decimal strings.

Example

Print process information about the processes in a pipeline using the SysV **ps** program before reading the output of that pipeline:

```
set pipeline [open "| zcat somefile.gz | grep foobar | sort -u"]
# Print process information
exec ps -fp [pid $pipeline] >@stdout
# Print a separator and then the output of the pipeline
puts [string repeat - 70]
puts [read $pipeline]
close $pipeline
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

pkgMkIndex

Keywords: pkgMkIndex, auto-load, index, package, version

Name

pkg_mkIndex - Build an index for automatic loading of packages

Synopsis

pkg_mkIndex ?*options...*? *dir* ?*pattern pattern ...*?

Description

Pkg_mkIndex is a utility procedure that is part of the standard Tcl library. It is used to create index files that allow packages to be loaded automatically when package require commands are executed. To use **pkg_mkIndex**, follow these steps:

1. Create the package(s). Each package may consist of one or more Tcl script files or binary files. Binary files must be suitable for loading with the load command with a single argument; for example, if the file is **test.so** it must be possible to load this file with the command load test.so. Each script file must contain a package provide command to declare the package and version number, and each binary file must contain a call to Tcl_PkgProvide.
2. Create the index by invoking **pkg_mkIndex**. The *dir* argument gives the name of a directory and each *pattern* argument is a glob-style pattern that selects script or binary files in *dir*. The default pattern is ***.tcl** and ***.[info sharedlibextension]**.

   Pkg_mkIndex will create a file **pkgIndex.tcl** in *dir* with package information about all the files given by the *pattern* arguments. It does this by loading each file into a child interpreter and seeing what packages and new commands appear (this is why it is essential to have package provide commands or Tcl_PkgProvide calls in the files, as described above). If you have a package split among scripts and binary files, or if you have dependencies among files, you may have to use the **-load** option or adjust the order in which **pkg_mkIndex** processes the files. See Complex cases below.
3. Install the package as a subdirectory of one of the directories given by the tcl_pkgPath

variable. If **$tcl_pkgPath** contains more than one directory, machine-dependent packages (e.g., those that contain binary shared libraries) should normally be installed under the first directory and machine-independent packages (e.g., those that contain only Tcl scripts) should be installed under the second directory. The subdirectory should include the package's script and/or binary files as well as the **pkgIndex.tcl** file. As long as the package is installed as a subdirectory of a directory in **$tcl_pkgPath** it will automatically be found during package require commands.

If you install the package anywhere else, then you must ensure that the directory containing the package is in the auto_path global variable or an immediate subdirectory of one of the directories in auto_path. **Auto_path** contains a list of directories that are searched by both the auto-loader and the package loader; by default it includes **$tcl_pkgPath**. The package loader also checks all of the subdirectories of the directories in auto_path. You can add a directory to auto_path explicitly in your application, or you can add the directory to your **TCLLIBPATH** environment variable: if this environment variable is present, Tcl initializes auto_path from it during application startup.

4. Once the above steps have been taken, all you need to do to use a package is to invoke package require. For example, if versions 2.1, 2.3, and 3.1 of package **Test** have been indexed by **pkg_mkIndex**, the command **package require Test** will make version 3.1 available and the command **package require -exact Test 2.1** will make version 2.1 available. There may be many versions of a package in the various index files in auto_path, but only one will actually be loaded in a given interpreter, based on the first call to package require. Different versions of a package may be loaded in different interpreters.

Options

The optional switches are:

-direct
: The generated index will implement direct loading of the package upon package require. This is the default.

-lazy
: The generated index will manage to delay loading the package until the use of one of the commands provided by the package, instead of loading it immediately upon package require. This is not compatible with the use of *auto_reset*, and therefore its use is discouraged.

-load *pkgPat*
: The index process will preload any packages that exist in the current interpreter and match *pkgPat* into the child interpreter used to generate the index. The pattern match uses string match rules, but without making case distinctions. See Complex cases below.

-verbose
: Generate output during the indexing process. Output is via the **tclLog** procedure, which by default prints to stderr.

--
: End of the flags, in case *dir* begins with a dash.

Packages and the auto-loader

The package management facilities overlap somewhat with the auto-loader, in that both arrange for files to be loaded on-demand. However, package management is a higher-level mechanism

that uses the auto-loader for the last step in the loading process. It is generally better to index a package with **pkg_mkIndex** rather than auto_mkindex because the package mechanism provides version control: several versions of a package can be made available in the index files, with different applications using different versions based on package require commands. In contrast, auto_mkindex does not understand versions so it can only handle a single version of each package. It is probably not a good idea to index a given package with both **pkg_mkIndex** and auto_mkindex. If you use **pkg_mkIndex** to index a package, its commands cannot be invoked until package require has been used to select a version; in contrast, packages indexed with auto_mkindex can be used immediately since there is no version control.

How it works

Pkg_mkIndex depends on the package unknown command, the package ifneeded command, and the auto-loader. The first time a package require command is invoked, the package unknown script is invoked. This is set by Tcl initialization to a script that evaluates all of the **pkgIndex.tcl** files in the auto_path. The **pkgIndex.tcl** files contain package ifneeded commands for each version of each available package; these commands invoke package provide commands to announce the availability of the package, and they setup auto-loader information to load the files of the package. If the **-lazy** flag was provided when the **pkgIndex.tcl** was generated, a given file of a given version of a given package is not actually loaded until the first time one of its commands is invoked. Thus, after invoking package require you may not see the package's commands in the interpreter, but you will be able to invoke the commands and they will be auto-loaded.

Direct loading

Some packages, for instance packages which use namespaces and export commands or those which require special initialization, might select that their package files be loaded immediately upon package require instead of delaying the actual loading to the first use of one of the package's command. This is the default mode when generating the package index. It can be overridden by specifying the **-lazy** argument.

Complex cases

Most complex cases of dependencies among scripts and binary files, and packages being split among scripts and binary files are handled OK. However, you may have to adjust the order in which files are processed by **pkg_mkIndex**. These issues are described in detail below.

If each script or file contains one package, and packages are only contained in one file, then things are easy. You simply specify all files to be indexed in any order with some glob patterns.

In general, it is OK for scripts to have dependencies on other packages. If scripts contain package require commands, these are stubbed out in the interpreter used to process the scripts, so these do not cause problems. If scripts call into other packages in global code, these calls are handled by a stub unknown command. However, if scripts make variable references to other package's variables in global code, these will cause errors. That is also bad coding style.

If binary files have dependencies on other packages, things can become tricky because it is not possible to stub out C-level APIs such as Tcl_PkgRequire API when loading a binary file. For example, suppose the BLT package requires Tk, and expresses this with a call to Tcl_PkgRequire in its **Blt_Init** routine. To support this, you must run **pkg_mkIndex** in an interpreter that has Tk loaded. You can achieve this with the **-load** *pkgPat* option. If you specify this option,

pkg_mkIndex will load any packages listed by info loaded and that match *pkgPat* into the interpreter used to process files. In most cases this will satisfy the Tcl_PkgRequire calls made by binary files.

If you are indexing two binary files and one depends on the other, you should specify the one that has dependencies last. This way the one without dependencies will get loaded and indexed, and then the package it provides will be available when the second file is processed. You may also need to load the first package into the temporary interpreter used to create the index by using the **-load** flag; it will not hurt to specify package patterns that are not yet loaded.

If you have a package that is split across scripts and a binary file, then you should avoid the **-load** flag. The problem is that if you load a package before computing the index it masks any other files that provide part of the same package. If you must use **-load**, then you must specify the scripts first; otherwise the package loaded from the binary file may mask the package defined by the scripts.

Copyright (c) 1996 Sun Microsystems, Inc.

platform

Keywords: platform, operating system, cpu architecture, architecture

Name

platform - System identification support code and utilities

Synopsis

package require platform ?**1.0.10**?
platform::generic
platform::identify
platform::patterns *identifier*

Description

The **platform** package provides several utility commands useful for the identification of the architecture of a machine running Tcl.

Whilst Tcl provides the tcl_platform array for identifying the current architecture (in particular, the platform and machine elements) this is not always sufficient. This is because (on Unix machines) tcl_platform reflects the values returned by the **uname** command and these are not standardized across platforms and architectures. In addition, on at least one platform (AIX) the **tcl_platform(machine)** contains the CPU serial number.

Consequently, individual applications need to manipulate the values in tcl_platform (along with the output of system specific utilities) - which is both inconvenient for developers, and introduces the potential for inconsistencies in identifying architectures and in naming conventions.

The **platform** package prevents such fragmentation - i.e., it establishes a standard naming convention for architectures running Tcl and makes it more convenient for developers to identify the current architecture a Tcl program is running on.

Commands

platform::identify

This command returns an identifier describing the platform the Tcl core is running on. The returned identifier has the general format *OS*-*CPU*. The *OS* part of the identifier may contain details like kernel version, libc version, etc., and this information may contain dashes as well. The *CPU* part will not contain dashes, making the preceding dash the last dash in the result.

platform::generic
This command returns a simplified identifier describing the platform the Tcl core is running on. In contrast to **platform::identify** it leaves out details like kernel version, libc version, etc. The returned identifier has the general format *OS*-*CPU*.

platform::patterns *identifier*
This command takes an identifier as returned by **platform::identify** and returns a list of identifiers describing compatible architectures.

Example

This can be used to allow an application to be shipped with multiple builds of a shared library, so that the same package works on many versions of an operating system. For example:

```
package require platform
# Assume that app script is .../theapp/bin/theapp.tcl
set binDir [file dirname [file normalize [info script]]]
set libDir [file join $binDir .. lib]
set platLibDir [file join $libDir [platform::identify]]
load [file join $platLibDir support[info sharedlibextension]]
```

Copyright (c) 2006 ActiveState Software Inc

platform_shell

Keywords: platform_shell, operating system, cpu architecture, platform, architecture

Name

platform::shell - System identification support code and utilities

Synopsis

package require platform::shell ?**1.1.4**?
platform::shell::generic *shell*
platform::shell::identify *shell*
platform::shell::platform *shell*

Description

The **platform::shell** package provides several utility commands useful for the identification of the architecture of a specific Tcl shell.

This package allows the identification of the architecture of a specific Tcl shell different from the shell running the package. The only requirement is that the other shell (identified by its path), is actually executable on the current machine.

While for most platform this means that the architecture of the interrogated shell is identical to the architecture of the running shell this is not generally true. A counter example are all platforms which have 32 and 64 bit variants and where a 64bit system is able to run 32bit code. For these running and interrogated shell may have different 32/64 bit settings and thus different

identifiers.

For applications like a code repository it is important to identify the architecture of the shell which will actually run the installed packages, versus the architecture of the shell running the repository software.

Commands

platform::shell::identify *shell*
This command does the same identification as platform::identify, for the specified Tcl shell, in contrast to the running shell.

platform::shell::generic *shell*
This command does the same identification as platform::generic, for the specified Tcl shell, in contrast to the running shell.

platform::shell::platform *shell*
This command returns the contents of **tcl_platform(platform)** for the specified Tcl shell.

Copyright (c) 2006-2008 ActiveState Software Inc

prefix

Keywords: prefix, table lookup

Name

tcl::prefix - facilities for prefix matching

Synopsis

::tcl::prefix all *table string*
::tcl::prefix longest *table string*
::tcl::prefix match ?*option* ...? *table string*

Description

This document describes commands looking up a prefix in a list of strings. The following commands are supported:

::tcl::prefix all *table string*
Returns a list of all elements in *table* that begin with the prefix *string*.

::tcl::prefix longest *table string*
Returns the longest common prefix of all elements in *table* that begin with the prefix *string*.

::tcl::prefix match ?*option ...*? *table string*
If *string* equals one element in *table* or is a prefix to exactly one element, the matched element is returned. If not, the result depends on the **-error** option. (It is recommended that the *table* be sorted before use with this subcommand, so that the list of matches presented in the error message also becomes sorted, though this is not strictly necessary for the operation of this subcommand itself.) The following options are supported:

-exact
Accept only exact matches.

-message *string*
Use *string* in the error message at a mismatch. Default is "option".

-error *options*

The *options* are used when no match is found. If *options* is empty, no error is generated and an empty string is returned. Otherwise the *options* are used as return options when generating the error message. The default corresponds to setting "-level 0". Example: If "**-error** {-errorcode MyError -level 1}" is used, an error would be generated as:

```
return -errorcode MyError -level 1 -code error \
       "ambiguous option ..."
```

Examples

Basic use:

```
namespace import ::tcl::prefix
prefix match {apa bepa cepa} apa
     → apa
prefix match {apa bepa cepa} a
     → apa
prefix match -exact {apa bepa cepa} a
     → bad option "a": must be apa, bepa, or cepa
prefix match -message "switch" {apa ada bepa cepa} a
     → ambiguous switch "a": must be apa, ada, bepa, or cepa
prefix longest {fblocked fconfigure fcopy file fileevent flush} fc
     → fco
prefix all {fblocked fconfigure fcopy file fileevent flush} fc
     → fconfigure fcopy
```

Simplifying option matching:

```
array set opts {-apa 1 -bepa "" -cepa 0}
foreach {arg val} $args {
    set opts([prefix match {-apa -bepa -cepa} $arg]) $val
}
```

Creating a switch that supports prefixes:

```
switch [prefix match {apa bepa cepa} $arg] {
    apa  { }
    bepa { }
    cepa { }
}
```

Copyright (c) 2008 Peter Spjuth pspjuth@users.sourceforge.net

proc

Keywords: proc, argument, procedure

Name

proc - Create a Tcl procedure

Synopsis

proc *name args body*

Description

The **proc** command creates a new Tcl procedure named *name*, replacing any existing command or procedure there may have been by that name. Whenever the new command is invoked, the contents of *body* will be executed by the Tcl interpreter. Normally, *name* is unqualified (does not include the names of any containing namespaces), and the new procedure is created in the current namespace. If *name* includes any namespace qualifiers, the procedure is created in the specified namespace. *Args* specifies the formal arguments to the procedure. It consists of a list, possibly empty, each of whose elements specifies one argument. Each argument specifier is also a list with either one or two fields. If there is only a single field in the specifier then it is the name of the argument; if there are two fields, then the first is the argument name and the second is its default value. Arguments with default values that are followed by non-defaulted arguments become required arguments; enough actual arguments must be supplied to allow all arguments up to and including the last required formal argument.

When *name* is invoked a local variable will be created for each of the formal arguments to the procedure; its value will be the value of corresponding argument in the invoking command or the argument's default value. Actual arguments are assigned to formal arguments strictly in order. Arguments with default values need not be specified in a procedure invocation. However, there must be enough actual arguments for all the formal arguments that do not have defaults, and there must not be any extra actual arguments. Arguments with default values that are followed by non-defaulted arguments become de-facto required arguments, though this may change in a future version of Tcl; portable code should ensure that all optional arguments come after all required arguments.

There is one special case to permit procedures with variable numbers of arguments. If the last formal argument has the name "**args**", then a call to the procedure may contain more actual arguments than the procedure has formal arguments. In this case, all of the actual arguments starting at the one that would be assigned to **args** are combined into a list (as if the list command had been used); this combined value is assigned to the local variable **args**.

When *body* is being executed, variable names normally refer to local variables, which are created automatically when referenced and deleted when the procedure returns. One local variable is automatically created for each of the procedure's arguments. Other variables can only be accessed by invoking one of the global, variable, upvar or namespace upvar commands. The current namespace when *body* is executed will be the namespace that the procedure's name exists in, which will be the namespace that it was created in unless it has been changed with rename.

The **proc** command returns an empty string. When a procedure is invoked, the procedure's return value is the value specified in a return command. If the procedure does not execute an explicit return, then its return value is the value of the last command executed in the procedure's body. If an error occurs while executing the procedure body, then the procedure-as-a-whole will return that same error.

Examples

This is a procedure that takes two arguments and prints both their sum and their product. It also

returns the string "OK" to the caller as an explicit result.

```
proc printSumProduct {x y} {
    set sum [expr {$x + $y}]
    set prod [expr {$x * $y}]
    puts "sum is $sum, product is $prod"
    return "OK"
}
```

This is a procedure that accepts arbitrarily many arguments and prints them out, one by one.

```
proc printArguments args {
    foreach arg $args {
        puts $arg
    }
}
```

This procedure is a bit like the incr command, except it multiplies the contents of the named variable by the value, which defaults to **2**:

```
proc mult {varName {multiplier 2}} {
    upvar 1 $varName var
    set var [expr {$var * $multiplier}]
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

puts

Keywords: puts, channel, newline, output, write

Name

puts - Write to a channel

Synopsis

puts ?**-nonewline**? ?*channelId*? *string*

Description

Writes the characters given by *string* to the channel given by *channelId*.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdout** or **stderr**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension. The channel must have been opened for output.

If no *channelId* is specified then it defaults to **stdout**. **Puts** normally outputs a newline character after *string*, but this feature may be suppressed by specifying the **-nonewline** switch.

Newline characters in the output are translated by **puts** to platform-specific end-of-line sequences according to the current value of the **-translation** option for the channel (for example, on PCs newlines are normally replaced with carriage-return-linefeed sequences. See the fconfigure manual entry for a discussion on ways in which fconfigure will alter output.

Tcl buffers output internally, so characters written with **puts** may not appear immediately on the output file or device; Tcl will normally delay output until the buffer is full or the channel is closed. You can force output to appear immediately with the flush command.

When the output buffer fills up, the **puts** command will normally block until all the buffered data has been accepted for output by the operating system. If *channelId* is in nonblocking mode then the **puts** command will not block even if the operating system cannot accept the data. Instead, Tcl continues to buffer the data and writes it in the background as fast as the underlying file or device can accept it. The application must use the Tcl event loop for nonblocking output to work; otherwise Tcl never finds out that the file or device is ready for more output data. It is possible for an arbitrarily large amount of data to be buffered for a channel in nonblocking mode, which could consume a large amount of memory. To avoid wasting memory, nonblocking I/O should normally be used in an event-driven fashion with the fileevent command (do not invoke **puts** unless you have recently been notified via a file event that the channel is ready for more output data).

Examples

Write a short message to the console (or wherever **stdout** is directed):

```
puts "Hello, World!"
```

Print a message in several parts:

```
puts -nonewline "Hello, "
puts "World!"
```

Print a message to the standard error channel:

```
puts stderr "Hello, World!"
```

Append a log message to a file:

```
set chan [open my.log a]
set timestamp [clock format [clock seconds]]
puts $chan "$timestamp - Hello, World!"
close $chan
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

pwd

Keywords: pwd, working directory

Name

pwd - Return the absolute path of the current working directory

Synopsis

pwd

Description

Returns the absolute path name of the current working directory.

Example

Sometimes it is useful to change to a known directory when running some external command using exec, but it is important to keep the application usually running in the directory that it was started in (unless the user specifies otherwise) since that minimizes user confusion. The way to do this is to save the current directory while the external command is being run:

```
set tarFile [file normalize somefile.tar]
set savedDir [pwd]
cd /tmp
exec tar -xf $tarFile
cd $savedDir
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

re_syntax

Keywords: re_syntax, match, regular expression, string

Name

re_syntax - Syntax of Tcl regular expressions

Description

A *regular expression* describes strings of characters. It's a pattern that matches certain strings and does not match others.

Different flavors of REs

Regular expressions ("RE"s), as defined by POSIX, come in two flavors: *extended* REs ("ERE"s) and *basic* REs ("BRE"s). EREs are roughly those of the traditional *egrep*, while BREs are roughly those of the traditional *ed*. This implementation adds a third flavor, *advanced* REs ("ARE"s), basically EREs with some significant extensions.

This manual page primarily describes AREs. BREs mostly exist for backward compatibility in some old programs; they will be discussed at the end. POSIX EREs are almost an exact subset of AREs. Features of AREs that are not present in EREs will be indicated.

Regular expression syntax

Tcl regular expressions are implemented using the package written by Henry Spencer, based on the 1003.2 spec and some (not quite all) of the Perl5 extensions (thanks, Henry!). Much of the description of regular expressions below is copied verbatim from his manual entry.

An ARE is one or more *branches*, separated by "|", matching anything that matches any of the branches.

A branch is zero or more *constraints* or *quantified atoms*, concatenated. It matches a match for the first, followed by a match for the second, etc; an empty branch matches the empty string.

Quantifiers

A quantified atom is an *atom* possibly followed by a single *quantifier*. Without a quantifier, it matches a single match for the atom. The quantifiers, and what a so-quantified atom matches, are:

*****
a sequence of 0 or more matches of the atom

+
a sequence of 1 or more matches of the atom

?
a sequence of 0 or 1 matches of the atom

{*m*}
a sequence of exactly *m* matches of the atom

{*m*,}
a sequence of *m* or more matches of the atom

{*m*,*n*}
a sequence of *m* through *n* (inclusive) matches of the atom; *m* may not exceed *n*

***? +? ?? {*m*}? {*m*,}? {*m*,*n*}?**
non-greedy quantifiers, which match the same possibilities, but prefer the smallest number rather than the largest number of matches (see Matching)

The forms using **{** and **}** are known as *bound*s. The numbers *m* and *n* are unsigned decimal integers with permissible values from 0 to 255 inclusive.

Atoms

An atom is one of:

(*re*)
matches a match for *re* (*re* is any regular expression) with the match noted for possible reporting

(?:*re*)
as previous, but does no reporting (a "non-capturing" set of parentheses)

()
matches an empty string, noted for possible reporting

(?:)
matches an empty string, without reporting

[*chars*]
a *bracket expression*, matching any one of the *chars* (see Bracket expressions for more detail)

.
matches any single character

k
matches the non-alphanumeric character *k* taken as an ordinary character, e.g. \\ matches a backslash character

c
where *c* is alphanumeric (possibly followed by other characters), an *escape* (AREs only), see Escapes below

{
when followed by a character other than a digit, matches the left-brace character "**{**"; when followed by a digit, it is the beginning of a *bound* (see above)

x
where *x* is a single character with no other significance, matches that character.

Constraints

A *constraint* matches an empty string when specific conditions are met. A constraint may not be followed by a quantifier. The simple constraints are as follows; some more constraints are described later, under Escapes.

^
matches at the beginning of the string or a line (according to whether matching is newline-sensitive or not, as described in Matching, below).

$
matches at the end of the string or a line (according to whether matching is newline-sensitive or not, as described in Matching, below).

The difference between string and line matching modes is immaterial when the string does not contain a newline character. The **\A** and **\Z** constraint escapes have a similar purpose but are always constraints for the overall string.

The default newline-sensitivity depends on the command that uses the regular expression, and can be overridden as described in Metasyntax, below.

(?=*re*)
positive lookahead (AREs only), matches at any point where a substring matching *re* begins

(?!*re*)
negative lookahead (AREs only), matches at any point where no substring matching *re* begins

The lookahead constraints may not contain back references (see later), and all parentheses within them are considered non-capturing.

An RE may not end with "\".

Bracket expressions

A *bracket expression* is a list of characters enclosed in "**[]**". It normally matches any single character from the list (but see below). If the list begins with "**^**", it matches any single character (but see below) *not* from the rest of the list.

If two characters in the list are separated by "**-**", this is shorthand for the full *range* of characters between those two (inclusive) in the collating sequence, e.g. "**[0-9]**" in Unicode matches any conventional decimal digit. Two ranges may not share an endpoint, so e.g. "**a-c-e**" is illegal. Ranges in Tcl always use the Unicode collating sequence, but other programs may use other collating sequences and this can be a source of incompatibility between programs.

To include a literal **]** or **-** in the list, the simplest method is to enclose it in **[.** and**.]** to make it a collating element (see below). Alternatively, make it the first character (following a possible "**^**"), or (AREs only) precede it with "\". Alternatively, for "**-**", make it the last character, or the second endpoint of a range. To use a literal **-** as the first endpoint of a range, make it a collating element or (AREs only) precede it with "\". With the exception of these, some combinations using **[** (see next paragraphs), and escapes, all other special characters lose their special significance within a bracket expression.

Character classes

Within a bracket expression, the name of a *character class* enclosed in **[:** and**:]** stands for the list of all characters (not all collating elements!) belonging to that class. Standard character classes

are:

alpha
A letter.

upper
An upper-case letter.

lower
A lower-case letter.

digit
A decimal digit.

xdigit
A hexadecimal digit.

alnum
An alphanumeric (letter or digit).

print
A "printable" (same as graph, except also including space).

blank
A space or tab character.

space
A character producing white space in displayed text.

punct
A punctuation character.

graph
A character with a visible representation (includes both **alnum** and **punct**).

cntrl
A control character.

A locale may provide others. A character class may not be used as an endpoint of a range.

(*Note:* the current Tcl implementation has only one locale, the Unicode locale, which supports exactly the above classes.)

Bracketed constraints

There are two special cases of bracket expressions: the bracket expressions "**[[:<:]]**" and "**[[:>:]]**" are constraints, matching empty strings at the beginning and end of a word respectively. A word is defined as a sequence of word characters that is neither preceded nor followed by word characters. A word character is an *alnum* character or an underscore ("_"). These special bracket expressions are deprecated; users of AREs should use constraint escapes instead (see below).

Collating elements

Within a bracket expression, a collating element (a character, a multi-character sequence that collates as if it were a single character, or a collating-sequence name for either) enclosed in **[.** and**.]** stands for the sequence of characters of that collating element. The sequence is a single element of the bracket expression's list. A bracket expression in a locale that has multi-character collating elements can thus match more than one character. So (insidiously), a bracket expression that starts with **^** can match multi-character collating elements even if none of them

appear in the bracket expression!

(*Note:* Tcl has no multi-character collating elements. This information is only for illustration.)

For example, assume the collating sequence includes a **ch** multi-character collating element. Then the RE "**[[.ch.]]*c**" (zero or more "**ch**s" followed by "**c**") matches the first five characters of "**chchcc**". Also, the RE "**[^c]b**" matches all of "**chb**" (because "**[^c]**" matches the multi-character "**ch**").

Equivalence classes

Within a bracket expression, a collating element enclosed in **[=** and **=]** is an equivalence class, standing for the sequences of characters of all collating elements equivalent to that one, including itself. (If there are no other equivalent collating elements, the treatment is as if the enclosing delimiters were "**[.**" and "**.]**".) For example, if **o** and **ô** are the members of an equivalence class, then "**[[=o=]]**", "**[[=ô=]]**", and "**[oô]**" are all synonymous. An equivalence class may not be an endpoint of a range.

(*Note:* Tcl implements only the Unicode locale. It does not define any equivalence classes. The examples above are just illustrations.)

Escapes

Escapes (AREs only), which begin with a **** followed by an alphanumeric character, come in several varieties: character entry, class shorthands, constraint escapes, and back references. A **** followed by an alphanumeric character but not constituting a valid escape is illegal in AREs. In EREs, there are no escapes: outside a bracket expression, a **** followed by an alphanumeric character merely stands for that character as an ordinary character, and inside a bracket expression, **** is an ordinary character. (The latter is the one actual incompatibility between EREs and AREs.)

Character-entry escapes

Character-entry escapes (AREs only) exist to make it easier to specify non-printing and otherwise inconvenient characters in REs:

\a
: alert (bell) character, as in C

\b
: backspace, as in C

\B
: synonym for **** to help reduce backslash doubling in some applications where there are multiple levels of backslash processing

\c*X*
: (where *X* is any character) the character whose low-order 5 bits are the same as those of *X*, and whose other bits are all zero

\e
: the character whose collating-sequence name is "**ESC**", or failing that, the character with octal value 033

\f
: formfeed, as in C

\n

newline, as in C

\r
carriage return, as in C

\t
horizontal tab, as in C

\u*wxyz*
(where *wxyz* is one up to four hexadecimal digits) the Unicode character **U+***wxyz* in the local byte ordering

\U*stuvwxyz*
(where *stuvwxyz* is one up to eight hexadecimal digits) reserved for a Unicode extension up to 21 bits. The digits are parsed until the first non-hexadecimal character is encountered, the maximun of eight hexadecimal digits are reached, or an overflow would occur in the maximum value of **U+***10ffff*.

\v
vertical tab, as in C are all available.

\x*hh*
(where *hh* is one or two hexadecimal digits) the character whose hexadecimal value is **0x***hh*.

\0
the character whose value is **0**

xyz
(where *xyz* is exactly three octal digits, and is not a *back reference* (see below)) the character whose octal value is **0***xyz*. The first digit must be in the range 0-3, otherwise the two-digit form is assumed.

xy
(where *xy* is exactly two octal digits, and is not a *back reference* (see below)) the character whose octal value is **0***xy*

Hexadecimal digits are "**0**-**9**", "**a**-**f**", and "**A**-**F**". Octal digits are "**0**-**7**".

The character-entry escapes are always taken as ordinary characters. For example, **\135** is **]** in Unicode, but **\135** does not terminate a bracket expression. Beware, however, that some applications (e.g., C compilers and the Tcl interpreter if the regular expression is not quoted with braces) interpret such sequences themselves before the regular-expression package gets to see them, which may require doubling (quadrupling, etc.) the "****".

Class-shorthand escapes

Class-shorthand escapes (AREs only) provide shorthands for certain commonly-used character classes:

\d
[[:digit:]]

\s
[[:space:]]

\w
[[:alnum:]_\u203F\u2040\u2054\uFE33\uFE34\uFE4D\uFE4E\uFE4F\uFF3F] (including punctuation connector characters)

\D

[^[:digit:]]

\S

[^[:space:]]

\W

[^[:alnum:]_\u203F\u2040\u2054\uFE33\uFE34\uFE4D\uFE4E\uFE4F\uFF3F] (including punctuation connector characters)

Within bracket expressions, "**\d**", "**\s**", and "**\w**" lose their outer brackets, and "**\D**", "**\S**", and "**\W**" are illegal. (So, for example, "**[a-c\d]**" is equivalent to "**[a-c[:digit:]]**". Also, "**[a-c\D]**", which is equivalent to "**[a-c^[:digit:]]**", is illegal.)

Constraint escapes

A constraint escape (AREs only) is a constraint, matching the empty string if specific conditions are met, written as an escape:

\A

matches only at the beginning of the string (see Matching, below, for how this differs from "**^**")

\m

matches only at the beginning of a word

\M

matches only at the end of a word

\y

matches only at the beginning or end of a word

\Y

matches only at a point that is not the beginning or end of a word

\Z

matches only at the end of the string (see Matching, below, for how this differs from "**$**")

\m

(where *m* is a nonzero digit) a *back reference*, see below

\mnn

(where *m* is a nonzero digit, and *nn* is some more digits, and the decimal value *mnn* is not greater than the number of closing capturing parentheses seen so far) a *back reference*, see below

A word is defined as in the specification of "**[[:<:]]**" and "**[[:>:]]**" above. Constraint escapes are illegal within bracket expressions.

Back references

A back reference (AREs only) matches the same string matched by the parenthesized subexpression specified by the number, so that (e.g.) "**([bc])\1**" matches "**bb**" or "**cc**" but not "**bc**". The subexpression must entirely precede the back reference in the RE. Subexpressions are numbered in the order of their leading parentheses. Non-capturing parentheses do not define subexpressions.

There is an inherent historical ambiguity between octal character-entry escapes and back references, which is resolved by heuristics, as hinted at above. A leading zero always indicates an octal escape. A single non-zero digit, not followed by another digit, is always taken as a back

reference. A multi-digit sequence not starting with a zero is taken as a back reference if it comes after a suitable subexpression (i.e. the number is in the legal range for a back reference), and otherwise is taken as octal.

Metasyntax

In addition to the main syntax described above, there are some special forms and miscellaneous syntactic facilities available.

Normally the flavor of RE being used is specified by application-dependent means. However, this can be overridden by a *director*. If an RE of any flavor begins with "*****:**", the rest of the RE is an ARE. If an RE of any flavor begins with "*****=**", the rest of the RE is taken to be a literal string, with all characters considered ordinary characters.

An ARE may begin with *embedded options*: a sequence **(?***xyz***)** (where *xyz* is one or more alphabetic characters) specifies options affecting the rest of the RE. These supplement, and can override, any options specified by the application. The available option letters are:

b
rest of RE is a BRE

c
case-sensitive matching (usual default)

e
rest of RE is an ERE

i
case-insensitive matching (see Matching, below)

m
historical synonym for **n**

n
newline-sensitive matching (see Matching, below)

p
partial newline-sensitive matching (see Matching, below)

q
rest of RE is a literal ("quoted") string, all ordinary characters

s
non-newline-sensitive matching (usual default)

t
tight syntax (usual default; see below)

w
inverse partial newline-sensitive ("weird") matching (see Matching, below)

x
expanded syntax (see below)

Embedded options take effect at the**)** terminating the sequence. They are available only at the start of an ARE, and may not be used later within it.

In addition to the usual (*tight*) RE syntax, in which all characters are significant, there is an *expanded* syntax, available in all flavors of RE with the **-expanded** switch, or in AREs with the embedded x option. In the expanded syntax, white-space characters are ignored and all

characters between a **#** and the following newline (or the end of the RE) are ignored, permitting paragraphing and commenting a complex RE. There are three exceptions to that basic rule:

- a white-space character or "**#**" preceded by "\" is retained
- white space or "**#**" within a bracket expression is retained
- white space and comments are illegal within multi-character symbols like the ARE "**(?:**" or the BRE "**\(**"

Expanded-syntax white-space characters are blank, tab, newline, and any character that belongs to the *space* character class.

Finally, in an ARE, outside bracket expressions, the sequence "**(?#***ttt***)**" (where *ttt* is any text not containing a "**)**") is a comment, completely ignored. Again, this is not allowed between the characters of multi-character symbols like "**(?:**". Such comments are more a historical artifact than a useful facility, and their use is deprecated; use the expanded syntax instead.

None of these metasyntax extensions is available if the application (or an initial "*****=**" director) has specified that the user's input be treated as a literal string rather than as an RE.

Matching

In the event that an RE could match more than one substring of a given string, the RE matches the one starting earliest in the string. If the RE could match more than one substring starting at that point, its choice is determined by its *preference*: either the longest substring, or the shortest.

Most atoms, and all constraints, have no preference. A parenthesized RE has the same preference (possibly none) as the RE. A quantified atom with quantifier **{***m***}** or **{***m***}?** has the same preference (possibly none) as the atom itself. A quantified atom with other normal quantifiers (including **{***m***,***n***}** with *m* equal to *n*) prefers longest match. A quantified atom with other non-greedy quantifiers (including **{***m***,***n***}?** with *m* equal to *n*) prefers shortest match. A branch has the same preference as the first quantified atom in it which has a preference. An RE consisting of two or more branches connected by the **|** operator prefers longest match.

Subject to the constraints imposed by the rules for matching the whole RE, subexpressions also match the longest or shortest possible substrings, based on their preferences, with subexpressions starting earlier in the RE taking priority over ones starting later. Note that outer subexpressions thus take priority over their component subexpressions.

The quantifiers **{1,1}** and **{1,1}?** can be used to force longest and shortest preference, respectively, on a subexpression or a whole RE.

NOTE: This means that you can usually make a RE be non-greedy overall by putting **{1,1}?** after one of the first non-constraint atoms or parenthesized sub-expressions in it. *It pays to experiment* with the placing of this non-greediness override on a suitable range of input texts when you are writing a RE if you are using this level of complexity.

For example, this regular expression is non-greedy, and will match the shortest substring possible given that "**abc**" will be matched as early as possible (the quantifier does not change that):

```
ab{1,1}?c.*x.*cba
```

The atom "**a**" has no greediness preference, we explicitly give one for "**b**", and the remaining quantifiers are overridden to be non-greedy by the preceding non-greedy quantifier.

Match lengths are measured in characters, not collating elements. An empty string is considered longer than no match at all. For example, "**bb***" matches the three middle characters of "**abbbc** ", "**(week|wee)(night|knights)**" matches all ten characters of "**weeknights**", when "**(.*).***" is matched against "**abc**" the parenthesized subexpression matches all three characters, and when "**(a*)***" is matched against "**bc**" both the whole RE and the parenthesized subexpression match an empty string.

If case-independent matching is specified, the effect is much as if all case distinctions had vanished from the alphabet. When an alphabetic that exists in multiple cases appears as an ordinary character outside a bracket expression, it is effectively transformed into a bracket expression containing both cases, so that **x** becomes "**[xX]**". When it appears inside a bracket expression, all case counterparts of it are added to the bracket expression, so that "**[x]**" becomes "**[xX]**" and "**[^x]**" becomes "**[^xX]**".

If newline-sensitive matching is specified,**.** and bracket expressions using **^** will never match the newline character (so that matches will never cross newlines unless the RE explicitly arranges it) and **^** and **$** will match the empty string after and before a newline respectively, in addition to matching at beginning and end of string respectively. ARE **\A** and **\Z** continue to match beginning or end of string *only*.

If partial newline-sensitive matching is specified, this affects**.** and bracket expressions as with newline-sensitive matching, but not **^** and **$**.

If inverse partial newline-sensitive matching is specified, this affects **^** and **$** as with newline-sensitive matching, but not**.** and bracket expressions. This is not very useful but is provided for symmetry.

Limits and compatibility

No particular limit is imposed on the length of REs. Programs intended to be highly portable should not employ REs longer than 256 bytes, as a POSIX-compliant implementation can refuse to accept such REs.

The only feature of AREs that is actually incompatible with POSIX EREs is that **** does not lose its special significance inside bracket expressions. All other ARE features use syntax which is illegal or has undefined or unspecified effects in POSIX EREs; the *** syntax of directors likewise is outside the POSIX syntax for both BREs and EREs.

Many of the ARE extensions are borrowed from Perl, but some have been changed to clean them up, and a few Perl extensions are not present. Incompatibilities of note include "**\b**", "**\B**", the lack of special treatment for a trailing newline, the addition of complemented bracket expressions to the things affected by newline-sensitive matching, the restrictions on parentheses and back references in lookahead constraints, and the longest/shortest-match (rather than first-match) matching semantics.

The matching rules for REs containing both normal and non-greedy quantifiers have changed since early beta-test versions of this package. (The new rules are much simpler and cleaner, but do not work as hard at guessing the user's real intentions.)

Henry Spencer's original 1986 *regexp* package, still in widespread use (e.g., in pre-8.1 releases of Tcl), implemented an early version of today's EREs. There are four incompatibilities between *regexp*'s near-EREs ("RREs" for short) and AREs. In roughly increasing order of significance:

- In AREs, **** followed by an alphanumeric character is either an escape or an error, while in RREs,

it was just another way of writing the alphanumeric. This should not be a problem because there was no reason to write such a sequence in RREs.
- **{** followed by a digit in an ARE is the beginning of a bound, while in RREs, **{** was always an ordinary character. Such sequences should be rare, and will often result in an error because following characters will not look like a valid bound.
- In AREs, **** remains a special character within "**[]**", so a literal **** within **[]** must be written "****". **** also gives a literal **** within **[]** in RREs, but only truly paranoid programmers routinely doubled the backslash.
- AREs report the longest/shortest match for the RE, rather than the first found in a specified search order. This may affect some RREs which were written in the expectation that the first match would be reported. (The careful crafting of RREs to optimize the search order for fast matching is obsolete (AREs examine all possible matches in parallel, and their performance is largely insensitive to their complexity) but cases where the search order was exploited to deliberately find a match which was *not* the longest/shortest will need rewriting.)

Basic regular expressions

BREs differ from EREs in several respects. "**|**", "**+**", and**?** are ordinary characters and there is no equivalent for their functionality. The delimiters for bounds are **\{** and "**\}**", with **{** and **}** by themselves ordinary characters. The parentheses for nested subexpressions are **\(** and "**\)**", with **(** and**)** by themselves ordinary characters. **^** is an ordinary character except at the beginning of the RE or the beginning of a parenthesized subexpression, **$** is an ordinary character except at the end of the RE or the end of a parenthesized subexpression, and ***** is an ordinary character if it appears at the beginning of the RE or the beginning of a parenthesized subexpression (after a possible leading "**^**"). Finally, single-digit back references are available, and **\<** and **\>** are synonyms for "**[[:<:]]**" and "**[[:>:]]**" respectively; no other escapes are available.

Copyright (c) 1998 Sun Microsystems, Inc.
Copyright (c) 1999 Scriptics Corporation

read

Keywords: read, blocking, channel, end of line, end of file, nonblocking, translation, encoding

Name

read - Read from a channel

Synopsis

read ?**-nonewline**? *channelId*
read *channelId numChars*

Description

In the first form, the **read** command reads all of the data from *channelId* up to the end of the file. If the **-nonewline** switch is specified then the last character of the file is discarded if it is a newline. In the second form, the extra argument specifies how many characters to read. Exactly that many characters will be read and returned, unless there are fewer than *numChars* left in the file; in this case all the remaining characters are returned. If the channel is configured to use a multi-byte encoding, then the number of characters read may not be the same as the number of bytes read.

ChannelId must be an identifier for an open channel such as the Tcl standard input channel (**stdin**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension. The channel must have been opened for input.

If *channelId* is in nonblocking mode, the command may not read as many characters as requested: once all available input has been read, the command will return the data that is available rather than blocking for more input. If the channel is configured to use a multi-byte encoding, then there may actually be some bytes remaining in the internal buffers that do not form a complete character. These bytes will not be returned until a complete character is available or end-of-file is reached. The **-nonewline** switch is ignored if the command returns before reaching the end of the file.

Read translates end-of-line sequences in the input into newline characters according to the **-translation** option for the channel. See the fconfigure manual entry for a discussion on ways in which fconfigure will alter input.

Use with serial ports

For most applications a channel connected to a serial port should be configured to be nonblocking: fconfigure *channelId* **-blocking** *0*. Then **read** behaves much like described above. Care must be taken when using **read** on blocking serial ports:

read *channelId numChars*
: In this form **read** blocks until *numChars* have been received from the serial port.

read *channelId*
: In this form **read** blocks until the reception of the end-of-file character, see fconfigure **-eofchar**. If there no end-of-file character has been configured for the channel, then **read** will block forever.

Example

This example code reads a file all at once, and splits it into a list, with each line in the file corresponding to an element in the list:

```
set fl [open /proc/meminfo]
set data [read $fl]
close $fl
set lines [split $data \n]
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

refchan

Keywords: refchan, API, channel, ensemble, prefix, reflection

Name

refchan - command handler API of reflected channels

Synopsis

cmdPrefix *option* ?*arg arg* ...?

Description

The Tcl-level handler for a reflected channel has to be a command with subcommands (termed an *ensemble*, as it is a command such as that created by namespace ensemble **create**, though the implementation of handlers for reflected channel *is not* tied to namespace ensembles in any way; see Example below for how to build an oo::class that supports the API). Note that *cmdPrefix* is whatever was specified in the call to chan create, and may consist of multiple arguments; this will be expanded to multiple words in place of the prefix.

Of all the possible subcommands, the handler *must* support **initialize**, **finalize**, and **watch**. Support for the other subcommands is optional.

Mandatory subcommands

cmdPrefix* initialize *channelId mode

An invocation of this subcommand will be the first call the *cmdPrefix* will receive for the specified new *channelId*. It is the responsibility of this subcommand to set up any internal data structures required to keep track of the channel and its state.

The return value of the method has to be a list containing the names of all subcommands supported by the *cmdPrefix*. This also tells the Tcl core which version of the API for reflected channels is used by this command handler.

Any error thrown by the method will abort the creation of the channel and no channel will be created. The thrown error will appear as error thrown by chan create. Any exception other than an error (e.g., break, etc.) is treated as (and converted to) an error.

Note: If the creation of the channel was aborted due to failures here, then the **finalize** subcommand will not be called.

The *mode* argument tells the handler whether the channel was opened for reading, writing, or both. It is a list containing any of the strings read or **write**. The list may be empty, but will usually contain at least one element.

The subcommand must throw an error if the chosen mode is not supported by the *cmdPrefix*.

cmdPrefix* finalize *channelId

An invocation of this subcommand will be the last call the *cmdPrefix* will receive for the specified *channelId*. It will be generated just before the destruction of the data structures of the channel held by the Tcl core. The command handler *must not* access the *channelId* anymore in no way. Upon this subcommand being called, any internal resources allocated to this channel must be cleaned up.

The return value of this subcommand is ignored.

If the subcommand throws an error the command which caused its invocation (usually chan close) will appear to have thrown this error. Any exception beyond error (e.g., break, etc.) is treated as (and converted to) an error.

This subcommand is not invoked if the creation of the channel was aborted during **initialize**

(See above).

cmdPrefix* watch *channelId eventspec

This subcommand notifies the *cmdPrefix* that the specified *channelId* is interested in the events listed in the *eventspec*. This argument is a list containing any of read and **write**. The list may be empty, which signals that the channel does not wish to be notified of any events. In that situation, the handler should disable event generation completely.

Warning: Any return value of the subcommand is ignored. This includes all errors thrown by the subcommand, break, continue, and custom return codes.

This subcommand interacts with chan postevent. Trying to post an event which was not listed in the last call to **watch** will cause chan postevent to throw an error.

Optional subcommands

cmdPrefix* read *channelId count

This *optional* subcommand is called when the user requests data from the channel *channelId*. *count* specifies how many *bytes* have been requested. If the subcommand is not supported then it is not possible to read from the channel handled by the command.

The return value of this subcommand is taken as the requested data *bytes*. If the returned data contains more bytes than requested, an error will be signaled and later thrown by the command which performed the read (usually gets or read). However, returning fewer bytes than requested is acceptable.

Note that returning nothing (0 bytes) is a signal to the higher layers that **EOF** has been reached on the channel. To signal that the channel is out of data right now, but has not yet reached **EOF**, it is necessary to throw the error "EAGAIN", i.e. to either

```
return -code error EAGAIN
```

or

```
error EAGAIN
```

For extensibility any error whose value is a negative integer number will cause the higher layers to set the C-level variable "**errno**" to the absolute value of this number, signaling a system error. However, note that the exact mapping between these error numbers and their meanings is operating system dependent.

For example, while on Linux both

```
return -code error -11
```

and

```
error -11
```

are equivalent to the examples above, using the more readable string "EAGAIN", this is not true for BSD, where the equivalent number is -35.

The symbolic string however is the same across systems, and internally translated to the correct number. No other error value has such a mapping to a symbolic string.

If the subcommand throws any other error, the command which caused its invocation (usually gets, or read) will appear to have thrown this error. Any exception beyond error, (e.g., break, etc.) is treated as and converted to an error.

cmdPrefix* write *channelId data

This *optional* subcommand is called when the user writes data to the channel *channelId*. The *data* argument contains *bytes*, not characters. Any type of transformation (EOL, encoding) configured for the channel has already been applied at this point. If this subcommand is not supported then it is not possible to write to the channel handled by the command.

The return value of the subcommand is taken as the number of bytes written by the channel. Anything non-numeric will cause an error to be signaled and later thrown by the command which performed the write. A negative value implies that the write failed. Returning a value greater than the number of bytes given to the handler, or zero, is forbidden and will cause the Tcl core to throw an error.

To signal that the channel is not able to accept data for writing right now, it is necessary to throw the error "EAGAIN", i.e. to either

```
return -code error EAGAIN
```

or

```
error EAGAIN
```

For extensibility any error whose value is a negative integer number will cause the higher layers to set the C-level variable "**errno**" to the absolute value of this number, signaling a system error. However, note that the exact mapping between these error numbers and their meanings is operating system dependent.

For example, while on Linux both

```
return -code error -11
```

and

```
error -11
```

are equivalent to the examples above, using the more readable string "EAGAIN", this is not true for BSD, where the equivalent number is -35.

The symbolic string however is the same across systems, and internally translated to the correct number. No other error value has such a mapping to a symbolic string.

If the subcommand throws any other error the command which caused its invocation (usually puts) will appear to have thrown this error. Any exception beyond error (e.g., break, etc.) is treated as and converted to an error.

cmdPrefix* seek *channelId offset base

This *optional* subcommand is responsible for the handling of chan seek and chan tell requests

on the channel *channelId*. If it is not supported then seeking will not be possible for the channel.

The *base* argument is the same as the equivalent argument of the builtin chan seek, namely:

start
Seeking is relative to the beginning of the channel.

current
Seeking is relative to the current seek position.

end
Seeking is relative to the end of the channel.

The *offset* is an integer number specifying the amount of **bytes** to seek forward or backward. A positive number should seek forward, and a negative number should seek backward. A channel may provide only limited seeking. For example sockets can seek forward, but not backward.

The return value of the subcommand is taken as the (new) location of the channel, counted from the start. This has to be an integer number greater than or equal to zero. If the subcommand throws an error the command which caused its invocation (usually chan seek, or chan tell) will appear to have thrown this error. Any exception beyond error (e.g., break, etc.) is treated as and converted to an error.

The offset/base combination of 0/**current** signals a chan tell request, i.e., seek nothing relative to the current location, making the new location identical to the current one, which is then returned.

cmdPrefix* configure *channelId option value
This *optional* subcommand is for setting the type-specific options of channel *channelId*. The *option* argument indicates the option to be written, and the *value* argument indicates the value to set the option to.

This subcommand will never try to update more than one option at a time; that is behavior implemented in the Tcl channel core.

The return value of the subcommand is ignored.

If the subcommand throws an error the command which performed the (re)configuration or query (usually fconfigure or chan configure) will appear to have thrown this error. Any exception beyond error (e.g., break, etc.) is treated as and converted to an error.

cmdPrefix* cget *channelId option
This *optional* subcommand is used when reading a single type-specific option of channel *channelId*. If this subcommand is supported then the subcommand **cgetall** must be supported as well.

The subcommand should return the value of the specified *option*.

If the subcommand throws an error, the command which performed the (re)configuration or query (usually fconfigure or chan configure) will appear to have thrown this error. Any exception beyond *error* (e.g., break, etc.) is treated as and converted to an error.

cmdPrefix* cgetall *channelId

This *optional* subcommand is used for reading all type-specific options of channel *channelId*. If this subcommand is supported then the subcommand **cget** has to be supported as well.

The subcommand should return a list of all options and their values. This list must have an even number of elements.

If the subcommand throws an error the command which performed the (re)configuration or query (usually fconfigure or chan configure) will appear to have thrown this error. Any exception beyond error (e.g., break, etc.) is treated as and converted to an error.

cmdPrefix* blocking *channelId mode

This *optional* subcommand handles changes to the blocking mode of the channel *channelId*. The *mode* is a boolean flag. A true value means that the channel has to be set to blocking, and a false value means that the channel should be non-blocking.

The return value of the subcommand is ignored.

If the subcommand throws an error the command which caused its invocation (usually fconfigure or chan configure) will appear to have thrown this error. Any exception beyond error (e.g., break, etc.) is treated as and converted to an error.

Notes

Some of the functions supported in channels defined in Tcl's C interface are not available to channels reflected to the Tcl level.

The function **Tcl_DriverGetHandleProc** is not supported; i.e., reflected channels do not have OS specific handles.

The function **Tcl_DriverHandlerProc** is not supported. This driver function is relevant only for stacked channels, i.e., transformations. Reflected channels are always base channels, not transformations.

The function **Tcl_DriverFlushProc** is not supported. This is because the current generic I/O layer of Tcl does not use this function anywhere at all. Therefore support at the Tcl level makes no sense either. This may be altered in the future (through extending the API defined here and changing its version number) should the function be used at some time in the future.

Example

This demonstrates how to make a channel that reads from a string.

```
oo::class create stringchan {
    variable data pos
    constructor {string {encoding {}}} {
        if {$encoding eq ""} {set encoding [encoding system]}
        set data [encoding convertto $encoding $string]
        set pos 0
    }

    method initialize {ch mode} {
```

```
        return "initialize finalize watch read seek"
    }
    method finalize {ch} {
        my destroy
    }
    method watch {ch events} {
        # Must be present but we ignore it because we do not
        # post any events
    }

    # Must be present on a readable channel
    method read {ch count} {
        set d [string range $data $pos [expr {$pos+$count-1}]]
        incr pos [string length $d]
        return $d
    }

    # This method is optional, but useful for the example below
    method seek {ch offset base} {
        switch $base {
            start {
                set pos $offset
            }
            current {
                incr pos $offset
            }
            end {
                set pos [string length $data]
                incr pos $offset
            }
        }
        if {$pos < 0} {
            set pos 0
        } elseif {$pos > [string length $data]} {
            set pos [string length $data]
        }
        return $pos
    }
}

# Now we create an instance...
set string "The quick brown fox jumps over the lazy dog.\n"
set ch [chan create read [stringchan new $string]]

puts [gets $ch];   # Prints the whole string
```

```
seek $ch -5 end;
puts [read $ch];   # Prints just the last word
```

Copyright (c) 2006 Andreas Kupries andreas_kupries@users.sourceforge.net

regexp

Keywords: regexp, match, parsing, pattern, regular expression, splitting, string

Name

regexp - Match a regular expression against a string

Synopsis

regexp ?*switches*? *exp string* ?*matchVar*? ?*subMatchVar subMatchVar* ...?

Description

Determines whether the regular expression *exp* matches part or all of *string* and returns 1 if it does, 0 if it does not, unless **-inline** is specified (see below). (Regular expression matching is described in the re_syntax reference page.)

If additional arguments are specified after *string* then they are treated as the names of variables in which to return information about which part(s) of *string* matched *exp*. *MatchVar* will be set to the range of *string* that matched all of *exp*. The first *subMatchVar* will contain the characters in *string* that matched the leftmost parenthesized subexpression within *exp*, the next *subMatchVar* will contain the characters that matched the next parenthesized subexpression to the right in *exp*, and so on.

If the initial arguments to **regexp** start with **-** then they are treated as switches. The following switches are currently supported:

-about
: Instead of attempting to match the regular expression, returns a list containing information about the regular expression. The first element of the list is a subexpression count. The second element is a list of property names that describe various attributes of the regular expression. This switch is primarily intended for debugging purposes.

-expanded
: Enables use of the expanded regular expression syntax where whitespace and comments are ignored. This is the same as specifying the **(?x)** embedded option (see the re_syntax manual page).

-indices
: Changes what is stored in the *matchVar* and *subMatchVar*s. Instead of storing the matching characters from *string*, each variable will contain a list of two decimal strings giving the indices in *string* of the first and last characters in the matching range of characters.

-line
: Enables newline-sensitive matching. By default, newline is a completely ordinary character with no special meaning. With this flag, "[^" bracket expressions and "." never match newline, "^" matches an empty string after any newline in addition to its normal function, and "$" matches an empty string before any newline in addition to its normal function. This flag is equivalent to specifying both **-linestop** and **-lineanchor**, or the **(?n)** embedded option (see the re_syntax manual page).

-linestop
Changes the behavior of "[^" bracket expressions and "." so that they stop at newlines. This is the same as specifying the **(?p)** embedded option (see the re_syntax manual page).

-lineanchor
Changes the behavior of "^" and "$" (the "anchors") so they match the beginning and end of a line respectively. This is the same as specifying the **(?w)** embedded option (see the re_syntax manual page).

-nocase
Causes upper-case characters in *string* to be treated as lower case during the matching process.

-all
Causes the regular expression to be matched as many times as possible in the string, returning the total number of matches found. If this is specified with match variables, they will contain information for the last match only.

-inline
Causes the command to return, as a list, the data that would otherwise be placed in match variables. When using **-inline**, match variables may not be specified. If used with **-all**, the list will be concatenated at each iteration, such that a flat list is always returned. For each match iteration, the command will append the overall match data, plus one element for each subexpression in the regular expression. Examples are:

```
regexp -inline -- {\w(\w)} " inlined "
      → in n
regexp -all -inline -- {\w(\w)} " inlined "
      → in n li i ne e
```

-start *index*
Specifies a character index offset into the string to start matching the regular expression at. The *index* value is interpreted in the same manner as the *index* argument to string index. When using this switch, "^" will not match the beginning of the line, and \A will still match the start of the string at *index*. If **-indices** is specified, the indices will be indexed starting from the absolute beginning of the input string. *index* will be constrained to the bounds of the input string.

--
Marks the end of switches. The argument following this one will be treated as *exp* even if it starts with a **-**.

If there are more *subMatchVar*s than parenthesized subexpressions within *exp*, or if a particular subexpression in *exp* does not match the string (e.g. because it was in a portion of the expression that was not matched), then the corresponding *subMatchVar* will be set to "**-1 -1**" if **-indices** has been specified or to an empty string otherwise.

Examples

Find the first occurrence of a word starting with **foo** in a string that is not actually an instance of **foobar**, and get the letters following it up to the end of the word into a variable:

```
regexp {\mfoo(?!bar\M)(\w*)} $string -> restOfWord
```

Note that the whole matched substring has been placed in the variable "**->**", which is a name chosen to look nice given that we are not actually interested in its contents.

Find the index of the word **badger** (in any case) within a string and store that in the variable **location**:

```
regexp -indices {(?i)\mbadger\M} $string location
```

This could also be written as a *basic* regular expression (as opposed to using the default syntax of *advanced* regular expressions) match by prefixing the expression with a suitable flag:

```
regexp -indices {(?ib)\<badger\>} $string location
```

This counts the number of octal digits in a string:

```
regexp -all {[0-7]} $string
```

This lists all words (consisting of all sequences of non-whitespace characters) in a string, and is useful as a more powerful version of the split command:

```
regexp -all -inline {\S+} $string
```

Copyright (c) 1998 Sun Microsystems, Inc.

registry

Keywords: registry

Name

registry - Manipulate the Windows registry

Synopsis

package require registry 1.3
registry ?*-mode*? *option keyName* ?*arg arg* ...?

Description

The **registry** package provides a general set of operations for manipulating the Windows registry. The package implements the **registry** Tcl command. This command is only supported on the Windows platform. Warning: this command should be used with caution as a corrupted registry can leave your system in an unusable state.

KeyName is the name of a registry key. Registry keys must be one of the following forms:

*hostname**rootname**keypath*

*rootname**keypath*

rootname

Hostname specifies the name of any valid Windows host that exports its registry. The *rootname* component must be one of **HKEY_LOCAL_MACHINE**, **HKEY_USERS**, **HKEY_CLASSES_ROOT**, **HKEY_CURRENT_USER**, **HKEY_CURRENT_CONFIG**, **HKEY_PERFORMANCE_DATA**, or **HKEY_DYN_DATA**. The *keypath* can be one or more registry key names separated by backslash (\) characters.

The optional -mode argument indicates which registry to work with; when it is -32bit the 32-bit

registry will be used, and when it is -64bit the 64-bit registry will be used. If this argument is omitted, the system's default registry will be the subject of the requested operation.

Option indicates what to do with the registry key name. Any unique abbreviation for *option* is acceptable. The valid options are:

registry broadcast *keyName* ?-timeout *milliseconds*?
Sends a broadcast message to the system and running programs to notify them of certain updates. This is necessary to propagate changes to key registry keys like Environment. The timeout specifies the amount of time, in milliseconds, to wait for applications to respond to the broadcast message. It defaults to 3000. The following example demonstrates how to add a path to the global Environment and notify applications of the change without requiring a logoff/logon step (assumes admin privileges):

```
set regPath [join {
    HKEY_LOCAL_MACHINE
    SYSTEM
    CurrentControlSet
    Control
    {Session Manager}
    Environment
} "\\"]
set curPath [registry get $regPath "Path"]
registry set $regPath "Path" "$curPath;$addPath"
registry broadcast "Environment"
```

registry delete *keyName* ?*valueName*?
If the optional *valueName* argument is present, the specified value under *keyName* will be deleted from the registry. If the optional *valueName* is omitted, the specified key and any subkeys or values beneath it in the registry hierarchy will be deleted. If the key could not be deleted then an error is generated. If the key did not exist, the command has no effect.

registry get *keyName valueName*
Returns the data associated with the value *valueName* under the key *keyName*. If either the key or the value does not exist, then an error is generated. For more details on the format of the returned data, see Supported types, below.

registry keys *keyName* ?*pattern*?
If *pattern* is not specified, returns a list of names of all the subkeys of *keyName*. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for string match. If the specified *keyName* does not exist, then an error is generated.

registry set *keyName* ?*valueName data* ?*type*??
If *valueName* is not specified, creates the key *keyName* if it does not already exist. If *valueName* is specified, creates the key *keyName* and value *valueName* if necessary. The contents of *valueName* are set to *data* with the type indicated by *type*. If *type* is not specified, the type **sz** is assumed. For more details on the data and type arguments, see Supported types below.

registry type *keyName valueName*
Returns the type of the value *valueName* in the key *keyName*. For more information on the

possible types, see Supported types, below.

registry values *keyName* ?*pattern*?
If *pattern* is not specified, returns a list of names of all the values of *keyName*. If *pattern* is specified, only those names matching *pattern* are returned. Matching is determined using the same rules as for string match.

Supported types

Each value under a key in the registry contains some data of a particular type in a type-specific representation. The **registry** command converts between this internal representation and one that can be manipulated by Tcl scripts. In most cases, the data is simply returned as a Tcl string. The type indicates the intended use for the data, but does not actually change the representation. For some types, the **registry** command returns the data in a different form to make it easier to manipulate. The following types are recognized by the registry command:

binary
The registry value contains arbitrary binary data. The data is represented exactly in Tcl, including any embedded nulls.

none
The registry value contains arbitrary binary data with no defined type. The data is represented exactly in Tcl, including any embedded nulls.

sz
The registry value contains a null-terminated string. The data is represented in Tcl as a string.

expand_sz
The registry value contains a null-terminated string that contains unexpanded references to environment variables in the normal Windows style (for example, "%PATH%"). The data is represented in Tcl as a string.

dword
The registry value contains a little-endian 32-bit number. The data is represented in Tcl as a decimal string.

dword_big_endian
The registry value contains a big-endian 32-bit number. The data is represented in Tcl as a decimal string.

link
The registry value contains a symbolic link. The data is represented exactly in Tcl, including any embedded nulls.

multi_sz
The registry value contains an array of null-terminated strings. The data is represented in Tcl as a list of strings.

resource_list
The registry value contains a device-driver resource list. The data is represented exactly in Tcl, including any embedded nulls.

In addition to the symbolically named types listed above, unknown types are identified using a 32-bit integer that corresponds to the type code returned by the system interfaces. In this case, the data is represented exactly in Tcl, including any embedded nulls.

Portability issues
The registry command is only available on Windows.

Example
Print out how double-clicking on a Tcl script file will invoke a Tcl interpreter:

```
package require registry
set ext .tcl

# Read the type name
set type [registry get HKEY_CLASSES_ROOT\\$ext {}]
# Work out where to look for the command
set path HKEY_CLASSES_ROOT\\$type\\Shell\\Open\\command
# Read the command!
set command [registry get $path {}]

puts "$ext opens with $command"
```

Copyright (c) 1997 Sun Microsystems, Inc.
Copyright (c) 2002 ActiveState Corporation.

regsub
Keywords: regsub, match, pattern, quoting, regular expression, substitution

Name
regsub - Perform substitutions based on regular expression pattern matching

Synopsis
regsub ?*switches*? *exp string subSpec* ?*varName*?

Description
This command matches the regular expression *exp* against *string*, and either copies *string* to the variable whose name is given by *varName* or returns *string* if *varName* is not present. (Regular expression matching is described in the re_syntax reference page.) If there is a match, then while copying *string* to *varName* (or to the result of this command if *varName* is not present) the portion of *string* that matched *exp* is replaced with *subSpec*. If *subSpec* contains a "&" or "\0", then it is replaced in the substitution with the portion of *string* that matched *exp*. If *subSpec* contains a "*n*", where *n* is a digit between 1 and 9, then it is replaced in the substitution with the portion of *string* that matched the *n*'th parenthesized subexpression of *exp*. Additional backslashes may be used in *subSpec* to prevent special interpretation of "&", "\0", "*n*" and backslashes. The use of backslashes in *subSpec* tends to interact badly with the Tcl parser's use of backslashes, so it is generally safest to enclose *subSpec* in braces if it includes backslashes.

If the initial arguments to **regsub** start with **-** then they are treated as switches. The following switches are currently supported:

-all
: All ranges in *string* that match *exp* are found and substitution is performed for each of these ranges. Without this switch only the first matching range is found and substituted. If **-all** is

specified, then "&" and "*n*" sequences are handled for each substitution using the information from the corresponding match.

-expanded
Enables use of the expanded regular expression syntax where whitespace and comments are ignored. This is the same as specifying the **(?x)** embedded option (see the re_syntax manual page).

-line
Enables newline-sensitive matching. By default, newline is a completely ordinary character with no special meaning. With this flag, "[^" bracket expressions and "." never match newline, "^" matches an empty string after any newline in addition to its normal function, and "$" matches an empty string before any newline in addition to its normal function. This flag is equivalent to specifying both **-linestop** and **-lineanchor**, or the **(?n)** embedded option (see the re_syntax manual page).

-linestop
Changes the behavior of "[^" bracket expressions and "." so that they stop at newlines. This is the same as specifying the **(?p)** embedded option (see the re_syntax manual page).

-lineanchor
Changes the behavior of "^" and "$" (the "anchors") so they match the beginning and end of a line respectively. This is the same as specifying the **(?w)** embedded option (see the re_syntax manual page).

-nocase
Upper-case characters in *string* will be converted to lower-case before matching against *exp*; however, substitutions specified by *subSpec* use the original unconverted form of *string*.

-start *index*
Specifies a character index offset into the string to start matching the regular expression at. The *index* value is interpreted in the same manner as the *index* argument to string index. When using this switch, "^" will not match the beginning of the line, and \A will still match the start of the string at *index*. *index* will be constrained to the bounds of the input string.

--
Marks the end of switches. The argument following this one will be treated as *exp* even if it starts with a **-**.

If *varName* is supplied, the command returns a count of the number of matching ranges that were found and replaced, otherwise the string after replacement is returned. See the manual entry for regexp for details on the interpretation of regular expressions.

Examples

Replace (in the string in variable *string*) every instance of **foo** which is a word by itself with **bar**:

```
regsub -all {\mfoo\M} $string bar string
```

or (using the "basic regular expression" syntax):

```
regsub -all {(?b)\<foo\>} $string bar string
```

Insert double-quotes around the first instance of the word **interesting**, however it is capitalized.

```
regsub -nocase {\yinteresting\y} $string {"&"} string
```

Convert all non-ASCII and Tcl-significant characters into \u escape sequences by using **regsub** and subst in combination:

```
# This RE is just a character class for almost everything "bad"
set RE {[][{};#\\\$ \r\t\u0080-\uffff]}

# We will substitute with a fragment of Tcl script in brackets
set substitution {[format \\\\u%04x [scan "\\&" %c]]}

# Now we apply the substitution to get a subst-string that
# will perform the computational parts of the conversion. Note
# that newline is handled specially through string map since
# backslash-newline is a special sequence.
set quoted [subst [string map {\n {\\u000a}} \
        [regsub -all $RE $string $substitution]]]
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2000 Scriptics Corporation.

rename

Keywords: rename, command, delete, namespace

Name

rename - Rename or delete a command

Synopsis

rename *oldName newName*

Description

Rename the command that used to be called *oldName* so that it is now called *newName*. If *newName* is an empty string then *oldName* is deleted. *oldName* and *newName* may include namespace qualifiers (names of containing namespaces). If a command is renamed into a different namespace, future invocations of it will execute in the new namespace. The **rename** command returns an empty string as result.

Example

The **rename** command can be used to wrap the standard Tcl commands with your own monitoring machinery. For example, you might wish to count how often the source command is called:

```
rename ::source ::theRealSource
set sourceCount 0
proc ::source args {
    global sourceCount
    puts "called source for the [incr sourceCount]'th time"
    uplevel 1 ::theRealSource $args
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

return

Keywords: return, break, catch, continue, error, exception, procedure, result

Name

return - Return from a procedure, or set return code of a script

Synopsis

return ?*result*?
return ?*-code code*? ?*result*?
return ?*option value* ...? ?*result*?

Description

In its simplest usage, the **return** command is used without options in the body of a procedure to immediately return control to the caller of the procedure. If a *result* argument is provided, its value becomes the result of the procedure passed back to the caller. If *result* is not specified then an empty string will be returned to the caller as the result of the procedure.

The **return** command serves a similar function within script files that are evaluated by the source command. When source evaluates the contents of a file as a script, an invocation of the **return** command will cause script evaluation to immediately cease, and the value *result* (or an empty string) will be returned as the result of the source command.

Exceptional return codes

In addition to the result of a procedure, the return code of a procedure may also be set by **return** through use of the **-code** option. In the usual case where the **-code** option is not specified the procedure will return normally. However, the **-code** option may be used to generate an exceptional return from the procedure. *Code* may have any of the following values:

ok (or 0)
: Normal return: same as if the option is omitted. The return code of the procedure is 0 (TCL_OK).

error **(or 1)**
: Error return: the return code of the procedure is 1 (TCL_ERROR). The procedure command behaves in its calling context as if it were the command error *result*. See below for additional options.

return (or 2)
: The return code of the procedure is 2 (TCL_RETURN). The procedure command behaves in its calling context as if it were the command **return** (with no arguments).

break **(or 3)**
: The return code of the procedure is 3 (TCL_BREAK). The procedure command behaves in its calling context as if it were the command break.

continue **(or 4)**
: The return code of the procedure is 4 (TCL_CONTINUE). The procedure command behaves in its calling context as if it were the command continue.

value
Value must be an integer; it will be returned as the return code for the current procedure.

When a procedure wants to signal that it has received invalid arguments from its caller, it may use **return -code error** with *result* set to a suitable error message. Otherwise usage of the **return -code** option is mostly limited to procedures that implement a new control structure.

The **return -code** command acts similarly within script files that are evaluated by the source command. During the evaluation of the contents of a file as a script by source, an invocation of the **return -code** *code* command will cause the return code of source to be *code*.

Return options

In addition to a result and a return code, evaluation of a command in Tcl also produces a dictionary of return options. In general usage, all *option value* pairs given as arguments to **return** become entries in the return options dictionary, and any values at all are acceptable except as noted below. The catch command may be used to capture all of this information — the return code, the result, and the return options dictionary — that arise from evaluation of a script.

As documented above, the **-code** entry in the return options dictionary receives special treatment by Tcl. There are other return options also recognized and treated specially by Tcl. They are:

-errorcode *list*
The **-errorcode** option receives special treatment only when the value of the **-code** option is TCL_ERROR. Then the *list* value is meant to be additional information about the error, presented as a Tcl list for further processing by programs. If no **-errorcode** option is provided to **return** when the **-code error** option is provided, Tcl will set the value of the **-errorcode** entry in the return options dictionary to the default value of **NONE**. The **-errorcode** return option will also be stored in the global variable errorCode.

-errorinfo *info*
The **-errorinfo** option receives special treatment only when the value of the **-code** option is TCL_ERROR. Then *info* is the initial stack trace, meant to provide to a human reader additional information about the context in which the error occurred. The stack trace will also be stored in the global variable errorInfo. If no **-errorinfo** option is provided to **return** when the **-code error** option is provided, Tcl will provide its own initial stack trace value in the entry for **-errorinfo**. Tcl's initial stack trace will include only the call to the procedure, and stack unwinding will append information about higher stack levels, but there will be no information about the context of the error within the procedure. Typically the *info* value is supplied from the value of **-errorinfo** in a return options dictionary captured by the catch command (or from the copy of that information stored in the global variable errorInfo).

-errorstack *list*
The -errorstack option receives special treatment only when the value of the -code option is TCL_ERROR. Then list is the initial error stack, recording actual argument values passed to each proc level. The error stack will also be reachable through info errorstack. If no -errorstack option is provided to return when the -code error option is provided, Tcl will provide its own initial error stack in the entry for -errorstack. Tcl's initial error stack will include only the call to the procedure, and stack unwinding will append information about higher stack levels, but there will be no information about the context of the error within the procedure. Typically the list value is supplied from the value of -errorstack in a return options dictionary captured by

the catch command (or from the copy of that information from info errorstack).

-level *level*
The **-level** and **-code** options work together to set the return code to be returned by one of the commands currently being evaluated. The *level* value must be a non-negative integer representing a number of levels on the call stack. It defines the number of levels up the stack at which the return code of a command currently being evaluated should be *code*. If no **-level** option is provided, the default value of *level* is 1, so that **return** sets the return code that the current procedure returns to its caller, 1 level up the call stack. The mechanism by which these options work is described in more detail below.

-options *options*
The value *options* must be a valid dictionary. The entries of that dictionary are treated as additional *option value* pairs for the **return** command.

Return code handling mechanisms

Return codes are used in Tcl to control program flow. A Tcl script is a sequence of Tcl commands. So long as each command evaluation returns a return code of TCL_OK, evaluation will continue to the next command in the script. Any exceptional return code (non-TCL_OK) returned by a command evaluation causes the flow on to the next command to be interrupted. Script evaluation ceases, and the exceptional return code from the command becomes the return code of the full script evaluation. This is the mechanism by which errors during script evaluation cause an interruption and unwinding of the call stack. It is also the mechanism by which commands like break, continue, and **return** cause script evaluation to terminate without evaluating all commands in sequence.

Some of Tcl's built-in commands evaluate scripts as part of their functioning. These commands can make use of exceptional return codes to enable special features. For example, the built-in Tcl commands that provide loops — such as while, for, and foreach — evaluate a script that is the body of the loop. If evaluation of the loop body returns the return code of TCL_BREAK or TCL_CONTINUE, the loop command can react in such a way as to give the break and continue commands their documented interpretation in loops.

Procedure invocation also involves evaluation of a script, the body of the procedure. Procedure invocation provides special treatment when evaluation of the procedure body returns the return code TCL_RETURN. In that circumstance, the **-level** entry in the return options dictionary is decremented. If after decrementing, the value of the **-level** entry is 0, then the value of the **-code** entry becomes the return code of the procedure. If after decrementing, the value of the **-level** entry is greater than zero, then the return code of the procedure is TCL_RETURN. If the procedure invocation occurred during the evaluation of the body of another procedure, the process will repeat itself up the call stack, decrementing the value of the **-level** entry at each level, so that the *code* will be the return code of the current command *level* levels up the call stack. The source command performs the same handling of the TCL_RETURN return code, which explains the similarity of **return** invocation during a source to **return** invocation within a procedure.

The return code of the **return** command itself triggers this special handling by procedure invocation. If **return** is provided the option **-level 0**, then the return code of the **return** command itself will be the value *code* of the **-code** option (or TCL_OK by default). Any other value for the **-level** option (including the default value of 1) will cause the return code of the

return command itself to be TCL_RETURN, triggering a return from the enclosing procedure.

Examples

First, a simple example of using **return** to return from a procedure, interrupting the procedure body.

```
proc printOneLine {} {
    puts "line 1"    ;# This line will be printed.
    return
    puts "line 2"    ;# This line will not be printed.
}
```

Next, an example of using **return** to set the value returned by the procedure.

```
proc returnX {} {return X}
puts [returnX]    ;# prints "X"
```

Next, a more complete example, using **return -code error** to report invalid arguments.

```
proc factorial {n} {
    if {![string is integer $n] || ($n < 0)} {
        return -code error \
                "expected non-negative integer,\
                but got \"$n\""
    }
    if {$n < 2} {
        return 1
    }
    set factor [factorial [expr {$n - 1}]]
    set product [expr {$n * $factor}]
    return $product
}
```

Next, a procedure replacement for break.

```
proc myBreak {} {
    return -code break
}
```

With the **-level 0** option, **return** itself can serve as a replacement for break, with the help of interp alias.

```
interp alias {} Break {} return -level 0 -code break
```

An example of using catch and **return -options** to re-raise a caught error:

```
proc doSomething {} {
    set resource [allocate]
    catch {
        # Long script of operations
        # that might raise an error
```

```
    } result options
    deallocate $resource
    return -options $options $result
}
```

Finally an example of advanced use of the **return** options to create a procedure replacement for **return** itself:

```
proc myReturn {args} {
    set result ""
    if {[llength $args] % 2} {
        set result [lindex $args end]
        set args [lrange $args 0 end-1]
    }
    set options [dict merge {-level 1} $args]
    dict incr options -level
    return -options $options $result
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

safe

Keywords: safe, alias, auto-loading, auto_mkindex, load, parent interpreter, safe interpreter, child interpreter, source

Name

safe - Creating and manipulating safe interpreters

Synopsis

::safe::interpCreate ?*child*? ?*options...*?
::safe::interpInit *child* ?*options...*?
::safe::interpConfigure *child* ?*options...*?
::safe::interpDelete *child*
::safe::interpAddToAccessPath *child directory*
::safe::interpFindInAccessPath *child directory*
::safe::setLogCmd ?*cmd arg...*?
?-accessPath *pathList?*
?-statics *boolean?* ?**-noStatics**?
?-nested *boolean?* ?**-nestedLoadOk**?
?-deleteHook *script?*

Description

Safe Tcl is a mechanism for executing untrusted Tcl scripts safely and for providing mediated access by such scripts to potentially dangerous functionality.

Safe Tcl ensures that untrusted Tcl scripts cannot harm the hosting application. It prevents integrity and privacy attacks. Untrusted Tcl scripts are prevented from corrupting the state of the

hosting application or computer. Untrusted scripts are also prevented from disclosing information stored on the hosting computer or in the hosting application to any party.

Safe Tcl allows a parent interpreter to create safe, restricted interpreters that contain a set of predefined aliases for the source, load, file, encoding, and exit commands and are able to use the auto-loading and package mechanisms.

No knowledge of the file system structure is leaked to the safe interpreter, because it has access only to a virtualized path containing tokens. When the safe interpreter requests to source a file, it uses the token in the virtual path as part of the file name to source; the parent interpreter transparently translates the token into a real directory name and executes the requested operation (see the section Security below for details). Different levels of security can be selected by using the optional flags of the commands described below.

All commands provided in the parent interpreter by Safe Tcl reside in the **safe** namespace.

Commands

The following commands are provided in the parent interpreter:

::safe::interpCreate ?*child*? ?*options...*?

Creates a safe interpreter, installs the aliases described in the section Aliases and initializes the auto-loading and package mechanism as specified by the supplied *options*. See the Options section below for a description of the optional arguments. If the *child* argument is omitted, a name will be generated. **::safe::interpCreate** always returns the interpreter name.

The interpreter name *child* may include namespace separators, but may not have leading or trailing namespace separators, or excess colon characters in namespace separators. The interpreter name is qualified relative to the global namespace ::, not the namespace in which the **::safe::interpCreate** command is evaluated.

::safe::interpInit *child* ?*options...*?

This command is similar to **interpCreate** except it that does not create the safe interpreter. *child* must have been created by some other means, like interp create **-safe**. The interpreter name *child* may include namespace separators, subject to the same restrictions as for **interpCreate**.

::safe::interpConfigure *child* ?*options...*?

If no *options* are given, returns the settings for all options for the named safe interpreter as a list of options and their current values for that *child*. If a single additional argument is provided, it will return a list of 2 elements *name* and *value* where *name* is the full name of that option and *value* the current value for that option and the *child*. If more than two additional arguments are provided, it will reconfigure the safe interpreter and change each and only the provided options. See the section on Options below for options description. Example of use:

```
# Create new interp with the same configuration as "$i0":
set i1 [safe::interpCreate {*}[safe::interpConfigure $i0]]

# Get the current deleteHook
set dh [safe::interpConfigure $i0  -del]
```

```
# Change (only) the statics loading ok attribute of an
# interp and its deleteHook (leaving the rest unchanged):
safe::interpConfigure $i0  -delete {foo bar} -statics 0
```

::safe::interpDelete *child*

Deletes the safe interpreter and cleans up the corresponding parent interpreter data structures. If a *deleteHook* script was specified for this interpreter it is evaluated before the interpreter is deleted, with the name of the interpreter as an additional argument.

::safe::interpFindInAccessPath *child directory*

This command finds and returns the token for the real directory *directory* in the safe interpreter's current virtual access path. It generates an error if the directory is not found. Example of use:

```
$child eval [list set tk_library \
      [::safe::interpFindInAccessPath $name $tk_library]]
```

::safe::interpAddToAccessPath *child directory*

This command adds *directory* to the virtual path maintained for the safe interpreter in the parent, and returns the token that can be used in the safe interpreter to obtain access to files in that directory. If the directory is already in the virtual path, it only returns the token without adding the directory to the virtual path again. Example of use:

```
$child eval [list set tk_library \
      [::safe::interpAddToAccessPath $name $tk_library]]
```

::safe::setLogCmd ?*cmd arg...*?

This command installs a script that will be called when interesting life cycle events occur for a safe interpreter. When called with no arguments, it returns the currently installed script. When called with one argument, an empty string, the currently installed script is removed and logging is turned off. The script will be invoked with one additional argument, a string describing the event of interest. The main purpose is to help in debugging safe interpreters. Using this facility you can get complete error messages while the safe interpreter gets only generic error messages. This prevents a safe interpreter from seeing messages about failures and other events that might contain sensitive information such as real directory names.

Example of use:

```
::safe::setLogCmd puts stderr
```

Below is the output of a sample session in which a safe interpreter attempted to source a file not found in its virtual access path. Note that the safe interpreter only received an error message saying that the file was not found:

```
NOTICE for child interp10 : Created
NOTICE for child interp10 : Setting accessPath=(/foo/bar)
    staticsok=1 nestedok=0 deletehook=()
NOTICE for child interp10 : auto_path in interp10 has been set to
    {$p(:0:)}
ERROR for child interp10 : /foo/bar/init.tcl: no such file or
    directory
```

Options

The following options are common to **::safe::interpCreate**, **::safe::interpInit**, and **::safe::interpConfigure**. Any option name can be abbreviated to its minimal non-ambiguous name. Option names are not case sensitive.

-accessPath *directoryList*
: This option sets the list of directories from which the safe interpreter can source and load files. If this option is not specified, or if it is given as the empty list, the safe interpreter will use the same directories as its parent for auto-loading. See the section Security below for more detail about virtual paths, tokens and access control.

-statics *boolean*
: This option specifies if the safe interpreter will be allowed to load statically linked packages (like **load {} Tk**). The default value is **true**: safe interpreters are allowed to load statically linked packages.

-noStatics
: This option is a convenience shortcut for **-statics false** and thus specifies that the safe interpreter will not be allowed to load statically linked packages.

-nested *boolean*
: This option specifies if the safe interpreter will be allowed to load packages into its own sub-interpreters. The default value is **false**: safe interpreters are not allowed to load packages into their own sub-interpreters.

-nestedLoadOk
: This option is a convenience shortcut for **-nested true** and thus specifies the safe interpreter will be allowed to load packages into its own sub-interpreters.

-deleteHook *script*
: When this option is given a non-empty *script*, it will be evaluated in the parent with the name of the safe interpreter as an additional argument just before actually deleting the safe interpreter. Giving an empty value removes any currently installed deletion hook script for that safe interpreter. The default value (**{}**) is not to have any deletion call back.

Aliases

The following aliases are provided in a safe interpreter:

source ***fileName***
: The requested file, a Tcl source file, is sourced into the safe interpreter if it is found. The source alias can only source files from directories in the virtual path for the safe interpreter. The source alias requires the safe interpreter to use one of the token names in its virtual path to denote the directory in which the file to be sourced can be found. See the section on

Security for more discussion of restrictions on valid filenames.

load ***fileName***

The requested file, a shared object file, is dynamically loaded into the safe interpreter if it is found. The filename must contain a token name mentioned in the virtual path for the safe interpreter for it to be found successfully. Additionally, the shared object file must contain a safe entry point; see the manual page for the load command for more details.

file **?*subCmd args...*?**

The file alias provides access to a safe subset of the subcommands of the file command; it allows only **dirname**, **join**, **extension**, **root**, **tail**, **pathtype** and **split** subcommands. For more details on what these subcommands do see the manual page for the file command.

encoding **?*subCmd args...*?**

The encoding alias provides access to a safe subset of the subcommands of the encoding command; it disallows setting of the system encoding, but allows all other subcommands including **system** to check the current encoding.

exit

The calling interpreter is deleted and its computation is stopped, but the Tcl process in which this interpreter exists is not terminated.

Security

Safe Tcl does not attempt to completely prevent annoyance and denial of service attacks. These forms of attack prevent the application or user from temporarily using the computer to perform useful work, for example by consuming all available CPU time or all available screen real estate. These attacks, while aggravating, are deemed to be of lesser importance in general than integrity and privacy attacks that Safe Tcl is to prevent.

The commands available in a safe interpreter, in addition to the safe set as defined in interp manual page, are mediated aliases for source, load, exit, and safe subsets of file and encoding. The safe interpreter can also auto-load code and it can request that packages be loaded.

Because some of these commands access the local file system, there is a potential for information leakage about its directory structure. To prevent this, commands that take file names as arguments in a safe interpreter use tokens instead of the real directory names. These tokens are translated to the real directory name while a request to, e.g., source a file is mediated by the parent interpreter. This virtual path system is maintained in the parent interpreter for each safe interpreter created by **::safe::interpCreate** or initialized by **::safe::interpInit** and the path maps tokens accessible in the safe interpreter into real path names on the local file system thus preventing safe interpreters from gaining knowledge about the structure of the file system of the host on which the interpreter is executing. The only valid file names arguments for the source and load aliases provided to the child are path in the form of **[file join** *token filename***]** (i.e. when using the native file path formats: *token*/*filename* on Unix and *token**filename* on Windows), where *token* is representing one of the directories of the *accessPath* list and *filename* is one file in that directory (no sub directories access are allowed).

When a token is used in a safe interpreter in a request to source or load a file, the token is checked and translated to a real path name and the file to be sourced or loaded is located on the file system. The safe interpreter never gains knowledge of the actual path name under which the file is stored on the file system.

To further prevent potential information leakage from sensitive files that are accidentally

included in the set of files that can be sourced by a safe interpreter, the source alias restricts access to files meeting the following constraints: the file name must fourteen characters or shorter, must not contain more than one dot ("**.**"), must end up with the extension ("**.tcl**") or be called ("**tclIndex**".)

Each element of the initial access path list will be assigned a token that will be set in the child auto_path and the first element of that list will be set as the tcl_library for that child.

If the access path argument is not given or is the empty list, the default behavior is to let the child access the same packages as the parent has access to (Or to be more precise: only packages written in Tcl (which by definition cannot be dangerous as they run in the child interpreter) and C extensions that provides a _SafeInit entry point). For that purpose, the parent's auto_path will be used to construct the child access path. In order that the child successfully loads the Tcl library files (which includes the auto-loading mechanism itself) the tcl_library will be added or moved to the first position if necessary, in the child access path, so the child tcl_library will be the same as the parent's (its real path will still be invisible to the child though). In order that auto-loading works the same for the child and the parent in this by default case, the first-level sub directories of each directory in the parent auto_path will also be added (if not already included) to the child access path. You can always specify a more restrictive path for which sub directories will never be searched by explicitly specifying your directory list with the **-accessPath** flag instead of relying on this default mechanism.

When the *accessPath* is changed after the first creation or initialization (i.e. through **interpConfigure -accessPath** *list*), an auto_reset is automatically evaluated in the safe interpreter to synchronize its **auto_index** with the new token list.

Copyright (c) 1995-1996 Sun Microsystems, Inc.

scan

Keywords: scan, conversion specifier, parse

Name

scan - Parse string using conversion specifiers in the style of sscanf

Synopsis

scan *string format* ?*varName varName* ...?

Introduction

This command parses substrings from an input string in a fashion similar to the ANSI C **sscanf** procedure and returns a count of the number of conversions performed, or -1 if the end of the input string is reached before any conversions have been performed. *String* gives the input to be parsed and *format* indicates how to parse it, using **%** conversion specifiers as in **sscanf**. Each *varName* gives the name of a variable; when a substring is scanned from *string* that matches a conversion specifier, the substring is assigned to the corresponding variable. If no *varName* variables are specified, then **scan** works in an inline manner, returning the data that would otherwise be stored in the variables as a list. In the inline case, an empty string is returned when the end of the input string is reached before any conversions have been performed.

Details on scanning

Scan operates by scanning *string* and *format* together. If the next character in *format* is a blank

or tab then it matches any number of white space characters in *string* (including zero). Otherwise, if it is not a **%** character then it must match the next character of *string*. When a **%** is encountered in *format*, it indicates the start of a conversion specifier. A conversion specifier contains up to four fields after the **%**: a XPG3 position specifier (or a * to indicate the converted value is to be discarded instead of assigned to any variable); a number indicating a maximum substring width; a size modifier; and a conversion character. All of these fields are optional except for the conversion character. The fields that are present must appear in the order given above.

When **scan** finds a conversion specifier in *format*, it first skips any white-space characters in *string* (unless the conversion character is **[** or **c**). Then it converts the next input characters according to the conversion specifier and stores the result in the variable given by the next argument to **scan**.

Optional positional specifier

If the **%** is followed by a decimal number and a **$**, as in "**%2$d**", then the variable to use is not taken from the next sequential argument. Instead, it is taken from the argument indicated by the number, where 1 corresponds to the first *varName*. If there are any positional specifiers in *format* then all of the specifiers must be positional. Every *varName* on the argument list must correspond to exactly one conversion specifier or an error is generated, or in the inline case, any position can be specified at most once and the empty positions will be filled in with empty strings.

Optional size modifier

The size modifier field is used only when scanning a substring into one of Tcl's integer values. The size modifier field dictates the integer range acceptable to be stored in a variable, or, for the inline case, in a position in the result list. The syntactically valid values for the size modifier are **h**, **L**, **l**, and **ll**. The **h** size modifier value is equivalent to the absence of a size modifier in the the conversion specifier. Either one indicates the integer range to be stored is limited to the same range produced by the **int()** function of the expr command. The **L** size modifier is equivalent to the **l** size modifier. Either one indicates the integer range to be stored is limited to the same range produced by the **wide()** function of the expr command. The **ll** size modifier indicates that the integer range to be stored is unlimited.

Mandatory conversion character

The following conversion characters are supported:

d
: The input substring must be a decimal integer. It is read in and the integer value is stored in the variable, truncated as required by the size modifier value.

o
: The input substring must be an octal integer. It is read in and the integer value is stored in the variable, truncated as required by the size modifier value.

x or X
: The input substring must be a hexadecimal integer. It is read in and the integer value is stored in the variable, truncated as required by the size modifier value.

b
: The input substring must be a binary integer. It is read in and the integer value is stored in

the variable, truncated as required by the size modifier value.

u
The input substring must be a decimal integer. The integer value is truncated as required by the size modifier value, and the corresponding unsigned value for that truncated range is computed and stored in the variable as a decimal string. The conversion makes no sense without reference to a truncation range, so the size modifier **ll** is not permitted in combination with conversion character **u**.

i
The input substring must be an integer. The base (i.e. decimal, octal, or hexadecimal) is determined by the C convention (leading 0 for octal; prefix 0x for hexadecimal). The integer value is stored in the variable, truncated as required by the size modifier value.

c
A single character is read in and its Unicode value is stored in the variable as an integer value. Initial white space is not skipped in this case, so the input substring may be a white-space character.

s
The input substring consists of all the characters up to the next white-space character; the characters are copied to the variable.

e or f or g or E or G
The input substring must be a floating-point number consisting of an optional sign, a string of decimal digits possibly containing a decimal point, and an optional exponent consisting of an **e** or **E** followed by an optional sign and a string of decimal digits. It is read in and stored in the variable as a floating-point value.

[*chars*]
The input substring consists of one or more characters in *chars*. The matching string is stored in the variable. If the first character between the brackets is a **]** then it is treated as part of *chars* rather than the closing bracket for the set. If *chars* contains a sequence of the form *a***-***b* then any character between *a* and *b* (inclusive) will match. If the first or last character between the brackets is a **-**, then it is treated as part of *chars* rather than indicating a range.

[^*chars*]
The input substring consists of one or more characters not in *chars*. The matching string is stored in the variable. If the character immediately following the **^** is a **]** then it is treated as part of the set rather than the closing bracket for the set. If *chars* contains a sequence of the form *a***-***b* then any character between *a* and *b* (inclusive) will be excluded from the set. If the first or last character between the brackets is a **-**, then it is treated as part of *chars* rather than indicating a range value.

n
No input is consumed from the input string. Instead, the total number of characters scanned from the input string so far is stored in the variable.

The number of characters read from the input for a conversion is the largest number that makes sense for that particular conversion (e.g. as many decimal digits as possible for **%d**, as many octal digits as possible for **%o**, and so on). The input substring for a given conversion terminates either when a white-space character is encountered or when the maximum substring width has been reached, whichever comes first. If a ***** is present in the conversion specifier then no variable is assigned and the next scan argument is not consumed.

Differences from ANSI sscanf

The behavior of the **scan** command is the same as the behavior of the ANSI C **sscanf** procedure except for the following differences:

1. **%p** conversion specifier is not supported.
2. For **%c** conversions a single character value is converted to a decimal string, which is then assigned to the corresponding *varName*; no substring width may be specified for this conversion.
3. The **h** modifier is always ignored and the **l** and **L** modifiers are ignored when converting real values (i.e. type **double** is used for the internal representation). The **ll** modifier has no **sscanf** counterpart.
4. If the end of the input string is reached before any conversions have been performed and no variables are given, an empty string is returned.

Examples

Convert a UNICODE character to its numeric value:

```
set char "x"
set value [scan $char %c]
```

Parse a simple color specification of the form *#RRGGBB* using hexadecimal conversions with substring sizes:

```
set string "#08D03F"
scan $string "#%2x%2x%2x" r g b
```

Parse a *HH:MM* time string, noting that this avoids problems with octal numbers by forcing interpretation as decimals (if we did not care, we would use the **%i** conversion instead):

```
set string "08:08"   ;# *Not* octal!
if {[scan $string "%d:%d" hours minutes] != 2} {
    error "not a valid time string"
}
# We have to understand numeric ranges ourselves...
if {$minutes < 0 || $minutes > 59} {
    error "invalid number of minutes"
}
```

Break a string up into sequences of non-whitespace characters (note the use of the **%n** conversion so that we get skipping over leading whitespace correct):

```
set string " a string {with braced words} + leading space "
set words {}
while {[scan $string %s%n word length] == 2} {
    lappend words $word
    set string [string range $string $length end]
}
```

Parse a simple coordinate string, checking that it is complete by looking for the terminating character explicitly:

```
set string "(5.2,-4e-2)"
# Note that the spaces before the literal parts of
# the scan pattern are significant, and that ")" is
# the Unicode character \u0029
if {
    [scan $string " (%f ,%f %c" x y last] != 3
    || $last != 0x0029
} then {
    error "invalid coordinate string"
}
puts "X=$x, Y=$y"
```

An interactive session demonstrating the truncation of integer values determined by size modifiers:

```
% set tcl_platform(wordSize)
4
% scan 20000000000000000000 %d
2147483647
% scan 20000000000000000000 %ld
9223372036854775807
% scan 20000000000000000000 %lld
20000000000000000000
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2000 Scriptics Corporation.

seek

Keywords: seek, access position, file

Name

seek - Change the access position for an open channel

Synopsis

seek *channelId offset* ?*origin*?

Description

Changes the current access position for *channelId*.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension.

The *offset* and *origin* arguments specify the position at which the next read or write will occur for *channelId*. *Offset* must be an integer (which may be negative) and *origin* must be one of the following:

start
The new access position will be *offset* bytes from the start of the underlying file or device.

current
The new access position will be *offset* bytes from the current access position; a negative *offset* moves the access position backwards in the underlying file or device.

end
The new access position will be *offset* bytes from the end of the file or device. A negative *offset* places the access position before the end of file, and a positive *offset* places the access position after the end of file.

The *origin* argument defaults to **start**.

The command flushes all buffered output for the channel before the command returns, even if the channel is in non-blocking mode. It also discards any buffered and unread input. This command returns an empty string. An error occurs if this command is applied to channels whose underlying file or device does not support seeking.

Note that *offset* values are byte offsets, not character offsets. Both **seek** and tell operate in terms of bytes, not characters, unlike read.

Examples

Read a file twice:

```
set f [open file.txt]
set data1 [read $f]
seek $f 0
set data2 [read $f]
close $f
# $data1 eq $data2 if the file wasn't updated
```

Read the last 10 bytes from a file:

```
set f [open file.data]
# This is guaranteed to work with binary data but
# may fail with other encodings...
fconfigure $f -translation binary
seek $f -10 end
set data [read $f 10]
close $f
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

self

Keywords: self, call, introspection, object

Name

self - method call internal introspection

Synopsis

package require TclOO

self ?*subcommand*?

Description

The **self** command, which should only be used from within the context of a call to a method (i.e. inside a method, constructor or destructor body) is used to allow the method to discover information about how it was called. It takes an argument, *subcommand*, that tells it what sort of information is actually desired; if omitted the result will be the same as if **self object** was invoked. The supported subcommands are:

self call
: This returns a two-element list describing the method implementations used to implement the current call chain. The first element is the same as would be reported by info object **call** for the current method (except that this also reports useful values from within constructors and destructors, whose names are reported as and respectively), and the second element is an index into the first element's list that indicates which actual implementation is currently executing (the first implementation to execute is always at index 0).

self caller
: When the method was invoked from inside another object method, this subcommand returns a three element list describing the containing object and method. The first element describes the declaring object or class of the method, the second element is the name of the object on which the containing method was invoked, and the third element is the name of the method (with the strings and indicating constructors and destructors respectively).

self class
: This returns the name of the class that the current method was defined within. Note that this will change as the chain of method implementations is traversed with next, and that if the method was defined on an object then this will fail.

  If you want the class of the current object, you need to use this other construct:

```
info object class [self object]
```

self filter
: When invoked inside a filter, this subcommand returns a three element list describing the filter. The first element gives the name of the object or class that declared the filter (note that this may be different from the object or class that provided the implementation of the filter), the second element is either **object** or **class** depending on whether the declaring entity was an object or class, and the third element is the name of the filter.

self method
: This returns the name of the current method (with the strings and indicating constructors and destructors respectively).

self namespace
: This returns the name of the unique namespace of the object that the method was invoked upon.

self next
: When invoked from a method that is not at the end of a call chain (i.e. where the next command will invoke an actual method implementation), this subcommand returns a two element list describing the next element in the method call chain; the first element is the name of the class or object that declares the next part of the call chain, and the second element is the name of the method (with the strings and indicating constructors and

destructors respectively). If invoked from a method that is at the end of a call chain, this subcommand returns the empty string.

self object
This returns the name of the object that the method was invoked upon.

self target
When invoked inside a filter implementation, this subcommand returns a two element list describing the method being filtered. The first element will be the name of the declarer of the method, and the second element will be the actual name of the method.

Examples

This example shows basic use of **self** to provide information about the current object:

```
oo::class create c {
    method foo {} {
        puts "this is the [self] object"
    }
}
c create a
c create b
a foo               → prints "this is the ::a object"
b foo               → prints "this is the ::b object"
```

This demonstrates what a method call chain looks like, and how traversing along it changes the index into it:

```
oo::class create c {
    method x {} {
        puts "Cls: [self call]"
    }
}
c create a
oo::objdefine a {
    method x {} {
        puts "Obj: [self call]"
        next
        puts "Obj: [self call]"
    }
}
a x     → Obj: {{method x object method} {method x ::c method}} 0
        → Cls: {{method x object method} {method x ::c method}} 1
        → Obj: {{method x object method} {method x ::c method}} 0
```

Copyright (c) 2007 Donal K. Fellows

set

Keywords: set, read, write, variable

Name

set - Read and write variables

Synopsis

set *varName* ?*value*?

Description

Returns the value of variable *varName*. If *value* is specified, then set the value of *varName* to *value*, creating a new variable if one does not already exist, and return its value. If *varName* contains an open parenthesis and ends with a close parenthesis, then it refers to an array element: the characters before the first open parenthesis are the name of the array, and the characters between the parentheses are the index within the array. Otherwise *varName* refers to a scalar variable.

If *varName* includes namespace qualifiers (in the array name if it refers to an array element), or if *varName* is unqualified (does not include the names of any containing namespaces) but no procedure is active, *varName* refers to a namespace variable resolved according to the rules described under **NAME RESOLUTION** in the namespace manual page.

If a procedure is active and *varName* is unqualified, then *varName* refers to a parameter or local variable of the procedure, unless *varName* was declared to resolve differently through one of the global, variable or upvar commands.

Examples

Store a random number in the variable *r*:

```
set r [expr {rand()}]
```

Store a short message in an array element:

```
set anAry(msg) "Hello, World!"
```

Store a short message in an array element specified by a variable:

```
set elemName "msg"
set anAry($elemName) "Hello, World!"
```

Copy a value into the variable *out* from a variable whose name is stored in the *vbl* (note that it is often easier to use arrays in practice instead of doing double-dereferencing):

```
set in0 "small random"
set in1 "large random"
set vbl in[expr {rand() >= 0.5}]
set out [set $vbl]
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

socket

Keywords: socket, asynchronous I/O, bind, channel, connection, domain name, host, network address, tcp

Name

socket - Open a TCP network connection

Synopsis

socket ?*options*? *host port*
socket -server *command* ?*options*? *port*

Description

This command opens a network socket and returns a channel identifier that may be used in future invocations of commands like read, puts and flush. At present only the TCP network protocol is supported over IPv4 and IPv6; future releases may include support for additional protocols. The **socket** command may be used to open either the client or server side of a connection, depending on whether the **-server** switch is specified.

Note that the default encoding for *all* sockets is the system encoding, as returned by encoding system. Most of the time, you will need to use chan configure to alter this to something else, such as *utf-8* (ideal for communicating with other Tcl processes) or *iso8859-1* (useful for many network protocols, especially the older ones).

Client sockets

If the **-server** option is not specified, then the client side of a connection is opened and the command returns a channel identifier that can be used for both reading and writing. *Port* and *host* specify a port to connect to; there must be a server accepting connections on this port. *Port* is an integer port number (or service name, where supported and understood by the host operating system) and *host* is either a domain-style name such as **www.tcl.tk** or a numerical IPv4 or IPv6 address such as **127.0.0.1** or **2001:DB8::1**. Use *localhost* to refer to the host on which the command is invoked.

The following options may also be present before *host* to specify additional information about the connection:

-myaddr *addr*
: *Addr* gives the domain-style name or numerical IP address of the client-side network interface to use for the connection. This option may be useful if the client machine has multiple network interfaces. If the option is omitted then the client-side interface will be chosen by the system software.

-myport *port*
: *Port* specifies an integer port number (or service name, where supported and understood by the host operating system) to use for the client's side of the connection. If this option is omitted, the client's port number will be chosen at random by the system software.

-async
: This option will cause the client socket to be connected asynchronously. This means that the socket will be created immediately but may not yet be connected to the server, when the call to **socket** returns.

  When a gets or flush is done on the socket before the connection attempt succeeds or fails, if the socket is in blocking mode, the operation will wait until the connection is completed or fails. If the socket is in nonblocking mode and a gets or flush is done on the socket before the connection attempt succeeds or fails, the operation returns immediately and fblocked on the socket returns 1. Synchronous client sockets may be switched (after they have connected) to

operating in asynchronous mode using:

```
chan configure chan -blocking 0
```

See the chan configure command for more details.

The Tcl event loop should be running while an asynchronous connection is in progress, because it may have to do several connection attempts in the background. Running the event loop also allows you to set up a writable channel event on the socket to get notified when the asynchronous connection has succeeded or failed. See the vwait and the chan commands for more details on the event loop and channel events.

The chan configure option **-connecting** may be used to check if the connect is still running. To verify a successful connect, the option **-error** may be checked when **-connecting** returned 0.

Operation without the event queue requires at the moment calls to chan configure to advance the internal state machine.

Server sockets

If the **-server** option is specified then the new socket will be a server that listens on the given *port* (either an integer or a service name, where supported and understood by the host operating system; if *port* is zero, the operating system will allocate a free port to the server socket which may be discovered by using chan configure to read the **-sockname** option). If the host supports both, IPv4 and IPv6, the socket will listen on both address families. Tcl will automatically accept connections to the given port. For each connection Tcl will create a new channel that may be used to communicate with the client. Tcl then invokes *command* (properly a command prefix list, see the Examples below) with three additional arguments: the name of the new channel, the address, in network address notation, of the client's host, and the client's port number.

The following additional option may also be specified before *port*:

-myaddr *addr*
: *Addr* gives the domain-style name or numerical IP address of the server-side network interface to use for the connection. This option may be useful if the server machine has multiple network interfaces. If the option is omitted then the server socket is bound to the wildcard address so that it can accept connections from any interface. If *addr* is a domain name that resolves to multiple IP addresses that are available on the local machine, the socket will listen on all of them.

Server channels cannot be used for input or output; their sole use is to accept new client connections. The channels created for each incoming client connection are opened for input and output. Closing the server channel shuts down the server so that no new connections will be accepted; however, existing connections will be unaffected.

Server sockets depend on the Tcl event mechanism to find out when new connections are opened. If the application does not enter the event loop, for example by invoking the vwait command or calling the C procedure Tcl_DoOneEvent, then no connections will be accepted.

If *port* is specified as zero, the operating system will allocate an unused port for use as a server socket. The port number actually allocated may be retrieved from the created server socket

using the chan configure command to retrieve the **-sockname** option as described below.

Configuration options

The chan configure command can be used to query several readonly configuration options for socket channels:

-error
: This option gets the current error status of the given socket. This is useful when you need to determine if an asynchronous connect operation succeeded. If there was an error, the error message is returned. If there was no error, an empty string is returned.

  Note that the error status is reset by the read operation; this mimics the underlying getsockopt(SO_ERROR) call.

-sockname
: For client sockets (including the channels that get created when a client connects to a server socket) this option returns a list of three elements, the address, the host name and the port number for the socket. If the host name cannot be computed, the second element is identical to the address, the first element of the list.

  For server sockets this option returns a list of a multiple of three elements each group of which have the same meaning as described above. The list contains more than one group when the server socket was created without **-myaddr** or with the argument to **-myaddr** being a domain name that resolves multiple IP addresses that are local to the invoking host.

-peername
: This option is not supported by server sockets. For client and accepted sockets, this option returns a list of three elements; these are the address, the host name and the port to which the peer socket is connected or bound. If the host name cannot be computed, the second element of the list is identical to the address, its first element.

-connecting
: This option is not supported by server sockets. For client sockets, this option returns 1 if an asyncroneous connect is still in progress, 0 otherwise.

Examples

Here is a very simple time server:

```
proc Server {startTime channel clientaddr clientport} {
    puts "Connection from $clientaddr registered"
    set now [clock seconds]
    puts $channel [clock format $now]
    puts $channel "[expr {$now - $startTime}] since start"
    close $channel
}

socket -server [list Server [clock seconds]] 9900
vwait forever
```

And here is the corresponding client to talk to the server and extract some information:

```
set server localhost
set sockChan [socket $server 9900]
gets $sockChan line1
gets $sockChan line2
close $sockChan
puts "The time on $server is $line1"
puts "That is [lindex $line2 0]s since the server started"
```

History
Support for IPv6 was added in Tcl 8.6.

Copyright (c) 1996 Sun Microsystems, Inc.
Copyright (c) 1998-1999 Scriptics Corporation.

source
Keywords: source, file, script

Name
source - Evaluate a file or resource as a Tcl script

Synopsis
source *fileName*
source -encoding *encodingName fileName*

Description
This command takes the contents of the specified file or resource and passes it to the Tcl interpreter as a text script. The return value from **source** is the return value of the last command executed in the script. If an error occurs in evaluating the contents of the script then the **source** command will return that error. If a return command is invoked from within the script then the remainder of the file will be skipped and the **source** command will return normally with the result from the return command.

The end-of-file character for files is "\32" (^Z) for all platforms. The source command will read files up to this character. This restriction does not exist for the read or gets commands, allowing for files containing code and data segments (scripted documents). If you require a "^Z" in code for string comparison, you can use "\032" or "\u001a", which will be safely substituted by the Tcl interpreter into "^Z".

A leading BOM (Byte order mark) contained in the file is ignored for unicode encodings (utf-8, unicode).

The **-encoding** option is used to specify the encoding of the data stored in *fileName*. When the **-encoding** option is omitted, the system encoding is assumed.

Example
Run the script in the file **foo.tcl** and then the script in the file **bar.tcl**:

```
source foo.tcl
source bar.tcl
```

Alternatively:

```
foreach scriptFile {foo.tcl bar.tcl} {
    source $scriptFile
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2000 Scriptics Corporation.

split

Keywords: split, list, string

Name

split - Split a string into a proper Tcl list

Synopsis

split *string* ?*splitChars*?

Description

Returns a list created by splitting *string* at each character that is in the *splitChars* argument. Each element of the result list will consist of the characters from *string* that lie between instances of the characters in *splitChars*. Empty list elements will be generated if *string* contains adjacent characters in *splitChars*, or if the first or last character of *string* is in *splitChars*. If *splitChars* is an empty string then each character of *string* becomes a separate element of the result list. *SplitChars* defaults to the standard white-space characters.

Examples

Divide up a USENET group name into its hierarchical components:

```
split "comp.lang.tcl" .
      → comp lang tcl
```

See how the **split** command splits on *every* character in *splitChars*, which can result in information loss if you are not careful:

```
split "alpha beta gamma" "temp"
      → al {ha b} {} {a ga} {} a
```

Extract the list words from a string that is not a well-formed list:

```
split "Example with {unbalanced brace character"
      → Example with \{unbalanced brace character
```

Split a string into its constituent characters

```
split "Hello world" {}
      → H e l l o { } w o r l d
```

Parsing record-oriented files

Parse a Unix /etc/passwd file, which consists of one entry per line, with each line consisting of a

colon-separated list of fields:

```
## Read the file
set fid [open /etc/passwd]
set content [read $fid]
close $fid

## Split into records on newlines
set records [split $content "\n"]

## Iterate over the records
foreach rec $records {

    ## Split into fields on colons
    set fields [split $rec ":"]

    ## Assign fields to variables and print some out...
    lassign $fields \
            userName password uid grp longName homeDir shell
    puts "$longName uses [file tail $shell] for a login shell"
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

string

Keywords: string, case conversion, compare, index, integer value, match, pattern, word, equal, ctype, character, reverse

Name

string - Manipulate strings

Synopsis

string *option arg* ?*arg* ...?

Description

Performs one of several string operations, depending on *option*. The legal *option*s (which may be abbreviated) are:

string cat ?*string1*? ?*string2*...?

Concatenate the given strings just like placing them directly next to each other and return the resulting compound string. If no strings are present, the result is an empty string.

This primitive is occasionally handier than juxtaposition of strings when mixed quoting is wanted, or when the aim is to return the result of a concatenation without resorting to return **-level 0**, and is more efficient than building a list of arguments and using join with an empty join string.

string compare ?-nocase? ?*-length length*? *string1 string2*

Perform a character-by-character comparison of strings *string1* and *string2*. Returns -1, 0, or 1, depending on whether *string1* is lexicographically less than, equal to, or greater than *string2*. If **-length** is specified, then only the first *length* characters are used in the comparison. If **-length** is negative, it is ignored. If **-nocase** is specified, then the strings are compared in a case-insensitive manner.

string equal ?-nocase? ?*-length length*? *string1 string2*

Perform a character-by-character comparison of strings *string1* and *string2*. Returns 1 if *string1* and *string2* are identical, or 0 when not. If **-length** is specified, then only the first *length* characters are used in the comparison. If **-length** is negative, it is ignored. If **-nocase** is specified, then the strings are compared in a case-insensitive manner.

string first *needleString haystackString* ?*startIndex*?

Search *haystackString* for a sequence of characters that exactly match the characters in *needleString*. If found, return the index of the first character in the first such match within *haystackString*. If not found, return -1. If *startIndex* is specified (in any of the forms described in String indices), then the search is constrained to start with the character in *haystackString* specified by the index. For example,

```
string first a 0a23456789abcdef 5
```

will return **10**, but

```
string first a 0123456789abcdef 11
```

will return **-1**.

string index *string charIndex*

Returns the *charIndex*'th character of the *string* argument. A *charIndex* of 0 corresponds to the first character of the string. *charIndex* may be specified as described in the String indices section.

If *charIndex* is less than 0 or greater than or equal to the length of the string then this command returns an empty string.

string is *class* ?-strict? ?*-failindex varname*? *string*

Returns 1 if *string* is a valid member of the specified character class, otherwise returns 0. If **-strict** is specified, then an empty string returns 0, otherwise an empty string will return 1 on any class. If **-failindex** is specified, then if the function returns 0, the index in the string where the class was no longer valid will be stored in the variable named *varname*. The *varname* will not be set if **string is** returns 1. The following character classes are recognized (the class name can be abbreviated):

alnum

Any Unicode alphabet or digit character.

alpha

Any Unicode alphabet character.

ascii

Any character with a value less than \u0080 (those that are in the 7-bit ascii range).

boolean

Any of the forms allowed to Tcl_GetBoolean.

control
Any Unicode control character.
digit
Any Unicode digit character. Note that this includes characters outside of the [0-9] range.
double
Any of the forms allowed to Tcl_GetDoubleFromObj.
entier
Any of the valid string formats for an integer value of arbitrary size in Tcl, with optional surrounding whitespace. The formats accepted are exactly those accepted by the C routine Tcl_GetBignumFromObj.
false
Any of the forms allowed to Tcl_GetBoolean where the value is false.
graph
Any Unicode printing character, except space.
integer
Any of the valid string formats for a 32-bit integer value in Tcl, with optional surrounding whitespace. In case of overflow in the value, 0 is returned and the *varname* will contain -1.
list
Any proper list structure, with optional surrounding whitespace. In case of improper list structure, 0 is returned and the *varname* will contain the index of the "element" where the list parsing fails, or -1 if this cannot be determined.
lower
Any Unicode lower case alphabet character.
print
Any Unicode printing character, including space.
punct
Any Unicode punctuation character.
space
Any Unicode whitespace character, mongolian vowel separator (U+180e), zero width space (U+200b), word joiner (U+2060) or zero width no-break space (U+feff) (=BOM).
true
Any of the forms allowed to Tcl_GetBoolean where the value is true.
upper
Any upper case alphabet character in the Unicode character set.
wideinteger
Any of the valid forms for a wide integer in Tcl, with optional surrounding whitespace. In case of overflow in the value, 0 is returned and the *varname* will contain -1.
wordchar
Any Unicode word character. That is any alphanumeric character, and any Unicode connector punctuation characters (e.g. underscore).
xdigit
Any hexadecimal digit character ([0-9A-Fa-f]).

In the case of **boolean**, **true** and **false**, if the function will return 0, then the *varname* will

always be set to 0, due to the varied nature of a valid boolean value.

string last *needleString haystackString* ?*lastIndex*?

Search *haystackString* for a sequence of characters that exactly match the characters in *needleString*. If found, return the index of the first character in the last such match within *haystackString*. If there is no match, then return -1. If *lastIndex* is specified (in any of the forms described in String indices), then only the characters in *haystackString* at or before the specified *lastIndex* will be considered by the search. For example,

```
string last a 0a23456789abcdef 15
```

will return **10**, but

```
string last a 0a23456789abcdef 9
```

will return **1**.

string length *string*

Returns a decimal string giving the number of characters in *string*. Note that this is not necessarily the same as the number of bytes used to store the string. If the value is a byte array value (such as those returned from reading a binary encoded channel), then this will return the actual byte length of the value.

string map ?-nocase? *mapping string*

Replaces substrings in *string* based on the key-value pairs in *mapping*. *mapping* is a list of *key value key value ...* as in the form returned by array get. Each instance of a key in the string will be replaced with its corresponding value. If **-nocase** is specified, then matching is done without regard to case differences. Both *key* and *value* may be multiple characters. Replacement is done in an ordered manner, so the key appearing first in the list will be checked first, and so on. *string* is only iterated over once, so earlier key replacements will have no affect for later key matches. For example,

```
string map {abc 1 ab 2 a 3 1 0} 1abcaababcababababc
```

will return the string **01321221**.

Note that if an earlier *key* is a prefix of a later one, it will completely mask the later one. So if the previous example is reordered like this,

```
string map {1 0 ab 2 a 3 abc 1} 1abcaababcababababc
```

it will return the string **02c322c222c**.

string match ?-nocase? *pattern string*

See if *pattern* matches *string*; return 1 if it does, 0 if it does not. If **-nocase** is specified, then the pattern attempts to match against the string in a case insensitive manner. For the two strings to match, their contents must be identical except that the following special sequences may appear in *pattern*:

*****

Matches any sequence of characters in *string*, including a null string.

?

Matches any single character in *string*.

[*chars*]
Matches any character in the set given by *chars*. If a sequence of the form *x*-*y* appears in *chars*, then any character between *x* and *y*, inclusive, will match. When used with **-nocase**, the end points of the range are converted to lower case first. Whereas {[A-z]} matches "_" when matching case-sensitively (since "_" falls between the "Z" and "a"), with **-nocase** this is considered like {[A-Za-z]} (and probably what was meant in the first place).

x
Matches the single character *x*. This provides a way of avoiding the special interpretation of the characters ***?[]*** in *pattern*.

string range *string first last*
Returns a range of consecutive characters from *string*, starting with the character whose index is *first* and ending with the character whose index is *last* (using the forms described in String indices). An index of **0** refers to the first character of the string; an index of **end** refers to last character of the string. *first* and *last* may be specified as for the **index** method. If *first* is less than zero then it is treated as if it were zero, and if *last* is greater than or equal to the length of the string then it is treated as if it were **end**. If *first* is greater than *last* then an empty string is returned.

string repeat *string count*
Returns a string consisting of *string* concatenated with itself *count* times. If *count* is 0, the empty string will be returned.

string replace *string first last ?newstring?*
Removes a range of consecutive characters from *string*, starting with the character whose index is *first* and ending with the character whose index is *last* (using the forms described in String indices). An index of 0 refers to the first character of the string. *First* and *last* may be specified as for the **index** method. If *newstring* is specified, then it is placed in the removed character range. If *first* is less than zero then it is treated as if it were zero, and if *last* is greater than or equal to the length of the string then it is treated as if it were **end**. The initial string is returned untouched, if *first* is greater than *last*, or if *first* is equal to or greater than the length of the initial string, or *last* is less than 0.

string reverse *string*
Returns a string that is the same length as *string* but with its characters in the reverse order.

string tolower *string ?first? ?last?*
Returns a value equal to *string* except that all upper (or title) case letters have been converted to lower case. If *first* is specified, it refers to the first char index in the string to start modifying. If *last* is not specified, only the character at position *first* is converted, otherwise *last* refers to the char index in the string to stop at (inclusive). *first* and *last* may be specified using the forms described in String indices.

string totitle *string ?first? ?last?*
Returns a value equal to *string* except that the first character in *string* is converted to its Unicode title case variant (or upper case if there is no title case variant) and the rest of the string is converted to lower case. If *first* is specified, it refers to the first char index in the string to start modifying. If *last* is not specified, only the character at position *first* is converted, otherwise *last* refers to the char index in the string to stop at (inclusive). *first* and *last* may be specified using the forms described in String indices.

string toupper *string* ?*first*? ?*last*?
Returns a value equal to *string* except that all lower (or title) case letters have been converted to upper case. If *first* is specified, it refers to the first char index in the string to start modifying. If *last* is not specified, only the character at position *first* is converted, otherwise *last* refers to the char index in the string to stop at (inclusive). *first* and *last* may be specified using the forms described in String indices.

string trim *string* ?*chars*?
Returns a value equal to *string* except that any leading or trailing characters present in the string given by *chars* are removed. If *chars* is not specified then white space is removed (any character for which **string is space** returns 1, and "\0").

string trimleft *string* ?*chars*?
Returns a value equal to *string* except that any leading characters present in the string given by *chars* are removed. If *chars* is not specified then white space is removed (any character for which **string is space** returns 1, and "\0").

string trimright *string* ?*chars*?
Returns a value equal to *string* except that any trailing characters present in the string given by *chars* are removed. If *chars* is not specified then white space is removed (any character for which **string is space** returns 1, and "\0").

Obsolete subcommands

These subcommands are currently supported, but are likely to go away in a future release as their functionality is either virtually never used or highly misleading.

string bytelength *string*
Returns a decimal string giving the number of bytes used to represent *string* in memory when encoded as Tcl's internal modified UTF-8; Tcl may use other encodings for *string* as well, and does not guarantee to only use a single encoding for a particular *string*. Because UTF-8 uses a variable number of bytes to represent Unicode characters, the byte length will not be the same as the character length in general. The cases where a script cares about the byte length are rare.

In almost all cases, you should use the **string length** operation (including determining the length of a Tcl byte array value). Refer to the Tcl_NumUtfChars manual entry for more details on the UTF-8 representation.

Formally, the **string bytelength** operation returns the content of the *length* field of the Tcl_Obj structure, after calling Tcl_GetString to ensure that the *bytes* field is populated. This is highly unlikely to be useful to Tcl scripts, as Tcl's internal encoding is not strict UTF-8, but rather a modified CESU-8 with a denormalized NUL (identical to that used in a number of places by Java's serialization mechanism) to enable basic processing with non-Unicode-aware C functions. As this representation should only ever be used by Tcl's implementation, the number of bytes used to store the representation is of very low value (except to C extension code, which has direct access for the purpose of memory management, etc.)

Compatibility note: it is likely that this subcommand will be withdrawn in a future version of Tcl. It is better to use the encoding convertto command to convert a string to a known encoding and then apply **string length** to that.

```
string length [encoding convertto utf-8 $theString]
```

string wordend *string charIndex*
Returns the index of the character just after the last one in the word containing character *charIndex* of *string*. *charIndex* may be specified using the forms in String indices. A word is considered to be any contiguous range of alphanumeric (Unicode letters or decimal digits) or underscore (Unicode connector punctuation) characters, or any single character other than these.

string wordstart *string charIndex*
Returns the index of the first character in the word containing character *charIndex* of *string*. *charIndex* may be specified using the forms in String indices. A word is considered to be any contiguous range of alphanumeric (Unicode letters or decimal digits) or underscore (Unicode connector punctuation) characters, or any single character other than these.

String indices

When referring to indices into a string (e.g., for **string index** or **string range**) the following formats are supported:

integer
For any index value that passes **string is integer -strict**, the char specified at this integral index (e.g., **2** would refer to the "c" in "abcd").

end
The last char of the string (e.g., **end** would refer to the "d" in "abcd").

end-*N*
The last char of the string minus the specified integer offset *N* (e.g., "**end-1**" would refer to the "c" in "abcd").

end+*N*
The last char of the string plus the specified integer offset *N* (e.g., "**end+-1**" would refer to the "c" in "abcd").

M+N
The char specified at the integral index that is the sum of integer values *M* and *N* (e.g., "**1+1**" would refer to the "c" in "abcd").

M-N
The char specified at the integral index that is the difference of integer values *M* and *N* (e.g., "**2-1**" would refer to the "b" in "abcd").

In the specifications above, the integer value *M* contains no trailing whitespace and the integer value *N* contains no leading whitespace.

Example

Test if the string in the variable *string* is a proper non-empty prefix of the string **foobar**.

```
set length [string length $string]
if {$length == 0} {
    set isPrefix 0
} else {
    set isPrefix [string equal -length $length $string "foobar"]
}
```

Copyright (c) 1993 The Regents of the University of California.

Copyright (c) 1994-1996 Sun Microsystems, Inc.

subst

Keywords: subst, backslash substitution, command substitution, quoting, substitution, variable substitution

Name

subst - Perform backslash, command, and variable substitutions

Synopsis

subst ?**-nobackslashes**? ?**-nocommands**? ?**-novariables**? *string*

Description

This command performs variable substitutions, command substitutions, and backslash substitutions on its *string* argument and returns the fully-substituted result. The substitutions are performed in exactly the same way as for Tcl commands. As a result, the *string* argument is actually substituted twice, once by the Tcl parser in the usual fashion for Tcl commands, and again by the *subst* command.

If any of the **-nobackslashes**, **-nocommands**, or **-novariables** are specified, then the corresponding substitutions are not performed. For example, if **-nocommands** is specified, command substitution is not performed: open and close brackets are treated as ordinary characters with no special interpretation.

Note that the substitution of one kind can include substitution of other kinds. For example, even when the **-novariables** option is specified, command substitution is performed without restriction. This means that any variable substitution necessary to complete the command substitution will still take place. Likewise, any command substitution necessary to complete a variable substitution will take place, even when **-nocommands** is specified. See the Examples below.

If an error occurs during substitution, then **subst** will return that error. If a break exception occurs during command or variable substitution, the result of the whole substitution will be the string (as substituted) up to the start of the substitution that raised the exception. If a continue exception occurs during the evaluation of a command or variable substitution, an empty string will be substituted for that entire command or variable substitution (as long as it is well-formed Tcl.) If a return exception occurs, or any other return code is returned during command or variable substitution, then the returned value is substituted for that substitution. See the Examples below. In this way, all exceptional return codes are "caught" by **subst**. The **subst** command itself will either return an error, or will complete successfully.

Examples

When it performs its substitutions, *subst* does not give any special treatment to double quotes or curly braces (except within command substitutions) so the script

```
set a 44
subst {xyz {$a}}
```

returns "**xyz {44}**", not "**xyz {$a}**" and the script

```
set a "p\} q \{r"
subst {xyz {$a}}
```

returns "**xyz {p} q {r}**", not "**xyz {p\} q \{r}**".

When command substitution is performed, it includes any variable substitution necessary to evaluate the script.

```
set a 44
subst -novariables {$a [format $a]}
```

returns "**$a 44**", not "**$a $a**". Similarly, when variable substitution is performed, it includes any command substitution necessary to retrieve the value of the variable.

```
proc b {} {return c}
array set a {c c [b] tricky}
subst -nocommands {[b] $a([b])}
```

returns "**[b] c**", not "**[b] tricky**".

The continue and break exceptions allow command substitutions to prevent substitution of the rest of the command substitution and the rest of *string* respectively, giving script authors more options when processing text using *subst*. For example, the script

```
subst {abc,[break],def}
```

returns "**abc,**", not "**abc,,def**" and the script

```
subst {abc,[continue;expr {1+2}],def}
```

returns "**abc,,def**", not "**abc,3,def**".

Other exceptional return codes substitute the returned value

```
subst {abc,[return foo;expr {1+2}],def}
```

returns "**abc,foo,def**", not "**abc,3,def**" and

```
subst {abc,[return -code 10 foo;expr {1+2}],def}
```

also returns "**abc,foo,def**", not "**abc,3,def**".

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2001 Donal K. Fellows

switch

Keywords: switch, match, regular expression

Name

switch - Evaluate one of several scripts, depending on a given value

Synopsis

switch ?*options*? *string pattern body* ?*pattern body* ...?
switch ?*options*? *string {pattern body ?pattern body ...?}*

Description

The **switch** command matches its *string* argument against each of the *pattern* arguments in order. As soon as it finds a *pattern* that matches *string* it evaluates the following *body* argument by passing it recursively to the Tcl interpreter and returns the result of that evaluation. If the last *pattern* argument is **default** then it matches anything. If no *pattern* argument matches *string* and no default is given, then the **switch** command returns an empty string.

If the initial arguments to **switch** start with **-** then they are treated as options unless there are exactly two arguments to **switch** (in which case the first must the *string* and the second must be the *pattern*/*body* list). The following options are currently supported:

-exact
: Use exact matching when comparing *string* to a pattern. This is the default.

-glob
: When matching *string* to the patterns, use glob-style matching (i.e. the same as implemented by the string match command).

-regexp
: When matching *string* to the patterns, use regular expression matching (as described in the re_syntax reference page).

-nocase
: Causes comparisons to be handled in a case-insensitive manner.

-matchvar *varName*
: This option (only legal when **-regexp** is also specified) specifies the name of a variable into which the list of matches found by the regular expression engine will be written. The first element of the list written will be the overall substring of the input string (i.e. the *string* argument to **switch**) matched, the second element of the list will be the substring matched by the first capturing parenthesis in the regular expression that matched, and so on. When a **default** branch is taken, the variable will have the empty list written to it. This option may be specified at the same time as the **-indexvar** option.

-indexvar *varName*
: This option (only legal when **-regexp** is also specified) specifies the name of a variable into which the list of indices referring to matching substrings found by the regular expression engine will be written. The first element of the list written will be a two-element list specifying the index of the start and index of the first character after the end of the overall substring of the input string (i.e. the *string* argument to **switch**) matched, in a similar way to the **-indices** option to the regexp can obtain. Similarly, the second element of the list refers to the first capturing parenthesis in the regular expression that matched, and so on. When a **default** branch is taken, the variable will have the empty list written to it. This option may be specified at the same time as the **-matchvar** option.

--
: Marks the end of options. The argument following this one will be treated as *string* even if it starts with a **-**. This is not required when the matching patterns and bodies are grouped together in a single argument.

Two syntaxes are provided for the *pattern* and *body* arguments. The first uses a separate argument for each of the patterns and commands; this form is convenient if substitutions are desired on some of the patterns or commands. The second form places all of the patterns and

commands together into a single argument; the argument must have proper list structure, with the elements of the list being the patterns and commands. The second form makes it easy to construct multi-line switch commands, since the braces around the whole list make it unnecessary to include a backslash at the end of each line. Since the *pattern* arguments are in braces in the second form, no command or variable substitutions are performed on them; this makes the behavior of the second form different than the first form in some cases.

If a *body* is specified as "**-**" it means that the *body* for the next pattern should also be used as the body for this pattern (if the next pattern also has a body of "**-**" then the body after that is used, and so on). This feature makes it possible to share a single *body* among several patterns.

Beware of how you place comments in **switch** commands. Comments should only be placed **inside** the execution body of one of the patterns, and not intermingled with the patterns.

Examples

The **switch** command can match against variables and not just literals, as shown here (the result is *2*):

```
set foo "abc"
switch abc a - b {expr {1}} $foo {expr {2}} default {expr {3}}
```

Using glob matching and the fall-through body is an alternative to writing regular expressions with alternations, as can be seen here (this returns *1*):

```
switch -glob aaab {
    a*b     -
    b       {expr {1}}
    a*      {expr {2}}
    default {expr {3}}
}
```

Whenever nothing matches, the **default** clause (which must be last) is taken. This example has a result of *3*:

```
switch xyz {
    a -
    b {
        # Correct Comment Placement
        expr {1}
    }
    c {
        expr {2}
    }
    default {
        expr {3}
    }
}
```

When matching against regular expressions, information about what exactly matched is easily obtained using the **-matchvar** option:

```
switch -regexp -matchvar foo -- $bar {
    a(b*)c {
        puts "Found [string length [lindex $foo 1]] 'b's"
    }
    d(e*)f(g*)h {
        puts "Found [string length [lindex $foo 1]] 'e's and\
                [string length [lindex $foo 2]] 'g's"
    }
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

tailcall

Keywords: tailcall, call, recursion, tail recursion

Name

tailcall - Replace the current procedure with another command

Synopsis

tailcall *command* ?*arg* ...?

Description

The **tailcall** command replaces the currently executing procedure, lambda application, or method with another command. The *command*, which will have *arg ...* passed as arguments if they are supplied, will be looked up in the current namespace context, not in the caller's. Apart from that difference in resolution, it is equivalent to:

```
return [uplevel 1 [list command ?arg ...?]]
```

This command may not be invoked from within an uplevel into a procedure or inside a catch inside a procedure or lambda.

Example

Compute the factorial of a number.

```
proc factorial {n {accum 1}} {
    if {$n < 2} {
        return $accum
    }
    tailcall factorial [expr {$n - 1}] [expr {$accum * $n}]
}
```

Print the elements of a list with alternating lines having different indentations.

```
proc printList {theList} {
    if {[llength $theList]} {
        puts "> [lindex $theList 0]"
        tailcall printList2 [lrange $theList 1 end]
```

```
    }
}
proc printList2 {theList} {
    if {[llength $theList]} {
        puts "< [lindex $theList 0]"
        tailcall printList [lrange $theList 1 end]
    }
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

tclsh

Keywords: tclsh, application, argument, interpreter, prompt, script file, shell

Name

tclsh - Simple shell containing Tcl interpreter

Synopsis

tclsh ?*-encoding name*? ?*fileName arg arg* ...?

Description

Tclsh is a shell-like application that reads Tcl commands from its standard input or from a file and evaluates them. If invoked with no arguments then it runs interactively, reading Tcl commands from standard input and printing command results and error messages to standard output. It runs until the exit command is invoked or until it reaches end-of-file on its standard input. If there exists a file **.tclshrc** (or **tclshrc.tcl** on the Windows platforms) in the home directory of the user, interactive **tclsh** evaluates the file as a Tcl script just before reading the first command from standard input.

Script files

If **tclsh** is invoked with arguments then the first few arguments specify the name of a script file, and, optionally, the encoding of the text data stored in that script file. Any additional arguments are made available to the script as variables (see below). Instead of reading commands from standard input **tclsh** will read Tcl commands from the named file; **tclsh** will exit when it reaches the end of the file. The end of the file may be marked either by the physical end of the medium, or by the character, "\032" ("\u001a", control-Z). If this character is present in the file, the **tclsh** application will read text up to but not including the character. An application that requires this character in the file may safely encode it as "\032", "\x1A", or "\u001a"; or may generate it by use of commands such as format or binary. There is no automatic evaluation of **.tclshrc** when the name of a script file is presented on the **tclsh** command line, but the script file can always source it if desired.

If you create a Tcl script in a file whose first line is

```
#!/usr/local/bin/tclsh
```

then you can invoke the script file directly from your shell if you mark the file as executable. This assumes that **tclsh** has been installed in the default location in /usr/local/bin; if it is installed

somewhere else then you will have to modify the above line to match. Many UNIX systems do not allow the **#!** line to exceed about 30 characters in length, so be sure that the **tclsh** executable can be accessed with a short file name.

An even better approach is to start your script files with the following three lines:

```
#!/bin/sh
# the next line restarts using tclsh \
exec tclsh "$0" ${1+"$@"}
```

This approach has three advantages over the approach in the previous paragraph. First, the location of the **tclsh** binary does not have to be hard-wired into the script: it can be anywhere in your shell search path. Second, it gets around the 30-character file name limit in the previous approach. Third, this approach will work even if **tclsh** is itself a shell script (this is done on some systems in order to handle multiple architectures or operating systems: the **tclsh** script selects one of several binaries to run). The three lines cause both **sh** and **tclsh** to process the script, but the exec is only executed by **sh**. **sh** processes the script first; it treats the second line as a comment and executes the third line. The exec statement cause the shell to stop processing and instead to start up **tclsh** to reprocess the entire script. When **tclsh** starts up, it treats all three lines as comments, since the backslash at the end of the second line causes the third line to be treated as part of the comment on the second line.

You should note that it is also common practice to install tclsh with its version number as part of the name. This has the advantage of allowing multiple versions of Tcl to exist on the same system at once, but also the disadvantage of making it harder to write scripts that start up uniformly across different versions of Tcl.

Variables

Tclsh sets the following global Tcl variables in addition to those created by the Tcl library itself (such as env, which maps environment variables such as **PATH** into Tcl):

argc
: Contains a count of the number of *arg* arguments (0 if none), not including the name of the script file.

argv
: Contains a Tcl list whose elements are the *arg* arguments, in order, or an empty string if there are no *arg* arguments.

argv0
: Contains *fileName* if it was specified. Otherwise, contains the name by which **tclsh** was invoked.

tcl_interactive
: Contains 1 if **tclsh** is running interactively (no *fileName* was specified and standard input is a terminal-like device), 0 otherwise.

Prompts

When **tclsh** is invoked interactively it normally prompts for each command with "**%** ". You can change the prompt by setting the global variables **tcl_prompt1** and **tcl_prompt2**. If variable **tcl_prompt1** exists then it must consist of a Tcl script to output a prompt; instead of outputting a prompt **tclsh** will evaluate the script in **tcl_prompt1**. The variable **tcl_prompt2** is used in a

similar way when a newline is typed but the current command is not yet complete; if **tcl_prompt2** is not set then no prompt is output for incomplete commands.

Standard channels

See Tcl_StandardChannels for more explanations.

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

tcltest

Keywords: tcltest, test, test harness, test suite

Name

tcltest - Test harness support code and utilities

Synopsis

package require tcltest ?**2.5**?

tcltest::test *name description* ?*-option value ...*?
tcltest::test *name description* ?*constraints*? *body result*

tcltest::loadTestedCommands
tcltest::makeDirectory *name* ?*directory*?
tcltest::removeDirectory *name* ?*directory*?
tcltest::makeFile *contents name* ?*directory*?
tcltest::removeFile *name* ?*directory*?
tcltest::viewFile *name* ?*directory*?
tcltest::cleanupTests ?*runningMultipleTests*?
tcltest::runAllTests

tcltest::configure
tcltest::configure *-option*
tcltest::configure *-option value* ?*-option value ...*?
tcltest::customMatch *mode command*
tcltest::testConstraint *constraint* ?*value*?
tcltest::outputChannel ?*channel*?
tcltest::errorChannel ?*channel*?
tcltest::interpreter ?*interp*?

tcltest::debug ?*level*?
tcltest::errorFile ?*filename*?
tcltest::limitConstraints ?*boolean*?
tcltest::loadFile ?*filename*?
tcltest::loadScript ?*script*?
tcltest::match ?*patternList*?
tcltest::matchDirectories ?*patternList*?
tcltest::matchFiles ?*patternList*?
tcltest::outputFile ?*filename*?
tcltest::preserveCore ?*level*?
tcltest::singleProcess ?*boolean*?

tcltest::skip ?*patternList*?
tcltest::skipDirectories ?*patternList*?
tcltest::skipFiles ?*patternList*?
tcltest::temporaryDirectory ?*directory*?
tcltest::testsDirectory ?*directory*?
tcltest::verbose ?*level*?

tcltest::test *name description optionList*
tcltest::normalizeMsg *msg*
tcltest::normalizePath *pathVar*
tcltest::workingDirectory ?*dir*?

Description

The **tcltest** package provides several utility commands useful in the construction of test suites for code instrumented to be run by evaluation of Tcl commands. Notably the built-in commands of the Tcl library itself are tested by a test suite using the tcltest package.

All the commands provided by the **tcltest** package are defined in and exported from the **::tcltest** namespace, as indicated in the Synopsis above. In the following sections, all commands will be described by their simple names, in the interest of brevity.

The central command of **tcltest** is **test** that defines and runs a test. Testing with **test** involves evaluation of a Tcl script and comparing the result to an expected result, as configured and controlled by a number of options. Several other commands provided by **tcltest** govern the configuration of **test** and the collection of many **test** commands into test suites.

See Creating test suites with tcltest below for an extended example of how to use the commands of **tcltest** to produce test suites for your Tcl-enabled code.

Commands

test *name description* ?*-option value* ...?
: Defines and possibly runs a test with the name *name* and description *description*. The name and description of a test are used in messages reported by **test** during the test, as configured by the options of **tcltest**. The remaining *option value* arguments to **test** define the test, including the scripts to run, the conditions under which to run them, the expected result, and the means by which the expected and actual results should be compared. See Tests below for a complete description of the valid options and how they define a test. The **test** command returns an empty string.

test *name description* ?*constraints*? *body result*
: This form of **test** is provided to support test suites written for version 1 of the **tcltest** package, and also a simpler interface for a common usage. It is the same as "**test** *name description* **-constraints** *constraints* **-body** *body* **-result** *result*". All other options to **test** take their default values. When *constraints* is omitted, this form of **test** can be distinguished from the first because all *option*s begin with "-".

loadTestedCommands
: Evaluates in the caller's context the script specified by **configure -load** or **configure -loadfile**. Returns the result of that script evaluation, including any error raised by the script. Use this command and the related configuration options to provide the commands to be tested to the interpreter running the test suite.

makeFile *contents name* ?*directory*?
Creates a file named *name* relative to directory *directory* and write *contents* to that file using the encoding encoding system. If *contents* does not end with a newline, a newline will be appended so that the file named *name* does end with a newline. Because the system encoding is used, this command is only suitable for making text files. The file will be removed by the next evaluation of **cleanupTests**, unless it is removed by **removeFile** first. The default value of *directory* is the directory **configure -tmpdir**. Returns the full path of the file created. Use this command to create any text file required by a test with contents as needed.

removeFile *name* ?*directory*?
Forces the file referenced by *name* to be removed. This file name should be relative to *directory*. The default value of *directory* is the directory **configure -tmpdir**. Returns an empty string. Use this command to delete files created by **makeFile**.

makeDirectory *name* ?*directory*?
Creates a directory named *name* relative to directory *directory*. The directory will be removed by the next evaluation of **cleanupTests**, unless it is removed by **removeDirectory** first. The default value of *directory* is the directory **configure -tmpdir**. Returns the full path of the directory created. Use this command to create any directories that are required to exist by a test.

removeDirectory *name* ?*directory*?
Forces the directory referenced by *name* to be removed. This directory should be relative to *directory*. The default value of *directory* is the directory **configure -tmpdir**. Returns an empty string. Use this command to delete any directories created by **makeDirectory**.

viewFile *file* ?*directory*?
Returns the contents of *file*, except for any final newline, just as read -nonewline would return. This file name should be relative to *directory*. The default value of *directory* is the directory **configure -tmpdir**. Use this command as a convenient way to turn the contents of a file generated by a test into the result of that test for matching against an expected result. The contents of the file are read using the system encoding, so its usefulness is limited to text files.

cleanupTests
Intended to clean up and summarize after several tests have been run. Typically called once per test file, at the end of the file after all tests have been completed. For best effectiveness, be sure that the **cleanupTests** is evaluated even if an error occurs earlier in the test file evaluation.

Prints statistics about the tests run and removes files that were created by **makeDirectory** and **makeFile** since the last **cleanupTests**. Names of files and directories in the directory **configure -tmpdir** created since the last **cleanupTests**, but not created by **makeFile** or **makeDirectory** are printed to **outputChannel**. This command also restores the original shell environment, as described by the global env array. Returns an empty string.

runAllTests
This is a main command meant to run an entire suite of tests, spanning multiple files and/or directories, as governed by the configurable options of **tcltest**. See Running all tests below for a complete description of the many variations possible with **runAllTests**.

Configuration commands

configure

Returns the list of configurable options supported by **tcltest**. See Configurable options below for the full list of options, their valid values, and their effect on **tcltest** operations.

configure *option*

Returns the current value of the supported configurable option *option*. Raises an error if *option* is not a supported configurable option.

configure *option value* ?*-option value* ...?

Sets the value of each configurable option *option* to the corresponding value *value*, in order. Raises an error if an *option* is not a supported configurable option, or if *value* is not a valid value for the corresponding *option*, or if a *value* is not provided. When an error is raised, the operation of **configure** is halted, and subsequent *option value* arguments are not processed.

If the environment variable **::env(TCLTEST_OPTIONS)** exists when the **tcltest** package is loaded (by package require **tcltest**) then its value is taken as a list of arguments to pass to **configure**. This allows the default values of the configuration options to be set by the environment.

customMatch *mode script*

Registers *mode* as a new legal value of the **-match** option to **test**. When the **-match** *mode* option is passed to **test**, the script *script* will be evaluated to compare the actual result of evaluating the body of the test to the expected result. To perform the match, the *script* is completed with two additional words, the expected result, and the actual result, and the completed script is evaluated in the global namespace. The completed script is expected to return a boolean value indicating whether or not the results match. The built-in matching modes of **test** are **exact**, glob, and regexp.

testConstraint *constraint* ?*boolean*?

Sets or returns the boolean value associated with the named *constraint*. See Test constraints below for more information.

interpreter ?*executableName*?

Sets or returns the name of the executable to be execed by **runAllTests** to run each test file when **configure -singleproc** is false. The default value for **interpreter** is the name of the currently running program as returned by info nameofexecutable.

outputChannel ?*channel*?

Sets or returns the output channel. This defaults to **stdout**. Any test that prints test related output should send that output to **outputChannel** rather than letting that output default to **stdout**.

errorChannel ?*channel*?

Sets or returns the error channel. This defaults to **stderr**. Any test that prints error messages should send that output to **errorChannel** rather than printing directly to **stderr**.

Shortcut configuration commands

debug ?*level*?

Same as "**configure -debug** ?*level*?".

errorFile ?*filename*?

Same as "**configure -errfile** ?*filename*?".

limitConstraints ?*boolean*?
Same as "**configure -limitconstraints** ?*boolean*?".

loadFile ?*filename*?
Same as "**configure -loadfile** ?*filename*?".

loadScript ?*script*?
Same as "**configure -load** ?*script*?".

match ?*patternList*?
Same as "**configure -match** ?*patternList*?".

matchDirectories ?*patternList*?
Same as "**configure -relateddir** ?*patternList*?".

matchFiles ?*patternList*?
Same as "**configure -file** ?*patternList*?".

outputFile ?*filename*?
Same as "**configure -outfile** ?*filename*?".

preserveCore ?*level*?
Same as "**configure -preservecore** ?*level*?".

singleProcess ?*boolean*?
Same as "**configure -singleproc** ?*boolean*?".

skip ?*patternList*?
Same as "**configure -skip** ?*patternList*?".

skipDirectories ?*patternList*?
Same as "**configure -asidefromdir** ?*patternList*?".

skipFiles ?*patternList*?
Same as "**configure -notfile** ?*patternList*?".

temporaryDirectory ?*directory*?
Same as "**configure -tmpdir** ?*directory*?".

testsDirectory ?*directory*?
Same as "**configure -testdir** ?*directory*?".

verbose ?*level*?
Same as "**configure -verbose** ?*level*?".

Other commands

The remaining commands provided by **tcltest** have better alternatives provided by **tcltest** or **Tcl** itself. They are retained to support existing test suites, but should be avoided in new code.

test *name description optionList*
This form of **test** was provided to enable passing many options spanning several lines to **test** as a single argument quoted by braces, rather than needing to backslash quote the newlines between arguments to **test**. The *optionList* argument is expected to be a list with an even number of elements representing *option* and *value* arguments to pass to **test**. However, these values are not passed directly, as in the alternate forms of switch. Instead, this form makes an unfortunate attempt to overthrow Tcl's substitution rules by performing substitutions on some of the list elements as an attempt to implement a "do what I mean" interpretation of a brace-enclosed "block". The result is nearly impossible to document clearly, and for that reason this form is not recommended. See the examples in Creating test

suites with tcltest below to see that this form is really not necessary to avoid backslash-quoted newlines. If you insist on using this form, examine the source code of **tcltest** if you want to know the substitution details, or just enclose the third through last argument to **test** in braces and hope for the best.

workingDirectory ?*directoryName*?
Sets or returns the current working directory when the test suite is running. The default value for workingDirectory is the directory in which the test suite was launched. The Tcl commands cd and pwd are sufficient replacements.

normalizeMsg *msg*
Returns the result of removing the "extra" newlines from *msg*, where "extra" is rather imprecise. Tcl offers plenty of string processing commands to modify strings as you wish, and **customMatch** allows flexible matching of actual and expected results.

normalizePath *pathVar*
Resolves symlinks in a path, thus creating a path without internal redirection. It is assumed that *pathVar* is absolute. *pathVar* is modified in place. The Tcl command file normalize is a sufficient replacement.

Tests

The **test** command is the heart of the **tcltest** package. Its essential function is to evaluate a Tcl script and compare the result with an expected result. The options of **test** define the test script, the environment in which to evaluate it, the expected result, and how the compare the actual result to the expected result. Some configuration options of **tcltest** also influence how **test** operates.

The valid options for **test** are summarized:

```
test name description
        ?-constraints keywordList|expression?
        ?-setup setupScript?
        ?-body testScript?
        ?-cleanup cleanupScript?
        ?-result expectedAnswer?
        ?-output expectedOutput?
        ?-errorOutput expectedError?
        ?-returnCodes codeList?
        ?-errorCode expectedErrorCode?
        ?-match mode?
```

The *name* may be any string. It is conventional to choose a *name* according to the pattern:

```
target-majorNum.minorNum
```

For white-box (regression) tests, the target should be the name of the C function or Tcl procedure being tested. For black-box tests, the target should be the name of the feature being tested. Some conventions call for the names of black-box tests to have the suffix **_bb**. Related tests should share a major number. As a test suite evolves, it is best to have the same test name continue to correspond to the same test, so that it remains meaningful to say things like "Test foo-1.3 passed in all releases up to 3.4, but began failing in release 3.5."

During evaluation of **test**, the *name* will be compared to the lists of string matching patterns returned by **configure -match**, and **configure -skip**. The test will be run only if *name* matches any of the patterns from **configure -match** and matches none of the patterns from **configure -skip**.

The *description* should be a short textual description of the test. The *description* is included in output produced by the test, typically test failure messages. Good *description* values should briefly explain the purpose of the test to users of a test suite. The name of a Tcl or C function being tested should be included in the description for regression tests. If the test case exists to reproduce a bug, include the bug ID in the description.

Valid attributes and associated values are:

-constraints *keywordList*|*expression*
: The optional **-constraints** attribute can be list of one or more keywords or an expression. If the **-constraints** value is a list of keywords, each of these keywords should be the name of a constraint defined by a call to **testConstraint**. If any of the listed constraints is false or does not exist, the test is skipped. If the **-constraints** value is an expression, that expression is evaluated. If the expression evaluates to true, then the test is run.

  Note that the expression form of **-constraints** may interfere with the operation of **configure -constraints** and **configure -limitconstraints**, and is not recommended.

  Appropriate constraints should be added to any tests that should not always be run. That is, conditional evaluation of a test should be accomplished by the **-constraints** option, not by conditional evaluation of **test**. In that way, the same number of tests are always reported by the test suite, though the number skipped may change based on the testing environment. The default value is an empty list. See Test constraints below for a list of built-in constraints and information on how to add your own constraints.

-setup *script*
: The optional **-setup** attribute indicates a *script* that will be run before the script indicated by the **-body** attribute. If evaluation of *script* raises an error, the test will fail. The default value is an empty script.

-body *script*
: The **-body** attribute indicates the *script* to run to carry out the test, which must return a result that can be checked for correctness. If evaluation of *script* raises an error, the test will fail (unless the **-returnCodes** option is used to state that an error is expected). The default value is an empty script.

-cleanup *script*
: The optional **-cleanup** attribute indicates a *script* that will be run after the script indicated by the **-body** attribute. If evaluation of *script* raises an error, the test will fail. The default value is an empty script.

-match *mode*
: The **-match** attribute determines how expected answers supplied by **-result**, **-output**, and **-errorOutput** are compared. Valid values for *mode* are regexp, glob, **exact**, and any value registered by a prior call to **customMatch**. The default value is **exact**.

-result *expectedValue*
: The **-result** attribute supplies the *expectedValue* against which the return value from script

will be compared. The default value is an empty string.

-output *expectedValue*
The **-output** attribute supplies the *expectedValue* against which any output sent to **stdout** or **outputChannel** during evaluation of the script(s) will be compared. Note that only output printed using the global puts command is used for comparison. If **-output** is not specified, output sent to **stdout** and **outputChannel** is not processed for comparison.

-errorOutput *expectedValue*
The **-errorOutput** attribute supplies the *expectedValue* against which any output sent to **stderr** or **errorChannel** during evaluation of the script(s) will be compared. Note that only output printed using the global puts command is used for comparison. If **-errorOutput** is not specified, output sent to **stderr** and **errorChannel** is not processed for comparison.

-returnCodes *expectedCodeList*
The optional **-returnCodes** attribute supplies *expectedCodeList*, a list of return codes that may be accepted from evaluation of the **-body** script. If evaluation of the **-body** script returns a code not in the *expectedCodeList*, the test fails. All return codes known to return, in both numeric and symbolic form, including extended return codes, are acceptable elements in the *expectedCodeList*. Default value is "**ok return**".

-errorCode *expectedErrorCode*
The optional **-errorCode** attribute supplies *expectedErrorCode*, a glob pattern that should match the error code reported from evaluation of the **-body** script. If evaluation of the **-body** script returns a code not matching *expectedErrorCode*, the test fails. Default value is "*". If **-returnCodes** does not include **error** it is set to **error**.

To pass, a test must successfully evaluate its **-setup**, **-body**, and **-cleanup** scripts. The return code of the **-body** script and its result must match expected values, and if specified, output and error data from the test must match expected **-output** and **-errorOutput** values. If any of these conditions are not met, then the test fails. Note that all scripts are evaluated in the context of the caller of **test**.

As long as **test** is called with valid syntax and legal values for all attributes, it will not raise an error. Test failures are instead reported as output written to **outputChannel**. In default operation, a successful test produces no output. The output messages produced by **test** are controlled by the **configure -verbose** option as described in Configurable options below. Any output produced by the test scripts themselves should be produced using puts to **outputChannel** or **errorChannel**, so that users of the test suite may easily capture output with the **configure -outfile** and **configure -errfile** options, and so that the **-output** and **-errorOutput** attributes work properly.

Test constraints

Constraints are used to determine whether or not a test should be skipped. Each constraint has a name, which may be any string, and a boolean value. Each **test** has a **-constraints** value which is a list of constraint names. There are two modes of constraint control. Most frequently, the default mode is used, indicated by a setting of **configure -limitconstraints** to false. The test will run only if all constraints in the list are true-valued. Thus, the **-constraints** option of **test** is a convenient, symbolic way to define any conditions required for the test to be possible or meaningful. For example, a **test** with **-constraints unix** will only be run if the constraint **unix** is true, which indicates the test suite is being run on a Unix platform.

Each **test** should include whatever **-constraints** are required to constrain it to run only where appropriate. Several constraints are predefined in the **tcltest** package, listed below. The registration of user-defined constraints is performed by the **testConstraint** command. User-defined constraints may appear within a test file, or within the script specified by the **configure -load** or **configure -loadfile** options.

The following is a list of constraints predefined by the **tcltest** package itself:

singleTestInterp
This test can only be run if all test files are sourced into a single interpreter.

unix
This test can only be run on any Unix platform.

win
This test can only be run on any Windows platform.

nt
This test can only be run on any Windows NT platform.

mac
This test can only be run on any Mac platform.

unixOrWin
This test can only be run on a Unix or Windows platform.

macOrWin
This test can only be run on a Mac or Windows platform.

macOrUnix
This test can only be run on a Mac or Unix platform.

tempNotWin
This test can not be run on Windows. This flag is used to temporarily disable a test.

tempNotMac
This test can not be run on a Mac. This flag is used to temporarily disable a test.

unixCrash
This test crashes if it is run on Unix. This flag is used to temporarily disable a test.

winCrash
This test crashes if it is run on Windows. This flag is used to temporarily disable a test.

macCrash
This test crashes if it is run on a Mac. This flag is used to temporarily disable a test.

emptyTest
This test is empty, and so not worth running, but it remains as a place-holder for a test to be written in the future. This constraint has value false to cause tests to be skipped unless the user specifies otherwise.

knownBug
This test is known to fail and the bug is not yet fixed. This constraint has value false to cause tests to be skipped unless the user specifies otherwise.

nonPortable
This test can only be run in some known development environment. Some tests are inherently non-portable because they depend on things like word length, file system configuration, window manager, etc. This constraint has value false to cause tests to be skipped unless the user specifies otherwise.

userInteraction
This test requires interaction from the user. This constraint has value false to causes tests to be skipped unless the user specifies otherwise.

interactive
This test can only be run in if the interpreter is in interactive mode (when the global **::tcl_interactive** variable is set to 1).

nonBlockFiles
This test can only be run if platform supports setting files into nonblocking mode.

asyncPipeClose
This test can only be run if platform supports async flush and async close on a pipe.

unixExecs
This test can only be run if this machine has Unix-style commands **cat**, **echo**, **sh**, **wc**, **rm**, **sleep**, **fgrep**, **ps**, **chmod**, and **mkdir** available.

hasIsoLocale
This test can only be run if can switch to an ISO locale.

root
This test can only run if Unix user is root.

notRoot
This test can only run if Unix user is not root.

eformat
This test can only run if app has a working version of sprintf with respect to the "e" format of floating-point numbers.

stdio
This test can only be run if **interpreter** can be opened as a pipe.

The alternative mode of constraint control is enabled by setting **configure -limitconstraints** to true. With that configuration setting, all existing constraints other than those in the constraint list returned by **configure -constraints** are set to false. When the value of **configure -constraints** is set, all those constraints are set to true. The effect is that when both options **configure -constraints** and **configure -limitconstraints** are in use, only those tests including only constraints from the **configure -constraints** list are run; all others are skipped. For example, one might set up a configuration with

```
configure -constraints knownBug \
          -limitconstraints true \
          -verbose pass
```

to run exactly those tests that exercise known bugs, and discover whether any of them pass, indicating the bug had been fixed.

Running all tests

The single command **runAllTests** is evaluated to run an entire test suite, spanning many files and directories. The configuration options of **tcltest** control the precise operations. The **runAllTests** command begins by printing a summary of its configuration to **outputChannel**.

Test files to be evaluated are sought in the directory **configure -testdir**. The list of files in that directory that match any of the patterns in **configure -file** and match none of the patterns in **configure -notfile** is generated and sorted. Then each file will be evaluated in turn. If

configure -singleproc is true, then each file will be sourced in the caller's context. If it is false, then a copy of **interpreter** will be exec'd to evaluate each file. The multi-process operation is useful when testing can cause errors so severe that a process terminates. Although such an error may terminate a child process evaluating one file, the main process can continue with the rest of the test suite. In multi-process operation, the configuration of **tcltest** in the main process is passed to the child processes as command line arguments, with the exception of **configure -outfile**. The **runAllTests** command in the main process collects all output from the child processes and collates their results into one main report. Any reports of individual test failures, or messages requested by a **configure -verbose** setting are passed directly on to **outputChannel** by the main process.

After evaluating all selected test files, a summary of the results is printed to **outputChannel**. The summary includes the total number of **test**s evaluated, broken down into those skipped, those passed, and those failed. The summary also notes the number of files evaluated, and the names of any files with failing tests or errors. A list of the constraints that caused tests to be skipped, and the number of tests skipped for each is also printed. Also, messages are printed if it appears that evaluation of a test file has caused any temporary files to be left behind in **configure -tmpdir**.

Having completed and summarized all selected test files, **runAllTests** then recursively acts on subdirectories of **configure -testdir**. All subdirectories that match any of the patterns in **configure -relateddir** and do not match any of the patterns in **configure -asidefromdir** are examined. If a file named **all.tcl** is found in such a directory, it will be sourced in the caller's context. Whether or not an examined directory contains an **all.tcl** file, its subdirectories are also scanned against the **configure -relateddir** and **configure -asidefromdir** patterns. In this way, many directories in a directory tree can have all their test files evaluated by a single **runAllTests** command.

Configurable options

The **configure** command is used to set and query the configurable options of **tcltest**. The valid options are:

-singleproc *boolean*
: Controls whether or not **runAllTests** spawns a child process for each test file. No spawning when *boolean* is true. Default value is false.

-debug *level*
: Sets the debug level to *level*, an integer value indicating how much debugging information should be printed to **stdout**. Note that debug messages always go to **stdout**, independent of the value of **configure -outfile**. Default value is 0. Levels are defined as:

  0
  : Do not display any debug information.

  1
  : Display information regarding whether a test is skipped because it does not match any of the tests that were specified using by **configure -match** (userSpecifiedNonMatch) or matches any of the tests specified by **configure -skip** (userSpecifiedSkip). Also print warnings about possible lack of cleanup or balance in test files. Also print warnings about any re-use of test names.

  2

Display the flag array parsed by the command line processor, the contents of the global env array, and all user-defined variables that exist in the current namespace as they are used.

3

Display information regarding what individual procs in the test harness are doing.

-verbose *level*

Sets the type of output verbosity desired to *level*, a list of zero or more of the elements **body**, **pass**, **skip**, **start**, **error**, **line**, **msec** and **usec**. Default value is "**body error**". Levels are defined as:

body (b)

Display the body of failed tests

pass (p)

Print output when a test passes

skip (s)

Print output when a test is skipped

start (t)

Print output whenever a test starts

error (e)

Print errorInfo and errorCode, if they exist, when a test return code does not match its expected return code

line (l)

Print source file line information of failed tests

msec (m)

Print each test's execution time in milliseconds

usec (u)

Print each test's execution time in microseconds

Note that the **msec** and **usec** verbosity levels are provided as indicative measures only. They do not tackle the problem of repeatability which should be considered in performance tests or benchmarks. To use these verbosity levels to thoroughly track performance degradations, consider wrapping your test bodies with time commands.

The single letter abbreviations noted above are also recognized so that "**configure -verbose pt**" is the same as "**configure -verbose {pass start}**".

-preservecore *level*

Sets the core preservation level to *level*. This level determines how stringent checks for core files are. Default value is 0. Levels are defined as:

0

No checking — do not check for core files at the end of each test command, but do check for them in **runAllTests** after all test files have been evaluated.

1

Also check for core files at the end of each **test** command.

2

Check for core files at all times described above, and save a copy of each core file produced in **configure -tmpdir**.

-limitconstraints *boolean*
Sets the mode by which **test** honors constraints as described in Tests above. Default value is false.

-constraints *list*
Sets all the constraints in *list* to true. Also used in combination with **configure -limitconstraints true** to control an alternative constraint mode as described in Tests above. Default value is an empty list.

-tmpdir *directory*
Sets the temporary directory to be used by **makeFile**, **makeDirectory**, **viewFile**, **removeFile**, and **removeDirectory** as the default directory where temporary files and directories created by test files should be created. Default value is **workingDirectory**.

-testdir *directory*
Sets the directory searched by **runAllTests** for test files and subdirectories. Default value is **workingDirectory**.

-file *patternList*
Sets the list of patterns used by **runAllTests** to determine what test files to evaluate. Default value is "***.test**".

-notfile *patternList*
Sets the list of patterns used by **runAllTests** to determine what test files to skip. Default value is "**l.*.test**", so that any SCCS lock files are skipped.

-relateddir *patternList*
Sets the list of patterns used by **runAllTests** to determine what subdirectories to search for an **all.tcl** file. Default value is "*".

-asidefromdir *patternList*
Sets the list of patterns used by **runAllTests** to determine what subdirectories to skip when searching for an **all.tcl** file. Default value is an empty list.

-match *patternList*
Set the list of patterns used by **test** to determine whether a test should be run. Default value is "*".

-skip *patternList*
Set the list of patterns used by **test** to determine whether a test should be skipped. Default value is an empty list.

-load *script*
Sets a script to be evaluated by **loadTestedCommands**. Default value is an empty script.

-loadfile *filename*
Sets the filename from which to read a script to be evaluated by **loadTestedCommands**. This is an alternative to **-load**. They cannot be used together.

-outfile *filename*
Sets the file to which all output produced by tcltest should be written. A file named *filename* will be opened for writing, and the resulting channel will be set as the value of **outputChannel**.

-errfile *filename*
Sets the file to which all error output produced by tcltest should be written. A file named *filename* will be opened for writing, and the resulting channel will be set as the value of

errorChannel.

Creating test suites with tcltest

The fundamental element of a test suite is the individual **test** command. We begin with several examples.

1. Test of a script that returns normally.

```
test example-1.0 {normal return} {
    format %s value
} value
```

2. Test of a script that requires context setup and cleanup. Note the bracing and indenting style that avoids any need for line continuation.

```
test example-1.1 {test file existence} -setup {
    set file [makeFile {} test]
} -body {
    file exists $file
} -cleanup {
    removeFile test
} -result 1
```

3. Test of a script that raises an error.

```
test example-1.2 {error return} -body {
    error message
} -returnCodes error -result message
```

4. Test with a constraint.

```
test example-1.3 {user owns created files} -constraints {
    unix
} -setup {
    set file [makeFile {} test]
} -body {
    file attributes $file -owner
} -cleanup {
    removeFile test
} -result $::tcl_platform(user)
```

At the next higher layer of organization, several **test** commands are gathered together into a single test file. Test files should have names with the "**.test**" extension, because that is the default pattern used by **runAllTests** to find test files. It is a good rule of thumb to have one test file for each source code file of your project. It is good practice to edit the test file and the source code file together, keeping tests synchronized with code changes.

Most of the code in the test file should be the **test** commands. Use constraints to skip tests, rather than conditional evaluation of **test**.

1. Recommended system for writing conditional tests, using constraints to guard:

```
testConstraint X [expr $myRequirement]
test goodConditionalTest {} X {
    # body
} result
```

2. Discouraged system for writing conditional tests, using if to guard:

```
if $myRequirement {
    test badConditionalTest {} {
        #body
    } result
}
```

Use the **-setup** and **-cleanup** options to establish and release all context requirements of the test body. Do not make tests depend on prior tests in the file. Those prior tests might be skipped. If several consecutive tests require the same context, the appropriate setup and cleanup scripts may be stored in variable for passing to each tests **-setup** and **-cleanup** options. This is a better solution than performing setup outside of **test** commands, because the setup will only be done if necessary, and any errors during setup will be reported, and not cause the test file to abort.

A test file should be able to be combined with other test files and not interfere with them, even when **configure -singleproc 1** causes all files to be evaluated in a common interpreter. A simple way to achieve this is to have your tests define all their commands and variables in a namespace that is deleted when the test file evaluation is complete. A good namespace to use is a child namespace **test** of the namespace of the module you are testing.

A test file should also be able to be evaluated directly as a script, not depending on being called by a main **runAllTests**. This means that each test file should process command line arguments to give the tester all the configuration control that **tcltest** provides.

After all **test**s in a test file, the command **cleanupTests** should be called.

1. Here is a sketch of a sample test file illustrating those points:

```
package require tcltest 2.5
::tcltest::configure {*}$::argv
package require example
namespace eval ::example::test {
    namespace import ::tcltest::*
    testConstraint X [expr {...}]
    variable SETUP {#common setup code}
    variable CLEANUP {#common cleanup code}
    test example-1 {} -setup $SETUP -body {
        # First test
    } -cleanup $CLEANUP -result {...}
    test example-2 {} -constraints X -setup $SETUP -body {
        # Second test; constrained
    } -cleanup $CLEANUP -result {...}
    test example-3 {} {
        # Third test; no context required
```

```
    } {...}
    cleanupTests
}
namespace delete ::example::test
```

The next level of organization is a full test suite, made up of several test files. One script is used to control the entire suite. The basic function of this script is to call **runAllTests** after doing any necessary setup. This script is usually named **all.tcl** because that is the default name used by **runAllTests** when combining multiple test suites into one testing run.

1. Here is a sketch of a sample test suite main script:

```
package require tcltest 2.5
package require example
::tcltest::configure -testdir \
        [file dirname [file normalize [info script]]]
::tcltest::configure {*}$::argv
::tcltest::runAllTests
```

Compatibility

A number of commands and variables in the **::tcltest** namespace provided by earlier releases of **tcltest** have not been documented here. They are no longer part of the supported public interface of **tcltest** and should not be used in new test suites. However, to continue to support existing test suites written to the older interface specifications, many of those deprecated commands and variables still work as before. For example, in many circumstances, **configure** will be automatically called shortly after package require **tcltest 2.1** succeeds with arguments from the variable **::argv**. This is to support test suites that depend on the old behavior that **tcltest** was automatically configured from command line arguments. New test files should not depend on this, but should explicitly include

```
::tcltest::configure {*}$::argv
```

to establish a configuration from command line arguments.

Known issues

There are two known issues related to nested evaluations of **test**. The first issue relates to the stack level in which test scripts are executed. Tests nested within other tests may be executed at the same stack level as the outermost test. For example, in the following code:

```
test level-1.1 {level 1} {
    -body {
        test level-2.1 {level 2} {
        }
    }
}
```

any script executed in level-2.1 may be executed at the same stack level as the script defined for level-1.1.

In addition, while two **test**s have been run, results will only be reported by **cleanupTests** for

tests at the same level as test level-1.1. However, test results for all tests run prior to level-1.1 will be available when test level-2.1 runs. What this means is that if you try to access the test results for test level-2.1, it will may say that "m" tests have run, "n" tests have been skipped, "o" tests have passed and "p" tests have failed, where "m", "n", "o", and "p" refer to tests that were run at the same test level as test level-1.1.

Implementation of output and error comparison in the test command depends on usage of puts in your application code. Output is intercepted by redefining the global puts command while the defined test script is being run. Errors thrown by C procedures or printed directly from C applications will not be caught by the **test** command. Therefore, usage of the **-output** and **-errorOutput** options to **test** is useful only for pure Tcl applications that use puts to produce output.

Copyright (c) 1990-1994 The Regents of the University of California
Copyright (c) 1994-1997 Sun Microsystems, Inc.
Copyright (c) 1998-1999 Scriptics Corporation
Copyright (c) 2000 Ajuba Solutions

tclvars

Keywords: tclvars, arithmetic, bytecode, compiler, error, environment, POSIX, precision, subprocess, user, variables

Name

argc, argv, argv0, auto_path, env, errorCode, errorInfo, tcl_interactive, tcl_library, tcl_patchLevel, tcl_pkgPath, tcl_platform, tcl_precision, tcl_rcFileName, tcl_traceCompile, tcl_traceExec, tcl_version - Variables used by Tcl

Description

The following global variables are created and managed automatically by the Tcl library. Except where noted below, these variables should normally be treated as read-only by application-specific code and by users.

auto_path
: If set, then it must contain a valid Tcl list giving directories to search during auto-load operations (including for package index files when using the default package unknown handler). This variable is initialized during startup to contain, in order: the directories listed in the **TCLLIBPATH** environment variable, the directory named by the **tcl_library** global variable, the parent directory of **tcl_library**, **[file dirname [file dirname [info nameofexecutable]]]/lib**, the directories listed in the **tcl_pkgPath** variable. Additional locations to look for files and package indices should normally be added to this variable using lappend.

  Additional variables relating to package management exist. More details are listed in the **VARIABLES** section of the library manual page.

env
: This variable is maintained by Tcl as an array whose elements are the environment variables for the process. Reading an element will return the value of the corresponding environment variable. Setting an element of the array will modify the corresponding environment variable

or create a new one if it does not already exist. Unsetting an element of **env** will remove the corresponding environment variable. Changes to the **env** array will affect the environment passed to children by commands like exec. If the entire **env** array is unset then Tcl will stop monitoring **env** accesses and will not update environment variables.

Under Windows, the environment variables PATH and COMSPEC in any capitalization are converted automatically to upper case. For instance, the PATH variable could be exported by the operating system as "path", "Path", "PaTh", etc., causing otherwise simple Tcl code to have to support many special cases. All other environment variables inherited by Tcl are left unmodified. Setting an env array variable to blank is the same as unsetting it as this is the behavior of the underlying Windows OS. It should be noted that relying on an existing and empty environment variable will not work on Windows and is discouraged for cross-platform usage.

The following elements of **env** are special to Tcl:

env(HOME)
: This environment variable, if set, gives the location of the directory considered to be the current user's home directory, and to which a call of cd without arguments or with just "~" as an argument will change into. Most platforms set this correctly by default; it does not normally need to be set by user code.

env(TCL_LIBRARY)
: If set, then it specifies the location of the directory containing library scripts (the value of this variable will be assigned to the **tcl_library** variable and therefore returned by the command info library). If this variable is not set then a default value is used.

  Note that this environment variable should *not* normally be set.

env(TCLLIBPATH)
: If set, then it must contain a valid Tcl list giving directories to search during auto-load operations. Directories must be specified in Tcl format, using "/" as the path separator, regardless of platform. This variable is only used when initializing the **auto_path** variable.

env(TCL_TZ), env(TZ)
: These specify the default timezone used for parsing and formatting times and dates in the clock command. On many platforms, the TZ environment variable is set up by the operating system.

env(LC_ALL), env(LC_MESSAGES), env(LANG)
: These environment variables are used by the msgcat package to determine what locale to format messages using.

env(TCL_INTERP_DEBUG_FRAME)
: If existing, it has the same effect as running interp debug **{} -frame 1** as the very first command of each new Tcl interpreter.

errorCode
: This variable holds the value of the **-errorcode** return option set by the most recent error that occurred in this interpreter. This list value represents additional information about the error in a form that is easy to process with programs. The first element of the list identifies a general class of errors, and determines the format of the rest of the list. The following formats for **-errorcode** return options are used by the Tcl core; individual applications may define

additional formats.

ARITH *code msg*
This format is used when an arithmetic error occurs (e.g. an attempt to divide zero by zero in the expr command). *Code* identifies the precise error and *msg* provides a human-readable description of the error. *Code* will be either DIVZERO (for an attempt to divide by zero), DOMAIN (if an argument is outside the domain of a function, such as acos(-3)), IOVERFLOW (for integer overflow), OVERFLOW (for a floating-point overflow), or UNKNOWN (if the cause of the error cannot be determined).

Detection of these errors depends in part on the underlying hardware and system libraries.

CHILDKILLED *pid sigName msg*
This format is used when a child process has been killed because of a signal. The *pid* element will be the process's identifier (in decimal). The *sigName* element will be the symbolic name of the signal that caused the process to terminate; it will be one of the names from the include file signal.h, such as **SIGPIPE**. The *msg* element will be a short human-readable message describing the signal, such as "write on pipe with no readers" for **SIGPIPE**.

CHILDSTATUS *pid code*
This format is used when a child process has exited with a non-zero exit status. The *pid* element will be the process's identifier (in decimal) and the *code* element will be the exit code returned by the process (also in decimal).

CHILDSUSP *pid sigName msg*
This format is used when a child process has been suspended because of a signal. The *pid* element will be the process's identifier, in decimal. The *sigName* element will be the symbolic name of the signal that caused the process to suspend; this will be one of the names from the include file signal.h, such as **SIGTTIN**. The *msg* element will be a short human-readable message describing the signal, such as "background tty read" for **SIGTTIN**.

NONE
This format is used for errors where no additional information is available for an error besides the message returned with the error. In these cases the **-errorcode** return option will consist of a list containing a single element whose contents are **NONE**.

POSIX *errName msg*
If the first element is **POSIX**, then the error occurred during a POSIX kernel call. The *errName* element will contain the symbolic name of the error that occurred, such as **ENOENT**; this will be one of the values defined in the include file errno.h. The *msg* element will be a human-readable message corresponding to *errName*, such as "no such file or directory" for the **ENOENT** case.

TCL ...
Indicates some sort of problem generated in relation to Tcl itself, e.g. a failure to look up a channel or variable.

To set the **-errorcode** return option, applications should use library procedures such as Tcl_SetObjErrorCode, Tcl_SetReturnOptions, and Tcl_PosixError, or they may invoke the **-errorcode** option of the return command. If none of these methods for setting the error code has been used, the Tcl interpreter will reset the variable to **NONE** after the next error.

errorInfo
This variable holds the value of the **-errorinfo** return option set by the most recent error that occurred in this interpreter. This string value will contain one or more lines identifying the Tcl commands and procedures that were being executed when the most recent error occurred. Its contents take the form of a stack trace showing the various nested Tcl commands that had been invoked at the time of the error.

tcl_library
This variable holds the name of a directory containing the system library of Tcl scripts, such as those used for auto-loading. The value of this variable is returned by the info library command. See the library manual entry for details of the facilities provided by the Tcl script library. Normally each application or package will have its own application-specific script library in addition to the Tcl script library; each application should set a global variable with a name like **$***app***_library** (where *app* is the application's name) to hold the network file name for that application's library directory. The initial value of **tcl_library** is set when an interpreter is created by searching several different directories until one is found that contains an appropriate Tcl startup script. If the **TCL_LIBRARY** environment variable exists, then the directory it names is checked first. If **TCL_LIBRARY** is not set or doesn't refer to an appropriate directory, then Tcl checks several other directories based on a compiled-in default location, the location of the binary containing the application, and the current working directory.

tcl_patchLevel
When an interpreter is created Tcl initializes this variable to hold a string giving the current patch level for Tcl, such as **8.4.16** for Tcl 8.4 with the first sixteen official patches, or **8.5b3** for the third beta release of Tcl 8.5. The value of this variable is returned by the info patchlevel command.

tcl_pkgPath
This variable holds a list of directories indicating where packages are normally installed. It is not used on Windows. It typically contains either one or two entries; if it contains two entries, the first is normally a directory for platform-dependent packages (e.g., shared library binaries) and the second is normally a directory for platform-independent packages (e.g., script files). Typically a package is installed as a subdirectory of one of the entries in the **tcl_pkgPath** variable. The directories in the **tcl_pkgPath** variable are included by default in the **auto_path** variable, so they and their immediate subdirectories are automatically searched for packages during package require commands. Note: **tcl_pkgPath** is not intended to be modified by the application. Its value is added to **auto_path** at startup; changes to **tcl_pkgPath** are not reflected in **auto_path**. If you want Tcl to search additional directories for packages you should add the names of those directories to **auto_path**, not **tcl_pkgPath**.

tcl_platform
This is an associative array whose elements contain information about the platform on which the application is running, such as the name of the operating system, its current release number, and the machine's instruction set. The elements listed below will always be defined, but they may have empty strings as values if Tcl could not retrieve any relevant information. In addition, extensions and applications may add additional values to the array. The predefined elements are:

byteOrder
The native byte order of this machine: either **littleEndian** or **bigEndian**.

debug
If this variable exists, then the interpreter was compiled with and linked to a debug-enabled C run-time. This variable will only exist on Windows, so extension writers can specify which package to load depending on the C run-time library that is in use. This is not an indication that this core contains symbols.

engine
The name of the Tcl language implementation. When the interpreter is first created, this is always set to the string **Tcl**.

machine
The instruction set executed by this machine, such as **intel**, **PPC**, **68k**, or **sun4m**. On UNIX machines, this is the value returned by **uname -m**.

os
The name of the operating system running on this machine, such as **Windows NT** or **SunOS**. On UNIX machines, this is the value returned by **uname -s**.

osVersion
The version number for the operating system running on this machine. On UNIX machines, this is the value returned by **uname -r**.

pathSeparator
The character that should be used to split PATH-like environment variables into their corresponding list of directory names.

platform
Either **windows**, or **unix**. This identifies the general operating environment of the machine.

pointerSize
This gives the size of the native-machine pointer in bytes (strictly, it is same as the result of evaluating *sizeof(void*)* in C.)

threaded
If this variable exists, then the interpreter was compiled with threads enabled.

user
This identifies the current user based on the login information available on the platform. This value comes from the getuid() and getpwuid() system calls on Unix, and the value from the GetUserName() system call on Windows.

wordSize
This gives the size of the native-machine word in bytes (strictly, it is same as the result of evaluating *sizeof(long)* in C.)

tcl_precision
This variable controls the number of digits to generate when converting floating-point values to strings. It defaults to 0. *Applications should not change this value;* it is provided for compatibility with legacy code.

The default value of 0 is special, meaning that Tcl should convert numbers using as few digits as possible while still distinguishing any floating point number from its nearest neighbours. It differs from using an arbitrarily high value for *tcl_precision* in that an inexact number like *1.4* will convert as *1.4* rather than *1.3999999999999999* even though the latter is nearer to the

exact value of the binary number.

If **tcl_precision** is not zero, then when Tcl converts a floating point number, it creates a decimal representation of at most **tcl_precision** significant digits; the result may be shorter if the shorter result represents the original number exactly. If no result of at most **tcl_precision** digits is an exact representation of the original number, the one that is closest to the original number is chosen. If the original number lies precisely between two equally accurate decimal representations, then the one with an even value for the least significant digit is chosen; for instance, if **tcl_precision** is 3, then 0.3125 will convert to 0.312, not 0.313, while 0.6875 will convert to 0.688, not 0.687. Any string of trailing zeroes that remains is trimmed.

a **tcl_precision** value of 17 digits is "perfect" for IEEE floating-point in that it allows double-precision values to be converted to strings and back to binary with no loss of information. For this reason, you will often see it as a value in legacy code that must run on Tcl versions before 8.5. It is no longer recommended; as noted above, a zero value is the preferred method.

All interpreters in a thread share a single **tcl_precision** value: changing it in one interpreter will affect all other interpreters as well. Safe interpreters are not allowed to modify the variable.

Valid values for **tcl_precision** range from 0 to 17.

tcl_rcFileName
This variable is used during initialization to indicate the name of a user-specific startup file. If it is set by application-specific initialization, then the Tcl startup code will check for the existence of this file and source it if it exists. For example, for wish the variable is set to **~/.wishrc** for Unix and **~/wishrc.tcl** for Windows.

tcl_traceCompile
The value of this variable can be set to control how much tracing information is displayed during bytecode compilation. By default, **tcl_traceCompile** is zero and no information is displayed. Setting **tcl_traceCompile** to 1 generates a one-line summary in **stdout** whenever a procedure or top-level command is compiled. Setting it to 2 generates a detailed listing in **stdout** of the bytecode instructions emitted during every compilation. This variable is useful in tracking down suspected problems with the Tcl compiler.

This variable and functionality only exist if **TCL_COMPILE_DEBUG** was defined during Tcl's compilation.

tcl_traceExec
The value of this variable can be set to control how much tracing information is displayed during bytecode execution. By default, **tcl_traceExec** is zero and no information is displayed. Setting **tcl_traceExec** to 1 generates a one-line trace in **stdout** on each call to a Tcl procedure. Setting it to 2 generates a line of output whenever any Tcl command is invoked that contains the name of the command and its arguments. Setting it to 3 produces a detailed trace showing the result of executing each bytecode instruction. Note that when **tcl_traceExec** is 2 or 3, commands such as set and incr that have been entirely replaced by a sequence of bytecode instructions are not shown. Setting this variable is useful in tracking

down suspected problems with the bytecode compiler and interpreter.

This variable and functionality only exist if **TCL_COMPILE_DEBUG** was defined during Tcl's compilation.

tcl_version
When an interpreter is created Tcl initializes this variable to hold the version number for this version of Tcl in the form *x.y*. Changes to *x* represent major changes with probable incompatibilities and changes to *y* represent small enhancements and bug fixes that retain backward compatibility. The value of this variable is returned by the info tclversion command.

Other global variables

The following variables are only guaranteed to exist in tclsh and wish executables; the Tcl library does not define them itself but many Tcl environments do.

argc
The number of arguments to tclsh or wish.

argv
Tcl list of arguments to tclsh or wish.

argv0
The script that tclsh or wish started executing (if it was specified) or otherwise the name by which tclsh or wish was invoked.

tcl_interactive
Contains 1 if tclsh or wish is running interactively (no script was specified and standard input is a terminal-like device), 0 otherwise.

Examples

To add a directory to the collection of locations searched by package require, e.g., because of some application-specific packages that are used, the **auto_path** variable needs to be updated:

```
lappend ::auto_path [file join [pwd] "theLibDir"]
```

A simple though not very robust way to handle command line arguments of the form "-foo 1 -bar 2" is to load them into an array having first loaded in the default settings:

```
array set arguments {-foo 0 -bar 0 -grill 0}
array set arguments $::argv
puts "foo is $arguments(-foo)"
puts "bar is $arguments(-bar)"
puts "grill is $arguments(-grill)"
```

The **argv0** global variable can be used (in conjunction with the info script command) to determine whether the current script is being executed as the main script or loaded as a library. This is useful because it allows a single script to be used as both a library and a demonstration of that library:

```
if {$::argv0 eq [info script]} {
    # running as: tclsh example.tcl
} else {
    package provide Example 1.0
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

tell

Keywords: tell, access position, channel, seeking

Name

tell - Return current access position for an open channel

Synopsis

tell *channelId*

Description

Returns an integer string giving the current access position in *channelId*. This value returned is a byte offset that can be passed to seek in order to set the channel to a particular position. Note that this value is in terms of bytes, not characters like read. The value returned is -1 for channels that do not support seeking.

ChannelId must be an identifier for an open channel such as a Tcl standard channel (**stdin**, **stdout**, or **stderr**), the return value from an invocation of open or socket, or the result of a channel creation command provided by a Tcl extension.

Example

Read a line from a file channel only if it starts with **foobar**:

```
# Save the offset in case we need to undo the read...
set offset [tell $chan]
if {[read $chan 6] eq "foobar"} {
    gets $chan line
} else {
    set line {}
    # Undo the read...
    seek $chan $offset
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

throw

Keywords: throw, error, exception

Name

throw - Generate a machine-readable error

Synopsis
throw *type message*

Description
This command causes the current evaluation to be unwound with an error. The error created is described by the *type* and *message* arguments: *type* must contain a list of words describing the error in a form that is machine-readable (and which will form the error-code part of the result dictionary), and *message* should contain text that is intended for display to a human being.

The stack will be unwound until the error is trapped by a suitable catch or try command. If it reaches the event loop without being trapped, it will be reported through the bgerror mechanism. If it reaches the top level of script evaluation in tclsh, it will be printed on the console before, in the non-interactive case, causing an exit (the behavior in other programs will depend on the details of how Tcl is embedded and used).

By convention, the words in the *type* argument should go from most general to most specific.

Examples
The following produces an error that is identical to that produced by expr when trying to divide a value by zero.

```
throw {ARITH DIVZERO {divide by zero}} {divide by zero}
```

Copyright (c) 2008 Donal K. Fellows

time
Keywords: time, script

Name
time - Time the execution of a script

Synopsis
time *script* ?*count*?

Description
This command will call the Tcl interpreter *count* times to evaluate *script* (or once if *count* is not specified). It will then return a string of the form

```
503.2 microseconds per iteration
```

which indicates the average amount of time required per iteration, in microseconds. Time is measured in elapsed time, not CPU time.

Example
Estimate how long it takes for a simple Tcl for loop to count to a thousand:

```
time {
    for {set i 0} {$i<1000} {incr i} {
        # empty body
    }
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

timerate

Keywords: timerate, performance measurement, script, time

Name

timerate - Calibrated performance measurements of script execution time

Synopsis

timerate *script* ?*time*? ?*max-count*?
timerate ?**-direct**? ?*-overhead double*? *script* ?*time*? ?*max-count*?
timerate ?**-calibrate**? ?**-direct**? *script* ?*time*? ?*max-count*?

Description

The **timerate** command does calibrated performance measurement of a Tcl command or script, *script*. The *script* should be written so that it can be executed multiple times during the performance measurement process. Time is measured in elapsed time using the finest timer resolution as possible, not CPU time; if *script* interacts with the OS, the cost of that interaction is included. This command may be used to provide information as to how well a script or Tcl command is performing, and can help determine bottlenecks and fine-tune application performance.

The first and second form will evaluate *script* until the interval *time* given in milliseconds elapses, or for 1000 milliseconds (1 second) if *time* is not specified. The parameter *max-count* could additionally impose a further restriction by the maximal number of iterations to evaluate the script. If *max-count* is specified, the evalution will stop either this count of iterations is reached or the time is exceeded. It will then return a canonical tcl-list of the form:

```
0.095977 µs/# 52095836 # 10419167 #/sec 5000.000 net-ms
```

which indicates:

- the average amount of time required per iteration, in microseconds ([**lindex** $result 0])
- the count how many times it was executed ([**lindex** $result 2])
- the estimated rate per second ([**lindex** $result 4])
- the estimated real execution time without measurement overhead ([**lindex** $result 6])

The following options may be supplied to the **timerate** command:

-calibrate
: To measure very fast scripts as exactly as possible, a calibration process may be required. The **-calibrate** option is used to calibrate **timerate** itself, calculating the estimated overhead of the given script as the default overhead for future invocations of the **timerate** command. If the *time* parameter is not specified, the calibrate procedure runs for up to 10 seconds.

Note that calibration is not thread safe in the current implementation.

-overhead *double*
The **-overhead** parameter supplies an estimate (in microseconds) of the measurement overhead of each iteration of the tested script. The passed value overrides, for the current invocation of **timerate**, the overhead estimated by a previous calibration. Overrides may themselves be measured using **timerate** as illustrated by a later example.

-direct
The **-direct** option causes direct execution of the supplied script, without compilation, in a manner similar to the time command. It can be used to measure the cost of Tcl_EvalObjEx, of the invocation of canonical lists, and of the uncompiled versions of bytecoded commands.

As opposed to the time commmand, which runs the tested script for a fixed number of iterations, the timerate command runs it for a fixed time. Additionally, the compiled variant of the script will be used during the entire measurement, as if the script were part of a compiled procedure, if the **-direct** option is not specified. The fixed time period and possibility of compilation allow for more precise results and prevent very long execution times by slow scripts, making it practical for measuring scripts with highly uncertain execution times.

Examples

Estimate how fast it takes for a simple Tcl for loop (including operations on variable *i*) to count to ten:

```
# calibrate
timerate -calibrate {}

# measure
timerate { for {set i 0} {$i<10} {incr i} {} } 5000
```

Estimate how fast it takes for a simple Tcl for loop, ignoring the overhead of the management of the variable that controls the loop:

```
# calibrate for overhead of variable operations
set i 0; timerate -calibrate {expr {$i<10}; incr i} 1000

# measure
timerate {
    for {set i 0} {$i<10} {incr i} {}
} 5000
```

Estimate the speed of calculating the hour of the day using clock format only, ignoring overhead of the portion of the script that prepares the time for it to calculate:

```
# calibrate
timerate -calibrate {}

# estimate overhead
set tm 0
set ovh [lindex [timerate -overhead 0 {
```

```
    incr tm [expr {24*60*60}]
}] 0]

# measure using estimated overhead
set tm 0
timerate -overhead $ovh {
    clock format $tm -format %H
    incr tm [expr {24*60*60}]; # overhead for this is ignored
} 5000
```

In this last example, note that the overhead itself is measured using **timerate** invoked with **-overhead 0**. This is necessary because explicit overheads are assumed to be absolute values, and not an increment over the default calibrated overhead. It is therefore important that the calibrated overhead is excluded in the measurement of the overhead value itself. This is accomplished by passing **-overhead 0** when measuring the overhead.

Copyright (c) 2005 Sergey Brester aka sebres.

tm

Keywords: tm, modules, package

Name

tm - Facilities for locating and loading of Tcl Modules

Synopsis

::tcl::tm::path add ?*path...*?
::tcl::tm::path remove ?*path...*?
::tcl::tm::path list
::tcl::tm::roots *paths*

Description

This document describes the facilities for locating and loading Tcl Modules (see Module definition for the definition of a Tcl Module). The following commands are supported:

::tcl::tm::path add ?*path*...?

The paths are added at the head to the list of module paths, in order of appearance. This means that the last argument ends up as the new head of the list.

The command enforces the restriction that no path may be an ancestor directory of any other path on the list. If any of the new paths violates this restriction an error will be raised, before any of the paths have been added. In other words, if only one path argument violates the restriction then none will be added.

If a path is already present as is, no error will be raised and no action will be taken.

Paths are searched later in the order of their appearance in the list. As they are added to the front of the list they are searched in reverse order of addition. In other words, the paths added last are looked at first.

::tcl::tm::path remove ?*path*...?
Removes the paths from the list of module paths. The command silently ignores all paths which are not on the list.

::tcl::tm::path list
Returns a list containing all registered module paths, in the order that they are searched for modules.

::tcl::tm::roots *paths*
Similar to **path add**, and layered on top of it. This command takes a single argument containing a list of paths, extends each with "**tcl***X*/**site-tcl**", and "**tcl***X*/*X*.*y*", for major version *X* of the Tcl interpreter and minor version *y* less than or equal to the minor version of the interpreter, and adds the resulting set of paths to the list of paths to search.

This command is used internally by the system to set up the system-specific default paths.

The command has been exposed to allow a build system to define additional root paths beyond those described by this document.

Module definition

A Tcl Module is a Tcl Package contained in a single file, and no other files required by it. This file has to be sourceable. In other words, a Tcl Module is always imported via:

```
source module_file
```

The load command is not directly used. This restriction is not an actual limitation, as some may believe. Ever since 8.4 the Tcl source command reads only until the first ^Z character. This allows us to combine an arbitrary Tcl script with arbitrary binary data into one file, where the script processes the attached data in any it chooses to fully import and activate the package.

The name of a module file has to match the regular expression:

```
([_[:alpha:]][:_[:alnum:]]*)-([[:digit:]].*)\.tm
```

The first capturing parentheses provides the name of the package, the second clause its version. In addition to matching the pattern, the extracted version number must not raise an error when used in the command:

```
package vcompare $version 0
```

Finding modules

The directory tree for storing Tcl modules is separate from other parts of the filesystem and independent of auto_path.

Tcl Modules are searched for in all directories listed in the result of the command **::tcl::tm::path list**. This is called the *Module path*. Neither the auto_path nor the tcl_pkgPath variables are used. All directories on the module path have to obey one restriction:

For any two directories, neither is an ancestor directory of the other.

This is required to avoid ambiguities in package naming. If for example the two directories "*foo/*" and "*foo/cool*" were on the path a package named **cool::ice** could be found via the names **cool::ice** or **ice**, the latter potentially obscuring a package named **ice**, unqualified.

Before the search is started, the name of the requested package is translated into a partial path, using the following algorithm:

All occurrences of "**::**" in the package name are replaced by the appropriate directory separator character for the platform we are on. On Unix, for example, this is "**/**".

Example:

The requested package is **encoding::base64**. The generated partial path is "*encoding/base64*".

After this translation the package is looked for in all module paths, by combining them one-by-one, first to last with the partial path to form a complete search pattern. Note that the search algorithm rejects all files where the filename does not match the regular expression given in the section Module definition. For the remaining files *provide scripts* are generated and added to the package ifneeded database.

The algorithm falls back to the previous unknown handler when none of the found module files satisfy the request. If the request was satisfied the fall-back is ignored.

Note that packages in module form have *no* control over the *index* and *provide script*s entered into the package database for them. For a module file **MF** the *index script* is always:

```
package ifneeded PNAME PVERSION [list source MF]
```

and the *provide script* embedded in the above is:

```
source MF
```

Both package name **PNAME** and package version **PVERSION** are extracted from the filename **MF** according to the definition below:

```
MF = /module_path/PNAME'-PVERSION.tm
```

Where **PNAME'** is the partial path of the module as defined in section Finding modules, and translated into **PNAME** by changing all directory separators to "**::**", and **module_path** is the path (from the list of paths to search) that we found the module file under.

Note also that we are here creating a connection between package names and paths. Tcl is case-sensitive when it comes to comparing package names, but there are filesystems which are not, like NTFS. Luckily these filesystems do store the case of the name, despite not using the information when comparing.

Given the above we allow the names for packages in Tcl modules to have mixed-case, but also require that there are no collisions when comparing names in a case-insensitive manner. In other words, if a package **Foo** is deployed in the form of a Tcl Module, packages like **foo**, **fOo**, etc. are not allowed anymore.

Default paths

The default list of paths on the module path is computed by a tclsh as follows, where *X* is the major version of the Tcl interpreter and *y* is less than or equal to the minor version of the Tcl interpreter.

All the default paths are added to the module path, even those paths which do not exist. Non-existent paths are filtered out during actual searches. This enables a user to create one of the paths searched when needed and all running applications will automatically pick up any modules placed in them.

The paths are added in the order as they are listed below, and for lists of paths defined by an environment variable in the order they are found in the variable.

System specific paths

file normalize [info library]/../tcl*X*/*X*.*y*

In other words, the interpreter will look into a directory specified by its major version and whose minor versions are less than or equal to the minor version of the interpreter.

For example for Tcl 8.4 the paths searched are:

```
[info library]/../tcl8/8.4
[info library]/../tcl8/8.3
[info library]/../tcl8/8.2
[info library]/../tcl8/8.1
[info library]/../tcl8/8.0
```

This definition assumes that a package defined for Tcl *X*.*y* can also be used by all interpreters which have the same major number *X* and a minor number greater than *y*.

file normalize EXEC/tcl*X*/*X*.*y*

Where **EXEC** is **file normalize [info nameofexecutable]/../lib** or **file normalize [::tcl::pkgconfig get libdir,runtime]**

This sets of paths is handled equivalently to the set coming before, except that it is anchored in **EXEC_PREFIX**. For a build with **PREFIX** = **EXEC_PREFIX** the two sets are identical.

Site specific paths

file normalize [info library]/../tcl*X*/site-tcl

Note that this is always a single entry because *X* is always a specific value (the current major version of Tcl).

User specific paths

$::env(TCL*X*_*y*_TM_PATH)

A list of paths, separated by either**:** (Unix) or**;** (Windows). This is user and site specific as this environment variable can be set not only by the user's profile, but by system configuration scripts as well.

$::env(TCL*X*.*y*_TM_PATH)

Same meaning and content as the previous variable. However the use of dot '.' to separate major and minor version number makes this name less to non-portable and its use is discouraged. Support of this variable has been kept only for backward compatibility with the original specification, i.e. TIP 189.

These paths are seen and therefore shared by all Tcl shells in the **$::env(PATH)** of the user.

Note that *X* and *y* follow the general rules set out above. In other words, Tcl 8.4, for example, will look at these 10 environment variables:

```
$::env(TCL8.4_TM_PATH)  $::env(TCL8_4_TM_PATH)
$::env(TCL8.3_TM_PATH)  $::env(TCL8_3_TM_PATH)
$::env(TCL8.2_TM_PATH)  $::env(TCL8_2_TM_PATH)
$::env(TCL8.1_TM_PATH)  $::env(TCL8_1_TM_PATH)
$::env(TCL8.0_TM_PATH)  $::env(TCL8_0_TM_PATH)
```

Copyright (c) 2004-2010 Andreas Kupries andreas_kupries@users.sourceforge.net

trace

Keywords: trace, read, command, rename, variable, write, unset

Name

trace - Monitor variable accesses, command usages and command executions

Synopsis

trace *option* ?*arg arg* ...?

Description

This command causes Tcl commands to be executed whenever certain operations are invoked. The legal *option*s (which may be abbreviated) are:

trace add *type name ops ?args?*

Where *type* is **command**, **execution**, or **variable**.

trace add command *name ops commandPrefix*

Arrange for *commandPrefix* to be executed (with additional arguments) whenever command *name* is modified in one of the ways given by the list *ops*. *Name* will be resolved using the usual namespace resolution rules used by commands. If the command does not exist, an error will be thrown.

Ops indicates which operations are of interest, and is a list of one or more of the following items:

rename

Invoke *commandPrefix* whenever the traced command is renamed. Note that renaming to the empty string is considered deletion, and will not be traced with "**rename**".

delete

Invoke *commandPrefix* when the traced command is deleted. Commands can be deleted explicitly by using the **rename** command to rename the command to an empty string. Commands are also deleted when the interpreter is deleted, but traces will not be invoked because there is no interpreter in which to execute them.

When the trace triggers, depending on the operations being traced, a number of arguments are appended to *commandPrefix* so that the actual command is as follows:

```
commandPrefix oldName newName op
```

OldName and *newName* give the traced command's current (old) name, and the name to which it is being renamed (the empty string if this is a "delete" operation). *Op* indicates what operation is being performed on the command, and is one of **rename** or **delete** as defined above. The trace operation cannot be used to stop a command from being deleted.

Tcl will always remove the command once the trace is complete. Recursive renaming or deleting will not cause further traces of the same type to be evaluated, so a delete trace which itself deletes the command, or a rename trace which itself renames the command will not cause further trace evaluations to occur. Both *oldName* and *newName* are fully qualified with any namespace(s) in which they appear.

trace add execution *name ops commandPrefix*

Arrange for *commandPrefix* to be executed (with additional arguments) whenever command *name* is executed, with traces occurring at the points indicated by the list *ops*. *Name* will be resolved using the usual namespace resolution rules used by commands. If the command does not exist, an error will be thrown.

Ops indicates which operations are of interest, and is a list of one or more of the following items:

enter
Invoke *commandPrefix* whenever the command *name* is executed, just before the actual execution takes place.

leave
Invoke *commandPrefix* whenever the command *name* is executed, just after the actual execution takes place.

enterstep
Invoke *commandPrefix* for every Tcl command which is executed from the start of the execution of the procedure *name* until that procedure finishes. *CommandPrefix* is invoked just before the actual execution of the Tcl command being reported takes place. For example if we have "proc foo {} { puts "hello" }", then an *enterstep* trace would be invoked just before "*puts* "*hello*"" is executed. Setting an *enterstep* trace on a command *name* that does not refer to a procedure will not result in an error and is simply ignored.

leavestep
Invoke *commandPrefix* for every Tcl command which is executed from the start of the execution of the procedure *name* until that procedure finishes. *CommandPrefix* is invoked just after the actual execution of the Tcl command being reported takes place. Setting a *leavestep* trace on a command *name* that does not refer to a procedure will not result in an error and is simply ignored.

When the trace triggers, depending on the operations being traced, a number of arguments are appended to *commandPrefix* so that the actual command is as follows:

For **enter** and **enterstep** operations:

```
commandPrefix command-string op
```

Command-string gives the complete current command being executed (the traced command for a **enter** operation, an arbitrary command for a **enterstep** operation), including all arguments in their fully expanded form. *Op* indicates what operation is being performed on the command execution, and is one of **enter** or **enterstep** as defined above. The trace operation can be used to stop the command from executing, by deleting the command in question. Of course when the command is subsequently executed, an "invalid command" error will occur.

For **leave** and **leavestep** operations:

```
commandPrefix command-string code result op
```

Command-string gives the complete current command being executed (the traced command for a **enter** operation, an arbitrary command for a **enterstep** operation), including all arguments in their fully expanded form. *Code* gives the result code of that execution, and *result* the result string. *Op* indicates what operation is being performed on the command execution, and is one of **leave** or **leavestep** as defined above. Note that the creation of many **enterstep** or **leavestep** traces can lead to unintuitive results, since the invoked commands from one trace can themselves lead to further command invocations for other traces.

CommandPrefix executes in the same context as the code that invoked the traced operation: thus the *commandPrefix*, if invoked from a procedure, will have access to the same local variables as code in the procedure. This context may be different than the context in which the trace was created. If *commandPrefix* invokes a procedure (which it normally does) then the procedure will have to use upvar or uplevel commands if it wishes to access the local variables of the code which invoked the trace operation.

While *commandPrefix* is executing during an execution trace, traces on *name* are temporarily disabled. This allows the *commandPrefix* to execute *name* in its body without invoking any other traces again. If an error occurs while executing the *commandPrefix*, then the command *name* as a whole will return that same error.

When multiple traces are set on *name*, then for *enter* and *enterstep* operations, the traced commands are invoked in the reverse order of how the traces were originally created; and for *leave* and *leavestep* operations, the traced commands are invoked in the original order of creation.

The behavior of execution traces is currently undefined for a command *name* imported into another namespace.

trace add variable *name ops commandPrefix*

Arrange for *commandPrefix* to be executed whenever variable *name* is accessed in one of the ways given by the list *ops*. *Name* may refer to a normal variable, an element of an array, or to an array as a whole (i.e. *name* may be just the name of an array, with no parenthesized index). If *name* refers to a whole array, then *commandPrefix* is invoked whenever any element of the array is manipulated. If the variable does not exist, it will be created but will not be given a value, so it will be visible to namespace which queries, but not to info exists queries.

Ops indicates which operations are of interest, and is a list of one or more of the following items:

array

Invoke *commandPrefix* whenever the variable is accessed or modified via the array command, provided that *name* is not a scalar variable at the time that the array command is invoked. If *name* is a scalar variable, the access via the array command will not trigger the trace.

read

Invoke *commandPrefix* whenever the variable is read.

write
Invoke *commandPrefix* whenever the variable is written.

unset
Invoke *commandPrefix* whenever the variable is unset. Variables can be unset explicitly with the **unset** command, or implicitly when procedures return (all of their local variables are unset). Variables are also unset when interpreters are deleted, but traces will not be invoked because there is no interpreter in which to execute them.

When the trace triggers, three arguments are appended to *commandPrefix* so that the actual command is as follows:

```
commandPrefix name1 name2 op
```

Name1 and *name2* give the name(s) for the variable being accessed: if the variable is a scalar then *name1* gives the variable's name and *name2* is an empty string; if the variable is an array element then *name1* gives the name of the array and name2 gives the index into the array; if an entire array is being deleted and the trace was registered on the overall array, rather than a single element, then *name1* gives the array name and *name2* is an empty string. *Name1* and *name2* are not necessarily the same as the name used in the **trace variable** command: the upvar command allows a procedure to reference a variable under a different name. *Op* indicates what operation is being performed on the variable, and is one of **read**, **write**, or **unset** as defined above.

CommandPrefix executes in the same context as the code that invoked the traced operation: if the variable was accessed as part of a Tcl procedure, then *commandPrefix* will have access to the same local variables as code in the procedure. This context may be different than the context in which the trace was created. If *commandPrefix* invokes a procedure (which it normally does) then the procedure will have to use upvar or uplevel if it wishes to access the traced variable. Note also that *name1* may not necessarily be the same as the name used to set the trace on the variable; differences can occur if the access is made through a variable defined with the upvar command.

For read and write traces, *commandPrefix* can modify the variable to affect the result of the traced operation. If *commandPrefix* modifies the value of a variable during a read or write trace, then the new value will be returned as the result of the traced operation. The return value from *commandPrefix* is ignored except that if it returns an error of any sort then the traced operation also returns an error with the same error message returned by the trace command (this mechanism can be used to implement read-only variables, for example). For write traces, *commandPrefix* is invoked after the variable's value has been changed; it can write a new value into the variable to override the original value specified in the write operation. To implement read-only variables, *commandPrefix* will have to restore the old value of the variable.

While *commandPrefix* is executing during a read or write trace, traces on the variable are temporarily disabled. This means that reads and writes invoked by *commandPrefix* will occur directly, without invoking *commandPrefix* (or any other traces) again. However, if *commandPrefix* unsets the variable then unset traces will be invoked.

When an unset trace is invoked, the variable has already been deleted: it will appear to be undefined with no traces. If an unset occurs because of a procedure return, then the trace will be invoked in the variable context of the procedure being returned to: the stack frame of the returning procedure will no longer exist. Traces are not disabled during unset traces, so if an unset trace command creates a new trace and accesses the variable, the trace will be invoked. Any errors in unset traces are ignored.

If there are multiple traces on a variable they are invoked in order of creation, most-recent first. If one trace returns an error, then no further traces are invoked for the variable. If an array element has a trace set, and there is also a trace set on the array as a whole, the trace on the overall array is invoked before the one on the element.

Once created, the trace remains in effect either until the trace is removed with the **trace remove variable** command described below, until the variable is unset, or until the interpreter is deleted. Unsetting an element of array will remove any traces on that element, but will not remove traces on the overall array.

This command returns an empty string.

trace remove *type name opList commandPrefix*

Where *type* is either **command**, **execution** or **variable**.

trace remove command *name opList commandPrefix*

If there is a trace set on command *name* with the operations and command given by *opList* and *commandPrefix*, then the trace is removed, so that *commandPrefix* will never again be invoked. Returns an empty string. If *name* does not exist, the command will throw an error.

trace remove execution *name opList commandPrefix*

If there is a trace set on command *name* with the operations and command given by *opList* and *commandPrefix*, then the trace is removed, so that *commandPrefix* will never again be invoked. Returns an empty string. If *name* does not exist, the command will throw an error.

trace remove variable *name opList commandPrefix*

If there is a trace set on variable *name* with the operations and command given by *opList* and *commandPrefix*, then the trace is removed, so that *commandPrefix* will never again be invoked. Returns an empty string.

trace info *type name*

Where *type* is either **command**, **execution** or **variable**.

trace info command *name*

Returns a list containing one element for each trace currently set on command *name*. Each element of the list is itself a list containing two elements, which are the *opList* and *commandPrefix* associated with the trace. If *name* does not have any traces set, then the result of the command will be an empty string. If *name* does not exist, the command will throw an error.

trace info execution *name*

Returns a list containing one element for each trace currently set on command *name*. Each element of the list is itself a list containing two elements, which are the *opList* and *commandPrefix* associated with the trace. If *name* does not have any traces set, then the result of the command will be an empty string. If *name* does not exist, the command will throw an error.

trace info variable *name*
Returns a list containing one element for each trace currently set on variable *name*. Each element of the list is itself a list containing two elements, which are the *opList* and *commandPrefix* associated with the trace. If *name* does not exist or does not have any traces set, then the result of the command will be an empty string.

For backwards compatibility, three other subcommands are available:

trace variable *name ops command*
This is equivalent to **trace add variable** *name ops command*.

trace vdelete *name ops command*
This is equivalent to **trace remove variable** *name ops command*

trace vinfo *name*
This is equivalent to **trace info variable** *name*

These subcommands are deprecated and will likely be removed in a future version of Tcl. They use an older syntax in which array, **read**, **write**, **unset** are replaced by **a**, **r**, **w** and **u** respectively, and the *ops* argument is not a list, but simply a string concatenation of the operations, such as **rwua**.

Examples

Print a message whenever either of the global variables **foo** and **bar** are updated, even if they have a different local name at the time (which can be done with the upvar command):

```
proc tracer {varname args} {
    upvar #0 $varname var
    puts "$varname was updated to be \"$var\""
}
trace add variable foo write "tracer foo"
trace add variable bar write "tracer bar"
```

Ensure that the global variable **foobar** always contains the product of the global variables **foo** and **bar**:

```
proc doMult args {
    global foo bar foobar
    set foobar [expr {$foo * $bar}]
}
trace add variable foo write doMult
trace add variable bar write doMult
```

Print a trace of what commands are executed during the processing of a Tcl procedure:

```
proc x {} { y }
proc y {} { z }
proc z {} { puts hello }
proc report args {puts [info level 0]}
trace add execution x enterstep report
x
  → report y enterstep
```

```
    report z enterstep
    report {puts hello} enterstep
    hello
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2000 Ajuba Solutions.

transchan

Keywords: transchan, API, channel, ensemble, prefix, transformation

Name

transchan - command handler API of channel transforms

Synopsis

cmdPrefix *option* ?*arg arg* ...?

Description

The Tcl-level handler for a channel transformation has to be a command with subcommands (termed an *ensemble* despite not implying that it must be created with **namespace ensemble create**; this mechanism is not tied to namespace ensemble in any way). Note that *cmdPrefix* is whatever was specified in the call to chan push, and may consist of multiple arguments; this will be expanded to multiple words in place of the prefix.

Of all the possible subcommands, the handler *must* support **initialize** and **finalize**. Transformations for writable channels must also support **write**, and transformations for readable channels must also support **read**.

Note that in the descriptions below *cmdPrefix* may be more than one word, and *handle* is the value returned by the chan push call used to create the transformation.

Generic subcommands

The following subcommands are relevant to all types of channel.

cmdPrefix* clear *handle
: This optional subcommand is called to signify to the transformation that any data stored in internal buffers (either incoming or outgoing) must be cleared. It is called when a chan seek is performed on the channel being transformed.

cmdPrefix* finalize *handle
: This mandatory subcommand is called last for the given *handle*, and then never again, and it exists to allow for cleaning up any Tcl-level data structures associated with the transformation. *Warning!* Any errors thrown by this subcommand will be ignored. It is not guaranteed to be called if the interpreter is deleted.

cmdPrefix* initialize *handle mode
: This mandatory subcommand is called first, and then never again (for the given *handle*). Its responsibility is to initialize all parts of the transformation at the Tcl level. The *mode* is a list containing any of **read** and **write**.

  write
  : implies that the channel is writable.

read
implies that the channel is readable.

The return value of the subcommand should be a list containing the names of all subcommands supported by this handler. Any error thrown by the subcommand will prevent the creation of the transformation. The thrown error will appear as error thrown by chan push.

Read-related subcommands

These subcommands are used for handling transformations applied to readable channels; though strictly **read** is optional, it must be supported if any of the others is or the channel will be made non-readable.

cmdPrefix* drain *handle
This optional subcommand is called whenever data in the transformation input (i.e. read) buffer has to be forced upward, i.e. towards the user or script. The result returned by the method is taken as the *binary* data to push upward to the level above this transformation (the reader or a higher-level transformation).

In other words, when this method is called the transformation cannot defer the actual transformation operation anymore and has to transform all data waiting in its internal read buffers and return the result of that action.

cmdPrefix* limit? *handle
This optional subcommand is called to allow the Tcl I/O engine to determine how far ahead it should read. If present, it should return an integer number greater than zero which indicates how many bytes ahead should be read, or an integer less than zero to indicate that the I/O engine may read as far ahead as it likes.

cmdPrefix* read *handle buffer
This subcommand, which must be present if the transformation is to work with readable channels, is called whenever the base channel, or a transformation below this transformation, pushes data upward. The *buffer* contains the binary data which has been given to us from below. It is the responsibility of this subcommand to actually transform the data. The result returned by the subcommand is taken as the binary data to push further upward to the transformation above this transformation. This can also be the user or script that originally read from the channel.

Note that the result is allowed to be empty, or even less than the data we received; the transformation is not required to transform everything given to it right now. It is allowed to store incoming data in internal buffers and to defer the actual transformation until it has more data.

Write-related subcommands

These subcommands are used for handling transformations applied to writable channels; though strictly **write** is optional, it must be supported if any of the others is or the channel will be made non-writable.

cmdPrefix* flush *handle
This optional subcommand is called whenever data in the transformation 'write' buffer has to be forced downward, i.e. towards the base channel. The result returned by the subcommand

is taken as the binary data to write to the transformation below the current transformation. This can be the base channel as well.

In other words, when this subcommand is called the transformation cannot defer the actual transformation operation anymore and has to transform all data waiting in its internal write buffers and return the result of that action.

cmdPrefix* write *handle buffer

This subcommand, which must be present if the transformation is to work with writable channels, is called whenever the user, or a transformation above this transformation, writes data downward. The *buffer* contains the binary data which has been written to us. It is the responsibility of this subcommand to actually transform the data.

The result returned by the subcommand is taken as the binary data to write to the transformation below this transformation. This can be the base channel as well. Note that the result is allowed to be empty, or less than the data we got; the transformation is not required to transform everything which was written to it right now. It is allowed to store this data in internal buffers and to defer the actual transformation until it has more data.

Copyright (c) 2008 Donal K. Fellows

try

Keywords: try, cleanup, error, exception, final, resource management

Name

try - Trap and process errors and exceptions

Synopsis

try *body* ?*handler...*? ?*finally script*?

Description

This command executes the script *body* and, depending on what the outcome of that script is (normal exit, error, or some other exceptional result), runs a handler script to deal with the case. Once that has all happened, if the **finally** clause is present, the *script* it includes will be run and the result of the handler (or the *body* if no handler matched) is allowed to continue to propagate. Note that the **finally** clause is processed even if an error occurs and irrespective of which, if any, *handler* is used.

The *handler* clauses are each expressed as several words, and must have one of the following forms:

on *code variableList script*

This clause matches if the evaluation of *body* completed with the exception code *code*. The *code* may be expressed as an integer or one of the following literal words: **ok**, **error**, **return**, **break**, or **continue**. Those literals correspond to the integers 0 through 4 respectively.

trap *pattern variableList script*

This clause matches if the evaluation of *body* resulted in an error and the prefix of the **-errorcode** from the interpreter's status dictionary is equal to the *pattern*. The number of prefix words taken from the **-errorcode** is equal to the list-length of *pattern*, and inter-word

spaces are normalized in both the **-errorcode** and *pattern* before comparison.

The *variableList* word in each *handler* is always interpreted as a list of variable names. If the first word of the list is present and non-empty, it names a variable into which the result of the evaluation of *body* (from the main **try**) will be placed; this will contain the human-readable form of any errors. If the second word of the list is present and non-empty, it names a variable into which the options dictionary of the interpreter at the moment of completion of execution of *body* will be placed.

The *script* word of each *handler* is also always interpreted the same: as a Tcl script to evaluate if the clause is matched. If *script* is a literal "-" and the *handler* is not the last one, the *script* of the following *handler* is invoked instead (just like with the switch command).

Note that *handler* clauses are matched against in order, and that the first matching one is always selected. At most one *handler* clause will selected. As a consequence, an **on error** will mask any subsequent **trap** in the **try**. Also note that **on error** is equivalent to **trap {}**.

If an exception (i.e. any non-**ok** result) occurs during the evaluation of either the *handler* or the **finally** clause, the original exception's status dictionary will be added to the new exception's status dictionary under the **-during** key.

Examples

Ensure that a file is closed no matter what:

```
set f [open /some/file/name a]
try {
    puts $f "some message"
    # ...
} finally {
    close $f
}
```

Handle different reasons for a file to not be openable for reading:

```
try {
    set f [open /some/file/name r]
} trap {POSIX EISDIR} {} {
    puts "failed to open /some/file/name: it's a directory"
} trap {POSIX ENOENT} {} {
    puts "failed to open /some/file/name: it doesn't exist"
}
```

Copyright (c) 2008 Donal K. Fellows

unknown

Keywords: unknown, error, non-existent command

Name

unknown - Handle attempts to use non-existent commands

Synopsis

unknown *cmdName* ?*arg arg* ...?

Description

This command is invoked by the Tcl interpreter whenever a script tries to invoke a command that does not exist. The default implementation of **unknown** is a library procedure defined when Tcl initializes an interpreter. You can override the default **unknown** to change its functionality, or you can register a new handler for individual namespaces using the namespace unknown command. Note that there is no default implementation of **unknown** in a safe interpreter.

If the Tcl interpreter encounters a command name for which there is not a defined command (in either the current namespace, or the global namespace), then Tcl checks for the existence of an unknown handler for the current namespace. By default, this handler is a command named **::unknown**. If there is no such command, then the interpreter returns an error. If the **unknown** command exists (or a new handler has been registered for the current namespace), then it is invoked with arguments consisting of the fully-substituted name and arguments for the original non-existent command. The **unknown** command typically does things like searching through library directories for a command procedure with the name *cmdName*, or expanding abbreviated command names to full-length, or automatically executing unknown commands as sub-processes. In some cases (such as expanding abbreviations) **unknown** will change the original command slightly and then (re-)execute it. The result of the **unknown** command is used as the result for the original non-existent command.

The default implementation of **unknown** behaves as follows. It first calls the auto_load library procedure to load the command. If this succeeds, then it executes the original command with its original arguments. If the auto-load fails and Tcl is run interactively then **unknown** calls auto_execok to see if there is an executable file by the name *cmd*. If so, it invokes the Tcl exec command with *cmd* and all the *args* as arguments. If *cmd* cannot be auto-executed, **unknown** checks to see if the command was invoked at top-level and outside of any script. If so, then **unknown** takes two additional steps. First, it sees if *cmd* has one of the following three forms:**!!**, **!***event*, or **^***old***^***new*?**^**?. If so, then **unknown** carries out history substitution in the same way that **csh** would for these constructs. Finally, **unknown** checks to see if *cmd* is a unique abbreviation for an existing Tcl command. If so, it expands the command name and executes the command with the original arguments. If none of the above efforts has been able to execute the command, **unknown** generates an error return. If the global variable **auto_noload** is defined, then the auto-load step is skipped. If the global variable **auto_noexec** is defined then the auto-exec step is skipped. Under normal circumstances the return value from **unknown** is the return value from the command that was eventually executed.

Example

Arrange for the **unknown** command to have its standard behavior except for first logging the fact that a command was not found:

```
# Save the original one so we can chain to it
rename unknown _original_unknown

# Provide our own implementation
proc unknown args {
    puts stderr "WARNING: unknown command: $args"
    uplevel 1 [list _original_unknown {*}$args]
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

unload

Keywords: unload, binary code, unloading, safe interpreter, shared library

Name

unload - Unload machine code

Synopsis

unload ?*switches*? *fileName*
unload ?*switches*? *fileName prefix*
unload ?*switches*? *fileName prefix interp*

Description

This command tries to unload shared libraries previously loaded with load from the application's address space. *fileName* is the name of the file containing the library file to be unload; it must be the same as the filename provided to load for loading the library. The *prefix* argument is the prefix (as determined by or passed to load), and is used to compute the name of the unload procedure; if not supplied, it is computed from *fileName* in the same manner as load. The *interp* argument is the path name of the interpreter from which to unload the package (see the interp manual entry for details); if *interp* is omitted, it defaults to the interpreter in which the **unload** command was invoked.

If the initial arguments to **unload** start with **-** then they are treated as switches. The following switches are currently supported:

-nocomplain
Suppresses all error messages. If this switch is given, **unload** will never report an error.

-keeplibrary
This switch will prevent **unload** from issuing the operating system call that will unload the library from the process.

--
Marks the end of switches. The argument following this one will be treated as a *fileName* even if it starts with a **-**.

Unload operation

When a file containing a shared library is loaded through the load command, Tcl associates two reference counts to the library file. The first counter shows how many times the library has been

loaded into normal (trusted) interpreters while the second describes how many times the library has been loaded into safe interpreters. As a file containing a shared library can be loaded only once by Tcl (with the first load call on the file), these counters track how many interpreters use the library. Each subsequent call to load after the first simply increments the proper reference count.

unload works in the opposite direction. As a first step, **unload** will check whether the library is unloadable: an unloadable library exports a special unload procedure. The name of the unload procedure is determined by *prefix* and whether or not the target interpreter is a safe one. For normal interpreters the name of the initialization procedure will have the form *pfx*_**Unload**, where *pfx* is the same as *prefix* except that the first letter is converted to upper case and all other letters are converted to lower case. For example, if *prefix* is **foo** or **FOo**, the initialization procedure's name will be **Foo_Unload**. If the target interpreter is a safe interpreter, then the name of the initialization procedure will be *pkg*_**SafeUnload** instead of *pkg*_**Unload**.

If **unload** determines that a library is not unloadable (or unload functionality has been disabled during compilation), an error will be returned. If the library is unloadable, then **unload** will call the unload procedure. If the unload procedure returns TCL_OK, **unload** will proceed and decrease the proper reference count (depending on the target interpreter type). When both reference counts have reached 0, the library will be detached from the process.

Unload hook prototype

The unload procedure must match the following prototype:

```
typedef int Tcl_PackageUnloadProc(
        Tcl_Interp *interp,
        int flags);
```

The *interp* argument identifies the interpreter from which the library is to be unloaded. The unload procedure must return TCL_OK or TCL_ERROR to indicate whether or not it completed successfully; in the event of an error it should set the interpreter's result to point to an error message. In this case, the result of the **unload** command will be the result returned by the unload procedure.

The *flags* argument can be either **TCL_UNLOAD_DETACH_FROM_INTERPRETER** or **TCL_UNLOAD_DETACH_FROM_PROCESS**. In case the library will remain attached to the process after the unload procedure returns (i.e. because the library is used by other interpreters), **TCL_UNLOAD_DETACH_FROM_INTERPRETER** will be defined. However, if the library is used only by the target interpreter and the library will be detached from the application as soon as the unload procedure returns, the *flags* argument will be set to **TCL_UNLOAD_DETACH_FROM_PROCESS**.

Notes

The **unload** command cannot unload libraries that are statically linked with the application. If *fileName* is an empty string, then the *prefix* argument must be specified.

If *prefix* is omitted or specified as an empty string, Tcl tries to guess the prefix. This may be done differently on different platforms. The default guess, which is used on most UNIX platforms, is to take the last element of *fileName*, strip off the first three characters if they are **lib**, and use any following alphabetic and underline characters, converted to titlecase as the prefix. For example, the command **unload libxyz4.2.so** uses the prefix **Xyz** and the command **unload bin/last.so**

{} uses the prefix **Last**.

Portability issues

Unix

Not all unix operating systems support library unloading. Under such an operating system **unload** returns an error (unless **-nocomplain** has been specified).

Bugs

If the same file is loaded by different *fileName*s, it will be loaded into the process's address space multiple times. The behavior of this varies from system to system (some systems may detect the redundant loads, others may not). In case a library has been silently detached by the operating system (and as a result Tcl thinks the library is still loaded), it may be dangerous to use **unload** on such a library (as the library will be completely detached from the application while some interpreters will continue to use it).

Example

If an unloadable module in the file **foobar.dll** had been loaded using the load command like this (on Windows):

```
load c:/some/dir/foobar.dll
```

then it would be unloaded like this:

```
unload c:/some/dir/foobar.dll
```

This allows a C code module to be installed temporarily into a long-running Tcl program and then removed again (either because it is no longer needed or because it is being updated with a new version) without having to shut down the overall Tcl process.

Copyright (c) 2003 George Petasis petasis@iit.demokritos.gr.

unset

Keywords: unset, remove, variable

Name

unset - Delete variables

Synopsis

unset ?**-nocomplain**? ?**--**? ?*name name name* ...?

Description

This command removes one or more variables. Each *name* is a variable name, specified in any of the ways acceptable to the set command. If a *name* refers to an element of an array then that element is removed without affecting the rest of the array. If a *name* consists of an array name with no parenthesized index, then the entire array is deleted. The **unset** command returns an empty string as result. If **-nocomplain** is specified as the first argument, any possible errors are suppressed. The option may not be abbreviated, in order to disambiguate it from possible variable names. The option **--** indicates the end of the options, and should be used if you wish to remove a variable with the same name as any of the options. If an error occurs during variable

deletion, any variables after the named one causing the error are not deleted. An error can occur when the named variable does not exist, or the name refers to an array element but the variable is a scalar, or the name refers to a variable in a non-existent namespace.

Example

Create an array containing a mapping from some numbers to their squares and remove the array elements for non-prime numbers:

```
array set squares {
    1 1    6 36
    2 4    7 49
    3 9    8 64
    4 16   9 81
    5 25  10 100
}

puts "The squares are:"
parray squares

unset squares(1) squares(4) squares(6)
unset squares(8) squares(9) squares(10)

puts "The prime squares are:"
parray squares
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2000 Ajuba Solutions.

update

Keywords: update, asynchronous I/O, event, flush, handler, idle

Name

update - Process pending events and idle callbacks

Synopsis

update ?**idletasks**?

Description

This command is used to bring the application "up to date" by entering the event loop repeatedly until all pending events (including idle callbacks) have been processed.

If the **idletasks** keyword is specified as an argument to the command, then no new events or errors are processed; only idle callbacks are invoked. This causes operations that are normally deferred, such as display updates and window layout calculations, to be performed immediately.

The **update idletasks** command is useful in scripts where changes have been made to the application's state and you want those changes to appear on the display immediately, rather than waiting for the script to complete. Most display updates are performed as idle callbacks, so

update idletasks will cause them to run. However, there are some kinds of updates that only happen in response to events, such as those triggered by window size changes; these updates will not occur in **update idletasks**.

The **update** command with no options is useful in scripts where you are performing a long-running computation but you still want the application to respond to events such as user interactions; if you occasionally call **update** then user input will be processed during the next call to **update**.

Example

Run computations for about a second and then finish:

```
set x 1000
set done 0
after 1000 set done 1
while {!$done} {
    # A very silly example!
    set x [expr {log($x) ** 2.8}]

    # Test to see if our time-limit has been hit.  This would
    # also give a chance for serving network sockets and, if
    # the Tk package is loaded, updating a user interface.
    update
}
```

Copyright (c) 1990-1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

uplevel

Keywords: uplevel, context, level, namespace, stack frame, variable

Name

uplevel - Execute a script in a different stack frame

Synopsis

uplevel ?*level*? *arg* ?*arg* ...?

Description

All of the *arg* arguments are concatenated as if they had been passed to concat; the result is then evaluated in the variable context indicated by *level*. **Uplevel** returns the result of that evaluation.

If *level* is an integer then it gives a distance (up the procedure calling stack) to move before executing the command. If *level* consists of **#** followed by a number then the number gives an absolute level number. If *level* is omitted then it defaults to **1**. *Level* cannot be defaulted if the first *command* argument starts with a digit or **#**.

For example, suppose that procedure **a** was invoked from top-level, and that it called **b**, and that **b** called **c**. Suppose that **c** invokes the **uplevel** command. If *level* is **1** or **#2** or omitted, then the command will be executed in the variable context of **b**. If *level* is **2** or **#1** then the command will

be executed in the variable context of **a**. If *level* is **3** or **#0** then the command will be executed at top-level (only global variables will be visible).

The **uplevel** command causes the invoking procedure to disappear from the procedure calling stack while the command is being executed. In the above example, suppose **c** invokes the command

```
uplevel 1 {set x 43; d}
```

where **d** is another Tcl procedure. The set command will modify the variable **x** in **b**'s context, and **d** will execute at level 3, as if called from **b**. If it in turn executes the command

```
uplevel {set x 42}
```

then the set command will modify the same variable **x** in **b**'s context: the procedure **c** does not appear to be on the call stack when **d** is executing. The info level command may be used to obtain the level of the current procedure.

Uplevel makes it possible to implement new control constructs as Tcl procedures (for example, **uplevel** could be used to implement the while construct as a Tcl procedure).

The namespace eval and apply commands offer other ways (besides procedure calls) that the Tcl naming context can change. They add a call frame to the stack to represent the namespace context. This means each namespace eval command counts as another call level for **uplevel** and upvar commands. For example, **info level 1** will return a list describing a command that is either the outermost procedure call or the outermost namespace eval command. Also, **uplevel #0** evaluates a script at top-level in the outermost namespace (the global namespace).

Example

As stated above, the **uplevel** command is useful for creating new control constructs. This example shows how (without error handling) it can be used to create a **do** command that is the counterpart of while except for always performing the test after running the loop body:

```
proc do {body while condition} {
    if {$while ne "while"} {
        error "required word missing"
    }
    set conditionCmd [list expr $condition]
    while {1} {
        uplevel 1 $body
        if {![uplevel 1 $conditionCmd]} {
            break
        }
    }
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

upvar

Keywords: upvar, context, frame, global, level, namespace, procedure, variable

Name

upvar - Create link to variable in a different stack frame

Synopsis

upvar ?*level*? *otherVar myVar* ?*otherVar myVar* ...?

Description

This command arranges for one or more local variables in the current procedure to refer to variables in an enclosing procedure call or to global variables. *Level* may have any of the forms permitted for the uplevel command, and may be omitted (it defaults to **1**). For each *otherVar* argument, **upvar** makes the variable by that name in the procedure frame given by *level* (or at global level, if *level* is **#0**) accessible in the current procedure by the name given in the corresponding *myVar* argument. The variable named by *otherVar* need not exist at the time of the call; it will be created the first time *myVar* is referenced, just like an ordinary variable. There must not exist a variable by the name *myVar* at the time **upvar** is invoked. *MyVar* is always treated as the name of a variable, not an array element. An error is returned if the name looks like an array element, such as **a(b)**. *OtherVar* may refer to a scalar variable, an array, or an array element. **Upvar** returns an empty string.

The **upvar** command simplifies the implementation of call-by-name procedure calling and also makes it easier to build new control constructs as Tcl procedures. For example, consider the following procedure:

```
proc add2 name {
    upvar $name x
    set x [expr {$x + 2}]
}
```

If *add2* is invoked with an argument giving the name of a variable, it adds two to the value of that variable. Although *add2* could have been implemented using uplevel instead of **upvar**, **upvar** makes it simpler for *add2* to access the variable in the caller's procedure frame.

namespace eval is another way (besides procedure calls) that the Tcl naming context can change. It adds a call frame to the stack to represent the namespace context. This means each namespace eval command counts as another call level for uplevel and **upvar** commands. For example, info level **1** will return a list describing a command that is either the outermost procedure call or the outermost namespace eval command. Also, uplevel #0 evaluates a script at top-level in the outermost namespace (the global namespace).

If an upvar variable is unset (e.g. **x** in **add2** above), the unset operation affects the variable it is linked to, not the upvar variable. There is no way to unset an upvar variable except by exiting the procedure in which it is defined. However, it is possible to retarget an upvar variable by executing another **upvar** command.

Traces and upvar

Upvar interacts with traces in a straightforward but possibly unexpected manner. If a variable trace is defined on *otherVar*, that trace will be triggered by actions involving *myVar*. However, the trace procedure will be passed the name of *myVar*, rather than the name of *otherVar*. Thus, the output of the following code will be "*localVar*" rather than "*originalVar*":

```
proc traceproc { name index op } {
    puts $name
}
proc setByUpvar { name value } {
    upvar $name localVar
    set localVar $value
}
set originalVar 1
trace variable originalVar w traceproc
setByUpvar originalVar 2
```

If *otherVar* refers to an element of an array, then variable traces set for the entire array will not be invoked when *myVar* is accessed (but traces on the particular element will still be invoked). In particular, if the array is env, then changes made to *myVar* will not be passed to subprocesses correctly.

Example

A **decr** command that works like incr except it subtracts the value from the variable instead of adding it:

```
proc decr {varName {decrement 1}} {
    upvar 1 $varName var
    incr var [expr {-$decrement}]
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

variable

Keywords: variable, global, namespace, procedure

Name

variable - create and initialize a namespace variable

Synopsis

variable *name*
variable ?*name value...*?

Description

This command is normally used within a namespace eval command to create one or more variables within a namespace. Each variable *name* is initialized with *value*. The *value* for the last variable is optional.

If a variable *name* does not exist, it is created. In this case, if *value* is specified, it is assigned to the newly created variable. If no *value* is specified, the new variable is left undefined. If the variable already exists, it is set to *value* if *value* is specified or left unchanged if no *value* is given. Normally, *name* is unqualified (does not include the names of any containing namespaces), and the variable is created in the current namespace. If *name* includes any namespace qualifiers, the variable is created in the specified namespace. If the variable is not

defined, it will be visible to the namespace which command, but not to the info exists command.

If the **variable** command is executed inside a Tcl procedure, it creates local variables linked to the corresponding namespace variables (and therefore these variables are listed by info vars.) In this way the **variable** command resembles the global command, although the global command resolves variable names with respect to the global namespace instead of the current namespace of the procedure. If any *value*s are given, they are used to modify the values of the associated namespace variables. If a namespace variable does not exist, it is created and optionally initialized.

A *name* argument cannot reference an element within an array. Instead, *name* should reference the entire array, and the initialization *value* should be left off. After the variable has been declared, elements within the array can be set using ordinary set or array commands.

Examples

Create a variable in a namespace:

```
namespace eval foo {
    variable bar 12345
}
```

Create an array in a namespace:

```
namespace eval someNS {
    variable someAry
    array set someAry {
        someName  someValue
        otherName otherValue
    }
}
```

Access variables in namespaces from a procedure:

```
namespace eval foo {
    proc spong {} {
        # Variable in this namespace
        variable bar
        puts "bar is $bar"

        # Variable in another namespace
        variable ::someNS::someAry
        parray someAry
    }
}
```

Copyright (c) 1993-1997 Bell Labs Innovations for Lucent Technologies
Copyright (c) 1997 Sun Microsystems, Inc.

vwait

Keywords: vwait, asynchronous I/O, event, variable, wait

Name
vwait - Process events until a variable is written

Synopsis
vwait *varName*

Description
This command enters the Tcl event loop to process events, blocking the application if no events are ready. It continues processing events until some event handler sets the value of the global variable *varName*. Once *varName* has been set, the **vwait** command will return as soon as the event handler that modified *varName* completes. The *varName* argument is always interpreted as a variable name with respect to the global namespace, but can refer to any namespace's variables if the fully-qualified name is given.

In some cases the **vwait** command may not return immediately after *varName* is set. This happens if the event handler that sets *varName* does not complete immediately. For example, if an event handler sets *varName* and then itself calls **vwait** to wait for a different variable, then it may not return for a long time. During this time the top-level **vwait** is blocked waiting for the event handler to complete, so it cannot return either. (See the Nested vwaits by example below.)

To be clear, *multiple* **vwait** *calls will nest and will not happen in parallel*. The outermost call to **vwait** will not return until all the inner ones do. It is recommended that code should never nest **vwait** calls (by avoiding putting them in event callbacks) but when that is not possible, care should be taken to add interlock variables to the code to prevent all reentrant calls to **vwait** that are not *strictly* necessary. Be aware that the synchronous modes of operation of some Tcl packages (e.g., **http**) use **vwait** internally; if using the event loop, it is best to use the asynchronous callback-based modes of operation of those packages where available.

Examples
Run the event-loop continually until some event calls exit. (You can use any variable not mentioned elsewhere, but the name *forever* reminds you at a glance of the intent.)

```
vwait forever
```

Wait five seconds for a connection to a server socket, otherwise close the socket and continue running the script:

```
# Initialise the state
after 5000 set state timeout
set server [socket -server accept 12345]
proc accept {args} {
    global state connectionInfo
    set state accepted
    set connectionInfo $args
}

# Wait for something to happen
vwait state
```

```
# Clean up events that could have happened
close $server
after cancel set state timeout

# Do something based on how the vwait finished...
switch $state {
    timeout {
        puts "no connection on port 12345"
    }
    accepted {
       puts "connection: $connectionInfo"
       puts [lindex $connectionInfo 0] "Hello there!"
    }
}
```

A command that will wait for some time delay by waiting for a namespace variable to be set. Includes an interlock to prevent nested waits.

```
namespace eval example {
    variable v done
    proc wait {delay} {
        variable v
        if {$v ne "waiting"} {
            set v waiting
            after $delay [namespace code {set v done}]
            vwait [namespace which -variable v]
        }
        return $v
    }
}
```

When running inside a coroutine, an alternative to using **vwait** is to yield to an outer event loop and to get recommenced when the variable is set, or at an idle moment after that.

```
coroutine task apply {{} {
    # simulate [after 1000]
    after 1000 [info coroutine]
    yield

    # schedule the setting of a global variable, as normal
    after 2000 {set var 1}

    # simulate [vwait var]
    proc updatedVar {task args} {
        after idle $task
        trace remove variable ::var write "updatedVar $task"
    }
    trace add variable ::var write "updatedVar [info coroutine]"
    yield
}}
```

Nested vwaits by example

This example demonstrates what can happen when the **vwait** command is nested. The script will never finish because the waiting for the *a* variable never finishes; that **vwait** command is still waiting for a script scheduled with after to complete, which just happens to be running an inner **vwait** (for *b*) even though the event that the outer **vwait** was waiting for (the setting of *a*) has occurred.

```
after 500 {
    puts "waiting for b"
    vwait b
    puts "b was set"
}
after 1000 {
    puts "setting a"
    set a 10
}
puts "waiting for a"
vwait a
puts "a was set"
puts "setting b"
set b 42
```

If you run the above code, you get this output:

```
waiting for a
waiting for b
setting a
```

The script will never print "a was set" until after it has printed "b was set" because of the nesting of **vwait** commands, and yet *b* will not be set until after the outer **vwait** returns, so the script has deadlocked. The only ways to avoid this are to either structure the overall program in

continuation-passing style or to use coroutine to make the continuations implicit. The first of these options would be written as:

```
after 500 {
    puts "waiting for b"
    trace add variable b write {apply {args {
        global a b
        trace remove variable ::b write \
                [lrange [info level 0] 0 1]
        puts "b was set"
        set ::done ok
    }}}
}
after 1000 {
    puts "setting a"
    set a 10
}
puts "waiting for a"
trace add variable a write {apply {args {
    global a b
    trace remove variable a write [lrange [info level 0] 0 1]
    puts "a was set"
    puts "setting b"
    set b 42
}}}
vwait done
```

The second option, with coroutine and some helper procedures, is done like this:

```
# A coroutine-based wait-for-variable command
proc waitvar globalVar {
    trace add variable ::$globalVar write \
            [list apply {{v c args} {
        trace remove variable $v write \
                [lrange [info level 0] 0 3]
        after 0 $c
    }} ::$globalVar [info coroutine]]
    yield
}
# A coroutine-based wait-for-some-time command
proc waittime ms {
    after $ms [info coroutine]
    yield
}

coroutine task-1 eval {
    puts "waiting for a"
```

```
    waitvar a
    puts "a was set"
    puts "setting b"
    set b 42
}
coroutine task-2 eval {
    waittime 500
    puts "waiting for b"
    waitvar b
    puts "b was set"
    set done ok
}
coroutine task-3 eval {
    waittime 1000
    puts "setting a"
    set a 10
}
vwait done
```

Copyright (c) 1995-1996 Sun Microsystems, Inc.

while

Keywords: while, boolean, loop, test

Name

while - Execute script repeatedly as long as a condition is met

Synopsis

while *test body*

Description

The **while** command evaluates *test* as an expression (in the same way that expr evaluates its argument). The value of the expression must a proper boolean value; if it is a true value then *body* is executed by passing it to the Tcl interpreter. Once *body* has been executed then *test* is evaluated again, and the process repeats until eventually *test* evaluates to a false boolean value. **Continue** commands may be executed inside *body* to terminate the current iteration of the loop, and break commands may be executed inside *body* to cause immediate termination of the **while** command. The **while** command always returns an empty string.

Note: *test* should almost always be enclosed in braces. If not, variable substitutions will be made before the **while** command starts executing, which means that variable changes made by the loop body will not be considered in the expression. This is likely to result in an infinite loop. If *test* is enclosed in braces, variable substitutions are delayed until the expression is evaluated (before each loop iteration), so changes in the variables will be visible. For an example, try the following script with and without the braces around **$x<10**:

```
set x 0
while {$x<10} {
    puts "x is $x"
    incr x
}
```

Example

Read lines from a channel until we get to the end of the stream, and print them out with a line-number prepended:

```
set lineCount 0
while {[gets $chan line] >= 0} {
    puts "[incr lineCount]: $line"
}
```

Copyright (c) 1993 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

zlib

Keywords: zlib, compress, decompress, deflate, gzip, inflate

Name

zlib - compression and decompression operations

Synopsis

zlib *subcommand arg ...*

Description

The **zlib** command provides access to the compression and check-summing facilities of the Zlib library by Jean-loup Gailly and Mark Adler. It has the following subcommands.

Compression subcommands

zlib compress *string* ?*level*?
: Returns the zlib-format compressed binary data of the binary string in *string*. If present, *level* gives the compression level to use (from 0, which is uncompressed, to 9, maximally compressed).

zlib decompress *string* ?*bufferSize*?
: Returns the uncompressed version of the raw compressed binary data in *string*. If present, *bufferSize* is a hint as to what size of buffer is to be used to receive the data.

zlib deflate *string* ?*level*?
: Returns the raw compressed binary data of the binary string in *string*. If present, *level* gives the compression level to use (from 0, which is uncompressed, to 9, maximally compressed).

zlib gunzip *string* ?-headerVar *varName*?
: Return the uncompressed contents of binary string *string*, which must have been in gzip format. If **-headerVar** is given, store a dictionary describing the contents of the gzip header in the variable called *varName*. The keys of the dictionary that may be present are: **comment**

The comment field from the header, if present.

crc
A boolean value describing whether a CRC of the header is computed.

filename
The filename field from the header, if present.

os
The operating system type code field from the header (if not the QW unknown value). See RFC 1952 for the meaning of these codes.

size
The size of the uncompressed data.

time
The time field from the header if non-zero, expected to be time that the file named by the **filename** field was modified. Suitable for use with clock format.

type
The type of the uncompressed data (**binary** or **text**) if known.

zlib gzip *string* ?-level *level*? ?-header *dict*?
Return the compressed contents of binary string *string* in gzip format. If **-level** is given, *level* gives the compression level to use (from 0, which is uncompressed, to 9, maximally compressed). If **-header** is given, *dict* is a dictionary containing values used for the gzip header. The following keys may be defined:

comment
Add the given comment to the header of the gzip-format data.

crc
A boolean saying whether to compute a CRC of the header. Note that if the data is to be interchanged with the **gzip** program, a header CRC should *not* be computed.

filename
The name of the file that the data to be compressed came from.

os
The operating system type code, which should be one of the values described in RFC 1952.

time
The time that the file named in the **filename** key was last modified. This will be in the same as is returned by clock seconds or file mtime.

type
The type of the data being compressed, being **binary** or **text**.

zlib inflate *string* ?*bufferSize*?
Returns the uncompressed version of the raw compressed binary data in *string*. If present, *bufferSize* is a hint as to what size of buffer is to be used to receive the data.

Channel subcommand

zlib push *mode channel* ?*options ...*?
Pushes a compressing or decompressing transformation onto the channel *channel*. The transformation can be removed again with chan pop. The *mode* argument determines what type of transformation is pushed; the following are supported:

compress
The transformation will be a compressing transformation that produces zlib-format data on *channel*, which must be writable.

decompress
The transformation will be a decompressing transformation that reads zlib-format data from *channel*, which must be readable.

deflate
The transformation will be a compressing transformation that produces raw compressed data on *channel*, which must be writable.

gunzip
The transformation will be a decompressing transformation that reads gzip-format data from *channel*, which must be readable.

gzip
The transformation will be a compressing transformation that produces gzip-format data on *channel*, which must be writable.

inflate
The transformation will be a decompressing transformation that reads raw compressed data from *channel*, which must be readable.

The following options may be set when creating a transformation via the "*options ...*" to the **zlib push** command:

-dictionary *binData*
Sets the compression dictionary to use when working with compressing or decompressing the data to be binData. Not valid for transformations that work with gzip-format data. The dictionary should consist of strings (byte sequences) that are likely to be encountered later in the data to be compressed, with the most commonly used strings preferably put towards the end of the dictionary. Tcl provides no mechanism for choosing a good such dictionary for a particular data sequence.

-header *dictionary*
Passes a description of the gzip header to create, in the same format that **zlib gzip** understands.

-level *compressionLevel*
How hard to compress the data. Must be an integer from 0 (uncompressed) to 9 (maximally compressed).

-limit *readaheadLimit*
The maximum number of bytes ahead to read when decompressing.

This option has become **irrelevant**. It was originally introduced to prevent Tcl from reading beyond the end of a compressed stream in multi-stream channels to ensure that the data after was left alone for further reading, at the cost of speed.

Tcl now automatically returns any bytes it has read beyond the end of a compressed stream back to the channel, making them appear as unread to further readers.

Both compressing and decompressing channel transformations add extra configuration options that may be accessed through chan configure. The options are:

-checksum *checksum*
This read-only option gets the current checksum for the uncompressed data that the compression engine has seen so far. It is valid for both compressing and decompressing transforms, but not for the raw inflate and deflate formats. The compression algorithm depends on what format is being produced or consumed.

-dictionary *binData*
This read-write options gets or sets the initial compression dictionary to use when working with compressing or decompressing the data to be binData. It is not valid for transformations that work with gzip-format data, and should not normally be set on compressing transformations other than at the point where the transformation is stacked. Note that this cannot be used to get the current active compression dictionary mid-stream, as that information is not exposed by the underlying library.

-flush *type*
This write-only operation flushes the current state of the compressor to the underlying channel. It is only valid for compressing transformations. The *type* must be either **sync** or **full** for a normal flush or an expensive flush respectively. Flushing degrades the compression ratio, but makes it easier for a decompressor to recover more of the file in the case of data corruption.

-header *dictionary*
This read-only option, only valid for decompressing transforms that are processing gzip-format data, returns the dictionary describing the header read off the data stream.

-limit *readaheadLimit*
This read-write option is used by decompressing channels to control the maximum number of bytes ahead to read from the underlying data source. See above for more information.

Streaming subcommand

zlib stream *mode* ?*options*?
Creates a streaming compression or decompression command based on the *mode*, and return the name of the command. For a description of how that command works, see Streaming instance command below. The following modes and *options* are supported:

zlib stream compress ?-dictionary *bindata*? ?-level *level*?
The stream will be a compressing stream that produces zlib-format output, using compression level *level* (if specified) which will be an integer from 0 to 9, and the compression dictionary bindata (if specified).

zlib stream decompress ?-dictionary *bindata*?
The stream will be a decompressing stream that takes zlib-format input and produces uncompressed output. If bindata is supplied, it is a compression dictionary to use if required.

zlib stream deflate ?-dictionary *bindata*? ?-level *level*?
The stream will be a compressing stream that produces raw output, using compression level *level* (if specified) which will be an integer from 0 to 9, and the compression dictionary bindata (if specified). Note that the raw compressed data includes no metadata about what compression dictionary was used, if any; that is a feature of the zlib-format data.

zlib stream gunzip

The stream will be a decompressing stream that takes gzip-format input and produces uncompressed output.

zlib stream gzip ?-header *header*? ?-level *level*?
The stream will be a compressing stream that produces gzip-format output, using compression level *level* (if specified) which will be an integer from 0 to 9, and the header descriptor dictionary *header* (if specified; for keys see **zlib gzip**).

zlib stream inflate ?-dictionary *bindata*?
The stream will be a decompressing stream that takes raw compressed input and produces uncompressed output. If bindata is supplied, it is a compression dictionary to use. Note that there are no checks in place to determine whether the compression dictionary is correct.

Checksumming subcommands

zlib adler32 *string* ?*initValue*?
Compute a checksum of binary string *string* using the Adler-32 algorithm. If given, *initValue* is used to initialize the checksum engine.

zlib crc32 *string* ?*initValue*?
Compute a checksum of binary string *string* using the CRC-32 algorithm. If given, *initValue* is used to initialize the checksum engine.

Streaming instance command

Streaming compression instance commands are produced by the **zlib stream** command. They are used by calling their **put** subcommand one or more times to load data in, and their **get** subcommand one or more times to extract the transformed data.

The full set of subcommands supported by a streaming instance command, *stream*, is as follows:

stream* add ?*option...*? *data
A short-cut for "*stream* **put** ?*option...*? *data*" followed by "*stream* **get**".

***stream* checksum**
Returns the checksum of the uncompressed data seen so far by this stream.

***stream* close**
Deletes this stream and frees up all resources associated with it.

***stream* eof**
Returns a boolean indicating whether the end of the stream (as determined by the compressed data itself) has been reached. Not all formats support detection of the end of the stream.

***stream* finalize**
A short-cut for "*stream* **put -finalize {}**".

***stream* flush**
A short-cut for "*stream* **put -flush {}**".

***stream* fullflush**
A short-cut for "*stream* **put -fullflush {}**".

***stream* get ?*count*?**
Return up to *count* bytes from *stream*'s internal buffers with the transformation applied. If *count* is omitted, the entire contents of the buffers are returned. *stream* **header** Return the

gzip header description dictionary extracted from the stream. Only supported for streams created with their *mode* parameter set to **gunzip**.

stream* put ?*option...*? *data

Append the contents of the binary string *data* to *stream*'s internal buffers while applying the transformation. The following *option*s are supported (or an unambiguous prefix of them), which are used to modify the way in which the transformation is applied:

-dictionary *binData*

Sets the compression dictionary to use when working with compressing or decompressing the data to be binData.

-finalize

Mark the stream as finished, ensuring that all bytes have been wholly compressed or decompressed. For gzip streams, this also ensures that the footer is written to the stream. The stream will need to be reset before having more data written to it after this, though data can still be read out of the stream with the **get** subcommand.

This option is mutually exclusive with the **-flush** and **-fullflush** options.

-flush

Ensure that a decompressor consuming the bytes that the current (compressing) stream is producing will be able to produce all the bytes that have been compressed so far, at some performance penalty.

This option is mutually exclusive with the **-finalize** and **-fullflush** options.

-fullflush

Ensure that not only can a decompressor handle all the bytes produced so far (as with **-flush** above) but also that it can restart from this point if it detects that the stream is partially corrupt. This incurs a substantial performance penalty.

This option is mutually exclusive with the **-finalize** and **-flush** options.

***stream* reset**

Puts any stream, including those that have been finalized or that have reached eof, back into a state where it can process more data. Throws away all internally buffered data.

Examples

To compress a Tcl string, it should be first converted to a particular charset encoding since the **zlib** command always operates on binary strings.

```
set binData [encoding convertto utf-8 $string]
set compData [zlib compress $binData]
```

When converting back, it is also important to reverse the charset encoding:

```
set binData [zlib decompress $compData]
set string [encoding convertfrom utf-8 $binData]
```

The compression operation from above can also be done with streams, which is especially helpful when you want to accumulate the data by stages:

```
set strm [zlib stream compress]
$strm put [encoding convertto utf-8 $string]
# ...
$strm finalize
set compData [$strm get]
$strm close
```

Copyright (c) 2008-2012 Donal K. Fellows

Tk

bell

Keywords: bell, beep, ring

Name

bell - Ring a display's bell

Synopsis

bell ?*-displayof window*? ?**-nice**?

Description

This command rings the bell on the display for *window* and returns an empty string. If the **-displayof** option is omitted, the display of the application's main window is used by default. The command uses the current bell-related settings for the display, which may be modified with programs such as **xset**.

If **-nice** is not specified, this command also resets the screen saver for the screen. Some screen savers will ignore this, but others will reset so that the screen becomes visible again.

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2000 Ajuba Solutions.

bind

Keywords: bind, binding, event

Name

bind - Arrange for X events to invoke Tcl scripts

Synopsis

bind *tag* ?*sequence*? ?**+**? ?*script*?

Introduction

The **bind** command associates Tcl scripts with X events. If all three arguments are specified, **bind** will arrange for *script* (a Tcl script called the "binding script") to be evaluated whenever the event(s) given by *sequence* occur in the window(s) identified by *tag*. If *script* is prefixed with a "+", then it is appended to any existing binding for *sequence*; otherwise *script* replaces any existing binding. If *script* is an empty string then the current binding for *sequence* is destroyed,

leaving *sequence* unbound. In all of the cases where a *script* argument is provided, **bind** returns an empty string.

If *sequence* is specified without a *script*, then the script currently bound to *sequence* is returned, or an empty string is returned if there is no binding for *sequence*. If neither *sequence* nor *script* is specified, then the return value is a list whose elements are all the sequences for which there exist bindings for *tag*.

The *tag* argument determines which window(s) the binding applies to. If *tag* begins with a dot, as in **.a.b.c**, then it must be the path name for a window; otherwise it may be an arbitrary string. Each window has an associated list of tags, and a binding applies to a particular window if its tag is among those specified for the window. Although the bindtags command may be used to assign an arbitrary set of binding tags to a window, the default binding tags provide the following behavior:

- If a tag is the name of an internal window the binding applies to that window.
- If the tag is the name of a class of widgets, such as **Button**, the binding applies to all widgets in that class.
- If the tag is the name of a toplevel window the binding applies to the toplevel window and all its internal windows.
- If *tag* has the value **all**, the binding applies to all windows in the application.

Event patterns

The *sequence* argument specifies a sequence of one or more event patterns, with optional white space between the patterns. Each event pattern may take one of three forms. In the simplest case it is a single printing ASCII character, such as **a** or **[**. The character may not be a space character or the character **<**. This form of pattern matches a **KeyPress** event for the particular character. The second form of pattern is longer but more general. It has the following syntax:

```
<modifier-modifier-type-detail>
```

The entire event pattern is surrounded by angle brackets. Inside the angle brackets are zero or more modifiers, an event type, and an extra piece of information (*detail*) identifying a particular button or keysym. Any of the fields may be omitted, as long as at least one of *type* and *detail* is present. The fields must be separated by white space or dashes.

The third form of pattern is used to specify a user-defined, named virtual event. It has the following syntax:

```
<<name>>
```

The entire virtual event pattern is surrounded by double angle brackets. Inside the angle brackets is the user-defined name of the virtual event. Modifiers, such as **Shift** or **Control**, may not be combined with a virtual event to modify it. Bindings on a virtual event may be created before the virtual event is defined, and if the definition of a virtual event changes dynamically, all windows bound to that virtual event will respond immediately to the new definition.

Some widgets (e.g. menu and text) issue virtual events when their internal state is updated in some ways. Please see the manual page for each widget for details.

Modifiers

Modifiers consist of any of the following values:

Control Mod1, **M1**, **Command**
Alt Mod2, **M2**, **Option**
Shift Mod3, **M3**
Lock Mod4, **M4**
Extended Mod5, **M5**
Button1, **B1 Meta**, **M**
Button2, **B2 Double**
Button3, **B3 Triple**
Button4, **B4 Quadruple**
Button5, **B5**

Where more than one value is listed, separated by commas, the values are equivalent. Most of the modifiers have the obvious X meanings. For example, **Button1** requires that button 1 be depressed when the event occurs. For a binding to match a given event, the modifiers in the event must include all of those specified in the event pattern. An event may also contain additional modifiers not specified in the binding. For example, if button 1 is pressed while the shift and control keys are down, the pattern will match the event, but will not. If no modifiers are specified, then any combination of modifiers may be present in the event.

Meta and **M** refer to whichever of the **M1** through **M5** modifiers is associated with the Meta key(s) on the keyboard (keysyms **Meta_R** and **Meta_L**). If there are no Meta keys, or if they are not associated with any modifiers, then **Meta** and **M** will not match any events. Similarly, the **Alt** modifier refers to whichever modifier is associated with the alt key(s) on the keyboard (keysyms **Alt_L** and **Alt_R**).

The **Double**, **Triple** and **Quadruple** modifiers are a convenience for specifying double mouse clicks and other repeated events. They cause a particular event pattern to be repeated 2, 3 or 4 times, and also place a time and space requirement on the sequence: for a sequence of events to match a **Double**, **Triple** or **Quadruple** pattern, all of the events must occur close together in time and without substantial mouse motion in between. For example, is equivalent to with the extra time and space requirement.

The **Command** and **Option** modifiers are equivalents of **Mod1** resp. **Mod2**, they correspond to Macintosh-specific modifier keys.

The **Extended** modifier is, at present, specific to Windows. It appears on events that are associated with the keys on the "extended keyboard". On a US keyboard, the extended keys include the **Alt** and **Control** keys at the right of the keyboard, the cursor keys in the cluster to the left of the numeric pad, the **NumLock** key, the **Break** key, the **PrintScreen** key, and the **/** and **Enter** keys in the numeric keypad.

Event types

The *type* field may be any of the standard X event types, with a few extra abbreviations. The *type* field will also accept a couple non-standard X event types that were added to better support the Macintosh and Windows platforms. Below is a list of all the valid types; where two names appear together, they are synonyms.

Activate Destroy Map
ButtonPress, **Button Enter MapRequest**
ButtonRelease Expose Motion
Circulate FocusIn MouseWheel

CirculateRequest FocusOut Property
Colormap Gravity Reparent
Configure KeyPress, **Key ResizeRequest**
ConfigureRequest KeyRelease Unmap
Create Leave Visibility
Deactivate

Most of the above events have the same fields and behaviors as events in the X Windowing system. You can find more detailed descriptions of these events in any X window programming book. A couple of the events are extensions to the X event system to support features unique to the Macintosh and Windows platforms. We provide a little more detail on these events here. These include:

Activate, Deactivate
These two events are sent to every sub-window of a toplevel when they change state. In addition to the focus Window, the Macintosh platform and Windows platforms have a notion of an active window (which often has but is not required to have the focus). On the Macintosh, widgets in the active window have a different appearance than widgets in deactive windows. The **Activate** event is sent to all the sub-windows in a toplevel when it changes from being deactive to active. Likewise, the **Deactive** event is sent when the window's state changes from active to deactive. There are no useful percent substitutions you would make when binding to these events.

MouseWheel
Many contemporary mice support a mouse wheel, which is used for scrolling documents without using the scrollbars. By rolling the wheel, the system will generate **MouseWheel** events that the application can use to scroll. The event is routed to the window currently under the mouse pointer. When the event is received you can use the **%D** substitution to get the *delta* field for the event, which is an integer value describing how the mouse wheel has moved. The smallest value for which the system will report is defined by the OS. The sign of the value determines which direction your widget should scroll. Positive values should scroll up and negative values should scroll down.

Horizontal scrolling uses **Shift-MouseWheel** events, with positive **%D** *delta* substitution indicating left scrolling and negative right scrolling. Only Windows and macOS Aqua typically fire **MouseWheel** and **Shift-MouseWheel** events. On X11 vertical scrolling is rather supported through **Button-4** and **Button-5** events, and horizontal scrolling through **Shift-Button-4** and **Shift-Button-5** events. Horizontal scrolling events may fire from many different hardware units such as tilt wheels or touchpads. Horizontal scrolling can also be emulated by holding Shift and scrolling vertically.

KeyPress, KeyRelease
The **KeyPress** and **KeyRelease** events are generated whenever a key is pressed or released. **KeyPress** and **KeyRelease** events are sent to the window which currently has the keyboard focus.

ButtonPress, ButtonRelease, Motion
The **ButtonPress** and **ButtonRelease** events are generated when the user presses or releases a mouse button. **Motion** events are generated whenever the pointer is moved. **ButtonPress**, **ButtonRelease**, and **Motion** events are normally sent to the window

containing the pointer.

When a mouse button is pressed, the window containing the pointer automatically obtains a temporary pointer grab. Subsequent **ButtonPress**, **ButtonRelease**, and **Motion** events will be sent to that window, regardless of which window contains the pointer, until all buttons have been released.

Configure

A **Configure** event is sent to a window whenever its size, position, or border width changes, and sometimes when it has changed position in the stacking order.

Map, Unmap

The **Map** and **Unmap** events are generated whenever the mapping state of a window changes.

Windows are created in the unmapped state. Top-level windows become mapped when they transition to the **normal** state, and are unmapped in the **withdrawn** and **iconic** states. Other windows become mapped when they are placed under control of a geometry manager (for example pack or grid).

A window is *viewable* only if it and all of its ancestors are mapped. Note that geometry managers typically do not map their children until they have been mapped themselves, and unmap all children when they become unmapped; hence in Tk **Map** and **Unmap** events indicate whether or not a window is viewable.

Visibility

A window is said to be *obscured* when another window above it in the stacking order fully or partially overlaps it. **Visibility** events are generated whenever a window's obscurity state changes; the *state* field (**%s**) specifies the new state.

Expose

An **Expose** event is generated whenever all or part of a window should be redrawn (for example, when a window is first mapped or if it becomes unobscured). It is normally not necessary for client applications to handle **Expose** events, since Tk handles them internally.

Destroy

A **Destroy** event is delivered to a window when it is destroyed.

When the **Destroy** event is delivered to a widget, it is in a "half-dead" state: the widget still exists, but operations that involve it may return invalid results, or return an error.

FocusIn, FocusOut

The **FocusIn** and **FocusOut** events are generated whenever the keyboard focus changes. A **FocusOut** event is sent to the old focus window, and a **FocusIn** event is sent to the new one.

In addition, if the old and new focus windows do not share a common parent, "virtual crossing" focus events are sent to the intermediate windows in the hierarchy. Thus a **FocusIn** event indicates that the target window or one of its descendants has acquired the focus, and a **FocusOut** event indicates that the focus has been changed to a window outside the target window's hierarchy.

The keyboard focus may be changed explicitly by a call to focus, or implicitly by the window manager.

Enter, Leave

An **Enter** event is sent to a window when the pointer enters that window, and a **Leave** event is sent when the pointer leaves it.

If there is a pointer grab in effect, **Enter** and **Leave** events are only delivered to the window owning the grab.

In addition, when the pointer moves between two windows, **Enter** and **Leave** "virtual crossing" events are sent to intermediate windows in the hierarchy in the same manner as for **FocusIn** and **FocusOut** events.

Property

A **Property** event is sent to a window whenever an X property belonging to that window is changed or deleted. **Property** events are not normally delivered to Tk applications as they are handled by the Tk core.

Colormap

A **Colormap** event is generated whenever the colormap associated with a window has been changed, installed, or uninstalled.

Widgets may be assigned a private colormap by specifying a **-colormap** option; the window manager is responsible for installing and uninstalling colormaps as necessary.

Note that Tk provides no useful details for this event type.

MapRequest, CirculateRequest, ResizeRequest, ConfigureRequest, Create

These events are not normally delivered to Tk applications. They are included for completeness, to make it possible to write X11 window managers in Tk. (These events are only delivered when a client has selected **SubstructureRedirectMask** on a window; the Tk core does not use this mask.)

Gravity, Reparent, Circulate

The events **Gravity** and **Reparent** are not normally delivered to Tk applications. They are included for completeness.

A **Circulate** event indicates that the window has moved to the top or to the bottom of the stacking order as a result of an **XCirculateSubwindows** protocol request. Note that the stacking order may be changed for other reasons which do not generate a **Circulate** event, and that Tk does not use **XCirculateSubwindows()** internally. This event type is included only for completeness; there is no reliable way to track changes to a window's position in the stacking order.

Event details

The last part of a long event specification is *detail*. In the case of a **ButtonPress** or **ButtonRelease** event, it is the number of a button (1-5). If a button number is given, then only an event on that particular button will match; if no button number is given, then an event on any button will match. Note: giving a specific button number is different than specifying a button modifier; in the first case, it refers to a button being pressed or released, while in the second it

refers to some other button that is already depressed when the matching event occurs. If a button number is given then *type* may be omitted: if will default to **ButtonPress**. For example, the specifier **<1>** is equivalent to.

If the event type is **KeyPress** or **KeyRelease**, then *detail* may be specified in the form of an X keysym. Keysyms are textual specifications for particular keys on the keyboard; they include all the alphanumeric ASCII characters (e.g. "a" is the keysym for the ASCII character "a"), plus descriptions for non-alphanumeric characters ("comma"is the keysym for the comma character), plus descriptions for all the non-ASCII keys on the keyboard (e.g. "Shift_L" is the keysym for the left shift key, and "F1" is the keysym for the F1 function key, if it exists). The complete list of keysyms is not presented here; it is available in other X documentation and may vary from system to system. If necessary, you can use the **%K** notation described below to print out the keysym name for a particular key. If a keysym *detail* is given, then the *type* field may be omitted; it will default to **KeyPress**. For example, is equivalent to.

Binding scripts and substitutions

The *script* argument to **bind** is a Tcl script, called the "binding script", which will be executed whenever the given event sequence occurs. *Command* will be executed in the same interpreter that the **bind** command was executed in, and it will run at global level (only global variables will be accessible). If *script* contains any **%** characters, then the script will not be executed directly. Instead, a new script will be generated by replacing each **%**, and the character following it, with information from the current event. The replacement depends on the character following the **%**, as defined in the list below. Unless otherwise indicated, the replacement string is the decimal value of the given field from the current event. Some of the substitutions are only valid for certain types of events; if they are used for other types of events the value substituted is undefined.

%%
Replaced with a single percent.

%#
The number of the last client request processed by the server (the *serial* field from the event). Valid for all event types.

%a
The *above* field from the event, formatted as a hexadecimal number. Valid only for **Configure** events. Indicates the sibling window immediately below the receiving window in the stacking order, or **0** if the receiving window is at the bottom.

%b
The number of the button that was pressed or released. Valid only for **ButtonPress** and **ButtonRelease** events.

%c
The *count* field from the event. Valid only for **Expose** events. Indicates that there are *count* pending **Expose** events which have not yet been delivered to the window.

%d
The *detail* or *user_data* field from the event. The **%d** is replaced by a string identifying the detail. For **Enter**, **Leave**, **FocusIn**, and **FocusOut** events, the string will be one of the following:

NotifyAncestor NotifyNonlinearVirtual

NotifyDetailNone NotifyPointer
NotifyInferior NotifyPointerRoot
NotifyNonlinear NotifyVirtual

For **ConfigureRequest** events, the string will be one of:

Above Opposite
Below None
BottomIf TopIf

For virtual events, the string will be whatever value is stored in the *user_data* field when the event was created (typically with event generate), or the empty string if the field is NULL. Virtual events corresponding to key sequence presses (see event add for details) set the *user_data* to NULL. For events other than these, the substituted string is undefined.

%f
The *focus* field from the event (**0** or **1**). Valid only for **Enter** and **Leave** events. **1** if the receiving window is the focus window or a descendant of the focus window, **0** otherwise.

%h
The *height* field from the event. Valid for the **Configure**, **ConfigureRequest**, **Create**, **ResizeRequest**, and **Expose** events. Indicates the new or requested height of the window.

%i
The *window* field from the event, represented as a hexadecimal integer. Valid for all event types.

%k
The *keycode* field from the event. Valid only for **KeyPress** and **KeyRelease** events.

%m
The *mode* field from the event. The substituted string is one of **NotifyNormal**, **NotifyGrab**, **NotifyUngrab**, or **NotifyWhileGrabbed**. Valid only for **Enter**, **FocusIn**, **FocusOut**, and **Leave** events.

%o
The *override_redirect* field from the event. Valid only for **Map**, **Reparent**, and **Configure** events.

%p
The *place* field from the event, substituted as one of the strings **PlaceOnTop** or **PlaceOnBottom**. Valid only for **Circulate** and **CirculateRequest** events.

%s
The *state* field from the event. For **ButtonPress**, **ButtonRelease**, **Enter**, **KeyPress**, **KeyRelease**, **Leave**, and **Motion** events, a decimal string is substituted. For **Visibility**, one of the strings **VisibilityUnobscured**, **VisibilityPartiallyObscured**, and **VisibilityFullyObscured** is substituted. For **Property** events, substituted with either the string **NewValue** (indicating that the property has been created or modified) or **Delete** (indicating that the property has been removed).

%t
The *time* field from the event. This is the X server timestamp (typically the time since the last server reset) in milliseconds, when the event occurred. Valid for most events.

%w

The *width* field from the event. Indicates the new or requested width of the window. Valid only for **Configure**, **ConfigureRequest**, **Create**, **ResizeRequest**, and **Expose** events.

%x, %y

The *x* and *y* fields from the event. For **ButtonPress**, **ButtonRelease**, **Motion**, **KeyPress**, **KeyRelease**, and **MouseWheel** events, **%x** and **%y** indicate the position of the mouse pointer relative to the receiving window. For key events on the Macintosh these are the coordinates of the mouse at the moment when an X11 KeyEvent is sent to Tk, which could be slightly later than the time of the physical press or release. For **Enter** and **Leave** events, the position where the mouse pointer crossed the window, relative to the receiving window. For **Configure** and **Create** requests, the *x* and *y* coordinates of the window relative to its parent window.

%A

Substitutes the UNICODE character corresponding to the event, or the empty string if the event does not correspond to a UNICODE character (e.g. the shift key was pressed). On X11, **XmbLookupString** (or **XLookupString** when input method support is turned off) does all the work of translating from the event to a UNICODE character. On X11, valid only for **KeyPress** event. On Windows and macOS/aqua, valid only for **KeyPress** and **KeyRelease** events.

%B

The *border_width* field from the event. Valid only for **Configure**, **ConfigureRequest**, and **Create** events.

%D

This reports the *delta* value of a **MouseWheel** event. The *delta* value represents the rotation units the mouse wheel has been moved. The sign of the value represents the direction the mouse wheel was scrolled.

%E

The *send_event* field from the event. Valid for all event types. **0** indicates that this is a "normal" event, **1** indicates that it is a "synthetic" event generated by **SendEvent**.

%K

The keysym corresponding to the event, substituted as a textual string. Valid only for **KeyPress** and **KeyRelease** events.

%M

The number of script-based binding patterns matched so far for the event. Valid for all event types.

%N

The keysym corresponding to the event, substituted as a decimal number. Valid only for **KeyPress** and **KeyRelease** events.

%P

The name of the property being updated or deleted (which may be converted to an XAtom using winfo atom.) Valid only for **Property** events.

%R

The *root* window identifier from the event. Valid only for events containing a *root* field.

%S

The *subwindow* window identifier from the event, formatted as a hexadecimal number. Valid only for events containing a *subwindow* field.

%T
The *type* field from the event. Valid for all event types.

%W
The path name of the window to which the event was reported (the *window* field from the event). Valid for all event types.

%X, %Y
The *x_root* and *y_root* fields from the event. If a virtual-root window manager is being used then the substituted values are the corresponding x-coordinate and y-coordinate in the virtual root. Valid only for **ButtonPress**, **ButtonRelease**, **Enter**, **KeyPress**, **KeyRelease**, **Leave** and **Motion** events. Same meaning as **%x** and **%y**, except relative to the (virtual) root window.

The replacement string for a %-replacement is formatted as a proper Tcl list element. This means that spaces or special characters such as **$** and **{** may be preceded by backslashes. This guarantees that the string will be passed through the Tcl parser when the binding script is evaluated. Most replacements are numbers or well-defined strings such as **Above**; for these replacements no special formatting is ever necessary. The most common case where reformatting occurs is for the **%A** substitution. For example, if *script* is

```
insert %A
```

and the character typed is an open square bracket, then the script actually executed will be

```
insert \[
```

This will cause the **insert** to receive the original replacement string (open square bracket) as its first argument. If the extra backslash had not been added, Tcl would not have been able to parse the script correctly.

Multiple matches

It is possible for several bindings to match a given X event. If the bindings are associated with different *tag*'s, then each of the bindings will be executed, in order. By default, a binding for the widget will be executed first, followed by a class binding, a binding for its toplevel, and an **all** binding. The bindtags command may be used to change this order for a particular window or to associate additional binding tags with the window.

The continue and break commands may be used inside a binding script to control the processing of matching scripts. If continue is invoked within a binding script, then this binding script, including all other "+" appended scripts, is terminated but Tk will continue processing binding scripts associated with other *tag*'s. If the break command is invoked within a binding script, then that script terminates and no other scripts will be invoked for the event.

Within a script called from the binding script, return **-code ok** may be used to continue processing (including "+" appended scripts), or return **-code break** may be used to stop processing all other binding scripts.

If more than one binding matches a particular event and they have the same *tag*, then the most specific binding is chosen and its script is evaluated. The following tests are applied, in order, to determine which of several matching sequences is more specific:

(a)

an event pattern that specifies a specific button or key is more specific than one that does not;

(b)
a longer sequence (in terms of number of events matched) is more specific than a shorter sequence;

(c)
if the modifiers specified in one pattern are a subset of the modifiers in another pattern, then the pattern with more modifiers is more specific;

(d)
a virtual event whose physical pattern matches the sequence is less specific than the same physical pattern that is not associated with a virtual event;

(e)
given a sequence that matches two or more virtual events, one of the virtual events will be chosen, but the order is undefined.

If the matching sequences contain more than one event, then tests (c)-(e) are applied in order from the most recent event to the least recent event in the sequences. If these tests fail to determine a winner, then the most recently registered sequence is the winner.

If there are two (or more) virtual events that are both triggered by the same sequence, and both of those virtual events are bound to the same window tag, then only one of the virtual events will be triggered, and it will be picked at random:

```
event add <<Paste>> <Control-y>
event add <<Paste>> <Button-2>
event add <<Scroll>> <Button-2>
bind Entry <<Paste>> {puts Paste}
bind Entry <<Scroll>> {puts Scroll}
```

If the user types Control-y, the **<>** binding will be invoked, but if the user presses button 2 then one of either the **<>** or the **<>** bindings will be invoked, but exactly which one gets invoked is undefined.

If an X event does not match any of the existing bindings, then the event is ignored. An unbound event is not considered to be an error.

Multi-event sequences and ignored events

When a *sequence* specified in a **bind** command contains more than one event pattern, then its script is executed whenever the recent events (leading up to and including the current event) match the given sequence. This means, for example, that if button 1 is clicked repeatedly the sequence will match each button press but the first. If extraneous events that would prevent a match occur in the middle of an event sequence then the extraneous events are ignored unless they are **KeyPress** or **ButtonPress** events. For example, will match a sequence of presses of button 1, even though there will be **ButtonRelease** events (and possibly **Motion** events) between the **ButtonPress** events. Furthermore, a **KeyPress** event may be preceded by any number of other **KeyPress** events for modifier keys without the modifier keys preventing a match. For example, the event sequence **aB** will match a press of the **a** key, a release of the **a** key, a press of the **Shift** key, and a press of the **b** key: the press of **Shift** is ignored because it is a modifier key. Finally, if several **Motion** events occur in a row, only the last one is used for

purposes of matching binding sequences.

Errors
If an error occurs in executing the script for a binding then the bgerror mechanism is used to report the error. The bgerror command will be executed at global level (outside the context of any Tcl procedure).

Examples
Arrange for a string describing the motion of the mouse to be printed out when the mouse is double-clicked:

```
bind . <Double-1> {
    puts "hi from (%x,%y)"
}
```

A little GUI that displays what the keysym name of the last key pressed is:

```
set keysym "Press any key"
pack [label .l -textvariable keysym -padx 2m -pady 1m]
bind . <Key> {
    set keysym "You pressed %K"
}
```

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 1998 by Scriptics Corporation.

bindtags
Keywords: bindtags, binding, event, tag

Name
bindtags - Determine which bindings apply to a window, and order of evaluation

Synopsis
bindtags *window* ?*tagList*?

Description
When a binding is created with the bind command, it is associated either with a particular window such as **.a.b.c**, a class name such as **Button**, the keyword **all**, or any other string. All of these forms are called *binding tags*. Each window contains a list of binding tags that determine how events are processed for the window. When an event occurs in a window, it is applied to each of the window's tags in order: for each tag, the most specific binding that matches the given tag and event is executed. See the bind command for more information on the matching process.

By default, each window has four binding tags consisting of the name of the window, the window's class name, the name of the window's nearest toplevel ancestor, and **all**, in that order. Toplevel windows have only three tags by default, since the toplevel name is the same as that of the window. The **bindtags** command allows the binding tags for a window to be read and

modified.

If **bindtags** is invoked with only one argument, then the current set of binding tags for *window* is returned as a list. If the *tagList* argument is specified to **bindtags**, then it must be a proper list; the tags for *window* are changed to the elements of the list. The elements of *tagList* may be arbitrary strings; however, any tag starting with a dot is treated as the name of a window; if no window by that name exists at the time an event is processed, then the tag is ignored for that event. The order of the elements in *tagList* determines the order in which binding scripts are executed in response to events. For example, the command

```
bindtags .b {all . Button .b}
```

reverses the order in which binding scripts will be evaluated for a button named **.b** so that **all** bindings are invoked first, following by bindings for **.b**'s toplevel ("."), followed by class bindings, followed by bindings for **.b**. If *tagList* is an empty list then the binding tags for *window* are returned to the default state described above.

The **bindtags** command may be used to introduce arbitrary additional binding tags for a window, or to remove standard tags. For example, the command

```
bindtags .b {.b TrickyButton . all}
```

replaces the **Button** tag for **.b** with **TrickyButton**. This means that the default widget bindings for buttons, which are associated with the **Button** tag, will no longer apply to **.b**, but any bindings associated with **TrickyButton** (perhaps some new button behavior) will apply.

Example

If you have a set of nested frame widgets and you want events sent to a button widget to also be delivered to all the widgets up to the current toplevel (in contrast to Tk's default behavior, where events are not delivered to those intermediate windows) to make it easier to have accelerators that are only active for part of a window, you could use a helper procedure like this to help set things up:

```
proc setupBindtagsForTreeDelivery {widget} {
    set tags [list $widget [winfo class $widget]]
    set w $widget
    set t [winfo toplevel $w]
    while {$w ne $t} {
        set w [winfo parent $w]
        lappend tags $w
    }
    lappend tags all
    bindtags $widget $tags
}
```

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

bitmap

Keywords: bitmap, image

Name

bitmap - Images that display two colors

Synopsis

image create bitmap ?*name*? ?*options*?

imageName **cget** *option*
imageName **configure** ?*option*? ?*value option value* ...?

Description

A bitmap is an image whose pixels can display either of two colors or be transparent. A bitmap image is defined by four things: a background color, a foreground color, and two bitmaps, called the *source* and the *mask*. Each of the bitmaps specifies 0/1 values for a rectangular array of pixels, and the two bitmaps must have the same dimensions. For pixels where the mask is zero, the image displays nothing, producing a transparent effect. For other pixels, the image displays the foreground color if the source data is one and the background color if the source data is zero.

Creating bitmaps

Like all images, bitmaps are created using the image create command. Bitmaps support the following *options*:

-background *color*
: Specifies a background color for the image in any of the standard ways accepted by Tk. If this option is set to an empty string then the background pixels will be transparent. This effect is achieved by using the source bitmap as the mask bitmap, ignoring any **-maskdata** or **-maskfile** options.

-data *string*
: Specifies the contents of the source bitmap as a string. The string must adhere to X11 bitmap format (e.g., as generated by the **bitmap** program). If both the **-data** and **-file** options are specified, the **-data** option takes precedence.

-file *name*
: *name* gives the name of a file whose contents define the source bitmap. The file must adhere to X11 bitmap format (e.g., as generated by the **bitmap** program).

-foreground *color*
: Specifies a foreground color for the image in any of the standard ways accepted by Tk.

-maskdata *string*
: Specifies the contents of the mask as a string. The string must adhere to X11 bitmap format (e.g., as generated by the **bitmap** program). If both the **-maskdata** and **-maskfile** options are specified, the **-maskdata** option takes precedence.

-maskfile *name*
: *name* gives the name of a file whose contents define the mask. The file must adhere to X11 bitmap format (e.g., as generated by the **bitmap** program).

Image command

When a bitmap image is created, Tk also creates a new command whose name is the same as the image. This command may be used to invoke various operations on the image. It has the following general form:

```
imageName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following commands are possible for bitmap images:

imageName* cget *option
: Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the image create **bitmap** command.

***imageName* configure ?*option*? ?*value option value ...*?**
: Query or modify the configuration options for the image. If no *option* is specified, returns a list describing all of the available options for *imageName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the image create **bitmap** command.

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

busy

Keywords: busy, keyboard events, pointer events, window

Name

busy - Make Tk widgets busy, temporarily blocking user interactions

Synopsis

tk busy *window* ?*options*?
tk busy hold *window* ?*options*?
tk busy configure *window* ?*option value* ...?
tk busy forget *window* ?*window* ...?
tk busy current ?*pattern*?
tk busy status *window*

Description

The tk busy command provides a simple means to block mouse pointer events from Tk widgets, while overriding the widget's cursor with a configurable busy cursor. Note this command does not prevent keyboard events from being sent to the widgets made busy.

Introduction

There are many times in applications where you want to temporarily restrict what actions the user can take. For example, an application could have a "Run" button that when pressed causes some processing to occur. However, while the application is busy processing, you probably don't want the user to be able to click the "Run" button again. You may also want restrict the user from other tasks such as clicking a "Print" button.

The tk busy command lets you make Tk widgets busy. This means that user interactions such as button clicks, moving the mouse, typing at the keyboard, etc. are ignored by the widget. You can

set a special cursor (like a watch) that overrides the widget's normal cursor, providing feedback that the application (widget) is temporarily busy.

When a widget is made busy, the widget and all of its descendants will ignore pointer events. It's easy to make an entire panel of widgets busy. You can simply make the toplevel widget (such as ".") busy. This is easier and far much more efficient than recursively traversing the widget hierarchy, disabling each widget and re-configuring its cursor.

Often, the tk busy command can be used instead of Tk's grab command. Unlike grab which restricts all user interactions to one widget, with the tk busy command you can have more than one widget active (for example, a "Cancel" dialog and a "Help" button).

Example

You can make several widgets busy by simply making its ancestor widget busy using the **hold** operation.

```
frame .top
button .top.button; canvas .top.canvas
pack .top.button .top.canvas
pack .top
# . . .
tk busy hold .top
update
```

All the widgets within **.top** (including **.top**) are now busy. Using update insures that tk busy command will take effect before any other user events can occur.

When the application is no longer busy processing, you can allow user interactions again and free any resources it allocated by the **forget** operation.

```
tk busy forget .top
```

The busy window has a configurable cursor. You can change the busy cursor using the **configure** operation.

```
tk busy configure .top -cursor "watch"
```

Destroying the widget will also clean up any resources allocated by the tk busy command.

Operations

The following operations are available for the tk busy command:

tk busy *window* ?*option value*?...
Shortcut for **tk busy hold** command.

tk busy cget *window option*
Queries the tk busy command configuration options for *window*. *Window* must be the path name of a widget previously made busy by the **hold** operation. The command returns the present value of the specified *option*. *Option* may have any of the values accepted by the **hold** operation.

tk busy configure *window* ?*option value* ...?
Queries or modifies the tk busy command configuration options for *window*. *Window* must be the path name of a widget previously made busy by the **hold** operation. If no options are

specified, a list describing all of the available options for *window* (see Tk_ConfigureInfo for information on the format of this list) is returned. If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns the empty string. *Option* may have any of the values accepted by the **hold** operation.

Please note that the option database is referenced through *window*. For example, if the widget **.frame** is to be made busy, the busy cursor can be specified for it by either option command:

```
option add *frame.busyCursor gumby
option add *Frame.BusyCursor gumby
```

tk busy current ?*pattern*?
Returns the pathnames of all widgets that are currently busy. If a *pattern* is given, only the path names of busy widgets matching *pattern* are returned.

tk busy forget *window* ?*window?...*?
Releases resources allocated by the tk busy command for *window*, including the transparent window. User events will again be received by *window*. Resources are also released when *window* is destroyed. *Window* must be the name of a widget specified in the **hold** operation, otherwise an error is reported.

tk busy hold *window* ?*option value* ...?
Makes the specified *window* (and its descendants in the Tk window hierarchy) appear busy. *Window* must be a valid path name of a Tk widget. A transparent window is put in front of the specified window. This transparent window is mapped the next time idle tasks are processed, and the specified window and its descendants will be blocked from user interactions. Normally update should be called immediately afterward to insure that the hold operation is in effect before the application starts its processing. The following configuration options are valid:

-cursor *cursorName*
Specifies the cursor to be displayed when the widget is made busy. *CursorName* can be in any form accepted by Tk_GetCursor. The default cursor is **wait** on Windows and **watch** on other platforms.

tk busy status *window*
Returns the status of a widget *window*. If *window* presently can not receive user interactions, **1** is returned, otherwise **0**.

Event handling

Bindings

The event blocking feature is implemented by creating and mapping a transparent window that completely covers the widget. When the busy window is mapped, it invisibly shields the widget and its hierarchy from all events that may be sent. Like Tk widgets, busy windows have widget names in the Tk window hierarchy. This means that you can use the bind command to handle events in the busy window:

```
tk busy hold .frame.canvas
bind .frame.canvas_Busy <Enter> { ... }
```

Normally the busy window is a sibling of the widget. The name of the busy window is "*widget* **_Busy**" where *widget* is the name of the widget to be made busy. In the previous example, the pathname of the busy window is "**.frame.canvas_Busy**". The exception is when the widget is a toplevel widget (such as ".") where the busy window can't be made a sibling. The busy window is then a child of the widget named "*widget***._Busy**" where *widget* is the name of the toplevel widget. In the following example, the pathname of the busy window is "**._Busy**".

```
tk busy hold .
bind ._Busy <Enter> { ... }
```

Enter/Leave events

Mapping and unmapping busy windows generates Enter/Leave events for all widgets they cover. Please note this if you are tracking Enter/Leave events in widgets.

Keyboard events

When a widget is made busy, the widget is prevented from gaining the keyboard focus by a user clicking on it by the busy window. But if the widget already had focus, it still may receive keyboard events. The widget can also still receive focus through keyboard traversal. To prevent this, you must move focus to another window and make sure the focus can not go back to the widgets made busy (e.g. but restricting focus to a cancel button).

```
pack [frame .frame]
pack [text .frame.text]
tk busy hold .frame
pack [button .cancel -text "Cancel" -command exit]
focus .cancel
bind .cancel <Tab> {break}
bind .cancel <Shift-Tab> {break}
update
```

The above example moves the focus from .frame immediately after invoking the **hold** so that no keyboard events will be sent to **.frame** or any of its descendants. It also makes sure it's not possible to leave button **.cancel** using the keyboard.

Portability

Note that the tk busy command does not currently have any effect on macOS when Tk is built using Aqua support.

Copyright (c) 1993-1998 Lucent Technologies, Inc.
Copyright (c) 2008 Jos Decoster

button

Keywords: button, widget

Name

button - Create and manipulate 'button' action widgets

Synopsis

button *pathName* ?*options*?

Standard options

-activebackground, **-font**, **-relief**, **-activeforeground**, **-foreground**, **-repeatdelay**, **-anchor**, **-highlightbackground**, **-repeatinterval**, **-background**, **-highlightcolor**, **-takefocus**, **-bitmap**, **-highlightthickness**, **-text**, **-borderwidth**, **-image**, **-textvariable**, **-compound**, **-justify**, **-underline**, **-cursor**, **-padx**, **-wraplength**, **-disabledforeground**, **-pady**

See the options manual entry for details on the standard options.

Widget-specific options

-command (command/Command)
Specifies a Tcl command to associate with the button. This command is typically invoked when mouse button 1 is released over the button window.

-default (default/Default)
Specifies one of three states for the default ring: **normal**, **active**, or **disabled**. In active state, the button is drawn with the platform specific appearance for a default button. In normal state, the button is drawn with the platform specific appearance for a non-default button, leaving enough space to draw the default button appearance. The normal and active states will result in buttons of the same size. In disabled state, the button is drawn with the non-default button appearance without leaving space for the default appearance. The disabled state may result in a smaller button than the active state.

-height (height/Height)
Specifies a desired height for the button. If an image or bitmap is being displayed in the button then the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels); for text it is in lines of text. If this option is not specified, the button's desired height is computed from the size of the image or bitmap or text being displayed in it.

-overrelief (overRelief/OverRelief)
Specifies an alternative relief for the button, to be used when the mouse cursor is over the widget. This option can be used to make toolbar buttons, by configuring **-relief flat -overrelief raised**. If the value of this option is the empty string, then no alternative relief is used when the mouse cursor is over the button. The empty string is the default value.

-state (state/State)
Specifies one of three states for the button: **normal**, **active**, or **disabled**. In normal state the button is displayed using the **-foreground** and **-background** options. The active state is typically used when the pointer is over the button. In active state the button is displayed using the **-activeforeground** and **-activebackground** options. Disabled state means that the button should be insensitive: the default bindings will refuse to activate the widget and will ignore mouse button presses. In this state the **-disabledforeground** and **-background** options determine how the button is displayed.

-width (width/Width)
Specifies a desired width for the button. If an image or bitmap is being displayed in the button then the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels). For a text button (no image or with **-compound none**) then the width specifies how much space in

characters to allocate for the text label. If the width is negative then this specifies a minimum width. If this option is not specified, the button's desired width is computed from the size of the image or bitmap or text being displayed in it.

Description

The **button** command creates a new window (given by the *pathName* argument) and makes it into a button widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the button such as its colors, font, text, and initial relief. The **button** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A button is a widget that displays a textual string, bitmap or image. If text is displayed, it must all be in a single font, but it can occupy multiple lines on the screen (if it contains newlines or if wrapping occurs because of the **-wraplength** option) and one of the characters may optionally be underlined using the **-underline** option. It can display itself in either of three different ways, according to the **-state** option; it can be made to appear raised, sunken, or flat; and it can be made to flash. When a user invokes the button (by pressing mouse button 1 with the cursor over the button), then the Tcl command specified in the **-command** option is invoked.

Widget command

The **button** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following commands are possible for button widgets:

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **button** command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **button** command.

***pathName* flash**
Flash the button. This is accomplished by redisplaying the button several times, alternating between the configured activebackground and background colors. At the end of the flash the button is left in the same normal/active state as when the command was invoked. This command is ignored if the button's state is **disabled**.

***pathName* invoke**
Invoke the Tcl command associated with the button, if there is one. The return value is the return value from the Tcl command, or an empty string if there is no command associated

with the button. This command is ignored if the button's state is **disabled**.

Default bindings

Tk automatically creates class bindings for buttons that give them default behavior:

1. A button activates whenever the mouse passes over it and deactivates whenever the mouse leaves the button. Under Windows, this binding is only active when mouse button 1 has been pressed over the button.
2. A button's relief is changed to sunken whenever mouse button 1 is pressed over the button, and the relief is restored to its original value when button 1 is later released.
3. If mouse button 1 is pressed over a button and later released over the button, the button is invoked. However, if the mouse is not over the button when button 1 is released, then no invocation occurs.
4. When a button has the input focus, the space key causes the button to be invoked.

If the button's state is **disabled** then none of the above actions occur: the button is completely non-responsive.

The behavior of buttons can be changed by defining new bindings for individual widgets or by redefining the class bindings.

Platform notes

On Aqua/macOS, some configuration options are ignored for the purpose of drawing of the widget because they would otherwise conflict with platform guidelines. The **configure** and **cget** subcommands can still manipulate the values, but do not cause any variation to the look of the widget. The options affected notably include **-background** and **-relief**.

Examples

This is the classic Tk "Hello, World!" demonstration:

```
button .b -text "Hello, World!" -command exit
pack .b
```

This example demonstrates how to handle button accelerators:

```
button .b1 -text Hello -underline 0
button .b2 -text World -underline 0
bind . <Key-h> {.b1 flash; .b1 invoke}
bind . <Key-w> {.b2 flash; .b2 invoke}
pack .b1 .b2
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

canvas

Keywords: canvas, widget

Name

canvas - Create and manipulate 'canvas' hypergraphics drawing surface widgets

Synopsis

canvas *pathName* ?*options*?

Standard options

-background, **-borderwidth**, **-cursor**, **-highlightbackground**, **-highlightcolor**, **-highlightthickness**, **-insertbackground**, **-insertborderwidth**, **-insertofftime**, **-insertontime**, **-insertwidth**, **-relief**, **-selectbackground**, **-selectborderwidth**, **-selectforeground**, **-takefocus**, **-xscrollcommand**, **-yscrollcommand**

See the options manual entry for details on the standard options.

Widget-specific options

-closeenough (closeEnough/CloseEnough)
Specifies a floating-point value indicating how close the mouse cursor must be to an item before it is considered to be "inside" the item. Defaults to 1.0.

-confine (confine/Confine)
Specifies a boolean value that indicates whether or not it should be allowable to set the canvas's view outside the region defined by the **scrollRegion** argument. Defaults to true, which means that the view will be constrained within the scroll region.

-height (height/Height)
Specifies a desired window height that the canvas widget should request from its geometry manager. The value may be specified in any of the forms described in the Coordinates section below.

-scrollregion (scrollRegion/ScrollRegion)
Specifies a list with four coordinates describing the left, top, right, and bottom coordinates of a rectangular region. This region is used for scrolling purposes and is considered to be the boundary of the information in the canvas. Each of the coordinates may be specified in any of the forms given in the Coordinates section below.

-state (state/State)
Modifies the default state of the canvas where *state* may be set to one of: **normal**, **disabled**, or **hidden**. Individual canvas objects all have their own state option which may override the default state. Many options can take separate specifications such that the appearance of the item can be different in different situations. The options that start with **active** control the appearance when the mouse pointer is over it, while the option starting with **disabled** controls the appearance when the state is disabled. Canvas items which are **disabled** will not react to canvas bindings.

-width (width/width)
Specifies a desired window width that the canvas widget should request from its geometry manager. The value may be specified in any of the forms described in the Coordinates section below.

-xscrollincrement (xScrollIncrement/ScrollIncrement)
Specifies an increment for horizontal scrolling, in any of the usual forms permitted for screen distances. If the value of this option is greater than zero, the horizontal view in the window will be constrained so that the canvas x coordinate at the left edge of the window is always an even multiple of **xScrollIncrement**; furthermore, the units for scrolling (e.g., the change in view when the left and right arrows of a scrollbar are selected) will also be **xScrollIncrement**

. If the value of this option is less than or equal to zero, then horizontal scrolling is unconstrained.

-yscrollincrement (yScrollIncrement/ScrollIncrement)
Specifies an increment for vertical scrolling, in any of the usual forms permitted for screen distances. If the value of this option is greater than zero, the vertical view in the window will be constrained so that the canvas y coordinate at the top edge of the window is always an even multiple of **yScrollIncrement**; furthermore, the units for scrolling (e.g., the change in view when the top and bottom arrows of a scrollbar are selected) will also be **yScrollIncrement**. If the value of this option is less than or equal to zero, then vertical scrolling is unconstrained.

Introduction

The **canvas** command creates a new window (given by the *pathName* argument) and makes it into a canvas widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the canvas such as its colors and 3-D relief. The **canvas** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

Canvas widgets implement structured graphics. A canvas displays any number of *items*, which may be things like rectangles, circles, lines, and text. Items may be manipulated (e.g. moved or re-colored) and commands may be associated with items in much the same way that the bind command allows commands to be bound to widgets. For example, a particular command may be associated with the event so that the command is invoked whenever button 1 is pressed with the mouse cursor over an item. This means that items in a canvas can have behaviors defined by the Tcl scripts bound to them.

Display list

The items in a canvas are ordered for purposes of display, with the first item in the display list being displayed first, followed by the next item in the list, and so on. Items later in the display list obscure those that are earlier in the display list and are sometimes referred to as being "on top" of earlier items. When a new item is created it is placed at the end of the display list, on top of everything else. Widget commands may be used to re-arrange the order of the display list.

Window items are an exception to the above rules. The underlying window systems require them always to be drawn on top of other items. In addition, the stacking order of window items is not affected by any of the canvas widget commands; you must use the Tk raise command and lower command instead.

Item ids and tags

Items in a canvas widget may be named in either of two ways: by id or by tag. Each item has a unique identifying number, which is assigned to that item when it is created. The id of an item never changes and id numbers are never re-used within the lifetime of a canvas widget.

Each item may also have any number of *tags* associated with it. A tag is just a string of characters, and it may take any form except that of an integer. For example, "x123" is OK but "123" is not. The same tag may be associated with many different items. This is commonly done to group items in various interesting ways; for example, all selected items might be given the tag "selected".

The tag **all** is implicitly associated with every item in the canvas; it may be used to invoke operations on all the items in the canvas.

The tag **current** is managed automatically by Tk; it applies to the *current item*, which is the topmost item whose drawn area covers the position of the mouse cursor (different item types interpret this in varying ways; see the individual item type documentation for details). If the mouse is not in the canvas widget or is not over an item, then no item has the **current** tag.

When specifying items in canvas widget commands, if the specifier is an integer then it is assumed to refer to the single item with that id. If the specifier is not an integer, then it is assumed to refer to all of the items in the canvas that have a tag matching the specifier. The symbol *tagOrId* is used below to indicate that an argument specifies either an id that selects a single item or a tag that selects zero or more items.

tagOrId may contain a logical expressions of tags by using operators: "**&&**", "**||**", "**^**", "**!**", and parenthesized subexpressions. For example:

```
      .c find withtag {(a&&!b)||(!a&&b)}
```

or equivalently:

```
      .c find withtag {a^b}
```

will find only those items with either "a" or "b" tags, but not both.

Some widget commands only operate on a single item at a time; if *tagOrId* is specified in a way that names multiple items, then the normal behavior is for the command to use the first (lowest) of these items in the display list that is suitable for the command. Exceptions are noted in the widget command descriptions below.

Coordinates

All coordinates related to canvases are stored as floating-point numbers. Coordinates and distances are specified in screen units, which are floating-point numbers optionally followed by one of several letters. If no letter is supplied then the distance is in pixels. If the letter is **m** then the distance is in millimeters on the screen; if it is **c** then the distance is in centimeters; **i** means inches, and **p** means printers points (1/72 inch). Larger y-coordinates refer to points lower on the screen; larger x-coordinates refer to points farther to the right. Coordinates can be specified either as an even number of parameters, or as a single list parameter containing an even number of x and y coordinate values.

Transformations

Normally the origin of the canvas coordinate system is at the upper-left corner of the window containing the canvas. It is possible to adjust the origin of the canvas coordinate system relative to the origin of the window using the **xview** and **yview** widget commands; this is typically used for scrolling. Canvases do not support scaling or rotation of the canvas coordinate system relative to the window coordinate system.

Individual items may be moved or scaled using widget commands described below, but they may not be rotated.

Note that the default origin of the canvas's visible area is coincident with the origin for the whole window as that makes bindings using the mouse position easier to work with; you only need to use the **canvasx** and **canvasy** widget commands if you adjust the origin of the visible area.

However, this also means that any focus ring (as controlled by the **-highlightthickness** option) and window border (as controlled by the **-borderwidth** option) must be taken into account before you get to the visible area of the canvas.

Indices

Text items support the notion of an *index* for identifying particular positions within the item. In a similar fashion, line and polygon items support *index* for identifying, inserting and deleting subsets of their coordinates. Indices are used for commands such as inserting or deleting a range of characters or coordinates, and setting the insertion cursor position. An index may be specified in any of a number of ways, and different types of items may support different forms for specifying indices. Text items support the following forms for an index; if you define new types of text-like items, it would be advisable to support as many of these forms as practical. Note that it is possible to refer to the character just after the last one in the text item; this is necessary for such tasks as inserting new text at the end of the item. Lines and Polygons do not support the insertion cursor and the selection. Their indices are supposed to be even always, because coordinates always appear in pairs.

number
: A decimal number giving the position of the desired character within the text item. 0 refers to the first character, 1 to the next character, and so on. If indexes are odd for lines and polygons, they will be automatically decremented by one. A number less than 0 is treated as if it were zero, and a number greater than the length of the text item is treated as if it were equal to the length of the text item. For polygons, numbers less than 0 or greater than the length of the coordinate list will be adjusted by adding or subtracting the length until the result is between zero and the length, inclusive.

end
: Refers to the character or coordinate just after the last one in the item (same as the number of characters or coordinates in the item).

insert
: Refers to the character just before which the insertion cursor is drawn in this item. Not valid for lines and polygons.

sel.first
: Refers to the first selected character in the item. If the selection is not in this item then this form is illegal.

sel.last
: Refers to the last selected character in the item. If the selection is not in this item then this form is illegal.

@x,y
: Refers to the character or coordinate at the point given by *x* and *y*, where *x* and *y* are specified in the coordinate system of the canvas. If *x* and *y* lie outside the coordinates covered by the text item, then they refer to the first or last character in the line that is closest to the given point.

Dash patterns

Many items support the notion of a dash pattern for outlines.

The first possible syntax is a list of integers. Each element represents the number of pixels of a

line segment. Only the odd segments are drawn using the "outline" color. The other segments are drawn transparent.

The second possible syntax is a character list containing only 5 possible characters "**.,-_** ". The space can be used to enlarge the space between other line elements, and cannot occur as the first position in the string. Some examples:

```
-dash .     → -dash {2 4}
-dash -     → -dash {6 4}
-dash -.    → -dash {6 4 2 4}
-dash -..   → -dash {6 4 2 4 2 4}
-dash {. }  → -dash {2 8}
-dash ,     → -dash {4 4}
```

The main difference of this syntax with the previous is that it is shape-conserving. This means that all values in the dash list will be multiplied by the line width before display. This assures that "." will always be displayed as a dot and "-" always as a dash regardless of the line width.

On systems which support only a limited set of dash patterns, the dash pattern will be displayed as the closest dash pattern that is available. For example, on Windows only the first 4 of the above examples are available. The last 2 examples will be displayed identically to the first one.

Widget command

The **canvas** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following widget commands are possible for canvas widgets:

***pathName* addtag *tag searchSpec* ?*arg arg ...*?**

For each item that meets the constraints specified by *searchSpec* and the *arg*s, add *tag* to the list of tags associated with the item if it is not already present on that list. It is possible that no items will satisfy the constraints given by *searchSpec* and *arg*s, in which case the command has no effect. This command returns an empty string as result. *SearchSpec* and *arg* 's may take any of the following forms:

above *tagOrId*

Selects the item just after (above) the one given by *tagOrId* in the display list. If *tagOrId* denotes more than one item, then the last (topmost) of these items in the display list is used.

all

Selects all the items in the canvas.

below *tagOrId*

Selects the item just before (below) the one given by *tagOrId* in the display list. If *tagOrId* denotes more than one item, then the first (lowest) of these items in the display list is used.

closest *x y* ?*halo*? ?*start*?

Selects the item closest to the point given by *x* and *y*. If more than one item is at the same closest distance (e.g. two items overlap the point), then the top-most of these items (the

last one in the display list) is used. If *halo* is specified, then it must be a non-negative value. Any item closer than *halo* to the point is considered to overlap it. The *start* argument may be used to step circularly through all the closest items. If *start* is specified, it names an item using a tag or id (if by tag, it selects the first item in the display list with the given tag). Instead of selecting the topmost closest item, this form will select the topmost closest item that is below *start* in the display list; if no such item exists, then the selection behaves as if the *start* argument had not been specified.

enclosed *x1 y1 x2 y2*
Selects all the items completely enclosed within the rectangular region given by *x1*, *y1*, *x2*, and *y2*. *X1* must be no greater than *x2* and *y1* must be no greater than *y2*.

overlapping *x1 y1 x2 y2*
Selects all the items that overlap or are enclosed within the rectangular region given by *x1*, *y1*, *x2*, and *y2*. *X1* must be no greater than *x2* and *y1* must be no greater than *y2*.

withtag *tagOrId*
Selects all the items given by *tagOrId*.

***pathName* bbox *tagOrId* ?*tagOrId tagOrId ...*?**
Returns a list with four elements giving an approximate bounding box for all the items named by the *tagOrId* arguments. The list has the form "*x1 y1 x2 y2*" such that the drawn areas of all the named elements are within the region bounded by *x1* on the left, *x2* on the right, *y1* on the top, and *y2* on the bottom. The return value may overestimate the actual bounding box by a few pixels. If no items match any of the *tagOrId* arguments or if the matching items have empty bounding boxes (i.e. they have nothing to display) then an empty string is returned.

***pathName* bind *tagOrId* ?*sequence*? ?*command*?**
This command associates *command* with all the items given by *tagOrId* such that whenever the event sequence given by *sequence* occurs for one of the items the command will be invoked. This widget command is similar to the bind command except that it operates on items in a canvas rather than entire widgets. See the bind manual entry for complete details on the syntax of *sequence* and the substitutions performed on *command* before invoking it. If all arguments are specified then a new binding is created, replacing any existing binding for the same *sequence* and *tagOrId* (if the first character of *command* is "+" then *command* augments an existing binding rather than replacing it). In this case the return value is an empty string. If *command* is omitted then the command returns the *command* associated with *tagOrId* and *sequence* (an error occurs if there is no such binding). If both *command* and *sequence* are omitted then the command returns a list of all the sequences for which bindings have been defined for *tagOrId*.

The only events for which bindings may be specified are those related to the mouse and keyboard (such as **Enter**, **Leave**, **ButtonPress**, **Motion**, and **KeyPress**) or virtual events. The handling of events in canvases uses the current item defined in Item ids and tags above. **Enter** and **Leave** events trigger for an item when it becomes the current item or ceases to be the current item; note that these events are different than **Enter** and **Leave** events for windows. Mouse-related events are directed to the current item, if any. Keyboard-related events are directed to the focus item, if any (see the **focus** widget command below for more on this). If a virtual event is used in a binding, that binding can trigger only if the virtual event is defined by an underlying mouse-related or keyboard-related event.

It is possible for multiple bindings to match a particular event. This could occur, for example, if one binding is associated with the item's id and another is associated with one of the item's tags. When this occurs, all of the matching bindings are invoked. A binding associated with the **all** tag is invoked first, followed by one binding for each of the item's tags (in order), followed by a binding associated with the item's id. If there are multiple matching bindings for a single tag, then only the most specific binding is invoked. A continue command in a binding script terminates that script, and a break command terminates that script and skips any remaining scripts for the event, just as for the bind command.

If bindings have been created for a canvas window using the bind command, then they are invoked in addition to bindings created for the canvas's items using the **bind** widget command. The bindings for items will be invoked before any of the bindings for the window as a whole.

***pathName* canvasx *screenx* ?*gridspacing*?**
Given a window x-coordinate in the canvas *screenx*, this command returns the canvas x-coordinate that is displayed at that location. If *gridspacing* is specified, then the canvas coordinate is rounded to the nearest multiple of *gridspacing* units.

***pathName* canvasy *screeny* ?*gridspacing*?**
Given a window y-coordinate in the canvas *screeny* this command returns the canvas y-coordinate that is displayed at that location. If *gridspacing* is specified, then the canvas coordinate is rounded to the nearest multiple of *gridspacing* units.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **canvas** command.

***pathName* configure ?*option*? ?*value*? ?*option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **canvas** command.

***pathName* coords *tagOrId* ?*x0 y0 ...*?**
see below ...

***pathName* coords *tagOrId* ?*coordList*?**
Query or modify the coordinates that define an item. If no coordinates are specified, this command returns a list whose elements are the coordinates of the item named by *tagOrId*. If coordinates are specified, then they replace the current coordinates for the named item. If *tagOrId* refers to multiple items, then the first one in the display list is used.

Note that for rectangles, ovals and arcs the returned list of coordinates has a fixed order, namely the left, top, right and bottom coordinates, which may not be the order originally given. Also the coordinates are always returned in screen units with no units (that is, in

pixels). So if the original coordinates were specified for instance in centimeters or inches, the returned values will nevertheless be in pixels.

pathName* create *type x y ?x y ...? ?option value ...?
see below ...

pathName* create *type coordList ?option value ...?
Create a new item in *pathName* of type *type*. The exact format of the arguments after *type* depends on *type*, but usually they consist of the coordinates for one or more points, followed by specifications for zero or more item options. See the subsections on individual item types below for more on the syntax of this command. This command returns the id for the new item.

pathName* dchars *tagOrId first ?last?
For each item given by *tagOrId*, delete the characters, or coordinates, in the range given by *first* and *last*, inclusive. If some of the items given by *tagOrId* do not support indexing operations then they ignore this operation. Text items interpret *first* and *last* as indices to a character, line and polygon items interpret them as indices to a coordinate (an x,y pair). Indices are described in Indices above. If *last* is omitted, it defaults to *first*. This command returns an empty string.

***pathName* delete ?*tagOrId tagOrId ...*?**
Delete each of the items given by each *tagOrId*, and return an empty string.

pathName* dtag *tagOrId ?tagToDelete?
For each of the items given by *tagOrId*, delete the tag given by *tagToDelete* from the list of those associated with the item. If an item does not have the tag *tagToDelete* then the item is unaffected by the command. If *tagToDelete* is omitted then it defaults to *tagOrId*. This command returns an empty string.

pathName* find *searchCommand ?arg arg ...?
This command returns a list consisting of all the items that meet the constraints specified by *searchCommand* and *arg*'s. *SearchCommand* and *args* have any of the forms accepted by the **addtag** command. The items are returned in stacking order, with the lowest item first.

***pathName* focus ?*tagOrId*?**
Set the keyboard focus for the canvas widget to the item given by *tagOrId*. If *tagOrId* refers to several items, then the focus is set to the first such item in the display list that supports the insertion cursor. If *tagOrId* does not refer to any items, or if none of them support the insertion cursor, then the focus is not changed. If *tagOrId* is an empty string, then the focus item is reset so that no item has the focus. If *tagOrId* is not specified then the command returns the id for the item that currently has the focus, or an empty string if no item has the focus.

Once the focus has been set to an item, the item will display the insertion cursor and all keyboard events will be directed to that item. The focus item within a canvas and the focus window on the screen (set with the focus command) are totally independent: a given item does not actually have the input focus unless (a) its canvas is the focus window and (b) the item is the focus item within the canvas. In most cases it is advisable to follow the **focus** widget command with the focus command to set the focus window to the canvas (if it was not there already).

pathName* gettags *tagOrId

Return a list whose elements are the tags associated with the item given by *tagOrId*. If *tagOrId* refers to more than one item, then the tags are returned from the first such item in the display list. If *tagOrId* does not refer to any items, or if the item contains no tags, then an empty string is returned.

pathName* icursor *tagOrId index

Set the position of the insertion cursor for the item(s) given by *tagOrId* to just before the character whose position is given by *index*. If some or all of the items given by *tagOrId* do not support an insertion cursor then this command has no effect on them. See Indices above for a description of the legal forms for *index*. Note: the insertion cursor is only displayed in an item if that item currently has the keyboard focus (see the **focus** widget command, above), but the cursor position may be set even when the item does not have the focus. This command returns an empty string.

pathName* imove *tagOrId index x y

This command causes the index'th coordinate of each of the items indicated by tagOrId to be relocated to the location (x,y). Each item interprets index independently according to the rules described in Indices above. Out of the standard set of items, only line and polygon items may have their coordinates relocated this way.

pathName* index *tagOrId index

This command returns a decimal string giving the numerical index within *tagOrId* corresponding to *index*. *Index* gives a textual description of the desired position as described in Indices above. Text items interpret *index* as an index to a character, line and polygon items interpret it as an index to a coordinate (an x,y pair). The return value is guaranteed to lie between 0 and the number of characters, or coordinates, within the item, inclusive. If *tagOrId* refers to multiple items, then the index is processed in the first of these items that supports indexing operations (in display list order).

pathName* insert *tagOrId beforeThis string

For each of the items given by *tagOrId*, if the item supports text or coordinate, insertion then *string* is inserted into the item's text just before the character, or coordinate, whose index is *beforeThis*. Text items interpret *beforeThis* as an index to a character, line and polygon items interpret it as an index to a coordinate (an x,y pair). For lines and polygons the *string* must be a valid coordinate sequence. See Indices above for information about the forms allowed for *beforeThis*. This command returns an empty string.

pathName* itemcget *tagOrId option

Returns the current value of the configuration option for the item given by *tagOrId* whose name is *option*. This command is similar to the **cget** widget command except that it applies to a particular item rather than the widget as a whole. *Option* may have any of the values accepted by the **create** widget command when the item was created. If *tagOrId* is a tag that refers to more than one item, the first (lowest) such item is used.

***pathName* itemconfigure *tagOrId* ?*option*? ?*value*? ?*option value ...*?**

This command is similar to the **configure** widget command except that it modifies item-specific options for the items given by *tagOrId* instead of modifying options for the overall canvas widget. If no *option* is specified, returns a list describing all of the available options for the first item given by *tagOrId* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned

if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s) in each of the items given by *tagOrId*; in this case the command returns an empty string. The *option*s and *value*s are the same as those permissible in the **create** widget command when the item(s) were created; see the sections describing individual item types below for details on the legal options.

***pathName* lower *tagOrId* ?*belowThis*?**

Move all of the items given by *tagOrId* to a new position in the display list just before the item given by *belowThis*. If *tagOrId* refers to more than one item then all are moved but the relative order of the moved items will not be changed. *BelowThis* is a tag or id; if it refers to more than one item then the first (lowest) of these items in the display list is used as the destination location for the moved items. Note: this command has no effect on window items. Window items always obscure other item types, and the stacking order of window items is determined by the raise command and lower command, not the **raise** widget command and **lower** widget command for canvases. This command returns an empty string.

pathName* move *tagOrId xAmount yAmount

Move each of the items given by *tagOrId* in the canvas coordinate space by adding *xAmount* to the x-coordinate of each point associated with the item and *yAmount* to the y-coordinate of each point associated with the item. This command returns an empty string.

pathName* moveto *tagOrId xPos yPos

Move the items given by tagOrId in the canvas coordinate space so that the first coordinate pair (the upper-left corner of the bounding box) of the first item (the lowest in the display list) with tag tagOrId is located at position (xPos,yPos). xPos and yPos may be the empty string, in which case the corresponding coordinate will be unchanged. All items matching tagOrId remain in the same positions relative to each other. This command returns an empty string.

***pathName* postscript ?*option value option value ...*?**

Generate a Postscript representation for part or all of the canvas. If the **-file** option is specified then the Postscript is written to a file and an empty string is returned; otherwise the Postscript is returned as the result of the command. If the interpreter that owns the canvas is marked as safe, the operation will fail because safe interpreters are not allowed to write files. If the **-channel** option is specified, the argument denotes the name of a channel already opened for writing. The Postscript is written to that channel, and the channel is left open for further writing at the end of the operation. The Postscript is created in Encapsulated Postscript form using version 3.0 of the Document Structuring Conventions. Note: by default Postscript is only generated for information that appears in the canvas's window on the screen. If the canvas is freshly created it may still have its initial size of 1x1 pixel so nothing will appear in the Postscript. To get around this problem either invoke the update command to wait for the canvas window to reach its final size, or else use the **-width** and **-height** options to specify the area of the canvas to print. The *option-value* argument pairs provide additional information to control the generation of Postscript. The following options are supported:

-channel *channelName*

Specifies the name of the channel to which to write the Postscript. If this option and the **-file** option are not specified then the Postscript is returned as the result of the command.

-colormap *varName*

VarName must be the name of an array variable that specifies a color mapping to use in the Postscript. Each element of *varName* must consist of Postscript code to set a particular color value (e.g. "**1.0 1.0 0.0 setrgbcolor**"). When outputting color information in the

Postscript, Tk checks to see if there is an element of *varName* with the same name as the color. If so, Tk uses the value of the element as the Postscript command to set the color. If this option has not been specified, or if there is no entry in *varName* for a given color, then Tk uses the red, green, and blue intensities from the X color.

-colormode *mode*
Specifies how to output color information. *Mode* must be either **color** (for full color output), **gray** (convert all colors to their gray-scale equivalents) or **mono** (convert all colors to black or white).

-file *fileName*
Specifies the name of the file in which to write the Postscript. If this option and the **-channel** option are not specified then the Postscript is returned as the result of the command.

-fontmap *varName*
VarName must be the name of an array variable that specifies a font mapping to use in the Postscript. Each element of *varName* must consist of a Tcl list with two elements, which are the name and point size of a Postscript font. When outputting Postscript commands for a particular font, Tk checks to see if *varName* contains an element with the same name as the font. If there is such an element, then the font information contained in that element is used in the Postscript. Otherwise Tk attempts to guess what Postscript font to use. Tk's guesses generally only work for well-known fonts such as Times and Helvetica and Courier, and only if the X font name does not omit any dashes up through the point size. For example, **-*-Courier-Bold-R-Normal--*-120-*** will work but ***Courier-Bold-R-Normal*120*** will not; Tk needs the dashes to parse the font name).

-height *size*
Specifies the height of the area of the canvas to print. Defaults to the height of the canvas window.

-pageanchor *anchor*
Specifies which point of the printed area of the canvas should appear over the positioning point on the page (which is given by the **-pagex** and **-pagey** options). For example, **-pageanchor n** means that the top center of the area of the canvas being printed (as it appears in the canvas window) should be over the positioning point. Defaults to **center**.

-pageheight *size*
Specifies that the Postscript should be scaled in both x and y so that the printed area is *size* high on the Postscript page. *Size* consists of a floating-point number followed by **c** for centimeters, **i** for inches, **m** for millimeters, or **p** or nothing for printer's points (1/72 inch). Defaults to the height of the printed area on the screen. If both **-pageheight** and **-pagewidth** are specified then the scale factor from **-pagewidth** is used (non-uniform scaling is not implemented).

-pagewidth *size*
Specifies that the Postscript should be scaled in both x and y so that the printed area is *size* wide on the Postscript page. *Size* has the same form as for **-pageheight**. Defaults to the width of the printed area on the screen. If both **-pageheight** and **-pagewidth** are specified then the scale factor from **-pagewidth** is used (non-uniform scaling is not implemented).

-pagex *position*

Position gives the x-coordinate of the positioning point on the Postscript page, using any of the forms allowed for **-pageheight**. Used in conjunction with the **-pagey** and **-pageanchor** options to determine where the printed area appears on the Postscript page. Defaults to the center of the page.

-pagey *position*
Position gives the y-coordinate of the positioning point on the Postscript page, using any of the forms allowed for **-pageheight**. Used in conjunction with the **-pagex** and **-pageanchor** options to determine where the printed area appears on the Postscript page. Defaults to the center of the page.

-rotate *boolean*
Boolean specifies whether the printed area is to be rotated 90 degrees. In non-rotated output the x-axis of the printed area runs along the short dimension of the page ("portrait" orientation); in rotated output the x-axis runs along the long dimension of the page ("landscape" orientation). Defaults to non-rotated.

-width *size*
Specifies the width of the area of the canvas to print. Defaults to the width of the canvas window.

-x *position*
Specifies the x-coordinate of the left edge of the area of the canvas that is to be printed, in canvas coordinates, not window coordinates. Defaults to the coordinate of the left edge of the window.

-y *position*
Specifies the y-coordinate of the top edge of the area of the canvas that is to be printed, in canvas coordinates, not window coordinates. Defaults to the coordinate of the top edge of the window.

***pathName* raise *tagOrId* ?*aboveThis*?**
Move all of the items given by *tagOrId* to a new position in the display list just after the item given by *aboveThis*. If *tagOrId* refers to more than one item then all are moved but the relative order of the moved items will not be changed. *AboveThis* is a tag or id; if it refers to more than one item then the last (topmost) of these items in the display list is used as the destination location for the moved items. This command returns an empty string.

Note: this command has no effect on window items. Window items always obscure other item types, and the stacking order of window items is determined by the raise command and lower command, not the **raise** widget command and **lower** widget command for canvases.

pathName* rchars *tagOrId first last string
This command causes the text or coordinates between first and last for each of the items indicated by tagOrId to be replaced by string. Each item interprets first and last independently according to the rules described in Indices above. Out of the standard set of items, text items support this operation by altering their text as directed, and line and polygon items support this operation by altering their coordinate list (in which case string should be a list of coordinates to use as a replacement). The other items ignore this operation.

pathName* scale *tagOrId xOrigin yOrigin xScale yScale

Rescale the coordinates of all of the items given by *tagOrId* in canvas coordinate space. *XOrigin* and *yOrigin* identify the origin for the scaling operation and *xScale* and *yScale* identify the scale factors for x- and y-coordinates, respectively (a scale factor of 1.0 implies no change to that coordinate). For each of the points defining each item, the x-coordinate is adjusted to change the distance from *xOrigin* by a factor of *xScale*. Similarly, each y-coordinate is adjusted to change the distance from *yOrigin* by a factor of *yScale*. This command returns an empty string.

Note that some items have only a single pair of coordinates (e.g., text, images and windows) and so scaling of them by this command can only move them around.

pathName* scan *option args

This command is used to implement scanning on canvases. It has two forms, depending on *option*:

pathName* scan mark *x y

Records *x* and *y* and the canvas's current view; used in conjunction with later **scan dragto** commands. Typically this command is associated with a mouse button press in the widget and *x* and *y* are the coordinates of the mouse. It returns an empty string.

pathName* scan dragto *x y ?gain?

This command computes the difference between its *x* and *y* arguments (which are typically mouse coordinates) and the *x* and *y* arguments to the last **scan mark** command for the widget. It then adjusts the view by *gain* times the difference in coordinates, where *gain* defaults to 10. This command is typically associated with mouse motion events in the widget, to produce the effect of dragging the canvas at high speed through its window. The return value is an empty string.

***pathName* select *option* ?*tagOrId arg*?**

Manipulates the selection in one of several ways, depending on *option*. The command may take any of the forms described below. In all of the descriptions below, *tagOrId* must refer to an item that supports indexing and selection; if it refers to multiple items then the first of these that supports indexing and the selection is used. *Index* gives a textual description of a position within *tagOrId*, as described in Indices above.

pathName* select adjust *tagOrId index

Locate the end of the selection in *tagOrId* nearest to the character given by *index*, and adjust that end of the selection to be at *index* (i.e. including but not going beyond *index*). The other end of the selection is made the anchor point for future **select to** commands. If the selection is not currently in *tagOrId* then this command behaves the same as the **select to** widget command. Returns an empty string.

***pathName* select clear**

Clear the selection if it is in this widget. If the selection is not in this widget then the command has no effect. Returns an empty string.

pathName* select from *tagOrId index

Set the selection anchor point for the widget to be just before the character given by *index* in the item given by *tagOrId*. This command does not change the selection; it just sets the fixed end of the selection for future **select to** commands. Returns an empty string.

***pathName* select item**

Returns the id of the selected item, if the selection is in an item in this canvas. If the

selection is not in this canvas then an empty string is returned.

pathName* select to *tagOrId index

Set the selection to consist of those characters of *tagOrId* between the selection anchor point and *index*. The new selection will include the character given by *index*; it will include the character given by the anchor point only if *index* is greater than or equal to the anchor point. The anchor point is determined by the most recent **select adjust** or **select from** command for this widget. If the selection anchor point for the widget is not currently in *tagOrId*, then it is set to the same character given by *index*. Returns an empty string.

pathName* type *tagOrId

Returns the type of the item given by *tagOrId*, such as **rectangle** or **text**. If *tagOrId* refers to more than one item, then the type of the first item in the display list is returned. If *tagOrId* does not refer to any items at all then an empty string is returned.

***pathName* xview ?*args*?**

This command is used to query and change the horizontal position of the information displayed in the canvas's window. It can take any of the following forms:

***pathName* xview**

Returns a list containing two elements. Each element is a real fraction between 0 and 1; together they describe the horizontal span that is visible in the window. For example, if the first element is .2 and the second element is .6, 20% of the canvas's area (as defined by the **-scrollregion** option) is off-screen to the left, the middle 40% is visible in the window, and 40% of the canvas is off-screen to the right. These are the same values passed to scrollbars via the **-xscrollcommand** option.

pathName* xview moveto *fraction

Adjusts the view in the window so that *fraction* of the total width of the canvas is off-screen to the left. *Fraction* must be a fraction between 0 and 1.

pathName* xview scroll *number what

This command shifts the view in the window left or right according to *number* and *what*. *Number* must be an integer. *What* must be either **units** or **pages** or an abbreviation of one of these. If *what* is **units**, the view adjusts left or right in units of the **xScrollIncrement** option, if it is greater than zero, or in units of one-tenth the window's width otherwise. If *what is* **pages** then the view adjusts in units of nine-tenths the window's width. If *number* is negative then information farther to the left becomes visible; if it is positive then information farther to the right becomes visible.

***pathName* yview ?*args*?**

This command is used to query and change the vertical position of the information displayed in the canvas's window. It can take any of the following forms:

***pathName* yview**

Returns a list containing two elements. Each element is a real fraction between 0 and 1; together they describe the vertical span that is visible in the window. For example, if the first element is .6 and the second element is 1.0, the lowest 40% of the canvas's area (as defined by the **-scrollregion** option) is visible in the window. These are the same values passed to scrollbars via the **-yscrollcommand** option.

pathName* yview moveto *fraction

Adjusts the view in the window so that *fraction* of the canvas's area is off-screen to the top.

Fraction is a fraction between 0 and 1.

pathName* yview scroll *number what
This command adjusts the view in the window up or down according to *number* and *what*. *Number* must be an integer. *What* must be either **units** or **pages**. If *what* is **units**, the view adjusts up or down in units of the **yScrollIncrement** option, if it is greater than zero, or in units of one-tenth the window's height otherwise. If *what* is **pages** then the view adjusts in units of nine-tenths the window's height. If *number* is negative then higher information becomes visible; if it is positive then lower information becomes visible.

Overview of item types

The sections below describe the various types of items supported by canvas widgets. Each item type is characterized by two things: first, the form of the **create** command used to create instances of the type; and second, a set of configuration options for items of that type, which may be used in the **create** and **itemconfigure** widget commands. Most items do not support indexing or selection or the commands related to them, such as **index** and **insert**. Where items do support these facilities, it is noted explicitly in the descriptions below. At present, text, line and polygon items provide this support. For lines and polygons the indexing facility is used to manipulate the coordinates of the item.

Common item options

Many items share a common set of options. These options are explained here, and then referred to be each widget type for brevity.

-anchor *anchorPos*
AnchorPos tells how to position the item relative to the positioning point for the item; it may have any of the forms accepted by Tk_GetAnchor. For example, if *anchorPos* is **center** then the item is centered on the point; if *anchorPos* is **n** then the item will be drawn so that its top center point is at the positioning point. This option defaults to **center**.

-dash *pattern*
see below ...

-activedash *pattern*
see below ...

-disableddash *pattern*
This option specifies dash patterns for the normal, active state, and disabled state of an item. *pattern* may have any of the forms accepted by Tk_GetDash. If the dash options are omitted then the default is a solid outline. See Dash patterns for more information.

-dashoffset *offset*
The starting *offset* in pixels into the pattern provided by the **-dash** option. **-dashoffset** is ignored if there is no **-dash** pattern. The *offset* may have any of the forms described in the Coordinates section above.

-fill *color*
see below ...

-activefill *color*
see below ...

-disabledfill *color*
Specifies the color to be used to fill item's area. in its normal, active, and disabled states. The

even-odd fill rule is used. *Color* may have any of the forms accepted by Tk_GetColor. For the line item, it specifies the color of the line drawn. For the text item, it specifies the foreground color of the text. If *color* is an empty string (the default for all canvas items except line and text), then the item will not be filled.

-outline *color*
see below ...

-activeoutline *color*
see below ...

-disabledoutline *color*
This option specifies the color that should be used to draw the outline of the item in its normal, active and disabled states. *Color* may have any of the forms accepted by Tk_GetColor . If *color* is specified as an empty string then no outline is drawn for the item.

-offset *offset*
Specifies the offset of stipples. The offset value can be of the form **x,y** or *side*, where side can be **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, or **center**. In the first case the origin is the origin of the toplevel of the current window. For the canvas itself and canvas objects the origin is the canvas origin, but putting **#** in front of the coordinate pair indicates using the toplevel origin instead. For canvas objects, the **-offset** option is used for stippling as well. For the line and polygon canvas items you can also specify an index as argument, which connects the stipple origin to one of the coordinate points of the line/polygon. Note that stipple offsets are *only supported on X11*; they are silently ignored on other platforms.

-outlinestipple *bitmap*
see below ...

-activeoutlinestipple *bitmap*
see below ...

-disabledoutlinestipple *bitmap*
This option specifies stipple patterns that should be used to draw the outline of the item in its normal, active and disabled states. Indicates that the outline for the item should be drawn with a stipple pattern; *bitmap* specifies the stipple pattern to use, in any of the forms accepted by Tk_GetBitmap. If the **-outline** option has not been specified then this option has no effect. If *bitmap* is an empty string (the default), then the outline is drawn in a solid fashion. *Note that stipples are not well supported on platforms that do not use X11 as their drawing API.*

-outlineoffset *offset*
Specifies the offset of the stipple pattern used for outlines, in the same way that the **-outline** option controls fill stipples. (See the **-outline** option for a description of the syntax of *offset*.)

-stipple *bitmap*
see below ...

-activestipple *bitmap*
see below ...

-disabledstipple *bitmap*
This option specifies stipple patterns that should be used to fill the item in its normal, active and disabled states. *bitmap* specifies the stipple pattern to use, in any of the forms accepted by Tk_GetBitmap. If the **-fill** option has not been specified then this option has no effect. If *bitmap* is an empty string (the default), then filling is done in a solid fashion. For the text

item, it affects the actual text. *Note that stipples are not well supported on platforms that do not use X11 as their drawing API.*

-state *state*
This allows an item to override the canvas widget's global *state* option. It takes the same values: *normal*, *disabled* or *hidden*.

-tags *tagList*
Specifies a set of tags to apply to the item. *TagList* consists of a list of tag names, which replace any existing tags for the item. *TagList* may be an empty list.

-width *outlineWidth*
see below ...

-activewidth *outlineWidth*
see below ...

-disabledwidth *outlineWidth*
Specifies the width of the outline to be drawn around the item's region, in its normal, active and disabled states. *outlineWidth* may be in any of the forms described in the Coordinates section above. If the **-outline** option has been specified as an empty string then this option has no effect. This option defaults to 1.0. For arcs, wide outlines will be drawn centered on the edges of the arc's region.

Standard item types

Arc items

Items of type **arc** appear on the display as arc-shaped regions. An arc is a section of an oval delimited by two angles (specified by the **-start** and **-extent** options) and displayed in one of several ways (specified by the **-style** option). Arcs are created with widget commands of the following form:

```
pathName create arc x1 y1 x2 y2 ?option value ...?
pathName create arc coordList ?option value ...?
```

The arguments *x1*, *y1*, *x2*, and *y2* or *coordList* give the coordinates of two diagonally opposite corners of a rectangular region enclosing the oval that defines the arc. After the coordinates there may be any number of *option-value* pairs, each of which sets one of the configuration options for the item. These same *option-value* pairs may be used in **itemconfigure** widget commands to change the item's configuration. An arc item becomes the current item when the mouse pointer is over any part that is painted or (when fully transparent) that would be painted if both the **-fill** and **-outline** options were non-empty.

The following standard options are supported by arcs:

-dash -activedash
-disableddash -dashoffset
-fill -activefill
-disabledfill -offset
-outline -activeoutline
-disabledoutline -outlineoffset
-outlinestipple -activeoutlinestipple
-disabledoutlinestipple -stipple

-activestipple -disabledstipple
-state -tags
-width -activewidth
-disabledwidth

The following extra options are supported for arcs:

-extent *degrees*
Specifies the size of the angular range occupied by the arc. The arc's range extends for *degrees* degrees counter-clockwise from the starting angle given by the **-start** option. *Degrees* may be negative. If it is greater than 360 or less than -360, then *degrees* modulo 360 is used as the extent.

-start *degrees*
Specifies the beginning of the angular range occupied by the arc. *Degrees* is given in units of degrees measured counter-clockwise from the 3-o'clock position; it may be either positive or negative.

-style *type*
Specifies how to draw the arc. If *type* is **pieslice** (the default) then the arc's region is defined by a section of the oval's perimeter plus two line segments, one between the center of the oval and each end of the perimeter section. If *type* is **chord** then the arc's region is defined by a section of the oval's perimeter plus a single line segment connecting the two end points of the perimeter section. If *type* is **arc** then the arc's region consists of a section of the perimeter alone. In this last case the **-fill** option is ignored.

Bitmap items

Items of type **bitmap** appear on the display as images with two colors, foreground and background. Bitmaps are created with widget commands of the following form:

```
pathName create bitmap x y ?option value ...?
pathName create bitmap coordList ?option value ...?
```

The arguments *x* and *y* or *coordList* (which must have two elements) specify the coordinates of a point used to position the bitmap on the display, as controlled by the **-anchor** option. After the coordinates there may be any number of *option-value* pairs, each of which sets one of the configuration options for the item. These same *option-value* pairs may be used in **itemconfigure** widget commands to change the item's configuration. A bitmap item becomes the current item when the mouse pointer is over any part of its bounding box.

The following standard options are supported by bitmaps:

-anchor -state
-tags

The following extra options are supported for bitmaps:

-background *color*
see below ...

-activebackground *color*
see below ...

-disabledbackground *color*
Specifies the color to use for each of the bitmap's "0" valued pixels in its normal, active and

disabled states. *Color* may have any of the forms accepted by Tk_GetColor. If this option is not specified, or if it is specified as an empty string, then nothing is displayed where the bitmap pixels are 0; this produces a transparent effect.

-bitmap *bitmap*
see below ...

-activebitmap *bitmap*
see below ...

-disabledbitmap *bitmap*
Specifies the bitmaps to display in the item in its normal, active and disabled states. *Bitmap* may have any of the forms accepted by Tk_GetBitmap.

-foreground *color*
see below ...

-activeforeground *color*
see below ...

-disabledforeground *color*
Specifies the color to use for each of the bitmap's "1" valued pixels in its normal, active and disabled states. *Color* may have any of the forms accepted by Tk_GetColor.

Image items

Items of type image are used to display images on a canvas. Images are created with widget commands of the following form:

```
pathName create image x y ?option value ...?
pathName create image coordList ?option value ...?
```

The arguments *x* and *y* or *coordList* specify the coordinates of a point used to position the image on the display, as controlled by the **-anchor** option. After the coordinates there may be any number of *option-value* pairs, each of which sets one of the configuration options for the item. These same *option-value* pairs may be used in **itemconfigure** widget commands to change the item's configuration. An image item becomes the current item when the mouse pointer is over any part of its bounding box.

The following standard options are supported by images:

-anchor -state
-tags

The following extra options are supported for images:

-image *name*
see below ...

-activeimage *name*
see below ...

-disabledimage *name*
Specifies the name of the images to display in the item in is normal, active and disabled states. This image must have been created previously with the image create command.

Line items

Items of type **line** appear on the display as one or more connected line segments or curves. Line

items support coordinate indexing operations using the **dchars**, **index** and **insert** widget commands. Lines are created with widget commands of the following form:

```
pathName create line x1 y1... xn yn ?option value ...?
pathName create line coordList ?option value ...?
```

The arguments *x1* through *yn* or *coordList* give the coordinates for a series of two or more points that describe a series of connected line segments. After the coordinates there may be any number of *option-value* pairs, each of which sets one of the configuration options for the item. These same *option-value* pairs may be used in **itemconfigure** widget commands to change the item's configuration. A line item is the current item whenever the mouse pointer is over any segment of the line, whether drawn or not and whether or not the line is smoothed.

The following standard options are supported by lines:

-dash -activedash
-disableddash -dashoffset
-fill -activefill
-disabledfill -stipple
-activestipple -disabledstipple
-state -tags
-width -activewidth
-disabledwidth

The following extra options are supported for lines:

-arrow *where*
: Indicates whether or not arrowheads are to be drawn at one or both ends of the line. *Where* must have one of the values **none** (for no arrowheads), **first** (for an arrowhead at the first point of the line), **last** (for an arrowhead at the last point of the line), or **both** (for arrowheads at both ends). This option defaults to **none**. When requested to draw an arrowhead, Tk internally adjusts the corresponding line end point so that the rendered line ends at the neck of the arrowhead rather than at its tip so that the line doesn't extend past the edge of the arrowhead. This may trigger a **Leave** event if the mouse is hovering this line end. Conversely, when removing an arrowhead Tk adjusts the corresponding line point the other way round, which may trigger an **Enter** event.

-arrowshape *shape*
: This option indicates how to draw arrowheads. The *shape* argument must be a list with three elements, each specifying a distance in any of the forms described in the Coordinates section above. The first element of the list gives the distance along the line from the neck of the arrowhead to its tip. The second element gives the distance along the line from the trailing points of the arrowhead to the tip, and the third element gives the distance from the outside edge of the line to the trailing points. If this option is not specified then Tk picks a "reasonable" shape.

-capstyle *style*
: Specifies the ways in which caps are to be drawn at the endpoints of the line. *Style* may have any of the forms accepted by Tk_GetCapStyle (**butt**, **projecting**, or **round**). If this option is not specified then it defaults to **butt**. Where arrowheads are drawn the cap style is ignored.

-joinstyle *style*
: Specifies the ways in which joints are to be drawn at the vertices of the line. *Style* may have

any of the forms accepted by Tk_GetJoinStyle (**bevel**, **miter**, or **round**). If this option is not specified then it defaults to **round**. If the line only contains two points then this option is irrelevant.

-smooth *smoothMethod*
smoothMethod must have one of the forms accepted by Tcl_GetBoolean or a line smoothing method. Only **true** and **raw** are supported in the core (with **bezier** being an alias for **true**), but more can be added at runtime. If a boolean false value or empty string is given, no smoothing is applied. A boolean truth value assumes **true** smoothing. If the smoothing method is **true**, this indicates that the line should be drawn as a curve, rendered as a set of quadratic splines: one spline is drawn for the first and second line segments, one for the second and third, and so on. Straight-line segments can be generated within a curve by duplicating the end-points of the desired line segment. If the smoothing method is **raw**, this indicates that the line should also be drawn as a curve but where the list of coordinates is such that the first coordinate pair (and every third coordinate pair thereafter) is a knot point on a cubic Bezier curve, and the other coordinates are control points on the cubic Bezier curve. Straight line segments can be generated within a curve by making control points equal to their neighbouring knot points. If the last point is a control point and not a knot point, the point is repeated (one or two times) so that it also becomes a knot point.

-splinesteps *number*
Specifies the degree of smoothness desired for curves: each spline will be approximated with *number* line segments. This option is ignored unless the **-smooth** option is true or **raw**.

Oval items

Items of type **oval** appear as circular or oval regions on the display. Each oval may have an outline, a fill, or both. Ovals are created with widget commands of the following form:

```
pathName create oval x1 y1 x2 y2 ?option value ...?
pathName create oval coordList ?option value ...?
```

The arguments *x1*, *y1*, *x2*, and *y2* or *coordList* give the coordinates of two diagonally opposite corners of a rectangular region enclosing the oval. The oval will include the top and left edges of the rectangle not the lower or right edges. If the region is square then the resulting oval is circular; otherwise it is elongated in shape. After the coordinates there may be any number of *option-value* pairs, each of which sets one of the configuration options for the item. These same *option-value* pairs may be used in **itemconfigure** widget commands to change the item's configuration. An oval item becomes the current item when the mouse pointer is over any part that is painted or (when fully transparent) that would be painted if both the **-fill** and **-outline** options were non-empty.

The following standard options are supported by ovals:

-dash -activedash
-disableddash -dashoffset
-fill -activefill
-disabledfill -offset
-outline -activeoutline
-disabledoutline -outlineoffset
-outlinestipple -activeoutlinestipple

-disabledoutlinestipple -stipple
-activestipple -disabledstipple
-state -tags
-width -activewidth
-disabledwidth

There are no oval-specific options.

Polygon items

Items of type **polygon** appear as polygonal or curved filled regions on the display. Polygon items support coordinate indexing operations using the **dchars**, **index** and **insert** widget commands. Polygons are created with widget commands of the following form:

```
pathName create polygon x1 y1 ... xn yn ?option value ...?
pathName create polygon coordList ?option value ...?
```

The arguments *x1* through *yn* or *coordList* specify the coordinates for three or more points that define a polygon. The first point should not be repeated as the last to close the shape; Tk will automatically close the periphery between the first and last points. After the coordinates there may be any number of *option-value* pairs, each of which sets one of the configuration options for the item. These same *option-value* pairs may be used in **itemconfigure** widget commands to change the item's configuration. A polygon item is the current item whenever the mouse pointer is over any part of the polygon, whether drawn or not and whether or not the outline is smoothed.

The following standard options are supported by polygons:

-dash -activedash
-disableddash -dashoffset
-fill -activefill
-disabledfill -offset
-outline -activeoutline
-disabledoutline -outlineoffset
-outlinestipple -activeoutlinestipple
-disabledoutlinestipple -stipple
-activestipple -disabledstipple
-state -tags
-width -activewidth
-disabledwidth

The following extra options are supported for polygons:

-joinstyle *style*
: Specifies the ways in which joints are to be drawn at the vertices of the outline. *Style* may have any of the forms accepted by Tk_GetJoinStyle (**bevel**, **miter**, or **round**). If this option is not specified then it defaults to **round**.

-smooth *boolean*
: *Boolean* must have one of the forms accepted by Tcl_GetBoolean or a line smoothing method. Only **true** and **raw** are supported in the core (with **bezier** being an alias for **true**), but more can be added at runtime. If a boolean false value or empty string is given, no smoothing is applied. A boolean truth value assumes **true** smoothing. If the smoothing method is **true**,

this indicates that the polygon should be drawn as a curve, rendered as a set of quadratic splines: one spline is drawn for the first and second line segments, one for the second and third, and so on. Straight-line segments can be generated within a curve by duplicating the end-points of the desired line segment. If the smoothing method is **raw**, this indicates that the polygon should also be drawn as a curve but where the list of coordinates is such that the first coordinate pair (and every third coordinate pair thereafter) is a knot point on a cubic Bezier curve, and the other coordinates are control points on the cubic Bezier curve. Straight line segments can be generated within a curve by making control points equal to their neighbouring knot points. If the last point is not the second point of a pair of control points, the point is repeated (one or two times) so that it also becomes the second point of a pair of control points (the associated knot point will be the first control point).

-splinesteps *number*
Specifies the degree of smoothness desired for curves: each spline will be approximated with *number* line segments. This option is ignored unless the **-smooth** option is true or **raw**.

Polygon items are different from other items such as rectangles, ovals and arcs in that interior points are considered to be "inside" a polygon (e.g. for purposes of the **find closest** and **find overlapping** widget commands) even if it is not filled. For most other item types, an interior point is considered to be inside the item only if the item is filled or if it has neither a fill nor an outline. If you would like an unfilled polygon whose interior points are not considered to be inside the polygon, use a line item instead.

Rectangle items

Items of type **rectangle** appear as rectangular regions on the display. Each rectangle may have an outline, a fill, or both. Rectangles are created with widget commands of the following form:

```
pathName create rectangle x1 y1 x2 y2 ?option value ...?
pathName create rectangle coordList ?option value ...?
```

The arguments *x1*, *y1*, *x2*, and *y2* or *coordList* (which must have four elements) give the coordinates of two diagonally opposite corners of the rectangle (the rectangle will include its upper and left edges but not its lower or right edges). After the coordinates there may be any number of *option-value* pairs, each of which sets one of the configuration options for the item. These same *option-value* pairs may be used in **itemconfigure** widget commands to change the item's configuration. A rectangle item becomes the current item when the mouse pointer is over any part that is painted or (when fully transparent) that would be painted if both the **-fill** and **-outline** options were non-empty.

The following standard options are supported by rectangles:

-dash -activedash
-disableddash -dashoffset
-fill -activefill
-disabledfill -offset
-outline -activeoutline
-disabledoutline -outlineoffset
-outlinestipple -activeoutlinestipple
-disabledoutlinestipple -stipple
-activestipple -disabledstipple

-state -tags
-width -activewidth
-disabledwidth

There are no rectangle-specific options.

Text items

A text item displays a string of characters on the screen in one or more lines. Text items support indexing, editing and selection through the **dchars** widget command, the **focus** widget command, the **icursor** widget command, the **index** widget command, the **insert** widget command, and the **select** widget command. Text items are created with widget commands of the following form:

```
pathName create text x y ?option value ...?
pathName create text coordList ?option value ...?
```

The arguments *x* and *y* or *coordList* (which must have two elements) specify the coordinates of a point used to position the text on the display (see the options below for more information on how text is displayed). After the coordinates there may be any number of *option-value* pairs, each of which sets one of the configuration options for the item. These same *option-value* pairs may be used in **itemconfigure** widget commands to change the item's configuration. A text item becomes the current item when the mouse pointer is over any part of its bounding box.

The following standard options are supported by text items:

-anchor -fill
-activefill -disabledfill
-stipple -activestipple
-disabledstipple -state
-tags

The following extra options are supported for text items:

-angle *rotationDegrees*
RotationDegrees tells how many degrees to rotate the text anticlockwise about the positioning point for the text; it may have any floating-point value from 0.0 to 360.0. For example, if rotationDegrees is 90, then the text will be drawn vertically from bottom to top. This option defaults to 0.0.

-font *fontName*
Specifies the font to use for the text item. *FontName* may be any string acceptable to Tk_GetFont. If this option is not specified, it defaults to a system-dependent font.

-justify *how*
Specifies how to justify the text within its bounding region. *How* must be one of the values **left**, **right**, or **center**. This option will only matter if the text is displayed as multiple lines. If the option is omitted, it defaults to **left**.

-text *string*
String specifies the characters to be displayed in the text item. Newline characters cause line breaks. The characters in the item may also be changed with the **insert** and **delete** widget commands. This option defaults to an empty string.

-underline
Specifies the integer index of a character within the text to be underlined. 0 corresponds to

the first character of the text displayed, 1 to the next character, and so on. -1 means that no underline should be drawn (if the whole text item is to be underlined, the appropriate font should be used instead).

-width *lineLength*
Specifies a maximum line length for the text, in any of the forms described in the Coordinates section above. If this option is zero (the default) the text is broken into lines only at newline characters. However, if this option is non-zero then any line that would be longer than *lineLength* is broken just before a space character to make the line shorter than *lineLength*; the space character is treated as if it were a newline character.

Window items

Items of type **window** cause a particular window to be displayed at a given position on the canvas. Window items are created with widget commands of the following form:

```
pathName create window x y ?option value ...?
pathName create window coordList ?option value ...?
```

The arguments *x* and *y* or *coordList* (which must have two elements) specify the coordinates of a point used to position the window on the display, as controlled by the **-anchor** option. After the coordinates there may be any number of *option-value* pairs, each of which sets one of the configuration options for the item. These same *option-value* pairs may be used in **itemconfigure** widget commands to change the item's configuration. Theoretically, a window item becomes the current item when the mouse pointer is over any part of its bounding box, but in practice this typically does not happen because the mouse pointer ceases to be over the canvas at that point.

The following standard options are supported by window items:

-anchor -state
-tags

The following extra options are supported for window items:

-height *pixels*
Specifies the height to assign to the item's window. *Pixels* may have any of the forms described in the Coordinates section above. If this option is not specified, or if it is specified as zero, then the window is given whatever height it requests internally.

-width *pixels*
Specifies the width to assign to the item's window. *Pixels* may have any of the forms described in the Coordinates section above. If this option is not specified, or if it is specified as zero, then the window is given whatever width it requests internally.

-window *pathName*
Specifies the window to associate with this item. The window specified by *pathName* must either be a child of the canvas widget or a child of some ancestor of the canvas widget. *PathName* may not refer to a top-level window.

Note: due to restrictions in the ways that windows are managed, it is not possible to draw other graphical items (such as lines and images) on top of window items. A window item always obscures any graphics that overlap it, regardless of their order in the display list. Also note that window items, unlike other canvas items, are not clipped for display by their containing canvas's

border, and are instead clipped by the parent widget of the window specified by the **-window** option; when the parent widget is the canvas, this means that the window item can overlap the canvas's border.

Application-defined item types

It is possible for individual applications to define new item types for canvas widgets using C code. See the documentation for Tk_CreateItemType.

Bindings

In the current implementation, new canvases are not given any default behavior: you will have to execute explicit Tcl commands to give the canvas its behavior.

Credits

Tk's canvas widget is a blatant ripoff of ideas from Joel Bartlett's *ezd* program. *Ezd* provides structured graphics in a Scheme environment and preceded canvases by a year or two. Its simple mechanisms for placing and animating graphical objects inspired the functions of canvases.

Copyright (c) 1992-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 1997-1999 Scriptics Corporation.

checkbutton

Keywords: checkbutton, widget

Name

checkbutton - Create and manipulate 'checkbutton' boolean selection widgets

Synopsis

checkbutton *pathName* ?*options*?

Standard options

-activebackground, **-disabledforeground**, **-padx**, **-activeforeground**, **-font**, **-pady**, **-anchor**, **-foreground**, **-relief**, **-background**, **-highlightbackground**, **-takefocus**, **-bitmap**, **-highlightcolor**, **-text**, **-borderwidth**, **-highlightthickness**, **-textvariable**, **-compound**, **-image**, **-underline**, **-cursor**, **-justify**, **-wraplength**

See the options manual entry for details on the standard options.

Widget-specific options

-command (command/Command)

Specifies a Tcl command to associate with the button. This command is typically invoked when mouse button 1 is released over the button window. The button's global variable (**-variable** option) will be updated before the command is invoked.

-height (height/Height)

Specifies a desired height for the button. If an image or bitmap is being displayed in the button then the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels); for text it is in lines of text. If this option is not specified, the button's desired height is computed

from the size of the image or bitmap or text being displayed in it.

-indicatoron (indicatorOn/IndicatorOn)
Specifies whether or not the indicator should be drawn. Must be a proper boolean value. If false, the **-relief** option is ignored and the widget's relief is always sunken if the widget is selected and raised otherwise.

-offrelief (offRelief/OffRelief)
Specifies the relief for the checkbutton when the indicator is not drawn and the checkbutton is off. The default value is "raised". By setting this option to "flat" and setting **-indicatoron** to false and **-overrelief** to "raised", the effect is achieved of having a flat button that raises on mouse-over and which is depressed when activated. This is the behavior typically exhibited by the Bold, Italic, and Underline checkbuttons on the toolbar of a word-processor, for example.

-offvalue (offValue/Value)
Specifies value to store in the button's associated variable whenever this button is deselected. Defaults to "0".

-onvalue (onValue/Value)
Specifies value to store in the button's associated variable whenever this button is selected. Defaults to "1".

-overrelief (overRelief/OverRelief)
Specifies an alternative relief for the checkbutton, to be used when the mouse cursor is over the widget. This option can be used to make toolbar buttons, by configuring **-relief flat -overrelief raised**. If the value of this option is the empty string, then no alternative relief is used when the mouse cursor is over the checkbutton. The empty string is the default value.

-selectcolor (selectColor/Background)
Specifies a background color to use when the button is selected. If **indicatorOn** is true then the color is used as the background for the indicator regardless of the select state. If **indicatorOn** is false, this color is used as the background for the entire widget, in place of **background** or **activeBackground**, whenever the widget is selected. If specified as an empty string then no special color is used for displaying when the widget is selected.

-selectimage (selectImage/SelectImage)
Specifies an image to display (in place of the **-image** option) when the checkbutton is selected. This option is ignored unless the **-image** option has been specified.

-state (state/State)
Specifies one of three states for the checkbutton: **normal**, **active**, or **disabled**. In normal state the checkbutton is displayed using the **-foreground** and **-background** options. The active state is typically used when the pointer is over the checkbutton. In active state the checkbutton is displayed using the **-activeforeground** and **-activebackground** options. Disabled state means that the checkbutton should be insensitive: the default bindings will refuse to activate the widget and will ignore mouse button presses. In this state the **-disabledforeground** and **-background** options determine how the checkbutton is displayed.

-tristateimage (tristateImage/TristateImage)
Specifies an image to display (in place of the **-image** option) when the checkbutton is in tri-state mode. This option is ignored unless the **-image** option has been specified.

-tristatevalue (tristateValue/Value)

Specifies the value that causes the checkbutton to display the multi-value selection, also known as the tri-state mode. Defaults to "".

-variable (variable**/Variable)**
Specifies the name of a global variable to set to indicate whether or not this button is selected. Defaults to the name of the button within its parent (i.e. the last element of the button window's path name).

-width (width/Width)
Specifies a desired width for the button. If an image or bitmap is being displayed in the button then the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels); for text it is in characters. If this option is not specified, the button's desired width is computed from the size of the image or bitmap or text being displayed in it.

Description

The **checkbutton** command creates a new window (given by the *pathName* argument) and makes it into a checkbutton widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the checkbutton such as its colors, font, text, and initial relief. The **checkbutton** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A checkbutton is a widget that displays a textual string, bitmap or image and a square called an *indicator*. If text is displayed, it must all be in a single font, but it can occupy multiple lines on the screen (if it contains newlines or if wrapping occurs because of the **-wraplength** option) and one of the characters may optionally be underlined using the **-underline** option. A checkbutton has all of the behavior of a simple button, including the following: it can display itself in either of three different ways, according to the **-state** option; it can be made to appear raised, sunken, or flat; it can be made to flash; and it invokes a Tcl command whenever mouse button 1 is clicked over the checkbutton.

In addition, checkbuttons can be *selected*. If a checkbutton is selected then the indicator is normally drawn with a selected appearance, and a Tcl variable associated with the checkbutton is set to a particular value (normally 1). The indicator is drawn with a check mark inside. If the checkbutton is not selected, then the indicator is drawn with a deselected appearance, and the associated variable is set to a different value (typically 0). The indicator is drawn without a check mark inside. In the special case where the variable (if specified) has a value that matches the tristatevalue, the indicator is drawn with a tri-state appearance and is in the tri-state mode indicating mixed or multiple values. (This is used when the check box represents the state of multiple items.) The indicator is drawn in a platform dependent manner. Under Unix and Windows, the background interior of the box is "grayed". Under Mac, the indicator is drawn with a dash mark inside. By default, the name of the variable associated with a checkbutton is the same as the *name* used to create the checkbutton. The variable name, and the "on", "off" and "tristate" values stored in it, may be modified with options on the command line or in the option database. Configuration options may also be used to modify the way the indicator is displayed (or whether it is displayed at all). By default a checkbutton is configured to select and deselect itself on alternate button clicks. In addition, each checkbutton monitors its associated variable and automatically selects and deselects itself when the variables value changes to and from the button's "on", "off" and "tristate" values.

Widget command

The **checkbutton** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following commands are possible for checkbutton widgets:

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **checkbutton** command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **checkbutton** command.

***pathName* deselect**
Deselects the checkbutton and sets the associated variable to its "off" value.

***pathName* flash**
Flashes the checkbutton. This is accomplished by redisplaying the checkbutton several times, alternating between active and normal colors. At the end of the flash the checkbutton is left in the same normal/active state as when the command was invoked. This command is ignored if the checkbutton's state is **disabled**.

***pathName* invoke**
Does just what would have happened if the user invoked the checkbutton with the mouse: toggle the selection state of the button and invoke the Tcl command associated with the checkbutton, if there is one. The return value is the return value from the Tcl command, or an empty string if there is no command associated with the checkbutton. This command is ignored if the checkbutton's state is **disabled**.

***pathName* select**
Selects the checkbutton and sets the associated variable to its "on" value.

***pathName* toggle**
Toggles the selection state of the button, redisplaying it and modifying its associated variable to reflect the new state.

Bindings

Tk automatically creates class bindings for checkbuttons that give them the following default behavior:

1. On Unix systems, a checkbutton activates whenever the mouse passes over it and deactivates whenever the mouse leaves the checkbutton. On Mac and Windows systems, when mouse button 1 is pressed over a checkbutton, the button activates whenever the

mouse pointer is inside the button, and deactivates whenever the mouse pointer leaves the button.
2. When mouse button 1 is pressed over a checkbutton, it is invoked (its selection state toggles and the command associated with the button is invoked, if there is one).
3. When a checkbutton has the input focus, the space key causes the checkbutton to be invoked. Under Windows, there are additional key bindings; plus (**+**) and equal (**=**) select the button, and minus (**-**) deselects the button.

If the checkbutton's state is **disabled** then none of the above actions occur: the checkbutton is completely non-responsive.

The behavior of checkbuttons can be changed by defining new bindings for individual widgets or by redefining the class bindings.

Example

This example shows a group of uncoupled checkbuttons.

```
labelframe .lbl -text "Steps:"
checkbutton .c1 -text Lights  -variable lights
checkbutton .c2 -text Cameras -variable cameras
checkbutton .c3 -text Action! -variable action
pack .c1 .c2 .c3 -in .lbl
pack .lbl
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

chooseColor

Keywords: chooseColor, color, color selection, dialog

Name

tk_chooseColor - pops up a dialog box for the user to select a color.

Synopsis

tk_chooseColor ?*option value* ...?

Description

The procedure **tk_chooseColor** pops up a dialog box for the user to select a color. The following *option-value* pairs are possible as command line arguments:

-initialcolor *color*
Specifies the color to display in the color dialog when it pops up. *color* must be in a form acceptable to the Tk_GetColor function.

-parent *window*
Makes *window* the logical parent of the color dialog. The color dialog is displayed on top of its parent window.

-title *titleString*
Specifies a string to display as the title of the dialog box. If this option is not specified, then a default title will be displayed.

If the user selects a color, **tk_chooseColor** will return the name of the color in a form acceptable to Tk_GetColor. If the user cancels the operation, both commands will return the empty string.

Example

```
button .b -bg [tk_chooseColor -initialcolor gray -title "Choose color"]
```

Copyright (c) 1996 Sun Microsystems, Inc.

chooseDirectory

Keywords: chooseDirectory, directory, selection, dialog, platform-specific

Name

tk_chooseDirectory - pops up a dialog box for the user to select a directory.

Synopsis

tk_chooseDirectory ?*option value* ...?

Description

The procedure **tk_chooseDirectory** pops up a dialog box for the user to select a directory. The following *option-value* pairs are possible as command line arguments:

-command *string*
: Specifies the prefix of a Tcl command to invoke when the user closes the dialog after having selected an item. This callback is not called if the user cancelled the dialog. The actual command consists of *string* followed by a space and the value selected by the user in the dialog. This is only available on macOS.

-initialdir *dirname*
: Specifies that the directories in *directory* should be displayed when the dialog pops up. If this parameter is not specified, the initial directory defaults to the current working directory on non-Windows systems and on Windows systems prior to Vista. On Vista and later systems, the initial directory defaults to the last user-selected directory for the application. If the parameter specifies a relative path, the return value will convert the relative path to an absolute path.

-message *string*
: Specifies a message to include in the client area of the dialog. This is only available on macOS.

-mustexist *boolean*
: Specifies whether the user may specify non-existent directories. If this parameter is true, then the user may only select directories that already exist. The default value is *false*.

-parent *window*
: Makes *window* the logical parent of the dialog. The dialog is displayed on top of its parent window. On macOS, this turns the file dialog into a sheet attached to the parent window.

-title *titleString*
: Specifies a string to display as the title of the dialog box. If this option is not specified, then a default title will be displayed.

Example

```
set dir [tk_chooseDirectory \
        -initialdir ~ -title "Choose a directory"]
if {$dir eq ""} {
   label .l -text "No directory selected"
} else {
   label .l -text "Selected $dir"
}
```

Copyright (c) 1998-2000 Scriptics Corporation.

clipboard

Keywords: clipboard, clear, format, append, selection, type

Name

clipboard - Manipulate Tk clipboard

Synopsis

clipboard *option* ?*arg arg* ...?

Description

This command provides a Tcl interface to the Tk clipboard, which stores data for later retrieval using the selection mechanism (via the **-selection CLIPBOARD** option). In order to copy data into the clipboard, **clipboard clear** must be called, followed by a sequence of one or more calls to **clipboard append**. To ensure that the clipboard is updated atomically, all appends should be completed before returning to the event loop.

The first argument to **clipboard** determines the format of the rest of the arguments and the behavior of the command. The following forms are currently supported:

clipboard append ?-displayof *window*? ?-format *format*? ?-type *type*? ?--? *data*

Appends *data* to the clipboard on *window*'s display in the form given by *type* with the representation given by *format* and claims ownership of the clipboard on *window*'s display.

Type specifies the form in which the selection is to be returned (the desired "target" for conversion, in ICCCM terminology), and should be an atom name such as **STRING** or **FILE_NAME**; see the Inter-Client Communication Conventions Manual for complete details. *Type* defaults to **STRING**.

The *format* argument specifies the representation that should be used to transmit the selection to the requester (the second column of Table 2 of the ICCCM), and defaults to **STRING**. If *format* is **STRING**, the selection is transmitted as 8-bit ASCII characters. If *format* is **ATOM**, then the *data* is divided into fields separated by white space; each field is converted to its atom value, and the 32-bit atom value is transmitted instead of the atom name. For any other *format*, *data* is divided into fields separated by white space and each field is converted to a 32-bit integer; an array of integers is transmitted to the selection requester. Note that strings passed to **clipboard append** are concatenated before conversion, so the caller must take care to ensure appropriate spacing across string boundaries. All items appended to the clipboard with the same *type* must have the same *format*.

The *format* argument is needed only for compatibility with clipboard requesters that do not use Tk. If the Tk toolkit is being used to retrieve the **CLIPBOARD** selection then the value is converted back to a string at the requesting end, so *format* is irrelevant.

A **--** argument may be specified to mark the end of options: the next argument will always be used as *data*. This feature may be convenient if, for example, *data* starts with a **-**.

clipboard clear ?-displayof *window*?
Claims ownership of the clipboard on *window*'s display and removes any previous contents. *Window* defaults to ".". Returns an empty string.

clipboard get ?-displayof *window*? ?-type *type*?
Retrieve data from the clipboard on *window*'s display. *Window* defaults to ".". *Type* specifies the form in which the data is to be returned and should be an atom name such as **STRING** or **FILE_NAME**. *Type* defaults to **STRING**. This command is equivalent to "selection get **-selection CLIPBOARD**".

Note that on modern X11 systems, the most useful type to retrieve for transferred strings is not **STRING**, but rather **UTF8_STRING**.

Examples

Get the current contents of the clipboard.

```
if {[catch {clipboard get} contents]} {
    # There were no clipboard contents at all
}
```

Set the clipboard to contain a fixed string.

```
clipboard clear
clipboard append "some fixed string"
```

You can put custom data into the clipboard by using a custom **-type** option. This is not necessarily portable, but can be very useful. The method of passing Tcl scripts this way is effective, but should be mixed with safe interpreters in production code.

```
# This is a very simple canvas serializer;
# it produces a script that recreates the item(s) when executed
proc getItemConfig {canvas tag} {
   set script {}
   foreach item [$canvas find withtag $tag] {
      append script {$canvas create } [$canvas type $item]
      append script { } [$canvas coords $item] { }
      foreach config [$canvas itemconf $item] {
         lassign $config name - - value
         append script [list $name $value] { }
      }
      append script \n
   }
```

```
   return [string trim $script]
}

# Set up a binding on a canvas to cut and paste an item
set c [canvas .c]
pack $c
$c create text 150 30 -text "cut and paste me"
bind $c <<Cut>> {
   clipboard clear
   clipboard append -type TkCanvasItem \
         [getItemConfig %W current]
   # Delete because this is cut, not copy.
   %W delete current
}
bind $c <<Paste>> {
   catch {
      set canvas %W
      eval [clipboard get -type TkCanvasItem]
   }
}
```

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

colors

Keywords: colors, color, option

Name

colors - symbolic color names recognized by Tk

Description

Tk recognizes many symbolic color names (e.g., **red**) when specifying colors. The symbolic names recognized by Tk and their 8-bit-per-channel RGB values are:

Name **Red Green Blue**
alice blue 240 248 255
AliceBlue 240 248 255
antique white 250 235 215
AntiqueWhite 250 235 215
AntiqueWhite1 255 239 219
AntiqueWhite2 238 223 204
AntiqueWhite3 205 192 176
AntiqueWhite4 139 131 120
aqua 0 255 255
aquamarine 127 255 212
aquamarine1 127 255 212
aquamarine2 118 238 198

aquamarine3 102 205 170
aquamarine4 69 139 116
azure 240 255 255
azure1 240 255 255
azure2 224 238 238
azure3 193 205 205
azure4 131 139 139
beige 245 245 220
bisque 255 228 196
bisque1 255 228 196
bisque2 238 213 183
bisque3 205 183 158
bisque4 139 125 107
black 0 0 0
blanched almond 255 235 205
BlanchedAlmond 255 235 205
blue 0 0 255
blue violet 138 43 226
blue1 0 0 255
blue2 0 0 238
blue3 0 0 205
blue4 0 0 139
BlueViolet 138 43 226
brown 165 42 42
brown1 255 64 64
brown2 238 59 59
brown3 205 51 51
brown4 139 35 35
burlywood 222 184 135
burlywood1 255 211 155
burlywood2 238 197 145
burlywood3 205 170 125
burlywood4 139 115 85
cadet blue 95 158 160
CadetBlue 95 158 160
CadetBlue1 152 245 255
CadetBlue2 142 229 238
CadetBlue3 122 197 205
CadetBlue4 83 134 139
chartreuse 127 255 0
chartreuse1 127 255 0
chartreuse2 118 238 0
chartreuse3 102 205 0
chartreuse4 69 139 0
chocolate 210 105 30
chocolate1 255 127 36

chocolate2 238 118 33
chocolate3 205 102 29
chocolate4 139 69 19
coral 255 127 80
coral1 255 114 86
coral2 238 106 80
coral3 205 91 69
coral4 139 62 47
cornflower blue 100 149 237
CornflowerBlue 100 149 237
cornsilk 255 248 220
cornsilk1 255 248 220
cornsilk2 238 232 205
cornsilk3 205 200 177
cornsilk4 139 136 120
crimson 220 20 60
cyan 0 255 255
cyan1 0 255 255
cyan2 0 238 238
cyan3 0 205 205
cyan4 0 139 139
dark blue 0 0 139
dark cyan 0 139 139
dark goldenrod 184 134 11
dark gray 169 169 169
dark green 0 100 0
dark grey 169 169 169
dark khaki 189 183 107
dark magenta 139 0 139
dark olive green 85 107 47
dark orange 255 140 0
dark orchid 153 50 204
dark red 139 0 0
dark salmon 233 150 122
dark sea green 143 188 143
dark slate blue 72 61 139
dark slate gray 47 79 79
dark slate grey 47 79 79
dark turquoise 0 206 209
dark violet 148 0 211
DarkBlue 0 0 139
DarkCyan 0 139 139
DarkGoldenrod 184 134 11
DarkGoldenrod1 255 185 15
DarkGoldenrod2 238 173 14
DarkGoldenrod3 205 149 12

DarkGoldenrod4 139 101 8
DarkGray 169 169 169
DarkGreen 0 100 0
DarkGrey 169 169 169
DarkKhaki 189 183 107
DarkMagenta 139 0 139
DarkOliveGreen 85 107 47
DarkOliveGreen1 202 255 112
DarkOliveGreen2 188 238 104
DarkOliveGreen3 162 205 90
DarkOliveGreen4 110 139 61
DarkOrange 255 140 0
DarkOrange1 255 127 0
DarkOrange2 238 118 0
DarkOrange3 205 102 0
DarkOrange4 139 69 0
DarkOrchid 153 50 204
DarkOrchid1 191 62 255
DarkOrchid2 178 58 238
DarkOrchid3 154 50 205
DarkOrchid4 104 34 139
DarkRed 139 0 0
DarkSalmon 233 150 122
DarkSeaGreen 143 188 143
DarkSeaGreen1 193 255 193
DarkSeaGreen2 180 238 180
DarkSeaGreen3 155 205 155
DarkSeaGreen4 105 139 105
DarkSlateBlue 72 61 139
DarkSlateGray 47 79 79
DarkSlateGray1 151 255 255
DarkSlateGray2 141 238 238
DarkSlateGray3 121 205 205
DarkSlateGray4 82 139 139
DarkSlateGrey 47 79 79
DarkTurquoise 0 206 209
DarkViolet 148 0 211
deep pink 255 20 147
deep sky blue 0 191 255
DeepPink 255 20 147
DeepPink1 255 20 147
DeepPink2 238 18 137
DeepPink3 205 16 118
DeepPink4 139 10 80
DeepSkyBlue 0 191 255
DeepSkyBlue1 0 191 255

DeepSkyBlue2 0 178 238
DeepSkyBlue3 0 154 205
DeepSkyBlue4 0 104 139
dim gray 105 105 105
dim grey 105 105 105
DimGray 105 105 105
DimGrey 105 105 105
dodger blue 30 144 255
DodgerBlue 30 144 255
DodgerBlue1 30 144 255
DodgerBlue2 28 134 238
DodgerBlue3 24 116 205
DodgerBlue4 16 78 139
firebrick 178 34 34
firebrick1 255 48 48
firebrick2 238 44 44
firebrick3 205 38 38
firebrick4 139 26 26
floral white 255 250 240
FloralWhite 255 250 240
forest green 34 139 34
ForestGreen 34 139 34
fuchsia 255 0 255
gainsboro 220 220 220
ghost white 248 248 255
GhostWhite 248 248 255
gold 255 215 0
gold1 255 215 0
gold2 238 201 0
gold3 205 173 0
gold4 139 117 0
goldenrod 218 165 32
goldenrod1 255 193 37
goldenrod2 238 180 34
goldenrod3 205 155 29
goldenrod4 139 105 20
gray 128 128 128
gray0 0 0 0
gray1 3 3 3
gray2 5 5 5
gray3 8 8 8
gray4 10 10 10
gray5 13 13 13
gray6 15 15 15
gray7 18 18 18
gray8 20 20 20

```
gray9  23 23 23
gray10 26 26 26
gray11 28 28 28
gray12 31 31 31
gray13 33 33 33
gray14 36 36 36
gray15 38 38 38
gray16 41 41 41
gray17 43 43 43
gray18 46 46 46
gray19 48 48 48
gray20 51 51 51
gray21 54 54 54
gray22 56 56 56
gray23 59 59 59
gray24 61 61 61
gray25 64 64 64
gray26 66 66 66
gray27 69 69 69
gray28 71 71 71
gray29 74 74 74
gray30 77 77 77
gray31 79 79 79
gray32 82 82 82
gray33 84 84 84
gray34 87 87 87
gray35 89 89 89
gray36 92 92 92
gray37 94 94 94
gray38 97 97 97
gray39 99 99 99
gray40 102 102   102
gray41 105 105   105
gray42 107 107   107
gray43 110 110   110
gray44 112 112   112
gray45 115 115   115
gray46 117 117   117
gray47 120 120   120
gray48 122 122   122
gray49 125 125   125
gray50 127 127   127
gray51 130 130   130
gray52 133 133   133
gray53 135 135   135
gray54 138 138   138
```

```
gray55  140 140  140
gray56  143 143  143
gray57  145 145  145
gray58  148 148  148
gray59  150 150  150
gray60  153 153  153
gray61  156 156  156
gray62  158 158  158
gray63  161 161  161
gray64  163 163  163
gray65  166 166  166
gray66  168 168  168
gray67  171 171  171
gray68  173 173  173
gray69  176 176  176
gray70  179 179  179
gray71  181 181  181
gray72  184 184  184
gray73  186 186  186
gray74  189 189  189
gray75  191 191  191
gray76  194 194  194
gray77  196 196  196
gray78  199 199  199
gray79  201 201  201
gray80  204 204  204
gray81  207 207  207
gray82  209 209  209
gray83  212 212  212
gray84  214 214  214
gray85  217 217  217
gray86  219 219  219
gray87  222 222  222
gray88  224 224  224
gray89  227 227  227
gray90  229 229  229
gray91  232 232  232
gray92  235 235  235
gray93  237 237  237
gray94  240 240  240
gray95  242 242  242
gray96  245 245  245
gray97  247 247  247
gray98  250 250  250
gray99  252 252  252
gray100 255 255   255
```

green 0 128 0
green yellow 173 255 47
green1 0 255 0
green2 0 238 0
green3 0 205 0
green4 0 139 0
GreenYellow 173 255 47
grey 128 128 128
grey0 0 0 0
grey1 3 3 3
grey2 5 5 5
grey3 8 8 8
grey4 10 10 10
grey5 13 13 13
grey6 15 15 15
grey7 18 18 18
grey8 20 20 20
grey9 23 23 23
grey10 26 26 26
grey11 28 28 28
grey12 31 31 31
grey13 33 33 33
grey14 36 36 36
grey15 38 38 38
grey16 41 41 41
grey17 43 43 43
grey18 46 46 46
grey19 48 48 48
grey20 51 51 51
grey21 54 54 54
grey22 56 56 56
grey23 59 59 59
grey24 61 61 61
grey25 64 64 64
grey26 66 66 66
grey27 69 69 69
grey28 71 71 71
grey29 74 74 74
grey30 77 77 77
grey31 79 79 79
grey32 82 82 82
grey33 84 84 84
grey34 87 87 87
grey35 89 89 89
grey36 92 92 92
grey37 94 94 94

```
grey38  97  97  97
grey39  99  99  99
grey40  102 102   102
grey41  105 105   105
grey42  107 107   107
grey43  110 110   110
grey44  112 112   112
grey45  115 115   115
grey46  117 117   117
grey47  120 120   120
grey48  122 122   122
grey49  125 125   125
grey50  127 127   127
grey51  130 130   130
grey52  133 133   133
grey53  135 135   135
grey54  138 138   138
grey55  140 140   140
grey56  143 143   143
grey57  145 145   145
grey58  148 148   148
grey59  150 150   150
grey60  153 153   153
grey61  156 156   156
grey62  158 158   158
grey63  161 161   161
grey64  163 163   163
grey65  166 166   166
grey66  168 168   168
grey67  171 171   171
grey68  173 173   173
grey69  176 176   176
grey70  179 179   179
grey71  181 181   181
grey72  184 184   184
grey73  186 186   186
grey74  189 189   189
grey75  191 191   191
grey76  194 194   194
grey77  196 196   196
grey78  199 199   199
grey79  201 201   201
grey80  204 204   204
grey81  207 207   207
grey82  209 209   209
grey83  212 212   212
```

grey84 214 214 214
grey85 217 217 217
grey86 219 219 219
grey87 222 222 222
grey88 224 224 224
grey89 227 227 227
grey90 229 229 229
grey91 232 232 232
grey92 235 235 235
grey93 237 237 237
grey94 240 240 240
grey95 242 242 242
grey96 245 245 245
grey97 247 247 247
grey98 250 250 250
grey99 252 252 252
grey100 255 255 255
honeydew 240 255 240
honeydew1 240 255 240
honeydew2 224 238 224
honeydew3 193 205 193
honeydew4 131 139 131
hot pink 255 105 180
HotPink 255 105 180
HotPink1 255 110 180
HotPink2 238 106 167
HotPink3 205 96 144
HotPink4 139 58 98
indian red 205 92 92
IndianRed 205 92 92
IndianRed1 255 106 106
IndianRed2 238 99 99
IndianRed3 205 85 85
IndianRed4 139 58 58
indigo 75 0 130
ivory 255 255 240
ivory1 255 255 240
ivory2 238 238 224
ivory3 205 205 193
ivory4 139 139 131
khaki 240 230 140
khaki1 255 246 143
khaki2 238 230 133
khaki3 205 198 115
khaki4 139 134 78
lavender 230 230 250

lavender blush 255 240 245
LavenderBlush 255 240 245
LavenderBlush1 255 240 245
LavenderBlush2 238 224 229
LavenderBlush3 205 193 197
LavenderBlush4 139 131 134
lawn green 124 252 0
LawnGreen 124 252 0
lemon chiffon 255 250 205
LemonChiffon 255 250 205
LemonChiffon1 255 250 205
LemonChiffon2 238 233 191
LemonChiffon3 205 201 165
LemonChiffon4 139 137 112
light blue 173 216 230
light coral 240 128 128
light cyan 224 255 255
light goldenrod 238 221 130
light goldenrod yellow 250 250 210
light gray 211 211 211
light green 144 238 144
light grey 211 211 211
light pink 255 182 193
light salmon 255 160 122
light sea green 32 178 170
light sky blue 135 206 250
light slate blue 132 112 255
light slate gray 119 136 153
light slate grey 119 136 153
light steel blue 176 196 222
light yellow 255 255 224
LightBlue 173 216 230
LightBlue1 191 239 255
LightBlue2 178 223 238
LightBlue3 154 192 205
LightBlue4 104 131 139
LightCoral 240 128 128
LightCyan 224 255 255
LightCyan1 224 255 255
LightCyan2 209 238 238
LightCyan3 180 205 205
LightCyan4 122 139 139
LightGoldenrod 238 221 130
LightGoldenrod1 255 236 139
LightGoldenrod2 238 220 130
LightGoldenrod3 205 190 112

LightGoldenrod4 139 129 76
LightGoldenrodYellow 250 250 210
LightGray 211 211 211
LightGreen 144 238 144
LightGrey 211 211 211
LightPink 255 182 193
LightPink1 255 174 185
LightPink2 238 162 173
LightPink3 205 140 149
LightPink4 139 95 101
LightSalmon 255 160 122
LightSalmon1 255 160 122
LightSalmon2 238 149 114
LightSalmon3 205 129 98
LightSalmon4 139 87 66
LightSeaGreen 32 178 170
LightSkyBlue 135 206 250
LightSkyBlue1 176 226 255
LightSkyBlue2 164 211 238
LightSkyBlue3 141 182 205
LightSkyBlue4 96 123 139
LightSlateBlue 132 112 255
LightSlateGray 119 136 153
LightSlateGrey 119 136 153
LightSteelBlue 176 196 222
LightSteelBlue1 202 225 255
LightSteelBlue2 188 210 238
LightSteelBlue3 162 181 205
LightSteelBlue4 110 123 139
LightYellow 255 255 224
LightYellow1 255 255 224
LightYellow2 238 238 209
LightYellow3 205 205 180
LightYellow4 139 139 122
lime 0 255 0
lime green 50 205 50
LimeGreen 50 205 50
linen 250 240 230
magenta 255 0 255
magenta1 255 0 255
magenta2 238 0 238
magenta3 205 0 205
magenta4 139 0 139
maroon 128 0 0
maroon1 255 52 179
maroon2 238 48 167

maroon3 205 41 144
maroon4 139 28 98
medium aquamarine 102 205 170
medium blue 0 0 205
medium orchid 186 85 211
medium purple 147 112 219
medium sea green 60 179 113
medium slate blue 123 104 238
medium spring green 0 250 154
medium turquoise 72 209 204
medium violet red 199 21 133
MediumAquamarine 102 205 170
MediumBlue 0 0 205
MediumOrchid 186 85 211
MediumOrchid1 224 102 255
MediumOrchid2 209 95 238
MediumOrchid3 180 82 205
MediumOrchid4 122 55 139
MediumPurple 147 112 219
MediumPurple1 171 130 255
MediumPurple2 159 121 238
MediumPurple3 137 104 205
MediumPurple4 93 71 139
MediumSeaGreen 60 179 113
MediumSlateBlue 123 104 238
MediumSpringGreen 0 250 154
MediumTurquoise 72 209 204
MediumVioletRed 199 21 133
midnight blue 25 25 112
MidnightBlue 25 25 112
mint cream 245 255 250
MintCream 245 255 250
misty rose 255 228 225
MistyRose 255 228 225
MistyRose1 255 228 225
MistyRose2 238 213 210
MistyRose3 205 183 181
MistyRose4 139 125 123
moccasin 255 228 181
navajo white 255 222 173
NavajoWhite 255 222 173
NavajoWhite1 255 222 173
NavajoWhite2 238 207 161
NavajoWhite3 205 179 139
NavajoWhite4 139 121 94
navy 0 0 128

navy blue 0 0 128
NavyBlue 0 0 128
old lace 253 245 230
OldLace 253 245 230
olive 128 128 0
olive drab 107 142 35
OliveDrab 107 142 35
OliveDrab1 192 255 62
OliveDrab2 179 238 58
OliveDrab3 154 205 50
OliveDrab4 105 139 34
orange 255 165 0
orange red 255 69 0
orange1 255 165 0
orange2 238 154 0
orange3 205 133 0
orange4 139 90 0
OrangeRed 255 69 0
OrangeRed1 255 69 0
OrangeRed2 238 64 0
OrangeRed3 205 55 0
OrangeRed4 139 37 0
orchid 218 112 214
orchid1 255 131 250
orchid2 238 122 233
orchid3 205 105 201
orchid4 139 71 137
pale goldenrod 238 232 170
pale green 152 251 152
pale turquoise 175 238 238
pale violet red 219 112 147
PaleGoldenrod 238 232 170
PaleGreen 152 251 152
PaleGreen1 154 255 154
PaleGreen2 144 238 144
PaleGreen3 124 205 124
PaleGreen4 84 139 84
PaleTurquoise 175 238 238
PaleTurquoise1 187 255 255
PaleTurquoise2 174 238 238
PaleTurquoise3 150 205 205
PaleTurquoise4 102 139 139
PaleVioletRed 219 112 147
PaleVioletRed1 255 130 171
PaleVioletRed2 238 121 159
PaleVioletRed3 205 104 127

PaleVioletRed4 139 71 93
papaya whip 255 239 213
PapayaWhip 255 239 213
peach puff 255 218 185
PeachPuff 255 218 185
PeachPuff1 255 218 185
PeachPuff2 238 203 173
PeachPuff3 205 175 149
PeachPuff4 139 119 101
peru 205 133 63
pink 255 192 203
pink1 255 181 197
pink2 238 169 184
pink3 205 145 158
pink4 139 99 108
plum 221 160 221
plum1 255 187 255
plum2 238 174 238
plum3 205 150 205
plum4 139 102 139
powder blue 176 224 230
PowderBlue 176 224 230
purple 128 0 128
purple1 155 48 255
purple2 145 44 238
purple3 125 38 205
purple4 85 26 139
red 255 0 0
red1 255 0 0
red2 238 0 0
red3 205 0 0
red4 139 0 0
rosy brown 188 143 143
RosyBrown 188 143 143
RosyBrown1 255 193 193
RosyBrown2 238 180 180
RosyBrown3 205 155 155
RosyBrown4 139 105 105
royal blue 65 105 225
RoyalBlue 65 105 225
RoyalBlue1 72 118 255
RoyalBlue2 67 110 238
RoyalBlue3 58 95 205
RoyalBlue4 39 64 139
saddle brown 139 69 19
SaddleBrown 139 69 19

salmon 250 128 114
salmon1 255 140 105
salmon2 238 130 98
salmon3 205 112 84
salmon4 139 76 57
sandy brown 244 164 96
SandyBrown 244 164 96
sea green 46 139 87
SeaGreen 46 139 87
SeaGreen1 84 255 159
SeaGreen2 78 238 148
SeaGreen3 67 205 128
SeaGreen4 46 139 87
seashell 255 245 238
seashell1 255 245 238
seashell2 238 229 222
seashell3 205 197 191
seashell4 139 134 130
sienna 160 82 45
sienna1 255 130 71
sienna2 238 121 66
sienna3 205 104 57
sienna4 139 71 38
silver 192 192 192
sky blue 135 206 235
SkyBlue 135 206 235
SkyBlue1 135 206 255
SkyBlue2 126 192 238
SkyBlue3 108 166 205
SkyBlue4 74 112 139
slate blue 106 90 205
slate gray 112 128 144
slate grey 112 128 144
SlateBlue 106 90 205
SlateBlue1 131 111 255
SlateBlue2 122 103 238
SlateBlue3 105 89 205
SlateBlue4 71 60 139
SlateGray 112 128 144
SlateGray1 198 226 255
SlateGray2 185 211 238
SlateGray3 159 182 205
SlateGray4 108 123 139
SlateGrey 112 128 144
snow 255 250 250
snow1 255 250 250

snow2 238 233 233
snow3 205 201 201
snow4 139 137 137
spring green 0 255 127
SpringGreen 0 255 127
SpringGreen1 0 255 127
SpringGreen2 0 238 118
SpringGreen3 0 205 102
SpringGreen4 0 139 69
steel blue 70 130 180
SteelBlue 70 130 180
SteelBlue1 99 184 255
SteelBlue2 92 172 238
SteelBlue3 79 148 205
SteelBlue4 54 100 139
tan 210 180 140
tan1 255 165 79
tan2 238 154 73
tan3 205 133 63
tan4 139 90 43
teal 0 128 128
thistle 216 191 216
thistle1 255 225 255
thistle2 238 210 238
thistle3 205 181 205
thistle4 139 123 139
tomato 255 99 71
tomato1 255 99 71
tomato2 238 92 66
tomato3 205 79 57
tomato4 139 54 38
turquoise 64 224 208
turquoise1 0 245 255
turquoise2 0 229 238
turquoise3 0 197 205
turquoise4 0 134 139
violet 238 130 238
violet red 208 32 144
VioletRed 208 32 144
VioletRed1 255 62 150
VioletRed2 238 58 140
VioletRed3 205 50 120
VioletRed4 139 34 82
wheat 245 222 179
wheat1 255 231 186
wheat2 238 216 174

wheat3 205 186 150
wheat4 139 126 102
white 255 255 255
white smoke 245 245 245
WhiteSmoke 245 245 245
yellow 255 255 0
yellow green 154 205 50
yellow1 255 255 0
yellow2 238 238 0
yellow3 205 205 0
yellow4 139 139 0
YellowGreen 154 205 50

Portability issues

macOS

On macOS, the following additional system colors are available. This first group contains all of the HIBrush colors available in the HIToolbox library. Note that on macOS 10.14 (Mojave) and later these colors are unlikely to match the color actually used for the purpose suggested by the color name.

systemActiveAreaFill
systemAlertBackgroundActive
systemAlertBackgroundInactive
systemAlternatePrimaryHighlightColor
systemAppleGuideCoachmark
systemBevelActiveDark
systemBevelActiveLight
systemBevelInactiveDark
systemBevelInactiveLight
systemBlack
systemButtonActiveDarkHighlight
systemButtonActiveDarkShadow
systemButtonActiveLightHighlight
systemButtonActiveLightShadow
systemButtonFace
systemButtonFaceActive
systemButtonFaceInactive
systemButtonFacePressed
systemButtonFrame
systemButtonFrameActive
systemButtonFrameInactive
systemButtonInactiveDarkHighlight
systemButtonInactiveDarkShadow
systemButtonInactiveLightHighlight
systemButtonInactiveLightShadow
systemButtonPressedDarkHighlight
systemButtonPressedDarkShadow

systemButtonPressedLightHighlight
systemButtonPressedLightShadow
systemChasingArrows
systemDialogBackgroundActive
systemDialogBackgroundInactive
systemDocumentWindowBackground
systemDragHilite
systemDrawerBackground
systemFinderWindowBackground
systemFocusHighlight
systemHighlight
systemHighlightAlternate
systemHighlightSecondary
systemIconLabelBackground
systemIconLabelBackgroundSelected
systemListViewBackground
systemListViewColumnDivider
systemListViewEvenRowBackground
systemListViewOddRowBackground
systemListViewSeparator
systemListViewSortColumnBackground
systemMenu
systemMenuActive
systemMenuBackground
systemMenuBackgroundSelected
systemModelessDialogBackgroundActive
systemModelessDialogBackgroundInactive
systemMovableModalBackground
systemNotificationWindowBackground
systemPopupArrowActive
systemPopupArrowInactive
systemPopupArrowPressed
systemPrimaryHighlightColor
systemScrollBarDelimiterActive
systemScrollBarDelimiterInactive
systemSecondaryHighlightColor
systemSelectedTabTextColor
systemSheetBackground
systemSheetBackgroundOpaque
systemSheetBackgroundTransparent
systemStaticAreaFill
systemToolbarBackground
systemTransparent
systemUtilityWindowBackgroundActive
systemUtilityWindowBackgroundInactive
systemWhite

systemWindowBody

Tk supports all of the NSColors in the macOS System ColorList. The convention for naming these colors is that the Tk name is generated by capitalizing the macOS name and adding the prefix "system". On macOS 10.14 (Mojave) and later many of these "semantic" colors will appear differently depending on whether the NSWindow in which they are used has the Aqua or DarkAqua appearance. The System ColorList differs between releases of macOS and some colors, such as systemLinkColor and systemControlAccentColor, are simulated on older systems which did not provide them. All of the colors below are available on all supported macOS releases, but newer systems will support additional colors.

systemControlAccentColor
systemControlTextColor
systemDisabledControlTextColor
systemLabelColor
systemLinkColor
systemPlaceholderTextColor
systemSelectedTextBackgroundColor
systemSelectedTextColor
systemSeparatorColor
systemTextBackgroundColor
systemTextColor

The numbered systemWindowBackgroundColors below are used in the ttk::notebook and ttk::labelframe widgets to provide a contrasting background. Each numbered color contrasts with its predecessor.

systemWindowBackgroundColor
systemWindowBackgroundColor1
systemWindowBackgroundColor2
systemWindowBackgroundColor3
systemWindowBackgroundColor4
systemWindowBackgroundColor5
systemWindowBackgroundColor6
systemWindowBackgroundColor7

Windows

On Windows, the following additional system colors are available (note that the actual color values depend on the currently active OS theme):

system3dDarkShadow systemHighlight
system3dLight systemHighlightText
systemActiveBorder systemInactiveBorder
systemActiveCaption systemInactiveCaption
systemAppWorkspace systemInactiveCaptionText
systemBackground systemInfoBackground
systemButtonFace systemInfoText
systemButtonHighlight systemMenu
systemButtonShadow systemMenuText

systemButtonText systemScrollbar
systemCaptionText systemWindow
systemDisabledText systemWindowFrame
systemGrayText systemWindowText

Copyright (c) 1998-2000 Scriptics Corporation.
Copyright (c) 2003 ActiveState Corporation.
Copyright (c) 2006-2007 Daniel A. Steffen das@users.sourceforge.net
Copyright (c) 2008 Donal K. Fellows

console

Keywords: console, interpreter, window, interactive, output channels

Name

console - Control the console on systems without a real console

Synopsis

console *subcommand* ?*arg* ...?

Description

The console window is a replacement for a real console to allow input and output on the standard I/O channels on platforms that do not have a real console. It is implemented as a separate interpreter with the Tk toolkit loaded, and control over this interpreter is given through the **console** command. The behaviour of the console window is defined mainly through the contents of the *console.tcl* file in the Tk library. Except for TkAqua, this command is not available when Tk is loaded into a tclsh interpreter with "**package require Tk**", as a conventional terminal is expected to be present in that case. In TkAqua, this command is disabled when there is a startup script and stdin is **/dev/null** (as is the case e.g. when a bundled application embedding Tk is started by the macOS Launcher). To enable the command in that case, define the environment variable **TK_CONSOLE**. This can be done by modifying the Info.plist file by adding the LSEnvironment key to the main dict and setting its value to be a dict with the key **TK_CONSOLE**.

console eval *script*
Evaluate the *script* argument as a Tcl script in the console interpreter. The normal interpreter is accessed through the **consoleinterp** command in the console interpreter.

console hide
Hide the console window from view. Precisely equivalent to withdrawing the**.** window in the console interpreter.

console show
Display the console window. Precisely equivalent to deiconifying the**.** window in the console interpreter.

console title ?*string*?
Query or modify the title of the console window. If *string* is not specified, queries the title of the console window, and sets the title of the console window to *string* otherwise. Precisely equivalent to using the wm title command in the console interpreter.

Access to the main interpreter

The **consoleinterp** command in the console interpreter allows scripts to be evaluated in the main interpreter. It supports two subcommands: **eval** and **record**.

consoleinterp eval *script*
Evaluates *script* as a Tcl script at the global level in the main interpreter.

consoleinterp record *script*
Records and evaluates *script* as a Tcl script at the global level in the main interpreter as if *script* had been typed in at the console.

Additional trap calls

There are several additional commands in the console interpreter that are called in response to activity in the main interpreter. *These are documented here for completeness only; they form part of the internal implementation of the console and are likely to change or be modified without warning.*

Output to the console from the main interpreter via the stdout and stderr channels is handled by invoking the **tk::ConsoleOutput** command in the console interpreter with two arguments. The first argument is the name of the channel being written to, and the second argument is the string being written to the channel (after encoding and end-of-line translation processing has been performed.)

When the**.** window of the main interpreter is destroyed, the **tk::ConsoleExit** command in the console interpreter is called (assuming the console interpreter has not already been deleted itself, that is.)

Default bindings

The default script creates a console window (implemented using a text widget) that has the following behaviour:

1. Pressing the tab key inserts a TAB character (as defined by the Tcl \t escape.)
2. Pressing the return key causes the current line (if complete by the rules of info complete) to be passed to the main interpreter for evaluation.
3. Pressing the delete key deletes the selected text (if any text is selected) or the character to the right of the cursor (if not at the end of the line.)
4. Pressing the backspace key deletes the selected text (if any text is selected) or the character to the left of the cursor (of not at the start of the line.)
5. Pressing either Control+A or the home key causes the cursor to go to the start of the line (but after the prompt, if a prompt is present on the line.)
6. Pressing either Control+E or the end key causes the cursor to go to the end of the line.
7. Pressing either Control+P or the up key causes the previous entry in the command history to be selected.
8. Pressing either Control+N or the down key causes the next entry in the command history to be selected.
9. Pressing either Control+B or the left key causes the cursor to move one character backward as long as the cursor is not at the prompt.
10. Pressing either Control+F or the right key causes the cursor to move one character forward.
11. Pressing F9 rebuilds the console window by destroying all its children and reloading the Tcl script that defined the console's behaviour.

Most other behaviour is the same as a conventional text widget except for the way that the <> event is handled identically to the <> event.

Example

Not all platforms have the **console** command, so debugging code often has the following code fragment in it so output produced by puts can be seen while during development:

```
catch {console show}
```

Copyright (c) 2001 Donal K. Fellows

cursors

Keywords: cursors, cursor, option

Name

cursors - mouse cursors available in Tk

Description

The **-cursor** widget option allows a Tk programmer to change the mouse cursor for a particular widget. The cursor names recognized by Tk on all platforms are:

```
X_cursor
arrow
based_arrow_down
based_arrow_up
boat
bogosity
bottom_left_corner
bottom_right_corner
bottom_side
bottom_tee
box_spiral
center_ptr
circle
clock
coffee_mug
cross
cross_reverse
crosshair
diamond_cross
dot
dotbox
double_arrow
draft_large
draft_small
draped_box
exchange
fleur
```

```
gobbler
gumby
hand1
hand2
heart
icon
iron_cross
left_ptr
left_side
left_tee
leftbutton
ll_angle
lr_angle
man
middlebutton
mouse
none
pencil
pirate
plus
question_arrow
right_ptr
right_side
right_tee
rightbutton
rtl_logo
sailboat
sb_down_arrow
sb_h_double_arrow
sb_left_arrow
sb_right_arrow
sb_up_arrow
sb_v_double_arrow
shuttle
sizing
spider
spraycan
star
target
tcross
top_left_arrow
top_left_corner
top_right_corner
top_side
top_tee
trek
```

```
ul_angle
umbrella
ur_angle
watch
xterm
```

The **none** cursor can be specified to eliminate the cursor.

Portability issues

Windows

On Windows systems, the following cursors are mapped to native cursors:

```
arrow
center_ptr
crosshair
fleur
ibeam
icon
none
sb_h_double_arrow
sb_v_double_arrow
watch
xterm
```

And the following additional cursors are available:

```
no
starting
size
size_ne_sw
size_ns
size_nw_se
size_we
uparrow
wait
```

macOS

On macOS, the following cursors are mapped to native cursors:

```
arrow
top_left_arrow
left_ptr
cross
crosshair
tcross
ibeam
none
xterm
```

And the following additional native cursors are available:

```
copyarrow
aliasarrow
contextualmenuarrow
movearrow
text
cross-hair
hand
openhand
closedhand
fist
pointinghand
resize
resizeleft
resizeright
resizeleftright
resizeup
resizedown
resizeupdown
resizebottomleft
resizetopleft
resizebottomright
resizetopright
notallowed
poof
wait
countinguphand
countingdownhand
countingupanddownhand
spinning
help
bucket
cancel
eyedrop
eyedrop-full
zoom-in
zoom-out
```

Copyright (c) 1998-2000 Scriptics Corporation.
Copyright (c) 2006-2007 Daniel A. Steffen das@users.sourceforge.net

destroy

Keywords: destroy, application, window

Name

destroy - Destroy one or more windows

Synopsis

destroy ?*window window* ...?

Description

This command deletes the windows given by the *window* arguments, plus all of their descendants. If a *window* "." is deleted then all windows will be destroyed and the application will (normally) exit. The *window*s are destroyed in order, and if an error occurs in destroying a window the command aborts without destroying the remaining windows. No error is returned if *window* does not exist.

Example

Destroy all checkbuttons that are direct children of the given widget:

```
proc killCheckbuttonChildren {parent} {
   foreach w [winfo children $parent] {
      if {[winfo class $w] eq "Checkbutton"} {
         destroy $w
      }
   }
}
```

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

dialog

Keywords: dialog, bitmap, modal

Name

tk_dialog - Create modal dialog and wait for response

Synopsis

tk_dialog *window title text bitmap default string string ...*

Description

This procedure is part of the Tk script library. It is largely *deprecated* by the tk_messageBox. Its arguments describe a dialog box:

window
Name of top-level window to use for dialog. Any existing window by this name is destroyed.

title
Text to appear in the window manager's title bar for the dialog.

text
Message to appear in the top portion of the dialog box.

bitmap
If non-empty, specifies a bitmap (in a form suitable for Tk_GetBitmap) to display in the top portion of the dialog, to the left of the text. If this is an empty string then no bitmap is displayed in the dialog.

default

If this is an integer greater than or equal to zero, then it gives the index of the button that is to be the default button for the dialog (0 for the leftmost button, and so on). If less than zero or an empty string then there will not be any default button.

string
There will be one button for each of these arguments. Each *string* specifies text to display in a button, in order from left to right.

After creating a dialog box, **tk_dialog** waits for the user to select one of the buttons either by clicking on the button with the mouse or by typing return to invoke the default button (if any). Then it returns the index of the selected button: 0 for the leftmost button, 1 for the button next to it, and so on. If the dialog's window is destroyed before the user selects one of the buttons, then -1 is returned.

While waiting for the user to respond, **tk_dialog** sets a local grab. This prevents the user from interacting with the application in any way except to invoke the dialog box.

Example

```
set reply [tk_dialog .foo "The Title" "Do you want to say yes?" \
        questhead 0 Yes No "I'm not sure"]
```

Copyright (c) 1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

entry

Keywords: entry, widget

Name

entry - Create and manipulate 'entry' one-line text entry widgets

Synopsis

entry *pathName* ?*options*?

Standard options

-background, **-highlightthickness**, **-selectbackground**, **-borderwidth**, **-insertbackground**, **-selectborderwidth**, **-cursor**, **-insertborderwidth**, **-selectforeground**, **-exportselection**, **-insertofftime**, **-takefocus**, **-font**, **-insertontime**, **-textvariable**, **-foreground**, **-insertwidth**, **-xscrollcommand**, **-highlightbackground**, **-justify**, **-highlightcolor**, **-relief**

See the options manual entry for details on the standard options.

Widget-specific options

-disabledbackground (disabledBackground/DisabledBackground)
Specifies the background color to use when the entry is disabled. If this option is the empty string, the normal background color is used.

-disabledforeground (disabledForeground/DisabledForeground)
Specifies the foreground color to use when the entry is disabled. If this option is the empty string, the normal foreground color is used.

-invalidcommand or -invcmd (invalidCommand/InvalidCommand)

Specifies a script to eval when **-validatecommand** returns 0. Setting it to {} disables this feature (the default). The best use of this option is to set it to *bell*. See Validation below for more information.

-readonlybackground (readonlyBackground/ReadonlyBackground)
Specifies the background color to use when the entry is readonly. If this option is the empty string, the normal background color is used.

-show (show/Show)
If this option is specified, then the true contents of the entry are not displayed in the window. Instead, each character in the entry's value will be displayed as the first character in the value of this option, such as "*". This is useful, for example, if the entry is to be used to enter a password. If characters in the entry are selected and copied elsewhere, the information copied will be what is displayed, not the true contents of the entry.

-state (state/State)
Specifies one of three states for the entry: **normal**, **disabled**, or **readonly**. If the entry is readonly, then the value may not be changed using widget commands and no insertion cursor will be displayed, even if the input focus is in the widget; the contents of the widget may still be selected. If the entry is disabled, the value may not be changed, no insertion cursor will be displayed, the contents will not be selectable, and the entry may be displayed in a different color, depending on the values of the **-disabledforeground** and **-disabledbackground** options.

-validate (validate/Validate)
Specifies the mode in which validation should operate: **none**, **focus**, **focusin**, **focusout**, **key** , or **all**. It defaults to **none**. When you want validation, you must explicitly state which mode you wish to use. See Validation below for more.

-validatecommand or -vcmd (validateCommand/ValidateCommand)
Specifies a script to eval when you want to validate the input into the entry widget. Setting it to {} disables this feature (the default). This command must return a valid Tcl boolean value. If it returns 0 (or the valid Tcl boolean equivalent) then it means you reject the new edition and it will not occur and the **-invalidcommand** will be evaluated if it is set. If it returns 1, then the new edition occurs. See Validation below for more information.

-width (width/Width)
Specifies an integer value indicating the desired width of the entry window, in average-size characters of the widget's font. If the value is less than or equal to zero, the widget picks a size just large enough to hold its current text.

Description

The **entry** command creates a new window (given by the *pathName* argument) and makes it into an entry widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the entry such as its colors, font, and relief. The **entry** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

An entry is a widget that displays a one-line text string and allows that string to be edited using widget commands described below, which are typically bound to keystrokes and mouse actions. When first created, an entry's string is empty. A portion of the entry may be selected as described below. If an entry is exporting its selection (see the **-exportselection** option), then it

will observe the standard X11 protocols for handling the selection; entry selections are available as type **STRING**. Entries also observe the standard Tk rules for dealing with the input focus. When an entry has the input focus it displays an *insertion cursor* to indicate where new characters will be inserted.

Entries are capable of displaying strings that are too long to fit entirely within the widget's window. In this case, only a portion of the string will be displayed; commands described below may be used to change the view in the window. Entries use the standard **-xscrollcommand** mechanism for interacting with scrollbars (see the description of the **-xscrollcommand** option for details). They also support scanning, as described below.

Validation

Validation works by setting the **-validatecommand** option to a script (*validateCommand*) which will be evaluated according to the **-validate** option as follows:

none
Default. This means no validation will occur.

focus
validateCommand will be called when the entry receives or loses focus.

focusin
validateCommand will be called when the entry receives focus.

focusout
validateCommand will be called when the entry loses focus.

key
validateCommand will be called when the entry is edited.

all
validateCommand will be called for all above conditions.

It is possible to perform percent substitutions on the value of the **-validatecommand** and **-invalidcommand** options, just as you would in a bind script. The following substitutions are recognized:

%d
Type of action: 1 for **insert**, 0 for **delete**, or -1 for focus, forced or textvariable validation.

%i
Index of char string to be inserted/deleted, if any, otherwise -1.

%P
The value of the entry if the edit is allowed. If you are configuring the entry widget to have a new textvariable, this will be the value of that textvariable.

%s
The current value of entry prior to editing.

%S
The text string being inserted/deleted, if any, {} otherwise.

%v
The type of validation currently set.

%V
The type of validation that triggered the callback (key, focusin, focusout, forced).

%W

The name of the entry widget.

In general, the **-textvariable** and **-validatecommand** options can be dangerous to mix. Any problems have been overcome so that using the **-validatecommand** will not interfere with the traditional behavior of the entry widget. Using the **-textvariable** for read-only purposes will never cause problems. The danger comes when you try set the **-textvariable** to something that the **-validatecommand** would not accept, which causes **-validate** to become *none* (the **-invalidcommand** will not be triggered). The same happens when an error occurs evaluating the **-validatecommand**.

Primarily, an error will occur when the **-validatecommand** or **-invalidcommand** encounters an error in its script while evaluating or **-validatecommand** does not return a valid Tcl boolean value. The **-validate** option will also set itself to **none** when you edit the entry widget from within either the **-validatecommand** or the **-invalidcommand**. Such editions will override the one that was being validated. If you wish to edit the entry widget (for example set it to {}) during validation and still have the **-validate** option set, you should include the command

```
after idle {%W config -validate %v}
```

in the **-validatecommand** or **-invalidcommand** (whichever one you were editing the entry widget from). It is also recommended to not set an associated **-textvariable** during validation, as that can cause the entry widget to become out of sync with the **-textvariable**.

Widget command

The **entry** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName subcommand ?arg arg ...?
```

Subcommand and the *arg*s determine the exact behavior of the command.

Indices

Many of the widget commands for entries take one or more indices as arguments. An index specifies a particular character in the entry's string, in any of the following ways:

number
Specifies the character as a numerical index, where 0 corresponds to the first character in the string.

anchor
Indicates the anchor point for the selection, which is set with the **select from** and **select adjust** widget commands.

end
Indicates the character just after the last one in the entry's string. This is equivalent to specifying a numerical index equal to the length of the entry's string.

insert
Indicates the character adjacent to and immediately following the insertion cursor.

sel.first
Indicates the first character in the selection. It is an error to use this form if the selection is not in the entry window.

sel.last

Indicates the character just after the last one in the selection. It is an error to use this form if the selection is not in the entry window.

@number
In this form, *number* is treated as an x-coordinate in the entry's window; the character spanning that x-coordinate is used. For example, "**@0**" indicates the left-most character in the window.

Abbreviations may be used for any of the forms above, e.g. "**e**" or "**sel.f**". In general, out-of-range indices are automatically rounded to the nearest legal value.

Subcommands

The following commands are possible for entry widgets:

pathName* bbox *index
Returns a list of four numbers describing the bounding box of the character given by *index*. The first two elements of the list give the x and y coordinates of the upper-left corner of the screen area covered by the character (in pixels relative to the widget) and the last two elements give the width and height of the character, in pixels. The bounding box may refer to a region outside the visible area of the window.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **entry** command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **entry** command.

***pathName* delete *first* ?*last*?**
Delete one or more elements of the entry. *First* is the index of the first character to delete, and *last* is the index of the character just after the last one to delete. If *last* is not specified it defaults to *first*+1, i.e. a single character is deleted. This command returns an empty string.

***pathName* get**
Returns the entry's string.

pathName* icursor *index
Arrange for the insertion cursor to be displayed just before the character given by *index*. Returns an empty string.

pathName* index *index
Returns the numerical index corresponding to *index*.

pathName* insert *index string
Insert the characters of *string* just before the character indicated by *index*. Returns an empty string.

pathName* scan *option args
This command is used to implement scanning on entries. It has two forms, depending on

option:

pathName* scan mark *x

Records *x* and the current view in the entry window; used in conjunction with later **scan dragto** commands. Typically this command is associated with a mouse button press in the widget. It returns an empty string.

pathName* scan dragto *x

This command computes the difference between its *x* argument and the *x* argument to the last **scan mark** command for the widget. It then adjusts the view left or right by 10 times the difference in x-coordinates. This command is typically associated with mouse motion events in the widget, to produce the effect of dragging the entry at high speed through the window. The return value is an empty string.

pathName* selection *option arg

This command is used to adjust the selection within an entry. It has several forms, depending on *option*:

pathName* selection adjust *index

Locate the end of the selection nearest to the character given by *index*, and adjust that end of the selection to be at *index* (i.e. including but not going beyond *index*). The other end of the selection is made the anchor point for future **select to** commands. If the selection is not currently in the entry, then a new selection is created to include the characters between *index* and the most recent selection anchor point, inclusive. Returns an empty string.

***pathName* selection clear**

Clear the selection if it is currently in this widget. If the selection is not in this widget then the command has no effect. Returns an empty string.

pathName* selection from *index

Set the selection anchor point to just before the character given by *index*. Does not change the selection. Returns an empty string.

***pathName* selection present**

Returns 1 if there is are characters selected in the entry, 0 if nothing is selected.

pathName* selection range *start end

Sets the selection to include the characters starting with the one indexed by *start* and ending with the one just before *end*. If *end* refers to the same character as *start* or an earlier one, then the entry's selection is cleared.

pathName* selection to *index

If *index* is before the anchor point, set the selection to the characters from *index* up to but not including the anchor point. If *index* is the same as the anchor point, do nothing. If *index* is after the anchor point, set the selection to the characters from the anchor point up to but not including *index*. The anchor point is determined by the most recent **select from** or **select adjust** command in this widget. If the selection is not in this widget then a new selection is created using the most recent anchor point specified for the widget. Returns an empty string.

***pathName* validate**

This command is used to force an evaluation of the **-validatecommand** independent of the conditions specified by the **-validate** option. This is done by temporarily setting the

-validate option to **all**. It returns 0 or 1.

pathName* xview *args

This command is used to query and change the horizontal position of the text in the widget's window. It can take any of the following forms:

***pathName* xview**

Returns a list containing two elements. Each element is a real fraction between 0 and 1; together they describe the horizontal span that is visible in the window. For example, if the first element is .2 and the second element is .6, 20% of the entry's text is off-screen to the left, the middle 40% is visible in the window, and 40% of the text is off-screen to the right. These are the same values passed to scrollbars via the **-xscrollcommand** option.

pathName* xview *index

Adjusts the view in the window so that the character given by *index* is displayed at the left edge of the window.

pathName* xview moveto *fraction

Adjusts the view in the window so that the character *fraction* of the way through the text appears at the left edge of the window. *Fraction* must be a fraction between 0 and 1.

pathName* xview scroll *number what

This command shifts the view in the window left or right according to *number* and *what*. *Number* must be an integer. *What* must be either **units** or **pages** or an abbreviation of one of these. If *what* is **units**, the view adjusts left or right by *number* average-width characters on the display; if it is **pages** then the view adjusts by *number* screenfuls. If *number* is negative then characters farther to the left become visible; if it is positive then characters farther to the right become visible.

Default bindings

Tk automatically creates class bindings for entries that give them the following default behavior. In the descriptions below, "word" refers to a contiguous group of letters, digits, or "_" characters, or any single character other than these.

1. Clicking mouse button 1 positions the insertion cursor just before the character underneath the mouse cursor, sets the input focus to this widget, and clears any selection in the widget. Dragging with mouse button 1 strokes out a selection between the insertion cursor and the character under the mouse.
2. Double-clicking with mouse button 1 selects the word under the mouse and positions the insertion cursor at the end of the word. Dragging after a double click will stroke out a selection consisting of whole words.
3. Triple-clicking with mouse button 1 selects all of the text in the entry and positions the insertion cursor at the end of the line.
4. The ends of the selection can be adjusted by dragging with mouse button 1 while the Shift key is down; this will adjust the end of the selection that was nearest to the mouse cursor when button 1 was pressed. If the button is double-clicked before dragging then the selection will be adjusted in units of whole words.
5. Clicking mouse button 1 with the Control key down will position the insertion cursor in the entry without affecting the selection.
6. If any normal printing characters are typed in an entry, they are inserted at the point of the insertion cursor.

7. The view in the entry can be adjusted by dragging with the middle mouse button (button 2, or button 3 in TkAqua). If the middle mouse button is clicked without moving the mouse, the selection is copied into the entry at the position of the mouse cursor.
8. If the mouse is dragged out of the entry on the left or right sides while button 1 is pressed, the entry will automatically scroll to make more text visible (if there is more text off-screen on the side where the mouse left the window).
9. The Left and Right keys move the insertion cursor one character to the left or right; they also clear any selection in the entry and set the selection anchor. If Left or Right is typed with the Shift key down, then the insertion cursor moves and the selection is extended to include the new character. Control-Left and Control-Right move the insertion cursor by words, and Control-Shift-Left and Control-Shift-Right move the insertion cursor by words and also extend the selection. Control-b and Control-f behave the same as Left and Right, respectively. Meta-b and Meta-f behave the same as Control-Left and Control-Right, respectively.
10. The Home key, or Control-a, will move the insertion cursor to the beginning of the entry and clear any selection in the entry. Shift-Home moves the insertion cursor to the beginning of the entry and also extends the selection to that point.
11. The End key, or Control-e, will move the insertion cursor to the end of the entry and clear any selection in the entry. Shift-End moves the cursor to the end and extends the selection to that point.
12. The Select key and Control-Space set the selection anchor to the position of the insertion cursor. They do not affect the current selection. Shift-Select and Control-Shift-Space adjust the selection to the current position of the insertion cursor, selecting from the anchor to the insertion cursor if there was not any selection previously.
13. Control-/ selects all the text in the entry.
14. Control-\ clears any selection in the entry.
15. The F16 key (labelled Copy on many Sun workstations) or Meta-w copies the selection in the widget to the clipboard, if there is a selection.
16. The F20 key (labelled Cut on many Sun workstations) or Control-w copies the selection in the widget to the clipboard and deletes the selection. If there is no selection in the widget then these keys have no effect.
17. The F18 key (labelled Paste on many Sun workstations) or Control-y inserts the contents of the clipboard at the position of the insertion cursor.
18. The Delete key deletes the selection, if there is one in the entry. If there is no selection, it deletes the character to the right of the insertion cursor.
19. The BackSpace key and Control-h delete the selection, if there is one in the entry. If there is no selection, it deletes the character to the left of the insertion cursor.
20. Control-d deletes the character to the right of the insertion cursor.
21. Meta-d deletes the word to the right of the insertion cursor.
22. Control-k deletes all the characters to the right of the insertion cursor.
23. Control-t reverses the order of the two characters to the right of the insertion cursor.

If the entry is disabled using the **-state** option, then the entry's view can still be adjusted and text in the entry can still be selected, but no insertion cursor will be displayed and no text modifications will take place except if the entry is linked to a variable using the **-textvariable** option, in which case any changes to the variable are reflected by the entry whatever the value of its **-state** option.

The behavior of entries can be changed by defining new bindings for individual widgets or by

redefining the class bindings.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 1998-2000 Scriptics Corporation.

event

Keywords: event, binding, define, handle, virtual event

Name

event - Miscellaneous event facilities: define virtual events and generate events

Synopsis

event *option* ?*arg arg* ...?

Description

The **event** command provides several facilities for dealing with window system events, such as defining virtual events and synthesizing events. The command has several different forms, determined by the first argument. The following forms are currently supported:

event add <<*virtual*>> *sequence* ?*sequence ...*?
Associates the virtual event *virtual* with the physical event sequence(s) given by the *sequence* arguments, so that the virtual event will trigger whenever any one of the *sequence* s occurs. *Virtual* may be any string value and *sequence* may have any of the values allowed for the *sequence* argument to the bind command. If *virtual* is already defined, the new physical event sequences add to the existing sequences for the event.

event delete <<*virtual*>> ?*sequence sequence ...*?
Deletes each of the *sequence*s from those associated with the virtual event given by *virtual*. *Virtual* may be any string value and *sequence* may have any of the values allowed for the *sequence* argument to the bind command. Any *sequence*s not currently associated with *virtual* are ignored. If no *sequence* argument is provided, all physical event sequences are removed for *virtual*, so that the virtual event will not trigger anymore.

event generate *window event* ?*option value option value ...*?
Generates a window event and arranges for it to be processed just as if it had come from the window system. *Window* gives the path name of the window for which the event will be generated; it may also be an identifier (such as returned by winfo id) as long as it is for a window in the current application. *Event* provides a basic description of the event, such as or **<>**. If *Window* is empty the whole screen is meant, and coordinates are relative to the screen. *Event* may have any of the forms allowed for the *sequence* argument of the bind command except that it must consist of a single event pattern, not a sequence. *Option-value* pairs may be used to specify additional attributes of the event, such as the x and y mouse position; see Event fields below. If the **-when** option is not specified, the event is processed immediately: all of the handlers for the event will complete before the **event generate** command returns. If the **-when** option is specified then it determines when the event is processed. Certain events, such as key events, require that the window has focus to receive the event properly.

event info ?<<*virtual*>>?
Returns information about virtual events. If the **<<***virtual***>>** argument is omitted, the return value is a list of all the virtual events that are currently defined. If **<<***virtual***>>** is specified

then the return value is a list whose elements are the physical event sequences currently defined for the given virtual event; if the virtual event is not defined then an empty string is returned.

Note that virtual events that are not bound to physical event sequences are *not* returned by **event info**.

Event fields

The following options are supported for the **event generate** command. These correspond to the "%" expansions allowed in binding scripts for the bind command.

-above *window*
Window specifies the *above* field for the event, either as a window path name or as an integer window id. Valid for **Configure** events. Corresponds to the **%a** substitution for binding scripts.

-borderwidth *size*
Size must be a screen distance; it specifies the *border_width* field for the event. Valid for **Configure** events. Corresponds to the **%B** substitution for binding scripts.

-button *number*
Number must be an integer; it specifies the *detail* field for a **ButtonPress** or **ButtonRelease** event, overriding any button number provided in the base *event* argument. Corresponds to the **%b** substitution for binding scripts.

-count *number*
Number must be an integer; it specifies the *count* field for the event. Valid for **Expose** events. Corresponds to the **%c** substitution for binding scripts.

-data *string*
String may be any value; it specifies the *user_data* field for the event. Only valid for virtual events. Corresponds to the **%d** substitution for virtual events in binding scripts.

-delta *number*
Number must be an integer; it specifies the *delta* field for the **MouseWheel** event. The *delta* refers to the direction and magnitude the mouse wheel was rotated. Note the value is not a screen distance but are units of motion in the mouse wheel. Typically these values are multiples of 120. For example, 120 should scroll the text widget up 4 lines and -240 would scroll the text widget down 8 lines. Of course, other widgets may define different behaviors for mouse wheel motion. This field corresponds to the **%D** substitution for binding scripts.

-detail *detail*
Detail specifies the *detail* field for the event and must be one of the following:

NotifyAncestor NotifyNonlinearVirtual
NotifyDetailNone NotifyPointer
NotifyInferior NotifyPointerRoot
NotifyNonlinear NotifyVirtual

Valid for **Enter**, **Leave**, **FocusIn** and **FocusOut** events. Corresponds to the **%d** substitution for binding scripts.

-focus *boolean*
Boolean must be a boolean value; it specifies the *focus* field for the event. Valid for **Enter** and **Leave** events. Corresponds to the **%f** substitution for binding scripts.

-height *size*
Size must be a screen distance; it specifies the *height* field for the event. Valid for **Configure** events. Corresponds to the **%h** substitution for binding scripts.

-keycode *number*
Number must be an integer; it specifies the *keycode* field for the event. Valid for **KeyPress** and **KeyRelease** events. Corresponds to the **%k** substitution for binding scripts.

-keysym *name*
Name must be the name of a valid keysym, such as **g**, **space**, or **Return**; its corresponding keycode value is used as the *keycode* field for event, overriding any detail specified in the base *event* argument. Valid for **KeyPress** and **KeyRelease** events. Corresponds to the **%K** substitution for binding scripts.

-mode *notify*
Notify specifies the *mode* field for the event and must be one of **NotifyNormal**, **NotifyGrab**, **NotifyUngrab**, or **NotifyWhileGrabbed**. Valid for **Enter**, **Leave**, **FocusIn**, and **FocusOut** events. Corresponds to the **%m** substitution for binding scripts.

-override *boolean*
Boolean must be a boolean value; it specifies the *override_redirect* field for the event. Valid for **Map**, **Reparent**, and **Configure** events. Corresponds to the **%o** substitution for binding scripts.

-place *where*
Where specifies the *place* field for the event; it must be either **PlaceOnTop** or **PlaceOnBottom**. Valid for **Circulate** events. Corresponds to the **%p** substitution for binding scripts.

-root *window*
Window must be either a window path name or an integer window identifier; it specifies the *root* field for the event. Valid for **KeyPress**, **KeyRelease**, **ButtonPress**, **ButtonRelease**, **Enter**, **Leave**, and **Motion** events. Corresponds to the **%R** substitution for binding scripts.

-rootx *coord*
Coord must be a screen distance; it specifies the *x_root* field for the event. Valid for **KeyPress**, **KeyRelease**, **ButtonPress**, **ButtonRelease**, **Enter**, **Leave**, and **Motion** events. Corresponds to the **%X** substitution for binding scripts.

-rooty *coord*
Coord must be a screen distance; it specifies the *y_root* field for the event. Valid for **KeyPress**, **KeyRelease**, **ButtonPress**, **ButtonRelease**, **Enter**, **Leave**, and **Motion** events. Corresponds to the **%Y** substitution for binding scripts.

-sendevent *boolean*
Boolean must be a boolean value; it specifies the *send_event* field for the event. Valid for all events. Corresponds to the **%E** substitution for binding scripts.

-serial *number*
Number must be an integer; it specifies the *serial* field for the event. Valid for all events. Corresponds to the **%#** substitution for binding scripts.

-state *state*
State specifies the *state* field for the event. For **KeyPress**, **KeyRelease**, **ButtonPress**, **ButtonRelease**, **Enter**, **Leave**, and **Motion** events it must be an integer value. For **Visibility** events it must be one of **VisibilityUnobscured**, **VisibilityPartiallyObscured**, or

VisibilityFullyObscured. This option overrides any modifiers such as **Meta** or **Control** specified in the base *event*. Corresponds to the **%s** substitution for binding scripts.

-subwindow *window*
Window specifies the *subwindow* field for the event, either as a path name for a Tk widget or as an integer window identifier. Valid for **KeyPress**, **KeyRelease**, **ButtonPress**, **ButtonRelease**, **Enter**, **Leave**, and **Motion** events. Similar to **%S** substitution for binding scripts.

-time *integer*
Integer must be an integer value; it specifies the *time* field for the event. Additonally the special value **current** is allowed, this value will be substituted by the current event time. Valid for **KeyPress**, **KeyRelease**, **ButtonPress**, **ButtonRelease**, **Enter**, **Leave**, **Motion**, and **Property** events. Corresponds to the **%t** substitution for binding scripts.

-warp *boolean*
boolean must be a boolean value; it specifies whether the screen pointer should be warped as well. Valid for **KeyPress**, **KeyRelease**, **ButtonPress**, **ButtonRelease**, and **Motion** events. Tk supports pointer warping relative to the screen's root window, and relative to a Tk window. In the latter case, the pointer will only warp if the window is mapped.

-width *size*
Size must be a screen distance; it specifies the *width* field for the event. Valid for **Configure** events. Corresponds to the **%w** substitution for binding scripts.

-when *when*
When determines when the event will be processed; it must have one of the following values:

now
Process the event immediately, before the command returns. This also happens if the **-when** option is omitted.

tail
Place the event on Tcl's event queue behind any events already queued for this application.

head
Place the event at the front of Tcl's event queue, so that it will be handled before any other events already queued.

mark
Place the event at the front of Tcl's event queue but behind any other events already queued with **-when mark**. This option is useful when generating a series of events that should be processed in order but at the front of the queue.

-x *coord*
Coord must be a screen distance; it specifies the *x* field for the event. Valid for **KeyPress**, **KeyRelease**, **ButtonPress**, **ButtonRelease**, **Motion**, **Enter**, **Leave**, **Expose**, **Configure**, **Gravity**, and **Reparent** events. Corresponds to the **%x** substitution for binding scripts. If *Window* is empty the coordinate is relative to the screen, and this option corresponds to the **%X** substitution for binding scripts.

-y *coord*
Coord must be a screen distance; it specifies the *y* field for the event. Valid for **KeyPress**, **KeyRelease**, **ButtonPress**, **ButtonRelease**, **Motion**, **Enter**, **Leave**, **Expose**, **Configure**, **Gravity**, and **Reparent** events. Corresponds to the **%y** substitution for binding scripts. If

Window is empty the coordinate is relative to the screen, and this option corresponds to the **%Y** substitution for binding scripts.

Any options that are not specified when generating an event are filled with the value 0, except for *serial*, which is filled with the next X event serial number.

Predefined virtual events

Tk defines the following virtual events for the purposes of notification:

<<Activate>>
This is sent to widget to notify it that the letter it has underlined (as an accelerator indicator) with the **-underline** option has been pressed in combination with the Alt key. The usual response to this is to either focus into the widget (or some related widget) or to invoke the widget.

<<Deactivate>>
This can be sent to some widgets (e.g. button, listbox, menu) as an alternative to.

<<ListboxSelect>>
This is sent to a listbox when the set of selected item(s) in the listbox is updated.

<<MenuSelect>>
This is sent to a menu when the currently selected item in the menu changes. It is intended for use with context-sensitive help systems.

<<Modified>>
This is sent to a text widget when the contents of the widget are changed.

<<Selection>>
This is sent to a text widget when the selection in the widget is changed.

<<ThemeChanged>>
This is sent to all widgets when the ttk theme changed. The ttk widgets listen to this event and redisplay themselves when it fires. The legacy widgets ignore this event.

<<TkFontChanged>>
This event is sent to all widgets when a font is changed, for example, by the use of [font configure]. The user_data field (%d) will have the value "FontChanged". For other system wide changes, this event will be sent to all widgets, and the user_data field will indicate the cause of the change. NOTE: all tk and ttk widgets already handle this event internally.

<<TraverseIn>>
This is sent to a widget when the focus enters the widget because of a user-driven "tab to widget" action.

<<TraverseOut>>
This is sent to a widget when the focus leaves the widget because of a user-driven "tab to widget" action.

<<UndoStack>>
This is sent to a text widget when its undo stack or redo stack becomes empty or unempty.

<<WidgetViewSync>>
This is sent to a text widget when its internal data become obsolete, and again when these internal data are back in sync with the widget view. The detail field (%d substitution) is either true (when the widget is in sync) or false (when it is not).

Tk defines the following virtual events for the purposes of unifying bindings across multiple

platforms. Users expect them to behave in the following way:

<>
Delete the currently selected widget contents.

<>
Copy the currently selected widget contents to the clipboard.

<>
Move the currently selected widget contents to the clipboard.

<>
Move to the end of the line in the current widget while deselecting any selected contents.

<>
Move to the start of the line in the current widget while deselecting any selected contents.

<>
Move to the next item (i.e., visible character) in the current widget while deselecting any selected contents.

<>
Move to the next line in the current widget while deselecting any selected contents.

<>
Move to the next paragraph in the current widget while deselecting any selected contents.

<>
Move to the next group of items (i.e., visible word) in the current widget while deselecting any selected contents.

<>
Replace the currently selected widget contents with the contents of the clipboard.

<>
Insert the contents of the selection at the mouse location. (This event has meaningful **%x** and **%y** substitutions).

<>
Move to the previous item (i.e., visible character) in the current widget while deselecting any selected contents.

<>
Move to the previous line in the current widget while deselecting any selected contents.

<>
Move to the previous paragraph in the current widget while deselecting any selected contents.

<>
Traverse to the previous window.

<>
Move to the previous group of items (i.e., visible word) in the current widget while deselecting any selected contents.

<>
Redo one undone action.

<>
Set the range of selected contents to the complete widget.

<>

Move to the end of the line in the current widget while extending the range of selected contents.

<>

Move to the start of the line in the current widget while extending the range of selected contents.

<>

Move to the next item (i.e., visible character) in the current widget while extending the range of selected contents.

<>

Move to the next line in the current widget while extending the range of selected contents.

<>

Move to the next paragraph in the current widget while extending the range of selected contents.

<>

Move to the next group of items (i.e., visible word) in the current widget while extending the range of selected contents.

<>

Reset the range of selected contents to be empty.

<>

Move to the previous item (i.e., visible character) in the current widget while extending the range of selected contents.

<>

Move to the previous line in the current widget while extending the range of selected contents.

<>

Move to the previous paragraph in the current widget while extending the range of selected contents.

<>

Move to the previous group of items (i.e., visible word) in the current widget while extending the range of selected contents.

<>

Toggle the selection.

<>

Undo the last action.

Examples

Mapping keys to virtual events

In order for a virtual event binding to trigger, two things must happen. First, the virtual event must be defined with the **event add** command. Second, a binding must be created for the virtual event with the bind command. Consider the following virtual event definitions:

```
event add <<Paste>> <Control-y>
event add <<Paste>> <Button-2>
event add <<Save>> <Control-X><Control-S>
event add <<Save>> <Shift-F12>
if {[tk windowingsystem] eq "aqua"} {
    event add <<Save>> <Command-s>
}
```

In the bind command, a virtual event can be bound like any other builtin event type as follows:

```
bind Entry <<Paste>> {%W insert [selection get]}
```

The double angle brackets are used to specify that a virtual event is being bound. If the user types Control-y or presses button 2, or if a **<>** virtual event is synthesized with **event generate** , then the **<>** binding will be invoked.

If a virtual binding has the exact same sequence as a separate physical binding, then the physical binding will take precedence. Consider the following example:

```
event add <<Paste>> <Control-y> <Meta-Control-y>
bind Entry <Control-y> {puts Control-y}
bind Entry <<Paste>> {puts Paste}
```

When the user types Control-y the binding will be invoked, because a physical event is considered more specific than a virtual event, all other things being equal. However, when the user types Meta-Control-y the **<>** binding will be invoked, because the **Meta** modifier in the physical pattern associated with the virtual binding is more specific than the ** sequence for the physical event.

Bindings on a virtual event may be created before the virtual event exists. Indeed, the virtual event never actually needs to be defined, for instance, on platforms where the specific virtual event would be meaningless or ungeneratable.

When a definition of a virtual event changes at run time, all windows will respond immediately to the new definition. Starting from the preceding example, if the following code is executed:

```
bind Entry <Control-y> {}
event add <<Paste>> <Key-F6>
```

the behavior will change such in two ways. First, the shadowed **<>** binding will emerge. Typing Control-y will no longer invoke the binding, but instead invoke the virtual event **<>**. Second, pressing the F6 key will now also invoke the **<>** binding.

Moving the mouse pointer

Sometimes it is useful to be able to really move the mouse pointer. For example, if you have some software that is capable of demonstrating directly to the user how to use the program. To do this, you need to "warp" the mouse around by using **event generate**, like this:

```
for {set xy 0} {$xy < 200} {incr xy} {
    event generate . <Motion> -x $xy -y $xy -warp 1
    update
    after 50
}
```

Note that it is usually considered bad style to move the mouse pointer for the user because it removes control from them. Therefore this technique should be used with caution. Also note that it is not guaranteed to function on all platforms.

Copyright (c) 1996 Sun Microsystems, Inc.
Copyright (c) 1998-2000 Ajuba Solutions.

focus

Keywords: focus, events, keyboard, top-level, window manager

Name

focus - Manage the input focus

Synopsis

focus
focus *window*
focus *option* ?*arg arg* ...?

Description

The **focus** command is used to manage the Tk input focus. At any given time, one window on each display is designated as the *focus window*; any key press or key release events for the display are sent to that window. It is normally up to the window manager to redirect the focus among the top-level windows of a display. For example, some window managers automatically set the input focus to a top-level window whenever the mouse enters it; others redirect the input focus only when the user clicks on a window. Usually the window manager will set the focus only to top-level windows, leaving it up to the application to redirect the focus among the children of the top-level.

Tk remembers one focus window for each top-level (the most recent descendant of that top-level to receive the focus); when the window manager gives the focus to a top-level, Tk automatically redirects it to the remembered window. Within a top-level Tk uses an *explicit* focus model by default. Moving the mouse within a top-level does not normally change the focus; the focus changes only when a widget decides explicitly to claim the focus (e.g., because of a button click), or when the user types a key such as Tab that moves the focus.

The Tcl procedure tk_focusFollowsMouse may be invoked to create an *implicit* focus model: it reconfigures Tk so that the focus is set to a window whenever the mouse enters it. The Tcl procedures tk_focusNext and tk_focusPrev implement a focus order among the windows of a top-level; they are used in the default bindings for Tab and Shift-Tab, among other things.

The **focus** command can take any of the following forms:

focus
Returns the path name of the focus window on the display containing the application's main

window, or an empty string if no window in this application has the focus on that display. Note: it is better to specify the display explicitly using **-displayof** (see below) so that the code will work in applications using multiple displays.

focus *window*
If the application currently has the input focus on *window*'s display, this command resets the input focus for *window*'s display to *window* and returns an empty string. If the application does not currently have the input focus on *window*'s display, *window* will be remembered as the focus for its top-level; the next time the focus arrives at the top-level, Tk will redirect it to *window*. If *window* is an empty string then the command does nothing.

focus -displayof *window*
Returns the name of the focus window on the display containing *window*. If the focus window for *window*'s display is not in this application, the return value is an empty string.

focus -force *window*
Sets the focus of *window*'s display to *window*, even if the application does not currently have the input focus for the display. This command should be used sparingly, if at all. In normal usage, an application should not claim the focus for itself; instead, it should wait for the window manager to give it the focus. If *window* is an empty string then the command does nothing.

focus -lastfor *window*
Returns the name of the most recent window to have the input focus among all the windows in the same top-level as *window*. If no window in that top-level has ever had the input focus, or if the most recent focus window has been deleted, then the name of the top-level is returned. The return value is the window that will receive the input focus the next time the window manager gives the focus to the top-level.

Quirks

When an internal window receives the input focus, Tk does not actually set the X focus to that window; as far as X is concerned, the focus will stay on the top-level window containing the window with the focus. However, Tk generates FocusIn and FocusOut events just as if the X focus were on the internal window. This approach gets around a number of problems that would occur if the X focus were actually moved; the fact that the X focus is on the top-level is invisible unless you use C code to query the X server directly.

Example

To make a window that only participates in the focus traversal ring when a variable is set, add the following bindings to the widgets *before* and *after* it in that focus ring:

```
button .before -text "Before"
button .middle -text "Middle"
button .after  -text "After"
checkbutton .flag -variable traverseToMiddle -takefocus 0
pack .flag -side left
pack .before .middle .after
bind .before <Tab> {
   if {!$traverseToMiddle} {
      focus .after
```

```
      break
   }
}
bind .after <Shift-Tab> {
   if {!$traverseToMiddle} {
      focus .before
      break
   }
}
focus .before
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

focusNext

Keywords: focusNext, focus, keyboard traversal, top-level

Name

tk_focusNext, tk_focusPrev, tk_focusFollowsMouse - Utility procedures for managing the input focus.

Synopsis

tk_focusNext *window*
tk_focusPrev *window*
tk_focusFollowsMouse

Description

tk_focusNext is a utility procedure used for keyboard traversal. It returns the "next" window after *window* in focus order. The focus order is determined by the stacking order of windows and the structure of the window hierarchy. Among siblings, the focus order is the same as the stacking order, with the lowest window being first. If a window has children, the window is visited first, followed by its children (recursively), followed by its next sibling. Top-level windows other than *window* are skipped, so that **tk_focusNext** never returns a window in a different top-level from *window*.

After computing the next window, **tk_focusNext** examines the window's **-takefocus** option to see whether it should be skipped. If so, **tk_focusNext** continues on to the next window in the focus order, until it eventually finds a window that will accept the focus or returns back to *window*.

tk_focusPrev is similar to **tk_focusNext** except that it returns the window just before *window* in the focus order.

tk_focusFollowsMouse changes the focus model for the application to an implicit one where the window under the mouse gets the focus. After this procedure is called, whenever the mouse enters a window Tk will automatically give it the input focus. The focus command may be used to move the focus to a window other than the one under the mouse, but as soon as the mouse moves into a new window the focus will jump to that window. Note: at present there is no built-in support for returning the application to an explicit focus model; to do this you will have to write a

script that deletes the bindings created by **tk_focusFollowsMouse**.

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

font

Keywords: font

Name

font - Create and inspect fonts.

Synopsis

font *option* ?*arg* ...?

Description

The **font** command provides several facilities for dealing with fonts, such as defining named fonts and inspecting the actual attributes of a font. The command has several different forms, determined by the first argument. The following forms are currently supported:

font actual *font* ?*-displayof window*? ?*option*? ?*--*? ?*char*?

Returns information about the actual attributes that are obtained when *font* is used on *window*'s display; the actual attributes obtained may differ from the attributes requested due to platform-dependent limitations, such as the availability of font families and point sizes. *font* is a font description; see Font descriptions below. If the *window* argument is omitted, it defaults to the main window. If *option* is specified, returns the value of that attribute; if it is omitted, the return value is a list of all the attributes and their values. See Font options below for a list of the possible attributes. If the *char* argument is supplied, it must be a single character. The font attributes returned will be those of the specific font used to render that character, which will be different from the base font if the base font does not contain the given character. If *char* may be a hyphen, it should be preceded by **--** to distinguish it from a misspelled *option*.

font configure *fontname* ?*option*? ?*value option value* ...?

Query or modify the desired attributes for the named font called *fontname*. If no *option* is specified, returns a list describing all the options and their values for *fontname*. If a single *option* is specified with no *value*, then returns the current value of that attribute. If one or more *option-value* pairs are specified, then the command modifies the given named font to have the given values; in this case, all widgets using that font will redisplay themselves using the new attributes for the font. See Font options below for a list of the possible attributes.

Note that on Aqua/macOS, the system fonts (see Platform specific fonts below) may not be actually altered because they are implemented by the system theme. To achieve the effect of modification, use **font actual** to get their configuration and **font create** to synthesize a copy of the font which can be modified.

font create ?*fontname*? ?*option value* ...?

Creates a new named font and returns its name. *fontname* specifies the name for the font; if it is omitted, then Tk generates a new name of the form **font***x*, where *x* is an integer. There may be any number of *option-value* pairs, which provide the desired attributes for the new named font. See Font options below for a list of the possible attributes.

font delete *fontname* ?*fontname* ...?
Delete the specified named fonts. If there are widgets using the named font, the named font will not actually be deleted until all the instances are released. Those widgets will continue to display using the last known values for the named font. If a deleted named font is subsequently recreated with another call to **font create**, the widgets will use the new named font and redisplay themselves using the new attributes of that font.

font families ?*-displayof window*?
The return value is a list of the case-insensitive names of all font families that exist on *window* 's display. If the *window* argument is omitted, it defaults to the main window.

font measure *font* ?*-displayof window*? *text*
Measures the amount of space the string *text* would use in the given *font* when displayed in *window*. *font* is a font description; see Font descriptions below. If the *window* argument is omitted, it defaults to the main window. The return value is the total width in pixels of *text*, not including the extra pixels used by highly exaggerated characters such as cursive "f". If the string contains newlines or tabs, those characters are not expanded or treated specially when measuring the string.

font metrics *font* ?*-displayof window*? ?*option*?
Returns information about the metrics (the font-specific data), for *font* when it is used on *window*'s display. *font* is a font description; see Font descriptions below. If the *window* argument is omitted, it defaults to the main window. If *option* is specified, returns the value of that metric; if it is omitted, the return value is a list of all the metrics and their values. See Font metrics below for a list of the possible metrics.

font names
The return value is a list of all the named fonts that are currently defined.

Font descriptions

The following formats are accepted as a font description anywhere *font* is specified as an argument above; these same forms are also permitted when specifying the **-font** option for widgets.

[1] *fontname*
The name of a named font, created using the **font create** command. When a widget uses a named font, it is guaranteed that this will never cause an error, as long as the named font exists, no matter what potentially invalid or meaningless set of attributes the named font has. If the named font cannot be displayed with exactly the specified attributes, some other close font will be substituted automatically.

[2] *systemfont*
The platform-specific name of a font, interpreted by the graphics server. This also includes, under X, an XLFD (see [4]) for which a single "*" character was used to elide more than one field in the middle of the name. See Platform specific fonts for a list of the system fonts.

[3] *family* ?*size*? ?*style*? ?*style ...*?
A properly formed list whose first element is the desired font *family* and whose optional second element is the desired *size*. The interpretation of the *size* attribute follows the same rules described for **-size** in Font options below. Any additional optional arguments following the *size* are font *style*s. Possible values for the *style* arguments are as follows:

normal bold roman italic

underline overstrike

[4] X-font names (XLFD)
A Unix-centric font name of the form *-foundry-family-weight-slant-setwidth-addstyle-pixel-point-resx-resy-spacing-width-charset-encoding*. The "*" character may be used to skip individual fields that the user does not care about. There must be exactly one "*" for each field skipped, except that a "*" at the end of the XLFD skips any remaining fields; the shortest valid XLFD is simply "*", signifying all fields as defaults. Any fields that were skipped are given default values. For compatibility, an XLFD always chooses a font of the specified pixel size (not point size); although this interpretation is not strictly correct, all existing applications using XLFDs assumed that one "point" was in fact one pixel and would display incorrectly (generally larger) if the correct size font were actually used.

[5] *option value ?option value ...?*
A properly formed list of *option-value* pairs that specify the desired attributes of the font, in the same format used when defining a named font; see Font options below.

When font description *font* is used, the system attempts to parse the description according to each of the above five rules, in the order specified. Cases [1] and [2] must match the name of an existing named font or of a system font. Cases [3], [4], and [5] are accepted on all platforms and the closest available font will be used. In some situations it may not be possible to find any close font (e.g., the font family was a garbage value); in that case, some system-dependent default font is chosen. If the font description does not match any of the above patterns, an error is generated.

Font metrics

The following options are used by the **font metrics** command to query font-specific data determined when the font was created. These properties are for the whole font itself and not for individual characters drawn in that font. In the following definitions, the "baseline" of a font is the horizontal line where the bottom of most letters line up; certain letters, such as lower-case "g" stick below the baseline.

-ascent
The amount in pixels that the tallest letter sticks up above the baseline of the font, plus any extra blank space added by the designer of the font.

-descent
The largest amount in pixels that any letter sticks down below the baseline of the font, plus any extra blank space added by the designer of the font.

-linespace
Returns how far apart vertically in pixels two lines of text using the same font should be placed so that none of the characters in one line overlap any of the characters in the other line. This is generally the sum of the ascent above the baseline line plus the descent below the baseline.

-fixed
Returns a boolean flag that is "**1**" if this is a fixed-width font, where each normal character is the same width as all the other characters, or is "**0**" if this is a proportionally-spaced font, where individual characters have different widths. The widths of control characters, tab characters, and other non-printing characters are not included when calculating this value.

Font options

The following options are supported on all platforms, and are used when constructing a named font or when specifying a font using style [5] as above:

-family *name*
The case-insensitive font family name. Tk guarantees to support the font families named **Courier** (a monospaced "typewriter" font), **Times** (a serifed "newspaper" font), and **Helvetica** (a sans-serif "European" font). The most closely matching native font family will automatically be substituted when one of the above font families is used. The *name* may also be the name of a native, platform-specific font family; in that case it will work as desired on one platform but may not display correctly on other platforms. If the family is unspecified or unrecognized, a platform-specific default font will be chosen.

-size *size*
The desired size of the font. If the *size* argument is a positive number, it is interpreted as a size in points. If *size* is a negative number, its absolute value is interpreted as a size in pixels. If a font cannot be displayed at the specified size, a nearby size will be chosen. If *size* is unspecified or zero, a platform-dependent default size will be chosen.

Sizes should normally be specified in points so the application will remain the same ruler size on the screen, even when changing screen resolutions or moving scripts across platforms. However, specifying pixels is useful in certain circumstances such as when a piece of text must line up with respect to a fixed-size bitmap. The mapping between points and pixels is set when the application starts, based on properties of the installed monitor, but it can be overridden by calling the tk scaling command.

-weight *weight*
The nominal thickness of the characters in the font. The value **normal** specifies a normal weight font, while **bold** specifies a bold font. The closest available weight to the one specified will be chosen. The default weight is **normal**.

-slant *slant*
The amount the characters in the font are slanted away from the vertical. Valid values for slant are **roman** and **italic**. A roman font is the normal, upright appearance of a font, while an italic font is one that is tilted some number of degrees from upright. The closest available slant to the one specified will be chosen. The default slant is **roman**.

-underline *boolean*
The value is a boolean flag that specifies whether characters in this font should be underlined. The default value for underline is **false**.

-overstrike *boolean*
The value is a boolean flag that specifies whether a horizontal line should be drawn through the middle of characters in this font. The default value for overstrike is **false**.

Standard fonts

The following named fonts are supported on all systems, and default to values that match appropriate system defaults.

TkDefaultFont
This font is the default for all GUI items not otherwise specified.

TkTextFont

This font should be used for user text in entry widgets, listboxes etc.

TkFixedFont
This font is the standard fixed-width font.

TkMenuFont
This font is used for menu items.

TkHeadingFont
This font should be used for column headings in lists and tables.

TkCaptionFont
This font should be used for window and dialog caption bars.

TkSmallCaptionFont
This font should be used for captions on contained windows or tool dialogs.

TkIconFont
This font should be used for icon captions.

TkTooltipFont
This font should be used for tooltip windows (transient information windows).

It is *not* advised to change these fonts, as they may be modified by Tk itself in response to system changes. Instead, make a copy of the font and modify that.

Platform specific fonts

The following system fonts are supported:

X Windows
All valid X font names, including those listed by xlsfonts(1), are available.

MS Windows
The following fonts are supported, and are mapped to the user's style defaults.

system ansi device
systemfixed ansifixed oemfixed

macOS
The following fonts are supported, and are mapped to the user's style defaults.

system application menu

Additionally, the following named fonts provide access to the Aqua theme fonts:

systemSystemFont systemEmphasizedSystemFont
systemSmallSystemFont systemSmallEmphasizedSystemFont
systemApplicationFont systemLabelFont
systemViewsFont systemMenuTitleFont
systemMenuItemFont systemMenuItemMarkFont
systemMenuItemCmdKeyFont systemWindowTitleFont
systemPushButtonFont systemUtilityWindowTitleFont
systemAlertHeaderFont systemToolbarFont
systemMiniSystemFont systemDetailSystemFont
systemDetailEmphasizedSystemFont

Example

Fill a text widget with lots of font demonstrators, one for every font family installed on your system:

```
pack [text .t -wrap none] -fill both -expand 1
set count 0
set tabwidth 0
foreach family [lsort -dictionary [font families]] {
    .t tag configure f[incr count] -font [list $family 10]
    .t insert end ${family}:\t {} \
            "This is a simple sampler\n" f$count
    set w [font measure [.t cget -font] ${family}:]
    if {$w + 5 > $tabwidth} {
        set tabwidth [expr {$w + 5}]
        .t configure -tabs $tabwidth
    }
}
```

Copyright (c) 1996 Sun Microsystems, Inc.
Copyright (c) 2006-2007 Daniel A. Steffen das@users.sourceforge.net

fontchooser

Keywords: fontchooser, dialog, font, font selection, font chooser, font panel

Name

fontchooser - control font selection dialog

Synopsis

tk fontchooser configure ?*-option value* ...?
tk fontchooser show
tk fontchooser hide

Description

The tk fontchooser command controls the Tk font selection dialog. It uses the native platform font selection dialog where available, or a dialog implemented in Tcl otherwise.

Unlike most of the other Tk dialog commands, tk fontchooser does not return an immediate result, as on some platforms (macOS) the standard font dialog is modeless while on others (Windows) it is modal. To accommodate this difference, all user interaction with the dialog will be communicated to the caller via callbacks or virtual events.

The tk fontchooser command can have one of the following forms:

tk fontchooser configure ?*-option value* ...?
Set or query one or more of the configurations options below (analogous to Tk widget configuration).

tk fontchooser show
Show the font selection dialog. Depending on the platform, may return immediately or only once the dialog has been withdrawn.

tk fontchooser hide

Hide the font selection dialog if it is visible and cause any pending tk fontchooser **show** command to return.

Configuration options

-parent
Specifies/returns the logical parent window of the font selection dialog (similar to the **-parent** option to other dialogs). The font selection dialog is hidden if it is visible when the parent window is destroyed.

-title
Specifies/returns the title of the dialog. Has no effect on platforms where the font selection dialog does not support titles.

-font
Specifies/returns the font that is currently selected in the dialog if it is visible, or that will be initially selected when the dialog is shown (if supported by the platform). Can be set to the empty string to indicate that no font should be selected. Fonts can be specified in any form given by the "FONT DESCRIPTION" section in the font manual page.

-command
Specifies/returns the command prefix to be called when a font selection has been made by the user. The command prefix is evaluated at the global level after having the specification of the selected font appended. On platforms where the font selection dialog offers the user control of further font attributes (such as color), additional key/value pairs may be appended before evaluation. Can be set to the empty string to indicate that no callback should be invoked. Fonts are specified by a list of form [3] of the "FONT DESCRIPTION" section in the font manual page (i.e. a list of the form *{family size style ?style ...?}*).

-visible
Read-only option that returns a boolean indicating whether the font selection dialog is currently visible. Attempting to set this option results in an error.

Virtual events

<<FontDialogVisibility>>
Sent to the dialog parent whenever the visibility of the font selection dialog changes, both as a result of user action (e.g. disposing of the dialog via OK/Cancel button or close box) and of the tk fontchooser **show**/**hide** commands being called. Binding scripts can determine the current visibility of the dialog by querying the **-visible** configuration option.

<<FontDialogFont>>
Sent to the dialog parent whenever the font selection dialog is visible and the selected font changes, both as a result of user action and of the **-font** configuration option being set. Binding scripts can determine the currently selected font by querying the **-font** configuration option.

Notes

Callers should not expect a result from tk fontchooser **show** and may not assume that the dialog has been withdrawn or closed when the command returns. All user interaction with the dialog is communicated to the caller via the **-command** callback and the **<<FontDialogFont>>** virtual events. It is implementation dependent which exact user actions result in the callback being called resp. the virtual events being sent. Where an Apply or OK button is present in the dialog, that button will

trigger the **-command** callback and **<>** virtual event. On some implementations other user actions may also have that effect; on macOS for instance, the standard font selection dialog immediately reflects all user choices to the caller.

In the presence of multiple widgets intended to be influenced by the font selection dialog, care needs to be taken to correctly handle focus changes: the font selected in the dialog should always match the current font of the widget with the focus, and the **-command** callback should only act on the widget with the focus. The recommended practice is to set font dialog **-font** and **-command** configuration options in per-widget handlers (and if necessary to unset them - i.e. set to the empty string - in corresponding handlers). This is particularly important for implementers of library code using the font selection dialog, to avoid conflicting with application code that may also want to use the dialog.

Because the font selection dialog is application-global, in the presence of multiple interpreters calling tk fontchooser, only the **-command** callback set by the interpreter that most recently called tk fontchooser **configure** or tk fontchooser **show** will be invoked in response to user action and only the **-parent** set by that interpreter will receive **<>** virtual events.

The font dialog implementation may only store (and return) font **actual** data as the value of the **-font** configuration option. This can be an issue when **-font** is set to a named font, if that font is subsequently changed, the font dialog **-font** option needs to be set again to ensure its selected font matches the new value of the named font.

Example

```
proc fontchooserDemo {} {
    wm title . "Font Chooser Demo"
    tk fontchooser configure -parent .
    button .b -command fontchooserToggle -takefocus 0
    fontchooserVisibility .b
    bind . <<TkFontchooserVisibility>> \
            [list fontchooserVisibility .b]
    foreach w {.t1 .t2} {
        text $w -width 20 -height 4 -borderwidth 1 -relief solid
        bind $w <FocusIn> [list fontchooserFocus $w]
        $w insert end "Text Widget $w"
    }
    .t1 configure -font {Courier 14}
    .t2 configure -font {Times 16}
    pack .b .t1 .t2; focus .t1
}
proc fontchooserToggle {} {
    tk fontchooser [expr {
            [tk fontchooser configure -visible] ?
            "hide" : "show"}]
}
proc fontchooserVisibility {w} {
    $w configure -text [expr {
            [tk fontchooser configure -visible] ?
```

```
            "Hide Font Dialog" : "Show Font Dialog"}]
}
proc fontchooserFocus {w} {
    tk fontchooser configure -font [$w cget -font] \
            -command [list fontchooserFontSelection $w]
}
proc fontchooserFontSelection {w font args} {
    $w configure -font [font actual $font]
}
fontchooserDemo
```

Copyright (c) 2008 Daniel A. Steffen das@users.sourceforge.net

frame

Keywords: frame, widget

Name

frame - Create and manipulate 'frame' simple container widgets

Synopsis

frame *pathName* ?*options*?

Standard options

-borderwidth, **-highlightcolor**, **-pady**, **-cursor**, **-highlightthickness**, **-relief**, **-highlightbackground**, **-padx**, **-takefocus**

See the options manual entry for details on the standard options.

Widget-specific options

-background (background/Background)
This option is the same as the standard **-background** option except that its value may also be specified as an empty string. In this case, the widget will display no background or border, and no colors will be consumed from its colormap for its background and border.

-class (class/Class)
Specifies a class for the window. This class will be used when querying the option database for the window's other options, and it will also be used later for other purposes such as bindings. The **-class** option may not be changed with the **configure** widget command.

-colormap (colormap/Colormap)
Specifies a colormap to use for the window. The value may be either **new**, in which case a new colormap is created for the window and its children, or the name of another window (which must be on the same screen and have the same visual as *pathName*), in which case the new window will use the colormap from the specified window. If the **-colormap** option is not specified, the new window uses the same colormap as its parent. This option may not be changed with the **configure** widget command.

-container (container/Container)
The value must be a boolean. If true, it means that this window will be used as a container in which some other application will be embedded (for example, a Tk toplevel can be embedded

using the **-use** option). The window will support the appropriate window manager protocols for things like geometry requests. The window should not have any children of its own in this application. This option may not be changed with the **configure** widget command. Note that **-borderwidth**, **-padx** and **-pady** are ignored when configured as a container since a container has no border.

-height (height/Height)
Specifies the desired height for the window in any of the forms acceptable to Tk_GetPixels. If this option is less than or equal to zero then the window will not request any size at all. Note that this sets the total height of the frame, any **-borderwidth** or similar is not added. Normally **-height** should not be used if a propagating geometry manager, such as grid or pack, is used within the frame since the geometry manager will override the height of the frame.

-visual (visual/Visual)
Specifies visual information for the new window in any of the forms accepted by Tk_GetVisual. If this option is not specified, the new window will use the same visual as its parent. The **-visual** option may not be modified with the **configure** widget command.

-width (width/Width)
Specifies the desired width for the window in any of the forms acceptable to Tk_GetPixels. If this option is less than or equal to zero then the window will not request any size at all. Note that this sets the total width of the frame, any **-borderwidth** or similar is not added. Normally **-width** should not be used if a propagating geometry manager, such as grid or pack , is used within the frame since the geometry manager will override the width of the frame.

Description

The **frame** command creates a new window (given by the *pathName* argument) and makes it into a frame widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the frame such as its background color and relief. The **frame** command returns the path name of the new window.

A frame is a simple widget. Its primary purpose is to act as a spacer or container for complex window layouts. The only features of a frame are its background color and an optional 3-D border to make the frame appear raised or sunken.

Widget command

The **frame** command creates a new Tcl command whose name is the same as the path name of the frame's window. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

PathName is the name of the command, which is the same as the frame widget's path name. *Option* and the *arg*s determine the exact behavior of the command. The following commands are possible for frame widgets:

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **frame** command.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **frame** command.

Bindings

When a new frame is created, it has no default event bindings: frames are not intended to be interactive.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

getOpenFile

Keywords: getOpenFile, file selection dialog

Name

tk_getOpenFile, tk_getSaveFile - pop up a dialog box for the user to select a file to open or save.

Synopsis

tk_getOpenFile ?*option value* ...?
tk_getSaveFile ?*option value* ...?

Description

The procedures **tk_getOpenFile** and **tk_getSaveFile** pop up a dialog box for the user to select a file to open or save. The **tk_getOpenFile** command is usually associated with the **Open** command in the **File** menu. Its purpose is for the user to select an existing file *only*. If the user enters a non-existent file, the dialog box gives the user an error prompt and requires the user to give an alternative selection. If an application allows the user to create new files, it should do so by providing a separate **New** menu command.

The **tk_getSaveFile** command is usually associated with the **Save as** command in the **File** menu. If the user enters a file that already exists, the dialog box prompts the user for confirmation whether the existing file should be overwritten or not.

The following *option-value* pairs are possible as command line arguments to these two commands:

-command *string*
Specifies the prefix of a Tcl command to invoke when the user closes the dialog after having selected an item. This callback is not called if the user cancelled the dialog. The actual command consists of *string* followed by a space and the value selected by the user in the dialog. This is only available on macOS.

-confirmoverwrite *boolean*
Configures how the Save dialog reacts when the selected file already exists, and saving would overwrite it. A true value requests a confirmation dialog be presented to the user. A false

value requests that the overwrite take place without confirmation. Default value is true.

-defaultextension *extension*
Specifies a string that will be appended to the filename if the user enters a filename without an extension. The default value is the empty string, which means no extension will be appended to the filename in any case. This option is ignored on macOS, which does not require extensions to filenames, and the UNIX implementation guesses reasonable values for this from the **-filetypes** option when this is not supplied.

-filetypes *filePatternList*
If a **File types** listbox exists in the file dialog on the particular platform, this option gives the *filetype*s in this listbox. When the user choose a filetype in the listbox, only the files of that type are listed. If this option is unspecified, or if it is set to the empty list, or if the **File types** listbox is not supported by the particular platform then all files are listed regardless of their types. See the section Specifying file patterns below for a discussion on the contents of *filePatternList*.

-initialdir *directory*
Specifies that the files in *directory* should be displayed when the dialog pops up. If this parameter is not specified, the initial directory defaults to the current working directory on non-Windows systems and on Windows systems prior to Vista. On Vista and later systems, the initial directory defaults to the last user-selected directory for the application. If the parameter specifies a relative path, the return value will convert the relative path to an absolute path.

-initialfile *filename*
Specifies a filename to be displayed in the dialog when it pops up.

-message *string*
Specifies a message to include in the client area of the dialog. This is only available on macOS.

-multiple *boolean*
Allows the user to choose multiple files from the Open dialog.

-parent *window*
Makes *window* the logical parent of the file dialog. The file dialog is displayed on top of its parent window. On macOS, this turns the file dialog into a sheet attached to the parent window.

-title *titleString*
Specifies a string to display as the title of the dialog box. If this option is not specified, then a default title is displayed.

-typevariable *variableName*
The global variable *variableName* is used to preselect which filter is used from *filterList* when the dialog box is opened and is updated when the dialog box is closed, to the last selected filter. The variable is read once at the beginning to select the appropriate filter. If the variable does not exist, or its value does not match any filter typename, or is empty (**{}**), the dialog box will revert to the default behavior of selecting the first filter in the list. If the dialog is canceled, the variable is not modified.

If the user selects a file, both **tk_getOpenFile** and **tk_getSaveFile** return the full pathname of this file. If the user cancels the operation, both commands return the empty string.

Specifying file patterns

The *filePatternList* value given by the **-filetypes** option is a list of file patterns. Each file pattern is a list of the form

```
typeName {extension ?extension ...?} ?{macType ?macType ...?}?
```

typeName is the name of the file type described by this file pattern and is the text string that appears in the **File types** listbox. *extension* is a file extension for this file pattern. *macType* is a four-character Macintosh file type. The list of *macType*s is optional and may be omitted for applications that do not need to execute on the Macintosh platform.

Several file patterns may have the same *typeName,* in which case they refer to the same file type and share the same entry in the listbox. When the user selects an entry in the listbox, all the files that match at least one of the file patterns corresponding to that entry are listed. Usually, each file pattern corresponds to a distinct type of file. The use of more than one file pattern for one type of file is only necessary on the Macintosh platform.

On the Macintosh platform, a file matches a file pattern if its name matches at least one of the *extension*(s) AND it belongs to at least one of the *macType*(s) of the file pattern. For example, the **C Source Files** file pattern in the sample code matches with files that have a **.c** extension AND belong to the *macType* **TEXT**. To use the OR rule instead, you can use two file patterns, one with the *extensions* only and the other with the *macType* only. The **GIF Files** file type in the sample code matches files that *either* have a **.gif** extension OR belong to the *macType* **GIFF**.

On the Unix and Windows platforms, a file matches a file pattern if its name matches at least one of the *extension*(s) of the file pattern. The *macType*s are ignored.

Specifying extensions

On the Unix and Macintosh platforms, extensions are matched using glob-style pattern matching. On the Windows platform, extensions are matched by the underlying operating system. The types of possible extensions are:

1. the special extension "*" matches any file;
2. the special extension "" matches any files that do not have an extension (i.e., the filename contains no full stop character);
3. any character string that does not contain any wild card characters (* and ?).

Due to the different pattern matching rules on the various platforms, to ensure portability, wild card characters are not allowed in the extensions, except as in the special extension "*". Extensions without a full stop character (e.g. "~") are allowed but may not work on all platforms.

Example

```
set types {
    {{Text Files}       {.txt}        }
    {{TCL Scripts}      {.tcl}        }
    {{C Source Files}   {.c}      TEXT}
    {{GIF Files}        {.gif}        }
    {{GIF Files}        {}        GIFF}
    {{All Files}        *             }
}
```

```
set filename [tk_getOpenFile -filetypes $types]

if {$filename ne ""} {
    # Open the file ...
}
```

Copyright (c) 1996 Sun Microsystems, Inc.

grab

Keywords: grab, keyboard events, pointer events, window

Name

grab - Confine pointer and keyboard events to a window sub-tree

Synopsis

grab ?**-global**? *window*
grab *option* ?*arg arg* ...?

Description

This command implements simple pointer and keyboard grabs for Tk. Tk's grabs are different than the grabs described in the Xlib documentation. When a grab is set for a particular window, Tk restricts all pointer events to the grab window and its descendants in Tk's window hierarchy. Whenever the pointer is within the grab window's subtree, the pointer will behave exactly the same as if there had been no grab at all and all events will be reported in the normal fashion. When the pointer is outside *window*'s tree, button presses and releases and mouse motion events are reported to *window*, and window entry and window exit events are ignored. The grab subtree "owns" the pointer: windows outside the grab subtree will be visible on the screen but they will be insensitive until the grab is released. The tree of windows underneath the grab window can include top-level windows, in which case all of those top-level windows and their descendants will continue to receive mouse events during the grab.

Two forms of grabs are possible: local and global. A local grab affects only the grabbing application: events will be reported to other applications as if the grab had never occurred. Grabs are local by default. A global grab locks out all applications on the screen, so that only the given subtree of the grabbing application will be sensitive to pointer events (mouse button presses, mouse button releases, pointer motions, window entries, and window exits). During global grabs the window manager will not receive pointer events either.

During local grabs, keyboard events (key presses and key releases) are delivered as usual: the window manager controls which application receives keyboard events, and if they are sent to any window in the grabbing application then they are redirected to the focus window. During a global grab Tk grabs the keyboard so that all keyboard events are always sent to the grabbing application. The focus command is still used to determine which window in the application receives the keyboard events. The keyboard grab is released when the grab is released.

On macOS a global grab affects all windows created by one Tk process. No window in that process other than the grab window can even be focused, hence no other window receives key or mouse events. A local grab on macOS affects all windows created by one Tcl interpreter. It is possible to focus any window belonging to the Tk process during a local grab but the grab

window is the only window created by its interpreter which receives key or mouse events. Windows belonging to the same process but created by different interpreters continue to receive key and mouse events normally.

Grabs apply to particular displays. If an application has windows on multiple displays then it can establish a separate grab on each display. The grab on a particular display affects only the windows on that display. It is possible for different applications on a single display to have simultaneous local grabs, but only one application can have a global grab on a given display at once.

The **grab** command can take any of the following forms:

grab ?-global? *window*
: Same as **grab set**, described below.

grab current ?*window*?
: If *window* is specified, returns the name of the current grab window in this application for *window*'s display, or an empty string if there is no such window. If *window* is omitted, the command returns a list whose elements are all of the windows grabbed by this application for all displays, or an empty string if the application has no grabs.

grab release *window*
: Releases the grab on *window* if there is one, otherwise does nothing. Returns an empty string.

grab set ?-global? *window*
: Sets a grab on *window*. If **-global** is specified then the grab is global, otherwise it is local. If a grab was already in effect for this application on *window*'s display then it is automatically released. If there is already a grab on *window* and it has the same global/local form as the requested grab, then the command does nothing. Returns an empty string.

grab status *window*
: Returns **none** if no grab is currently set on *window*, **local** if a local grab is set on *window*, and **global** if a global grab is set.

Warning

It is very easy to use global grabs to render a display completely unusable (e.g. by setting a grab on a widget which does not respond to events and not providing any mechanism for releasing the grab). Take *extreme* care when using them!

Bugs

It took an incredibly complex and gross implementation to produce the simple grab effect described above. Given the current implementation, it is not safe for applications to use the Xlib grab facilities at all except through the Tk grab procedures. If applications try to manipulate X's grab mechanisms directly, things will probably break.

If a single process is managing several different Tk applications, only one of those applications can have a local grab for a given display at any given time. If the applications are in different processes, this restriction does not exist.

Example

Set a grab so that only one button may be clicked out of a group. The other buttons are unresponsive to the mouse until the middle button is clicked.

```
pack [button .b1 -text "Click me! #1" -command {destroy .b1}]
pack [button .b2 -text "Click me! #2" -command {destroy .b2}]
pack [button .b3 -text "Click me! #3" -command {destroy .b3}]
grab .b2
```

Copyright (c) 1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

grid

Keywords: grid, geometry manager, location, cell, propagation, size, pack

Name

grid - Geometry manager that arranges widgets in a grid

Synopsis

grid *option arg* ?*arg* ...?

Description

The **grid** command is used to communicate with the grid geometry manager that arranges widgets in rows and columns inside of another window, called the geometry container (or container window). The **grid** command can have any of several forms, depending on the *option* argument:

grid *window* ?*window ...*? ?*options*?
: If the first argument to **grid** is suitable as the first window argument to **grid configure**, either a window name (any value starting with **.**) or one of the characters **x** or **^** (see the Relative placement section below), then the command is processed in the same way as **grid configure**.

grid anchor *window* ?*anchor*?
: The anchor value controls how to place the grid within the container window when no row/column has any weight. See The grid algorithm below for further details. The default *anchor* is *nw*.

grid bbox *window* ?*column row*? ?*column2 row2*?
: With no arguments, the bounding box (in pixels) of the grid is returned. The return value consists of 4 integers. The first two are the pixel offset from the container window (x then y) of the top-left corner of the grid, and the second two integers are the width and height of the grid, also in pixels. If a single *column* and *row* is specified on the command line, then the bounding box for that cell is returned, where the top left cell is numbered from zero. If both *column* and *row* arguments are specified, then the bounding box spanning the rows and columns indicated is returned.

grid columnconfigure *window index* ?*-option value...*?
: Query or set the column properties of the *index* column of the geometry container, *window*. The valid options are **-minsize**, **-weight**, **-uniform** and **-pad**. If one or more options are provided, then *index* may be given as a list of column indices to which the configuration options will operate on. Indices may be integers, window names or the keyword *all*. For *all* the options apply to all columns currently occupied be content windows. For a window name, that window must be a content of this container and the options apply to all columns currently

occupied be the content. The **-minsize** option sets the minimum size, in screen units, that will be permitted for this column. The **-weight** option (an integer value) sets the relative weight for apportioning any extra spaces among columns. A weight of zero (0) indicates the column will not deviate from its requested size. A column whose weight is two will grow at twice the rate as a column of weight one when extra space is allocated to the layout. The **-uniform** option, when a non-empty value is supplied, places the column in a *uniform group* with other columns that have the same value for **-uniform**. The space for columns belonging to a uniform group is allocated so that their sizes are always in strict proportion to their **-weight** values. See The grid algorithm below for further details. The **-pad** option specifies the number of screen units that will be added to the largest window contained completely in that column when the grid geometry manager requests a size from the containing window. If only an option is specified, with no value, the current value of that option is returned. If only the container window and index is specified, all the current settings are returned in a list of "-option value" pairs.

grid configure *window* ?*window ...*? ?*options*?

The arguments consist of the names of one or more content windows followed by pairs of arguments that specify how to manage the content. The characters **-**, **x** and **^**, can be specified instead of a window name to alter the default location of a *window*, as described in the Relative placement section, below. The following options are supported:

-column *n*

Insert the window so that it occupies the *n*th column in the grid. Column numbers start with 0. If this option is not supplied, then the window is arranged just to the right of previous window specified on this call to **grid**, or column "0" if it is the first window. For each **x** that immediately precedes the *window*, the column position is incremented by one. Thus the **x** represents a blank column for this row in the grid.

-columnspan *n*

Insert the window so that it occupies *n* columns in the grid. The default is one column, unless the window name is followed by a **-**, in which case the columnspan is incremented once for each immediately following **-**.

-in *container*

Insert the window(s) in the container window given by *container*. The default is the first window's parent window.

-ipadx *amount*

The *amount* specifies how much horizontal internal padding to leave on each side of the content. This is space is added inside the content border. The *amount* must be a valid screen distance, such as **2** or **.5c**. It defaults to 0.

-ipady *amount*

The *amount* specifies how much vertical internal padding to leave on the top and bottom of the content. This space is added inside the content border. The *amount* defaults to 0.

-padx *amount*

The *amount* specifies how much horizontal external padding to leave on each side of the content, in screen units. *Amount* may be a list of two values to specify padding for left and right separately. The *amount* defaults to 0. This space is added outside the content border.

-pady *amount*

The *amount* specifies how much vertical external padding to leave on the top and bottom

of the content, in screen units. *Amount* may be a list of two values to specify padding for top and bottom separately. The *amount* defaults to 0. This space is added outside the content border.

-row *n*
Insert the content so that it occupies the *n*th row in the grid. Row numbers start with 0. If this option is not supplied, then the content is arranged on the same row as the previous content specified on this call to **grid**, or the next row after the highest occupied row if this is the first content.

-rowspan *n*
Insert the content so that it occupies *n* rows in the grid. The default is one row. If the next **grid** command contains **^** characters instead of *content* that line up with the columns of this *content*, then the **rowspan** of this *content* is extended by one.

-sticky *style*
If a content's cell is larger than its requested dimensions, this option may be used to position (or stretch) the content within its cell. *Style* is a string that contains zero or more of the characters **n**, **s**, **e** or **w**. The string can optionally contains spaces or commas, but they are ignored. Each letter refers to a side (north, south, east, or west) that the content will "stick" to. If both **n** and **s** (or **e** and **w**) are specified, the content will be stretched to fill the entire height (or width) of its cavity. The **-sticky** option subsumes the combination of **-anchor** and **-fill** that is used by pack. The default is "", which causes the content to be centered in its cavity, at its requested size.

If any of the content is already managed by the geometry manager then any unspecified options for them retain their previous values rather than receiving default values.

grid forget *window* ?*window ...*?
Removes each of the *window*s from grid for its container and unmaps their windows. The content will no longer be managed by the grid geometry manager. The configuration options for that window are forgotten, so that if the window is managed once more by the grid geometry manager, the initial default settings are used.

grid info *window*
Returns a list whose elements are the current configuration state of the content given by *window* in the same option-value form that might be specified to **grid configure**. The first two elements of the list are "**-in** *container*" where *container* is the windows's container window.

grid location *window x y*
Given *x* and *y* values in screen units relative to the container window, the column and row number at that *x* and *y* location is returned. For locations that are above or to the left of the grid, **-1** is returned.

grid propagate *window* ?*boolean*?
If *boolean* has a true boolean value such as **1** or **on** then propagation is enabled for *window*, which must be a window name (see Geometry propagation below). If *boolean* has a false boolean value then propagation is disabled for *window*. In either of these cases an empty string is returned. If *boolean* is omitted then the command returns **0** or **1** to indicate whether propagation is currently enabled for *window*. Propagation is enabled by default.

grid rowconfigure *window index* ?*-option value...*?

Query or set the row properties of the *index* row of the geometry window, *window*. The valid options are **-minsize**, **-weight**, **-uniform** and **-pad**. If one or more options are provided, then *index* may be given as a list of row indices to which the configuration options will operate on. Indices may be integers, window names or the keyword *all*. For *all* the options apply to all rows currently occupied by content windows. For a window name, that window must be a content window of this container and the options apply to all rows currently occupied by the container window. The **-minsize** option sets the minimum size, in screen units, that will be permitted for this row. The **-weight** option (an integer value) sets the relative weight for apportioning any extra spaces among rows. A weight of zero (0) indicates the row will not deviate from its requested size. A row whose weight is two will grow at twice the rate as a row of weight one when extra space is allocated to the layout. The **-uniform** option, when a non-empty value is supplied, places the row in a *uniform group* with other rows that have the same value for **-uniform**. The space for rows belonging to a uniform group is allocated so that their sizes are always in strict proportion to their **-weight** values. See The grid algorithm below for further details. The **-pad** option specifies the number of screen units that will be added to the largest window contained completely in that row when the grid geometry manager requests a size from the containing window. If only an option is specified, with no value, the current value of that option is returned. If only the container window and index is specified, all the current settings are returned in a list of "-option value" pairs.

grid remove *window* ?*window ...*?

Removes each of the *window*s from grid for its container and unmaps their windows. The content will no longer be managed by the grid geometry manager. However, the configuration options for that window are remembered, so that if the content window is managed once more by the grid geometry manager, the previous values are retained.

grid size *container*

Returns the size of the grid (in columns then rows) for *container*. The size is determined either by the *content* occupying the largest row or column, or the largest column or row with a **-minsize**, **-weight**, or **-pad** that is non-zero.

grid slaves *window* ?*-option value*?

If no options are supplied, a list of all of the content in *window* are returned, most recently managed first. *Option* can be either **-row** or **-column** which causes only the content in the row (or column) specified by *value* to be returned.

grid content window ?-option value?

Synonym for grid slaves window ?-option value?.

Relative placement

The **grid** command contains a limited set of capabilities that permit layouts to be created without specifying the row and column information for each content. This permits content to be rearranged, added, or removed without the need to explicitly specify row and column information. When no column or row information is specified for a *content*, default values are chosen for **-column**, **-row**, **-columnspan** and **-rowspan** at the time the *content* is managed. The values are chosen based upon the current layout of the grid, the position of the *content* relative to other *content*s in the same grid command, and the presence of the characters **-**, **x**, and **^** in **grid** command where *content* names are normally expected.

-

This increases the **-columnspan** of the *content* to the left. Several **-**'s in a row will successively increase the number of columns spanned. A **-** may not follow a **^** or a **x**, nor may it be the first *content* argument to **grid configure**.

x

This leaves an empty column between the *content* on the left and the *content* on the right.

^

This extends the **-rowspan** of the *content* above the **^**'s in the grid. The number of **^**'s in a row must match the number of columns spanned by the *content* above it.

The grid algorithm

The grid geometry manager lays out its content in three steps. In the first step, the minimum size needed to fit all of the content is computed, then (if propagation is turned on), a request is made of the container window to become that size. In the second step, the requested size is compared against the actual size of the container. If the sizes are different, then spaces is added to or taken away from the layout as needed. For the final step, each content is positioned in its row(s) and column(s) based on the setting of its *sticky* flag.

To compute the minimum size of a layout, the grid geometry manager first looks at all content whose **-columnspan** and **-rowspan** values are one, and computes the nominal size of each row or column to be either the *minsize* for that row or column, or the sum of the *pad*ding plus the size of the largest content, whichever is greater. After that the rows or columns in each uniform group adapt to each other. Then the content whose row-spans or column-spans are greater than one are examined. If a group of rows or columns need to be increased in size in order to accommodate these content, then extra space is added to each row or column in the group according to its *weight*. For each group whose weights are all zero, the additional space is apportioned equally.

When multiple rows or columns belong to a uniform group, the space allocated to them is always in proportion to their weights. (A weight of zero is considered to be 1.) In other words, a row or column configured with **-weight 1 -uniform a** will have exactly the same size as any other row or column configured with **-weight 1 -uniform a**. A row or column configured with **-weight 2 -uniform b** will be exactly twice as large as one that is configured with **-weight 1 -uniform b**.

More technically, each row or column in the group will have a size equal to *k***weight* for some constant *k*. The constant *k* is chosen so that no row or column becomes smaller than its minimum size. For example, if all rows or columns in a group have the same weight, then each row or column will have the same size as the largest row or column in the group.

For containers whose size is larger than the requested layout, the additional space is apportioned according to the row and column weights. If all of the weights are zero, the layout is placed within its container according to the *anchor* value. For containers whose size is smaller than the requested layout, space is taken away from columns and rows according to their weights. However, once a column or row shrinks to its minsize, its weight is taken to be zero. If more space needs to be removed from a layout than would be permitted, as when all the rows or columns are at their minimum sizes, the layout is placed and clipped according to the *anchor* value.

Geometry propagation

The grid geometry manager normally computes how large a container must be to just exactly

meet the needs of its content, and it sets the requested width and height of the container to these dimensions. This causes geometry information to propagate up through a window hierarchy to a top-level window so that the entire sub-tree sizes itself to fit the needs of the leaf windows. However, the **grid propagate** command may be used to turn off propagation for one or more containers. If propagation is disabled then grid will not set the requested width and height of the container window. This may be useful if, for example, you wish for a container window to have a fixed size that you specify.

Restrictions on container windows

The container for each content must either be the content's parent (the default) or a descendant of the content's parent. This restriction is necessary to guarantee that the content can be placed over any part of its container that is visible without danger of the content being clipped by its parent. In addition, all content in one call to **grid** must have the same container.

Stacking order

If the container for a content is not its parent then you must make sure that the content is higher in the stacking order than the container. Otherwise the container will obscure the content and it will appear as if the content has not been managed correctly. The easiest way to make sure the content is higher than the container is to create the container window first: the most recently created window will be highest in the stacking order.

Credits

The **grid** command is based on ideas taken from the *GridBag* geometry manager written by Doug. Stein, and the **blt_table** geometry manager, written by George Howlett.

Examples

A toplevel window containing a text widget and two scrollbars:

```
# Make the widgets
toplevel .t
text .t.txt -wrap none -xscroll {.t.h set} -yscroll {.t.v set}
scrollbar .t.v -orient vertical   -command {.t.txt yview}
scrollbar .t.h -orient horizontal -command {.t.txt xview}

# Lay them out
grid .t.txt .t.v -sticky nsew
grid .t.h        -sticky nsew

# Tell the text widget to take all the extra room
grid rowconfigure    .t .t.txt -weight 1
grid columnconfigure .t .t.txt -weight 1
```

Three widgets of equal width, despite their different "natural" widths:

```
button .b -text "Foo"
entry .e -textvariable foo ; set foo "Hello World!"
label .l -text "This is a fairly long piece of text"

grid .b .e .l -sticky ew
grid columnconfigure . "all" -uniform allTheSame
```

Copyright (c) 1996 Sun Microsystems, Inc.

image

Keywords: image, height, types of images, width

Name

image - Create and manipulate images

Synopsis

image *option* ?*arg arg* ...?

Description

The **image** command is used to create, delete, and query images. It can take several different forms, depending on the *option* argument. The legal forms are:

image create *type* ?*name*? ?*option value ...*?

Creates a new image and a command with the same name and returns its name. *type* specifies the type of the image, which must be one of the types currently defined (e.g., bitmap). *name* specifies the name for the image; if it is omitted then Tk picks a name of the form **image***x*, where *x* is an integer. There may be any number of *option-value* pairs, which provide configuration options for the new image. The legal set of options is defined separately for each image type; see below for details on the options for built-in image types. If an image already exists by the given name then it is replaced with the new image and any instances of that image will redisplay with the new contents. It is important to note that the image command will silently overwrite any procedure that may currently be defined by the given name, so choose the name wisely. It is recommended to use a separate namespace for image names (e.g., **::img::logo**, **::img::large**).

image delete ?*name name* ...?

Deletes each of the named images and returns an empty string. If there are instances of the images displayed in widgets, the images will not actually be deleted until all of the instances are released. However, the association between the instances and the image manager will be dropped. Existing instances will retain their sizes but redisplay as empty areas. If a deleted image is recreated with another call to **image create**, the existing instances will use the new image.

image height *name*

Returns a decimal string giving the height of image *name* in pixels.

image inuse *name*

Returns a boolean value indicating whether or not the image given by *name* is in use by any widgets.

image names

Returns a list containing the names of all existing images.

image type *name*
Returns the type of image *name* (the value of the *type* argument to **image create** when the image was created).

image types
Returns a list whose elements are all of the valid image types (i.e., all of the values that may be supplied for the *type* argument to **image create**).

image width *name*
Returns a decimal string giving the width of image *name* in pixels.

Additional operations (e.g. writing the image to a file) may be available as subcommands of the image instance command. See the manual page for the particular image type for details.

Built-in image types

The following image types are defined by Tk so they will be available in any Tk application. Individual applications or extensions may define additional types.

bitmap
Each pixel in the image displays a foreground color, a background color, or nothing. See the bitmap manual entry for more information.

photo
Displays a variety of full-color images, using dithering to approximate colors on displays with limited color capabilities. See the photo manual entry for more information.

Copyright (c) 1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

keysyms

Keywords: keysyms, bind, binding, event, keysym

Name

keysyms - keysyms recognized by Tk

Description

Tk recognizes many keysyms when specifying key bindings (e.g., "bind**.** "). The following list enumerates the keysyms that will be recognized by Tk. Note that not all keysyms will be valid on all platforms, and some keysyms are also available on platforms that have a different native name for that key. For example, on Unix systems, the presence of a particular keysym is dependent on the configuration of the keyboard modifier map. This list shows keysyms along with their decimal and hexadecimal values.

```
space                           32      0x20
exclam                          33      0x21
quotedbl                        34      0x22
numbersign                      35      0x23
dollar                          36      0x24
percent                         37      0x25
ampersand                       38      0x26
```

```
apostrophe                      39      0x27
parenleft                       40      0x28
parenright                      41      0x29
asterisk                        42      0x2A
plus                            43      0x2B
comma                           44      0x2C
minus                           45      0x2D
period                          46      0x2E
slash                           47      0x2F
0                               48      0x30
1                               49      0x31
2                               50      0x32
3                               51      0x33
4                               52      0x34
5                               53      0x35
6                               54      0x36
7                               55      0x37
8                               56      0x38
9                               57      0x39
colon                           58      0x3A
semicolon                       59      0x3B
less                            60      0x3C
equal                           61      0x3D
greater                         62      0x3E
question                        63      0x3F
at                              64      0x40
A                               65      0x41
B                               66      0x42
C                               67      0x43
D                               68      0x44
E                               69      0x45
F                               70      0x46
G                               71      0x47
H                               72      0x48
I                               73      0x49
J                               74      0x4A
K                               75      0x4B
L                               76      0x4C
M                               77      0x4D
N                               78      0x4E
O                               79      0x4F
P                               80      0x50
Q                               81      0x51
R                               82      0x52
S                               83      0x53
T                               84      0x54
```

```
U                                85     0x55
V                                86     0x56
W                                87     0x57
X                                88     0x58
Y                                89     0x59
Z                                90     0x5A
bracketleft                      91     0x5B
backslash                        92     0x5C
bracketright                     93     0x5D
asciicircum                      94     0x5E
underscore                       95     0x5F
grave                            96     0x60
a                                97     0x61
b                                98     0x62
c                                99     0x63
d                               100     0x64
e                               101     0x65
f                               102     0x66
g                               103     0x67
h                               104     0x68
i                               105     0x69
j                               106     0x6A
k                               107     0x6B
l                               108     0x6C
m                               109     0x6D
n                               110     0x6E
o                               111     0x6F
p                               112     0x70
q                               113     0x71
r                               114     0x72
s                               115     0x73
t                               116     0x74
u                               117     0x75
v                               118     0x76
w                               119     0x77
x                               120     0x78
y                               121     0x79
z                               122     0x7A
braceleft                       123     0x7B
bar                             124     0x7C
braceright                      125     0x7D
asciitilde                      126     0x7E
nobreakspace                    160     0xA0
exclamdown                      161     0xA1
cent                            162     0xA2
sterling                        163     0xA3
```

```
currency                        164     0xA4
yen                             165     0xA5
brokenbar                       166     0xA6
section                         167     0xA7
diaeresis                       168     0xA8
copyright                       169     0xA9
ordfeminine                     170     0xAA
guillemetleft                   171     0xAB
notsign                         172     0xAC
hyphen                          173     0xAD
registered                      174     0xAE
macron                          175     0xAF
degree                          176     0xB0
plusminus                       177     0xB1
twosuperior                     178     0xB2
threesuperior                   179     0xB3
acute                           180     0xB4
mu                              181     0xB5
paragraph                       182     0xB6
periodcentered                  183     0xB7
cedilla                         184     0xB8
onesuperior                     185     0xB9
ordmasculine                    186     0xBA
guillemetright                  187     0xBB
onequarter                      188     0xBC
onehalf                         189     0xBD
threequarters                   190     0xBE
questiondown                    191     0xBF
Agrave                          192     0xC0
Aacute                          193     0xC1
Acircumflex                     194     0xC2
Atilde                          195     0xC3
Adiaeresis                      196     0xC4
Aring                           197     0xC5
AE                              198     0xC6
Ccedilla                        199     0xC7
Egrave                          200     0xC8
Eacute                          201     0xC9
Ecircumflex                     202     0xCA
Ediaeresis                      203     0xCB
Igrave                          204     0xCC
Iacute                          205     0xCD
Icircumflex                     206     0xCE
Idiaeresis                      207     0xCF
ETH                             208     0xD0
Ntilde                          209     0xD1
```

```
Ograve                          210     0xD2
Oacute                          211     0xD3
Ocircumflex                     212     0xD4
Otilde                          213     0xD5
Odiaeresis                      214     0xD6
multiply                        215     0xD7
Oslash                          216     0xD8
Ugrave                          217     0xD9
Uacute                          218     0xDA
Ucircumflex                     219     0xDB
Udiaeresis                      220     0xDC
Yacute                          221     0xDD
THORN                           222     0xDE
ssharp                          223     0xDF
agrave                          224     0xE0
aacute                          225     0xE1
acircumflex                     226     0xE2
atilde                          227     0xE3
adiaeresis                      228     0xE4
aring                           229     0xE5
ae                              230     0xE6
ccedilla                        231     0xE7
egrave                          232     0xE8
eacute                          233     0xE9
ecircumflex                     234     0xEA
ediaeresis                      235     0xEB
igrave                          236     0xEC
iacute                          237     0xED
icircumflex                     238     0xEE
idiaeresis                      239     0xEF
eth                             240     0xF0
ntilde                          241     0xF1
ograve                          242     0xF2
oacute                          243     0xF3
ocircumflex                     244     0xF4
otilde                          245     0xF5
odiaeresis                      246     0xF6
division                        247     0xF7
oslash                          248     0xF8
ugrave                          249     0xF9
uacute                          250     0xFA
ucircumflex                     251     0xFB
udiaeresis                      252     0xFC
yacute                          253     0xFD
thorn                           254     0xFE
ydiaeresis                      255     0xFF
```

Aogonek	417	0x1A1
breve	418	0x1A2
Lstroke	419	0x1A3
Lcaron	421	0x1A5
Sacute	422	0x1A6
Scaron	425	0x1A9
Scedilla	426	0x1AA
Tcaron	427	0x1AB
Zacute	428	0x1AC
Zcaron	430	0x1AE
Zabovedot	431	0x1AF
aogonek	433	0x1B1
ogonek	434	0x1B2
lstroke	435	0x1B3
lcaron	437	0x1B5
sacute	438	0x1B6
caron	439	0x1B7
scaron	441	0x1B9
scedilla	442	0x1BA
tcaron	443	0x1BB
zacute	444	0x1BC
doubleacute	445	0x1BD
zcaron	446	0x1BE
zabovedot	447	0x1BF
Racute	448	0x1C0
Abreve	451	0x1C3
Lacute	453	0x1C5
Cacute	454	0x1C6
Ccaron	456	0x1C8
Eogonek	458	0x1CA
Ecaron	460	0x1CC
Dcaron	463	0x1CF
Dstroke	464	0x1D0
Nacute	465	0x1D1
Ncaron	466	0x1D2
Odoubleacute	469	0x1D5
Rcaron	472	0x1D8
Uring	473	0x1D9
Udoubleacute	475	0x1DB
Tcedilla	478	0x1DE
racute	480	0x1E0
abreve	483	0x1E3
lacute	485	0x1E5
cacute	486	0x1E6
ccaron	488	0x1E8
eogonek	490	0x1EA

ecaron	492	0x1EC
dcaron	495	0x1EF
dstroke	496	0x1F0
nacute	497	0x1F1
ncaron	498	0x1F2
odoubleacute	501	0x1F5
rcaron	504	0x1F8
uring	505	0x1F9
udoubleacute	507	0x1FB
tcedilla	510	0x1FE
abovedot	511	0x1FF
Hstroke	673	0x2A1
Hcircumflex	678	0x2A6
Iabovedot	681	0x2A9
Gbreve	683	0x2AB
Jcircumflex	684	0x2AC
hstroke	689	0x2B1
hcircumflex	694	0x2B6
idotless	697	0x2B9
gbreve	699	0x2BB
jcircumflex	700	0x2BC
Cabovedot	709	0x2C5
Ccircumflex	710	0x2C6
Gabovedot	725	0x2D5
Gcircumflex	728	0x2D8
Ubreve	733	0x2DD
Scircumflex	734	0x2DE
cabovedot	741	0x2E5
ccircumflex	742	0x2E6
gabovedot	757	0x2F5
gcircumflex	760	0x2F8
ubreve	765	0x2FD
scircumflex	766	0x2FE
kra	930	0x3A2
Rcedilla	931	0x3A3
Itilde	933	0x3A5
Lcedilla	934	0x3A6
Emacron	938	0x3AA
Gcedilla	939	0x3AB
Tslash	940	0x3AC
rcedilla	947	0x3B3
itilde	949	0x3B5
lcedilla	950	0x3B6
emacron	954	0x3BA
gcedilla	955	0x3BB
tslash	956	0x3BC

ENG	957	0x3BD
eng	959	0x3BF
Amacron	960	0x3C0
Iogonek	967	0x3C7
Eabovedot	972	0x3CC
Imacron	975	0x3CF
Ncedilla	977	0x3D1
Omacron	978	0x3D2
Kcedilla	979	0x3D3
Uogonek	985	0x3D9
Utilde	989	0x3DD
Umacron	990	0x3DE
amacron	992	0x3E0
iogonek	999	0x3E7
eabovedot	1004	0x3EC
imacron	1007	0x3EF
ncedilla	1009	0x3F1
omacron	1010	0x3F2
kcedilla	1011	0x3F3
uogonek	1017	0x3F9
utilde	1021	0x3FD
umacron	1022	0x3FE
overline	1150	0x47E
kana_fullstop	1185	0x4A1
kana_openingbracket	1186	0x4A2
kana_closingbracket	1187	0x4A3
kana_comma	1188	0x4A4
kana_conjunctive	1189	0x4A5
kana_WO	1190	0x4A6
kana_a	1191	0x4A7
kana_i	1192	0x4A8
kana_u	1193	0x4A9
kana_e	1194	0x4AA
kana_o	1195	0x4AB
kana_ya	1196	0x4AC
kana_yu	1197	0x4AD
kana_yo	1198	0x4AE
kana_tsu	1199	0x4AF
prolongedsound	1200	0x4B0
kana_A	1201	0x4B1
kana_I	1202	0x4B2
kana_U	1203	0x4B3
kana_E	1204	0x4B4
kana_O	1205	0x4B5
kana_KA	1206	0x4B6
kana_KI	1207	0x4B7

kana_KU	1208	0x4B8
kana_KE	1209	0x4B9
kana_KO	1210	0x4BA
kana_SA	1211	0x4BB
kana_SHI	1212	0x4BC
kana_SU	1213	0x4BD
kana_SE	1214	0x4BE
kana_SO	1215	0x4BF
kana_TA	1216	0x4C0
kana_CHI	1217	0x4C1
kana_TSU	1218	0x4C2
kana_TE	1219	0x4C3
kana_TO	1220	0x4C4
kana_NA	1221	0x4C5
kana_NI	1222	0x4C6
kana_NU	1223	0x4C7
kana_NE	1224	0x4C8
kana_NO	1225	0x4C9
kana_HA	1226	0x4CA
kana_HI	1227	0x4CB
kana_FU	1228	0x4CC
kana_HE	1229	0x4CD
kana_HO	1230	0x4CE
kana_MA	1231	0x4CF
kana_MI	1232	0x4D0
kana_MU	1233	0x4D1
kana_ME	1234	0x4D2
kana_MO	1235	0x4D3
kana_YA	1236	0x4D4
kana_YU	1237	0x4D5
kana_YO	1238	0x4D6
kana_RA	1239	0x4D7
kana_RI	1240	0x4D8
kana_RU	1241	0x4D9
kana_RE	1242	0x4DA
kana_RO	1243	0x4DB
kana_WA	1244	0x4DC
kana_N	1245	0x4DD
voicedsound	1246	0x4DE
semivoicedsound	1247	0x4DF

Arabic_comma	1452	0x5AC
Arabic_semicolon	1467	0x5BB
Arabic_question_mark	1471	0x5BF
Arabic_hamza	1473	0x5C1
Arabic_maddaonalef	1474	0x5C2

```
Arabic_hamzaonalef              1475     0x5C3
Arabic_hamzaonwaw               1476     0x5C4
Arabic_hamzaunderalef           1477     0x5C5
Arabic_hamzaonyeh               1478     0x5C6
Arabic_alef                     1479     0x5C7
Arabic_beh                      1480     0x5C8
Arabic_tehmarbuta               1481     0x5C9
Arabic_teh                      1482     0x5CA
Arabic_theh                     1483     0x5CB
Arabic_jeem                     1484     0x5CC
Arabic_hah                      1485     0x5CD
Arabic_khah                     1486     0x5CE
Arabic_dal                      1487     0x5CF
Arabic_thal                     1488     0x5D0
Arabic_ra                       1489     0x5D1
Arabic_zain                     1490     0x5D2
Arabic_seen                     1491     0x5D3
Arabic_sheen                    1492     0x5D4
Arabic_sad                      1493     0x5D5
Arabic_dad                      1494     0x5D6
Arabic_tah                      1495     0x5D7
Arabic_zah                      1496     0x5D8
Arabic_ain                      1497     0x5D9
Arabic_ghain                    1498     0x5DA
Arabic_tatweel                  1504     0x5E0
Arabic_feh                      1505     0x5E1
Arabic_qaf                      1506     0x5E2
Arabic_kaf                      1507     0x5E3
Arabic_lam                      1508     0x5E4
Arabic_meem                     1509     0x5E5
Arabic_noon                     1510     0x5E6
Arabic_ha                       1511     0x5E7
Arabic_waw                      1512     0x5E8
Arabic_alefmaksura              1513     0x5E9
Arabic_yeh                      1514     0x5EA
Arabic_fathatan                 1515     0x5EB
Arabic_dammatan                 1516     0x5EC
Arabic_kasratan                 1517     0x5ED
Arabic_fatha                    1518     0x5EE
Arabic_damma                    1519     0x5EF
Arabic_kasra                    1520     0x5F0
Arabic_shadda                   1521     0x5F1
Arabic_sukun                    1522     0x5F2
Serbian_dje                     1697     0x6A1
Macedonia_gje                   1698     0x6A2
Cyrillic_io                     1699     0x6A3
```

Ukrainian_ie	1700	0x6A4
Macedonia_dse	1701	0x6A5
Ukrainian_i	1702	0x6A6
Ukrainian_yi	1703	0x6A7
Cyrillic_je	1704	0x6A8
Cyrillic_lje	1705	0x6A9
Cyrillic_nje	1706	0x6AA
Serbian_tshe	1707	0x6AB
Macedonia_kje	1708	0x6AC
Ukrainian_ghe_with_upturn	1709	0x6AD
Byelorussian_shortu	1710	0x6AE
Cyrillic_dzhe	1711	0x6AF
numerosign	1712	0x6B0
Serbian_DJE	1713	0x6B1
Macedonia_GJE	1714	0x6B2
Cyrillic_IO	1715	0x6B3
Ukrainian_IE	1716	0x6B4
Macedonia_DSE	1717	0x6B5
Ukrainian_I	1718	0x6B6
Ukrainian_YI	1719	0x6B7
Cyrillic_JE	1720	0x6B8
Cyrillic_LJE	1721	0x6B9
Cyrillic_NJE	1722	0x6BA
Serbian_TSHE	1723	0x6BB
Macedonia_KJE	1724	0x6BC
Ukrainian_GHE_WITH_UPTURN	1725	0x6BD
Byelorussian_SHORTU	1726	0x6BE
Cyrillic_DZHE	1727	0x6BF
Cyrillic_yu	1728	0x6C0
Cyrillic_a	1729	0x6C1
Cyrillic_be	1730	0x6C2
Cyrillic_tse	1731	0x6C3
Cyrillic_de	1732	0x6C4
Cyrillic_ie	1733	0x6C5
Cyrillic_ef	1734	0x6C6
Cyrillic_ghe	1735	0x6C7
Cyrillic_ha	1736	0x6C8
Cyrillic_i	1737	0x6C9
Cyrillic_shorti	1738	0x6CA
Cyrillic_ka	1739	0x6CB
Cyrillic_el	1740	0x6CC
Cyrillic_em	1741	0x6CD
Cyrillic_en	1742	0x6CE
Cyrillic_o	1743	0x6CF
Cyrillic_pe	1744	0x6D0
Cyrillic_ya	1745	0x6D1

```
Cyrillic_er                     1746    0x6D2
Cyrillic_es                     1747    0x6D3
Cyrillic_te                     1748    0x6D4
Cyrillic_u                      1749    0x6D5
Cyrillic_zhe                    1750    0x6D6
Cyrillic_ve                     1751    0x6D7
Cyrillic_softsign               1752    0x6D8
Cyrillic_yeru                   1753    0x6D9
Cyrillic_ze                     1754    0x6DA
Cyrillic_sha                    1755    0x6DB
Cyrillic_e                      1756    0x6DC
Cyrillic_shcha                  1757    0x6DD
Cyrillic_che                    1758    0x6DE
Cyrillic_hardsign               1759    0x6DF
Cyrillic_YU                     1760    0x6E0
Cyrillic_A                      1761    0x6E1
Cyrillic_BE                     1762    0x6E2
Cyrillic_TSE                    1763    0x6E3
Cyrillic_DE                     1764    0x6E4
Cyrillic_IE                     1765    0x6E5
Cyrillic_EF                     1766    0x6E6
Cyrillic_GHE                    1767    0x6E7
Cyrillic_HA                     1768    0x6E8
Cyrillic_I                      1769    0x6E9
Cyrillic_SHORTI                 1770    0x6EA
Cyrillic_KA                     1771    0x6EB
Cyrillic_EL                     1772    0x6EC
Cyrillic_EM                     1773    0x6ED
Cyrillic_EN                     1774    0x6EE
Cyrillic_O                      1775    0x6EF
Cyrillic_PE                     1776    0x6F0
Cyrillic_YA                     1777    0x6F1
Cyrillic_ER                     1778    0x6F2
Cyrillic_ES                     1779    0x6F3
Cyrillic_TE                     1780    0x6F4
Cyrillic_U                      1781    0x6F5
Cyrillic_ZHE                    1782    0x6F6
Cyrillic_VE                     1783    0x6F7
Cyrillic_SOFTSIGN               1784    0x6F8
Cyrillic_YERU                   1785    0x6F9
Cyrillic_ZE                     1786    0x6FA
Cyrillic_SHA                    1787    0x6FB
Cyrillic_E                      1788    0x6FC
Cyrillic_SHCHA                  1789    0x6FD
Cyrillic_CHE                    1790    0x6FE
Cyrillic_HARDSIGN               1791    0x6FF
```

```
Greek_ALPHAaccent                1953     0x7A1
Greek_EPSILONaccent              1954     0x7A2
Greek_ETAaccent                  1955     0x7A3
Greek_IOTAaccent                 1956     0x7A4
Greek_IOTAdieresis               1957     0x7A5
Greek_IOTAaccentdiaeresis        1958     0x7A6
Greek_OMICRONaccent              1959     0x7A7
Greek_UPSILONaccent              1960     0x7A8
Greek_UPSILONdieresis            1961     0x7A9
Greek_UPSILONaccentdieresis      1962     0x7AA
Greek_OMEGAaccent                1963     0x7AB
Greek_accentdieresis             1966     0x7AE
Greek_horizbar                   1967     0x7AF
Greek_alphaaccent                1969     0x7B1
Greek_epsilonaccent              1970     0x7B2
Greek_etaaccent                  1971     0x7B3
Greek_iotaaccent                 1972     0x7B4
Greek_iotadieresis               1973     0x7B5
Greek_iotaaccentdieresis         1974     0x7B6
Greek_omicronaccent              1975     0x7B7
Greek_upsilonaccent              1976     0x7B8
Greek_upsilondieresis            1977     0x7B9
Greek_upsilonaccentdieresis      1978     0x7BA
Greek_omegaaccent                1979     0x7BB
Greek_ALPHA                      1985     0x7C1
Greek_BETA                       1986     0x7C2
Greek_GAMMA                      1987     0x7C3
Greek_DELTA                      1988     0x7C4
Greek_EPSILON                    1989     0x7C5
Greek_ZETA                       1990     0x7C6
Greek_ETA                        1991     0x7C7
Greek_THETA                      1992     0x7C8
Greek_IOTA                       1993     0x7C9
Greek_KAPPA                      1994     0x7CA
Greek_LAMDA                      1995     0x7CB
Greek_MU                         1996     0x7CC
Greek_NU                         1997     0x7CD
Greek_XI                         1998     0x7CE
Greek_OMICRON                    1999     0x7CF
Greek_PI                         2000     0x7D0
Greek_RHO                        2001     0x7D1
Greek_SIGMA                      2002     0x7D2
Greek_TAU                        2004     0x7D4
Greek_UPSILON                    2005     0x7D5
Greek_PHI                        2006     0x7D6
Greek_CHI                        2007     0x7D7
```

```
Greek_PSI                       2008    0x7D8
Greek_OMEGA                     2009    0x7D9
Greek_alpha                     2017    0x7E1
Greek_beta                      2018    0x7E2
Greek_gamma                     2019    0x7E3
Greek_delta                     2020    0x7E4
Greek_epsilon                   2021    0x7E5
Greek_zeta                      2022    0x7E6
Greek_eta                       2023    0x7E7
Greek_theta                     2024    0x7E8
Greek_iota                      2025    0x7E9
Greek_kappa                     2026    0x7EA
Greek_lamda                     2027    0x7EB
Greek_mu                        2028    0x7EC
Greek_nu                        2029    0x7ED
Greek_xi                        2030    0x7EE
Greek_omicron                   2031    0x7EF
Greek_pi                        2032    0x7F0
Greek_rho                       2033    0x7F1
Greek_sigma                     2034    0x7F2
Greek_finalsmallsigma           2035    0x7F3
Greek_tau                       2036    0x7F4
Greek_upsilon                   2037    0x7F5
Greek_phi                       2038    0x7F6
Greek_chi                       2039    0x7F7
Greek_psi                       2040    0x7F8
Greek_omega                     2041    0x7F9
```

```
leftradical                     2209    0x8A1
topleftradical                  2210    0x8A2
horizconnector                  2211    0x8A3
topintegral                     2212    0x8A4
botintegral                     2213    0x8A5
vertconnector                   2214    0x8A6
topleftsqbracket                2215    0x8A7
botleftsqbracket                2216    0x8A8
toprightsqbracket               2217    0x8A9
botrightsqbracket               2218    0x8AA
topleftparens                   2219    0x8AB
botleftparens                   2220    0x8AC
toprightparens                  2221    0x8AD
botrightparens                  2222    0x8AE
leftmiddlecurlybrace            2223    0x8AF
rightmiddlecurlybrace           2224    0x8B0
topleftsummation                2225    0x8B1
botleftsummation                2226    0x8B2
```

```
topvertsummationconnector          2227     0x8B3
botvertsummationconnector          2228     0x8B4
toprightsummation                  2229     0x8B5
botrightsummation                  2230     0x8B6
rightmiddlesummation               2231     0x8B7
lessthanequal                      2236     0x8BC
notequal                           2237     0x8BD
greaterthanequal                   2238     0x8BE
integral                           2239     0x8BF
therefore                          2240     0x8C0
variation                          2241     0x8C1
infinity                           2242     0x8C2
nabla                              2245     0x8C5
approximate                        2248     0x8C8
similarequal                       2249     0x8C9
ifonlyif                           2253     0x8CD
implies                            2254     0x8CE
identical                          2255     0x8CF
radical                            2262     0x8D6
includedin                         2266     0x8DA
includes                           2267     0x8DB
intersection                       2268     0x8DC
union                              2269     0x8DD
logicaland                         2270     0x8DE
logicalor                          2271     0x8DF
partialderivative                  2287     0x8EF
function                           2294     0x8F6
leftarrow                          2299     0x8FB
uparrow                            2300     0x8FC
rightarrow                         2301     0x8FD
downarrow                          2302     0x8FE
blank                              2527     0x9DF
soliddiamond                       2528     0x9E0
checkerboard                       2529     0x9E1
ht                                 2530     0x9E2
ff                                 2531     0x9E3
cr                                 2532     0x9E4
lf                                 2533     0x9E5
nl                                 2536     0x9E8
vt                                 2537     0x9E9
lowrightcorner                     2538     0x9EA
uprightcorner                      2539     0x9EB
upleftcorner                       2540     0x9EC
lowleftcorner                      2541     0x9ED
crossinglines                      2542     0x9EE
horizlinescan1                     2543     0x9EF
```

horizlinescan3	2544	0x9F0
horizlinescan5	2545	0x9F1
horizlinescan7	2546	0x9F2
horizlinescan9	2547	0x9F3
leftt	2548	0x9F4
rightt	2549	0x9F5
bott	2550	0x9F6
topt	2551	0x9F7
vertbar	2552	0x9F8
emspace	2721	0xAA1
enspace	2722	0xAA2
em3space	2723	0xAA3
em4space	2724	0xAA4
digitspace	2725	0xAA5
punctspace	2726	0xAA6
thinspace	2727	0xAA7
hairspace	2728	0xAA8
emdash	2729	0xAA9
endash	2730	0xAAA
signifblank	2732	0xAAC
ellipsis	2734	0xAAE
doubbaselinedot	2735	0xAAF
onethird	2736	0xAB0
twothirds	2737	0xAB1
onefifth	2738	0xAB2
twofifths	2739	0xAB3
threefifths	2740	0xAB4
fourfifths	2741	0xAB5
onesixth	2742	0xAB6
fivesixths	2743	0xAB7
careof	2744	0xAB8
figdash	2747	0xABB
leftanglebracket	2748	0xABC
decimalpoint	2749	0xABD
rightanglebracket	2750	0xABE
marker	2751	0xABF
oneeighth	2755	0xAC3
threeeighths	2756	0xAC4
fiveeighths	2757	0xAC5
seveneighths	2758	0xAC6
trademark	2761	0xAC9
signaturemark	2762	0xACA
trademarkincircle	2763	0xACB
leftopentriangle	2764	0xACC
rightopentriangle	2765	0xACD
emopencircle	2766	0xACE

emopenrectangle	2767	0xACF
leftsinglequotemark	2768	0xAD0
rightsinglequotemark	2769	0xAD1
leftdoublequotemark	2770	0xAD2
rightdoublequotemark	2771	0xAD3
prescription	2772	0xAD4
permille	2773	0xAD5
minutes	2774	0xAD6
seconds	2775	0xAD7
latincross	2777	0xAD9
hexagram	2778	0xADA
filledrectbullet	2779	0xADB
filledlefttribullet	2780	0xADC
filledrighttribullet	2781	0xADD
emfilledcircle	2782	0xADE
emfilledrect	2783	0xADF
enopencircbullet	2784	0xAE0
enopensquarebullet	2785	0xAE1
openrectbullet	2786	0xAE2
opentribulletup	2787	0xAE3
opentribulletdown	2788	0xAE4
openstar	2789	0xAE5
enfilledcircbullet	2790	0xAE6
enfilledsqbullet	2791	0xAE7
filledtribulletup	2792	0xAE8
filledtribulletdown	2793	0xAE9
leftpointer	2794	0xAEA
rightpointer	2795	0xAEB
club	2796	0xAEC
diamond	2797	0xAED
heart	2798	0xAEE
maltesecross	2800	0xAF0
dagger	2801	0xAF1
doubledagger	2802	0xAF2
checkmark	2803	0xAF3
ballotcross	2804	0xAF4
musicalsharp	2805	0xAF5
musicalflat	2806	0xAF6
malesymbol	2807	0xAF7
femalesymbol	2808	0xAF8
telephone	2809	0xAF9
telephonerecorder	2810	0xAFA
phonographcopyright	2811	0xAFB
caret	2812	0xAFC
singlelowquotemark	2813	0xAFD
doublelowquotemark	2814	0xAFE

cursor	2815	0xAFF
leftcaret	2979	0xBA3
rightcaret	2982	0xBA6
downcaret	2984	0xBA8
upcaret	2985	0xBA9
overbar	3008	0xBC0
downtack	3010	0xBC2
upshoe	3011	0xBC3
downstile	3012	0xBC4
underbar	3014	0xBC6
jot	3018	0xBCA
quad	3020	0xBCC
uptack	3022	0xBCE
circle	3023	0xBCF
upstile	3027	0xBD3
downshoe	3030	0xBD6
rightshoe	3032	0xBD8
leftshoe	3034	0xBDA
lefttack	3036	0xBDC
righttack	3068	0xBFC
hebrew_doublelowline	3295	0xCDF
hebrew_aleph	3296	0xCE0
hebrew_bet	3297	0xCE1
hebrew_gimel	3298	0xCE2
hebrew_dalet	3299	0xCE3
hebrew_he	3300	0xCE4
hebrew_waw	3301	0xCE5
hebrew_zain	3302	0xCE6
hebrew_chet	3303	0xCE7
hebrew_tet	3304	0xCE8
hebrew_yod	3305	0xCE9
hebrew_finalkaph	3306	0xCEA
hebrew_kaph	3307	0xCEB
hebrew_lamed	3308	0xCEC
hebrew_finalmem	3309	0xCED
hebrew_mem	3310	0xCEE
hebrew_finalnun	3311	0xCEF
hebrew_nun	3312	0xCF0
hebrew_samech	3313	0xCF1
hebrew_ayin	3314	0xCF2
hebrew_finalpe	3315	0xCF3
hebrew_pe	3316	0xCF4
hebrew_finalzade	3317	0xCF5
hebrew_zade	3318	0xCF6
hebrew_qoph	3319	0xCF7
hebrew_resh	3320	0xCF8

hebrew_shin	3321	0xCF9
hebrew_taw	3322	0xCFA

Thai_kokai	3489	0xDA1
Thai_khokhai	3490	0xDA2
Thai_khokhuat	3491	0xDA3
Thai_khokhwai	3492	0xDA4
Thai_khokhon	3493	0xDA5
Thai_khorakhang	3494	0xDA6
Thai_ngongu	3495	0xDA7
Thai_chochan	3496	0xDA8
Thai_choching	3497	0xDA9
Thai_chochang	3498	0xDAA
Thai_soso	3499	0xDAB
Thai_chochoe	3500	0xDAC
Thai_yoying	3501	0xDAD
Thai_dochada	3502	0xDAE
Thai_topatak	3503	0xDAF
Thai_thothan	3504	0xDB0
Thai_thonangmontho	3505	0xDB1
Thai_thophuthao	3506	0xDB2
Thai_nonen	3507	0xDB3
Thai_dodek	3508	0xDB4
Thai_totao	3509	0xDB5
Thai_thothung	3510	0xDB6
Thai_thothahan	3511	0xDB7
Thai_thothong	3512	0xDB8
Thai_nonu	3513	0xDB9
Thai_bobaimai	3514	0xDBA
Thai_popla	3515	0xDBB
Thai_phophung	3516	0xDBC
Thai_fofa	3517	0xDBD
Thai_phophan	3518	0xDBE
Thai_fofan	3519	0xDBF
Thai_phosamphao	3520	0xDC0
Thai_moma	3521	0xDC1
Thai_yoyak	3522	0xDC2
Thai_rorua	3523	0xDC3
Thai_ru	3524	0xDC4
Thai_loling	3525	0xDC5
Thai_lu	3526	0xDC6
Thai_wowaen	3527	0xDC7
Thai_sosala	3528	0xDC8
Thai_sorusi	3529	0xDC9
Thai_sosua	3530	0xDCA
Thai_hohip	3531	0xDCB

Thai_lochula	3532	0xDCC
Thai_oang	3533	0xDCD
Thai_honokhuk	3534	0xDCE
Thai_paiyannoi	3535	0xDCF
Thai_saraa	3536	0xDD0
Thai_maihanakat	3537	0xDD1
Thai_saraaa	3538	0xDD2
Thai_saraam	3539	0xDD3
Thai_sarai	3540	0xDD4
Thai_saraii	3541	0xDD5
Thai_saraue	3542	0xDD6
Thai_sarauee	3543	0xDD7
Thai_sarau	3544	0xDD8
Thai_sarauu	3545	0xDD9
Thai_phinthu	3546	0xDDA
Thai_maihanakat_maitho	3550	0xDDE
Thai_baht	3551	0xDDF
Thai_sarae	3552	0xDE0
Thai_saraae	3553	0xDE1
Thai_sarao	3554	0xDE2
Thai_saraaimaimuan	3555	0xDE3
Thai_saraaimaimalai	3556	0xDE4
Thai_lakkhangyao	3557	0xDE5
Thai_maiyamok	3558	0xDE6
Thai_maitaikhu	3559	0xDE7
Thai_maiek	3560	0xDE8
Thai_maitho	3561	0xDE9
Thai_maitri	3562	0xDEA
Thai_maichattawa	3563	0xDEB
Thai_thanthakhat	3564	0xDEC
Thai_nikhahit	3565	0xDED
Thai_leksun	3568	0xDF0
Thai_leknung	3569	0xDF1
Thai_leksong	3570	0xDF2
Thai_leksam	3571	0xDF3
Thai_leksi	3572	0xDF4
Thai_lekha	3573	0xDF5
Thai_lekhok	3574	0xDF6
Thai_lekchet	3575	0xDF7
Thai_lekpaet	3576	0xDF8
Thai_lekkao	3577	0xDF9
Hangul_Kiyeog	3745	0xEA1
Hangul_SsangKiyeog	3746	0xEA2
Hangul_KiyeogSios	3747	0xEA3
Hangul_Nieun	3748	0xEA4
Hangul_NieunJieuj	3749	0xEA5

```
Hangul_NieunHieuh               3750    0xEA6
Hangul_Dikeud                   3751    0xEA7
Hangul_SsangDikeud              3752    0xEA8
Hangul_Rieul                    3753    0xEA9
Hangul_RieulKiyeog              3754    0xEAA
Hangul_RieulMieum               3755    0xEAB
Hangul_RieulPieub               3756    0xEAC
Hangul_RieulSios                3757    0xEAD
Hangul_RieulTieut               3758    0xEAE
Hangul_RieulPhieuf              3759    0xEAF
Hangul_RieulHieuh               3760    0xEB0
Hangul_Mieum                    3761    0xEB1
Hangul_Pieub                    3762    0xEB2
Hangul_SsangPieub               3763    0xEB3
Hangul_PieubSios                3764    0xEB4
Hangul_Sios                     3765    0xEB5
Hangul_SsangSios                3766    0xEB6
Hangul_Ieung                    3767    0xEB7
Hangul_Jieuj                    3768    0xEB8
Hangul_SsangJieuj               3769    0xEB9
Hangul_Cieuc                    3770    0xEBA
Hangul_Khieuq                   3771    0xEBB
Hangul_Tieut                    3772    0xEBC
Hangul_Phieuf                   3773    0xEBD
Hangul_Hieuh                    3774    0xEBE
Hangul_A                        3775    0xEBF
Hangul_AE                       3776    0xEC0
Hangul_YA                       3777    0xEC1
Hangul_YAE                      3778    0xEC2
Hangul_EO                       3779    0xEC3
Hangul_E                        3780    0xEC4
Hangul_YEO                      3781    0xEC5
Hangul_YE                       3782    0xEC6
Hangul_O                        3783    0xEC7
Hangul_WA                       3784    0xEC8
Hangul_WAE                      3785    0xEC9
Hangul_OE                       3786    0xECA
Hangul_YO                       3787    0xECB
Hangul_U                        3788    0xECC
Hangul_WEO                      3789    0xECD
Hangul_WE                       3790    0xECE
Hangul_WI                       3791    0xECF
Hangul_YU                       3792    0xED0
Hangul_EU                       3793    0xED1
Hangul_YI                       3794    0xED2
Hangul_I                        3795    0xED3
```

```
Hangul_J_Kiyeog                 3796    0xED4
Hangul_J_SsangKiyeog            3797    0xED5
Hangul_J_KiyeogSios             3798    0xED6
Hangul_J_Nieun                  3799    0xED7
Hangul_J_NieunJieuj             3800    0xED8
Hangul_J_NieunHieuh             3801    0xED9
Hangul_J_Dikeud                 3802    0xEDA
Hangul_J_Rieul                  3803    0xEDB
Hangul_J_RieulKiyeog            3804    0xEDC
Hangul_J_RieulMieum             3805    0xEDD
Hangul_J_RieulPieub             3806    0xEDE
Hangul_J_RieulSios              3807    0xEDF
Hangul_J_RieulTieut             3808    0xEE0
Hangul_J_RieulPhieuf            3809    0xEE1
Hangul_J_RieulHieuh             3810    0xEE2
Hangul_J_Mieum                  3811    0xEE3
Hangul_J_Pieub                  3812    0xEE4
Hangul_J_PieubSios              3813    0xEE5
Hangul_J_Sios                   3814    0xEE6
Hangul_J_SsangSios              3815    0xEE7
Hangul_J_Ieung                  3816    0xEE8
Hangul_J_Jieuj                  3817    0xEE9
Hangul_J_Cieuc                  3818    0xEEA
Hangul_J_Khieuq                 3819    0xEEB
Hangul_J_Tieut                  3820    0xEEC
Hangul_J_Phieuf                 3821    0xEED
Hangul_J_Hieuh                  3822    0xEEE
Hangul_RieulYeorinHieuh         3823    0xEEF
Hangul_SunkyeongeumMieum        3824    0xEF0
Hangul_SunkyeongeumPieub        3825    0xEF1
Hangul_PanSios                  3826    0xEF2
Hangul_KkogjiDalrinIeung        3827    0xEF3
Hangul_SunkyeongeumPhieuf       3828    0xEF4
Hangul_YeorinHieuh              3829    0xEF5
Hangul_AraeA                    3830    0xEF6
Hangul_AraeAE                   3831    0xEF7
Hangul_J_PanSios                3832    0xEF8
Hangul_J_KkogjiDalrinIeung      3833    0xEF9
Hangul_J_YeorinHieuh            3834    0xEFA
Korean_Won                      3839    0xEFF
OE                              5052    0x13BC
oe                              5053    0x13BD
Ydiaeresis                      5054    0x13BE
```

```
ISO_Lock                        65025     0xFE01
ISO_Level2_Latch                65026     0xFE02
ISO_Level3_Shift                65027     0xFE03
ISO_Level3_Latch                65028     0xFE04
ISO_Level3_Lock                 65029     0xFE05
ISO_Group_Latch                 65030     0xFE06
ISO_Group_Lock                  65031     0xFE07
ISO_Next_Group                  65032     0xFE08
ISO_Next_Group_Lock             65033     0xFE09
ISO_Prev_Group                  65034     0xFE0A
ISO_Prev_Group_Lock             65035     0xFE0B
ISO_First_Group                 65036     0xFE0C
ISO_First_Group_Lock            65037     0xFE0D
ISO_Last_Group                  65038     0xFE0E
ISO_Last_Group_Lock             65039     0xFE0F
ISO_Level5_Shift                65041     0xFE11
ISO_Level5_Latch                65042     0xFE12
ISO_Level5_Lock                 65043     0xFE13
ISO_Left_Tab                    65056     0xFE20
ISO_Move_Line_Up                65057     0xFE21
ISO_Move_Line_Down              65058     0xFE22
ISO_Partial_Line_Up             65059     0xFE23
ISO_Partial_Line_Down           65060     0xFE24
ISO_Partial_Space_Left          65061     0xFE25
ISO_Partial_Space_Right         65062     0xFE26
ISO_Set_Margin_Left             65063     0xFE27
ISO_Set_Margin_Right            65064     0xFE28
ISO_Release_Margin_Left         65065     0xFE29
ISO_Release_Margin_Right        65066     0xFE2A
ISO_Release_Both_Margins        65067     0xFE2B
ISO_Fast_Cursor_Left            65068     0xFE2C
ISO_Fast_Cursor_Right           65069     0xFE2D
ISO_Fast_Cursor_Up              65070     0xFE2E
ISO_Fast_Cursor_Down            65071     0xFE2F
ISO_Continuous_Underline        65072     0xFE30
ISO_Discontinuous_Underline     65073     0xFE31
ISO_Emphasize                   65074     0xFE32
ISO_Center_Object               65075     0xFE33
ISO_Enter                       65076     0xFE34
dead_grave                      65104     0xFE50
dead_acute                      65105     0xFE51
dead_circumflex                 65106     0xFE52
dead_tilde                      65107     0xFE53
dead_macron                     65108     0xFE54
dead_breve                      65109     0xFE55
dead_abovedot                   65110     0xFE56
```

```
dead_diaeresis                  65111     0xFE57
dead_abovering                  65112     0xFE58
dead_doubleacute                65113     0xFE59
dead_caron                      65114     0xFE5A
dead_cedilla                    65115     0xFE5B
dead_ogonek                     65116     0xFE5C
dead_iota                       65117     0xFE5D
dead_voiced_sound               65118     0xFE5E
dead_semivoiced_sound           65119     0xFE5F
dead_belowdot                   65120     0xFE60
dead_hook                       65121     0xFE61
dead_horn                       65122     0xFE62
dead_stroke                     65123     0xFE63
dead_abovecomma                 65124     0xFE64
dead_abovereversedcomma         65125     0xFE65
dead_doublegrave                65126     0xFE66
dead_belowring                  65127     0xFE67
dead_belowmacron                65128     0xFE68
dead_belowcircumflex            65129     0xFE69
dead_belowtilde                 65130     0xFE6A
dead_belowbreve                 65131     0xFE6B
dead_belowdiaeresis             65132     0xFE6C
dead_invertedbreve              65133     0xFE6D
dead_belowcomma                 65134     0xFE6E
dead_currency                   65135     0xFE6F
AccessX_Enable                  65136     0xFE70
AccessX_Feedback_Enable         65137     0xFE71
RepeatKeys_Enable               65138     0xFE72
SlowKeys_Enable                 65139     0xFE73
BounceKeys_Enable               65140     0xFE74
StickyKeys_Enable               65141     0xFE75
MouseKeys_Enable                65142     0xFE76
MouseKeys_Accel_Enable          65143     0xFE77
Overlay1_Enable                 65144     0xFE78
Overlay2_Enable                 65145     0xFE79
AudibleBell_Enable              65146     0xFE7A
dead_a                          65152     0xFE80
dead_A                          65153     0xFE81
dead_e                          65154     0xFE82
dead_E                          65155     0xFE83
dead_i                          65156     0xFE84
dead_I                          65157     0xFE85
dead_o                          65158     0xFE86
dead_O                          65159     0xFE87
dead_u                          65160     0xFE88
dead_U                          65161     0xFE89
```

```
dead_schwa                      65162     0xFE8A
dead_SCHWA                      65163     0xFE8B
dead_greek                      65164     0xFE8C
dead_hamza                      65165     0xFE8D
dead_lowline                    65168     0xFE90
dead_aboveverticalline          65169     0xFE91
dead_belowverticalline          65170     0xFE92
dead_longsolidusoverlay         65171     0xFE93
ch                              65184     0xFEA0
Ch                              65185     0xFEA1
CH                              65186     0xFEA2
c_h                             65187     0xFEA3
C_h                             65188     0xFEA4
C_H                             65189     0xFEA5
First_Virtual_Screen            65232     0xFED0
Prev_Virtual_Screen             65233     0xFED1
Next_Virtual_Screen             65234     0xFED2
Last_Virtual_Screen             65236     0xFED4
Terminate_Server                65237     0xFED5
Pointer_Left                    65248     0xFEE0
Pointer_Right                   65249     0xFEE1
Pointer_Up                      65250     0xFEE2
Pointer_Down                    65251     0xFEE3
Pointer_UpLeft                  65252     0xFEE4
Pointer_UpRight                 65253     0xFEE5
Pointer_DownLeft                65254     0xFEE6
Pointer_DownRight               65255     0xFEE7
Pointer_Button_Dflt             65256     0xFEE8
Pointer_Button1                 65257     0xFEE9
Pointer_Button2                 65258     0xFEEA
Pointer_Button3                 65259     0xFEEB
Pointer_Button4                 65260     0xFEEC
Pointer_Button5                 65261     0xFEED
Pointer_DblClick_Dflt           65262     0xFEEE
Pointer_DblClick1               65263     0xFEEF
Pointer_DblClick2               65264     0xFEF0
Pointer_DblClick3               65265     0xFEF1
Pointer_DblClick4               65266     0xFEF2
Pointer_DblClick5               65267     0xFEF3
Pointer_Drag_Dflt               65268     0xFEF4
Pointer_Drag1                   65269     0xFEF5
Pointer_Drag2                   65270     0xFEF6
Pointer_Drag3                   65271     0xFEF7
Pointer_Drag4                   65272     0xFEF8
Pointer_EnableKeys              65273     0xFEF9
Pointer_Accelerate              65274     0xFEFA
```

Pointer_DfltBtnNext	65275	0xFEFB
Pointer_DfltBtnPrev	65276	0xFEFC
Pointer_Drag5	65277	0xFEFD

BackSpace	65288	0xFF08
Tab	65289	0xFF09
Linefeed	65290	0xFF0A
Clear	65291	0xFF0B
Return	65293	0xFF0D
Pause	65299	0xFF13
Scroll_Lock	65300	0xFF14
Sys_Req	65301	0xFF15
Escape	65307	0xFF1B
Multi_key	65312	0xFF20
Kanji	65313	0xFF21
Muhenkan	65314	0xFF22
Henkan_Mode	65315	0xFF23
Romaji	65316	0xFF24
Hiragana	65317	0xFF25
Katakana	65318	0xFF26
Hiragana_Katakana	65319	0xFF27
Zenkaku	65320	0xFF28
Hankaku	65321	0xFF29
Zenkaku_Hankaku	65322	0xFF2A
Touroku	65323	0xFF2B
Massyo	65324	0xFF2C
Kana_Lock	65325	0xFF2D
Kana_Shift	65326	0xFF2E
Eisu_Shift	65327	0xFF2F
Eisu_toggle	65328	0xFF30
Hangul	65329	0xFF31
Hangul_Start	65330	0xFF32
Hangul_End	65331	0xFF33
Hangul_Hanja	65332	0xFF34
Hangul_Jamo	65333	0xFF35
Hangul_Romaja	65334	0xFF36
Codeinput	65335	0xFF37
Hangul_Jeonja	65336	0xFF38
Hangul_Banja	65337	0xFF39
Hangul_PreHanja	65338	0xFF3A
Hangul_PostHanja	65339	0xFF3B
SingleCandidate	65340	0xFF3C
MultipleCandidate	65341	0xFF3D
PreviousCandidate	65342	0xFF3E
Hangul_Special	65343	0xFF3F
Home	65360	0xFF50

```
Left                    65361     0xFF51
Up                      65362     0xFF52
Right                   65363     0xFF53
Down                    65364     0xFF54
Prior                   65365     0xFF55
Next                    65366     0xFF56
End                     65367     0xFF57
Begin                   65368     0xFF58
Win_L                   65371     0xFF5B
Win_R                   65372     0xFF5C
App                     65373     0xFF5D
Select                  65376     0xFF60
Print                   65377     0xFF61
Execute                 65378     0xFF62
Insert                  65379     0xFF63
Undo                    65381     0xFF65
Redo                    65382     0xFF66
Menu                    65383     0xFF67
Find                    65384     0xFF68
Cancel                  65385     0xFF69
Help                    65386     0xFF6A
Break                   65387     0xFF6B
Mode_switch             65406     0xFF7E
Num_Lock                65407     0xFF7F
KP_Space                65408     0xFF80
KP_Tab                  65417     0xFF89
KP_Enter                65421     0xFF8D
KP_F1                   65425     0xFF91
KP_F2                   65426     0xFF92
KP_F3                   65427     0xFF93
KP_F4                   65428     0xFF94
KP_Home                 65429     0xFF95
KP_Left                 65430     0xFF96
KP_Up                   65431     0xFF97
KP_Right                65432     0xFF98
KP_Down                 65433     0xFF99
KP_Prior                65434     0xFF9A
KP_Next                 65435     0xFF9B
KP_End                  65436     0xFF9C
KP_Begin                65437     0xFF9D
KP_Insert               65438     0xFF9E
KP_Delete               65439     0xFF9F
KP_Multiply             65450     0xFFAA
KP_Add                  65451     0xFFAB
KP_Separator            65452     0xFFAC
KP_Subtract             65453     0xFFAD
```

```
KP_Decimal                      65454     0xFFAE
KP_Divide                       65455     0xFFAF
KP_0                            65456     0xFFB0
KP_1                            65457     0xFFB1
KP_2                            65458     0xFFB2
KP_3                            65459     0xFFB3
KP_4                            65460     0xFFB4
KP_5                            65461     0xFFB5
KP_6                            65462     0xFFB6
KP_7                            65463     0xFFB7
KP_8                            65464     0xFFB8
KP_9                            65465     0xFFB9
KP_Equal                        65469     0xFFBD
F1                              65470     0xFFBE
F2                              65471     0xFFBF
F3                              65472     0xFFC0
F4                              65473     0xFFC1
F5                              65474     0xFFC2
F6                              65475     0xFFC3
F7                              65476     0xFFC4
F8                              65477     0xFFC5
F9                              65478     0xFFC6
F10                             65479     0xFFC7
F11                             65480     0xFFC8
F12                             65481     0xFFC9
F13                             65482     0xFFCA
F14                             65483     0xFFCB
F15                             65484     0xFFCC
F16                             65485     0xFFCD
F17                             65486     0xFFCE
F18                             65487     0xFFCF
F19                             65488     0xFFD0
F20                             65489     0xFFD1
F21                             65490     0xFFD2
F22                             65491     0xFFD3
F23                             65492     0xFFD4
F24                             65493     0xFFD5
F25                             65494     0xFFD6
F26                             65495     0xFFD7
F27                             65496     0xFFD8
F28                             65497     0xFFD9
F29                             65498     0xFFDA
F30                             65499     0xFFDB
F31                             65500     0xFFDC
F32                             65501     0xFFDD
F33                             65502     0xFFDE
```

F34	65503	0xFFDF
F35	65504	0xFFE0
Shift_L	65505	0xFFE1
Shift_R	65506	0xFFE2
Control_L	65507	0xFFE3
Control_R	65508	0xFFE4
Caps_Lock	65509	0xFFE5
Shift_Lock	65510	0xFFE6
Meta_L	65511	0xFFE7
Meta_R	65512	0xFFE8
Alt_L	65513	0xFFE9
Alt_R	65514	0xFFEA
Super_L	65515	0xFFEB
Super_R	65516	0xFFEC
Hyper_L	65517	0xFFED
Hyper_R	65518	0xFFEE
braille_dot_1	65521	0xFFF1
braille_dot_2	65522	0xFFF2
braille_dot_3	65523	0xFFF3
braille_dot_4	65524	0xFFF4
braille_dot_5	65525	0xFFF5
braille_dot_6	65526	0xFFF6
braille_dot_7	65527	0xFFF7
braille_dot_8	65528	0xFFF8
braille_dot_9	65529	0xFFF9
braille_dot_10	65530	0xFFFA
Delete	65535	0xFFFF

SunFA_Grave	268828416	0x1005FF00
SunFA_Circum	268828417	0x1005FF01
SunFA_Tilde	268828418	0x1005FF02
SunFA_Acute	268828419	0x1005FF03
SunFA_Diaeresis	268828420	0x1005FF04
SunFA_Cedilla	268828421	0x1005FF05
SunF36	268828432	0x1005FF10
SunF37	268828433	0x1005FF11
SunSys_Req	268828512	0x1005FF60
SunProps	268828528	0x1005FF70
SunFront	268828529	0x1005FF71
SunCopy	268828530	0x1005FF72
SunOpen	268828531	0x1005FF73
SunPaste	268828532	0x1005FF74
SunCut	268828533	0x1005FF75
SunPowerSwitch	268828534	0x1005FF76
SunAudioLowerVolume	268828535	0x1005FF77
SunAudioMute	268828536	0x1005FF78

```
SunAudioRaiseVolume          268828537     0x1005FF79
SunVideoDegauss              268828538     0x1005FF7A
SunVideoLowerBrightness      268828539     0x1005FF7B
SunVideoRaiseBrightness      268828540     0x1005FF7C
SunPowerSwitchShift          268828541     0x1005FF7D
XF86BrightnessAuto           268964084     0x100810F4
XF86DisplayOff               268964085     0x100810F5
XF86Info                     268964198     0x10081166
XF86AspectRatio              268964215     0x10081177
XF86DVD                      268964229     0x10081185
XF86Audio                    268964232     0x10081188
XF86ChannelUp                268964242     0x10081192
XF86ChannelDown              268964243     0x10081193
XF86Break                    268964251     0x1008119B
XF86VideoPhone               268964256     0x100811A0
XF86ZoomReset                268964260     0x100811A4
XF86Editor                   268964262     0x100811A6
XF86GraphicsEditor           268964264     0x100811A8
XF86Presentation             268964265     0x100811A9
XF86Database                 268964266     0x100811AA
XF86Voicemail                268964268     0x100811AC
XF86Addressbook              268964269     0x100811AD
XF86DisplayToggle            268964271     0x100811AF
XF86SpellCheck               268964272     0x100811B0
XF86ContextMenu              268964278     0x100811B6
XF86MediaRepeat              268964279     0x100811B7
XF8610ChannelsUp             268964280     0x100811B8
XF8610ChannelsDown           268964281     0x100811B9
XF86Images                   268964282     0x100811BA
XF86NotificationCenter       268964284     0x100811BC
XF86PickupPhone              268964285     0x100811BD
XF86HangupPhone              268964286     0x100811BE
XF86Fn                       268964304     0x100811D0
XF86Fn_Esc                   268964305     0x100811D1
XF86FnRightShift             268964325     0x100811E5
XF86Numeric0                 268964352     0x10081200
XF86Numeric1                 268964353     0x10081201
XF86Numeric2                 268964354     0x10081202
XF86Numeric3                 268964355     0x10081203
XF86Numeric4                 268964356     0x10081204
XF86Numeric5                 268964357     0x10081205
XF86Numeric6                 268964358     0x10081206
XF86Numeric7                 268964359     0x10081207
XF86Numeric8                 268964360     0x10081208
XF86Numeric9                 268964361     0x10081209
XF86NumericStar              268964362     0x1008120A
```

```
XF86NumericPound              268964363     0x1008120B
XF86NumericA                  268964364     0x1008120C
XF86NumericB                  268964365     0x1008120D
XF86NumericC                  268964366     0x1008120E
XF86NumericD                  268964367     0x1008120F
XF86CameraFocus               268964368     0x10081210
XF86WPSButton                 268964369     0x10081211
XF86CameraZoomIn              268964373     0x10081215
XF86CameraZoomOut             268964374     0x10081216
XF86CameraUp                  268964375     0x10081217
XF86CameraDown                268964376     0x10081218
XF86CameraLeft                268964377     0x10081219
XF86CameraRight               268964378     0x1008121A
XF86AttendantOn               268964379     0x1008121B
XF86AttendantOff              268964380     0x1008121C
XF86AttendantToggle           268964381     0x1008121D
XF86LightsToggle              268964382     0x1008121E
XF86ALSToggle                 268964400     0x10081230
XF86Buttonconfig              268964416     0x10081240
XF86Taskmanager               268964417     0x10081241
XF86Journal                   268964418     0x10081242
XF86ControlPanel              268964419     0x10081243
XF86AppSelect                 268964420     0x10081244
XF86Screensaver               268964421     0x10081245
XF86VoiceCommand              268964422     0x10081246
XF86Assistant                 268964423     0x10081247
XF86EmojiPicker               268964425     0x10081249
XF86Dictate                   268964426     0x1008124A
XF86BrightnessMin             268964432     0x10081250
XF86BrightnessMax             268964433     0x10081251
XF86KbdInputAssistPrev        268964448     0x10081260
XF86KbdInputAssistNext        268964449     0x10081261
XF86KbdInputAssistPrevgroup   268964450     0x10081262
XF86KbdInputAssistNextgroup   268964451     0x10081263
XF86KbdInputAssistAccept      268964452     0x10081264
XF86KbdInputAssistCancel      268964453     0x10081265
XF86RightUp                   268964454     0x10081266
XF86RightDown                 268964455     0x10081267
XF86LeftUp                    268964456     0x10081268
XF86LeftDown                  268964457     0x10081269
XF86RootMenu                  268964458     0x1008126A
XF86MediaTopMenu              268964459     0x1008126B
XF86Numeric11                 268964460     0x1008126C
XF86Numeric12                 268964461     0x1008126D
XF86AudioDesc                 268964462     0x1008126E
XF863DMode                    268964463     0x1008126F
```

```
XF86NextFavorite            268964464     0x10081270
XF86StopRecord              268964465     0x10081271
XF86PauseRecord             268964466     0x10081272
XF86VOD                     268964467     0x10081273
XF86Unmute                  268964468     0x10081274
XF86FastReverse             268964469     0x10081275
XF86SlowReverse             268964470     0x10081276
XF86Data                    268964471     0x10081277
XF86OnScreenKeyboard        268964472     0x10081278
XF86PrivacyScreenToggle     268964473     0x10081279
XF86SelectiveScreenshot     268964474     0x1008127A
XF86Macro1                  268964496     0x10081290
XF86Macro2                  268964497     0x10081291
XF86Macro3                  268964498     0x10081292
XF86Macro4                  268964499     0x10081293
XF86Macro5                  268964500     0x10081294
XF86Macro6                  268964501     0x10081295
XF86Macro7                  268964502     0x10081296
XF86Macro8                  268964503     0x10081297
XF86Macro9                  268964504     0x10081298
XF86Macro10                 268964505     0x10081299
XF86Macro11                 268964506     0x1008129A
XF86Macro12                 268964507     0x1008129B
XF86Macro13                 268964508     0x1008129C
XF86Macro14                 268964509     0x1008129D
XF86Macro15                 268964510     0x1008129E
XF86Macro16                 268964511     0x1008129F
XF86Macro17                 268964512     0x100812A0
XF86Macro18                 268964513     0x100812A1
XF86Macro19                 268964514     0x100812A2
XF86Macro20                 268964515     0x100812A3
XF86Macro21                 268964516     0x100812A4
XF86Macro22                 268964517     0x100812A5
XF86Macro23                 268964518     0x100812A6
XF86Macro24                 268964519     0x100812A7
XF86Macro25                 268964520     0x100812A8
XF86Macro26                 268964521     0x100812A9
XF86Macro27                 268964522     0x100812AA
XF86Macro28                 268964523     0x100812AB
XF86Macro29                 268964524     0x100812AC
XF86Macro30                 268964525     0x100812AD
XF86MacroRecordStart        268964528     0x100812B0
XF86MacroRecordStop         268964529     0x100812B1
XF86MacroPresetCycle        268964530     0x100812B2
XF86MacroPreset1            268964531     0x100812B3
XF86MacroPreset2            268964532     0x100812B4
```

```
XF86MacroPreset3            268964533     0x100812B5
XF86KbdLcdMenu1             268964536     0x100812B8
XF86KbdLcdMenu2             268964537     0x100812B9
XF86KbdLcdMenu3             268964538     0x100812BA
XF86KbdLcdMenu4             268964539     0x100812BB
XF86KbdLcdMenu5             268964540     0x100812BC
XF86Switch_VT_1             269024769     0x1008FE01
XF86Switch_VT_2             269024770     0x1008FE02
XF86Switch_VT_3             269024771     0x1008FE03
XF86Switch_VT_4             269024772     0x1008FE04
XF86Switch_VT_5             269024773     0x1008FE05
XF86Switch_VT_6             269024774     0x1008FE06
XF86Switch_VT_7             269024775     0x1008FE07
XF86Switch_VT_8             269024776     0x1008FE08
XF86Switch_VT_9             269024777     0x1008FE09
XF86Switch_VT_10            269024778     0x1008FE0A
XF86Switch_VT_11            269024779     0x1008FE0B
XF86Switch_VT_12            269024780     0x1008FE0C
```

```
XF86Ungrab                  269024800     0x1008FE20
XF86ClearGrab               269024801     0x1008FE21
XF86Next_VMode              269024802     0x1008FE22
XF86Prev_VMode              269024803     0x1008FE23
XF86LogWindowTree           269024804     0x1008FE24
XF86LogGrabInfo             269024805     0x1008FE25
XF86ModeLock                269025025     0x1008FF01
XF86MonBrightnessUp         269025026     0x1008FF02
XF86MonBrightnessDown       269025027     0x1008FF03
XF86KbdLightOnOff           269025028     0x1008FF04
XF86KbdBrightnessUp         269025029     0x1008FF05
XF86KbdBrightnessDown       269025030     0x1008FF06
XF86MonBrightnessCycle      269025031     0x1008FF07
XF86Standby                 269025040     0x1008FF10
XF86AudioLowerVolume        269025041     0x1008FF11
XF86AudioMute               269025042     0x1008FF12
XF86AudioRaiseVolume        269025043     0x1008FF13
XF86AudioPlay               269025044     0x1008FF14
XF86AudioStop               269025045     0x1008FF15
XF86AudioPrev               269025046     0x1008FF16
XF86AudioNext               269025047     0x1008FF17
XF86HomePage                269025048     0x1008FF18
XF86Mail                    269025049     0x1008FF19
XF86Start                   269025050     0x1008FF1A
XF86Search                  269025051     0x1008FF1B
XF86AudioRecord             269025052     0x1008FF1C
XF86Calculator              269025053     0x1008FF1D
```

```
XF86Memo                269025054     0x1008FF1E
XF86ToDoList            269025055     0x1008FF1F
XF86Calendar            269025056     0x1008FF20
XF86PowerDown           269025057     0x1008FF21
XF86ContrastAdjust      269025058     0x1008FF22
XF86RockerUp            269025059     0x1008FF23
XF86RockerDown          269025060     0x1008FF24
XF86RockerEnter         269025061     0x1008FF25
XF86Back                269025062     0x1008FF26
XF86Forward             269025063     0x1008FF27
XF86Stop                269025064     0x1008FF28
XF86Refresh             269025065     0x1008FF29
XF86PowerOff            269025066     0x1008FF2A
XF86WakeUp              269025067     0x1008FF2B
XF86Eject               269025068     0x1008FF2C
XF86ScreenSaver         269025069     0x1008FF2D
XF86WWW                 269025070     0x1008FF2E
XF86Sleep               269025071     0x1008FF2F
XF86Favorites           269025072     0x1008FF30
XF86AudioPause          269025073     0x1008FF31
XF86AudioMedia          269025074     0x1008FF32
XF86MyComputer          269025075     0x1008FF33
XF86VendorHome          269025076     0x1008FF34
XF86LightBulb           269025077     0x1008FF35
XF86Shop                269025078     0x1008FF36
XF86History             269025079     0x1008FF37
XF86OpenURL             269025080     0x1008FF38
XF86AddFavorite         269025081     0x1008FF39
XF86HotLinks            269025082     0x1008FF3A
XF86BrightnessAdjust    269025083     0x1008FF3B
XF86Finance             269025084     0x1008FF3C
XF86Community           269025085     0x1008FF3D
XF86AudioRewind         269025086     0x1008FF3E
XF86BackForward         269025087     0x1008FF3F
XF86Launch0             269025088     0x1008FF40
XF86Launch1             269025089     0x1008FF41
XF86Launch2             269025090     0x1008FF42
XF86Launch3             269025091     0x1008FF43
XF86Launch4             269025092     0x1008FF44
XF86Launch5             269025093     0x1008FF45
XF86Launch6             269025094     0x1008FF46
XF86Launch7             269025095     0x1008FF47
XF86Launch8             269025096     0x1008FF48
XF86Launch9             269025097     0x1008FF49
XF86LaunchA             269025098     0x1008FF4A
XF86LaunchB             269025099     0x1008FF4B
```

```
XF86LaunchC                 269025100     0x1008FF4C
XF86LaunchD                 269025101     0x1008FF4D
XF86LaunchE                 269025102     0x1008FF4E
XF86LaunchF                 269025103     0x1008FF4F
XF86ApplicationLeft         269025104     0x1008FF50
XF86ApplicationRight        269025105     0x1008FF51
XF86Book                    269025106     0x1008FF52
XF86CD                      269025107     0x1008FF53
XF86Calculater              269025108     0x1008FF54
XF86Clear                   269025109     0x1008FF55
XF86Close                   269025110     0x1008FF56
XF86Copy                    269025111     0x1008FF57
XF86Cut                     269025112     0x1008FF58
XF86Display                 269025113     0x1008FF59
XF86DOS                     269025114     0x1008FF5A
XF86Documents               269025115     0x1008FF5B
XF86Excel                   269025116     0x1008FF5C
XF86Explorer                269025117     0x1008FF5D
XF86Game                    269025118     0x1008FF5E
XF86Go                      269025119     0x1008FF5F
XF86iTouch                  269025120     0x1008FF60
XF86LogOff                  269025121     0x1008FF61
XF86Market                  269025122     0x1008FF62
XF86Meeting                 269025123     0x1008FF63
XF86MenuKB                  269025125     0x1008FF65
XF86MenuPB                  269025126     0x1008FF66
XF86MySites                 269025127     0x1008FF67
XF86New                     269025128     0x1008FF68
XF86News                    269025129     0x1008FF69
XF86OfficeHome              269025130     0x1008FF6A
XF86Open                    269025131     0x1008FF6B
XF86Option                  269025132     0x1008FF6C
XF86Paste                   269025133     0x1008FF6D
XF86Phone                   269025134     0x1008FF6E
XF86Q                       269025136     0x1008FF70
XF86Reply                   269025138     0x1008FF72
XF86Reload                  269025139     0x1008FF73
XF86RotateWindows           269025140     0x1008FF74
XF86RotationPB              269025141     0x1008FF75
XF86RotationKB              269025142     0x1008FF76
XF86Save                    269025143     0x1008FF77
XF86ScrollUp                269025144     0x1008FF78
XF86ScrollDown              269025145     0x1008FF79
XF86ScrollClick             269025146     0x1008FF7A
XF86Send                    269025147     0x1008FF7B
XF86Spell                   269025148     0x1008FF7C
```

```
XF86SplitScreen             269025149     0x1008FF7D
XF86Support                 269025150     0x1008FF7E
XF86TaskPane                269025151     0x1008FF7F
XF86Terminal                269025152     0x1008FF80
XF86Tools                   269025153     0x1008FF81
XF86Travel                  269025154     0x1008FF82
XF86UserPB                  269025156     0x1008FF84
XF86User1KB                 269025157     0x1008FF85
XF86User2KB                 269025158     0x1008FF86
XF86Video                   269025159     0x1008FF87
XF86WheelButton             269025160     0x1008FF88
XF86Word                    269025161     0x1008FF89
XF86Xfer                    269025162     0x1008FF8A
XF86ZoomIn                  269025163     0x1008FF8B
XF86ZoomOut                 269025164     0x1008FF8C
XF86Away                    269025165     0x1008FF8D
XF86Messenger               269025166     0x1008FF8E
XF86WebCam                  269025167     0x1008FF8F
XF86MailForward             269025168     0x1008FF90
XF86Pictures                269025169     0x1008FF91
XF86Music                   269025170     0x1008FF92
XF86Battery                 269025171     0x1008FF93
XF86Bluetooth               269025172     0x1008FF94
XF86WLAN                    269025173     0x1008FF95
XF86UWB                     269025174     0x1008FF96
XF86AudioForward            269025175     0x1008FF97
XF86AudioRepeat             269025176     0x1008FF98
XF86AudioRandomPlay         269025177     0x1008FF99
XF86Subtitle                269025178     0x1008FF9A
XF86AudioCycleTrack         269025179     0x1008FF9B
XF86CycleAngle              269025180     0x1008FF9C
XF86FrameBack               269025181     0x1008FF9D
XF86FrameForward            269025182     0x1008FF9E
XF86Time                    269025183     0x1008FF9F
XF86Select                  269025184     0x1008FFA0
XF86View                    269025185     0x1008FFA1
XF86TopMenu                 269025186     0x1008FFA2
XF86Red                     269025187     0x1008FFA3
XF86Green                   269025188     0x1008FFA4
XF86Yellow                  269025189     0x1008FFA5
XF86Blue                    269025190     0x1008FFA6
XF86Suspend                 269025191     0x1008FFA7
XF86Hibernate               269025192     0x1008FFA8
XF86TouchpadToggle          269025193     0x1008FFA9
XF86TouchpadOn              269025200     0x1008FFB0
XF86TouchpadOff             269025201     0x1008FFB1
```

```
XF86AudioMicMute            269025202     0x1008FFB2
XF86Keyboard                269025203     0x1008FFB3
XF86WWAN                    269025204     0x1008FFB4
XF86RFKill                  269025205     0x1008FFB5
XF86AudioPreset             269025206     0x1008FFB6
XF86RotationLockToggle      269025207     0x1008FFB7
XF86FullScreen              269025208     0x1008FFB8
```

Copyright (c) 1998-2000 Scriptics Corporation.

label

Keywords: label, widget

Name

label - Create and manipulate 'label' non-interactive text or image widgets

Synopsis

label *pathName* ?*options*?

Standard options

-activebackground, **-disabledforeground**, **-padx**, **-activeforeground**, **-font**, **-pady**, **-anchor**, **-foreground**, **-relief**, **-background**, **-highlightbackground**, **-takefocus**, **-bitmap**, **-highlightcolor**, **-text**, **-borderwidth**, **-highlightthickness**, **-textvariable**, **-compound**, **-image**, **-underline**, **-cursor**, **-justify**, **-wraplength**

See the options manual entry for details on the standard options.

Widget-specific options

-height (height/Height)
: Specifies a desired height for the label. If an image or bitmap is being displayed in the label then the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels); for text it is in lines of text. If this option is not specified, the label's desired height is computed from the size of the image or bitmap or text being displayed in it.

-state (state/State)
: Specifies one of three states for the label: **normal**, **active**, or **disabled**. In normal state the button is displayed using the **-foreground** and **-background** options. In active state the label is displayed using the **-activeforeground** and **-activebackground** options. In the disabled state the **-disabledforeground** and **-background** options determine how the button is displayed.

-width (width/Width)
: Specifies a desired width for the label. If an image or bitmap is being displayed in the label then the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels); for text it is in characters. If this option is not specified, the label's desired width is computed from the size of the image or bitmap or text being displayed in it.

Description

The **label** command creates a new window (given by the *pathName* argument) and makes it

into a label widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the label such as its colors, font, text, and initial relief. The **label** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A label is a widget that displays a textual string, bitmap or image. If text is displayed, it must all be in a single font, but it can occupy multiple lines on the screen (if it contains newlines or if wrapping occurs because of the **-wraplength** option) and one of the characters may optionally be underlined using the **-underline** option. The label can be manipulated in a few simple ways, such as changing its relief or text, using the commands described below.

Widget command

The **label** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following commands are possible for label widgets:

pathName* cget *option
: Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **label** command.

***pathName* configure ?*option*? ?*value option value ...*?**
: Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **label** command.

Bindings

When a new label is created, it has no default event bindings: labels are not intended to be interactive.

Example

```
# Make the widgets
label .t -text "This widget is at the top"    -bg red
label .b -text "This widget is at the bottom" -bg green
label .l -text "Left\nHand\nSide"
label .r -text "Right\nHand\nSide"
text .mid
.mid insert end "This layout is like Java's BorderLayout"
# Lay them out
pack .t   -side top    -fill x
pack .b   -side bottom -fill x
```

```
pack .l   -side left   -fill y
pack .r   -side right  -fill y
pack .mid -expand 1    -fill both
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

labelframe

Keywords: labelframe, widget

Name

labelframe - Create and manipulate 'labelframe' labelled container widgets

Synopsis

labelframe *pathName* ?*options*?

Standard options

-borderwidth, **-highlightbackground**, **-pady**, **-cursor**, **-highlightcolor**, **-relief**, **-font**, **-highlightthickness**, **-takefocus**, **-foreground**, **-padx**, **-text**

See the options manual entry for details on the standard options.

Widget-specific options

-background (background/Background)
This option is the same as the standard **-background** option except that its value may also be specified as an empty string. In this case, the widget will display no background or border, and no colors will be consumed from its colormap for its background and border.

-class (class/Class)
Specifies a class for the window. This class will be used when querying the option database for the window's other options, and it will also be used later for other purposes such as bindings. The **-class** option may not be changed with the **configure** widget command.

-colormap (colormap/Colormap)
Specifies a colormap to use for the window. The value may be either **new**, in which case a new colormap is created for the window and its children, or the name of another window (which must be on the same screen and have the same visual as *pathName*), in which case the new window will use the colormap from the specified window. If the **-colormap** option is not specified, the new window uses the same colormap as its parent. This option may not be changed with the **configure** widget command.

-height (height/Height)
Specifies the desired height for the window in any of the forms acceptable to Tk_GetPixels. If this option is less than or equal to zero then the window will not request any size at all.

-labelanchor (labelAnchor/LabelAnchor)
Specifies where to place the label. A label is only displayed if the **-text** option is not the empty string. Valid values for this option are (listing them clockwise) **nw**, **n**, **ne**, **en**, **e**, **es**, **se**, **s**,**sw**, **ws**, **w** and **wn**. The default value is **nw**.

-labelwidget (labelWidget/LabelWidget)
Specifies a widget to use as label. This overrides any **-text** option. The widget must exist

before being used as **-labelwidget** and if it is not a descendant of this window, it will be raised above it in the stacking order.

-visual (visual/Visual)
Specifies visual information for the new window in any of the forms accepted by Tk_GetVisual. If this option is not specified, the new window will use the same visual as its parent. The **-visual** option may not be modified with the **configure** widget command.

-width (width/Width)
Specifies the desired width for the window in any of the forms acceptable to Tk_GetPixels. If this option is less than or equal to zero then the window will not request any size at all.

Description

The **labelframe** command creates a new window (given by the *pathName* argument) and makes it into a labelframe widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the labelframe such as its background color and relief. The **labelframe** command returns the path name of the new window.

A labelframe is a simple widget. Its primary purpose is to act as a spacer or container for complex window layouts. It has the features of a frame plus the ability to display a label.

Widget command

The **labelframe** command creates a new Tcl command whose name is the same as the path name of the labelframe's window. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

PathName is the name of the command, which is the same as the labelframe widget's path name. *Option* and the *arg*s determine the exact behavior of the command. The following commands are possible for frame widgets:

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **labelframe** command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **labelframe** command.

Bindings

When a new labelframe is created, it has no default event bindings: labelframes are not intended to be interactive.

Example

This shows how to build part of a GUI for a hamburger vendor. The **labelframe** widgets are used to organize the available choices by the kinds of things that the choices are being made over.

```
grid [labelframe .burger -text "Burger"] \
     [labelframe .bun    -text "Bun"] -sticky news
grid [labelframe .cheese -text "Cheese Option"] \
     [labelframe .pickle -text "Pickle Option"] -sticky news
foreach {type name val} {
    burger Beef    beef
    burger Lamb    lamb
    burger Vegetarian beans

    bun    Plain   white
    bun    Sesame  seeds
    bun    Wholemeal brown

    cheese None    none
    cheese Cheddar cheddar
    cheese Edam    edam
    cheese Brie    brie
    cheese Gruy\u00e8re gruyere
    cheese "Monterey Jack" jack

    pickle None    none
    pickle Gherkins gherkins
    pickle Onions  onion
    pickle Chili   chili
} {
    set w [radiobutton .$type.$val -text $name -anchor w \
            -variable $type -value $val]
    pack $w -side top -fill x
}
set burger beef
set bun    white
set cheese none
set pickle none
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

listbox

Keywords: listbox, widget

Name

listbox - Create and manipulate 'listbox' item list widgets

Synopsis

listbox *pathName* ?*options*?

Standard options

-background, **-borderwidth**, **-cursor**, **-disabledforeground**, **-exportselection**, **-font**, **-foreground**, **-highlightbackground**, **-highlightcolor**, **-highlightthickness**, **-justify**, **-relief**, **-selectbackground**, **-selectborderwidth**, **-selectforeground**, **-setgrid**, **-takefocus**, **-xscrollcommand**, **-yscrollcommand**

See the options manual entry for details on the standard options.

Widget-specific options

-activestyle (activeStyle/ActiveStyle)
Specifies the style in which to draw the active element. This must be one of **dotbox** (show a focus ring around the active element), **none** (no special indication of active element) or **underline** (underline the active element). The default is **underline** on Windows, and **dotbox** elsewhere.

-height (height/Height)
Specifies the desired height for the window, in lines. If zero or less, then the desired height for the window is made just large enough to hold all the elements in the listbox.

-listvariable (listVariable/Variable)
Specifies the name of a global variable. The value of the variable is a list to be displayed inside the widget; if the variable value changes then the widget will automatically update itself to reflect the new value. Attempts to assign a variable with an invalid list value to **-listvariable** will cause an error. Attempts to unset a variable in use as a **-listvariable** will fail but will not generate an error.

-selectmode (selectMode/SelectMode)
Specifies one of several styles for manipulating the selection. The value of the option may be arbitrary, but the default bindings expect it to be either **single**, **browse**, **multiple**, or **extended**; the default value is **browse**.

-state (state/State)
Specifies one of two states for the listbox: **normal** or **disabled**. If the listbox is disabled then items may not be inserted or deleted, items are drawn in the **-disabledforeground** color, and selection cannot be modified and is not shown (though selection information is retained).

-width (width/Width)
Specifies the desired width for the window in characters. If the font does not have a uniform width then the width of the character "0" is used in translating from character units to screen units. If zero or less, then the desired width for the window is made just large enough to hold all the elements in the listbox.

Description

The **listbox** command creates a new window (given by the *pathName* argument) and makes it into a listbox widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the listbox such as its colors, font, text, and relief. The **listbox** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A listbox is a widget that displays a list of strings, one per line. When first created, a new listbox has no elements. Elements may be added or deleted using widget commands described below. In addition, one or more elements may be selected as described below. If a listbox is exporting its selection (see **-exportselection** option), then it will observe the standard X11 protocols for handling the selection. Listbox selections are available as type **STRING**; the value of the selection will be the text of the selected elements, with newlines separating the elements.

It is not necessary for all the elements to be displayed in the listbox window at once; commands described below may be used to change the view in the window. Listboxes allow scrolling in both directions using the standard **-xscrollcommand** and **-yscrollcommand** options. They also support scanning, as described below.

Indices

Many of the widget commands for listboxes take one or more indices as arguments. An index specifies a particular element of the listbox, in any of the following ways:

number
: Specifies the element as a numerical index, where 0 corresponds to the first element in the listbox.

active
: Indicates the element that has the location cursor. This element will be displayed as specified by **-activestyle** when the listbox has the keyboard focus, and it is specified with the **activate** widget command.

anchor
: Indicates the anchor point for the selection, which is set with the **selection anchor** widget command.

end
: Indicates the end of the listbox. For most commands this refers to the last element in the listbox, but for a few commands such as **index** and **insert** it refers to the element just after the last one.

@x,y
: Indicates the element that covers the point in the listbox window specified by *x* and *y* (in pixel coordinates). If no element covers that point, then the closest element to that point is used.

In the widget command descriptions below, arguments named *index*, *first*, and *last* always contain text indices in one of the above forms.

Widget command

The **listbox** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following commands are possible for listbox widgets:

pathName* activate *index
: Sets the active element to the one indicated by *index*. If *index* is outside the range of elements in the listbox then the closest element is activated. The active element is drawn as specified by **-activestyle** when the widget has the input focus, and its index may be

retrieved with the index **active**.

pathName* bbox *index
Returns a list of four numbers describing the bounding box of the text in the element given by *index*. The first two elements of the list give the x and y coordinates of the upper-left corner of the screen area covered by the text (specified in pixels relative to the widget) and the last two elements give the width and height of the area, in pixels. If no part of the element given by *index* is visible on the screen, or if *index* refers to a non-existent element, then the result is an empty string; if the element is partially visible, the result gives the full area of the element, including any parts that are not visible.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **listbox** command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **listbox** command.

***pathName* curselection**
Returns a list containing the numerical indices of all of the elements in the listbox that are currently selected. If there are no elements selected in the listbox then an empty string is returned.

***pathName* delete *first* ?*last*?**
Deletes one or more elements of the listbox. *First* and *last* are indices specifying the first and last elements in the range to delete. If *last* is not specified it defaults to *first*, i.e. a single element is deleted.

***pathName* get *first* ?*last*?**
If *last* is omitted, returns the contents of the listbox element indicated by *first*, or an empty string if *first* refers to a non-existent element. If *last* is specified, the command returns a list whose elements are all of the listbox elements between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

pathName* index *index
Returns the integer index value that corresponds to *index*. If *index* is **end** the return value is a count of the number of elements in the listbox (not the index of the last element).

***pathName* insert *index* ?*element element ...*?**
Inserts zero or more new elements in the list just before the element given by *index*. If *index* is specified as **end** then the new elements are added to the end of the list. Returns an empty string.

pathName* itemcget *index option
Returns the current value of the item configuration option given by *option*. *Option* may have any of the values accepted by the **itemconfigure** command.

***pathName* itemconfigure *index* ?*option*? ?*value*? ?*option value ...*?**

Query or modify the configuration options of an item in the listbox. If no *option* is specified, returns a list describing all of the available options for the item (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. The following options are currently supported for items:

-background *color*
Color specifies the background color to use when displaying the item. It may have any of the forms accepted by Tk_GetColor.

-foreground *color*
Color specifies the foreground color to use when displaying the item. It may have any of the forms accepted by Tk_GetColor.

-selectbackground *color*
color specifies the background color to use when displaying the item while it is selected. It may have any of the forms accepted by Tk_GetColor.

-selectforeground *color*
color specifies the foreground color to use when displaying the item while it is selected. It may have any of the forms accepted by Tk_GetColor.

pathName* nearest *y
Given a y-coordinate within the listbox window, this command returns the index of the (visible) listbox element nearest to that y-coordinate.

pathName* scan *option args
This command is used to implement scanning on listboxes. It has two forms, depending on *option*:

pathName* scan mark *x y
Records *x* and *y* and the current view in the listbox window; used in conjunction with later **scan dragto** commands. Typically this command is associated with a mouse button press in the widget. It returns an empty string.

***pathName* scan dragto *x y*.**
This command computes the difference between its *x* and *y* arguments and the *x* and *y* arguments to the last **scan mark** command for the widget. It then adjusts the view by 10 times the difference in coordinates. This command is typically associated with mouse motion events in the widget, to produce the effect of dragging the list at high speed through the window. The return value is an empty string.

pathName* see *index
Adjust the view in the listbox so that the element given by *index* is visible. If the element is already visible then the command has no effect; if the element is near one edge of the window then the listbox scrolls to bring the element into view at the edge; otherwise the listbox scrolls to center the element.

pathName* selection *option arg
This command is used to adjust the selection within a listbox. It has several forms, depending on *option*:

pathName* selection anchor *index
Sets the selection anchor to the element given by *index*. If *index* refers to a non-existent element, then the closest element is used. The selection anchor is the end of the selection that is fixed while dragging out a selection with the mouse. The index **anchor** may be used to refer to the anchor element.

***pathName* selection clear *first* ?*last*?**
If any of the elements between *first* and *last* (inclusive) are selected, they are deselected. The selection state is not changed for elements outside this range.

pathName* selection includes *index
Returns 1 if the element indicated by *index* is currently selected, 0 if it is not.

***pathName* selection set *first* ?*last*?**
Selects all of the elements in the range between *first* and *last*, inclusive, without affecting the selection state of elements outside that range.

***pathName* size**
Returns a decimal string indicating the total number of elements in the listbox.

pathName* xview ?*args
This command is used to query and change the horizontal position of the information in the widget's window. It can take any of the following forms:

***pathName* xview**
Returns a list containing two elements. Each element is a real fraction between 0 and 1; together they describe the horizontal span that is visible in the window. For example, if the first element is .2 and the second element is .6, 20% of the listbox's text is off-screen to the left, the middle 40% is visible in the window, and 40% of the text is off-screen to the right. These are the same values passed to scrollbars via the **-xscrollcommand** option.

pathName* xview *index
Adjusts the view in the window so that the character position given by *index* is displayed at the left edge of the window. Character positions are defined by the width of the character **0** .

pathName* xview moveto *fraction
Adjusts the view in the window so that *fraction* of the total width of the listbox text is off-screen to the left. *fraction* must be a fraction between 0 and 1.

pathName* xview scroll *number what
This command shifts the view in the window left or right according to *number* and *what*. *Number* must be an integer. *What* must be either **units** or **pages** or an abbreviation of one of these. If *what* is **units**, the view adjusts left or right by *number* character units (the width of the **0** character) on the display; if it is **pages** then the view adjusts by *number* screenfuls. If *number* is negative then characters farther to the left become visible; if it is positive then characters farther to the right become visible.

***pathName* yview ?*args*?**
This command is used to query and change the vertical position of the text in the widget's window. It can take any of the following forms:

***pathName* yview**
Returns a list containing two elements, both of which are real fractions between 0 and 1. The first element gives the position of the listbox element at the top of the window, relative

to the listbox as a whole (0.5 means it is halfway through the listbox, for example). The second element gives the position of the listbox element just after the last one in the window, relative to the listbox as a whole. These are the same values passed to scrollbars via the **-yscrollcommand** option.

pathName* yview *index
Adjusts the view in the window so that the element given by *index* is displayed at the top of the window.

pathName* yview moveto *fraction
Adjusts the view in the window so that the element given by *fraction* appears at the top of the window. *Fraction* is a fraction between 0 and 1; 0 indicates the first element in the listbox, 0.33 indicates the element one-third the way through the listbox, and so on.

pathName* yview scroll *number what
This command adjusts the view in the window up or down according to *number* and *what*. *Number* must be an integer. *What* must be either **units** or **pages**. If *what* is **units**, the view adjusts up or down by *number* lines; if it is **pages** then the view adjusts by *number* screenfuls. If *number* is negative then earlier elements become visible; if it is positive then later elements become visible.

Default bindings

Tk automatically creates class bindings for listboxes that give them Motif-like behavior. Much of the behavior of a listbox is determined by its **-selectmode** option, which selects one of four ways of dealing with the selection.

If the selection mode is **single** or **browse**, at most one element can be selected in the listbox at once. In both modes, clicking button 1 on an element selects it and deselects any other selected item. In **browse** mode it is also possible to drag the selection with button 1. On button 1, the listbox will also take focus if it has a **normal** state.

If the selection mode is **multiple** or **extended**, any number of elements may be selected at once, including discontiguous ranges. In **multiple** mode, clicking button 1 on an element toggles its selection state without affecting any other elements. In **extended** mode, pressing button 1 on an element selects it, deselects everything else, and sets the anchor to the element under the mouse; dragging the mouse with button 1 down extends the selection to include all the elements between the anchor and the element under the mouse, inclusive.

Most people will probably want to use **browse** mode for single selections and **extended** mode for multiple selections; the other modes appear to be useful only in special situations.

Any time the set of selected item(s) in the listbox is updated by the user through the keyboard or mouse, the virtual event **<<ListboxSelect>>** will be generated. This virtual event will not be generated when adjusting the selection with the *pathName* selection command. It is easiest to bind to this event to be made aware of any user changes to listbox selection.

In addition to the above behavior, the following additional behavior is defined by the default bindings:

1. In **extended** mode, the selected range can be adjusted by pressing button 1 with the Shift key down: this modifies the selection to consist of the elements between the anchor and the element under the mouse, inclusive. The un-anchored end of this new selection can also be dragged with the button down.

2. In **extended** mode, pressing button 1 with the Control key down starts a toggle operation: the anchor is set to the element under the mouse, and its selection state is reversed. The selection state of other elements is not changed. If the mouse is dragged with button 1 down, then the selection state of all elements between the anchor and the element under the mouse is set to match that of the anchor element; the selection state of all other elements remains what it was before the toggle operation began.
3. If the mouse leaves the listbox window with button 1 down, the window scrolls away from the mouse, making information visible that used to be off-screen on the side of the mouse. The scrolling continues until the mouse re-enters the window, the button is released, or the end of the listbox is reached.
4. Mouse button 2 may be used for scanning. If it is pressed and dragged over the listbox, the contents of the listbox drag at high speed in the direction the mouse moves.
5. If the Up or Down key is pressed, the location cursor (active element) moves up or down one element. If the selection mode is **browse** or **extended** then the new active element is also selected and all other elements are deselected. In **extended** mode the new active element becomes the selection anchor.
6. In **extended** mode, Shift-Up and Shift-Down move the location cursor (active element) up or down one element and also extend the selection to that element in a fashion similar to dragging with mouse button 1.
7. The Left and Right keys scroll the listbox view left and right by the width of the character **0**. Control-Left and Control-Right scroll the listbox view left and right by the width of the window. Control-Prior and Control-Next also scroll left and right by the width of the window.
8. The Prior and Next keys scroll the listbox view up and down by one page (the height of the window).
9. The Home and End keys scroll the listbox horizontally to the left and right edges, respectively.
10. Control-Home sets the location cursor to the first element in the listbox, selects that element, and deselects everything else in the listbox.
11. Control-End sets the location cursor to the last element in the listbox, selects that element, and deselects everything else in the listbox.
12. In **extended** mode, Control-Shift-Home extends the selection to the first element in the listbox and Control-Shift-End extends the selection to the last element.
13. In **multiple** mode, Control-Shift-Home moves the location cursor to the first element in the listbox and Control-Shift-End moves the location cursor to the last element.
14. The space and Select keys make a selection at the location cursor (active element) just as if mouse button 1 had been pressed over this element.
15. In **extended** mode, Control-Shift-space and Shift-Select extend the selection to the active element just as if button 1 had been pressed with the Shift key down.
16. In **extended** mode, the Escape key cancels the most recent selection and restores all the elements in the selected range to their previous selection state.
17. Control-slash selects everything in the widget, except in **single** and **browse** modes, in which case it selects the active element and deselects everything else.
18. Control-backslash deselects everything in the widget, except in **browse** mode where it has no effect.
19. The F16 key (labelled Copy on many Sun workstations) or Meta-w copies the selection in the widget to the clipboard, if there is a selection.

The behavior of listboxes can be changed by defining new bindings for individual widgets or by

redefining the class bindings.

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

loadTk

Keywords: loadTk, alias, auto-loading, auto_mkindex, load, parent interpreter, safe interpreter, child interpreter, source

Name

safe::loadTk - Load Tk into a safe interpreter.

Synopsis

safe::loadTk *child* ?*-use windowId*? ?*-display displayName*?

Description

Safe Tk is based on Safe Tcl, which provides a mechanism that allows restricted and mediated access to auto-loading and packages for safe interpreters. Safe Tk adds the ability to configure the interpreter for safe Tk operations and load Tk into safe interpreters.

The **safe::loadTk** command initializes the required data structures in the named safe interpreter and then loads Tk into it. The interpreter must have been created with **safe::interpCreate** or have been initialized with **safe::interpInit**. The command returns the name of the safe interpreter. If **-use** is specified, the window identified by the specified system dependent identifier *windowId* is used to contain the "." window of the safe interpreter; it can be any valid id, eventually referencing a window belonging to another application. As a convenience, if the window you plan to use is a Tk Window of the application you can use the window name (e.g., "**.x.y**") instead of its window Id (e.g., from winfo id **.x.y**). When **-use** is not specified, a new toplevel window is created for the "." window of the safe interpreter. On X11 if you want the embedded window to use another display than the default one, specify it with **-display**. See the Security issues section below for implementation details.

Security issues

Please read the safe manual page for Tcl to learn about the basic security considerations for Safe Tcl.

safe::loadTk adds the value of tk_library taken from the parent interpreter to the virtual access path of the safe interpreter so that auto-loading will work in the safe interpreter.

Tk initialization is now safe with respect to not trusting the child's state for startup. **safe::loadTk** registers the child's name so when the Tk initialization (Tk_SafeInit) is called and in turn calls the parent's **safe::InitTk** it will return the desired argv equivalent (**-use** *windowId*, correct **-display**, etc.)

When **-use** is not used, the new toplevel created is specially decorated so the user is always aware that the user interface presented comes from a potentially unsafe code and can easily delete the corresponding interpreter.

On X11, conflicting **-use** and **-display** are likely to generate a fatal X error.

Copyright (c) 1995-1996 Sun Microsystems, Inc.

lower

Keywords: lower, obscure, stacking order

Name

lower - Change a window's position in the stacking order

Synopsis

lower *window* ?*belowThis*?

Description

If the *belowThis* argument is omitted then the command lowers *window* so that it is below all of its siblings in the stacking order (it will be obscured by any siblings that overlap it and will not obscure any siblings). If *belowThis* is specified then it must be the path name of a window that is either a sibling of *window* or the descendant of a sibling of *window*. In this case the **lower** command will insert *window* into the stacking order just below *belowThis* (or the ancestor of *belowThis* that is a sibling of *window*); this could end up either raising or lowering *window*.

All toplevel windows may be restacked with respect to each other, whatever their relative path names, but the window manager is not obligated to strictly honor requests to restack.

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

menu

Keywords: menu, widget

Name

menu, tk_menuSetFocus - Create and manipulate 'menu' widgets and menubars

Synopsis

menu *pathName* ?*options*?
tk_menuSetFocus *pathName*

Standard options

-activebackground, **-borderwidth**, **-foreground**, **-activeborderwidth**, **-cursor**, **-relief**, **-activeforeground**, **-disabledforeground**, **-takefocus**, **-background**, **-font**

See the options manual entry for details on the standard options.

Widget-specific options

-postcommand (postCommand/Command)
: If this option is specified then it provides a Tcl command to execute each time the menu is posted. The command is invoked by the **post** widget command before posting the menu. Note that in Tk 8.0 on Macintosh and Windows, all post-commands in a system of menus are executed before any of those menus are posted. This is due to the limitations in the individual platforms' menu managers.

-selectcolor (selectColor/Background)
: For menu entries that are check buttons or radio buttons, this option specifies the color to display in the indicator when the check button or radio button is selected.

-tearoff (tearOff/TearOff)
This option must have a proper boolean value, which specifies whether or not the menu should include a tear-off entry at the top. If so, it will exist as entry 0 of the menu and the other entries will number starting at 1. The default menu bindings arrange for the menu to be torn off when the tear-off entry is invoked. This option is ignored under Aqua/macOS, where menus cannot be torn off.

-tearoffcommand (tearOffCommand/TearOffCommand)
If this option has a non-empty value, then it specifies a Tcl command to invoke whenever the menu is torn off. The actual command will consist of the value of this option, followed by a space, followed by the name of the menu window, followed by a space, followed by the name of the name of the torn off menu window. For example, if the option's value is "**a b**" and menu **.x.y** is torn off to create a new menu **.x.tearoff1**, then the command "**a b .x.y .x.tearoff1**" will be invoked. This option is ignored under Aqua/macOS, where menus cannot be torn off.

-title (title/Title)
The string will be used to title the window created when this menu is torn off. If the title is NULL, then the window will have the title of the menubutton or the text of the cascade item from which this menu was invoked.

-type (type/Type)
This option can be one of **menubar**, **tearoff**, or **normal**, and is set when the menu is created. While the string returned by the configuration database will change if this option is changed, this does not affect the menu widget's behavior. This is used by the cloning mechanism and is not normally set outside of the Tk library.

Introduction

The **menu** command creates a new top-level window (given by the *pathName* argument) and makes it into a menu widget. That menu widget can either be used as a pop-up window or applied to a toplevel (with its **-menu** option) to make it into the menubar for that toplevel. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the menu such as its colors and font. The **menu** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A menu is a widget that displays a collection of one-line entries arranged in one or more columns. There exist several different types of entries, each with different properties. Entries of different types may be combined in a single menu. Menu entries are not the same as entry widgets. In fact, menu entries are not even distinct widgets; the entire menu is one widget.

Menu entries are displayed with up to three separate fields. The main field is a label in the form of a text string, a bitmap, or an image, controlled by the **-label**, **-bitmap**, and **-image** options for the entry. If the **-accelerator** option is specified for an entry then a second textual field is displayed to the right of the label. The accelerator typically describes a keystroke sequence that may be used in the application to cause the same result as invoking the menu entry. This is a display option, it does not actually set the corresponding binding (which can be achieved using the bind command). The third field is an *indicator*. The indicator is present only for checkbutton or radiobutton entries. It indicates whether the entry is selected or not, and is displayed to the left of the entry's string.

In normal use, an entry becomes active (displays itself differently) whenever the mouse pointer

is over the entry. If a mouse button is released over the entry then the entry is *invoked*. The effect of invocation is different for each type of entry; these effects are described below in the sections on individual entries.

Entries may be *disabled*, which causes their labels and accelerators to be displayed with dimmer colors. The default menu bindings will not allow a disabled entry to be activated or invoked. Disabled entries may be re-enabled, at which point it becomes possible to activate and invoke them again.

Whenever a menu's active entry is changed, a <> virtual event is send to the menu. The active item can then be queried from the menu, and an action can be taken, such as setting context-sensitive help text for the entry.

Types of entries

Command entries

The most common kind of menu entry is a command entry, which behaves much like a button widget. When a command entry is invoked, a Tcl command is executed. The Tcl command is specified with the **-command** option.

Separator entries

A separator is an entry that is displayed as a horizontal dividing line. A separator may not be activated or invoked, and it has no behavior other than its display appearance.

Checkbutton entries

A checkbutton menu entry behaves much like a checkbutton widget. When it is invoked it toggles back and forth between the selected and deselected states. When the entry is selected, a particular value is stored in a particular global variable (as determined by the **-onvalue** and **-variable** options for the entry); when the entry is deselected another value (determined by the **-offvalue** option) is stored in the global variable. An indicator box is displayed to the left of the label in a checkbutton entry. If the entry is selected then the indicator's center is displayed in the color given by the **-selectcolor** option for the entry; otherwise the indicator's center is displayed in the background color for the menu. If a **-command** option is specified for a checkbutton entry, then its value is evaluated as a Tcl command each time the entry is invoked; this happens after toggling the entry's selected state.

Radiobutton entries

A radiobutton menu entry behaves much like a radiobutton widget. Radiobutton entries are organized in groups of which only one entry may be selected at a time. Whenever a particular entry becomes selected it stores a particular value into a particular global variable (as determined by the **-value** and **-variable** options for the entry). This action causes any previously-selected entry in the same group to deselect itself. Once an entry has become selected, any change to the entry's associated variable will cause the entry to deselect itself. Grouping of radiobutton entries is determined by their associated variables: if two entries have the same associated variable then they are in the same group. An indicator diamond is displayed to the left of the label in each radiobutton entry. If the entry is selected then the indicator's center is displayed in the color given by the **-selectcolor** option for the entry; otherwise the indicator's center is displayed in the background color for the menu. If a **-command** option is specified for a radiobutton entry, then its value is evaluated as a Tcl

command each time the entry is invoked; this happens after selecting the entry.

Cascade entries

A cascade entry is one with an associated menu (determined by the **-menu** option). Cascade entries allow the construction of cascading menus. The **postcascade** widget command can be used to post and unpost the associated menu just next to of the cascade entry. The associated menu must be a child of the menu containing the cascade entry (this is needed in order for menu traversal to work correctly).

A cascade entry posts its associated menu by invoking a Tcl command of the form

```
menu post x y
```

where *menu* is the path name of the associated menu, and *x* and *y* are the root-window coordinates of the upper-right corner of the cascade entry. On Unix, the lower-level menu is unposted by executing a Tcl command with the form

```
menu unpost
```

where *menu* is the name of the associated menu. On other platforms, the platform's native code takes care of unposting the menu.

If a **-command** option is specified for a cascade entry then it is evaluated as a Tcl command whenever the entry is invoked. This is not supported on Windows.

Tear-off entries

A tear-off entry appears at the top of the menu if enabled with the **-tearoff** option. It is not like other menu entries in that it cannot be created with the **add** widget command and cannot be deleted with the **delete** widget command. When a tear-off entry is created it appears as a dashed line at the top of the menu. Under the default bindings, invoking the tear-off entry causes a torn-off copy to be made of the menu and all of its submenus.

Menubars

Any menu can be set as a menubar for a toplevel window (see toplevel command for syntax). On the Macintosh, whenever the toplevel is in front, this menu's cascade items will appear in the menubar across the top of the main monitor. On Windows and Unix, this menu's items will be displayed in a menubar across the top of the window. These menus will behave according to the interface guidelines of their platforms. For every menu set as a menubar, a clone menu is made. See the Clones section for more information.

As noted, menubars may behave differently on different platforms. One example of this concerns the handling of checkbuttons and radiobuttons within the menu. While it is permitted to put these menu elements on menubars, they may not be drawn with indicators on some platforms, due to system restrictions.

Special menus in menubars

Certain menus in a menubar will be treated specially. On the Macintosh, access to the special Application, Window and Help menus is provided. On Windows, access to the Windows System menu in each window is provided. On X Windows, a special right-justified help menu may be provided if Motif menu compatibility is enabled. In all cases, these menus must be created with the command name of the menubar menu concatenated with the special name. So for a

menubar named .menubar, on the Macintosh, the special menus would be .menubar.apple, .menubar.window and .menubar.help; on Windows, the special menu would be .menubar.system; on X Windows, the help menu would be .menubar.help.

When Tk sees a .menubar.apple menu as the first menu in a menubar on the Macintosh, that menu's contents make up the first items of the Application menu whenever the window containing the menubar is in front. After all of the Tk-defined items, the menu will have a separator, followed by all standard Application menu items. Such a .apple menu must be present in a menu when that menu is first configured as a toplevel's menubar, otherwise a default application menu (hidden from Tk) will be inserted into the menubar at that time and subsequent addition of a .apple menu will no longer result in it becoming the Application menu.

When Tk sees a .menubar.window menu on the Macintosh, the menu's contents are inserted into the standard Window menu of the user's menubar whenever the window's menubar is in front. The first items in the menu are provided by macOS, and the names of the current toplevels are automatically appended after all the Tk-defined items and a separator. The Window menu on the Mac also allows toggling the window into a fullscreen state, and managing a tabbed window interface (multiple windows grouped into a single window) if supported by that version of the operating system.

When Tk sees a .menubar.help menu on the Macintosh, the menu's contents are appended to the standard Help menu of the user's menubar whenever the window's menubar is in front. The first items in the menu are provided by macOS.

When Tk sees a System menu on Windows, its items are appended to the system menu that the menubar is attached to. This menu is tied to the application icon and can be invoked with the mouse or by typing Alt+Spacebar. Due to limitations in the Windows API, any font changes, colors, images, bitmaps, or tearoff images will not appear in the system menu.

When Tk sees a Help menu on X Windows and Motif menu compatibility is enabled the menu is moved to be last in the menubar and is right justified. Motif menu compatibility is enabled by setting the Tk option ***Menu.useMotifHelp** to true or by calling **tk::classic::restore menu**.

Clones

When a menu is set as a menubar for a toplevel window, or when a menu is torn off, a clone of the menu is made. This clone is a menu widget in its own right, but it is a child of the original. Changes in the configuration of the original are reflected in the clone. Additionally, any cascades that are pointed to are also cloned so that menu traversal will work right. Clones are destroyed when either the tearoff or menubar goes away, or when the original menu is destroyed.

Widget command

The **menu** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command.

Many of the widget commands for a menu take as one argument an indicator of which entry of the menu to operate on. These indicators are called *index*es and may be specified in any of the following forms:

active

Indicates the entry that is currently active. If no entry is active then this form is equivalent to **none**. This form may not be abbreviated.

end
Indicates the bottommost entry in the menu. If there are no entries in the menu then this form is equivalent to **none**. This form may not be abbreviated.

last
Same as **end**.

none
Indicates "no entry at all"; this is used most commonly with the **activate** option to deactivate all the entries in the menu. In most cases the specification of **none** causes nothing to happen in the widget command. This form may not be abbreviated.

@x,y
Indicates the entry that covers the point in the menu's window specified by *x* and *y* (in pixel coordinates). If no entry covers that point, then this form is equivalent to **none**. If only a single number is specified, it is treated as the y-coordinate.

number
Specifies the entry numerically, where 0 corresponds to the top-most entry of the menu, 1 to the entry below it, and so on.

pattern
If the index does not satisfy one of the above forms then this form is used. *Pattern* is pattern-matched against the label of each entry in the menu, in order from the top down, until a matching entry is found. The rules of string match are used.

If the index could match more than one of the above forms, then the form earlier in the above list takes precedence.

The following widget commands are possible for menu widgets:

pathName* activate *index
Change the state of the entry indicated by *index* to **active** and redisplay it using its active colors. Any previously-active entry is deactivated. If *index* is specified as **none**, or if the specified entry is disabled, then the menu ends up with no active entry. Returns an empty string.

***pathName* add *type* ?*option value option value ...*?**
Add a new entry to the bottom of the menu. The new entry's type is given by *type* and must be one of **cascade**, **checkbutton**, **command**, **radiobutton**, or **separator**, or a unique abbreviation of one of the above. If additional arguments are present, they specify the options listed in the Menu entry options section below. The **add** widget command returns an empty string.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **menu** command.

***pathName* clone *newPathname* ?*cloneType*?**
Makes a clone of the current menu named *newPathName*. This clone is a menu in its own right, but any changes to the clone are propagated to the original menu and vice versa. *cloneType* can be **normal**, **menubar**, or **tearoff**. Should not normally be called outside of the Tk library. See the Clones section for more information.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **menu** command.

***pathName* delete *index1* ?*index2*?**
Delete all of the menu entries between *index1* and *index2* inclusive. If *index2* is omitted then it defaults to *index1*. Attempts to delete a tear-off menu entry are ignored (instead, you should change the **-tearoff** option to remove the tear-off entry).

pathName* entrycget *index option
Returns the current value of a configuration option for the entry given by *index*. *Option* may have any of the names described in the Menu entry options section below.

***pathName* entryconfigure *index* ?*options...*?**
This command is similar to the **configure** command, except that it applies to the options for an individual entry, whereas **configure** applies to the options for the menu as a whole. *Options* may have any of the values described in the Menu entry options section below. If *options* are specified, options are modified as indicated in the command and the command returns an empty string. If no *options* are specified, returns a list describing the current options for entry *index* (see Tk_ConfigureInfo for information on the format of this list).

pathName* index *index
Returns the numerical index corresponding to *index*, or **none** if *index* was specified as **none**.

***pathName* insert *index type* ?*option value option value ...*?**
Same as the **add** widget command except that it inserts the new entry just before the entry given by *index*, instead of appending to the end of the menu. The *type*, *option*, and *value* arguments have the same interpretation as for the **add** widget command. It is not possible to insert new menu entries before the tear-off entry, if the menu has one.

pathName* invoke *index
Invoke the action of the menu entry. See the sections on the individual entries above for details on what happens. If the menu entry is disabled then nothing happens. If the entry has a command associated with it then the result of that command is returned as the result of the **invoke** widget command. Otherwise the result is an empty string. Note: invoking a menu entry does not automatically unpost the menu; the default bindings normally take care of this before invoking the **invoke** widget command.

***pathName* post *x y* ?*index*?**
Arrange for the menu to be displayed on the screen at the root-window coordinates given by *x* and *y*. If an index is specified the menu will be located so that the entry with that index is displayed at the point. These coordinates are adjusted if necessary to guarantee that the entire menu is visible on the screen. This command normally returns an empty string. If the **-postcommand** option has been specified, then its value is executed as a Tcl script before posting the menu and the result of that script is returned as the result of the **post** widget command. If an error returns while executing the command, then the error is returned without

posting the menu.

pathName* postcascade *index
Posts the submenu associated with the cascade entry given by *index*, and unposts any previously posted submenu. If *index* does not correspond to a cascade entry, or if *pathName* is not posted, the command has no effect except to unpost any currently posted submenu.

pathName* type *index
Returns the type of the menu entry given by *index*. This is the *type* argument passed to the **add** or **insert** widget command when the entry was created, such as **command** or **separator**, or **tearoff** for a tear-off entry.

***pathName* unpost**
Unmap the window so that it is no longer displayed. If a lower-level cascaded menu is posted, unpost that menu. Returns an empty string. This subcommand does not work on Windows and the Macintosh, as those platforms have their own way of unposting menus.

pathName* xposition *index
Returns a decimal string giving the x-coordinate within the menu window of the leftmost pixel in the entry specified by *index*.

pathName* yposition *index
Returns a decimal string giving the y-coordinate within the menu window of the topmost pixel in the entry specified by *index*.

Menu entry options

The following options are allowed on menu entries. Most options are not supported by all entry types.

-activebackground *value*
Specifies a background color to use for displaying this entry when it is active. This option is ignored on Aqua/macOS. If it is specified as an empty string (the default), then the **-activebackground** option for the overall menu is used. If the tk_strictMotif variable has been set to request strict Motif compliance, then this option is ignored and the **-background** option is used in its place. This option is not available for separator or tear-off entries.

-activeforeground *value*
Specifies a foreground color to use for displaying this entry when it is active. This option is ignored on Aqua/macOS. If this option is specified as an empty string (the default), then the **-activeforeground** option for the overall menu is used. This option is not available for separator or tear-off entries.

-accelerator *value*
Specifies a string to display at the right side of the menu entry. Normally describes an accelerator keystroke sequence that may be used to invoke the same function as the menu entry. This is a display option, it does not actually set the corresponding binding (which can be achieved using the bind command). This option is not available for separator or tear-off entries.

-background *value*
Specifies a background color to use for displaying this entry when it is in the normal state (neither active nor disabled). This option is ignored on Aqua/macOS. If it is specified as an empty string (the default), then the **-background** option for the overall menu is used. This option is not available for separator or tear-off entries.

-bitmap *value*
Specifies a bitmap to display in the menu instead of a textual label, in any of the forms accepted by Tk_GetBitmap. This option overrides the **-label** option (as controlled by the **-compound** option) but may be reset to an empty string to enable a textual label to be displayed. If a **-image** option has been specified, it overrides **-bitmap**. This option is not available for separator or tear-off entries.

-columnbreak *value*
When this option is zero, the entry appears below the previous entry. When this option is one, the entry appears at the top of a new column in the menu. This option is ignored on Aqua/macOS, where menus are always a single column.

-command *value*
Specifies a Tcl command to execute when the menu entry is invoked. Not available for separator or tear-off entries.

-compound *value*
Specifies whether the menu entry should display both an image and text, and if so, where the image should be placed relative to the text. Valid values for this option are **bottom**, **center**, **left**, **none**, **right** and **top**. The default value is **none**, meaning that the button will display either an image or text, depending on the values of the **-image** and **-bitmap** options.

-font *value*
Specifies the font to use when drawing the label or accelerator string in this entry. If this option is specified as an empty string (the default) then the **-font** option for the overall menu is used. This option is not available for separator or tear-off entries.

-foreground *value*
Specifies a foreground color to use for displaying this entry when it is in the normal state (neither active nor disabled). This option is ignored on Aqua/macOS. If it is specified as an empty string (the default), then the **-foreground** option for the overall menu is used. This option is not available for separator or tear-off entries.

-hidemargin *value*
Specifies whether the standard margins should be drawn for this menu entry. This is useful when creating palette with images in them, i.e., color palettes, pattern palettes, etc. 1 indicates that the margin for the entry is hidden; 0 means that the margin is used.

-image *value*
Specifies an image to display in the menu instead of a text string or bitmap. The image must have been created by some previous invocation of image create. This option overrides the **-label** and **-bitmap** options (as controlled by the **-compound** option) but may be reset to an empty string to enable a textual or bitmap label to be displayed. This option is not available for separator or tear-off entries.

-indicatoron *value*
Available only for checkbutton and radiobutton entries. *Value* is a boolean that determines whether or not the indicator should be displayed.

-label *value*
Specifies a string to display as an identifying label in the menu entry. Not available for separator or tear-off entries.

-menu *value*
Available only for cascade entries. Specifies the path name of the submenu associated with

this entry. The submenu must be a child of the menu.

-offvalue *value*
Available only for checkbutton entries. Specifies the value to store in the entry's associated variable when the entry is deselected.

-onvalue *value*
Available only for checkbutton entries. Specifies the value to store in the entry's associated variable when the entry is selected.

-selectcolor *value*
Available only for checkbutton and radiobutton entries. Specifies the color to display in the indicator when the entry is selected. If the value is an empty string (the default) then the **-selectcolor** option for the menu determines the indicator color.

-selectimage *value*
Available only for checkbutton and radiobutton entries. Specifies an image to display in the entry (in place of the **-image** option) when it is selected. *Value* is the name of an image, which must have been created by some previous invocation of image create. This option is ignored unless the **-image** option has been specified.

-state *value*
Specifies one of three states for the entry: **normal**, **active**, or **disabled**. In normal state the entry is displayed using the **-foreground** option for the menu and the **-background** option from the entry or the menu. The active state is typically used when the pointer is over the entry. In active state the entry is displayed using the **-activeforeground** option for the menu along with the **-activebackground** option from the entry. Disabled state means that the entry should be insensitive: the default bindings will refuse to activate or invoke the entry. In this state the entry is displayed according to the **-disabledforeground** option for the menu and the **-background** option from the entry. This option is not available for separator entries.

-underline *value*
Specifies the integer index of a character to underline in the entry. This option is also queried by the default bindings and used to implement keyboard traversal. 0 corresponds to the first character of the text displayed in the entry, 1 to the next character, and so on. If a bitmap or image is displayed in the entry then this option is ignored. This option is not available for separator or tear-off entries.

-value *value*
Available only for radiobutton entries. Specifies the value to store in the entry's associated variable when the entry is selected. If an empty string is specified, then the **-label** option for the entry as the value to store in the variable.

-variable *value*
Available only for checkbutton and radiobutton entries. Specifies the name of a global variable to set when the entry is selected. For checkbutton entries the variable is also set when the entry is deselected. For radiobutton entries, changing the variable causes the currently-selected entry to deselect itself.

For checkbutton entries, the default value of this option is taken from the **-label** option, and for radiobutton entries a single fixed value is used. It is recommended that you always set the **-variable** option when creating either a checkbutton or a radiobutton.

Menu configurations

The default bindings support four different ways of using menus:

Pulldown Menus in Menubar

This is the most common case. You create a menu widget that will become the menu bar. You then add cascade entries to this menu, specifying the pull down menus you wish to use in your menu bar. You then create all of the pulldowns. Once you have done this, specify the menu using the **-menu** option of the toplevel's widget command. See the toplevel manual entry for details.

Pulldown Menus in Menu Buttons

This is the compatible way to do menu bars. You create one menubutton widget for each top-level menu, and typically you arrange a series of menubuttons in a row in a menubar window. You also create the top-level menus and any cascaded submenus, and tie them together with **-menu** options in menubuttons and cascade menu entries. The top-level menu must be a child of the menubutton, and each submenu must be a child of the menu that refers to it. Once you have done this, the default bindings will allow users to traverse and invoke the tree of menus via its menubutton; see the menubutton manual entry for details.

Popup Menus

Popup menus typically post in response to a mouse button press or keystroke. You create the popup menus and any cascaded submenus, then you call the tk_popup procedure at the appropriate time to post the top-level menu.

Option Menus

An option menu consists of a menubutton with an associated menu that allows you to select one of several values. The current value is displayed in the menubutton and is also stored in a global variable. Use the tk_optionMenu procedure to create option menubuttons and their menus.

Torn-off Menus

You create a torn-off menu by invoking the tear-off entry at the top of an existing menu. The default bindings will create a new menu that is a copy of the original menu and leave it permanently posted as a top-level window. The torn-off menu behaves just the same as the original menu.

Default bindings

Tk automatically creates class bindings for menus that give them the following default behavior:

1. When the mouse enters a menu, the entry underneath the mouse cursor activates; as the mouse moves around the menu, the active entry changes to track the mouse.
2. When the mouse leaves a menu all of the entries in the menu deactivate, except in the special case where the mouse moves from a menu to a cascaded submenu.
3. When a button is released over a menu, the active entry (if any) is invoked. The menu also unposts unless it is a torn-off menu.
4. The Space and Return keys invoke the active entry and unpost the menu.
5. If any of the entries in a menu have letters underlined with the **-underline** option, then pressing one of the underlined letters (or its upper-case or lower-case equivalent) invokes that entry and unposts the menu.
6. The Escape key aborts a menu selection in progress without invoking any entry. It also unposts the menu unless it is a torn-off menu.

7. The Up and Down keys activate the next higher or lower entry in the menu. When one end of the menu is reached, the active entry wraps around to the other end.
8. The Left key moves to the next menu to the left. If the current menu is a cascaded submenu, then the submenu is unposted and the current menu entry becomes the cascade entry in the parent. If the current menu is a top-level menu posted from a menubutton, then the current menubutton is unposted and the next menubutton to the left is posted. Otherwise the key has no effect. The left-right order of menubuttons is determined by their stacking order: Tk assumes that the lowest menubutton (which by default is the first one created) is on the left.
9. The Right key moves to the next menu to the right. If the current entry is a cascade entry, then the submenu is posted and the current menu entry becomes the first entry in the submenu. Otherwise, if the current menu was posted from a menubutton, then the current menubutton is unposted and the next menubutton to the right is posted.

Disabled menu entries are non-responsive: they do not activate and they ignore mouse button presses and releases.

Several of the bindings make use of the command **tk_menuSetFocus**. It saves the current focus and sets the focus to its *pathName* argument, which is a menu widget.

The behavior of menus can be changed by defining new bindings for individual widgets or by redefining the class bindings.

Bugs

At present it is not possible to use the option database to specify values for the options to individual entries.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

menubar

Keywords: menubar, keyboard traversal, menu, menu bar, post

Name

tk_menuBar, tk_bindForTraversal - Obsolete support for menu bars

Synopsis

tk_menuBar *frame* ?*menu menu* ...?
tk_bindForTraversal *arg arg ...*

Description

These procedures were used in Tk 3.6 and earlier releases to help manage pulldown menus and to implement keyboard traversal of menus. In Tk 4.0 and later releases they are no longer needed. Stubs for these procedures have been retained for backward compatibility, but they have no effect. You should remove calls to these procedures from your code, since eventually the procedures will go away.

From Tk 8.0 onwards, you should instead construct your menubar as a normal menu and then attach it to the toplevel of your choice using the **-menu** option of that widget.

Copyright (c) 1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

menubutton

Keywords: menubutton, widget

Name

menubutton - Create and manipulate 'menubutton' pop-up menu indicator widgets

Synopsis

menubutton *pathName* ?*options*?

Standard options

-activebackground, **-disabledforeground**, **-padx**, **-activeforeground**, **-font**, **-pady**, **-anchor**, **-foreground**, **-relief**, **-background**, **-highlightbackground**, **-takefocus**, **-bitmap**, **-highlightcolor**, **-text**, **-borderwidth**, **-highlightthickness**, **-textvariable**, **-cursor**, **-image**, **-underline**, **-compound**, **-justify**, **-wraplength**

See the options manual entry for details on the standard options.

Widget-specific options

-direction (direction/Direction)

Specifies where the menu is going to be popup up. **above** tries to pop the menu above the menubutton. **below** tries to pop the menu below the menubutton. **left** tries to pop the menu to the left of the menubutton. **right** tries to pop the menu to the right of the menu button. flush pops the menu directly over the menubutton. In the case of **above** or **below**, the direction will be reversed if the menu would show offscreen.

-height (height/Height)

Specifies a desired height for the menubutton. If an image or bitmap is being displayed in the menubutton then the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels); for text it is in lines of text. If this option is not specified, the menubutton's desired height is computed from the size of the image or bitmap or text being displayed in it.

-indicatoron (indicatorOn/IndicatorOn)

The value must be a proper boolean value. If it is true then a small indicator rectangle will be displayed on the right side of the menubutton and the default menu bindings will treat this as an option menubutton. If false then no indicator will be displayed.

-menu (menu**/MenuName)**

Specifies the path name of the menu associated with this menubutton. The menu must be a child of the menubutton.

-state (state/State)

Specifies one of three states for the menubutton: **normal**, **active**, or **disabled**. In normal state the menubutton is displayed using the **foreground** and **background** options. The active state is typically used when the pointer is over the menubutton. In active state the menubutton is displayed using the **-activeforeground** and **-activebackground** options. Disabled state means that the menubutton should be insensitive: the default bindings will refuse to activate the widget and will ignore mouse button presses. In this state the **-disabledforeground** and **-background** options determine how the button is displayed.

-width (width/Width)

Specifies a desired width for the menubutton. If an image or bitmap is being displayed in the

menubutton then the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels); for text it is in characters. If this option is not specified, the menubutton's desired width is computed from the size of the image or bitmap or text being displayed in it.

Introduction

The **menubutton** command creates a new window (given by the *pathName* argument) and makes it into a menubutton widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the menubutton such as its colors, font, text, and initial relief. The **menubutton** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A menubutton is a widget that displays a textual string, bitmap, or image and is associated with a menu widget. If text is displayed, it must all be in a single font, but it can occupy multiple lines on the screen (if it contains newlines or if wrapping occurs because of the **-wraplength** option) and one of the characters may optionally be underlined using the **-underline** option. In normal usage, pressing mouse button 1 over the menubutton causes the associated menu to be posted just underneath the menubutton. If the mouse is moved over the menu before releasing the mouse button, the button release causes the underlying menu entry to be invoked. When the button is released, the menu is unposted.

Menubuttons are used to construct a tk_optionMenu, which is the preferred mechanism for allowing a user to select one item from a list on macOS.

Menubuttons were also typically organized into groups called menu bars that allow scanning: if the mouse button is pressed over one menubutton (causing it to post its menu) and the mouse is moved over another menubutton in the same menu bar without releasing the mouse button, then the menu of the first menubutton is unposted and the menu of the new menubutton is posted instead. *This use is deprecated* in favor of setting a menu directly as a menubar; see the toplevel's **-menu** option for how to do that.

There are several interactions between menubuttons and menus; see the menu manual entry for information on various menu configurations, such as pulldown menus and option menus.

Widget command

The **menubutton** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following commands are possible for menubutton widgets:

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **menubutton** command.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list

describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **menubutton** command.

Default bindings

Tk automatically creates class bindings for menubuttons that give them the following default behavior:

1. A menubutton activates whenever the mouse passes over it and deactivates whenever the mouse leaves it.
2. Pressing mouse button 1 over a menubutton posts the menubutton: its relief changes to raised and its associated menu is posted under the menubutton. If the mouse is dragged down into the menu with the button still down, and if the mouse button is then released over an entry in the menu, the menubutton is unposted and the menu entry is invoked.
3. If button 1 is pressed over a menubutton and then released over that menubutton, the menubutton stays posted: you can still move the mouse over the menu and click button 1 on an entry to invoke it. Once a menu entry has been invoked, the menubutton unposts itself.
4. If button 1 is pressed over a menubutton and then dragged over some other menubutton, the original menubutton unposts itself and the new menubutton posts.
5. If button 1 is pressed over a menubutton and released outside any menubutton or menu, the menubutton unposts without invoking any menu entry.
6. When a menubutton is posted, its associated menu claims the input focus to allow keyboard traversal of the menu and its submenus. See the menu manual entry for details on these bindings.
7. If the **-underline** option has been specified for a menubutton then keyboard traversal may be used to post the menubutton: Alt+*x*, where *x* is the underlined character (or its lower-case or upper-case equivalent), may be typed in any window under the menubutton's toplevel to post the menubutton.
8. The F10 key may be typed in any window to post the first menubutton under its toplevel window that is not disabled.
9. If a menubutton has the input focus, the space and return keys post the menubutton.

If the menubutton's state is **disabled** then none of the above actions occur: the menubutton is completely non-responsive.

The behavior of menubuttons can be changed by defining new bindings for individual widgets or by redefining the class bindings.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

message

Keywords: message, widget

Name

message - Create and manipulate 'message' non-interactive text widgets

Synopsis

message *pathName* ?*options*?

Standard options

-anchor, **-background**, **-borderwidth**, **-cursor**, **-font**, **-foreground**, **-highlightbackground**, **-highlightcolor**, **-highlightthickness**, **-padx**, **-pady**, **-relief**, **-takefocus**, **-text**, **-textvariable**

See the options manual entry for details on the standard options.

Widget-specific options

-aspect (aspect/Aspect)

Specifies a non-negative integer value indicating desired aspect ratio for the text. The aspect ratio is specified as 100*width/height. 100 means the text should be as wide as it is tall, 200 means the text should be twice as wide as it is tall, 50 means the text should be twice as tall as it is wide, and so on. Used to choose line length for text if **-width** option is not specified. Defaults to 150.

-justify (justify/Justify)

Specifies how to justify lines of text. Must be one of **left**, **center**, or **right**. Defaults to **left**. This option works together with the **-anchor**, **-aspect**, **-padx**, **-pady**, and **-width** options to provide a variety of arrangements of the text within the window. The **-aspect** and **-width** options determine the amount of screen space needed to display the text. The **-anchor**, **-padx**, and **-pady** options determine where this rectangular area is displayed within the widget's window, and the **-justify** option determines how each line is displayed within that rectangular region. For example, suppose **-anchor** is **e** and **-justify** is **left**, and that the message window is much larger than needed for the text. The text will be displayed so that the left edges of all the lines line up and the right edge of the longest line is **-padx** from the right side of the window; the entire text block will be centered in the vertical span of the window.

-width (width/Width)

Specifies the length of lines in the window. The value may have any of the forms acceptable to Tk_GetPixels. If this option has a value greater than zero then the **-aspect** option is ignored and the **-width** option determines the line length. If this option has a value less than or equal to zero, then the **-aspect** option determines the line length.

Description

The **message** command creates a new window (given by the *pathName* argument) and makes it into a message widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the message such as its colors, font, text, and initial relief. The **message** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A message is a widget that displays a textual string. A message widget has three special features that differentiate it from a label widget. First, it breaks up its string into lines in order to produce a given aspect ratio for the window. The line breaks are chosen at word boundaries wherever possible (if not even a single word would fit on a line, then the word will be split across

lines). Newline characters in the string will force line breaks; they can be used, for example, to leave blank lines in the display.

The second feature of a message widget is justification. The text may be displayed left-justified (each line starts at the left side of the window), centered on a line-by-line basis, or right-justified (each line ends at the right side of the window).

The third feature of a message widget is that it handles control characters and non-printing characters specially. Tab characters are replaced with enough blank space to line up on the next 8-character boundary. Newlines cause line breaks. Other control characters (ASCII code less than 0x20) and characters not defined in the font are displayed as a four-character sequence **\x***hh* where *hh* is the two-digit hexadecimal number corresponding to the character. In the unusual case where the font does not contain all of the characters in "0123456789abcdef\x" then control characters and undefined characters are not displayed at all.

Widget command

The **message** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following commands are possible for message widgets:

pathName* cget *option
: Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **message** command.

***pathName* configure ?*option*? ?*value option value ...*?**
: Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **message** command.

Default bindings

When a new message is created, it has no default event bindings: messages are intended for output purposes only.

Bugs

Tabs do not work very well with text that is centered or right-justified. The most common result is that the line is justified wrong.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

messageBox

Keywords: messageBox, message box

Name

tk_messageBox - pops up a message window and waits for user response.

Synopsis

tk_messageBox ?*option value* ...?

Description

This procedure creates and displays a message window with an application-specified message, an icon and a set of buttons. Each of the buttons in the message window is identified by a unique symbolic name (see the **-type** options). After the message window is popped up, **tk_messageBox** waits for the user to select one of the buttons. Then it returns the symbolic name of the selected button.

The following option-value pairs are supported:

-command *string*
Specifies the prefix of a Tcl command to invoke when the user closes the dialog. The actual command consists of *string* followed by a space and the name of the button clicked by the user to close the dialog. This is only available on macOS.

-default *name*
Name gives the symbolic name of the default button for this message window ("ok", "cancel", and so on). See **-type** for a list of the symbolic names. If this option is not specified, the first button in the dialog will be made the default.

-detail *string*
Specifies an auxiliary message to the main message given by the **-message** option. The message detail will be presented beneath the main message and, where supported by the OS, in a less emphasized font than the main message.

-icon *iconImage*
Specifies an icon to display. *IconImage* must be one of the following: **error**, **info**, **question** or **warning**. If this option is not specified, then the info icon will be displayed.

-message *string*
Specifies the message to display in this message box. The default value is an empty string.

-parent *window*
Makes *window* the logical parent of the message box. The message box is displayed on top of its parent window.

-title *titleString*
Specifies a string to display as the title of the message box. The default value is an empty string.

-type *predefinedType*
Arranges for a predefined set of buttons to be displayed. The following values are possible for *predefinedType*:

abortretryignore
Displays three buttons whose symbolic names are **abort**, **retry** and **ignore**.

ok
Displays one button whose symbolic name is **ok**. This is the default.

okcancel
Displays two buttons whose symbolic names are **ok** and **cancel**.

retrycancel
Displays two buttons whose symbolic names are **retry** and **cancel**.

yesno
Displays two buttons whose symbolic names are **yes** and **no**.

yesnocancel
Displays three buttons whose symbolic names are **yes**, **no** and **cancel**.

Example

```
set answer [tk_messageBox -message "Really quit?" \
        -icon question -type yesno \
        -detail "Select \"Yes\" to make the application exit"]
switch -- $answer {
    yes exit
    no {tk_messageBox -message "I know you like this application!" \
            -type ok}
}
```

Copyright (c) 1996 Sun Microsystems, Inc.

option

Keywords: option, database, priority, retrieve

Name

option - Add/retrieve window options to/from the option database

Synopsis

option add *pattern value* ?*priority*?
option clear
option get *window name class*
option readfile *fileName* ?*priority*?

Description

The **option** command allows you to add entries to the Tk option database or to retrieve options from the database. The **add** form of the command adds a new option to the database. *Pattern* contains the option being specified, and consists of names and/or classes separated by asterisks or dots, in the usual X format (see Pattern format). *Value* contains a text string to associate with *pattern*; this is the value that will be returned in calls to Tk_GetOption or by invocations of the **option get** command. If *priority* is specified, it indicates the priority level for this option (see below for legal values); it defaults to **interactive**. This command always returns an empty string.

The **option clear** command clears the option database. Default options (from the **RESOURCE_MANAGER** property or the **.Xdefaults** file) will be reloaded automatically the next time an option is added to the database or removed from it. This command always returns an empty string.

The **option get** command returns the value of the option specified for *window* under *name* and *class*. If several entries in the option database match *window*, *name*, and *class*, then the

command returns whichever was created with highest *priority* level. If there are several matching entries at the same priority level, then it returns whichever entry was most recently entered into the option database. If there are no matching entries, then the empty string is returned.

The **readfile** form of the command reads *fileName*, which should have the standard format for an X resource database such as **.Xdefaults**, and adds all the options specified in that file to the option database. If *priority* is specified, it indicates the priority level at which to enter the options; *priority* defaults to **interactive**.

The file is read through a channel which is in "utf-8" encoding, invalid byte sequences are automatically converted to valid ones. This means that encodings like ISO 8859-1 or cp1252 with high probability will work as well, but this cannot be guaranteed. This cannot be changed, setting the [encoding system] has no effect.

The *priority* arguments to the **option** command are normally specified symbolically using one of the following values:

widgetDefault
: Level 20. Used for default values hard-coded into widgets.

startupFile
: Level 40. Used for options specified in application-specific startup files.

userDefault
: Level 60. Used for options specified in user-specific defaults files, such as **.Xdefaults**, resource databases loaded into the X server, or user-specific startup files.

interactive
: Level 80. Used for options specified interactively after the application starts running. If *priority* is not specified, it defaults to this level.

Any of the above keywords may be abbreviated. In addition, priorities may be specified numerically using integers between 0 and 100, inclusive. The numeric form is probably a bad idea except for new priority levels other than the ones given above.

Pattern format

Patterns consist of a sequence of words separated by either periods, ".", or asterisks "*". The overall pattern may also be optionally preceded by an asterisk.

Each word in the pattern conventionally starts with either an upper-case letter (in which case it denotes the class of either a widget or an option) or any other character, when it denotes the name of a widget or option. The last word in the pattern always indicates the option; the preceding ones constrain which widgets that option will be looked for in.

When two words are separated by a period, the latter widget must be a direct child of the former (or the option must apply to only the indicated widgets). When two words are separated by an asterisk, any depth of widgets may lie between the former and latter widgets (and the option applies to all widgets that are children of the former widget).

If the overall pattern is preceded by an asterisk, then the overall pattern applies anywhere it can throughout the whole widget hierarchy. Otherwise the first word of the pattern is matched against the name and class of the "**.**" toplevel, which are usually set by options to wish.

Examples

Instruct every button in the application to have red text on it unless explicitly overridden, by setting the **foreground** for the **Button** class (note that on some platforms the option is ignored):

```
option add *Button.foreground red startupFile
```

Allow users to control what happens in an entry widget when the Return key is pressed by specifying a script in the option database and add a default option for that which rings the bell:

```
entry .e
bind .e <Return> [option get .e returnCommand Command]
option add *.e.returnCommand bell widgetDefault
```

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

optionMenu

Keywords: optionMenu, option menu

Name

tk_optionMenu - Create an option menubutton and its menu

Synopsis

tk_optionMenu *pathName varName value* ?*value value* ...?

Description

This procedure creates an option menubutton whose name is *pathName*, plus an associated menu. Together they allow the user to select one of the values given by the *value* arguments. The current value will be stored in the global variable whose name is given by *varName* and it will also be displayed as the label in the option menubutton. The user can click on the menubutton to display a menu containing all of the *value*s and thereby select a new value. Once a new value is selected, it will be stored in the variable and appear in the option menubutton. The current value can also be changed by setting the variable.

The return value from **tk_optionMenu** is the name of the menu associated with *pathName*, so that the caller can change its configuration options or manipulate it in other ways.

Example

```
tk_optionMenu .foo myVar Foo Bar Boo Spong Wibble
pack .foo
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

options

Keywords: options, class, name, standard option, switch

Name

options - Standard options supported by widgets

Description

This manual entry describes the common configuration options supported by widgets in the Tk toolkit. Every widget does not necessarily support every option (see the manual entries for individual widgets for a list of the standard options supported by that widget), but if a widget does support an option with one of the names listed below, then the option has exactly the effect described below.

In the descriptions below, "Command-Line Name" refers to the switch used in class commands and **configure** widget commands to set this value. For example, if an option's command-line switch is **-foreground** and there exists a widget **.a.b.c**, then the command

```
.a.b.c  configure  -foreground black
```

may be used to specify the value **black** for the option in the widget **.a.b.c**. Command-line switches may be abbreviated, as long as the abbreviation is unambiguous. "Database Name" refers to the option's name in the option database (e.g. in .Xdefaults files). "Database Class" refers to the option's class value in the option database.

-activebackground (activeBackground/Foreground)
Specifies background color to use when drawing active elements. An element (a widget or portion of a widget) is active if the mouse cursor is positioned over the element and pressing a mouse button will cause some action to occur. If strict Motif compliance has been requested by setting the tk_strictMotif variable, this option will normally be ignored; the normal background color will be used instead. For some elements on Windows and Macintosh systems, the active color will only be used while mouse button 1 is pressed over the element.

-activeborderwidth (activeBorderWidth/BorderWidth)
Specifies a non-negative value indicating the width of the 3-D border drawn around active elements. See above for definition of active elements. The value may have any of the forms acceptable to Tk_GetPixels. This option is typically only available in widgets displaying more than one element at a time (e.g. menus but not buttons).

-activeforeground (activeForeground/Background)
Specifies foreground color to use when drawing active elements. See above for definition of active elements.

-anchor (anchor/Anchor)
Specifies how the information in a widget (e.g. text or a bitmap) is to be displayed in the widget. Must be one of the values **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, or **center**. For example, **nw** means display the information such that its top-left corner is at the top-left corner of the widget.

-background or -bg (background/Background)
Specifies the normal background color to use when displaying the widget.

-bitmap (bitmap/Bitmap)
Specifies a bitmap to display in the widget, in any of the forms acceptable to Tk_GetBitmap. The exact way in which the bitmap is displayed may be affected by other options such as **-anchor** or **-justify**. Typically, if this option is specified then it overrides other options that specify a textual value to display in the widget but this is controlled by the **-compound** option; the **-bitmap** option may be reset to an empty string to re-enable a text display. In widgets that support both **-bitmap** and **-image** options, **-image** will usually override **-bitmap**.

-borderwidth or -bd (borderWidth/BorderWidth)
Specifies a non-negative value indicating the width of the 3-D border to draw around the outside of the widget (if such a border is being drawn; the **-relief** option typically determines this). The value may also be used when drawing 3-D effects in the interior of the widget. The value may have any of the forms acceptable to Tk_GetPixels.

-cursor (cursor/Cursor)
Specifies the mouse cursor to be used for the widget. The value may have any of the forms acceptable to Tk_GetCursor. In addition, if an empty string is specified, it indicates that the widget should defer to its parent for cursor specification.

-compound (compound/Compound)
Specifies if the widget should display text and bitmaps/images at the same time, and if so, where the bitmap/image should be placed relative to the text. Must be one of the values **none**, **bottom**, **top**, **left**, **right**, or **center**. For example, the (default) value **none** specifies that the bitmap or image should (if defined) be displayed instead of the text, the value **left** specifies that the bitmap or image should be displayed to the left of the text, and the value **center** specifies that the bitmap or image should be displayed on top of the text.

-disabledforeground (disabledForeground/DisabledForeground)
Specifies foreground color to use when drawing a disabled element. If the option is specified as an empty string (which is typically the case on monochrome displays), disabled elements are drawn with the normal foreground color but they are dimmed by drawing them with a stippled fill pattern.

-exportselection (exportSelection/ExportSelection)
Specifies whether or not a selection in the widget should also be the X selection. The value may have any of the forms accepted by Tcl_GetBoolean, such as **true**, **false**, **0**, **1**, **yes**, or **no** . If the selection is exported, then selecting in the widget deselects the current X selection, selecting outside the widget deselects any widget selection, and the widget will respond to selection retrieval requests when it has a selection. The default is usually for widgets to export selections.

-font (font/Font)
Specifies the font to use when drawing text inside the widget. The value may have any of the forms described in the font manual page under **FONT DESCRIPTION**.

-foreground or -fg (foreground/Foreground)
Specifies the normal foreground color to use when displaying the widget.

-highlightbackground (highlightBackground/HighlightBackground)
Specifies the color to display in the traversal highlight region when the widget does not have the input focus.

-highlightcolor (highlightColor/HighlightColor)
Specifies the color to use for the traversal highlight rectangle that is drawn around the widget when it has the input focus.

-highlightthickness (highlightThickness/HighlightThickness)
Specifies a non-negative value indicating the width of the highlight rectangle to draw around the outside of the widget when it has the input focus. The value may have any of the forms acceptable to Tk_GetPixels. If the value is zero, no focus highlight is drawn around the widget.

-image (image/Image)
Specifies an image to display in the widget, which must have been created with the **image**

create command. Typically, if the **-image** option is specified then it overrides other options that specify a bitmap or textual value to display in the widget, though this is controlled by the **-compound** option; the **-image** option may be reset to an empty string to re-enable a bitmap or text display.

-insertbackground (insertBackground/Foreground)
Specifies the color to use as background in the area covered by the insertion cursor. This color will normally override either the normal background for the widget (or the selection background if the insertion cursor happens to fall in the selection).

-insertborderwidth (insertBorderWidth/BorderWidth)
Specifies a non-negative value indicating the width of the 3-D border to draw around the insertion cursor. The value may have any of the forms acceptable to Tk_GetPixels.

-insertofftime (insertOffTime/OffTime)
Specifies a non-negative integer value indicating the number of milliseconds the insertion cursor should remain "off" in each blink cycle. If this option is zero then the cursor does not blink: it is on all the time.

-insertontime (insertOnTime/OnTime)
Specifies a non-negative integer value indicating the number of milliseconds the insertion cursor should remain "on" in each blink cycle.

-insertwidth (insertWidth/InsertWidth)
Specifies a non-negative value indicating the total width of the insertion cursor. The value may have any of the forms acceptable to Tk_GetPixels. If a border has been specified for the insertion cursor (using the **-insertborderwidth** option), the border will be drawn inside the width specified by the **-insertwidth** option.

-jump (jump/Jump)
For widgets with a slider that can be dragged to adjust a value, such as scrollbars, this option determines when notifications are made about changes in the value. The option's value must be a boolean of the form accepted by Tcl_GetBoolean. If the value is false, updates are made continuously as the slider is dragged. If the value is true, updates are delayed until the mouse button is released to end the drag; at that point a single notification is made (the value "jumps" rather than changing smoothly).

-justify (justify/Justify)
When there are multiple lines of text displayed in a widget, this option determines how the lines line up with each other. Must be one of **left**, **center**, or **right**. **Left** means that the lines' left edges all line up, **center** means that the lines' centers are aligned, and **right** means that the lines' right edges line up.

-orient (orient/Orient)
For widgets that can lay themselves out with either a horizontal or vertical orientation, such as scrollbars, this option specifies which orientation should be used. Must be either **horizontal** or **vertical** or an abbreviation of one of these.

-padx (padX/Pad)
Specifies a non-negative value indicating how much extra space to request for the widget in the X-direction. The value may have any of the forms acceptable to Tk_GetPixels. When computing how large a window it needs, the widget will add this amount to the width it would normally need (as determined by the width of the things displayed in the widget); if the geometry manager can satisfy this request, the widget will end up with extra internal space to

the left and/or right of what it displays inside. Most widgets only use this option for padding text: if they are displaying a bitmap or image, then they usually ignore padding options.

-pady (padY/Pad)
Specifies a non-negative value indicating how much extra space to request for the widget in the Y-direction. The value may have any of the forms acceptable to Tk_GetPixels. When computing how large a window it needs, the widget will add this amount to the height it would normally need (as determined by the height of the things displayed in the widget); if the geometry manager can satisfy this request, the widget will end up with extra internal space above and/or below what it displays inside. Most widgets only use this option for padding text: if they are displaying a bitmap or image, then they usually ignore padding options.

-relief (relief/Relief)
Specifies the 3-D effect desired for the widget. Acceptable values are **raised**, **sunken**, **flat**, **ridge**, **solid**, and **groove**. The value indicates how the interior of the widget should appear relative to its exterior; for example, **raised** means the interior of the widget should appear to protrude from the screen, relative to the exterior of the widget.

-repeatdelay (repeatDelay/RepeatDelay)
Specifies the number of milliseconds a button or key must be held down before it begins to auto-repeat. Used, for example, on the up- and down-arrows in scrollbars.

-repeatinterval (repeatInterval/RepeatInterval)
Used in conjunction with **-repeatdelay**: once auto-repeat begins, this option determines the number of milliseconds between auto-repeats.

-selectbackground (selectBackground/Foreground)
Specifies the background color to use when displaying selected items.

-selectborderwidth (selectBorderWidth/BorderWidth)
Specifies a non-negative value indicating the width of the 3-D border to draw around selected items. The value may have any of the forms acceptable to Tk_GetPixels.

-selectforeground (selectForeground/Background)
Specifies the foreground color to use when displaying selected items.

-setgrid (setGrid/SetGrid)
Specifies a boolean value that determines whether this widget controls the resizing grid for its top-level window. This option is typically used in text widgets, where the information in the widget has a natural size (the size of a character) and it makes sense for the window's dimensions to be integral numbers of these units. These natural window sizes form a grid. If the **-setgrid** option is set to true then the widget will communicate with the window manager so that when the user interactively resizes the top-level window that contains the widget, the dimensions of the window will be displayed to the user in grid units and the window size will be constrained to integral numbers of grid units. See the section **GRIDDED GEOMETRY MANAGEMENT** in the wm manual entry for more details.

-takefocus (takeFocus/TakeFocus)
Determines whether the window accepts the focus during keyboard traversal (e.g., Tab and Shift-Tab). Before setting the focus to a window, the traversal scripts consult the value of the **-takefocus** option. A value of **0** means that the window should be skipped entirely during keyboard traversal. **1** means that the window should receive the input focus as long as it is viewable (it and all of its ancestors are mapped). An empty value for the option means that

the traversal scripts make the decision about whether or not to focus on the window: the current algorithm is to skip the window if it is disabled, if it has no key bindings, or if it is not viewable. If the value has any other form, then the traversal scripts take the value, append the name of the window to it (with a separator space), and evaluate the resulting string as a Tcl script. The script must return **0**, **1**, or an empty string: a **0** or **1** value specifies whether the window will receive the input focus, and an empty string results in the default decision described above. Note: this interpretation of the option is defined entirely by the Tcl scripts that implement traversal: the widget implementations ignore the option entirely, so you can change its meaning if you redefine the keyboard traversal scripts.

-text (text**/Text)**
Specifies a string to be displayed inside the widget. The way in which the string is displayed depends on the particular widget and may be determined by other options, such as **-anchor** or **-justify**.

-textvariable (textVariable/Variable)
Specifies the name of a global variable. The value of the variable is a text string to be displayed inside the widget; if the variable value changes then the widget will automatically update itself to reflect the new value. The way in which the string is displayed in the widget depends on the particular widget and may be determined by other options, such as **-anchor** or **-justify**.

-troughcolor (troughColor/Background)
Specifies the color to use for the rectangular trough areas in widgets such as scrollbars and scales. This option is ignored for scrollbars on Windows (native widget does not recognize this option).

-underline (underline/Underline)
Specifies the integer index of a character to underline in the widget. This option is used by the default bindings to implement keyboard traversal for menu buttons and menu entries. 0 corresponds to the first character of the text displayed in the widget, 1 to the next character, and so on.

-wraplength (wrapLength/WrapLength)
For widgets that can perform word-wrapping, this option specifies the maximum line length. Lines that would exceed this length are wrapped onto the next line, so that no line is longer than the specified length. The value may be specified in any of the standard forms for screen distances. If this value is less than or equal to 0 then no wrapping is done: lines will break only at newline characters in the text.

-xscrollcommand (xScrollCommand/ScrollCommand)
Specifies the prefix for a command used to communicate with horizontal scrollbars. When the view in the widget's window changes (or whenever anything else occurs that could change the display in a scrollbar, such as a change in the total size of the widget's contents), the widget will generate a Tcl command by concatenating the scroll command and two numbers. Each of the numbers is a fraction between 0 and 1, which indicates a position in the document. 0 indicates the beginning of the document, 1 indicates the end, .333 indicates a position one third the way through the document, and so on. The first fraction indicates the first information in the document that is visible in the window, and the second fraction indicates the information just after the last portion that is visible. The command is then passed to the Tcl interpreter for execution. Typically the **-xscrollcommand** option consists of

the path name of a scrollbar widget followed by "set", e.g. ".x.scrollbar set": this will cause the scrollbar to be updated whenever the view in the window changes. If this option is not specified, then no command will be executed.

-yscrollcommand (yScrollCommand/ScrollCommand)
Specifies the prefix for a command used to communicate with vertical scrollbars. This option is treated in the same way as the **-xscrollcommand** option, except that it is used for vertical scrollbars and is provided by widgets that support vertical scrolling. See the description of **-xscrollcommand** for details on how this option is used.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

pack-old

Keywords: pack-old, geometry manager, location, packer, parcel, size

Name

pack-old - Obsolete syntax for packer geometry manager

Synopsis

pack after *sibling window options* ?*window options* ...?
pack append *parent window options* ?*window options* ...?
pack before *sibling window options* ?*window options* ...?
pack unpack *window*

Description

Note: this manual entry describes the syntax for the pack *command as it existed before Tk version 3.3. Although this syntax continues to be supported for backward compatibility, it is obsolete and should not be used anymore. At some point in the future it may cease to be supported.*

The packer is a geometry manager that arranges the children of a parent by packing them in order around the edges of the parent. The first child is placed against one side of the window, occupying the entire span of the window along that side. This reduces the space remaining for other children as if the side had been moved in by the size of the first child. Then the next child is placed against one side of the remaining cavity, and so on until all children have been placed or there is no space left in the cavity.

The **before**, **after**, and **append** forms of the pack command are used to insert one or more children into the packing order for their parent. The **before** form inserts the children before window *sibling* in the order; all of the other windows must be siblings of *sibling*. The **after** form inserts the windows after *sibling*, and the **append** form appends one or more windows to the end of the packing order for *parent*. If a *window* named in any of these commands is already packed in its parent, it is removed from its current position in the packing order and repositioned as indicated by the command. All of these commands return an empty string as result.

The **unpack** form of the pack command removes *window* from the packing order of its parent and unmaps it. After the execution of this command the packer will no longer manage *window*'s geometry.

The placement of each child is actually a four-step process; the *options* argument following each

window consists of a list of one or more fields that govern the placement of that window. In the discussion below, the term *cavity* refers to the space left in a parent when a particular child is placed (i.e. all the space that was not claimed by earlier children in the packing order). The term *parcel* refers to the space allocated to a particular child; this is not necessarily the same as the child window's final geometry.

The first step in placing a child is to determine which side of the cavity it will lie against. Any one of the following options may be used to specify a side:

top
Position the child's parcel against the top of the cavity, occupying the full width of the cavity.

bottom
Position the child's parcel against the bottom of the cavity, occupying the full width of the cavity.

left
Position the child's parcel against the left side of the cavity, occupying the full height of the cavity.

right
Position the child's parcel against the right side of the cavity, occupying the full height of the cavity.

At most one of these options should be specified for any given window. If no side is specified, then the default is **top**.

The second step is to decide on a parcel for the child. For **top** and **bottom** windows, the desired parcel width is normally the cavity width and the desired parcel height is the window's requested height, as passed to Tk_GeometryRequest. For **left** and **right** windows, the desired parcel height is normally the cavity height and the desired width is the window's requested width. However, extra space may be requested for the window using any of the following options:

padx *num*
Add *num* pixels to the window's requested width before computing the parcel size as described above.

pady *num*
Add *num* pixels to the window's requested height before computing the parcel size as described above.

expand
This option requests that the window's parcel absorb any extra space left over in the parent's cavity after packing all the children. The amount of space left over depends on the sizes requested by the other children, and may be zero. If several windows have all specified **expand** then the extra width will be divided equally among all the **left** and **right** windows that specified **expand** and the extra height will be divided equally among all the **top** and **bottom** windows that specified **expand**.

If the desired width or height for a parcel is larger than the corresponding dimension of the cavity, then the cavity's dimension is used instead.

The third step in placing the window is to decide on the window's width and height. The default is for the window to receive either its requested width and height or the those of the parcel, whichever is smaller. If the parcel is larger than the window's requested size, then the following

options may be used to expand the window to partially or completely fill the parcel:

fill
Set the window's size to equal the parcel size.

fillx
Increase the window's width to equal the parcel's width, but retain the window's requested height.

filly
Increase the window's height to equal the parcel's height, but retain the window's requested width.

The last step is to decide the window's location within its parcel. If the window's size equals the parcel's size, then the window simply fills the entire parcel. If the parcel is larger than the window, then one of the following options may be used to specify where the window should be positioned within its parcel:

frame center
Center the window in its parcel. This is the default if no framing option is specified.

frame n
Position the window with its top edge centered on the top edge of the parcel.

frame ne
Position the window with its upper-right corner at the upper-right corner of the parcel.

frame e
Position the window with its right edge centered on the right edge of the parcel.

frame se
Position the window with its lower-right corner at the lower-right corner of the parcel.

frame s
Position the window with its bottom edge centered on the bottom edge of the parcel.

frame sw
Position the window with its lower-left corner at the lower-left corner of the parcel.

frame w
Position the window with its left edge centered on the left edge of the parcel.

frame nw
Position the window with its upper-left corner at the upper-left corner of the parcel.

The packer manages the mapped/unmapped state of all the packed children windows. It automatically maps the windows when it packs them, and it unmaps any windows for which there was no space left in the cavity.

The packer makes geometry requests on behalf of the parent windows it manages. For each parent window it requests a size large enough to accommodate all the options specified by all the packed children, such that zero space would be leftover for **expand** options.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

pack

Keywords: pack, geometry manager, location, packer, parcel, propagation, size

Name

pack - Geometry manager that packs around edges of cavity

Synopsis

pack *option arg* ?*arg* ...?

Description

The **pack** command is used to communicate with the packer, a geometry manager that arranges the children of a parent by packing them in order around the edges of the parent. The **pack** command can have any of several forms, depending on the *option* argument:

pack *window* ?*window ...*? ?*options*?

If the first argument to **pack** is a window name (any value starting with "."), then the command is processed in the same way as **pack configure**.

pack configure *window* ?*window ...*? ?*options*?

The arguments consist of the names of one or more content windows followed by pairs of arguments that specify how to manage the content. See The packer algorithm below for details on how the options are used by the packer. The following options are supported:

-after *other*

Other must the name of another window. Use its container as the container for the content, and insert the content just after *other* in the packing order.

-anchor *anchor*

Anchor must be a valid anchor position such as **n** or **sw**; it specifies where to position each content in its parcel. Defaults to **center**.

-before *other*

Other must the name of another window. Use its container as the container for the content, and insert the content just before *other* in the packing order.

-expand *boolean*

Specifies whether the content should be expanded to consume extra space in their container. *Boolean* may have any proper boolean value, such as **1** or **no**. Defaults to 0.

-fill *style*

If a content's parcel is larger than its requested dimensions, this option may be used to stretch the content. *Style* must have one of the following values:

none

Give the content its requested dimensions plus any internal padding requested with **-ipadx** or **-ipady**. This is the default.

x

Stretch the content horizontally to fill the entire width of its parcel (except leave external padding as specified by **-padx**).

y

Stretch the content vertically to fill the entire height of its parcel (except leave external padding as specified by **-pady**).

both

Stretch the content both horizontally and vertically.

-in *container*

Insert the window at the end of the packing order for the container window given by *container*.

-ipadx *amount*
Amount specifies how much horizontal internal padding to leave on each side of the content. *Amount* must be a valid screen distance, such as **2** or **.5c**. It defaults to 0.

-ipady *amount*
Amount specifies how much vertical internal padding to leave on each side of the content. *Amount* defaults to 0.

-padx *amount*
Amount specifies how much horizontal external padding to leave on each side of the content. *Amount* may be a list of two values to specify padding for left and right separately. *Amount* defaults to 0.

-pady *amount*
Amount specifies how much vertical external padding to leave on each side of the content. *Amount* may be a list of two values to specify padding for top and bottom separately. *Amount* defaults to 0.

-side *side*
Specifies which side of the container the content will be packed against. Must be **left**, **right**, **top**, or **bottom**. Defaults to **top**.

If no **-in**, **-after** or **-before** option is specified then each of the content will be inserted at the end of the packing list for its parent unless it is already managed by the packer (in which case it will be left where it is). If one of these options is specified then all the content will be inserted at the specified point. If any of the content are already managed by the geometry manager then any unspecified options for them retain their previous values rather than receiving default values.

pack forget *window* ?*window ...*?
Removes each of the *window*s from the packing order for its container and unmaps their windows. The content will no longer be managed by the packer.

pack info *window*
Returns a list whose elements are the current configuration state of the window given by *window* in the same option-value form that might be specified to **pack configure**. The first two elements of the list are "**-in** *container*" where *container* is the window's container.

pack propagate *container* ?*boolean*?
If *boolean* has a true boolean value such as **1** or **on** then propagation is enabled for *container*, which must be a window name (see Geometry propagation below). If *boolean* has a false boolean value then propagation is disabled for *container*. In either of these cases an empty string is returned. If *boolean* is omitted then the command returns **0** or **1** to indicate whether propagation is currently enabled for *container*. Propagation is enabled by default.

pack slaves *window*
Returns a list of all of the content windows in the packing order for *window*. The order of the content windows in the list is the same as their order in the packing order. If *window* has no content then an empty string is returned.

pack content window
Synonym for pack slaves window.

The packer algorithm

For each container the packer maintains an ordered list of content windows called the *packing list*. The **-in**, **-after**, and **-before** configuration options are used to specify the container for each content and the content's position in the packing list. If none of these options is given for a content then the content is added to the end of the packing list for its parent.

The packer arranges the content windows for a container by scanning the packing list in order. At the time it processes each content, a rectangular area within the container is still unallocated. This area is called the *cavity*; for the first content it is the entire area of the container.

For each content the packer carries out the following steps:

1. The packer allocates a rectangular *parcel* for the content along the side of the cavity given by the content's **-side** option. If the side is top or bottom then the width of the parcel is the width of the cavity and its height is the requested height of the content plus the **-ipady** and **-pady** options. For the left or right side the height of the parcel is the height of the cavity and the width is the requested width of the content plus the **-ipadx** and **-padx** options. The parcel may be enlarged further because of the **-expand** option (see Expansion below)
2. The packer chooses the dimensions of the content. The width will normally be the content's requested width plus twice its **-ipadx** option and the height will normally be the content's requested height plus twice its **-ipady** option. However, if the **-fill** option is **x** or **both** then the width of the content is expanded to fill the width of the parcel, minus twice the **-padx** option. If the **-fill** option is **y** or **both** then the height of the content is expanded to fill the height of the parcel, minus twice the **-pady** option.
3. The packer positions the content over its parcel. If the content is smaller than the parcel then the **-anchor** option determines where in the parcel the content will be placed. If **-padx** or **-pady** is non-zero, then the given amount of external padding will always be left between the content and the edges of the parcel.

Once a given content has been packed, the area of its parcel is subtracted from the cavity, leaving a smaller rectangular cavity for the next content. If a content does not use all of its parcel, the unused space in the parcel will not be used by subsequent content. If the cavity should become too small to meet the needs of a content then the content will be given whatever space is left in the cavity. If the cavity shrinks to zero size, then all remaining content on the packing list will be unmapped from the screen until the container window becomes large enough to hold them again.

Expansion

If a container window is so large that there will be extra space left over after all of its content have been packed, then the extra space is distributed uniformly among all of the content for which the **-expand** option is set. Extra horizontal space is distributed among the expandable content whose **-side** is **left** or **right**, and extra vertical space is distributed among the expandable content whose **-side** is **top** or **bottom**.

Geometry propagation

The packer normally computes how large a container must be to just exactly meet the needs of its content, and it sets the requested width and height of the container to these dimensions. This causes geometry information to propagate up through a window hierarchy to a top-level window so that the entire sub-tree sizes itself to fit the needs of the leaf windows. However, the **pack**

propagate command may be used to turn off propagation for one or more containers. If propagation is disabled then the packer will not set the requested width and height of the packer. This may be useful if, for example, you wish for a container window to have a fixed size that you specify.

Restrictions on container windows

The container for each content must either be the content's parent (the default) or a descendant of the content's parent. This restriction is necessary to guarantee that the content can be placed over any part of its container that is visible without danger of the content being clipped by its parent.

Packing order

If the container for a content is not its parent then you must make sure that the content is higher in the stacking order than the container. Otherwise the container will obscure the content and it will appear as if the content has not been packed correctly. The easiest way to make sure the content is higher than the container is to create the container window first: the most recently created window will be highest in the stacking order. Or, you can use the raise and lower commands to change the stacking order of either the container or the content.

Example

```
# Make the widgets
label .t -text "This widget is at the top"    -bg red
label .b -text "This widget is at the bottom" -bg green
label .l -text "Left\nHand\nSide"
label .r -text "Right\nHand\nSide"
text .mid
.mid insert end "This layout is like Java's BorderLayout"
# Lay them out
pack .t   -side top    -fill x
pack .b   -side bottom -fill x
pack .l   -side left   -fill y
pack .r   -side right  -fill y
pack .mid -expand 1    -fill both
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

palette

Keywords: palette, bisque, color

Name

tk_setPalette, tk_bisque - Modify the Tk color palette

Synopsis

tk_setPalette *background*
tk_setPalette *name value* ?*name value* ...?
tk_bisque

Description

The **tk_setPalette** procedure changes the color scheme for Tk. It does this by modifying the colors of existing widgets and by changing the option database so that future widgets will use the new color scheme. If **tk_setPalette** is invoked with a single argument, the argument is the name of a color to use as the normal background color; **tk_setPalette** will compute a complete color palette from this background color. Alternatively, the arguments to **tk_setPalette** may consist of any number of *name-value* pairs, where the first argument of the pair is the name of an option in the Tk option database and the second argument is the new value to use for that option. The following database names are currently supported:

activeBackground foreground selectColor
activeForeground highlightBackground selectBackground
background highlightColor selectForeground
disabledForeground insertBackground troughColor

tk_setPalette tries to compute reasonable defaults for any options that you do not specify. You can specify options other than the above ones and Tk will change those options on widgets as well. This feature may be useful if you are using custom widgets with additional color options.

Once it has computed the new value to use for each of the color options, **tk_setPalette** scans the widget hierarchy to modify the options of all existing widgets. For each widget, it checks to see if any of the above options is defined for the widget. If so, and if the option's current value is the default, then the value is changed; if the option has a value other than the default, **tk_setPalette** will not change it. The default for an option is the one provided by the widget (**[lindex [$w configure $option] 3]**) unless **tk_setPalette** has been run previously, in which case it is the value specified in the previous invocation of **tk_setPalette**.

After modifying all the widgets in the application, **tk_setPalette** adds options to the option database to change the defaults for widgets created in the future. The new options are added at priority **widgetDefault**, so they will be overridden by options from the .Xdefaults file or options specified on the command-line that creates a widget.

The procedure **tk_bisque** is provided for backward compatibility: it restores the application's colors to the light brown ("bisque") color scheme used in Tk 3.6 and earlier versions.

Copyright (c) 1995-1996 Sun Microsystems, Inc.

panedwindow

Keywords: panedwindow, widget, geometry management

Name

panedwindow - Create and manipulate 'panedwindow' split container widgets

Synopsis

panedwindow *pathName* ?*options*?

Standard options

-background, **-borderwidth**, **-cursor**, **-orient**, **-relief**

See the options manual entry for details on the standard options.

Widget-specific options

-handlepad (handlePad/HandlePad)
When sash handles are drawn, specifies the distance from the top or left end of the sash (depending on the orientation of the widget) at which to draw the handle. May be any value accepted by Tk_GetPixels.

-handlesize (handleSize/HandleSize)
Specifies the side length of a sash handle. Handles are always drawn as squares. May be any value accepted by Tk_GetPixels.

-height (height/Height)
Specifies a desired height for the overall panedwindow widget. May be any value accepted by Tk_GetPixels. If an empty string, the widget will be made high enough to allow all contained widgets to have their natural height.

-opaqueresize (opaqueResize/OpaqueResize)
Specifies whether panes should be resized as a sash is moved (true), or if resizing should be deferred until the sash is placed (false). In the latter case, a "ghost" version of the sash is displayed during the resizing to show where the panes will be resized to when releasing the mouse button. This "ghost" version of the sash is the proxy. It's rendering can be configured using the **-proxybackground**, **-proxyborderwidth** and **-proxyrelief** options.

-proxybackground (proxyBackground/ProxyBackground)
Background color to use when drawing the proxy. If an empty string, the value of the **-background** option will be used.

-proxyborderwidth (proxyBorderWidth/ProxyBorderWidth)
Specifies the borderwidth of the proxy. May be any value accepted by Tk_GetPixels.

-proxyrelief (proxyRelief/ProxyRelief)
Relief to use when drawing the proxy. May be any of the standard Tk relief values. If an empty string, the value of the **-sashrelief** option will be used.

-sashcursor (sashCursor/SashCursor)
Mouse cursor to use when over a sash. If null, **sb_h_double_arrow** will be used for horizontal panedwindows, and **sb_v_double_arrow** will be used for vertical panedwindows.

-sashpad (sashPad/SashPad)
Specifies the amount of padding to leave of each side of a sash. May be any value accepted by Tk_GetPixels.

-sashrelief (sashRelief/SashRelief)
Relief to use when drawing a sash. May be any of the standard Tk relief values.

-sashwidth (sashWidth/SashWidth)
Specifies the width of each sash. May be any value accepted by Tk_GetPixels.

-showhandle (showHandle/ShowHandle)
Specifies whether sash handles should be shown. May be any valid Tcl boolean value.

-width (width/Width)
Specifies a desired width for the overall panedwindow widget. May be any value accepted by Tk_GetPixels. If an empty string, the widget will be made wide enough to allow all contained widgets to have their natural width.

Description

The **panedwindow** command creates a new window (given by the *pathName* argument) and makes it into a panedwindow widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the panedwindow such as its default background color and relief. The **panedwindow** command returns the path name of the new window.

A panedwindow widget contains any number of panes, arranged horizontally or vertically, according to the value of the **-orient** option. Each pane contains one widget, and each pair of panes is separated by a moveable (via mouse movements) sash. Moving a sash causes the widgets on either side of the sash to be resized.

Widget command

The **panedwindow** command creates a new Tcl command whose name is the same as the path name of the panedwindow's window. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

PathName is the name of the command, which is the same as the panedwindow widget's path name. *Option* and the *arg*s determine the exact behavior of the command. The following commands are possible for panedwindow widgets:

***pathName* add *window* ?*window ...*? ?*option value ...*?**
Add one or more windows to the panedwindow, each in a separate pane. The arguments consist of the names of one or more windows followed by pairs of arguments that specify how to manage the windows. *Option* may have any of the values accepted by the **configure** subcommand.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **panedwindow** command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **panedwindow** command.

***pathName* forget *window* ?*window ...*?**
Remove the pane containing *window* from the panedwindow. All geometry management options for *window* will be forgotten.

pathName* identify *x y
Identify the panedwindow component underneath the point given by *x* and *y*, in window coordinates. If the point is over a sash or a sash handle, the result is a two element list containing the index of the sash or handle, and a word indicating whether it is over a sash or a handle, such as {0 sash} or {2 handle}. If the point is over any other part of the panedwindow, the result is an empty list.

pathName* panecget *window option
Query a management option for *window*. *Option* may be any value allowed by the **paneconfigure** subcommand.

***pathName* paneconfigure *window* ?*option*? ?*value option value ...*?**
Query or modify the management options for *window*. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. The following options are supported:

-after *window*
Insert the window after the window specified. *window* should be the name of a window already managed by *pathName*.

-before *window*
Insert the window before the window specified. *window* should be the name of a window already managed by *pathName*.

-height *size*
Specify a height for the window. The height will be the outer dimension of the window including its border, if any. If *size* is an empty string, or if **-height** is not specified, then the height requested internally by the window will be used initially; the height may later be adjusted by the movement of sashes in the panedwindow. *Size* may be any value accepted by Tk_GetPixels.

-hide *boolean*
Controls the visibility of a pane. When the *boolean* is true (according to Tcl_GetBoolean) the pane will not be visible, but it will still be maintained in the list of panes.

-minsize *n*
Specifies that the size of the window cannot be made less than *n*. This constraint only affects the size of the widget in the paned dimension — the x dimension for horizontal panedwindows, the y dimension for vertical panedwindows. May be any value accepted by Tk_GetPixels.

-padx *n*
Specifies a non-negative value indicating how much extra space to leave on each side of the window in the X-direction. The value may have any of the forms accepted by Tk_GetPixels.

-pady *n*
Specifies a non-negative value indicating how much extra space to leave on each side of the window in the Y-direction. The value may have any of the forms accepted by Tk_GetPixels.

-sticky *style*
If a window's pane is larger than the requested dimensions of the window, this option may be used to position (or stretch) the window within its pane. *Style* is a string that contains zero or more of the characters **n**, **s**, **e** or **w**. The string can optionally contains spaces or commas, but they are ignored. Each letter refers to a side (north, south, east, or west) that the window will "stick" to. If both **n** and **s** (or **e** and **w**) are specified, the window will be

stretched to fill the entire height (or width) of its cavity.

-stretch *when*
Controls how extra space is allocated to each of the panes. *When* is one of **always**, **first**, **last**, **middle**, and **never**. The panedwindow will calculate the required size of all its panes. Any remaining (or deficit) space will be distributed to those panes marked for stretching. The space will be distributed based on each panes current ratio of the whole. The *when* values have the following definition:

always
This pane will always stretch.

first
Only if this pane is the first pane (left-most or top-most) will it stretch.

last
Only if this pane is the last pane (right-most or bottom-most) will it stretch. This is the default value.

middle
Only if this pane is not the first or last pane will it stretch.

never
This pane will never stretch.

-width *size*
Specify a width for the window. The width will be the outer dimension of the window including its border, if any. If *size* is an empty string, or if **-width** is not specified, then the width requested internally by the window will be used initially; the width may later be adjusted by the movement of sashes in the panedwindow. *Size* may be any value accepted by Tk_GetPixels.

***pathName* panes**
Returns an ordered list of the widgets managed by *pathName*.

***pathName* proxy ?*args*?**
This command is used to query and change the position of the sash proxy, used for rubberband-style pane resizing. It can take any of the following forms:

***pathName* proxy coord**
Return a list containing the x and y coordinates of the most recent proxy location.

***pathName* proxy forget**
Remove the proxy from the display.

pathName* proxy place *x y
Place the proxy at the given *x* and *y* coordinates.

***pathName* sash ?*args*?**
This command is used to query and change the position of sashes in the panedwindow. It can take any of the following forms:

pathName* sash coord *index
Return the current x and y coordinate pair for the sash given by *index*. *Index* must be an integer between 0 and 1 less than the number of panes in the panedwindow. The coordinates given are those of the top left corner of the region containing the sash.

pathName* sash dragto *index x y

This command computes the difference between the given coordinates and the coordinates given to the last **sash mark** command for the given sash. It then moves that sash the computed difference. The return value is the empty string.

pathName* sash mark *index x y
Records *x* and *y* for the sash given by *index*; used in conjunction with later **sash dragto** commands to move the sash.

pathName* sash place *index x y
Place the sash given by *index* at the given coordinates.

Resizing panes

A pane is resized by grabbing the sash (or sash handle if present) and dragging with the mouse. This is accomplished via mouse motion bindings on the widget. When a sash is moved, the sizes of the panes on each side of the sash, and thus the widgets in those panes, are adjusted.

When a pane is resized from outside (e.g. it is packed to expand and fill, and the containing toplevel is resized), space is added to the final (rightmost or bottommost) pane in the window.

Unlike child windows managed by e.g. pack or grid, the panes managed by a panedwindow do not change width or height to accommodate changes in the requested widths or heights of the panes, once these have become mapped. Therefore it may be advisable, particularly when creating layouts interactively, to not add a pane to the panedwindow widget until after the geometry requests of that pane has been finalized (i.e., all components of the pane inserted, all options affecting geometry set to their proper values, etc.).

Copyright (c) 1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

photo

Keywords: photo, image, color

Name

photo - Full-color images

Synopsis

image create photo ?*name*? ?*options*?

imageName **blank**
imageName **cget** *option*
imageName **configure** ?*option*? ?*value option value* ...?
imageName **copy** *sourceImage* ?*option value(s)* ...?
imageName **data** ?*option value(s)* ...?
imageName **get** *x y*
imageName **put** *data* ?*option value(s)* ...?
imageName **read** *filename* ?*option value(s)* ...?
imageName **redither**
imageName **transparency** *subcommand* ?*arg arg* ...?
imageName **write** *filename* ?*option value(s)* ...?

Description

A photo is an image whose pixels can display any color or be transparent. A photo image is stored internally in full color (32 bits per pixel), and is displayed using dithering if necessary. Image data for a photo image can be obtained from a file or a string, or it can be supplied from C code through a procedural interface. At present, only PNG, GIF and PPM/PGM formats are supported, but an interface exists to allow additional image file formats to be added easily. A photo image is transparent in regions where no image data has been supplied or where it has been set transparent by the **transparency set** subcommand.

Creating photos

Like all images, photos are created using the image create command. Photos support the following *options*:

-data *string*
Specifies the contents of the image as a string. The string should contain binary data or, for some formats, base64-encoded data (this is currently guaranteed to be supported for PNG and GIF images). The format of the string must be one of those for which there is an image file format handler that will accept string data. If both the **-data** and **-file** options are specified, the **-file** option takes precedence.

-format *format-name*
Specifies the name of the file format for the data specified with the **-data** or **-file** option.

-file *name*
name gives the name of a file that is to be read to supply data for the photo image. The file format must be one of those for which there is an image file format handler that can read data.

-gamma *value*
Specifies that the colors allocated for displaying this image in a window should be corrected for a non-linear display with the specified gamma exponent value. (The intensity produced by most CRT displays is a power function of the input value, to a good approximation; gamma is the exponent and is typically around 2). The value specified must be greater than zero. The default value is one (no correction). In general, values greater than one will make the image lighter, and values less than one will make it darker.

-height *number*
Specifies the height of the image, in pixels. This option is useful primarily in situations where the user wishes to build up the contents of the image piece by piece. A value of zero (the default) allows the image to expand or shrink vertically to fit the data stored in it.

-palette *palette-spec*
Specifies the resolution of the color cube to be allocated for displaying this image, and thus the number of colors used from the colormaps of the windows where it is displayed. The *palette-spec* string may be either a single decimal number, specifying the number of shades of gray to use, or three decimal numbers separated by slashes (/), specifying the number of shades of red, green and blue to use, respectively. If the first form (a single number) is used, the image will be displayed in monochrome (i.e., grayscale).

-width *number*
Specifies the width of the image, in pixels. This option is useful primarily in situations where the user wishes to build up the contents of the image piece by piece. A value of zero (the default) allows the image to expand or shrink horizontally to fit the data stored in it.

Image command

When a photo image is created, Tk also creates a new command whose name is the same as the image. This command may be used to invoke various operations on the image. It has the following general form:

```
imageName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command.

Those options that write data to the image generally expand the size of the image, if necessary, to accommodate the data written to the image, unless the user has specified non-zero values for the **-width** and/or **-height** configuration options, in which case the width and/or height, respectively, of the image will not be changed.

The following commands are possible for photo images:

***imageName* blank**
Blank the image; that is, set the entire image to have no data, so it will be displayed as transparent, and the background of whatever window it is displayed in will show through.

imageName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the image create **photo** command.

***imageName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options for the image. If no *option* is specified, returns a list describing all of the available options for *imageName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the image create **photo** command.

***imageName* copy *sourceImage* ?*option value(s) ...*?**
Copies a region from the image called *sourceImage* (which must be a photo image) to the image called *imageName*, possibly with pixel zooming and/or subsampling. If no options are specified, this command copies the whole of *sourceImage* into *imageName*, starting at coordinates (0,0) in *imageName*. The following options may be specified:

-from *x1 y1 x2 y2*
Specifies a rectangular sub-region of the source image to be copied. (*x1,y1*) and (*x2,y2*) specify diagonally opposite corners of the rectangle. If *x2* and *y2* are not specified, the default value is the bottom-right corner of the source image. The pixels copied will include the left and top edges of the specified rectangle but not the bottom or right edges. If the **-from** option is not given, the default is the whole source image.

-to *x1 y1 x2 y2*
Specifies a rectangular sub-region of the destination image to be affected. (*x1,y1*) and (*x2,y2*) specify diagonally opposite corners of the rectangle. If *x2* and *y2* are not specified, the default value is (*x1,y1*) plus the size of the source region (after subsampling and zooming, if specified). If *x2* and *y2* are specified, the source region will be replicated if necessary to fill the destination region in a tiled fashion.

-shrink
Specifies that the size of the destination image should be reduced, if necessary, so that the region being copied into is at the bottom-right corner of the image. This option will not affect the width or height of the image if the user has specified a non-zero value for the **-width** or **-height** configuration option, respectively.

-zoom *x y*
Specifies that the source region should be magnified by a factor of *x* in the X direction and *y* in the Y direction. If *y* is not given, the default value is the same as *x*. With this option, each pixel in the source image will be expanded into a block of *x* x *y* pixels in the destination image, all the same color. *x* and *y* must be greater than 0.

-subsample *x y*
Specifies that the source image should be reduced in size by using only every *x*th pixel in the X direction and *y*th pixel in the Y direction. Negative values will cause the image to be flipped about the Y or X axes, respectively. If *y* is not given, the default value is the same as *x*.

-compositingrule *rule*
Specifies how transparent pixels in the source image are combined with the destination image. When a compositing rule of *overlay* is set, the old contents of the destination image are visible, as if the source image were printed on a piece of transparent film and placed over the top of the destination. When a compositing rule of *set* is set, the old contents of the destination image are discarded and the source image is used as-is. The default compositing rule is *overlay*.

***imageName* data ?*option value(s) ...*?**
Returns image data in the form of a string. The following options may be specified:

-background *color*
If the color is specified, the data will not contain any transparency information. In all transparent pixels the color will be replaced by the specified color.

-format *format-name*
Specifies the name of the image file format handler to be used. Specifically, this subcommand searches for the first handler whose name matches an initial substring of *format-name* and which has the capability to write a string containing this image data. If this option is not given, this subcommand uses a format that consists of a list (one element per row) of lists (one element per pixel/column) of colors in "**#***rrggbb*" format (where *rr* is a pair of hexadecimal digits for the red channel, *gg* for green, and *bb* for blue).

-from *x1 y1 x2 y2*
Specifies a rectangular region of *imageName* to be returned. If only *x1* and *y1* are specified, the region extends from *(x1,y1)* to the bottom-right corner of *imageName*. If all four coordinates are given, they specify diagonally opposite corners of the rectangular region, including x1,y1 and excluding x2,y2. The default, if this option is not given, is the whole image.

-grayscale
If this options is specified, the data will not contain color information. All pixel data will be transformed into grayscale.

imageName* get *x y

Returns the color of the pixel at coordinates (*x*,*y*) in the image as a list of three integers between 0 and 255, representing the red, green and blue components respectively.

***imageName* put *data* ?*option value(s) ...*?**

Sets pixels in *imageName* to the data specified in *data*. This command first searches the list of image file format handlers for a handler that can interpret the data in *data*, and then reads the image encoded within into *imageName* (the destination image). If *data* does not match any known format, an attempt to interpret it as a (top-to-bottom) list of scan-lines is made, with each scan-line being a (left-to-right) list of pixel colors (see Tk_GetColor for a description of valid colors.) Every scan-line must be of the same length. Note that when *data* is a single color name, you are instructing Tk to fill a rectangular region with that color. The following options may be specified:

-format *format-name*

Specifies the format of the image data in *data*. Specifically, only image file format handlers whose names begin with *format-name* will be used while searching for an image data format handler to read the data.

-to *x1 y1* ?*x2 y2*?

Specifies the coordinates of the top-left corner (*x1*,*y1*) of the region of *imageName* into which the image data will be copied. The default position is (0,0). If *x2*,*y2* is given and *data* is not large enough to cover the rectangle specified by this option, the image data extracted will be tiled so it covers the entire destination rectangle. Note that if *data* specifies a single color value, then a region extending to the bottom-right corner represented by (*x2*,*y2*) will be filled with that color.

***imageName* read *filename* ?*option value(s) ...*?**

Reads image data from the file named *filename* into the image. This command first searches the list of image file format handlers for a handler that can interpret the data in *filename*, and then reads the image in *filename* into *imageName* (the destination image). The following options may be specified:

-format *format-name*

Specifies the format of the image data in *filename*. Specifically, only image file format handlers whose names begin with *format-name* will be used while searching for an image data format handler to read the data.

-from *x1 y1 x2 y2*

Specifies a rectangular sub-region of the image file data to be copied to the destination image. If only *x1* and *y1* are specified, the region extends from (*x1*,*y1*) to the bottom-right corner of the image in the image file. If all four coordinates are specified, they specify diagonally opposite corners or the region. The default, if this option is not specified, is the whole of the image in the image file.

-shrink

If this option, the size of *imageName* will be reduced, if necessary, so that the region into which the image file data are read is at the bottom-right corner of the *imageName*. This option will not affect the width or height of the image if the user has specified a non-zero value for the **-width** or **-height** configuration option, respectively.

-to *x y*

Specifies the coordinates of the top-left corner of the region of *imageName* into which data from *filename* are to be read. The default is (0,0).

***imageName* redither**
The dithering algorithm used in displaying photo images propagates quantization errors from one pixel to its neighbors. If the image data for *imageName* is supplied in pieces, the dithered image may not be exactly correct. Normally the difference is not noticeable, but if it is a problem, this command can be used to recalculate the dithered image in each window where the image is displayed.

***imageName* transparency *subcommand* ?*arg arg ...*?**
Allows examination and manipulation of the transparency information in the photo image. Several subcommands are available:

imageName* transparency get *x y
Returns a boolean indicating if the pixel at (*x*,*y*) is transparent.

imageName* transparency set *x y boolean
Makes the pixel at (*x*,*y*) transparent if *boolean* is true, and makes that pixel opaque otherwise.

***imageName* write *filename* ?*option value(s) ...*?**
Writes image data from *imageName* to a file named *filename*. The following options may be specified:

-background *color*
If the color is specified, the data will not contain any transparency information. In all transparent pixels the color will be replaced by the specified color.

-format *format-name*
Specifies the name of the image file format handler to be used to write the data to the file. Specifically, this subcommand searches for the first handler whose name matches an initial substring of *format-name* and which has the capability to write an image file. If this option is not given, the format is guessed from the file extension. If that cannot be determined, this subcommand uses the first handler that has the capability to write an image file.

-from *x1 y1 x2 y2*
Specifies a rectangular region of *imageName* to be written to the image file. If only *x1* and *y1* are specified, the region extends from *(x1,y1)* to the bottom-right corner of *imageName*. If all four coordinates are given, they specify diagonally opposite corners of the rectangular region. The default, if this option is not given, is the whole image.

-grayscale
If this options is specified, the data will not contain color information. All pixel data will be transformed into grayscale.

Image formats

The photo image code is structured to allow handlers for additional image file formats to be added easily. The photo image code maintains a list of these handlers. Handlers are added to the list by registering them with a call to Tk_CreatePhotoImageFormat. The standard Tk distribution comes with handlers for PPM/PGM, PNG and GIF formats, which are automatically registered on initialization.

When reading an image file or processing string data specified with the **-data** configuration option, the photo image code invokes each handler in turn until one is found that claims to be able to read the data in the file or string. Usually this will find the correct handler, but if it does

not, the user may give a format name with the **-format** option to specify which handler to use. In fact the photo image code will try those handlers whose names begin with the string specified for the **-format** option (the comparison is case-insensitive). For example, if the user specifies **-format gif**, then a handler named GIF87 or GIF89 may be invoked, but a handler named JPEG may not (assuming that such handlers had been registered).

When writing image data to a file, the processing of the **-format** option is slightly different: the string value given for the **-format** option must begin with the complete name of the requested handler, and may contain additional information following that, which the handler can use, for example, to specify which variant to use of the formats supported by the handler. Note that not all image handlers may support writing transparency data to a file, even where the target image format does.

Format suboptions

Some image formats support sub-options, which are specified at the time that the image is loaded using additional words in the -format option. At the time of writing, the following are supported:

gif -index indexValue
: When parsing a multi-part GIF image, Tk normally only accesses the first image. By giving the -index sub-option, the indexValue'th value may be used instead. The indexValue must be an integer from 0 up to the number of image parts in the GIF data.

png -alpha alphaValue
: An additional alpha filtering for the overall image, which allows the background on which the image is displayed to show through. This usually also has the effect of desaturating the image. The alphaValue must be between 0.0 and 1.0.

Color allocation

When a photo image is displayed in a window, the photo image code allocates colors to use to display the image and dithers the image, if necessary, to display a reasonable approximation to the image using the colors that are available. The colors are allocated as a color cube, that is, the number of colors allocated is the product of the number of shades of red, green and blue.

Normally, the number of colors allocated is chosen based on the depth of the window. For example, in an 8-bit PseudoColor window, the photo image code will attempt to allocate seven shades of red, seven shades of green and four shades of blue, for a total of 198 colors. In a 1-bit StaticGray (monochrome) window, it will allocate two colors, black and white. In a 24-bit DirectColor or TrueColor window, it will allocate 256 shades each of red, green and blue. Fortunately, because of the way that pixel values can be combined in DirectColor and TrueColor windows, this only requires 256 colors to be allocated. If not all of the colors can be allocated, the photo image code reduces the number of shades of each primary color and tries again.

The user can exercise some control over the number of colors that a photo image uses with the **-palette** configuration option. If this option is used, it specifies the maximum number of shades of each primary color to try to allocate. It can also be used to force the image to be displayed in shades of gray, even on a color display, by giving a single number rather than three numbers separated by slashes.

Credits

The photo image type was designed and implemented by Paul Mackerras, based on his earlier photo widget and some suggestions from John Ousterhout.

Example

Load an image from a file and tile it to the size of a window, which is useful for producing a tiled background:

```
# These lines should be called once
image create photo untiled -file "theFile.ppm"
image create photo tiled

# These lines should be called whenever .someWidget changes
# size; a <Configure> binding is useful here
set width  [winfo width .someWidget]
set height [winfo height .someWidget]
tiled copy untiled -to 0 0 $width $height -shrink
```

The PNG image loader allows the application of an additional alpha factor during loading, which is useful for generating images suitable for disabled buttons:

```
image create photo icon -file "icon.png"
image create photo iconDisabled -file "icon.png" \
        -format "png -alpha 0.5"
button .b -image icon -disabledimage iconDisabled
```

Copyright (c) 1994 The Australian National University
Copyright (c) 1994-1997 Sun Microsystems, Inc.

place

Keywords: place, geometry manager, height, location, container, rubber sheet, content, width

Name

place - Geometry manager for fixed or rubber-sheet placement

Synopsis

place *option arg* ?*arg* ...?

Description

The placer is a geometry manager for Tk. It provides simple fixed placement of windows, where you specify the exact size and location of one window, called the *content*, within another window, called the *container*. The placer also provides rubber-sheet placement, where you specify the size and location of the content in terms of the dimensions of the container, so that the content changes size and location in response to changes in the size of the container. Lastly, the placer allows you to mix these styles of placement so that, for example, the content has a fixed width and height but is centered inside the container.

place *window option value ?option value ...?*

Arrange for the placer to manage the geometry of a content whose pathName is *window*. The remaining arguments consist of one or more *option-value* pairs that specify the way in which

window's geometry is managed. *Option* may have any of the values accepted by the **place configure** command.

place configure *window* ?*option*? ?*value option value ...*?

Query or modify the geometry options of the content given by *window*. If no *option* is specified, this command returns a list describing the available options (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given option(s) to have the given value(s); in this case the command returns an empty string.

The following *option-value* pairs are supported:

-anchor *where*

Where specifies which point of *window* is to be positioned at the (x,y) location selected by the **-x**, **-y**, **-relx**, and **-rely** options. The anchor point is in terms of the outer area of *window* including its border, if any. Thus if *where* is **se** then the lower-right corner of *window*'s border will appear at the given (x,y) location in the container. The anchor position defaults to **nw**.

-bordermode *mode*

Mode determines the degree to which borders within the container are used in determining the placement of the content. The default and most common value is **inside**. In this case the placer considers the area of the container to be the innermost area of the container, inside any border: an option of **-x 0** corresponds to an x-coordinate just inside the border and an option of **-relwidth 1.0** means *window* will fill the area inside the container's border.

If *mode* is **outside** then the placer considers the area of the container to include its border; this mode is typically used when placing *window* outside its container, as with the options **-x 0 -y 0 -anchor ne**. Lastly, *mode* may be specified as **ignore**, in which case borders are ignored: the area of the container is considered to be its official X area, which includes any internal border but no external border. A bordermode of **ignore** is probably not very useful.

-height *size*

Size specifies the height for *window* in screen units (i.e. any of the forms accepted by Tk_GetPixels). The height will be the outer dimension of *window* including its border, if any. If *size* is an empty string, or if no **-height** or **-relheight** option is specified, then the height requested internally by the window will be used.

-in *container*

Container specifies the path name of the window relative to which *window* is to be placed. *Container* must either be *window*'s parent or a descendant of *window*'s parent. In addition, *container* and *window* must both be descendants of the same top-level window. These restrictions are necessary to guarantee that *window* is visible whenever *container* is visible. If this option is not specified then the other window defaults to *window*'s parent.

-relheight *size*

Size specifies the height for *window*. In this case the height is specified as a floating-point number relative to the height of the container: 0.5 means *window* will be half as high as the container, 1.0 means *window* will have the same height as the container, and so on. If

both **-height** and **-relheight** are specified for a content, their values are summed. For example, **-relheight 1.0 -height -2** makes the content 2 pixels shorter than the container.

-relwidth *size*
Size specifies the width for *window*. In this case the width is specified as a floating-point number relative to the width of the container: 0.5 means *window* will be half as wide as the container, 1.0 means *window* will have the same width as the container, and so on. If both **-width** and **-relwidth** are specified for a content, their values are summed. For example, **-relwidth 1.0 -width 5** makes the content 5 pixels wider than the container.

-relx *location*
Location specifies the x-coordinate within the container window of the anchor point for *window*. In this case the location is specified in a relative fashion as a floating-point number: 0.0 corresponds to the left edge of the container and 1.0 corresponds to the right edge of the container. *Location* need not be in the range 0.0-1.0. If both **-x** and **-relx** are specified for a content then their values are summed. For example, **-relx 0.5 -x -2** positions the left edge of the content 2 pixels to the left of the center of its container.

-rely *location*
Location specifies the y-coordinate within the container window of the anchor point for *window*. In this case the value is specified in a relative fashion as a floating-point number: 0.0 corresponds to the top edge of the container and 1.0 corresponds to the bottom edge of the container. *Location* need not be in the range 0.0-1.0. If both **-y** and **-rely** are specified for a content then their values are summed. For example, **-rely 0.5 -x 3** positions the top edge of the content 3 pixels below the center of its container.

-width *size*
Size specifies the width for *window* in screen units (i.e. any of the forms accepted by Tk_GetPixels). The width will be the outer width of *window* including its border, if any. If *size* is an empty string, or if no **-width** or **-relwidth** option is specified, then the width requested internally by the window will be used.

-x *location*
Location specifies the x-coordinate within the container window of the anchor point for *window*. The location is specified in screen units (i.e. any of the forms accepted by Tk_GetPixels) and need not lie within the bounds of the container window.

-y *location*
Location specifies the y-coordinate within the container window of the anchor point for *window*. The location is specified in screen units (i.e. any of the forms accepted by Tk_GetPixels) and need not lie within the bounds of the container window.

If the same value is specified separately with two different options, such as **-x** and **-relx**, then the most recent option is used and the older one is ignored.

place forget *window*
Causes the placer to stop managing the geometry of *window*. As a side effect of this command *window* will be unmapped so that it does not appear on the screen. If *window* is not currently managed by the placer then the command has no effect. This command returns an empty string.

place info *window*

Returns a list giving the current configuration of *window*. The list consists of *option-value* pairs in exactly the same form as might be specified to the **place configure** command.

place slaves *window*
Returns a list of all the content windows for which *window* is the container. If there is no content for *window* then an empty string is returned.

place content window
Synonym for place slaves window.

If the configuration of a window has been retrieved with **place info**, that configuration can be restored later by first using **place forget** to erase any existing information for the window and then invoking **place configure** with the saved information.

Fine points

It is not necessary for the container window to be the parent of the content window. This feature is useful in at least two situations. First, for complex window layouts it means you can create a hierarchy of subwindows whose only purpose is to assist in the layout of the parent. The "real children" of the parent (i.e. the windows that are significant for the application's user interface) can be children of the parent yet be placed inside the windows of the geometry-management hierarchy. This means that the path names of the "real children" do not reflect the geometry-management hierarchy and users can specify options for the real children without being aware of the structure of the geometry-management hierarchy.

A second reason for having a container different than the content's parent is to tie two siblings together. For example, the placer can be used to force a window always to be positioned centered just below one of its siblings by specifying the configuration

```
-in sibling -relx 0.5 -rely 1.0 -anchor n -bordermode outside
```

Whenever the sibling is repositioned in the future, the content will be repositioned as well.

Unlike many other geometry managers (such as the packer) the placer does not make any attempt to manipulate the geometry of the container windows or the parents of content windows (i.e. it does not set their requested sizes). To control the sizes of these windows, make them windows like frames and canvases that provide configuration options for this purpose.

Example

Make the label occupy the middle bit of the toplevel, no matter how it is resized:

```
label .l -text "In the\nMiddle!" -bg black -fg white
place .l -relwidth .3 -relx .35 -relheight .3 -rely .35
```

Copyright (c) 1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

popup

Keywords: popup, menu

Name

tk_popup - Post a popup menu

Synopsis
tk_popup *menu x y* ?*entry*?

Description
This procedure posts a menu at a given position on the screen and configures Tk so that the menu and its cascaded children can be traversed with the mouse or the keyboard. *Menu* is the name of a menu widget and *x* and *y* are the root coordinates at which to display the menu. If *entry* is omitted or an empty string, the menu's upper left corner is positioned at the given point. Otherwise *entry* gives the index of an entry in *menu* and the menu will be positioned so that the entry is positioned over the given point.

Example
How to attach a simple popup menu to a widget.

```
# Create a menu
set m [menu .popupMenu]
$m add command -label "Example 1" -command bell
$m add command -label "Example 2" -command bell

# Create something to attach it to
pack [label .l -text "Click me!"]

# Arrange for the menu to pop up when the label is clicked
bind .l <1> {tk_popup .popupMenu %X %Y}
```

Copyright (c) 1994-1996 Sun Microsystems, Inc.

radiobutton
Keywords: radiobutton, widget

Name
radiobutton - Create and manipulate 'radiobutton' pick-one widgets

Synopsis
radiobutton *pathName* ?*options*?

Standard options
-activebackground, **-disabledforeground**, **-padx**, **-activeforeground**, **-font**, **-pady**, **-anchor**, **-foreground**, **-relief**, **-background**, **-highlightbackground**, **-takefocus**, **-bitmap**, **-highlightcolor**, **-text**, **-borderwidth**, **-highlightthickness**, **-textvariable**, **-compound**, **-image**, **-underline**, **-cursor**, **-justify**, **-wraplength**

See the options manual entry for details on the standard options.

Widget-specific options
-command (command/Command)

Specifies a Tcl command to associate with the button. This command is typically invoked when mouse button 1 is released over the button window. The button's global variable (

-variable option) will be updated before the command is invoked.

-height (height/Height)
Specifies a desired height for the button. If an image or bitmap is being displayed in the button then the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels); for text it is in lines of text. If this option is not specified, the button's desired height is computed from the size of the image or bitmap or text being displayed in it.

-indicatoron (indicatorOn/IndicatorOn)
Specifies whether or not the indicator should be drawn. Must be a proper boolean value. If false, the **-relief** option is ignored and the widget's relief is always sunken if the widget is selected and raised otherwise.

-offrelief (offRelief/OffRelief)
Specifies the relief for the checkbutton when the indicator is not drawn and the checkbutton is off. The default value is "raised". By setting this option to "flat" and setting **-indicatoron** to false and **-overrelief** to "raised", the effect is achieved of having a flat button that raises on mouse-over and which is depressed when activated. This is the behavior typically exhibited by the Align-Left, Align-Right, and Center radiobuttons on the toolbar of a word-processor, for example.

-overrelief (overRelief/OverRelief)
Specifies an alternative relief for the radiobutton, to be used when the mouse cursor is over the widget. This option can be used to make toolbar buttons, by configuring **-relief flat -overrelief raised**. If the value of this option is the empty string, then no alternative relief is used when the mouse cursor is over the radiobutton. The empty string is the default value.

-selectcolor (selectColor/Background)
Specifies a background color to use when the button is selected. If **indicatorOn** is true then the color is used as the background for the indicator regardless of the select state. If **-indicatoron** is false, this color is used as the background for the entire widget, in place of **-background** or **-activeBackground**, whenever the widget is selected. If specified as an empty string then no special color is used for displaying when the widget is selected.

-selectimage (selectImage/SelectImage)
Specifies an image to display (in place of the **-image** option) when the radiobutton is selected. This option is ignored unless the **-image** option has been specified.

-state (state/State)
Specifies one of three states for the radiobutton: **normal**, **active**, or **disabled**. In normal state the radiobutton is displayed using the **-foreground** and **-background** options. The active state is typically used when the pointer is over the radiobutton. In active state the radiobutton is displayed using the **-activeforeground** and **-activebackground** options. Disabled state means that the radiobutton should be insensitive: the default bindings will refuse to activate the widget and will ignore mouse button presses. In this state the **-disabledforeground** and **-background** options determine how the radiobutton is displayed.

-tristateimage (tristateImage/TristateImage)
Specifies an image to display (in place of the **-image** option) when the radiobutton is selected. This option is ignored unless the **-image** option has been specified.

-tristatevalue (tristateValue/Value)
Specifies the value that causes the radiobutton to display the multi-value selection, also

known as the tri-state mode. Defaults to "".

-value (value/Value)
Specifies value to store in the button's associated variable whenever this button is selected.

-variable (variable**/Variable)**
Specifies the name of a global variable to set whenever this button is selected. Changes in this variable also cause the button to select or deselect itself. Defaults to the value **selectedButton**.

-width (width/Width)
Specifies a desired width for the button. If an image or bitmap is being displayed in the button, the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels); for text it is in characters. If this option is not specified, the button's desired width is computed from the size of the image or bitmap or text being displayed in it.

Description

The **radiobutton** command creates a new window (given by the *pathName* argument) and makes it into a radiobutton widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the radiobutton such as its colors, font, text, and initial relief. The **radiobutton** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A radiobutton is a widget that displays a textual string, bitmap or image and a diamond or circle called an *indicator*. If text is displayed, it must all be in a single font, but it can occupy multiple lines on the screen (if it contains newlines or if wrapping occurs because of the **-wraplength** option) and one of the characters may optionally be underlined using the **-underline** option. A radiobutton has all of the behavior of a simple button: it can display itself in either of three different ways, according to the **-state** option; it can be made to appear raised, sunken, or flat; it can be made to flash; and it invokes a Tcl command whenever mouse button 1 is clicked over the check button.

In addition, radiobuttons can be *selected*. If a radiobutton is selected, the indicator is normally drawn with a selected appearance, and a Tcl variable associated with the radiobutton is set to a particular value (normally 1). Under Unix, the indicator is drawn with a sunken relief and a special color. Under Windows, the indicator is drawn with a round mark inside. If the radiobutton is not selected, then the indicator is drawn with a deselected appearance, and the associated variable is set to a different value (typically 0). The indicator is drawn without a round mark inside. Typically, several radiobuttons share a single variable and the value of the variable indicates which radiobutton is to be selected. When a radiobutton is selected it sets the value of the variable to indicate that fact; each radiobutton also monitors the value of the variable and automatically selects and deselects itself when the variable's value changes. If the variable's value matches the **-tristatevalue**, then the radiobutton is drawn using the tri-state mode. This mode is used to indicate mixed or multiple values. (This is used when the radiobutton represents the state of multiple items.) By default the variable **selectedButton** is used; its contents give the name of the button that is selected, or the empty string if no button associated with that variable is selected. The name of the variable for a radiobutton, plus the variable to be stored into it, may be modified with options on the command line or in the option database. Configuration options may also be used to modify the way the indicator is displayed (or whether

it is displayed at all). By default a radiobutton is configured to select itself on button clicks.

Widget command

The **radiobutton** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following commands are possible for radiobutton widgets:

pathName* cget *option
: Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **radiobutton** command.

***pathName* configure ?*option*? ?*value option value ...*?**
: Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **radiobutton** command.

***pathName* deselect**
: Deselects the radiobutton and sets the associated variable to an empty string. If this radiobutton was not currently selected, the command has no effect.

***pathName* flash**
: Flashes the radiobutton. This is accomplished by redisplaying the radiobutton several times, alternating between active and normal colors. At the end of the flash the radiobutton is left in the same normal/active state as when the command was invoked. This command is ignored if the radiobutton's state is **disabled**.

***pathName* invoke**
: Does just what would have happened if the user invoked the radiobutton with the mouse: selects the button and invokes its associated Tcl command, if there is one. The return value is the return value from the Tcl command, or an empty string if there is no command associated with the radiobutton. This command is ignored if the radiobutton's state is **disabled**.

***pathName* select**
: Selects the radiobutton and sets the associated variable to the value corresponding to this widget.

Bindings

Tk automatically creates class bindings for radiobuttons that give them the following default behavior:

1. On Unix systems, a radiobutton activates whenever the mouse passes over it and deactivates whenever the mouse leaves the radiobutton. On Mac and Windows systems, when mouse button 1 is pressed over a radiobutton, the button activates whenever the mouse pointer is

inside the button, and deactivates whenever the mouse pointer leaves the button.
2. When mouse button 1 is pressed over a radiobutton it is invoked (it becomes selected and the command associated with the button is invoked, if there is one).
3. When a radiobutton has the input focus, the space key causes the radiobutton to be invoked.

If the radiobutton's state is **disabled** then none of the above actions occur: the radiobutton is completely non-responsive.

The behavior of radiobuttons can be changed by defining new bindings for individual widgets or by redefining the class bindings.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

raise

Keywords: raise, obscure, stacking order

Name

raise - Change a window's position in the stacking order

Synopsis

raise *window* ?*aboveThis*?

Description

If the *aboveThis* argument is omitted then the command raises *window* so that it is above all of its siblings in the stacking order (it will not be obscured by any siblings and will obscure any siblings that overlap it). If *aboveThis* is specified then it must be the path name of a window that is either a sibling of *window* or the descendant of a sibling of *window*. In this case the **raise** command will insert *window* into the stacking order just above *aboveThis* (or the ancestor of *aboveThis* that is a sibling of *window*); this could end up either raising or lowering *window*.

All toplevel windows may be restacked with respect to each other, whatever their relative path names, but the window manager is not obligated to strictly honor requests to restack.

On macOS raising an iconified toplevel window causes it to be deiconified.

Example

Make a button appear to be in a sibling frame that was created after it. This is is often necessary when building GUIs in the style where you create your activity widgets first before laying them out on the display:

```
button .b -text "Hi there!"
pack [frame .f -background blue]
pack [label .f.l1 -text "This is above"]
pack .b -in .f
pack [label .f.l2 -text "This is below"]
raise .b
```

Copyright (c) 1990 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

scale

Keywords: scale, slider, trough, widget

Name

scale - Create and manipulate 'scale' value-controlled slider widgets

Synopsis

scale *pathName* ?*options*?

Standard options

-activebackground, **-foreground**, **-relief**, **-background**, **-highlightbackground**, **-repeatdelay**, **-borderwidth**, **-highlightcolor**, **-repeatinterval**, **-cursor**, **-highlightthickness**, **-takefocus**, **-font**, **-orient**, **-troughcolor**

See the options manual entry for details on the standard options.

Widget-specific options

-bigincrement (bigIncrement/BigIncrement)
Some interactions with the scale cause its value to change by "large" increments; this option specifies the size of the large increments. If specified as 0, the large increments default to 1/10 the range of the scale.

-command (command/Command)
Specifies the prefix of a Tcl command to invoke whenever the scale's value is changed via a widget command. The actual command consists of this option followed by a space and a real number indicating the new value of the scale.

-digits (digits/Digits)
An integer specifying how many significant digits should be retained when converting the value of the scale to a string. If the number is less than or equal to zero, then the scale picks the smallest value that guarantees that every possible slider position prints as a different string.

-from (from/From)
A real value corresponding to the left or top end of the scale.

-label (label/Label)
A string to display as a label for the scale. For vertical scales the label is displayed just to the right of the top end of the scale. For horizontal scales the label is displayed just above the left end of the scale. If the option is specified as an empty string, no label is displayed.

-length (length/Length)
Specifies the desired long dimension of the scale in screen units (i.e. any of the forms acceptable to Tk_GetPixels). For vertical scales this is the scale's height; for horizontal scales it is the scale's width.

-resolution (resolution/Resolution)
A real value specifying the resolution for the scale. If this value is greater than zero then the scale's value will always be rounded to an even multiple of this value, as will the endpoints of the scale. If the value is less than zero then no rounding occurs. Defaults to 1 (i.e., the value will be integral).

-showvalue (showValue/ShowValue)

Specifies a boolean value indicating whether or not the current value of the scale is to be displayed.

-sliderlength (sliderLength/SliderLength)
Specifies the size of the slider, measured in screen units along the slider's long dimension. The value may be specified in any of the forms acceptable to Tk_GetPixels.

-sliderrelief (sliderRelief/SliderRelief)
Specifies the relief to use when drawing the slider, such as **raised** or **sunken**.

-state (state/State)
Specifies one of three states for the scale: **normal**, **active**, or **disabled**. If the scale is disabled then the value may not be changed and the scale will not activate. If the scale is active, the slider is displayed using the color specified by the **-activebackground** option.

-tickinterval (tickInterval/TickInterval)
Must be a real value. Determines the spacing between numerical tick marks displayed below or to the left of the slider. The values will all be displayed with the same number of decimal places, which will be enough to ensure they are all accurate to within 20% of a tick interval. If 0, no tick marks will be displayed.

-to (to/To)
Specifies a real value corresponding to the right or bottom end of the scale. This value may be either less than or greater than the **-from** option.

-variable (variable/Variable)
Specifies the name of a global variable to link to the scale. Whenever the value of the variable changes, the scale will update to reflect this value. Whenever the scale is manipulated interactively, the variable will be modified to reflect the scale's new value.

-width (width/Width)
Specifies the desired narrow dimension of the scale in screen units (i.e. any of the forms acceptable to Tk_GetPixels). For vertical scales this is the scale's width; for horizontal scales this is the scale's height.

Description

The **scale** command creates a new window (given by the *pathName* argument) and makes it into a scale widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the scale such as its colors, orientation, and relief. The **scale** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A scale is a widget that displays a rectangular *trough* and a small *slider*. The trough corresponds to a range of real values (determined by the **-from**, **-to**, and **-resolution** options), and the position of the slider selects a particular real value. The slider's position (and hence the scale's value) may be adjusted with the mouse or keyboard as described in the Bindings section below. Whenever the scale's value is changed, a Tcl command is invoked (using the **-command** option) to notify other interested widgets of the change. In addition, the value of the scale can be linked to a Tcl variable (using the **-variable** option), so that changes in either are reflected in the other.

Three annotations may be displayed in a scale widget: a label appearing at the top right of the widget (top left for horizontal scales), a number displayed just to the left of the slider (just above the slider for horizontal scales), and a collection of numerical tick marks just to the left of the current value (just below the trough for horizontal scales). Each of these three annotations may

be enabled or disabled using the configuration options.

Widget command

The **scale** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following commands are possible for scale widgets:

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **scale** command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **scale** command.

***pathName* coords ?*value*?**
Returns a list whose elements are the x and y coordinates of the point along the centerline of the trough that corresponds to *value*. If *value* is omitted then the scale's current value is used.

***pathName* get ?*x y*?**
If *x* and *y* are omitted, returns the current value of the scale. If *x* and *y* are specified, they give pixel coordinates within the widget; the command returns the scale value corresponding to the given pixel. Only one of *x* or *y* is used: for horizontal scales *y* is ignored, and for vertical scales *x* is ignored.

pathName* identify *x y
Returns a string indicating what part of the scale lies under the coordinates given by *x* and *y*. A return value of **slider** means that the point is over the slider; **trough1** means that the point is over the portion of the slider above or to the left of the slider; and **trough2** means that the point is over the portion of the slider below or to the right of the slider. If the point is not over one of these elements, an empty string is returned.

pathName* set *value
This command is invoked to change the current value of the scale, and hence the position at which the slider is displayed. *Value* gives the new value for the scale. The command has no effect if the scale is disabled.

Bindings

Tk automatically creates class bindings for scales that give them the following default behavior. Where the behavior is different for vertical and horizontal scales, the horizontal behavior is described in parentheses.

1. If button 1 is pressed in the trough, the scale's value will be incremented or decremented by

the value of the **-resolution** option so that the slider moves in the direction of the cursor. If the button is held down, the action auto-repeats.
2. If button 1 is pressed over the slider, the slider can be dragged with the mouse.
3. If button 1 is pressed in the trough with the Control key down, the slider moves all the way to the end of its range, in the direction towards the mouse cursor.
4. If button 2 is pressed, the scale's value is set to the mouse position. If the mouse is dragged with button 2 down, the scale's value changes with the drag.
5. The Up and Left keys move the slider up (left) by the value of the **-resolution** option.
6. The Down and Right keys move the slider down (right) by the value of the **-resolution** option.
7. Control-Up and Control-Left move the slider up (left) by the value of the **-bigincrement** option.
8. Control-Down and Control-Right move the slider down (right) by the value of the **-bigincrement** option.
9. Home moves the slider to the top (left) end of its range.
10. End moves the slider to the bottom (right) end of its range.

If the scale is disabled using the **-state** option then none of the above bindings have any effect.

The behavior of scales can be changed by defining new bindings for individual widgets or by redefining the class bindings.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

scrollbar

Keywords: scrollbar, widget

Name

scrollbar - Create and manipulate 'scrollbar' scrolling control and indicator widgets

Synopsis

scrollbar *pathName* ?*options*?

Standard options

-activebackground, **-highlightcolor**, **-repeatdelay**, **-background**, **-highlightthickness**, **-repeatinterval**, **-borderwidth**, **-jump**, **-takefocus**, **-cursor**, **-orient**, **-troughcolor**, **-highlightbackground**, **-relief**

See the options manual entry for details on the standard options.

Widget-specific options

-activerelief (activeRelief/ActiveRelief)
Specifies the relief to use when displaying the element that is active, if any. Elements other than the active element are always displayed with a raised relief.

-command (command/Command)
Specifies the prefix of a Tcl command to invoke to change the view in the widget associated with the scrollbar. When a user requests a view change by manipulating the scrollbar, a Tcl command is invoked. The actual command consists of this option followed by additional information as described later. This option almost always has a value such as **.t xview** or **.t**

yview, consisting of the name of a widget and either **xview** (if the scrollbar is for horizontal scrolling) or **yview** (for vertical scrolling). All scrollable widgets have **xview** and **yview** commands that take exactly the additional arguments appended by the scrollbar as described in Scrolling commands below.

-elementborderwidth (elementBorderWidth/BorderWidth)
Specifies the width of borders drawn around the internal elements of the scrollbar (the two arrows and the slider). The value may have any of the forms acceptable to Tk_GetPixels. If this value is less than zero, the value of the **-borderwidth** option is used in its place.

-width (width/Width)
Specifies the desired narrow dimension of the scrollbar window, not including 3-D border, if any. For vertical scrollbars this will be the width and for horizontal scrollbars this will be the height. The value may have any of the forms acceptable to Tk_GetPixels.

Description

The **scrollbar** command creates a new window (given by the *pathName* argument) and makes it into a scrollbar widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the scrollbar such as its colors, orientation, and relief. The **scrollbar** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A scrollbar is a widget that displays two arrows, one at each end of the scrollbar, and a *slider* in the middle portion of the scrollbar. It provides information about what is visible in an *associated window* that displays a document of some sort (such as a file being edited or a drawing). The position and size of the slider indicate which portion of the document is visible in the associated window. For example, if the slider in a vertical scrollbar covers the top third of the area between the two arrows, it means that the associated window displays the top third of its document.

Scrollbars can be used to adjust the view in the associated window by clicking or dragging with the mouse. See the Bindings section below for details.

Elements

A scrollbar displays five elements, which are referred to in the widget commands for the scrollbar:

arrow1
The top or left arrow in the scrollbar.

trough1
The region between the slider and **arrow1**.

slider
The rectangle that indicates what is visible in the associated widget.

trough2
The region between the slider and **arrow2**.

arrow2
The bottom or right arrow in the scrollbar.

Widget command

The **scrollbar** command creates a new Tcl command whose name is *pathName*. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg ...?
```

Option and the *arg*s determine the exact behavior of the command. The following commands are possible for scrollbar widgets:

***pathName* activate ?*element*?**
Marks the element indicated by *element* as active, which causes it to be displayed as specified by the **-activebackground** and **-activerelief** options. The only element values understood by this command are **arrow1**, **slider**, or **arrow2**. If any other value is specified then no element of the scrollbar will be active. If *element* is not specified, the command returns the name of the element that is currently active, or an empty string if no element is active.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **scrollbar** command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **scrollbar** command.

pathName* delta *deltaX deltaY
Returns a real number indicating the fractional change in the scrollbar setting that corresponds to a given change in slider position. For example, if the scrollbar is horizontal, the result indicates how much the scrollbar setting must change to move the slider *deltaX* pixels to the right (*deltaY* is ignored in this case). If the scrollbar is vertical, the result indicates how much the scrollbar setting must change to move the slider *deltaY* pixels down. The arguments and the result may be zero or negative.

pathName* fraction *x y
Returns a real number between 0 and 1 indicating where the point given by *x* and *y* lies in the trough area of the scrollbar. The value 0 corresponds to the top or left of the trough, the value 1 corresponds to the bottom or right, 0.5 corresponds to the middle, and so on. *X* and *y* must be pixel coordinates relative to the scrollbar widget. If *x* and *y* refer to a point outside the trough, the closest point in the trough is used.

***pathName* get**
Returns the scrollbar settings in the form of a list whose elements are the arguments to the most recent **set** widget command.

pathName* identify *x y
Returns the name of the element under the point given by *x* and *y* (such as **arrow1**), or an empty string if the point does not lie in any element of the scrollbar. *X* and *y* must be pixel coordinates relative to the scrollbar widget.

pathName* set *first last
This command is invoked by the scrollbar's associated widget to tell the scrollbar about the

current view in the widget. The command takes two arguments, each of which is a real fraction between 0 and 1. The fractions describe the range of the document that is visible in the associated widget. For example, if *first* is 0.2 and *last* is 0.4, it means that the first part of the document visible in the window is 20% of the way through the document, and the last visible part is 40% of the way through.

Scrolling commands

When the user interacts with the scrollbar, for example by dragging the slider, the scrollbar notifies the associated widget that it must change its view. The scrollbar makes the notification by evaluating a Tcl command generated from the scrollbar's **-command** option. The command may take any of the following forms. In each case, *prefix* is the contents of the **-command** option, which usually has a form like "**.t yview**".

prefix* moveto *fraction

Fraction is a real number between 0 and 1. The widget should adjust its view so that the point given by *fraction* appears at the beginning of the widget. If *fraction* is 0 it refers to the beginning of the document. 1.0 refers to the end of the document, 0.333 refers to a point one-third of the way through the document, and so on.

***prefix* scroll *number* units**

The widget should adjust its view by *number* units. The units are defined in whatever way makes sense for the widget, such as characters or lines in a text widget. *Number* is either 1, which means one unit should scroll off the top or left of the window, or -1, which means that one unit should scroll off the bottom or right of the window.

***prefix* scroll *number* pages**

The widget should adjust its view by *number* pages. It is up to the widget to define the meaning of a page; typically it is slightly less than what fits in the window, so that there is a slight overlap between the old and new views. *Number* is either 1, which means the next page should become visible, or -1, which means that the previous page should become visible.

Old command syntax

In versions of Tk before 4.0, the **set** and **get** widget commands used a different form. This form is still supported for backward compatibility, but it is deprecated. In the old command syntax, the **set** widget command has the following form:

pathName* set *totalUnits windowUnits firstUnit lastUnit

In this form the arguments are all integers. *TotalUnits* gives the total size of the object being displayed in the associated widget. The meaning of one unit depends on the associated widget; for example, in a text editor widget units might correspond to lines of text. *WindowUnits* indicates the total number of units that can fit in the associated window at one time. *FirstUnit* and *lastUnit* give the indices of the first and last units currently visible in the associated window (zero corresponds to the first unit of the object).

Under the old syntax the **get** widget command returns a list of four integers, consisting of the *totalUnits*, *windowUnits*, *firstUnit*, and *lastUnit* values from the last **set** widget command.

The commands generated by scrollbars also have a different form when the old syntax is being used:

prefix unit
Unit is an integer that indicates what should appear at the top or left of the associated widget's window. It has the same meaning as the *firstUnit* and *lastUnit* arguments to the **set** widget command.

The most recent **set** widget command determines whether or not to use the old syntax. If it is given two real arguments then the new syntax will be used in the future, and if it is given four integer arguments then the old syntax will be used.

Bindings

Tk automatically creates class bindings for scrollbars that give them the following default behavior. If the behavior is different for vertical and horizontal scrollbars, the horizontal behavior is described in parentheses.

1. Pressing button 1 over **arrow1** causes the view in the associated widget to shift up (left) by one unit so that the document appears to move down (right) one unit. If the button is held down, the action auto-repeats.
2. Pressing button 1 over **trough1** causes the view in the associated widget to shift up (left) by one screenful so that the document appears to move down (right) one screenful. If the button is held down, the action auto-repeats.
3. Pressing button 1 over the slider and dragging causes the view to drag with the slider. If the **jump** option is true, then the view does not drag along with the slider; it changes only when the mouse button is released.
4. Pressing button 1 over **trough2** causes the view in the associated widget to shift down (right) by one screenful so that the document appears to move up (left) one screenful. If the button is held down, the action auto-repeats.
5. Pressing button 1 over **arrow2** causes the view in the associated widget to shift down (right) by one unit so that the document appears to move up (left) one unit. If the button is held down, the action auto-repeats.
6. If button 2 is pressed over the trough or the slider, it sets the view to correspond to the mouse position; dragging the mouse with button 2 down causes the view to drag with the mouse. If button 2 is pressed over one of the arrows, it causes the same behavior as pressing button 1.
7. If button 1 is pressed with the Control key down, then if the mouse is over **arrow1** or **trough1** the view changes to the very top (left) of the document; if the mouse is over **arrow2** or **trough2** the view changes to the very bottom (right) of the document; if the mouse is anywhere else then the button press has no effect.
8. In vertical scrollbars the Up and Down keys have the same behavior as mouse clicks over **arrow1** and **arrow2**, respectively. In horizontal scrollbars these keys have no effect.
9. In vertical scrollbars Control-Up and Control-Down have the same behavior as mouse clicks over **trough1** and **trough2**, respectively. In horizontal scrollbars these keys have no effect.
10. In horizontal scrollbars the Up and Down keys have the same behavior as mouse clicks over **arrow1** and **arrow2**, respectively. In vertical scrollbars these keys have no effect.
11. In horizontal scrollbars Control-Up and Control-Down have the same behavior as mouse clicks over **trough1** and **trough2**, respectively. In vertical scrollbars these keys have no effect.
12. The Prior and Next keys have the same behavior as mouse clicks over **trough1** and **trough2**, respectively.
13. The Home key adjusts the view to the top (left edge) of the document.

14. The End key adjusts the view to the bottom (right edge) of the document.

Example

Create a window with a scrollable text widget:

```
toplevel .tl
text .tl.t -yscrollcommand {.tl.s set}
scrollbar .tl.s -command {.tl.t yview}
grid .tl.t .tl.s -sticky nsew
grid columnconfigure .tl 0 -weight 1
grid rowconfigure .tl 0 -weight 1
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

selection

Keywords: selection, clear, format, handler, ICCCM, own, target, type

Name

selection - Manipulate the X selection

Synopsis

selection *option* ?*arg arg* ...?

Description

This command provides a Tcl interface to the X selection mechanism and implements the full selection functionality described in the X Inter-Client Communication Conventions Manual (ICCCM).

Note that for management of the **CLIPBOARD** selection (see below), the clipboard command may also be used.

The first argument to **selection** determines the format of the rest of the arguments and the behavior of the command. The following forms are currently supported:

selection clear ?-displayof *window*? ?-selection *selection*?
: If *selection* exists anywhere on *window*'s display, clear it so that no window owns the selection anymore. *Selection* specifies the X selection that should be cleared, and should be an atom name such as **PRIMARY** or **CLIPBOARD**; see the Inter-Client Communication Conventions Manual for complete details. *Selection* defaults to **PRIMARY** and *window* defaults to ".". Returns an empty string.

selection get ?-displayof *window*? ?-selection *selection*? ?-type *type*?
: Retrieves the value of *selection* from *window*'s display and returns it as a result. *Selection* defaults to **PRIMARY** and *window* defaults to ".". *Type* specifies the form in which the selection is to be returned (the desired "target" for conversion, in ICCCM terminology), and should be an atom name such as **STRING** or **FILE_NAME**; see the Inter-Client Communication Conventions Manual for complete details. *Type* defaults to **STRING**. The selection owner may choose to return the selection in any of several different representation formats, such as **STRING**, **UTF8_STRING**, **ATOM**, **INTEGER**, etc. (this format is different than the selection type; see the ICCCM for all the confusing details). If the selection is returned in a non-string

format, such as **INTEGER** or **ATOM**, the **selection** command converts it to string format as a collection of fields separated by spaces: atoms are converted to their textual names, and anything else is converted to hexadecimal integers. Note that **selection get** does not retrieve the selection in the **UTF8_STRING** format unless told to.

selection handle ?-selection *s*? ?-type *t*? ?-format *f*? *window command*

Creates a handler for selection requests, such that *command* will be executed whenever selection *s* is owned by *window* and someone attempts to retrieve it in the form given by type *t* (e.g. *t* is specified in the **selection get** command). *S* defaults to **PRIMARY**, *t* defaults to **STRING**, and *f* defaults to **STRING**. If *command* is an empty string then any existing handler for *window*, *t*, and *s* is removed. Note that when the selection is handled as type **STRING** it is also automatically handled as type **UTF8_STRING** as well.

When *selection* is requested, *window* is the selection owner, and *type* is the requested type, *command* will be executed as a Tcl command with two additional numbers appended to it (with space separators). The two additional numbers are *offset* and *maxChars*: *offset* specifies a starting character position in the selection and *maxChars* gives the maximum number of characters to retrieve. The command should return a value consisting of at most *maxChars* of the selection, starting at position *offset*. For very large selections (larger than *maxChars*) the selection will be retrieved using several invocations of *command* with increasing *offset* values. If *command* returns a string whose length is less than *maxChars*, the return value is assumed to include all of the remainder of the selection; if the length of *command*'s result is equal to *maxChars* then *command* will be invoked again, until it eventually returns a result shorter than *maxChars*. The value of *maxChars* will always be relatively large (thousands of characters).

If *command* returns an error then the selection retrieval is rejected just as if the selection did not exist at all.

The *format* argument specifies the representation that should be used to transmit the selection to the requester (the second column of Table 2 of the ICCCM), and defaults to **STRING**. If *format* is **STRING**, the selection is transmitted as 8-bit ASCII characters (i.e. just in the form returned by *command*, in the system encoding; the **UTF8_STRING** format always uses UTF-8 as its encoding). If *format* is **ATOM**, then the return value from *command* is divided into fields separated by white space; each field is converted to its atom value, and the 32-bit atom value is transmitted instead of the atom name. For any other *format*, the return value from *command* is divided into fields separated by white space and each field is converted to a 32-bit integer; an array of integers is transmitted to the selection requester.

The *format* argument is needed only for compatibility with selection requesters that do not use Tk. If Tk is being used to retrieve the selection then the value is converted back to a string at the requesting end, so *format* is irrelevant.

selection own ?-displayof *window*? ?-selection *selection*?

see below ...

selection own ?-command *command*? ?-selection *selection*? *window*

The first form of **selection own** returns the path name of the window in this application that owns *selection* on the display containing *window*, or an empty string if no window in this

application owns the selection. *Selection* defaults to **PRIMARY** and *window* defaults to ".".

The second form of **selection own** causes *window* to become the new owner of *selection* on *window*'s display, returning an empty string as result. The existing owner, if any, is notified that it has lost the selection. If *command* is specified, it is a Tcl script to execute when some other window claims ownership of the selection away from *window*. *Selection* defaults to PRIMARY.

Widget facilities

The text, entry, ttk::entry, listbox, spinbox and ttk::spinbox widgets have the option **-exportselection**. If a widget has this option set to boolean **true**, then (in an unsafe interpreter) a selection made in the widget is automatically written to the **PRIMARY** selection.

A GUI event, for example **<>**, can copy the **PRIMARY** selection to certain widgets. This copy is implemented by a widget binding to the event. The binding script makes appropriate calls to the **selection** command.

Portability issues

On X11, the **PRIMARY** selection is a system-wide feature of the X server, allowing communication between different processes that are X11 clients.

On Windows, the **PRIMARY** selection is not provided by the system, but only by Tk, and so it is shared only between windows of a parent interpreter and its child interpreters. It is not shared between interpreters in different processes or different threads. Each parent interpreter has a separate **PRIMARY** selection that is shared only with its child interpreters which are not safe interpreters.

Security

A safe interpreter cannot read from the **PRIMARY** selection because its **selection** command is hidden. For this reason the **PRIMARY** selection cannot be written to the Tk widgets of a safe interpreter.

A Tk widget can have its option **-exportselection** set to boolean **true**, but in a safe interpreter this option has no effect: writing from the widget to the **PRIMARY** selection is disabled.

These are security features. A safe interpreter may run untrusted code, and it is a security risk if this untrusted code can read or write the **PRIMARY** selection used by other interpreters.

Examples

On X11 platforms, one of the standard selections available is the **SECONDARY** selection. Hardly anything uses it, but here is how to read it using Tk:

```
set selContents [selection get -selection SECONDARY]
```

Many different types of data may be available for a selection; the special type **TARGETS** allows you to get a list of available types:

```
foreach type [selection get -type TARGETS] {
   puts "Selection PRIMARY supports type $type"
}
```

To claim the selection, you must first set up a handler to supply the data for the selection. Then you have to claim the selection...

```
# Set up the data handler ready for incoming requests
set foo "This is a string with some data in it... blah blah"
selection handle -selection SECONDARY . getData
proc getData {offset maxChars} {
   puts "Retrieving selection starting at $offset"
   return [string range $::foo $offset [expr {$offset+$maxChars-1}]]
}

# Now we grab the selection itself
puts "Claiming selection"
selection own -command lost -selection SECONDARY .
proc lost {} {
   puts "Lost selection"
}
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

send

Keywords: send, application, dde, name, remote execution, security

Name

send - Execute a command in a different application

Synopsis

send ?*options*? *app cmd* ?*arg arg* ...?

Description

This command arranges for *cmd* (and *arg*s) to be executed in the application named by *app*. It returns the result or error from that command execution. *App* may be the name of any application whose main window is on the display containing the sender's main window; it need not be within the same process. If no *arg* arguments are present, then the command to be executed is contained entirely within the *cmd* argument. If one or more *arg*s are present, they are concatenated to form the command to be executed, just as for the eval command.

If the initial arguments of the command begin with "-" they are treated as options. The following options are currently defined:

-async
: Requests asynchronous invocation. In this case the **send** command will complete immediately without waiting for *cmd* to complete in the target application; no result will be available and errors in the sent command will be ignored. If the target application is in the same process as the sending application then the **-async** option is ignored.

-displayof *pathName*
: Specifies that the target application's main window is on the display of the window given by *pathName*, instead of the display containing the application's main window.

--

Serves no purpose except to terminate the list of options. This option is needed only if *app* could contain a leading "-" character.

Application names

The name of an application is set initially from the name of the program or script that created the application. You can query and change the name of an application with the tk appname command.

Disabling sends

If the **send** command is removed from an application (e.g. with the command rename **send {}**) then the application will not respond to incoming send requests anymore, nor will it be able to issue outgoing requests. Communication can be reenabled by invoking the tk appname command.

Security

The **send** command is potentially a serious security loophole. On Unix, any application that can connect to your X server can send scripts to your applications. These incoming scripts can use Tcl to read and write your files and invoke subprocesses under your name. Host-based access control such as that provided by **xhost** is particularly insecure, since it allows anyone with an account on particular hosts to connect to your server, and if disabled it allows anyone anywhere to connect to your server. In order to provide at least a small amount of security, Tk checks the access control being used by the server and rejects incoming sends unless (a) **xhost**-style access control is enabled (i.e. only certain hosts can establish connections) and (b) the list of enabled hosts is empty. This means that applications cannot connect to your server unless they use some other form of authorization such as that provide by **xauth**. Under Windows, **send** is currently disabled. Most of the functionality is provided by the dde command instead.

Example

This script fragment can be used to make an application that only runs once on a particular display.

```
if {[tk appname FoobarApp] ne "FoobarApp"} {
    send -async FoobarApp RemoteStart $argv
    exit
}
# The command that will be called remotely, which raises
# the application main window and opens the requested files
proc RemoteStart args {
    raise .
    foreach filename $args {
        OpenFile $filename
    }
}
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

spinbox

Keywords: spinbox, entry, widget

Name

spinbox - Create and manipulate 'spinbox' value spinner widgets

Synopsis

spinbox *pathName* ?*options*?

Standard options

-activebackground, **-highlightthickness**, **-repeatinterval**, **-background**, **-insertbackground**, **-selectbackground**, **-borderwidth**, **-insertborderwidth**, **-selectborderwidth**, **-cursor**, **-insertontime**, **-selectforeground**, **-exportselection**, **-insertwidth**, **-takefocus**, **-font**, **-insertofftime**, **-textvariable**, **-foreground**, **-justify**, **-xscrollcommand**, **-highlightbackground**, **-relief**, **-highlightcolor**, **-repeatdelay**

See the options manual entry for details on the standard options.

Widget-specific options

-buttonbackground (buttonBackground/Background)
The background color to be used for the spin buttons.

-buttoncursor (buttonCursor/Cursor)
The cursor to be used when over the spin buttons. If this is empty (the default), a default cursor will be used.

-buttondownrelief (buttonDownRelief/Relief)
The relief to be used for the upper spin button.

-buttonuprelief (buttonUpRelief/Relief)
The relief to be used for the lower spin button.

-command (command/Command)
Specifies a Tcl command to invoke whenever a spinbutton is invoked. The command recognizes several percent substitutions: **%W** for the widget path, **%s** for the current value of the widget, and **%d** for the direction of the button pressed (**up** or **down**).

-disabledbackground (disabledBackground/DisabledBackground)
Specifies the background color to use when the spinbox is disabled. If this option is the empty string, the normal background color is used.

-disabledforeground (disabledForeground/DisabledForeground)
Specifies the foreground color to use when the spinbox is disabled. If this option is the empty string, the normal foreground color is used.

-format (format**/Format)**
Specifies an alternate format to use when setting the string value when using the **-from** and **-to** range. This must be a format specifier of the form **%.f**, as it will format a floating-point number.

-from (from/From)
A floating-point value corresponding to the lowest value for a spinbox, to be used in conjunction with **-to** and **-increment**. When all are specified correctly, the spinbox will use these values to control its contents. This value must be less than the **-to** option. If **-values** is

specified, it supersedes this option.

-invalidcommand or -invcmd (invalidCommand/InvalidCommand)
Specifies a script to eval when **-validatecommand** returns 0. Setting it to an empty string disables this feature (the default). The best use of this option is to set it to *bell*. See Validation below for more information.

-increment (increment/Increment)
A floating-point value specifying the increment. When used with **-from** and **-to**, the value in the widget will be adjusted by **-increment** when a spin button is pressed (up adds the value, down subtracts the value).

-readonlybackground (readonlyBackground/ReadonlyBackground)
Specifies the background color to use when the spinbox is readonly. If this option is the empty string, the normal background color is used.

-state (state/State)
Specifies one of three states for the spinbox: **normal**, **disabled**, or **readonly**. If the spinbox is readonly, then the value may not be changed using widget commands and no insertion cursor will be displayed, even if the input focus is in the widget; the contents of the widget may still be selected. If the spinbox is disabled, the value may not be changed, no insertion cursor will be displayed, the contents will not be selectable, and the spinbox may be displayed in a different color, depending on the values of the **-disabledforeground** and **-disabledbackground** options.

-to (to/To)
A floating-point value corresponding to the highest value for the spinbox, to be used in conjunction with **-from** and **-increment**. When all are specified correctly, the spinbox will use these values to control its contents. This value must be greater than the **-from** option. If **-values** is specified, it supersedes this option.

-validate (validate/Validate)
Specifies the mode in which validation should operate: **none**, **focus**, **focusin**, **focusout**, **key**, or **all**. It defaults to **none**. When you want validation, you must explicitly state which mode you wish to use. See Validation below for more.

-validatecommand or -vcmd (validateCommand/ValidateCommand)
Specifies a script to evaluate when you want to validate the input in the widget. Setting it to an empty string disables this feature (the default). Validation occurs according to the value of **-validate**. This command must return a valid Tcl boolean value. If it returns 0 (or the valid Tcl boolean equivalent) then the value of the widget will not change and the **-invalidcommand** will be evaluated if it is set. If it returns 1, then value will be changed. See Validation below for more information.

-values (values/Values)
Must be a proper list value. If specified, the spinbox will use these values as to control its contents, starting with the first value. This option has precedence over the **-from** and **-to** range.

-width (width/Width)
Specifies an integer value indicating the desired width of the spinbox window, in average-size characters of the widget's font. If the value is less than or equal to zero, the widget picks a size just large enough to hold its current text.

-wrap (wrap/wrap)

Must be a proper boolean value. If on, the spinbox will wrap around the values of data in the widget.

Description

The **spinbox** command creates a new window (given by the *pathName* argument) and makes it into a spinbox widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the spinbox such as its colors, font, and relief. The **spinbox** command returns its *pathName* argument. At the time this command is invoked, there must not exist a window named *pathName*, but *pathName*'s parent must exist.

A **spinbox** is an extended entry widget that allows he user to move, or spin, through a fixed set of ascending or descending values such as times or dates in addition to editing the value as in an entry. When first created, a spinbox's string is empty. A portion of the spinbox may be selected as described below. If a spinbox is exporting its selection (see the **-exportselection** option), then it will observe the standard protocols for handling the selection; spinbox selections are available as type **STRING**. Spinboxes also observe the standard Tk rules for dealing with the input focus. When a spinbox has the input focus it displays an *insertion cursor* to indicate where new characters will be inserted.

Spinboxes are capable of displaying strings that are too long to fit entirely within the widget's window. In this case, only a portion of the string will be displayed; commands described below may be used to change the view in the window. Spinboxes use the standard **-xscrollcommand** mechanism for interacting with scrollbars (see the description of the **-xscrollcommand** option for details). They also support scanning, as described below.

Validation

Validation works by setting the **-validatecommand** option to a script which will be evaluated according to the **-validate** option as follows:

none
Default. This means no validation will occur.

focus
The **-validatecommand** will be called when the spinbox receives or loses focus.

focusin
The **-validatecommand** will be called when the spinbox receives focus.

focusout
The **-validatecommand** will be called when the spinbox loses focus.

key
The **-validatecommand** will be called when the spinbox is edited.

all
The **-validatecommand** will be called for all above conditions.

It is possible to perform percent substitutions on the **-validatecommand** and **-invalidcommand** scripts, just as you would in a bind script. The following substitutions are recognized:

%d
Type of action: 1 for **insert**, 0 for **delete**, or -1 for focus, forced or textvariable validation.

%i

Index of char string to be inserted/deleted, if any, otherwise -1.

%P
The value of the spinbox should edition occur. If you are configuring the spinbox widget to have a new textvariable, this will be the value of that textvariable.

%s
The current value of spinbox before edition.

%S
The text string being inserted/deleted, if any. Otherwise it is an empty string.

%v
The type of validation currently set.

%V
The type of validation that triggered the callback (key, focusin, focusout, forced).

%W
The name of the spinbox widget.

In general, the **-textvariable** and **-validatecommand** can be dangerous to mix. Any problems have been overcome so that using the **-validatecommand** will not interfere with the traditional behavior of the spinbox widget. Using the **-textvariable** for read-only purposes will never cause problems. The danger comes when you try set the **-textvariable** to something that the **-validatecommand** would not accept, which causes **-validate** to become **none** (the **-invalidcommand** will not be triggered). The same happens when an error occurs evaluating the **-validatecommand**.

Primarily, an error will occur when the **-validatecommand** or **-invalidcommand** encounters an error in its script while evaluating or **-validatecommand** does not return a valid Tcl boolean value. The **-validate** option will also set itself to **none** when you edit the spinbox widget from within either the **-validatecommand** or the **-invalidcommand**. Such editions will override the one that was being validated. If you wish to edit the value of the widget during validation and still have the **-validate** option set, you should include the command

```
%W config -validate %v
```

in the **-validatecommand** or **-invalidcommand** (whichever one you were editing the spinbox widget from). It is also recommended to not set an associated **-textvariable** during validation, as that can cause the spinbox widget to become out of sync with the **-textvariable**.

Also, the **-validate** option will set itself to **none** when the spinbox value gets changed because of adjustment of **-from** or **-to** and the **-validatecommand** returns false. For instance

```
spinbox pathName -from 1 -to 10 -validate all -vcmd {return 0}
```

will in fact set the **-validate** option to **none** because the default value for the spinbox gets changed (due to the **-from** and **-to** options) to a value not accepted by the validation script.

Moreover, forced validation is performed when invoking any spinbutton of the spinbox. If the validation script returns false in this situation, then the **-validate** option will be automatically set to **none**.

Widget command

The **spinbox** command creates a new Tcl command whose name is *pathName*. This command

may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

Option and the *arg*s determine the exact behavior of the command.

Indices

Many of the widget commands for spinboxes take one or more indices as arguments. An index specifies a particular character in the spinbox's string, in any of the following ways:

number
: Specifies the character as a numerical index, where 0 corresponds to the first character in the string.

anchor
: Indicates the anchor point for the selection, which is set with the **select from** and **select adjust** widget commands.

end
: Indicates the character just after the last one in the spinbox's string. This is equivalent to specifying a numerical index equal to the length of the spinbox's string.

insert
: Indicates the character adjacent to and immediately following the insertion cursor.

sel.first
: Indicates the first character in the selection. It is an error to use this form if the selection is not in the spinbox window.

sel.last
: Indicates the character just after the last one in the selection. It is an error to use this form if the selection is not in the spinbox window.

@*number*
: In this form, *number* is treated as an x-coordinate in the spinbox's window; the character spanning that x-coordinate is used. For example, "**@0**" indicates the left-most character in the window.

Abbreviations may be used for any of the forms above, e.g. "**e**" or "**sel.f**". In general, out-of-range indices are automatically rounded to the nearest legal value.

Subcommands

The following commands are possible for spinbox widgets:

pathName* bbox *index
: Returns a list of four numbers describing the bounding box of the character given by *index*. The first two elements of the list give the x and y coordinates of the upper-left corner of the screen area covered by the character (in pixels relative to the widget) and the last two elements give the width and height of the character, in pixels. The bounding box may refer to a region outside the visible area of the window.

pathName* cget *option
: Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **spinbox** command.

***pathName* configure ?*option*? ?*value option value ...*?**
: Query or modify the configuration options of the widget. If no *option* is specified, returns a list

describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **spinbox** command.

***pathName* delete *first* ?*last*?**
Delete one or more elements of the spinbox. *First* is the index of the first character to delete, and *last* is the index of the character just after the last one to delete. If *last* is not specified it defaults to *first*+1, i.e. a single character is deleted. This command returns an empty string.

***pathName* get**
Returns the spinbox's string.

pathName* icursor *index
Arrange for the insertion cursor to be displayed just before the character given by *index*. Returns an empty string.

pathName* identify *x y
Returns the name of the window element corresponding to coordinates *x* and *y* in the spinbox. Return value is one of: **none**, **buttondown**, **buttonup**, entry.

pathName* index *index
Returns the numerical index corresponding to *index*.

pathName* insert *index string
Insert the characters of *string* just before the character indicated by *index*. Returns an empty string.

pathName* invoke *element
Causes the specified element, either **buttondown** or **buttonup**, to be invoked, triggering the action associated with it.

pathName* scan *option args
This command is used to implement scanning on spinboxes. It has two forms, depending on *option*:

pathName* scan mark *x
Records *x* and the current view in the spinbox window; used in conjunction with later **scan dragto** commands. Typically this command is associated with a mouse button press in the widget. It returns an empty string.

pathName* scan dragto *x
This command computes the difference between its *x* argument and the *x* argument to the last **scan mark** command for the widget. It then adjusts the view left or right by 10 times the difference in x-coordinates. This command is typically associated with mouse motion events in the widget, to produce the effect of dragging the spinbox at high speed through the window. The return value is an empty string.

pathName* selection *option arg
This command is used to adjust the selection within a spinbox. It has several forms, depending on *option*:

pathName* selection adjust *index

Locate the end of the selection nearest to the character given by *index*, and adjust that end of the selection to be at *index* (i.e. including but not going beyond *index*). The other end of the selection is made the anchor point for future **select to** commands. If the selection is not currently in the spinbox, then a new selection is created to include the characters between *index* and the most recent selection anchor point, inclusive. Returns an empty string.

***pathName* selection clear**
Clear the selection if it is currently in this widget. If the selection is not in this widget then the command has no effect. Returns an empty string.

***pathName* selection element ?*element*?**
Sets or gets the currently selected element. If a spinbutton element is specified, it will be displayed depressed.

pathName* selection from *index
Set the selection anchor point to just before the character given by *index*. Does not change the selection. Returns an empty string.

***pathName* selection present**
Returns 1 if there is are characters selected in the spinbox, 0 if nothing is selected.

pathName* selection range *start end
Sets the selection to include the characters starting with the one indexed by *start* and ending with the one just before *end*. If *end* refers to the same character as *start* or an earlier one, then the spinbox's selection is cleared.

pathName* selection to *index
If *index* is before the anchor point, set the selection to the characters from *index* up to but not including the anchor point. If *index* is the same as the anchor point, do nothing. If *index* is after the anchor point, set the selection to the characters from the anchor point up to but not including *index*. The anchor point is determined by the most recent **select from** or **select adjust** command in this widget. If the selection is not in this widget then a new selection is created using the most recent anchor point specified for the widget. Returns an empty string.

***pathName* set ?*string*?**
If *string* is specified, the spinbox will try and set it to this value, otherwise it just returns the spinbox's string. If validation is on, it will occur when setting the string.

***pathName* validate**
This command is used to force an evaluation of the **-validatecommand** independent of the conditions specified by the **-validate** option. This is done by temporarily setting the **-validate** option to **all**. It returns 0 or 1.

pathName* xview *args
This command is used to query and change the horizontal position of the text in the widget's window. It can take any of the following forms:

***pathName* xview**
Returns a list containing two elements. Each element is a real fraction between 0 and 1; together they describe the horizontal span that is visible in the window. For example, if the first element is .2 and the second element is .6, 20% of the spinbox's text is off-screen to the left, the middle 40% is visible in the window, and 40% of the text is off-screen to the

right. These are the same values passed to scrollbars via the **-xscrollcommand** option.

pathName* xview *index
Adjusts the view in the window so that the character given by *index* is displayed at the left edge of the window.

pathName* xview moveto *fraction
Adjusts the view in the window so that the character *fraction* of the way through the text appears at the left edge of the window. *Fraction* must be a fraction between 0 and 1.

pathName* xview scroll *number what
This command shifts the view in the window left or right according to *number* and *what*. *Number* must be an integer. *What* must be either **units** or **pages** or an abbreviation of one of these. If *what* is **units**, the view adjusts left or right by *number* average-width characters on the display; if it is **pages** then the view adjusts by *number* screenfuls. If *number* is negative then characters farther to the left become visible; if it is positive then characters farther to the right become visible.

Default bindings

Tk automatically creates class bindings for spinboxes that give them the following default behavior. In the descriptions below, "word" refers to a contiguous group of letters, digits, or "_" characters, or any single character other than these.

1. Clicking mouse button 1 positions the insertion cursor just before the character underneath the mouse cursor, sets the input focus to this widget, and clears any selection in the widget. Dragging with mouse button 1 strokes out a selection between the insertion cursor and the character under the mouse.
2. Double-clicking with mouse button 1 selects the word under the mouse and positions the insertion cursor at the beginning of the word. Dragging after a double click will stroke out a selection consisting of whole words.
3. Triple-clicking with mouse button 1 selects all of the text in the spinbox and positions the insertion cursor before the first character.
4. The ends of the selection can be adjusted by dragging with mouse button 1 while the Shift key is down; this will adjust the end of the selection that was nearest to the mouse cursor when button 1 was pressed. If the button is double-clicked before dragging then the selection will be adjusted in units of whole words.
5. Clicking mouse button 1 with the Control key down will position the insertion cursor in the spinbox without affecting the selection.
6. If any normal printing characters are typed in a spinbox, they are inserted at the point of the insertion cursor.
7. The view in the spinbox can be adjusted by dragging with the middle mouse button (button 2, or button 3 in TkAqua). If the middle mouse button is clicked without moving the mouse, the selection is copied into the spinbox at the position of the mouse cursor.
8. If the mouse is dragged out of the spinbox on the left or right sides while button 1 is pressed, the spinbox will automatically scroll to make more text visible (if there is more text off-screen on the side where the mouse left the window).
9. The Left and Right keys move the insertion cursor one character to the left or right; they also clear any selection in the spinbox and set the selection anchor. If Left or Right is typed with the Shift key down, then the insertion cursor moves and the selection is extended to include

the new character. Control-Left and Control-Right move the insertion cursor by words, and Control-Shift-Left and Control-Shift-Right move the insertion cursor by words and also extend the selection. Control-b and Control-f behave the same as Left and Right, respectively. Meta-b and Meta-f behave the same as Control-Left and Control-Right, respectively.

10. The Home key, or Control-a, will move the insertion cursor to the beginning of the spinbox and clear any selection in the spinbox. Shift-Home moves the insertion cursor to the beginning of the spinbox and also extends the selection to that point.
11. The End key, or Control-e, will move the insertion cursor to the end of the spinbox and clear any selection in the spinbox. Shift-End moves the cursor to the end and extends the selection to that point.
12. The Select key and Control-Space set the selection anchor to the position of the insertion cursor. They do not affect the current selection. Shift-Select and Control-Shift-Space adjust the selection to the current position of the insertion cursor, selecting from the anchor to the insertion cursor if there was not any selection previously.
13. Control-/ selects all the text in the spinbox.
14. Control-\ clears any selection in the spinbox.
15. The F16 key (labelled Copy on many Sun workstations) or Meta-w copies the selection in the widget to the clipboard, if there is a selection.
16. The F20 key (labelled Cut on many Sun workstations) or Control-w copies the selection in the widget to the clipboard and deletes the selection. If there is no selection in the widget then these keys have no effect.
17. The F18 key (labelled Paste on many Sun workstations) or Control-y inserts the contents of the clipboard at the position of the insertion cursor.
18. The Delete key deletes the selection, if there is one in the spinbox. If there is no selection, it deletes the character to the right of the insertion cursor.
19. The BackSpace key and Control-h delete the selection, if there is one in the spinbox. If there is no selection, it deletes the character to the left of the insertion cursor.
20. Control-d deletes the character to the right of the insertion cursor.
21. Meta-d deletes the word to the right of the insertion cursor.
22. Control-k deletes all the characters to the right of the insertion cursor.
23. Control-t reverses the order of the two characters to the right of the insertion cursor.

If the spinbox is disabled using the **-state** option, then the spinbox's view can still be adjusted and text in the spinbox can still be selected, but no insertion cursor will be displayed and no text modifications will take place.

The behavior of spinboxes can be changed by defining new bindings for individual widgets or by redefining the class bindings.

Copyright (c) 2000 Jeffrey Hobbs.
Copyright (c) 2000 Ajuba Solutions.

text

Keywords: text, widget, tkvars

Name

text, tk_textCopy, tk_textCut, tk_textPaste - Create and manipulate 'text' hypertext editing widgets

Synopsis

text *pathName* ?*options*?
tk_textCopy *pathName*
tk_textCut *pathName*
tk_textPaste *pathName*

Standard options

-background, **-highlightthickness**, **-relief**, **-borderwidth**, **-insertbackground**, **-selectbackground**, **-cursor**, **-insertborderwidth**, **-selectborderwidth**, **-exportselection**, **-insertofftime**, **-selectforeground**, **-font**, **-insertontime**, **-setgrid**, **-foreground**, **-insertwidth**, **-takefocus**, **-highlightbackground**, **-padx**, **-xscrollcommand**, **-highlightcolor**, **-pady**, **-yscrollcommand**

See the options manual entry for details on the standard options.

Widget-specific options

-autoseparators (autoSeparators/AutoSeparators)
Specifies a boolean that says whether separators are automatically inserted in the undo stack. Only meaningful when the **-undo** option is true.

-blockcursor (blockCursor/BlockCursor)
Specifies a boolean that says whether the blinking insertion cursor should be drawn as a character-sized rectangular block. If false (the default) a thin vertical line is used for the insertion cursor.

-endline (endLine/EndLine)
Specifies an integer line index representing the line of the underlying textual data store that should be just after the last line contained in the widget. This allows a text widget to reflect only a portion of a larger piece of text. Instead of an integer, the empty string can be provided to this configuration option, which will configure the widget to end at the very last line in the textual data store.

-height (height/Height)
Specifies the desired height for the window, in units of characters in the font given by the **-font** option. Must be at least one.

-inactiveselectbackground (inactiveSelectBackground/Foreground)
Specifies the colour to use for the selection (the **sel** tag) when the window does not have the input focus. If empty, **{}**, then no selection is shown when the window does not have the focus.

-insertunfocussed (insertUnfocussed/InsertUnfocussed)
Specifies how to display the insertion cursor when the widget does not have the focus. Must be none (the default) which means to not display the cursor, hollow which means to display a hollow box, or solid which means to display a solid box. Note that hollow and solid will appear very similar when the -blockcursor option is false.

-maxundo (maxUndo/MaxUndo)
Specifies the maximum number of compound undo actions on the undo stack. A zero or a negative value imply an unlimited undo stack.

-spacing1 (spacing1/Spacing1)
Requests additional space above each text line in the widget, using any of the standard forms

for screen distances. If a line wraps, this option only applies to the first line on the display. This option may be overridden with **-spacing1** options in tags.

-spacing2 (spacing2/Spacing2)
For lines that wrap (so that they cover more than one line on the display) this option specifies additional space to provide between the display lines that represent a single line of text. The value may have any of the standard forms for screen distances. This option may be overridden with **-spacing2** options in tags.

-spacing3 (spacing3/Spacing3)
Requests additional space below each text line in the widget, using any of the standard forms for screen distances. If a line wraps, this option only applies to the last line on the display. This option may be overridden with **-spacing3** options in tags.

-startline (startLine/StartLine)
Specifies an integer line index representing the first line of the underlying textual data store that should be contained in the widget. This allows a text widget to reflect only a portion of a larger piece of text. Instead of an integer, the empty string can be provided to this configuration option, which will configure the widget to start at the very first line in the textual data store.

-state (state/State)
Specifies one of two states for the text: **normal** or **disabled**. If the text is disabled then characters may not be inserted or deleted and no insertion cursor will be displayed, even if the input focus is in the widget.

-tabs (tabs/Tabs)
Specifies a set of tab stops for the window. The option's value consists of a list of screen distances giving the positions of the tab stops, each of which is a distance relative to the left edge of the widget (excluding borders, padding, etc). Each position may optionally be followed in the next list element by one of the keywords **left**, **right**, **center**, or **numeric**, which specifies how to justify text relative to the tab stop. **Left** is the default; it causes the text following the tab character to be positioned with its left edge at the tab position. **Right** means that the right edge of the text following the tab character is positioned at the tab position, and **center** means that the text is centered at the tab position. **Numeric** means that the decimal point in the text is positioned at the tab position; if there is no decimal point then the least significant digit of the number is positioned just to the left of the tab position; if there is no number in the text then the text is right-justified at the tab position. For example, "**-tabs {2c left 4c 6c center}**" creates three tab stops at two-centimeter intervals; the first two use left justification and the third uses center justification.

If the list of tab stops does not have enough elements to cover all of the tabs in a text line, then Tk extrapolates new tab stops using the spacing and alignment from the last tab stop in the list. Tab distances must be strictly positive, and must always increase from one tab stop to the next (if not, an error is thrown). The value of the **-tabs** option may be overridden by **-tabs** options in tags.

If no **-tabs** option is specified, or if it is specified as an empty list, then Tk uses default tabs spaced every eight (average size) characters. To achieve a different standard spacing, for example every 4 characters, simply configure the widget with "**-tabs "[expr {4 * [font measure $font 0]}] left" -tabstyle wordprocessor**".

-tabstyle (tabStyle/TabStyle)
Specifies how to interpret the relationship between tab stops on a line and tabs in the text of that line. The value must be **tabular** (the default) or **wordprocessor**. Note that tabs are interpreted as they are encountered in the text. If the tab style is **tabular** then the *n*'th tab character in the line's text will be associated with the *n*'th tab stop defined for that line. If the tab character's x coordinate falls to the right of the *n*'th tab stop, then a gap of a single space will be inserted as a fallback. If the tab style is **wordprocessor** then any tab character being laid out will use (and be defined by) the first tab stop to the right of the preceding characters already laid out on that line. The value of the **-tabstyle** option may be overridden by **-tabstyle** options in tags.

-undo (undo/Undo)
Specifies a boolean that says whether the undo mechanism is active or not.

-width (width/Width)
Specifies the desired width for the window in units of characters in the font given by the **-font** option. If the font does not have a uniform width then the width of the character "0" is used in translating from character units to screen units.

-wrap (wrap/Wrap)
Specifies how to handle lines in the text that are too long to be displayed in a single line of the text's window. The value must be **none** or **char** or **word**. A wrap mode of **none** means that each line of text appears as exactly one line on the screen; extra characters that do not fit on the screen are not displayed. In the other modes each line of text will be broken up into several screen lines if necessary to keep all the characters visible. In **char** mode a screen line break may occur after any character; in **word** mode a line break will only be made at word boundaries.

Description

The **text** command creates a new window (given by the *pathName* argument) and makes it into a text widget. Additional options, described above, may be specified on the command line or in the option database to configure aspects of the text such as its default background color and relief. The **text** command returns the path name of the new window.

A text widget displays one or more lines of text and allows that text to be edited. Text widgets support four different kinds of annotations on the text, called tags, marks, embedded windows or embedded images. Tags allow different portions of the text to be displayed with different fonts and colors. In addition, Tcl commands can be associated with tags so that scripts are invoked when particular actions such as keystrokes and mouse button presses occur in particular ranges of the text. See Tags below for more details.

The second form of annotation consists of floating markers in the text called "marks". Marks are used to keep track of various interesting positions in the text as it is edited. See Marks below for more details.

The third form of annotation allows arbitrary windows to be embedded in a text widget. See Embedded windows below for more details.

The fourth form of annotation allows Tk images to be embedded in a text widget. See Embedded images below for more details.

The text widget also has a built-in undo/redo mechanism. See The undo mechanism below for

more details.

The text widget allows for the creation of peer widgets. These are other text widgets which share the same underlying data (text, marks, tags, images, etc). See Peer widgets below for more details.

Indices

Many of the widget commands for texts take one or more indices as arguments. An index is a string used to indicate a particular place within a text, such as a place to insert characters or one endpoint of a range of characters to delete. Indices have the syntax

```
base modifier modifier modifier ...
```

Where *base* gives a starting point and the *modifier*s adjust the index from the starting point (e.g. move forward or backward one character). Every index must contain a *base*, but the *modifier*s are optional. Most modifiers (as documented below) allow an optional submodifier. Valid submodifiers are **any** and **display**. If the submodifier is abbreviated, then it must be followed by whitespace, but otherwise there need be no space between the submodifier and the following *modifier*. Typically the **display** submodifier adjusts the meaning of the following *modifier* to make it refer to visual or non-elided units rather than logical units, but this is explained for each relevant case below. Lastly, where *count* is used as part of a modifier, it can be positive or negative, so "*base* - -3 lines" is perfectly valid (and equivalent to "*base* +3lines").

The *base* for an index must have one of the following forms:

line*.*char
: Indicates *char*'th character on line *line*. Lines are numbered from 1 for consistency with other UNIX programs that use this numbering scheme. Within a line, characters are numbered from 0. If *char* is **end** then it refers to the newline character that ends the line.

@x,y
: Indicates the character that covers the pixel whose x and y coordinates within the text's window are *x* and *y*.

end
: Indicates the end of the text (the character just after the last newline).

mark
: Indicates the character just after the mark whose name is *mark* (see Marks for details).

***tag*.first**
: Indicates the first character in the text that has been tagged with *tag*. This form generates an error if no characters are currently tagged with *tag*.

***tag*.last**
: Indicates the character just after the last one in the text that has been tagged with *tag*. This form generates an error if no characters are currently tagged with *tag*.

pathName
: Indicates the position of the embedded window whose name is *pathName*. This form generates an error if there is no embedded window by the given name.

imageName
: Indicates the position of the embedded image whose name is *imageName*. This form generates an error if there is no embedded image by the given name.

If the *base* could match more than one of the above forms, such as a *mark* and *imageName* both having the same value, then the form earlier in the above list takes precedence. If modifiers follow the base index, each one of them must have one of the forms listed below. Keywords such as **chars** and **wordend** may be abbreviated as long as the abbreviation is unambiguous.

+ *count* ?*submodifier*? chars
Adjust the index forward by *count* characters, moving to later lines in the text if necessary. If there are fewer than *count* characters in the text after the current index, then set the index to the last index in the text. Spaces on either side of *count* are optional. If the **display** submodifier is given, elided characters are skipped over without being counted. If **any** is given, then all characters are counted. For historical reasons, if neither modifier is given then the count actually takes place in units of index positions (see Indices for details). This behaviour may be changed in a future major release, so if you need an index count, you are encouraged to use **indices** instead wherever possible.

- *count* ?*submodifier*? chars
Adjust the index backward by *count* characters, moving to earlier lines in the text if necessary. If there are fewer than *count* characters in the text before the current index, then set the index to the first index in the text (1.0). Spaces on either side of *count* are optional. If the **display** submodifier is given, elided characters are skipped over without being counted. If **any** is given, then all characters are counted. For historical reasons, if neither modifier is given then the count actually takes place in units of index positions (see Indices for details). This behavior may be changed in a future major release, so if you need an index count, you are encouraged to use **indices** instead wherever possible.

+ *count* ?*submodifier*? indices
Adjust the index forward by *count* index positions, moving to later lines in the text if necessary. If there are fewer than *count* index positions in the text after the current index, then set the index to the last index position in the text. Spaces on either side of *count* are optional. Note that an index position is either a single character or a single embedded image or embedded window. If the **display** submodifier is given, elided indices are skipped over without being counted. If **any** is given, then all indices are counted; this is also the default behaviour if no modifier is given.

- *count* ?*submodifier*? indices
Adjust the index backward by *count* index positions, moving to earlier lines in the text if necessary. If there are fewer than *count* index positions in the text before the current index, then set the index to the first index position (1.0) in the text. Spaces on either side of *count* are optional. If the **display** submodifier is given, elided indices are skipped over without being counted. If **any** is given, then all indices are counted; this is also the default behaviour if no modifier is given.

+ *count* ?*submodifier*? lines
Adjust the index forward by *count* lines, retaining the same character position within the line. If there are fewer than *count* lines after the line containing the current index, then set the index to refer to the same character position on the last line of the text. Then, if the line is not long enough to contain a character at the indicated character position, adjust the character position to refer to the last character of the line (the newline). Spaces on either side of *count* are optional. If the **display** submodifier is given, then each visual display line is counted separately. Otherwise, if **any** (or no modifier) is given, then each logical line (no matter how

many times it is visually wrapped) counts just once. If the relevant lines are not wrapped, then these two methods of counting are equivalent.

- *count* ?*submodifier*? lines
Adjust the index backward by *count* logical lines, retaining the same character position within the line. If there are fewer than *count* lines before the line containing the current index, then set the index to refer to the same character position on the first line of the text. Then, if the line is not long enough to contain a character at the indicated character position, adjust the character position to refer to the last character of the line (the newline). Spaces on either side of *count* are optional. If the **display** submodifier is given, then each visual display line is counted separately. Otherwise, if **any** (or no modifier) is given, then each logical line (no matter how many times it is visually wrapped) counts just once. If the relevant lines are not wrapped, then these two methods of counting are equivalent.

?*submodifier*? linestart
Adjust the index to refer to the first index on the line. If the **display** submodifier is given, this is the first index on the display line, otherwise on the logical line.

?*submodifier*? lineend
Adjust the index to refer to the last index on the line (the newline). If the **display** submodifier is given, this is the last index on the display line, otherwise on the logical line.

?*submodifier*? wordstart
Adjust the index to refer to the first character of the word containing the current index. A word consists of any number of adjacent characters that are letters, digits, or underscores, or a single character that is not one of these. If the **display** submodifier is given, this only examines non-elided characters, otherwise all characters (elided or not) are examined.

?*submodifier*? wordend
Adjust the index to refer to the character just after the last one of the word containing the current index. If the current index refers to the last character of the text then it is not modified. If the **display** submodifier is given, this only examines non-elided characters, otherwise all characters (elided or not) are examined.

If more than one modifier is present then they are applied in left-to-right order. For example, the index "**end - 1 chars**" refers to the next-to-last character in the text and "**insert wordstart - 1 c**" refers to the character just before the first one in the word containing the insertion cursor. Modifiers are applied one by one in this left to right order, and after each step the resulting index is constrained to be a valid index in the text widget. So, for example, the index "**1.0 -1c +1c**" refers to the index "**2.0**".

Where modifiers result in index changes by display lines, display chars or display indices, and the *base* refers to an index inside an elided tag, that base index is considered to be equivalent to the first following non-elided index.

Tags

The first form of annotation in text widgets is a tag. A tag is a textual string that is associated with some of the characters in a text. Tags may contain arbitrary characters, but it is probably best to avoid using the characters " " (space), **+**, or **-**: these characters have special meaning in indices, so tags containing them cannot be used as indices. There may be any number of tags associated with characters in a text. Each tag may refer to a single character, a range of characters, or several ranges of characters. An individual character may have any number of

tags associated with it.

A priority order is defined among tags, and this order is used in implementing some of the tag-related functions described below. When a tag is defined (by associating it with characters or setting its display options or binding commands to it), it is given a priority higher than any existing tag. The priority order of tags may be redefined using the "*pathName* **tag raise**" and "*pathName* **tag lower**" widget commands.

Tags serve three purposes in text widgets. First, they control the way information is displayed on the screen. By default, characters are displayed as determined by the **-background**, **-font**, and **-foreground** options for the text widget. However, display options may be associated with individual tags using the "*pathName* **tag configure**" widget command. If a character has been tagged, then the display options associated with the tag override the default display style. The following options are currently supported for tags:

-background *color*
: *Color* specifies the background color to use for characters associated with the tag. It may have any of the forms accepted by Tk_GetColor.

-bgstipple *bitmap*
: *Bitmap* specifies a bitmap that is used as a stipple pattern for the background. It may have any of the forms accepted by Tk_GetBitmap. If *bitmap* has not been specified, or if it is specified as an empty string, then a solid fill will be used for the background.

-borderwidth *pixels*
: *Pixels* specifies the width of a border to draw around the tag using any of the forms accepted by Tk_GetPixels. This option should be used in conjunction with the **-relief** option to provide the desired border.

-elide *boolean*
: *Elide* specifies whether the data should be elided. Elided data (characters, images, embedded windows, etc.) is not displayed and takes no space on screen, but further on behaves just as normal data.

-fgstipple *bitmap*
: *Bitmap* specifies a bitmap that is used as a stipple pattern when drawing text and other foreground information such as underlines. It may have any of the forms accepted by Tk_GetBitmap. If *bitmap* has not been specified, or if it is specified as an empty string, then a solid fill will be used.

-font *fontName*
: *FontName* is the name of a font to use for drawing characters. It may have any of the forms accepted by Tk_GetFont.

-foreground *color*
: *Color* specifies the color to use when drawing text and other foreground information such as underlines. It may have any of the forms accepted by Tk_GetColor.

-justify *justify*
: If the first non-elided character of a display line has a tag for which this option has been specified, then *justify* determines how to justify the line. It must be one of **left**, **right**, or **center**. If a line wraps, then the justification for each line on the display is determined by the first non-elided character of that display line.

-lmargin1 *pixels*

If the first non-elided character of a text line has a tag for which this option has been specified, then *pixels* specifies how much the line should be indented from the left edge of the window. *Pixels* may have any of the standard forms for screen distances. If a line of text wraps, this option only applies to the first line on the display; the **-lmargin2** option controls the indentation for subsequent lines.

-lmargin2 ***pixels***
If the first non-elided character of a display line has a tag for which this option has been specified, and if the display line is not the first for its text line (i.e., the text line has wrapped), then *pixels* specifies how much the line should be indented from the left edge of the window. *Pixels* may have any of the standard forms for screen distances. This option is only used when wrapping is enabled, and it only applies to the second and later display lines for a text line.

-lmargincolor ***color***
Color specifies the background color to use in regions that do not contain characters because they are indented by **-lmargin1** or **-lmargin2**. It may have any of the forms accepted by Tk_GetColor. If *color* has not been specified, or if it is specified as an empty string, then the color used is specified by the **-background** tag option (or, if this is also unspecified, by the **-background** widget option).

-offset ***pixels***
Pixels specifies an amount by which the text's baseline should be offset vertically from the baseline of the overall line, in pixels. For example, a positive offset can be used for superscripts and a negative offset can be used for subscripts. *Pixels* may have any of the standard forms for screen distances.

-overstrike ***boolean***
Specifies whether or not to draw a horizontal rule through the middle of characters. *Boolean* may have any of the forms accepted by Tcl_GetBoolean.

-overstrikefg ***color***
Color specifies the color to use when displaying the overstrike. It may have any of the forms accepted by Tk_GetColor. If *color* has not been specified, or if it is specified as an empty string, then the color specified by the **-foreground** tag option is used.

-relief ***relief***
Relief specifies the relief style to use for drawing the border, in any of the forms accepted by Tk_GetRelief. This option is used in conjunction with the **-borderwidth** option to enable to the desired border appearance.

-rmargin ***pixels***
If the first non-elided character of a display line has a tag for which this option has been specified, then *pixels* specifies how wide a margin to leave between the end of the line and the right edge of the window. *Pixels* may have any of the standard forms for screen distances. This option is only used when wrapping is enabled. If a text line wraps, the right margin for each line on the display is determined by the first non-elided character of that display line.

-rmargincolor ***color***
Color specifies the background color to use in regions that do not contain characters because they are indented by **-rmargin**. It may have any of the forms accepted by Tk_GetColor. If *color* has not been specified, or if it is specified as an empty string, then the color used is specified by the **-background** tag option (or, if this is also unspecified, by the **-background** widget option).

-selectbackground *color*
Color specifies the background color to use when displaying selected items. It may have any of the forms accepted by Tk_GetColor. If *color* has not been specified, or if it is specified as an empty string, then the color specified by the **-background** tag option is used.

-selectforeground *color*
Color specifies the foreground color to use when displaying selected items. It may have any of the forms accepted by Tk_GetColor. If *color* has not been specified, or if it is specified as an empty string, then the color specified by the **-foreground** tag option is used.

-spacing1 *pixels*
Pixels specifies how much additional space should be left above each text line, using any of the standard forms for screen distances. If a line wraps, this option only applies to the first line on the display.

-spacing2 *pixels*
For lines that wrap, this option specifies how much additional space to leave between the display lines for a single text line. *Pixels* may have any of the standard forms for screen distances.

-spacing3 *pixels*
Pixels specifies how much additional space should be left below each text line, using any of the standard forms for screen distances. If a line wraps, this option only applies to the last line on the display.

-tabs *tabList*
TabList specifies a set of tab stops in the same form as for the **-tabs** option for the text widget. This option only applies to a display line if it applies to the first non-elided character on that display line. If this option is specified as an empty string, it cancels the option, leaving it unspecified for the tag (the default). If the option is specified as a non-empty string that is an empty list, such as **-tags { }**, then it requests default 8-character tabs as described for the **-tags** widget option.

-tabstyle *style*
Style specifies either the *tabular* or *wordprocessor* style of tabbing to use for the text widget. This option only applies to a display line if it applies to the first non-elided character on that display line. If this option is specified as an empty string, it cancels the option, leaving it unspecified for the tag (the default).

-underline *boolean*
Boolean specifies whether or not to draw an underline underneath characters. It may have any of the forms accepted by Tcl_GetBoolean.

-underlinefg *color*
Color specifies the color to use when displaying the underline. It may have any of the forms accepted by Tk_GetColor. If *color* has not been specified, or if it is specified as an empty string, then the color specified by the **-foreground** tag option is used.

-wrap *mode*
Mode specifies how to handle lines that are wider than the text's window. This option only applies to a display line if it applies to the first non-elided character on that display line. It has the same legal values as the **-wrap** option for the text widget: **none**, **char**, or **word**. If this tag option is specified, it overrides the **-wrap** option for the text widget.

If a character has several tags associated with it, and if their display options conflict, then the

options of the highest priority tag are used. If a particular display option has not been specified for a particular tag, or if it is specified as an empty string, then that option will never be used; the next-highest-priority tag's option will used instead. If no tag specifies a particular display option, then the default style for the widget will be used.

The second purpose for tags is event bindings. You can associate bindings with a tag in much the same way you can associate bindings with a widget class: whenever particular X events occur on characters with the given tag, a given Tcl command will be executed. Tag bindings can be used to give behaviors to ranges of characters; among other things, this allows hypertext-like features to be implemented. For details, see the description of the "*pathName* **tag bind**" widget command below. Tag bindings are shared between all peer widgets (including any bindings for the special **sel** tag).

The third use for tags is in managing the selection. See The selection below. With the exception of the special **sel** tag, all tags are shared between peer text widgets, and may be manipulated on an equal basis from any such widget. The **sel** tag exists separately and independently in each peer text widget (but any tag bindings to **sel** are shared).

Marks

The second form of annotation in text widgets is a mark. Marks are used for remembering particular places in a text. They are something like tags, in that they have names and they refer to places in the file, but a mark is not associated with particular characters. Instead, a mark is associated with the gap between two characters. Only a single position may be associated with a mark at any given time. If the characters around a mark are deleted the mark will still remain; it will just have new neighbor characters. In contrast, if the characters containing a tag are deleted then the tag will no longer have an association with characters in the file. Marks may be manipulated with the "*pathName* **mark**" widget command, and their current locations may be determined by using the mark name as an index in widget commands.

Each mark also has a "gravity", which is either **left** or **right**. The gravity for a mark specifies what happens to the mark when text is inserted at the point of the mark. If a mark has left gravity, then the mark is treated as if it were attached to the character on its left, so the mark will remain to the left of any text inserted at the mark position. If the mark has right gravity, new text inserted at the mark position will appear to the left of the mark (so that the mark remains rightmost). The gravity for a mark defaults to **right**.

The name space for marks is different from that for tags: the same name may be used for both a mark and a tag, but they will refer to different things.

Two marks have special significance. First, the mark **insert** is associated with the insertion cursor, as described under The insertion cursor below. Second, the mark **current** is associated with the character closest to the mouse and is adjusted automatically to track the mouse position and any changes to the text in the widget (one exception: **current** is not updated in response to mouse motions if a mouse button is down; the update will be deferred until all mouse buttons have been released). Neither of these special marks may be deleted. With the exception of these two special marks, all marks are shared between peer text widgets, and may be manipulated on an equal basis from any peer.

Embedded windows

The third form of annotation in text widgets is an embedded window. Each embedded window

annotation causes a window to be displayed at a particular point in the text. There may be any number of embedded windows in a text widget, and any widget may be used as an embedded window (subject to the usual rules for geometry management, which require the text window to be the parent of the embedded window or a descendant of its parent).

The embedded window's position on the screen will be updated as the text is modified or scrolled, and it will be mapped and unmapped as it moves into and out of the visible area of the text widget. Each embedded window occupies one unit's worth of index space in the text widget, and it may be referred to either by the name of its embedded window or by its position in the widget's index space. If the range of text containing the embedded window is deleted then the window is destroyed. Similarly if the text widget as a whole is deleted, then the window is destroyed.

Eliding an embedded window immediately after scheduling it for creation via *pathName* **window create** *index* **-create** will prevent it from being effectively created. Uneliding an elided embedded window scheduled for creation via *pathName* **window create** *index* **-create** will automatically trigger the associated creation script. After destroying an elided embedded window, the latter won't get automatically recreated.

When an embedded window is added to a text widget with the *pathName* **window create** widget command, several configuration options may be associated with it. These options may be modified later with the *pathName* **window configure** widget command. The following options are currently supported:

-align *where*
: If the window is not as tall as the line in which it is displayed, this option determines where the window is displayed in the line. *Where* must have one of the values **top** (align the top of the window with the top of the line), **center** (center the window within the range of the line), **bottom** (align the bottom of the window with the bottom of the line's area), or **baseline** (align the bottom of the window with the baseline of the line).

-create *script*
: Specifies a Tcl script that may be evaluated to create the window for the annotation. If no **-window** option has been specified for the annotation this script will be evaluated when the annotation is about to be displayed on the screen. *Script* must create a window for the annotation and return the name of that window as its result. Two substitutions will be performed in *script* before evaluation. *%W* will be substituted by the name of the parent text widget, and *%%* will be substituted by a single *%*. If the annotation's window should ever be deleted, *script* will be evaluated again the next time the annotation is displayed.

-padx *pixels*
: *Pixels* specifies the amount of extra space to leave on each side of the embedded window. It may have any of the usual forms defined for a screen distance.

-pady *pixels*
: *Pixels* specifies the amount of extra space to leave on the top and on the bottom of the embedded window. It may have any of the usual forms defined for a screen distance.

-stretch *boolean*
: If the requested height of the embedded window is less than the height of the line in which it is displayed, this option can be used to specify whether the window should be stretched vertically to fill its line. If the **-pady** option has been specified as well, then the requested padding will be retained even if the window is stretched.

-window *pathName*
Specifies the name of a window to display in the annotation. Note that if a *pathName* has been set, then later configuring a window to the empty string will not delete the widget corresponding to the old *pathName*. Rather it will remove the association between the old *pathName* and the text widget. If multiple peer widgets are in use, it is usually simpler to use the **-create** option if embedded windows are desired in each peer.

Embedded images

The final form of annotation in text widgets is an embedded image. Each embedded image annotation causes an image to be displayed at a particular point in the text. There may be any number of embedded images in a text widget, and a particular image may be embedded in multiple places in the same text widget.

The embedded image's position on the screen will be updated as the text is modified or scrolled. Each embedded image occupies one unit's worth of index space in the text widget, and it may be referred to either by its position in the widget's index space, or the name it is assigned when the image is inserted into the text widget with *pathName* **image create**. If the range of text containing the embedded image is deleted then that copy of the image is removed from the screen.

Eliding an embedded image immediately after scheduling it for creation via *pathName* **image create** *index* **-create** will prevent it from being effectively created. Uneliding an elided embedded image scheduled for creation via *pathName* **image create** *index* **-create** will automatically trigger the associated creation script. After destroying an elided embedded image, the latter won't get automatically recreated.

When an embedded image is added to a text widget with the *pathName* **image create** widget command, a name unique to this instance of the image is returned. This name may then be used to refer to this image instance. The name is taken to be the value of the **-name** option (described below). If the **-name** option is not provided, the **-image** name is used instead. If the *imageName* is already in use in the text widget, then **#***nn* is added to the end of the *imageName* , where *nn* is an arbitrary integer. This insures the *imageName* is unique. Once this name is assigned to this instance of the image, it does not change, even though the **-image** or **-name** values can be changed with *pathName* **image configure**.

When an embedded image is added to a text widget with the *pathName* **image create** widget command, several configuration options may be associated with it. These options may be modified later with the *pathName* **image configure** widget command. The following options are currently supported:

-align *where*
If the image is not as tall as the line in which it is displayed, this option determines where the image is displayed in the line. *Where* must have one of the values **top** (align the top of the image with the top of the line), **center** (center the image within the range of the line), **bottom** (align the bottom of the image with the bottom of the line's area), or **baseline** (align the bottom of the image with the baseline of the line).

-image *image*
Specifies the name of the Tk image to display in the annotation. If *image* is not a valid Tk image, then an error is returned.

-name *ImageName*

Specifies the name by which this image instance may be referenced in the text widget. If *ImageName* is not supplied, then the name of the Tk image is used instead. If the *imageName* is already in use, *#nn* is appended to the end of the name as described above.

-padx *pixels*
Pixels specifies the amount of extra space to leave on each side of the embedded image. It may have any of the usual forms defined for a screen distance.

-pady *pixels*
Pixels specifies the amount of extra space to leave on the top and on the bottom of the embedded image. It may have any of the usual forms defined for a screen distance.

The selection

Selection support is implemented via tags. If the **-exportselection** option for the text widget is true then the **sel** tag will be associated with the selection:

1. Whenever characters are tagged with **sel** the text widget will claim ownership of the selection.
2. Attempts to retrieve the selection will be serviced by the text widget, returning all the characters with the **sel** tag.
3. If the selection is claimed away by another application or by another window within this application, then the **sel** tag will be removed from all characters in the text.
4. Whenever the **sel** tag range changes a virtual event **<>** is generated. It might also be generated when selection is affected but not actually changed. Further, multiple selection changes could happen before events can be processed leading to multiple events with the same visible selection.

The **sel** tag is automatically defined when a text widget is created, and it may not be deleted with the "*pathName* **tag delete**" widget command. Furthermore, the **-selectbackground**, **-selectborderwidth**, and **-selectforeground** options for the text widget are tied to the **-background**, **-borderwidth**, and **-foreground** options for the **sel** tag: changes in either will automatically be reflected in the other. Also the **-inactiveselectbackground** option for the text widget is used instead of **-selectbackground** when the text widget does not have the focus. This allows programmatic control over the visualization of the **sel** tag for foreground and background windows, or to have **sel** not shown at all (when **-inactiveselectbackground** is empty) for background windows. Each peer text widget has its own **sel** tag which can be separately configured and set.

The insertion cursor

The mark named **insert** has special significance in text widgets. It is defined automatically when a text widget is created and it may not be unset with the "*pathName* **mark unset**" widget command. The **insert** mark represents the position of the insertion cursor, and the insertion cursor will automatically be drawn at this point whenever the text widget has the input focus.

The modified flag

The text widget can keep track of changes to the content of the widget by means of the modified flag. Inserting or deleting text will set this flag. The flag can be queried, set and cleared programmatically as well. Whenever the flag changes state a **<>** virtual event is generated. See the *pathName* **edit modified** widget command for more details.

The undo mechanism

The text widget has an unlimited undo and redo mechanism (when the **-undo** widget option is true) which records every insert and delete action on a stack.

Boundaries (called "separators") are inserted between edit actions. The purpose of these separators is to group inserts, deletes and replaces into one compound edit action. When undoing a change everything between two separators will be undone. The undone changes are then moved to the redo stack, so that an undone edit can be redone again. The redo stack is cleared whenever new edit actions are recorded on the undo stack. The undo and redo stacks can be cleared to keep their depth under control.

Separators are inserted automatically when the **-autoseparators** widget option is true. You can insert separators programmatically as well. If a separator is already present at the top of the undo stack no other will be inserted. That means that two separators on the undo stack are always separated by at least one insert or delete action.

The **<>** virtual event is generated every time the undo stack or the redo stack becomes empty or unempty.

The undo mechanism is also linked to the modified flag. This means that undoing or redoing changes can take a modified text widget back to the unmodified state or vice versa. The modified flag will be set automatically to the appropriate state. This automatic coupling does not work when the modified flag has been set by the user, until the flag has been reset again.

See below for the *pathName* **edit** widget command that controls the undo mechanism.

Peer widgets

The text widget has a separate store of all its data concerning each line's textual contents, marks, tags, images and windows, and the undo stack.

While this data store cannot be accessed directly (i.e. without a text widget as an intermediary), multiple text widgets can be created, each of which present different views on the same underlying data. Such text widgets are known as peer text widgets.

As text is added, deleted, edited and coloured in any one widget, and as images, marks, tags are adjusted, all such changes will be reflected in all peers.

All data and markup is shared, except for a few small details. First, the **sel** tag may be set and configured (in its display style) differently for each peer. Second, each peer has its own **insert** and **current** mark positions (but all other marks are shared). Third, embedded windows, which are arbitrary other widgets, cannot be shared between peers. This means the **-window** option of embedded windows is independently set for each peer (it is advisable to use the **-create** script capabilities to allow each peer to create its own embedded windows as needed). Fourth, all of the configuration options of each peer (e.g. **-font**, etc) can be set independently, with the exception of **-undo**, **-maxundo**, **-autoseparators** (i.e. all undo, redo and modified state issues are shared).

Finally any single peer need not contain all lines from the underlying data store. When creating a peer, a contiguous range of lines (e.g. only lines 52 through 125) may be specified. This allows a peer to contain just a small portion of the overall text. The range of lines will expand and contract as text is inserted or deleted. The peer will only ever display complete lines of text (one cannot share just part of a line). If the peer's contents contracts to nothing (i.e. all complete lines in the peer widget have been deleted from another widget), then it is impossible for new

lines to be inserted. The peer will simply become an empty shell on which the background can be configured, but which will never show any content (without manual reconfiguration of the start and end lines). Note that a peer which does not contain all of the underlying data store still has indices numbered from "1.0" to "end". It is simply that those indices reflect a subset of the total data, and data outside the contained range is not accessible to the peer. This means that the command *peerName* **index end** may return quite different values in different peers. Similarly, commands like *peerName* **tag ranges** will not return index ranges outside that which is meaningful to the peer. The configuration options **-startline** and **-endline** may be used to control how much of the underlying data is contained in any given text widget.

Note that peers are really peers. Deleting the "original" text widget will not cause any other peers to be deleted, or otherwise affected.

See below for the *pathName* **peer** widget command that controls the creation of peer widgets.

Asynchronous update of line heights

In order to maintain a responsive user-experience, the text widget calculates lines metrics (line heights in pixels) asynchronously. Because of this, some commands of the text widget may return wrong results if the asynchronous calculations are not finished at the time of calling. This applies to *pathName* **count -ypixels** and *pathName* **yview**.

Again for performance reasons, it would not be appropriate to let these commands always wait for the end of the update calculation each time they are called. In most use cases of these commands a more or less inaccurate result does not really matter compared to execution speed.

In case accurate result is needed (and if the text widget is managed by a geometry manager), one can resort to *pathName* **sync** and *pathName* **pendingsync** to control the synchronization of the view of text widgets.

The **<>** virtual event fires when the line heights of the text widget become obsolete (due to some editing command or configuration change), and again when the internal data of the text widget are back in sync with the widget view. The detail field (%d substitution) is either true (when the widget is in sync) or false (when it is not).

pathName **sync**, *pathName* **pendingsync** and **<>** apply to each text widget independently of its peers.

Examples of use:

```
## Example 1:
# immediately complete line metrics at any cost (GUI unresponsive)
$w sync
$w yview moveto $fraction

## Example 2:
# synchronously wait for up-to-date line metrics (GUI responsive)
# before executing the scheduled command, but don't block execution
    flow
$w sync -command [list $w yview moveto $fraction]

## Example 3:
# init
```

```
set yud($w) 0
proc updateaction w {
set ::yud($w) 1
# any other update action here...
}
# runtime, synchronously wait for up-to-date line metrics (GUI
    responsive)
$w sync -command [list updateaction $w]
vwait yud($w)
$w yview moveto $fraction

## Example 4:
# init
set todo($w) {}
proc updateaction w {
foreach cmd $::todo($w) {uplevel #0 $cmd}
set todo($w) {}
}
# runtime
lappend todo($w) [list $w yview moveto $fraction]
$w sync -command [list updateaction $w]

## Example 5:
# init
set todo($w) {}
bind $w <<WidgetViewSync>> {
if {%d} {
foreach cmd $todo(%W) {eval $cmd}
set todo(%W) {}
}
}
# runtime
if {![$w pendingsync]} {
$w yview moveto $fraction
} else {
lappend todo($w) [list $w yview moveto $fraction]
}
```

Widget command

The **text** command creates a new Tcl command whose name is the same as the path name of the text's window. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

PathName is the name of the command, which is the same as the text widget's path name. *Option* and the *arg*s determine the exact behavior of the command. The following commands are

possible for text widgets:

pathName* bbox *index
Returns a list of four elements describing the screen area of the character given by *index*. The first two elements of the list give the x and y coordinates of the upper-left corner of the area occupied by the character, and the last two elements give the width and height of the area. If the character is only partially visible on the screen, then the return value reflects just the visible part. If the character is not visible on the screen then the return value is an empty list.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **text** command.

pathName* compare *index1 op index2
Compares the indices given by *index1* and *index2* according to the relational operator given by *op*, and returns 1 if the relationship is satisfied and 0 if it is not. *Op* must be one of the operators <, <=, ==, >=, >, or!=. If *op* is == then 1 is returned if the two indices refer to the same character, if *op* is < then 1 is returned if *index1* refers to an earlier character in the text than *index2*, and so on.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **text** command.

pathName* count ?*options*? *index1 index2
Counts the number of relevant things between the two indices. If *index1* is after *index2*, the result will be a negative number (and this holds for each of the possible options). The actual items which are counted depend on the options given. The result is a list of integers, one for the result of each counting option given. Valid counting options are **-chars**, **-displaychars**, **-displayindices**, **-displaylines**, **-indices**, **-lines**, **-xpixels** and **-ypixels**. The default value, if no option is specified, is **-indices**. There is an additional possible option **-update** which is a modifier. If given (and if the text widget is managed by a geometry manager), then all subsequent options ensure that any possible out of date information is recalculated. This currently only has any effect for the **-ypixels** count (which, if **-update** is not given, will use the text widget's current cached value for each line). This **-update** option is obsoleted by *pathName* **sync**, *pathName* **pendingsync** and **<>**. The count options are interpreted as follows:

-chars
count all characters, whether elided or not. Do not count embedded windows or images.

-displaychars
count all non-elided characters.

-displayindices
count all non-elided characters, windows and images.

-displaylines

count all display lines (i.e. counting one for each time a line wraps) from the line of the first index up to, but not including the display line of the second index. Therefore if they are both on the same display line, zero will be returned. By definition displaylines are visible and therefore this only counts portions of actual visible lines.

-indices
count all characters and embedded windows or images (i.e. everything which counts in text-widget index space), whether they are elided or not.

-lines
count all logical lines (irrespective of wrapping) from the line of the first index up to, but not including the line of the second index. Therefore if they are both on the same line, zero will be returned. Logical lines are counted whether they are currently visible (non-elided) or not.

-xpixels
count the number of horizontal pixels from the first pixel of the first index to (but not including) the first pixel of the second index. To count the total desired width of the text widget (assuming wrapping is not enabled), first find the longest line and then use ".text count -xpixels "${line}.0" "${line}.0 lineend"".

-ypixels
count the number of vertical pixels from the first pixel of the first index to (but not including) the first pixel of the second index. If both indices are on the same display line, zero will be returned. To count the total number of vertical pixels in the text widget, use ".text count -ypixels 1.0 end", and to ensure this is up to date, use ".text count -update -ypixels 1.0 end".

The command returns a positive or negative integer corresponding to the number of items counted between the two indices. One such integer is returned for each counting option given, so a list is returned if more than one option was supplied. For example ".text count -xpixels -ypixels 1.3 4.5" is perfectly valid and will return a list of two elements.

***pathName* debug ?*boolean*?**
If *boolean* is specified, then it must have one of the true or false values accepted by Tcl_GetBoolean. If the value is a true one then internal consistency checks will be turned on in the B-tree code associated with text widgets. If *boolean* has a false value then the debugging checks will be turned off. In either case the command returns an empty string. If *boolean* is not specified then the command returns **on** or **off** to indicate whether or not debugging is turned on. There is a single debugging switch shared by all text widgets: turning debugging on or off in any widget turns it on or off for all widgets. For widgets with large amounts of text, the consistency checks may cause a noticeable slow-down.

When debugging is turned on, the drawing routines of the text widget set the global variables **tk_textRedraw** and **tk_textRelayout** to the lists of indices that are redrawn. The values of these variables are tested by Tk's test suite.

***pathName* delete *index1* ?*index2 ...*?**
Delete a range of characters from the text. If both *index1* and *index2* are specified, then delete all the characters starting with the one given by *index1* and stopping just before *index2* (i.e. the character at *index2* is not deleted). If *index2* does not specify a position later in the text than *index1* then no characters are deleted. If *index2* is not specified then the

single character at *index1* is deleted. Attempts to delete characters in a way that would leave the text without a newline as the last character will be tweaked by the text widget to avoid this. In particular, deletion of complete lines of text up to the end of the text will also delete the newline character just before the deleted block so that it is replaced by the new final newline of the text widget. The command returns an empty string. If more indices are given, multiple ranges of text will be deleted. All indices are first checked for validity before any deletions are made. They are sorted and the text is removed from the last range to the first range so deleted text does not cause an undesired index shifting side-effects. If multiple ranges with the same start index are given, then the longest range is used. If overlapping ranges are given, then they will be merged into spans that do not cause deletion of text outside the given ranges due to text shifted during deletion.

pathName* dlineinfo *index

Returns a list with five elements describing the area occupied by the display line containing *index*. The first two elements of the list give the x and y coordinates of the upper-left corner of the area occupied by the line, the third and fourth elements give the width and height of the area, and the fifth element gives the position of the baseline for the line, measured down from the top of the area. All of this information is measured in pixels. If the current wrap mode is **none** and the line extends beyond the boundaries of the window, the area returned reflects the entire area of the line, including the portions that are out of the window. If the line is shorter than the full width of the window then the area returned reflects just the portion of the line that is occupied by characters and embedded windows. If the display line containing *index* is not visible on the screen then the return value is an empty list.

***pathName* dump ?*switches*? *index1* ?*index2*?**

Return the contents of the text widget from *index1* up to, but not including *index2*, including the text and information about marks, tags, and embedded windows. If *index2* is not specified, then it defaults to one character past *index1*. The information is returned in the following format:

key1 value1 index1 key2 value2 index2 ...

The possible *key* values are **text**, **mark**, **tagon**, **tagoff**, **image**, and **window**. The corresponding *value* is the text, mark name, tag name, image name, or window name. The *index* information is the index of the start of the text, mark, tag transition, image or window. One or more of the following switches (or abbreviations thereof) may be specified to control the dump:

-all

Return information about all elements: text, marks, tags, images and windows. This is the default.

-command *command*

Instead of returning the information as the result of the dump operation, invoke the *command* on each element of the text widget within the range. The command has three arguments appended to it before it is evaluated: the *key*, *value*, and *index*.

-image

Include information about images in the dump results.

-mark

Include information about marks in the dump results.

-tag

Include information about tag transitions in the dump results. Tag information is returned as **tagon** and **tagoff** elements that indicate the begin and end of each range of each tag, respectively.

-text
Include information about text in the dump results. The value is the text up to the next element or the end of range indicated by *index2*. A text element does not span newlines. A multi-line block of text that contains no marks or tag transitions will still be dumped as a set of text segments that each end with a newline. The newline is part of the value.

-window
Include information about embedded windows in the dump results. The value of a window is its Tk pathname, unless the window has not been created yet. (It must have a create script.) In this case an empty string is returned, and you must query the window by its index position to get more information.

***pathName* edit *option* ?*arg arg ...*?**
This command controls the undo mechanism and the modified flag. The exact behavior of the command depends on the *option* argument that follows the **edit** argument. The following forms of the command are currently supported:

***pathName* edit canredo**
Returns a boolean true if redo is possible, i.e. when the redo stack is not empty. Otherwise returns false.

***pathName* edit canundo**
Returns a boolean true if undo is possible, i.e. when the undo stack is not empty. Otherwise returns false.

***pathName* edit modified ?*boolean*?**
If *boolean* is not specified, returns the modified flag of the widget. The insert, delete, edit undo and edit redo commands or the user can set or clear the modified flag. If *boolean* is specified, sets the modified flag of the widget to *boolean*.

***pathName* edit redo**
When the **-undo** option is true, reapplies the last undone edits provided no other edits were done since then. Generates an error when the redo stack is empty. Does nothing when the **-undo** option is false.

***pathName* edit reset**
Clears the undo and redo stacks.

***pathName* edit separator**
Inserts a separator (boundary) on the undo stack. Does nothing when the **-undo** option is false.

***pathName* edit undo**
Undoes the last edit action when the **-undo** option is true. An edit action is defined as all the insert and delete commands that are recorded on the undo stack in between two separators. Generates an error when the undo stack is empty. Does nothing when the **-undo** option is false.

***pathName* get ?-displaychars? ?--? *index1* ?*index2 ...*?**
Return a range of characters from the text. The return value will be all the characters in the text starting with the one whose index is *index1* and ending just before the one whose index

is *index2* (the character at *index2* will not be returned). If *index2* is omitted then the single character at *index1* is returned. If there are no characters in the specified range (e.g. *index1* is past the end of the file or *index2* is less than or equal to *index1*) then an empty string is returned. If the specified range contains embedded windows, no information about them is included in the returned string. If multiple index pairs are given, multiple ranges of text will be returned in a list. Invalid ranges will not be represented with empty strings in the list. The ranges are returned in the order passed to *pathName* **get**. If the **-displaychars** option is given, then, within each range, only those characters which are not elided will be returned. This may have the effect that some of the returned ranges are empty strings.

***pathName* image *option* ?*arg arg ...*?**

This command is used to manipulate embedded images. The behavior of the command depends on the *option* argument that follows the **image** argument. The following forms of the command are currently supported:

pathName* image cget *index option

Returns the value of a configuration option for an embedded image. *Index* identifies the embedded image, and *option* specifies a particular configuration option, which must be one of the ones listed in the section Embedded images.

***pathName* image configure *index* ?*option value ...*?**

Query or modify the configuration options for an embedded image. If no *option* is specified, returns a list describing all of the available options for the embedded image at *index* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given option(s) to have the given value(s); in this case the command returns an empty string. See Embedded images for information on the options that are supported.

***pathName* image create *index* ?*option value ...*?**

This command creates a new image annotation, which will appear in the text at the position given by *index*. Any number of *option-value* pairs may be specified to configure the annotation. Returns a unique identifier that may be used as an index to refer to this image. See Embedded images for information on the options that are supported, and a description of the identifier returned.

***pathName* image names**

Returns a list whose elements are the names of all image instances currently embedded in *window*.

pathName* index *index

Returns the position corresponding to *index* in the form *line.char* where *line* is the line number and *char* is the character number. *Index* may have any of the forms described under Indices above.

***pathName* insert *index chars* ?*tagList chars tagList ...*?**

Inserts all of the *chars* arguments just before the character at *index*. If *index* refers to the end of the text (the character after the last newline) then the new text is inserted just before the last newline instead. If there is a single *chars* argument and no *tagList*, then the new text will receive any tags that are present on both the character before and the character after the insertion point; if a tag is present on only one of these characters then it will not be applied to

the new text. If *tagList* is specified then it consists of a list of tag names; the new characters will receive all of the tags in this list and no others, regardless of the tags present around the insertion point. If multiple *chars-tagList* argument pairs are present, they produce the same effect as if a separate *pathName* **insert** widget command had been issued for each pair, in order. The last *tagList* argument may be omitted.

***pathName* mark *option* ?*arg arg ...*?**

This command is used to manipulate marks. The exact behavior of the command depends on the *option* argument that follows the **mark** argument. The following forms of the command are currently supported:

***pathName* mark gravity *markName* ?*direction*?**

If *direction* is not specified, returns **left** or **right** to indicate which of its adjacent characters *markName* is attached to. If *direction* is specified, it must be **left** or **right**; the gravity of *markName* is set to the given value.

***pathName* mark names**

Returns a list whose elements are the names of all the marks that are currently set.

pathName* mark next *index

Returns the name of the next mark at or after *index*. If *index* is specified in numerical form, then the search for the next mark begins at that index. If *index* is the name of a mark, then the search for the next mark begins immediately after that mark. This can still return a mark at the same position if there are multiple marks at the same index. These semantics mean that the **mark next** operation can be used to step through all the marks in a text widget in the same order as the mark information returned by the *pathName* **dump** operation. If a mark has been set to the special **end** index, then it appears to be *after* **end** with respect to the *pathName* **mark next** operation. An empty string is returned if there are no marks after *index*.

pathName* mark previous *index

Returns the name of the mark at or before *index*. If *index* is specified in numerical form, then the search for the previous mark begins with the character just before that index. If *index* is the name of a mark, then the search for the next mark begins immediately before that mark. This can still return a mark at the same position if there are multiple marks at the same index. These semantics mean that the *pathName* **mark previous** operation can be used to step through all the marks in a text widget in the reverse order as the mark information returned by the *pathName* **dump** operation. An empty string is returned if there are no marks before *index*.

pathName* mark set *markName index

Sets the mark named *markName* to a position just before the character at *index*. If *markName* already exists, it is moved from its old position; if it does not exist, a new mark is created. This command returns an empty string.

***pathName* mark unset *markName* ?*markName markName ...*?**

Remove the mark corresponding to each of the *markName* arguments. The removed marks will not be usable in indices and will not be returned by future calls to "*pathName* **mark names**". This command returns an empty string.

pathName* peer *option args

This command is used to create and query widget peers. It has two forms, depending on *option*:

***pathName* peer create *newPathName* ?*options*?**
Creates a peer text widget with the given *newPathName*, and any optional standard configuration options (as for the *text* command). By default the peer will have the same start and end line as the parent widget, but these can be overridden with the standard configuration options.

***pathName* peer names**
Returns a list of peers of this widget (this does not include the widget itself). The order within this list is undefined.

***pathName* pendingsync**
Returns 1 if the line heights calculations are not up-to-date, 0 otherwise.

***pathName* replace *index1 index2 chars* ?*tagList chars tagList ...*?**
Replaces the range of characters between *index1* and *index2* with the given characters and tags. See the section on *pathName* **insert** for an explanation of the handling of the *tagList...* arguments, and the section on *pathName* **delete** for an explanation of the handling of the indices. If *index2* corresponds to an index earlier in the text than *index1*, an error will be generated.

The deletion and insertion are arranged so that no unnecessary scrolling of the window or movement of insertion cursor occurs. In addition the undo/redo stack are correctly modified, if undo operations are active in the text widget. The command returns an empty string.

pathName* scan *option args
This command is used to implement scanning on texts. It has two forms, depending on *option* :

pathName* scan mark *x y
Records *x* and *y* and the current view in the text window, for use in conjunction with later *pathName* **scan dragto** commands. Typically this command is associated with a mouse button press in the widget. It returns an empty string.

pathName* scan dragto *x y
This command computes the difference between its *x* and *y* arguments and the *x* and *y* arguments to the last *pathName* **scan mark** command for the widget. It then adjusts the view by 10 times the difference in coordinates. This command is typically associated with mouse motion events in the widget, to produce the effect of dragging the text at high speed through the window. The return value is an empty string.

***pathName* search ?*switches*? *pattern index* ?*stopIndex*?**
Searches the text in *pathName* starting at *index* for a range of characters that matches *pattern*. If a match is found, the index of the first character in the match is returned as result; otherwise an empty string is returned. One or more of the following switches (or abbreviations thereof) may be specified to control the search:

-forwards
The search will proceed forward through the text, finding the first matching range starting at or after the position given by *index*. This is the default.

-backwards
The search will proceed backward through the text, finding the matching range closest to *index* whose first character is before *index* (it is not allowed to be at *index*). Note that, for a

variety of reasons, backwards searches can be substantially slower than forwards searches (particularly when using **-regexp**), so it is recommended that performance-critical code use forward searches.

-exact
Use exact matching: the characters in the matching range must be identical to those in *pattern*. This is the default.

-regexp
Treat *pattern* as a regular expression and match it against the text using the rules for regular expressions (see the regexp command and the re_syntax page for details). The default matching automatically passes both the **-lineanchor** and **-linestop** options to the regexp engine (unless **-nolinestop** is used), so that *^$* match beginning and end of line, and., *[^* sequences will never match the newline character *\n*.

-nolinestop
This allows. and *[^* sequences to match the newline character *\n*, which they will otherwise not do (see the regexp command for details). This option is only meaningful if **-regexp** is also given, and an error will be thrown otherwise. For example, to match the entire text, use "*pathName* **search -nolinestop -regexp** ".*" 1.0".

-nocase
Ignore case differences between the pattern and the text.

-count *varName*
The argument following **-count** gives the name of a variable; if a match is found, the number of index positions between beginning and end of the matching range will be stored in the variable. If there are no embedded images or windows in the matching range (and there are no elided characters if **-elide** is not given), this is equivalent to the number of characters matched. In either case, the range *matchIdx* to *matchIdx + $count chars* will return the entire matched text.

-all
Find all matches in the given range and return a list of the indices of the first character of each match. If a **-count** *varName* switch is given, then *varName* is also set to a list containing one element for each successful match. Note that, even for exact searches, the elements of this list may be different, if there are embedded images, windows or hidden text. Searches with **-all** behave very similarly to the Tcl command regexp -all, in that overlapping matches are not normally returned. For example, applying an **-all** search of the pattern "\w+" against "hello there" will just match twice, once for each word, and matching "Z[a-z]+Z" against "ZooZooZoo" will just match once.

-overlap
When performing **-all** searches, the normal behaviour is that matches which overlap an already-found match will not be returned. This switch changes that behaviour so that all matches which are not totally enclosed within another match are returned. For example, applying an **-overlap** search of the pattern "\w+" against "hello there" will just match twice (i.e. no different to just **-all**), but matching "Z[a-z]+Z" against "ZooZooZoo" will now match twice. An error will be thrown if this switch is used without **-all**.

-strictlimits
When performing any search, the normal behaviour is that the start and stop limits are checked with respect to the start of the matching text. With the **-strictlimits** flag, the

entire matching range must lie inside the start and stop limits specified for the match to be valid.

-elide
Find elided (hidden) text as well. By default only displayed text is searched.

--
This switch has no effect except to terminate the list of switches: the next argument will be treated as *pattern* even if it starts with **-**.

The matching range may be within a single line of text, or run across multiple lines (if parts of the pattern can match a new-line). For regular expression matching one can use the various newline-matching features such as **$** to match the end of a line, **^** to match the beginning of a line, and to control whether**.** is allowed to match a new-line. If *stopIndex* is specified, the search stops at that index: for forward searches, no match at or after *stopIndex* will be considered; for backward searches, no match earlier in the text than *stopIndex* will be considered. If *stopIndex* is omitted, the entire text will be searched: when the beginning or end of the text is reached, the search continues at the other end until the starting location is reached again; if *stopIndex* is specified, no wrap-around will occur. This means that, for example, if the search is **-forwards** but *stopIndex* is earlier in the text than *startIndex*, nothing will ever be found. See Known bugs below for a number of minor limitations of the *pathName* **search** command.

pathName* see *index
Adjusts the view in the window so that the character given by *index* is completely visible. If *index* is already visible then the command does nothing. If *index* is a short distance out of view, the command adjusts the view just enough to make *index* visible at the edge of the window. If *index* is far out of view, then the command centers *index* in the window.

***pathName* sync ?-command *command*?**
Controls the synchronization of the view of the text widget.

***pathName* sync**
Immediately brings the line metrics up-to-date by forcing computation of any outdated line heights. The command returns immediately if there is no such outdated line heights, otherwise it returns only at the end of the computation. The command returns an empty string.

pathName* sync -command *command
Schedules *command* to be executed (by the event loop) exactly once as soon as all line heights are up-to-date. If there are no pending line metrics calculations, the scheduling is immediate. The command returns the empty string. bgerror is called on *command* failure.

***pathName* tag *option* ?*arg arg ...*?**
This command is used to manipulate tags. The exact behavior of the command depends on the *option* argument that follows the **tag** argument. The following forms of the command are currently supported:

***pathName* tag add *tagName index1* ?*index2 index1 index2 ...*?**
Associate the tag *tagName* with all of the characters starting with *index1* and ending just before *index2* (the character at *index2* is not tagged). A single command may contain any number of *index1*-*index2* pairs. If the last *index2* is omitted then the single character at *index1* is tagged. If there are no characters in the specified range (e.g. *index1* is past the

end of the file or *index2* is less than or equal to *index1*) then the command has no effect.

***pathName* tag bind *tagName* ?*sequence*? ?*script*?**

This command associates *script* with the tag given by *tagName*. Whenever the event sequence given by *sequence* occurs for a character that has been tagged with *tagName*, the script will be invoked. This widget command is similar to the **bind** command except that it operates on characters in a text rather than entire widgets. See the **bind** manual entry for complete details on the syntax of *sequence* and the substitutions performed on *script* before invoking it. If all arguments are specified then a new binding is created, replacing any existing binding for the same *sequence* and *tagName* (if the first character of *script* is "+" then *script* augments an existing binding rather than replacing it). In this case the return value is an empty string. If *script* is omitted then the command returns the *script* associated with *tagName* and *sequence* (an error occurs if there is no such binding). If both *script* and *sequence* are omitted then the command returns a list of all the sequences for which bindings have been defined for *tagName*.

The only events for which bindings may be specified are those related to the mouse and keyboard (such as **Enter**, **Leave**, **ButtonPress**, **Motion**, and **KeyPress**) or virtual events. Mouse and keyboard event bindings for a text widget respectively use the **current** and **insert** marks described under Marks above. An **Enter** event triggers for a tag when the tag first becomes present on the current character, and a **Leave** event triggers for a tag when it ceases to be present on the current character. **Enter** and **Leave** events can happen either because the **current** mark moved or because the character at that position changed. Note that these events are different than **Enter** and **Leave** events for windows. Mouse events are directed to the current character, while keyboard events are directed to the insert character. If a virtual event is used in a binding, that binding can trigger only if the virtual event is defined by an underlying mouse-related or keyboard-related event.

It is possible for the current character to have multiple tags, and for each of them to have a binding for a particular event sequence. When this occurs, one binding is invoked for each tag, in order from lowest-priority to highest priority. If there are multiple matching bindings for a single tag, then the most specific binding is chosen (see the manual entry for the **bind** command for details). continue and break commands within binding scripts are processed in the same way as for bindings created with the **bind** command.

If bindings are created for the widget as a whole using the **bind** command, then those bindings will supplement the tag bindings. The tag bindings will be invoked first, followed by bindings for the window as a whole.

pathName* tag cget *tagName option

This command returns the current value of the option named *option* associated with the tag given by *tagName*. *Option* may have any of the values accepted by the *pathName* **tag configure** widget command.

***pathName* tag configure *tagName* ?*option*? ?*value*? ?*option value ...*?**

This command is similar to the *pathName* **configure** widget command except that it modifies options associated with the tag given by *tagName* instead of modifying options for the overall text widget. If no *option* is specified, the command returns a list describing all of the available options for *tagName* (see Tk_ConfigureInfo for information on the format

of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given option(s) to have the given value(s) in *tagName*; in this case the command returns an empty string. See Tags above for details on the options available for tags.

***pathName* tag delete *tagName* ?*tagName ...*?**
Deletes all tag information for each of the *tagName* arguments. The command removes the tags from all characters in the file and also deletes any other information associated with the tags, such as bindings and display information. The command returns an empty string.

***pathName* tag lower *tagName* ?*belowThis*?**
Changes the priority of tag *tagName* so that it is just lower in priority than the tag whose name is *belowThis*. If *belowThis* is omitted, then *tagName*'s priority is changed to make it lowest priority of all tags.

***pathName* tag names ?*index*?**
Returns a list whose elements are the names of all the tags that are active at the character position given by *index*. If *index* is omitted, then the return value will describe all of the tags that exist for the text (this includes all tags that have been named in a "*pathName* **tag**" widget command but have not been deleted by a "*pathName* **tag delete**" widget command, even if no characters are currently marked with the tag). The list will be sorted in order from lowest priority to highest priority.

***pathName* tag nextrange *tagName index1* ?*index2*?**
This command searches the text for a range of characters tagged with *tagName* where the first character of the range is no earlier than the character at *index1* and no later than the character just before *index2* (a range starting at *index2* will not be considered). If several matching ranges exist, the first one is chosen. The command's return value is a list containing two elements, which are the index of the first character of the range and the index of the character just after the last one in the range. If no matching range is found then the return value is an empty string. If *index2* is not given then it defaults to the end of the text.

***pathName* tag prevrange *tagName index1* ?*index2*?**
This command searches the text for a range of characters tagged with *tagName* where the first character of the range is before the character at *index1* and no earlier than the character at *index2* (a range starting at *index2* will be considered). If several matching ranges exist, the one closest to *index1* is chosen. The command's return value is a list containing two elements, which are the index of the first character of the range and the index of the character just after the last one in the range. If no matching range is found then the return value is an empty string. If *index2* is not given then it defaults to the beginning of the text.

***pathName* tag raise *tagName* ?*aboveThis*?**
Changes the priority of tag *tagName* so that it is just higher in priority than the tag whose name is *aboveThis*. If *aboveThis* is omitted, then *tagName*'s priority is changed to make it highest priority of all tags.

pathName* tag ranges *tagName
Returns a list describing all of the ranges of text that have been tagged with *tagName*. The

first two elements of the list describe the first tagged range in the text, the next two elements describe the second range, and so on. The first element of each pair contains the index of the first character of the range, and the second element of the pair contains the index of the character just after the last one in the range. If there are no characters tagged with *tag* then an empty string is returned.

pathName* tag remove *tagName index1 ?index2 index1 index2 ...?

Remove the tag *tagName* from all of the characters starting at *index1* and ending just before *index2* (the character at *index2* is not affected). A single command may contain any number of *index1*-*index2* pairs. If the last *index2* is omitted then the tag is removed from the single character at *index1*. If there are no characters in the specified range (e.g. *index1* is past the end of the file or *index2* is less than or equal to *index1*) then the command has no effect. This command returns an empty string.

pathName* window *option ?arg arg ...?

This command is used to manipulate embedded windows. The behavior of the command depends on the *option* argument that follows the **window** argument. The following forms of the command are currently supported:

pathName* window cget *index option

Returns the value of a configuration option for an embedded window. *Index* identifies the embedded window, and *option* specifies a particular configuration option, which must be one of the ones listed in the section Embedded windows.

pathName* window configure *index ?option value ...?

Query or modify the configuration options for an embedded window. If no *option* is specified, returns a list describing all of the available options for the embedded window at *index* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given option(s) to have the given value(s); in this case the command returns an empty string. See Embedded windows for information on the options that are supported.

pathName* window create *index ?option value ...?

This command creates a new window annotation, which will appear in the text at the position given by *index*. Any number of *option-value* pairs may be specified to configure the annotation. See Embedded windows for information on the options that are supported. Returns an empty string.

***pathName* window names**

Returns a list whose elements are the names of all windows currently embedded in *window* .

pathName* xview *option args

This command is used to query and change the horizontal position of the text in the widget's window. It can take any of the following forms:

***pathName* xview**

Returns a list containing two elements. Each element is a real fraction between 0 and 1; together they describe the portion of the document's horizontal span that is visible in the window. For example, if the first element is .2 and the second element is .6, 20% of the

text is off-screen to the left, the middle 40% is visible in the window, and 40% of the text is off-screen to the right. The fractions refer only to the lines that are actually visible in the window: if the lines in the window are all very short, so that they are entirely visible, the returned fractions will be 0 and 1, even if there are other lines in the text that are much wider than the window. These are the same values passed to scrollbars via the **-xscrollcommand** option.

pathName* xview moveto *fraction
Adjusts the view in the window so that *fraction* of the horizontal span of the text is off-screen to the left. *Fraction* is a fraction between 0 and 1.

pathName* xview scroll *number what
This command shifts the view in the window left or right according to *number* and *what*. *What* must be **units**, **pages** or **pixels**. If *what* is **units** or **pages** then *number* must be an integer, otherwise number may be specified in any of the forms acceptable to Tk_GetPixels, such as "2.0c" or "1i" (the result is rounded to the nearest integer value. If no units are given, pixels are assumed). If *what* is **units**, the view adjusts left or right by *number* average-width characters on the display; if it is **pages** then the view adjusts by *number* screenfuls; if it is **pixels** then the view adjusts by *number* pixels. If *number* is negative then characters farther to the left become visible; if it is positive then characters farther to the right become visible.

***pathName* yview ?*args*?**
This command is used to query and change the vertical position of the text in the widget's window. It can take any of the following forms:

***pathName* yview**
Returns a list containing two elements, both of which are real fractions between 0 and 1. The first element gives the position of the first visible pixel of the first character (or image, etc) in the top line in the window, relative to the text as a whole (0.5 means it is halfway through the text, for example). The second element gives the position of the first pixel just after the last visible one in the bottom line of the window, relative to the text as a whole. These are the same values passed to scrollbars via the **-yscrollcommand** option.

pathName* yview moveto *fraction
Adjusts the view in the window so that the pixel given by *fraction* appears at the top of the top line of the window. *Fraction* is a fraction between 0 and 1; 0 indicates the first pixel of the first character in the text, 0.33 indicates the pixel that is one-third the way through the text; and so on. Values close to 1 will indicate values close to the last pixel in the text (1 actually refers to one pixel beyond the last pixel), but in such cases the widget will never scroll beyond the last pixel, and so a value of 1 will effectively be rounded back to whatever fraction ensures the last pixel is at the bottom of the window, and some other pixel is at the top.

pathName* yview scroll *number what
This command adjust the view in the window up or down according to *number* and *what*. *What* must be **units**, **pages** or **pixels**. If *what* is **units** or **pages** then *number* must be an integer, otherwise number may be specified in any of the forms acceptable to Tk_GetPixels, such as "2.0c" or "1i" (the result is rounded to the nearest integer value. If no units are given, pixels are assumed). If *what* is **units**, the view adjusts up or down by *number* lines on the display; if it is **pages** then the view adjusts by *number* screenfuls; if it is **pixels**

then the view adjusts by *number* pixels. If *number* is negative then earlier positions in the text become visible; if it is positive then later positions in the text become visible.

pathName* yview ?-pickplace? *index

Changes the view in the widget's window to make *index* visible. If the **-pickplace** option is not specified then *index* will appear at the top of the window. If **-pickplace** is specified then the widget chooses where *index* appears in the window:

1. If *index* is already visible somewhere in the window then the command does nothing.
2. If *index* is only a few lines off-screen above the window then it will be positioned at the top of the window.
3. If *index* is only a few lines off-screen below the window then it will be positioned at the bottom of the window.
4. Otherwise, *index* will be centered in the window.

The **-pickplace** option has been obsoleted by the *pathName* **see** widget command (*pathName* **see** handles both x- and y-motion to make a location visible, whereas the **-pickplace** mode only handles motion in y).

pathName* yview *number

This command makes the first character on the line after the one given by *number* visible at the top of the window. *Number* must be an integer. This command used to be used for scrolling, but now it is obsolete.

Bindings

Tk automatically creates class bindings for texts that give them the following default behavior. In the descriptions below, "word" is dependent on the value of the **tcl_wordchars** variable. See tclvars(n).

1. Clicking mouse button 1 positions the insertion cursor at the closest edge of the character underneath the mouse cursor, sets the input focus to this widget, and clears any selection in the widget. Dragging with mouse button 1 strokes out a selection between the insertion cursor and the character under the mouse.
2. Double-clicking with mouse button 1 selects the word under the mouse and positions the insertion cursor at the start of the word. Dragging after a double click will stroke out a selection consisting of whole words.
3. Triple-clicking with mouse button 1 selects the line under the mouse and positions the insertion cursor at the start of the line. Dragging after a triple click will stroke out a selection consisting of whole lines.
4. The ends of the selection can be adjusted by dragging with mouse button 1 while the Shift key is down; this will adjust the end of the selection that was nearest to the mouse cursor when button 1 was pressed. If the button is double-clicked before dragging then the selection will be adjusted in units of whole words; if it is triple-clicked then the selection will be adjusted in units of whole lines.
5. Clicking mouse button 1 with the Control key down will reposition the insertion cursor without affecting the selection.
6. If any normal printing characters are typed, they are inserted at the point of the insertion cursor.
7. The view in the widget can be adjusted by dragging with the middle mouse button (button 2, or button 3 in TkAqua). If the middle mouse button is clicked without moving the mouse, the

selection is copied into the text at the position of the mouse cursor. The Insert key also inserts the selection, but at the position of the insertion cursor.

8. If the mouse is dragged out of the widget while button 1 is pressed, the entry will automatically scroll to make more text visible (if there is more text off-screen on the side where the mouse left the window).
9. The Left and Right keys move the insertion cursor one character to the left or right; they also clear any selection in the text. If Left or Right is typed with the Shift key down, then the insertion cursor moves and the selection is extended to include the new character. Control-Left and Control-Right move the insertion cursor by words, and Control-Shift-Left and Control-Shift-Right move the insertion cursor by words and also extend the selection. Control-b and Control-f behave the same as Left and Right, respectively. Meta-b and Meta-f behave the same as Control-Left and Control-Right, respectively.
10. The Up and Down keys move the insertion cursor one line up or down and clear any selection in the text. If Up or Right is typed with the Shift key down, then the insertion cursor moves and the selection is extended to include the new character. Control-Up and Control-Down move the insertion cursor by paragraphs (groups of lines separated by blank lines), and Control-Shift-Up and Control-Shift-Down move the insertion cursor by paragraphs and also extend the selection. Control-p and Control-n behave the same as Up and Down, respectively.
11. The Next and Prior keys move the insertion cursor forward or backwards by one screenful and clear any selection in the text. If the Shift key is held down while Next or Prior is typed, then the selection is extended to include the new character.
12. Control-Next and Control-Prior scroll the view right or left by one page without moving the insertion cursor or affecting the selection.
13. Home and Control-a move the insertion cursor to the beginning of its display line and clear any selection in the widget. Shift-Home moves the insertion cursor to the beginning of the display line and also extends the selection to that point.
14. End and Control-e move the insertion cursor to the end of the display line and clear any selection in the widget. Shift-End moves the cursor to the end of the display line and extends the selection to that point.
15. Control-Home and Meta-< move the insertion cursor to the beginning of the text and clear any selection in the widget. Control-Shift-Home moves the insertion cursor to the beginning of the text and also extends the selection to that point.
16. Control-End and Meta-> move the insertion cursor to the end of the text and clear any selection in the widget. Control-Shift-End moves the cursor to the end of the text and extends the selection to that point.
17. The Select key and Control-Space set the selection anchor to the position of the insertion cursor. They do not affect the current selection. Shift-Select and Control-Shift-Space adjust the selection to the current position of the insertion cursor, selecting from the anchor to the insertion cursor if there was not any selection previously.
18. Control-/ selects the entire contents of the widget.
19. Control-\ clears any selection in the widget.
20. The F16 key (labelled Copy on many Sun workstations) or Meta-w copies the selection in the widget to the clipboard, if there is a selection. This action is carried out by the command **tk_textCopy**.
21. The F20 key (labelled Cut on many Sun workstations) or Control-w copies the selection in the

widget to the clipboard and deletes the selection. This action is carried out by the command **tk_textCut**. If there is no selection in the widget then these keys have no effect.
22. The F18 key (labelled Paste on many Sun workstations) or Control-y inserts the contents of the clipboard at the position of the insertion cursor. This action is carried out by the command **tk_textPaste**.
23. The Delete key deletes the selection, if there is one in the widget. If there is no selection, it deletes the character to the right of the insertion cursor.
24. Backspace and Control-h delete the selection, if there is one in the widget. If there is no selection, they delete the character to the left of the insertion cursor.
25. Control-d deletes the character to the right of the insertion cursor.
26. Meta-d deletes the word to the right of the insertion cursor.
27. Control-k deletes from the insertion cursor to the end of its line; if the insertion cursor is already at the end of a line, then Control-k deletes the newline character.
28. Control-o opens a new line by inserting a newline character in front of the insertion cursor without moving the insertion cursor.
29. Meta-backspace and Meta-Delete delete the word to the left of the insertion cursor.
30. Control-x deletes whatever is selected in the text widget after copying it to the clipboard.
31. Control-t reverses the order of the two characters to the right of the insertion cursor.
32. Control-z undoes the last edit action if the **-undo** option is true. Does nothing otherwise.
33. Control-Z (or Control-y on Windows) reapplies the last undone edit action if the **-undo** option is true. Does nothing otherwise.

If the widget is disabled using the **-state** option, then its view can still be adjusted and text can still be selected, but no insertion cursor will be displayed and no text modifications will take place.

The behavior of texts can be changed by defining new bindings for individual widgets or by redefining the class bindings.

Known issues

Issues concerning chars and indices

Before Tk 8.5, the widget used the string "chars" to refer to index positions (which included characters, embedded windows and embedded images). As of Tk 8.5 the text widget deals separately and correctly with "chars" and "indices". For backwards compatibility, however, the index modifiers "+N chars" and "-N chars" continue to refer to indices. One must use any of the full forms "+N any chars" or "-N any chars" etc. to refer to actual character indices. This confusion may be fixed in a future release by making the widget correctly interpret "+N chars" as a synonym for "+N any chars".

Performance issues

Text widgets should run efficiently under a variety of conditions. The text widget uses about 2-3 bytes of main memory for each byte of text, so texts containing a megabyte or more should be practical on most workstations. Text is represented internally with a modified B-tree structure that makes operations relatively efficient even with large texts. Tags are included in the B-tree structure in a way that allows tags to span large ranges or have many disjoint smaller ranges without loss of efficiency. Marks are also implemented in a way that allows large numbers of marks. In most cases it is fine to have large numbers of unique tags, or a tag that has many distinct ranges.

One performance problem can arise if you have hundreds or thousands of different tags that all have the following characteristics: the first and last ranges of each tag are near the beginning and end of the text, respectively, or a single tag range covers most of the text widget. The cost of adding and deleting tags like this is proportional to the number of other tags with the same properties. In contrast, there is no problem with having thousands of distinct tags if their overall ranges are localized and spread uniformly throughout the text.

Very long text lines can be expensive, especially if they have many marks and tags within them.

The display line with the insert cursor is redrawn each time the cursor blinks, which causes a steady stream of graphics traffic. Set the **-insertofftime** attribute to 0 avoid this.

Known bugs

The *pathName* **search -regexp** sub-command attempts to perform sophisticated regexp matching across multiple lines in an efficient fashion (since Tk 8.5), examining each line individually, and then in small groups of lines, whether searching forwards or backwards. Under certain conditions the search result might differ from that obtained by applying the same regexp to the entire text from the widget in one go. For example, when searching with a greedy regexp, the widget will continue to attempt to add extra lines to the match as long as one of two conditions are true: either Tcl's regexp library returns a code to indicate a longer match is possible (but there are known bugs in Tcl which mean this code is not always correctly returned); or if each extra line added results in at least a partial match with the pattern. This means in the case where the first extra line added results in no match and Tcl's regexp system returns the incorrect code and adding a second extra line would actually match, the text widget will return the wrong result. In practice this is a rare problem, but it can occur, for example:

```
pack [text .t]
.t insert 1.0 "aaaa\nbbbb\ncccc\nbbbb\naaaa\n"
.t search -regexp -- {(a+|b+\nc+\nb+)+\na+} 1.0
```

will not find a match when one exists of 19 characters starting from the first "b".

Whenever one possible match is fully enclosed in another, the search command will attempt to ensure only the larger match is returned. When performing backwards regexp searches it is possible that Tcl will not always achieve this, in the case where a match is preceded by one or more short, non-overlapping matches, all of which are preceded by a large match which actually encompasses all of them. The search algorithm used by the widget does not look back arbitrarily far for a possible match which might cover large portions of the widget. For example:

```
pack [text .t]
.t insert 1.0 "aaaa\nbbbb\nbbbb\nbbbb\nbbbb\n"
.t search -regexp -backward -- {b+\n|a+\n(b+\n)+} end
```

matches at "5.0" when a true greedy match would match at "1.0". Similarly if we add **-all** to this case, it matches at all of "5.0", "4.0", "3.0" and "1.0", when really it should only match at "1.0" since that match encloses all the others.

Copyright (c) 1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

tk

Keywords: tk, application name, send

Name

tk - Manipulate Tk internal state

Synopsis

tk *option* ?*arg* ...?

Description

The **tk** command provides access to miscellaneous elements of Tk's internal state. Most of the information manipulated by this command pertains to the application as a whole, or to a screen or display, rather than to a particular window. The command can take any of a number of different forms depending on the *option* argument. The legal forms are:

tk appname ?*newName*?
: If *newName* is not specified, this command returns the name of the application (the name that may be used in send commands to communicate with the application). If *newName* is specified, then the name of the application is changed to *newName*. If the given name is already in use, then a suffix of the form " **#2**" or " **#3**" is appended in order to make the name unique. The command's result is the name actually chosen. *newName* should not start with a capital letter. This will interfere with option processing, since names starting with capitals are assumed to be classes; as a result, Tk may not be able to find some options for the application. If sends have been disabled by deleting the send command, this command will reenable them and recreate the send command.

tk busy *subcommand* ...
: This command controls the marking of window hierarchies as "busy", rendering them non-interactive while some other operation is proceeding. For more details see the busy manual page.

tk caret *window* ?*-x x*? ?*-y y*? ?*-height height*?
: Sets and queries the caret location for the display of the specified Tk window *window*. The caret is the per-display cursor location used for indicating global focus (e.g. to comply with Microsoft Accessibility guidelines), as well as for location of the over-the-spot XIM (X Input Methods) or Windows IME windows. If no options are specified, the last values used for setting the caret are return in option-value pair format. **-x** and **-y** represent window-relative coordinates, and **-height** is the height of the current cursor location, or the height of the specified *window* if none is given.

tk inactive ?*-displayof window*? ?reset?
: Returns a positive integer, the number of milliseconds since the last time the user interacted with the system. If the **-displayof** option is given then the return value refers to the display of *window*; otherwise it refers to the display of the application's main window.

  tk inactive will return -1, if querying the user inactive time is not supported by the system, and in safe interpreters.

  If the literal string **reset** is given as an additional argument, the timer is reset and an empty string is returned. Resetting the inactivity time is forbidden in safe interpreters and will throw an error if tried.

tk fontchooser *subcommand* ...
Controls the Tk font selection dialog. For more details see the fontchooser manual page.

tk scaling ?*-displayof window*? ?*number*?
Sets and queries the current scaling factor used by Tk to convert between physical units (for example, points, inches, or millimeters) and pixels. The *number* argument is a floating point number that specifies the number of pixels per point on *window*'s display. If the *window* argument is omitted, it defaults to the main window. If the *number* argument is omitted, the current value of the scaling factor is returned.

A "point" is a unit of measurement equal to 1/72 inch. A scaling factor of 1.0 corresponds to 1 pixel per point, which is equivalent to a standard 72 dpi monitor. A scaling factor of 1.25 would mean 1.25 pixels per point, which is the setting for a 90 dpi monitor; setting the scaling factor to 1.25 on a 72 dpi monitor would cause everything in the application to be displayed 1.25 times as large as normal. The initial value for the scaling factor is set when the application starts, based on properties of the installed monitor, but it can be changed at any time. Measurements made after the scaling factor is changed will use the new scaling factor, but it is undefined whether existing widgets will resize themselves dynamically to accommodate the new scaling factor.

tk useinputmethods ?*-displayof window*? ?*boolean*?
Sets and queries the state of whether Tk should use XIM (X Input Methods) for filtering events. The resulting state is returned. XIM is used in some locales (i.e., Japanese, Korean), to handle special input devices. This feature is only significant on X. If XIM support is not available, this will always return 0. If the *window* argument is omitted, it defaults to the main window. If the *boolean* argument is omitted, the current state is returned. This is turned on by default for the main display.

tk windowingsystem
Returns the current Tk windowing system, one of **x11** (X11-based), **win32** (MS Windows), or **aqua** (macOS Aqua).

Copyright (c) 1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

tk_mac

Keywords: tk_mac, about dialog, antialiasing, Apple event, icon, NSImage

Name

tk::mac - Access Mac-Specific Functionality on macOS from Tk

Synopsis

::tk::mac::DoScriptFile
::tk::mac::DoScriptText
::tk::mac::ShowPreferences
::tk::mac::OpenApplication
::tk::mac::ReopenApplication
::tk::mac::OpenDocument *file...*
::tk::mac::PrintDocument *file...*
::tk::mac::Quit

::tk::mac::OnHide
::tk::mac::OnShow
::tk::mac::ShowHelp
::tk::mac::PerformService
::tk::mac::LaunchURL *URL...*
::tk::mac::GetAppPath

::tk::mac::standardAboutPanel

::tk::mac::useCompatibilityMetrics *boolean*
::tk::mac::CGAntialiasLimit *limit*
::tk::mac::antialiasedtext *number*
::tk::mac::useThemedToplevel *boolean*

::tk::mac::iconBitmap *name width height -kind value*

Event handler callbacks

The Aqua/macOS application environment defines a number of additional events that applications should respond to. These events are mapped by Tk to calls to commands in the **::tk::mac** namespace; unless otherwise noted, if the command is absent, no action will be taken.

::tk::mac::DoScriptFile
: The default Apple Event handler for AEDoScriptHandler. This command executes a Tcl file when an AppleScript sends a "do script" command to Wish with a file path as a parameter.

::tk::mac::DoScriptText
: The default Apple Event handler for AEDoScriptHandler. This command executes Tcl code when an AppleScript sends a "do script" command to Wish with Tcl code or a Tcl procedure as a parameter.

::tk::mac::ShowPreferences
: The default Apple Event handler for kAEShowPreferences, "pref". The application menu "Preferences" menu item is only enabled when this proc is defined. Typically this command is used to wrap a specific own preferences command, which pops up a preferences window. Something like:

```
proc ::tk::mac::ShowPreferences {} {
    setPref
}
```

::tk::mac::OpenApplication
: If a proc of this name is defined, this proc fill fire when your application is initially opened. It is the default Apple Event handler for kAEOpenApplication, "oapp".

::tk::mac::ReopenApplication
: If a proc of this name is defined it is the default Apple Event handler for kAEReopenApplication, "rapp", the Apple Event sent when your application is opened when it is already running (e.g. by clicking its icon in the Dock). Here is a sample that raises a minimized window when the Dock icon is clicked:

```
proc ::tk::mac::ReopenApplication {} {
    if {[wm state .] eq "withdrawn"} {
        wm state . normal
    } else {
        wm deiconify .
    }
    raise .
}
```

::tk::mac::OpenDocument *file...*
If a proc of this name is defined it is the default Apple Event handler for kAEOpenDocuments, "odoc", the Apple Event sent when your application is asked to open one or more documents (e.g., by drag & drop onto the app or by opening a document of a type associated to the app). The proc should take as arguments paths to the files to be opened, like so:

```
proc ::tk::mac::OpenDocument {args} {
    foreach f $args {my_open_document $f}
}
```

::tk::mac::PrintDocument *file...*
If a proc of this name is defined it is the default Apple Event handler for kAEPrintDocuments, "pdoc", the Apple Event sent when your application is asked to print a document. It takes a single absolute file path as an argument.

::tk::mac::Quit
If a proc of this name is defined it is the default Apple Event handler for kAEQuitApplication, "quit", the Apple Event sent when your application is asked to be quit, e.g. via the quit menu item in the application menu, the quit menu item in the Dock menu, or during a logout/restart/shutdown etc. If this is not defined, exit is called instead.

::tk::mac::OnHide
If defined, this is called when your application receives a kEventAppHidden event, e.g. via the hide menu item in the application or Dock menus.

::tk::mac::OnShow
If defined, this is called when your application receives a kEventAppShown event, e.g. via the show all menu item in the application menu, or by clicking the Dock icon of a hidden application.

::tk::mac::ShowHelp
Customizes behavior of Apple Help menu; if this procedure is not defined, the platform-specific standard Help menu item "YourApp Help" performs the default Cocoa action of showing the Help Book configured in the application's Info.plist (or displaying an alert if no Help Book is set).

::tk::mac::PerformService
Executes a Tcl procedure called from the macOS "Services" menu in the Application menu item. The "Services" menu item allows for inter-application communication; data from one application, such as selected text, can be sent to another application for processing, for example to Safari as a search item for Google, or to TextEdit to be appended to a file. An example of the proc is below, and should be rewritten in an application script for

customization:

```
proc ::tk::mac::PerformService {} {
    set data [clipboard get]
    $w insert end $data
}
```

Note that the mechanism for retrieving the data is from the clipboard; there is no other supported way to obtain the data. If the Services process is not desired, the NSServices keys can be deleted from the application's Info.plist file. The underlying code supporting this command also allows the text, entry and ttk::entry widgets to access services from other applications via the Services menu. The NSPortName key in Wish's Info.plist file is currently set as "Wish"; if a developer changes the name of the Wish executable to something else, this key should be modified with the same name.

::tk::mac::LaunchURL *URL...*
If defined, launches a URL within Tk. This would be used if a Tk application wants to handle a URL itself, such as displaying data from an RSS feed, rather than launching a default application to handle the URL, although it can defined as such. Wish includes a stub URL scheme of "foo://" in the CFBundleURLSchemes key of its Info.plist file; this should be customized for the specific URL scheme the developer wants to support.

::tk::mac::GetAppPath
Returns the current applications's file path.

Additional dialogs

Aqua/macOS defines additional dialogs that applications should support.

::tk::mac::standardAboutPanel
Brings the standard Cocoa about panel to the front with information filled in from the application bundle files. The panel displays the application icon and the values associated to the info.plist keys named CFBundleName, CFBundleShortVersionString, NSAboutPanelOptionVersion and NSHumanReadableCopyright. If a file named *Credits.html* or *Credits.rtf* exists in the bundle's Resources directory then its contents will be displayed in a scrolling text box at the bottom of the dialog. See the documentation for -[NSApplication orderFrontStandardAboutPanelWithOptions:] for more details. A hook is also provided for a custom About dialog. If a Tcl proc named tkAboutDialog is defined in the main interpreter then that procedure will be called instead of opening the standardAboutPanel.

System configuration

There are a number of additional global configuration options that control the details of how Tk renders by default.

::tk::mac::useCompatibilityMetrics *boolean*
Preserves compatibility with older Tk/Aqua metrics; set to **false** for more native spacing.

::tk::mac::CGAntialiasLimit *limit*
Sets the antialiasing limit; lines thinner that *limit* pixels will not be antialiased. Integer, set to 0 by default, making all lines be antialiased.

::tk::mac::antialiasedtext *number*
Sets anti-aliased text. Controls text antialiasing, possible values for *number* are -1 (default,

use system default for text AA), 0 (no text AA), 1 (use text AA).

::tk::mac::useThemedToplevel *boolean*
Sets toplevel windows to draw with the modern grayish/ pinstripe Mac background. Equivalent to configuring the toplevel with "**-background systemWindowHeaderBackground**", or to using a ttk::frame.

Support commands

::tk::mac::iconBitmap *name width height -kind value*
Renders native icons and bitmaps in Tk applications (including any image file readable by NSImage). A native bitmap name is interpreted as follows (in order):
- predefined builtin 32x32 icon name (**stop**, **caution**, **document**, etc.)
- *name*, as defined by **tk::mac::iconBitmap**
- NSImage named image name
- NSImage url string
- 4-char OSType of IconServices icon

The *width* and *height* arguments to **tk::mac::iconBitmap** define the dimensions of the image to create, and *-kind* must be one of:

-file
icon of file at given path

-fileType
icon of given file type

-osType
icon of given 4-char OSType file type

-systemType
icon for given IconServices 4-char OSType

-namedImage
named NSImage for given name

-imageFile
image at given path

Copyright (c) 2011 Kevin Walzer.
Copyright (c) 2011 Donal K. Fellows.

tkerror

Keywords: tkerror, background error, reporting

Name

tkerror - Command invoked to process background errors

Synopsis

tkerror *message*

Description

Note: as of Tk 4.1 the **tkerror** command has been renamed to bgerror because the event loop (which is what usually invokes it) is now part of Tcl. For backward compatibility the bgerror

provided by the current Tk version still tries to call **tkerror** if there is one (or an auto loadable one), so old script defining that error handler should still work, but you should anyhow modify your scripts to use bgerror instead of **tkerror** because that support for the old name might vanish in the near future. If that call fails, bgerror posts a dialog showing the error and offering to see the stack trace to the user. If you want your own error management you should directly override bgerror instead of **tkerror**. Documentation for bgerror is available as part of Tcl's documentation.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

tkvars

Keywords: tkvars, environment, text, variables, version

Name

geometry, tk_library, tk_patchLevel, tk_strictMotif, tk_version - Variables used or set by Tk

Description

The following Tcl variables are either set or used by Tk at various times in its execution:

tk_library
: This variable holds the file name for a directory containing a library of Tcl scripts related to Tk. These scripts include an initialization file that is normally processed whenever a Tk application starts up, plus other files containing procedures that implement default behaviors for widgets.

  The initial value of **tk_library** is set when Tk is added to an interpreter; this is done by searching several different directories until one is found that contains an appropriate Tk startup script. If the **TK_LIBRARY** environment variable exists, then the directory it names is checked first. If **TK_LIBRARY** is not set or does not refer to an appropriate directory, then Tk checks several other directories based on a compiled-in default location, the location of the Tcl library directory, the location of the binary containing the application, and the current working directory.

  The variable can be modified by an application to switch to a different library.

tk_patchLevel
: Contains a dot-separated sequence of decimal integers giving the current patch level for Tk. The patch level is incremented for each new release or patch, and it uniquely identifies an official version of Tk.

  This value is normally the same as the result of "package require **Tk**".

tk_strictMotif
: This variable is set to zero by default. If an application sets it to one, then Tk attempts to adhere as closely as possible to Motif look-and-feel standards. For example, active elements such as buttons and scrollbar sliders will not change color when the pointer passes over them. Modern applications should not normally set this variable.

tk_version

Tk sets this variable in the interpreter for each application. The variable holds the current version number of the Tk library in the form *major.minor*. *Major* and *minor* are integers. The major version number increases in any Tk release that includes changes that are not backward compatible (i.e. whenever existing Tk applications and scripts may have to change to work with the new release). The minor version number increases with each new release of Tk, except that it resets to zero whenever the major version number changes.

Internal and debugging variables

These variables should not normally be set by user code.

tk::Priv
: This variable is an array containing several pieces of information that are private to Tk. The elements of **tk::Priv** are used by Tk library procedures and default bindings. They should not be accessed by any code outside Tk.

tk_textRedraw
: see below ...

tk_textRelayout
: These variables are set by text widgets when they have debugging turned on. The values written to these variables can be used to test or debug text widget operations. These variables are mostly used by Tk's test suite.

Other global variables

The following variables are only guaranteed to exist in wish executables; the Tk library does not define them itself but many Tk environments do.

geometry
: If set, contains the user-supplied geometry specification to use for the main Tk window.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

tkwait

Keywords: tkwait, variable, visibility, wait, window

Name

tkwait - Wait for variable to change or window to be destroyed

Synopsis

tkwait variable *name*
tkwait visibility *name*
tkwait window *name*

Description

The **tkwait** command waits for one of several things to happen, then it returns without taking any other actions. The return value is always an empty string. If the first argument is **variable** (or any abbreviation of it) then the second argument is the name of a global variable and the command waits for that variable to be modified. If the first argument is **visibility** (or any abbreviation of it) then the second argument is the name of a window and the **tkwait** command

waits for a change in its visibility state (as indicated by the arrival of a VisibilityNotify event). This form is typically used to wait for a newly-created window to appear on the screen before taking some action. If the first argument is **window** (or any abbreviation of it) then the second argument is the name of a window and the **tkwait** command waits for that window to be destroyed. This form is typically used to wait for a user to finish interacting with a dialog box before using the result of that interaction.

While the **tkwait** command is waiting it processes events in the normal fashion, so the application will continue to respond to user interactions. If an event handler invokes **tkwait** again, the nested call to **tkwait** must complete before the outer call can complete.

Copyright (c) 1992 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

toplevel

Keywords: toplevel, widget

Name

toplevel - Create and manipulate 'toplevel' main and popup window widgets

Synopsis

toplevel *pathName* ?*options*?

Standard options

-borderwidth, **-highlightcolor**, **-pady**, **-cursor**, **-highlightthickness**, **-relief**, **-highlightbackground**, **-padx**, **-takefocus**

See the options manual entry for details on the standard options.

Widget-specific options

-background (background/Background)
: This option is the same as the standard **-background** option except that its value may also be specified as an empty string. In this case, the widget will display no background or border, and no colors will be consumed from its colormap for its background and border.

-class (class/Class)
: Specifies a class for the window. This class will be used when querying the option database for the window's other options, and it will also be used later for other purposes such as bindings. Some window managers display the class name for windows in their dock while some others display the window title. The **-class** option may not be changed with the **configure** widget command.

-colormap (colormap/Colormap)
: Specifies a colormap to use for the window. The value may be either **new**, in which case a new colormap is created for the window and its children, or the name of another window (which must be on the same screen and have the same visual as *pathName*), in which case the new window will use the colormap from the specified window. If the **-colormap** option is not specified, the new window uses the default colormap of its screen. This option may not be changed with the **configure** widget command.

-container (container/Container)

The value must be a boolean. If true, it means that this window will be used as a container in which some other application will be embedded (for example, a Tk toplevel can be embedded using the **-use** option). The window will support the appropriate window manager protocols for things like geometry requests. The window should not have any children of its own in this application. This option may not be changed with the **configure** widget command.

-height (height/Height)
Specifies the desired height for the window in any of the forms acceptable to Tk_GetPixels. If this option is less than or equal to zero then the window will not request any size at all.

-menu (menu**/Menu)**
Specifies a menu widget to be used as a menubar. On the Macintosh, the menubar will be displayed across the top of the main monitor. On Microsoft Windows and all UNIX platforms, the menu will appear across the toplevel window as part of the window dressing maintained by the window manager.

-screen
Specifies the screen on which to place the new window. Any valid screen name may be used, even one associated with a different display. Defaults to the same screen as its parent. This option is special in that it may not be specified via the option database, and it may not be modified with the **configure** widget command.

-use (use/Use)
This option is used for embedding. If the value is not an empty string, it must be the window identifier of a container window, specified as a hexadecimal string like the ones returned by the winfo id command. The toplevel widget will be created as a child of the given container instead of the root window for the screen. If the container window is in a Tk application, it must be a frame or toplevel widget for which the **-container** option was specified. This option may not be changed with the **configure** widget command.

-visual (visual/Visual)
Specifies visual information for the new window in any of the forms accepted by Tk_GetVisual. If this option is not specified, the new window will use the default visual for its screen. The **-visual** option may not be modified with the **configure** widget command.

-width (width/Width)
Specifies the desired width for the window in any of the forms acceptable to Tk_GetPixels. If this option is less than or equal to zero then the window will not request any size at all.

Description

The **toplevel** command creates a new toplevel widget (given by the *pathName* argument). Additional options, described above, may be specified on the command line or in the option database to configure aspects of the toplevel such as its background color and relief. The **toplevel** command returns the path name of the new window.

A toplevel is similar to a frame except that it is created as a top-level window: its X parent is the root window of a screen rather than the logical parent from its Tk path name. The primary purpose of a toplevel is to serve as a container for dialog boxes and other collections of widgets. The only visible features of a toplevel are its background color and an optional 3-D border to make the toplevel appear raised or sunken.

Widget command

The **toplevel** command creates a new Tcl command whose name is the same as the path name of the toplevel's window. This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

PathName is the name of the command, which is the same as the toplevel widget's path name. *Option* and the *arg*s determine the exact behavior of the command. The following commands are possible for toplevel widgets:

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the **toplevel** command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see Tk_ConfigureInfo for information on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the **toplevel** command.

Bindings

When a new toplevel is created, it has no default event bindings: toplevels are not intended to be interactive.

Be aware that bindings on toplevels may receive events from subwidgets.

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

winfo

Keywords: winfo, atom, children, class, geometry, height, identifier, information, interpreters, mapped, parent, path name, screen, virtual root, width, window

Name

winfo - Return window-related information

Synopsis

winfo *option* ?*arg arg* ...?

Description

The **winfo** command is used to retrieve information about windows managed by Tk. It can take any of a number of different forms, depending on the *option* argument. The legal forms are:

winfo atom ?-displayof *window*? *name*
Returns a decimal string giving the integer identifier for the atom whose name is *name*. If no atom exists with the name *name* then a new one is created. If the **-displayof** option is given then the atom is looked up on the display of *window*; otherwise it is looked up on the display

of the application's main window.

winfo atomname ?-displayof *window*? *id*
Returns the textual name for the atom whose integer identifier is *id*. If the **-displayof** option is given then the identifier is looked up on the display of *window*; otherwise it is looked up on the display of the application's main window. This command is the inverse of the **winfo atom** command. It generates an error if no such atom exists.

winfo cells *window*
Returns a decimal string giving the number of cells in the color map for *window*.

winfo children *window*
Returns a list containing the path names of all the children of *window*. Top-level windows are returned as children of their logical parents. The list is in stacking order, with the lowest window first, except for Top-level windows which are not returned in stacking order. Use the wm stackorder command to query the stacking order of Top-level windows.

winfo class *window*
Returns the class name for *window*.

winfo colormapfull *window*
Returns 1 if the colormap for *window* is known to be full, 0 otherwise. The colormap for a window is "known" to be full if the last attempt to allocate a new color on that window failed and this application has not freed any colors in the colormap since the failed allocation.

winfo containing ?-displayof *window*? *rootX rootY*
Returns the path name for the window containing the point given by *rootX* and *rootY*. *RootX* and *rootY* are specified in screen units (i.e. any form acceptable to Tk_GetPixels) in the coordinate system of the root window (if a virtual-root window manager is in use then the coordinate system of the virtual root window is used). If the **-displayof** option is given then the coordinates refer to the screen containing *window*; otherwise they refer to the screen of the application's main window. If no window in this application contains the point then an empty string is returned. An empty string is also returned if the point lies in the title bar or border of its highest containing toplevel in this application. (Note that with some window managers the borders may be invisible.) In selecting the containing window, children are given higher priority than parents and among siblings the highest one in the stacking order is chosen.

winfo depth *window*
Returns a decimal string giving the depth of *window* (number of bits per pixel).

winfo exists *window*
Returns 1 if there exists a window named *window*, 0 if no such window exists.

winfo fpixels *window number*
Returns a floating-point value giving the number of pixels in *window* corresponding to the distance given by *number*. *Number* may be specified in any of the forms acceptable to Tk_GetScreenMM, such as "2.0c" or "1i". The return value may be fractional; for an integer value, use **winfo pixels**.

winfo geometry *window*
Returns the geometry for *window*, in the form *width***x***height*+*x*+*y*. All dimensions are in pixels.

winfo height *window*
Returns a decimal string giving *window*'s height in pixels. When a window is first created its height will be 1 pixel; the height will eventually be changed by a geometry manager to fulfil

the window's needs. If you need the true height immediately after creating a widget, invoke update to force the geometry manager to arrange it, or use **winfo reqheight** to get the window's requested height instead of its actual height.

winfo id *window*
Returns a hexadecimal string giving a low-level platform-specific identifier for *window*. On Unix platforms, this is the X window identifier. Under Windows, this is the Windows HWND. On the Macintosh the value has no meaning outside Tk.

winfo interps ?-displayof *window*?
Returns a list whose members are the names of all Tcl interpreters (e.g. all Tk-based applications) currently registered for a particular display. If the **-displayof** option is given then the return value refers to the display of *window*; otherwise it refers to the display of the application's main window.

winfo ismapped *window*
Returns **1** if *window* is currently mapped, **0** otherwise.

winfo manager *window*
Returns the name of the geometry manager currently responsible for *window*, or an empty string if *window* is not managed by any geometry manager. The name is usually the name of the Tcl command for the geometry manager, such as pack or place. If the geometry manager is a widget, such as canvases or text, the name is the widget's class command, such as canvas.

winfo name *window*
Returns *window*'s name (i.e. its name within its parent, as opposed to its full path name). The command **winfo name.** will return the name of the application.

winfo parent *window*
Returns the path name of *window*'s parent, or an empty string if *window* is the main window of the application.

winfo pathname ?-displayof *window*? *id*
Returns the path name of the window whose X identifier is *id*. *Id* must be a decimal, hexadecimal, or octal integer and must correspond to a window in the invoking application. If the **-displayof** option is given then the identifier is looked up on the display of *window*; otherwise it is looked up on the display of the application's main window.

winfo pixels *window number*
Returns the number of pixels in *window* corresponding to the distance given by *number*. *Number* may be specified in any of the forms acceptable to Tk_GetPixels, such as "2.0c" or "1i". The result is rounded to the nearest integer value; for a fractional result, use **winfo fpixels**.

winfo pointerx *window*
If the mouse pointer is on the same screen as *window*, returns the pointer's x coordinate, measured in pixels in the screen's root window. If a virtual root window is in use on the screen, the position is measured in the virtual root. If the mouse pointer is not on the same screen as *window* then -1 is returned.

winfo pointerxy *window*
If the mouse pointer is on the same screen as *window*, returns a list with two elements, which are the pointer's x and y coordinates measured in pixels in the screen's root window. If a virtual root window is in use on the screen, the position is computed in the virtual root. If the

mouse pointer is not on the same screen as *window* then both of the returned coordinates are -1.

winfo pointery *window*
If the mouse pointer is on the same screen as *window*, returns the pointer's y coordinate, measured in pixels in the screen's root window. If a virtual root window is in use on the screen, the position is computed in the virtual root. If the mouse pointer is not on the same screen as *window* then -1 is returned.

winfo reqheight *window*
Returns a decimal string giving *window*'s requested height, in pixels. This is the value used by *window*'s geometry manager to compute its geometry.

winfo reqwidth *window*
Returns a decimal string giving *window*'s requested width, in pixels. This is the value used by *window*'s geometry manager to compute its geometry.

winfo rgb *window color*
Returns a list containing three decimal values in the range 0 to 65535, which are the red, green, and blue intensities that correspond to *color* in the window given by *window*. *Color* may be specified in any of the forms acceptable for a color option.

winfo rootx *window*
Returns a decimal string giving the x-coordinate, in the root window of the screen, of the upper-left corner of *window*'s border (or *window* if it has no border).

winfo rooty *window*
Returns a decimal string giving the y-coordinate, in the root window of the screen, of the upper-left corner of *window*'s border (or *window* if it has no border).

winfo screen *window*
Returns the name of the screen associated with *window*, in the form *displayName*. *screenIndex*.

winfo screencells *window*
Returns a decimal string giving the number of cells in the default color map for *window*'s screen.

winfo screendepth *window*
Returns a decimal string giving the depth of the root window of *window*'s screen (number of bits per pixel).

winfo screenheight *window*
Returns a decimal string giving the height of *window*'s screen, in pixels.

winfo screenmmheight *window*
Returns a decimal string giving the height of *window*'s screen, in millimeters.

winfo screenmmwidth *window*
Returns a decimal string giving the width of *window*'s screen, in millimeters.

winfo screenvisual *window*
Returns one of the following strings to indicate the default visual class for *window*'s screen: **directcolor**, **grayscale**, **pseudocolor**, **staticcolor**, **staticgray**, or **truecolor**.

winfo screenwidth *window*
Returns a decimal string giving the width of *window*'s screen, in pixels.

winfo server *window*

Returns a string containing information about the server for *window*'s display. The exact format of this string may vary from platform to platform. For X servers the string has the form "**X***major***R***minor vendor vendorVersion*" where *major* and *minor* are the version and revision numbers provided by the server (e.g., **X11R5**), *vendor* is the name of the vendor for the server, and *vendorRelease* is an integer release number provided by the server.

winfo toplevel *window*
Returns the path name of the top-of-hierarchy window containing *window*. In standard Tk this will always be a toplevel widget, but extensions may create other kinds of top-of-hierarchy widgets.

winfo viewable *window*
Returns 1 if *window* and all of its ancestors up through the nearest toplevel window are mapped. Returns 0 if any of these windows are not mapped.

winfo visual *window*
Returns one of the following strings to indicate the visual class for *window*: **directcolor**, **grayscale**, **pseudocolor**, **staticcolor**, **staticgray**, or **truecolor**.

winfo visualid *window*
Returns the X identifier for the visual for *window*.

winfo visualsavailable *window* ?includeids?
Returns a list whose elements describe the visuals available for *window*'s screen. Each element consists of a visual class followed by an integer depth. The class has the same form as returned by **winfo visual**. The depth gives the number of bits per pixel in the visual. In addition, if the **includeids** argument is provided, then the depth is followed by the X identifier for the visual.

winfo vrootheight *window*
Returns the height of the virtual root window associated with *window* if there is one; otherwise returns the height of *window*'s screen.

winfo vrootwidth *window*
Returns the width of the virtual root window associated with *window* if there is one; otherwise returns the width of *window*'s screen.

winfo vrootx *window*
Returns the x-offset of the virtual root window associated with *window*, relative to the root window of its screen. This is normally either zero or negative. Returns 0 if there is no virtual root window for *window*.

winfo vrooty *window*
Returns the y-offset of the virtual root window associated with *window*, relative to the root window of its screen. This is normally either zero or negative. Returns 0 if there is no virtual root window for *window*.

winfo width *window*
Returns a decimal string giving *window*'s width in pixels. When a window is first created its width will be 1 pixel; the width will eventually be changed by a geometry manager to fulfil the window's needs. If you need the true width immediately after creating a widget, invoke update to force the geometry manager to arrange it, or use **winfo reqwidth** to get the window's requested width instead of its actual width.

winfo x *window*
Returns a decimal string giving the x-coordinate, in *window*'s parent, of the upper-left corner

of *window*'s border (or *window* if it has no border).

winfo y *window*
Returns a decimal string giving the y-coordinate, in *window*'s parent, of the upper-left corner of *window*'s border (or *window* if it has no border).

Example

Print where the mouse pointer is and what window it is currently over:

```
lassign [winfo pointerxy .] x y
puts -nonewline "Mouse pointer at ($x,$y) which is "
set win [winfo containing $x $y]
if {$win eq ""} {
    puts "over no window"
} else {
    puts "over $win"
}
```

Copyright (c) 1990-1994 The Regents of the University of California.
Copyright (c) 1994-1997 Sun Microsystems, Inc.

wish

Keywords: wish, application, argument, interpreter, prompt, script file, shell, toolkit, toplevel

Name

wish - Simple windowing shell

Synopsis

wish ?**options**? ?*fileName arg* ...?

Options

-encoding *name*
Specifies the encoding of the text stored in *fileName*. This option is only recognized prior to the *fileName* argument.

-colormap *new*
Specifies that the window should have a new private colormap instead of using the default colormap for the screen.

-display *display*
Display (and screen) on which to display window.

-geometry *geometry*
Initial geometry to use for window. If this option is specified, its value is stored in the geometry global variable of the application's Tcl interpreter.

-name *name*
Use *name* as the title to be displayed in the window, and as the name of the interpreter for send commands.

-sync
Execute all X server commands synchronously, so that errors are reported immediately. This will result in much slower execution, but it is useful for debugging.

-use *id*
Specifies that the main window for the application is to be embedded in the window whose identifier is *id*, instead of being created as an independent toplevel window. *Id* must be specified in the same way as the value for the **-use** option for toplevel widgets (i.e. it has a form like that returned by the winfo id command). Note that on some platforms this will only work correctly if *id* refers to a Tk frame or toplevel that has its **-container** option enabled.

-visual *visual*
Specifies the visual to use for the window. *Visual* may have any of the forms supported by the Tk_GetVisual procedure.

--
Pass all remaining arguments through to the script's argv variable without interpreting them. This provides a mechanism for passing arguments such as **-name** to a script instead of having **wish** interpret them.

Description

Wish is a simple program consisting of the Tcl command language, the Tk toolkit, and a main program that reads commands from standard input or from a file. It creates a main window and then processes Tcl commands. If **wish** is invoked with arguments, then the first few arguments, ?**-encoding** *name*? ?*fileName*?, specify the name of a script file, and, optionally, the encoding of the text data stored in that script file. A value for *fileName* is recognized if the appropriate argument does not start with "-".

If there are no arguments, or the arguments do not specify a *fileName*, then wish reads Tcl commands interactively from standard input. It will continue processing commands until all windows have been deleted or until end-of-file is reached on standard input. If there exists a file "**.wishrc**" in the home directory of the user, **wish** evaluates the file as a Tcl script just before reading the first command from standard input.

If arguments to **wish** do specify a *fileName*, then *fileName* is treated as the name of a script file. **Wish** will evaluate the script in *fileName* (which presumably creates a user interface), then it will respond to events until all windows have been deleted. Commands will not be read from standard input. There is no automatic evaluation of "**.wishrc**" when the name of a script file is presented on the **wish** command line, but the script file can always source it if desired.

Note that on Windows, the **wish***version***.exe** program varies from the tclsh*version***.exe** program in an additional important way: it does not connect to a standard Windows console and is instead a windowed program. Because of this, it additionally provides access to its own console command.

Option processing

Wish automatically processes all of the command-line options described in the Options summary above. Any other command-line arguments besides these are passed through to the application using the argc and argv variables described later.

Application name and class

The name of the application, which is used for purposes such as send commands, is taken from the **-name** option, if it is specified; otherwise it is taken from *fileName*, if it is specified, or from the command name by which **wish** was invoked. In the last two cases, if the name contains a "/"

character, then only the characters after the last slash are used as the application name.

The class of the application, which is used for purposes such as specifying options with a **RESOURCE_MANAGER** property or .Xdefaults file, is the same as its name except that the first letter is capitalized.

Variables

Wish sets the following global Tcl variables:

argc
: Contains a count of the number of *arg* arguments (0 if none), not including the options described above.

argv
: Contains a Tcl list whose elements are the *arg* arguments that follow a **--** option or do not match any of the options described in Options above, in order, or an empty string if there are no such arguments.

argv0
: Contains *fileName* if it was specified. Otherwise, contains the name by which **wish** was invoked.

geometry
: If the **-geometry** option is specified, **wish** copies its value into this variable. If the variable still exists after *fileName* has been evaluated, **wish** uses the value of the variable in a wm geometry command to set the main window's geometry.

tcl_interactive
: Contains 1 if **wish** is reading commands interactively (*fileName* was not specified and standard input is a terminal-like device), 0 otherwise.

Script files

If you create a Tcl script in a file whose first line is

```
#!/usr/local/bin/wish
```

then you can invoke the script file directly from your shell if you mark it as executable. This assumes that **wish** has been installed in the default location in /usr/local/bin; if it is installed somewhere else then you will have to modify the above line to match. Many UNIX systems do not allow the **#!** line to exceed about 30 characters in length, so be sure that the **wish** executable can be accessed with a short file name.

An even better approach is to start your script files with the following three lines:

```
#!/bin/sh
# the next line restarts using wish \
exec wish "$0" ${1+"$@"}
```

This approach has three advantages over the approach in the previous paragraph. First, the location of the **wish** binary does not have to be hard-wired into the script: it can be anywhere in your shell search path. Second, it gets around the 30-character file name limit in the previous approach. Third, this approach will work even if **wish** is itself a shell script (this is done on some systems in order to handle multiple architectures or operating systems: the **wish** script selects one of several binaries to run). The three lines cause both **sh** and **wish** to process the script, but

the exec is only executed by **sh**. **sh** processes the script first; it treats the second line as a comment and executes the third line. The exec statement cause the shell to stop processing and instead to start up **wish** to reprocess the entire script. When **wish** starts up, it treats all three lines as comments, since the backslash at the end of the second line causes the third line to be treated as part of the comment on the second line.

The end of a script file may be marked either by the physical end of the medium, or by the character, "\032" ("\u001a", control-Z). If this character is present in the file, the **wish** application will read text up to but not including the character. An application that requires this character in the file may encode it as "\032", "\x1a", or "\u001a"; or may generate it by use of commands such as format or binary.

Prompts

When **wish** is invoked interactively it normally prompts for each command with "**%** ". You can change the prompt by setting the variables **tcl_prompt1** and **tcl_prompt2**. If variable **tcl_prompt1** exists then it must consist of a Tcl script to output a prompt; instead of outputting a prompt **wish** will evaluate the script in **tcl_prompt1**. The variable **tcl_prompt2** is used in a similar way when a newline is typed but the current command is not yet complete; if **tcl_prompt2** is not set then no prompt is output for incomplete commands.

Copyright (c) 1991-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

wm

Keywords: wm, aspect ratio, deiconify, focus model, geometry, grid, group, icon, iconify, increments, position, size, title, top-level window, units, window manager

Name

wm - Communicate with window manager

Synopsis

wm *option window* ?*args*?

Description

The **wm** command is used to interact with window managers in order to control such things as the title for a window, its geometry, or the increments in terms of which it may be resized. The **wm** command can take any of a number of different forms, depending on the *option* argument. All of the forms expect at least one additional argument, *window*, which must be the path name of a top-level window.

The legal forms for the **wm** command are:

wm aspect *window* ?*minNumer minDenom maxNumer maxDenom*?

If *minNumer*, *minDenom*, *maxNumer*, and *maxDenom* are all specified, then they will be passed to the window manager and the window manager should use them to enforce a range of acceptable aspect ratios for *window*. The aspect ratio of *window* (width/length) will be constrained to lie between *minNumer*/*minDenom* and *maxNumer*/*maxDenom*. If *minNumer* etc. are all specified as empty strings, then any existing aspect ratio restrictions are removed. If *minNumer* etc. are specified, then the command returns an empty string. Otherwise, it returns a Tcl list containing four elements, which are the current values of *minNumer*,

minDenom, *maxNumer*, and *maxDenom* (if no aspect restrictions are in effect, then an empty string is returned).

wm attributes *window*
see below ...

wm attributes *window* ?option?
see below ...

wm attributes *window* ?option value option value...?
This subcommand returns or sets platform specific attributes associated with a window. The first form returns a list of the platform specific flags and their values. The second form returns the value for the specific option. The third form sets one or more of the values. The values are as follows:

All platforms support the following attributes (though X11 users should see the notes below):

-alpha
Specifies the alpha transparency level of the toplevel. It accepts a value from **0.0** (fully transparent) to **1.0** (opaque). Values outside that range will be constrained. Where not supported, the **-alpha** value remains at **1.0**.

-fullscreen
Places the window in a mode that takes up the entire screen, has no borders, and covers the general use area (i.e. Start menu and taskbar on Windows, dock and menubar on macOS, general window decorations on X11).

-topmost
Specifies whether this is a topmost window (displays above all other windows).

On Windows, the following attributes may be set.

-disabled
Specifies whether the window is in a disabled state.

-toolwindow
Specifies a toolwindow style window (as defined in the MSDN).

-transparentcolor
Specifies the transparent color index of the toplevel. It takes any color value accepted by Tk_GetColor. If the empty string is specified (default), no transparent color is used. This is supported on Windows 2000/XP+. Where not supported, the **-transparentcolor** value remains at **{}**.

On macOS, the following attributes may be set.

-modified
Specifies the modification state of the window (determines whether the window close widget contains the modification indicator and whether the proxy icon is draggable).

-notify
Specifies process notification state (bouncing of the application dock icon).

-titlepath
Specifies the path of the file referenced as the window proxy icon (which can be dragged and dropped in lieu of the file's finder icon).

-transparent
Makes the window content area transparent and turns off the window shadow. For the

transparency to be effective, the toplevel background needs to be set to a color with some alpha, e.g. "systemTransparent".

On X11, the following attributes may be set. These are not supported by all window managers, and will have no effect under older WMs.

-type

Requests that the window should be interpreted by the window manager as being of the specified type(s). This may cause the window to be decorated in a different way or otherwise managed differently, though exactly what happens is entirely up to the window manager. A list of types may be used, in order of preference. The following values are mapped to constants defined in the EWMH specification (using others is possible, but not advised):

desktop
indicates a desktop feature,

dock
indicates a dock/panel feature,

toolbar
indicates a toolbar window that should be acting on behalf of another window, as indicated with wm transient,

menu
indicates a torn-off menu that should be acting on behalf of another window, as indicated with wm transient,

utility
indicates a utility window (e.g., palette or toolbox) that should be acting on behalf of another window, as indicated with wm transient,

splash
indicates a splash screen, displayed during application start up,

dialog
indicates a general dialog window, that should be acting on behalf of another window, as indicated with wm transient,

dropdown_menu
indicates a menu summoned from a menu bar, which should usually also be set to be override-redirected (with wm overrideredirect),

popup_menu
indicates a popup menu, which should usually also be set to be override-redirected (with wm overrideredirect),

tooltip
indicates a tooltip window, which should usually also be set to be override-redirected (with wm overrideredirect),

notification
indicates a window that provides a background notification of some event, which should usually also be set to be override-redirected (with wm overrideredirect),

combo
indicates the drop-down list of a combobox widget, which should usually also be set to be override-redirected (with wm overrideredirect),

dnd
indicates a window that represents something being dragged, which should usually also be set to be override-redirected (with wm overrideredirect),

normal
indicates a window that has no special interpretation.

-zoomed
Requests that the window should be maximized. This is the same as **wm state zoomed** on Windows and macOS.

On X11, changes to window attributes are performed asynchronously. Querying the value of an attribute returns the current state, which will not be the same as the value most recently set if the window manager has not yet processed the request or if it does not support the attribute.

wm client *window* ?*name*?
If *name* is specified, this command stores *name* (which should be the name of the host on which the application is executing) in *window*'s **WM_CLIENT_MACHINE** property for use by the window manager or session manager. The command returns an empty string in this case. If *name* is not specified, the command returns the last name set in a **wm client** command for *window*. If *name* is specified as an empty string, the command deletes the **WM_CLIENT_MACHINE** property from *window*.

wm colormapwindows *window* ?*windowList*?
This command is used to manipulate the **WM_COLORMAP_WINDOWS** property, which provides information to the window managers about windows that have private colormaps.

If *windowList* is not specified, the command returns a list whose elements are the names of the windows in the **WM_COLORMAP_WINDOWS** property. If *windowList* is specified, it consists of a list of window path names; the command overwrites the **WM_COLORMAP_WINDOWS** property with the given windows and returns an empty string. The **WM_COLORMAP_WINDOWS** property should normally contain a list of the internal windows within *window* whose colormaps differ from their parents.

The order of the windows in the property indicates a priority order: the window manager will attempt to install as many colormaps as possible from the head of this list when *window* gets the colormap focus. If *window* is not included among the windows in *windowList*, Tk implicitly adds it at the end of the **WM_COLORMAP_WINDOWS** property, so that its colormap is lowest in priority. If **wm colormapwindows** is not invoked, Tk will automatically set the property for each top-level window to all the internal windows whose colormaps differ from their parents, followed by the top-level itself; the order of the internal windows is undefined. See the ICCCM documentation for more information on the **WM_COLORMAP_WINDOWS** property.

wm command *window* ?*value*?
If *value* is specified, this command stores *value* in *window*'s **WM_COMMAND** property for use by the window manager or session manager and returns an empty string. *Value* must have proper list structure; the elements should contain the words of the command used to invoke the application. If *value* is not specified then the command returns the last value set in a **wm**

command command for *window*. If *value* is specified as an empty string, the command deletes the **WM_COMMAND** property from *window*.

wm deiconify *window*

Arrange for *window* to be displayed in normal (non-iconified) form. This is done by mapping the window. If the window has never been mapped then this command will not map the window, but it will ensure that when the window is first mapped it will be displayed in de-iconified form. On Windows, a deiconified window will also be raised and be given the focus (made the active window). Returns an empty string.

wm focusmodel *window* ?active|passive?

If **active** or **passive** is supplied as an optional argument to the command, then it specifies the focus model for *window*. In this case the command returns an empty string. If no additional argument is supplied, then the command returns the current focus model for *window*.

An **active** focus model means that *window* will claim the input focus for itself or its descendants, even at times when the focus is currently in some other application. **Passive** means that *window* will never claim the focus for itself: the window manager should give the focus to *window* at appropriate times. However, once the focus has been given to *window* or one of its descendants, the application may re-assign the focus among *window*'s descendants. The focus model defaults to **passive**, and Tk's focus command assumes a passive model of focusing.

wm forget *window*

The *window* will be unmapped from the screen and will no longer be managed by **wm**. Windows created with the toplevel command will be treated like frame windows once they are no longer managed by **wm**, however, the **-menu** configuration will be remembered and the menus will return once the widget is managed again.

wm frame *window*

If *window* has been reparented by the window manager into a decorative frame, the command returns the platform specific window identifier for the outermost frame that contains *window* (the window whose parent is the root or virtual root). If *window* has not been reparented by the window manager then the command returns the platform specific window identifier for *window*.

wm geometry *window* ?*newGeometry*?

If *newGeometry* is specified, then the geometry of *window* is changed and an empty string is returned. Otherwise the current geometry for *window* is returned (this is the most recent geometry specified either by manual resizing or in a **wm geometry** command). *NewGeometry* has the form **=***width***x***height***±***x***±***y*, where any of **=**, *width***x***height*, or **±***x***±***y* may be omitted. *Width* and *height* are positive integers specifying the desired dimensions of *window*. If *window* is gridded (see Gridded geometry management below) then the dimensions are specified in grid units; otherwise they are specified in pixel units.

X and *y* specify the desired location of *window* on the screen, in pixels. If *x* is preceded by **+**, it specifies the number of pixels between the left edge of the screen and the left edge of *window*'s border; if preceded by **-** then *x* specifies the number of pixels between the right edge of the screen and the right edge of *window*'s border. If *y* is preceded by **+** then it specifies the number of pixels between the top of the screen and the top of *window*'s border;

if *y* is preceded by **-** then it specifies the number of pixels between the bottom of *window*'s border and the bottom of the screen.

If *newGeometry* is specified as an empty string then any existing user-specified geometry for *window* is cancelled, and the window will revert to the size requested internally by its widgets.

Note that this is related to winfo geometry, but not the same. That can only query the geometry, and always reflects Tk's current understanding of the actual size and location of *window*, whereas **wm geometry** allows both setting and querying of the *window manager*'s understanding of the size and location of the window. This can vary significantly, for example to reflect the addition of decorative elements to *window* such as title bars, and window managers are not required to precisely follow the requests made through this command.

wm grid *window* ?*baseWidth baseHeight widthInc heightInc*?

This command indicates that *window* is to be managed as a gridded window. It also specifies the relationship between grid units and pixel units. *BaseWidth* and *baseHeight* specify the number of grid units corresponding to the pixel dimensions requested internally by *window* using Tk_GeometryRequest. *WidthInc* and *heightInc* specify the number of pixels in each horizontal and vertical grid unit. These four values determine a range of acceptable sizes for *window*, corresponding to grid-based widths and heights that are non-negative integers. Tk will pass this information to the window manager; during manual resizing, the window manager will restrict the window's size to one of these acceptable sizes.

Furthermore, during manual resizing the window manager will display the window's current size in terms of grid units rather than pixels. If *baseWidth* etc. are all specified as empty strings, then *window* will no longer be managed as a gridded window. If *baseWidth* etc. are specified then the return value is an empty string.

Otherwise the return value is a Tcl list containing four elements corresponding to the current *baseWidth*, *baseHeight*, *widthInc*, and *heightInc*; if *window* is not currently gridded, then an empty string is returned.

Note that this command should not be needed very often, since the Tk_SetGrid library procedure and the **setGrid** option provide easier access to the same functionality.

wm group *window* ?*pathName*?

If *pathName* is specified, it gives the path name for the leader of a group of related windows. The window manager may use this information, for example, to unmap all of the windows in a group when the group's leader is iconified. *PathName* may be specified as an empty string to remove *window* from any group association. If *pathName* is specified then the command returns an empty string; otherwise it returns the path name of *window*'s current group leader, or an empty string if *window* is not part of any group.

wm iconbitmap *window* ?*bitmap*?

If *bitmap* is specified, then it names a bitmap in the standard forms accepted by Tk (see the Tk_GetBitmap manual entry for details). This bitmap is passed to the window manager to be displayed in *window*'s icon, and the command returns an empty string. If an empty string is specified for *bitmap*, then any current icon bitmap is cancelled for *window*. If *bitmap* is specified then the command returns an empty string. Otherwise it returns the name of the

current icon bitmap associated with *window*, or an empty string if *window* has no icon bitmap. On the Windows operating system, an additional flag is supported:

wm iconbitmap *window* ?-default? ?*image*?

If the **-default** flag is given, the icon is applied to all toplevel windows (existing and future) to which no other specific icon has yet been applied. In addition to bitmap image types, a full path specification to any file which contains a valid Windows icon is also accepted (usually .ico or .icr files), or any file for which the shell has assigned an icon. Tcl will first test if the file contains an icon, then if it has an assigned icon, and finally, if that fails, test for a bitmap.

wm iconify *window*

Arrange for *window* to be iconified. It *window* has not yet been mapped for the first time, this command will arrange for it to appear in the iconified state when it is eventually mapped.

wm iconmask *window* ?*bitmap*?

If *bitmap* is specified, then it names a bitmap in the standard forms accepted by Tk (see the Tk_GetBitmap manual entry for details). This bitmap is passed to the window manager to be used as a mask in conjunction with the **iconbitmap** option: where the mask has zeroes no icon will be displayed; where it has ones, the bits from the icon bitmap will be displayed. If an empty string is specified for *bitmap* then any current icon mask is cancelled for *window* (this is equivalent to specifying a bitmap of all ones). If *bitmap* is specified then the command returns an empty string. Otherwise it returns the name of the current icon mask associated with *window*, or an empty string if no mask is in effect.

wm iconname *window* ?*newName*?

If *newName* is specified, then it is passed to the window manager; the window manager should display *newName* inside the icon associated with *window*. In this case an empty string is returned as result. If *newName* is not specified then the command returns the current icon name for *window*, or an empty string if no icon name has been specified (in this case the window manager will normally display the window's title, as specified with the **wm title** command).

wm iconphoto *window* ?-default? *image1* ?*image2* ...?

Sets the titlebar icon for *window* based on the named photo images. If **-default** is specified, this is applied to all future created toplevels as well. The data in the images is taken as a snapshot at the time of invocation. If the images are later changed, this is not reflected to the titlebar icons. Multiple images are accepted to allow different images sizes (e.g., 16x16 and 32x32) to be provided. The window manager may scale provided icons to an appropriate size.

On Windows, the images are packed into a Windows icon structure. This will override an ico specified to **wm iconbitmap**, and vice versa.

On X, the images are arranged into the _NET_WM_ICON X property, which most modern window managers support. A **wm iconbitmap** may exist simultaneously. It is recommended to use not more than 2 icons, placing the larger icon first.

On Macintosh, the first image called is loaded into an OS-native icon format, and becomes the application icon in dialogs, the Dock, and other contexts. At the script level the command will accept only the first image passed in the parameters as support for multiple sizes/resolutions

on macOS is outside Tk's scope. Developers should use the largest icon they can support (preferably 512 pixels) to ensure smooth rendering on the Mac.

wm iconposition *window* ?*x y*?
If *x* and *y* are specified, they are passed to the window manager as a hint about where to position the icon for *window*. In this case an empty string is returned. If *x* and *y* are specified as empty strings then any existing icon position hint is cancelled. If neither *x* nor *y* is specified, then the command returns a Tcl list containing two values, which are the current icon position hints (if no hints are in effect then an empty string is returned).

wm iconwindow *window* ?*pathName*?
If *pathName* is specified, it is the path name for a window to use as icon for *window*: when *window* is iconified then *pathName* will be mapped to serve as icon, and when *window* is de-iconified then *pathName* will be unmapped again. If *pathName* is specified as an empty string then any existing icon window association for *window* will be cancelled. If the *pathName* argument is specified then an empty string is returned. Otherwise the command returns the path name of the current icon window for *window*, or an empty string if there is no icon window currently specified for *window*. Button press events are disabled for *window* as long as it is an icon window; this is needed in order to allow window managers to "own" those events. Note that not all window managers support the notion of an icon window, and the concept is entirely meaningless on non-X11 platforms.

wm manage *widget*
The *widget* specified will become a stand alone top-level window. The window will be decorated with the window managers title bar, etc. Only *frame*, *labelframe* and *toplevel* widgets can be used with this command. Attempting to pass any other widget type will raise an error. Attempting to manage a *toplevel* widget is benign and achieves nothing. See also Geometry management.

wm maxsize *window* ?*width height*?
If *width* and *height* are specified, they give the maximum permissible dimensions for *window*. For gridded windows the dimensions are specified in grid units; otherwise they are specified in pixel units. The window manager will restrict the window's dimensions to be less than or equal to *width* and *height*. If *width* and *height* are specified, then the command returns an empty string. Otherwise it returns a Tcl list with two elements, which are the maximum width and height currently in effect. The maximum size defaults to the size of the screen. See the sections on geometry management below for more information.

wm minsize *window* ?*width height*?
If *width* and *height* are specified, they give the minimum permissible dimensions for *window*. For gridded windows the dimensions are specified in grid units; otherwise they are specified in pixel units. The window manager will restrict the window's dimensions to be greater than or equal to *width* and *height*. If *width* and *height* are specified, then the command returns an empty string. Otherwise it returns a Tcl list with two elements, which are the minimum width and height currently in effect. The minimum size defaults to one pixel in each dimension. See the sections on geometry management below for more information.

wm overrideredirect *window* ?*boolean*?
If *boolean* is specified, it must have a proper boolean form and the override-redirect flag for *window* is set to that value. If *boolean* is not specified then **1** or **0** is returned to indicate whether or not the override-redirect flag is currently set for *window*. Setting the

override-redirect flag for a window causes it to be ignored by the window manager; among other things, this means that the window will not be reparented from the root window into a decorative frame and the user will not be able to manipulate the window using the normal window manager mechanisms.

Note that the override-redirect flag is only guaranteed to be taken notice of when the window is first mapped or when mapped after the state is changed from withdrawn to normal. Some, but not all, platforms will take notice at additional times.

wm positionfrom *window* ?*who*?

If *who* is specified, it must be either **program** or **user**, or an abbreviation of one of these two. It indicates whether *window*'s current position was requested by the program or by the user. Many window managers ignore program-requested initial positions and ask the user to manually position the window; if **user** is specified then the window manager should position the window at the given place without asking the user for assistance. If *who* is specified as an empty string, then the current position source is cancelled. If *who* is specified, then the command returns an empty string. Otherwise it returns **user** or **program** to indicate the source of the window's current position, or an empty string if no source has been specified yet. Most window managers interpret "no source" as equivalent to **program**. Tk will automatically set the position source to **user** when a **wm geometry** command is invoked, unless the source has been set explicitly to **program**.

wm protocol *window* ?*name*? ?*command*?

This command is used to manage window manager protocols. The *name* argument in the **wm protocol** command is the name of an atom corresponding to a window manager protocol. Examples include **WM_DELETE_WINDOW** or **WM_SAVE_YOURSELF** or **WM_TAKE_FOCUS**.

A *window manager protocol* is a class of messages sent from a window manager to a Tk application outside of the normal event processing system. The main example is the **WM_DELETE_WINDOW** protocol; these messages are sent when the user clicks the close widget in the title bar of a window. Handlers for window manager protocols are installed with the **wm protocol** command. As a rule, if no handler has been installed for a protocol by the **wm protocol** command then all messages of that protocol are ignored. The **WM_DELETE_WINDOW** protocol is an exception to this rule. At start-up Tk installs a handler for this protocol, which responds by destroying the window. The **wm protocol** command can be used to replace this default handler by one which responds differently.

The list of available window manager protocols depends on the window manager, but all window managers supported by Tk provide **WM_DELETE_WINDOW**. On the Windows platform, a **WM_SAVE_YOURSELF** message is sent on user logout or system restart.

If both *name* and *command* are specified, then *command* becomes the handler for the protocol specified by *name*. The atom for *name* will be added to *window*'s **WM_PROTOCOLS** property to tell the window manager that the application has a handler for the protocol specified by *name*, and *command* will be invoked in the future whenever the window manager sends a message of that protocol to the Tk application. In this case the **wm protocol** command returns an empty string. If *name* is specified but *command* is not, then the current handler for *name* is returned, or an empty string if there is no handler defined for *name* (as a

special case, the default handler for **WM_DELETE_WINDOW** is not returned). If *command* is specified as an empty string then the atom for *name* is removed from the **WM_PROTOCOLS** property of *window* and the handler is destroyed; an empty string is returned. Lastly, if neither *name* nor *command* is specified, the **wm protocol** command returns a list of all of the protocols for which handlers are currently defined for *window*.

wm resizable *window* ?*width height*?

This command controls whether or not the user may interactively resize a top-level window. If *width* and *height* are specified, they are boolean values that determine whether the width and height of *window* may be modified by the user. In this case the command returns an empty string. If *width* and *height* are omitted then the command returns a list with two 0/1 elements that indicate whether the width and height of *window* are currently resizable. By default, windows are resizable in both dimensions. If resizing is disabled, then the window's size will be the size from the most recent interactive resize or **wm geometry** command. If there has been no such operation then the window's natural size will be used.

wm sizefrom *window* ?*who*?

If *who* is specified, it must be either **program** or **user**, or an abbreviation of one of these two. It indicates whether *window*'s current size was requested by the program or by the user. Some window managers ignore program-requested sizes and ask the user to manually size the window; if **user** is specified then the window manager should give the window its specified size without asking the user for assistance. If *who* is specified as an empty string, then the current size source is cancelled. If *who* is specified, then the command returns an empty string. Otherwise it returns **user** or **window** to indicate the source of the window's current size, or an empty string if no source has been specified yet. Most window managers interpret "no source" as equivalent to **program**.

wm stackorder *window* ?*isabove*|*isbelow window*?

The **stackorder** command returns a list of toplevel windows in stacking order, from lowest to highest. When a single toplevel window is passed, the returned list recursively includes all of the window's children that are toplevels. Only those toplevels that are currently mapped to the screen are returned. The **stackorder** command can also be used to determine if one toplevel is positioned above or below a second toplevel. When two window arguments separated by either **isabove** or **isbelow** are passed, a boolean result indicates whether or not the first window is currently above or below the second window in the stacking order.

wm state *window* ?*newstate*?

If *newstate* is specified, the window will be set to the new state, otherwise it returns the current state of *window*: either **normal**, **iconic**, **withdrawn**, **icon**, or (Windows and macOS only) **zoomed**. The difference between **iconic** and **icon** is that **iconic** refers to a window that has been iconified (e.g., with the **wm iconify** command) while **icon** refers to a window whose only purpose is to serve as the icon for some other window (via the **wm iconwindow** command). The **icon** state cannot be set.

wm title *window* ?*string*?

If *string* is specified, then it will be passed to the window manager for use as the title for *window* (the window manager should display this string in *window*'s title bar). In this case the command returns an empty string. If *string* is not specified then the command returns the current title for the *window*. The title for a window defaults to its name.

wm transient *window* ?*container*?

If *container* is specified, then the window manager is informed that *window* is a transient window (e.g. pull-down menu) working on behalf of *container* (where *container* is the path name for a top-level window). If *container* is specified as an empty string then *window* is marked as not being a transient window any more. Otherwise the command returns the path name of *window*'s current container, or an empty string if *window* is not currently a transient window. A transient window will mirror state changes in the container and inherit the state of the container when initially mapped. The directed graph with an edge from each transient to its container must be acyclic. In particular, it is an error to attempt to make a window a transient of itself. The window manager may also decorate a transient window differently, removing some features normally present (e.g., minimize and maximize buttons) though this is entirely at the discretion of the window manager.

wm withdraw *window*

Arranges for *window* to be withdrawn from the screen. This causes the window to be unmapped and forgotten about by the window manager. If the window has never been mapped, then this command causes the window to be mapped in the withdrawn state. Not all window managers appear to know how to handle windows that are mapped in the withdrawn state. Note that it sometimes seems to be necessary to withdraw a window and then re-map it (e.g. with **wm deiconify**) to get some window managers to pay attention to changes in window attributes such as group.

Geometry management

By default a top-level window appears on the screen in its *natural size*, which is the one determined internally by its widgets and geometry managers. If the natural size of a top-level window changes, then the window's size changes to match. A top-level window can be given a size other than its natural size in two ways. First, the user can resize the window manually using the facilities of the window manager, such as resize handles. Second, the application can request a particular size for a top-level window using the **wm geometry** command. These two cases are handled identically by Tk; in either case, the requested size overrides the natural size. You can return the window to its natural by invoking **wm geometry** with an empty *geometry* string.

Normally a top-level window can have any size from one pixel in each dimension up to the size of its screen. However, you can use the **wm minsize** and **wm maxsize** commands to limit the range of allowable sizes. The range set by **wm minsize** and **wm maxsize** applies to all forms of resizing, including the window's natural size as well as manual resizes and the **wm geometry** command. You can also use the command **wm resizable** to completely disable interactive resizing in one or both dimensions.

The **wm manage** and **wm forget** commands may be used to perform undocking and docking of windows. After a widget is managed by **wm manage** command, all other **wm** subcommands may be used with the widget. Only widgets created using the toplevel command may have an attached menu via the **-menu** configure option. A toplevel widget may be used as a frame and managed with any of the other geometry managers after using the **wm forget** command. Any menu associated with a toplevel widget will be hidden when managed by another geometry managers. The menus will reappear once the window is managed by **wm**. All custom bindtags for widgets in a subtree that have their top-level widget changed via a **wm manage** or **wm forget** command, must be redone to adjust any top-level widget path in the bindtags. Bindtags that have not been customized do not have to be redone.

Gridded geometry management

Gridded geometry management occurs when one of the widgets of an application supports a range of useful sizes. This occurs, for example, in a text editor where the scrollbars, menus, and other adornments are fixed in size but the edit widget can support any number of lines of text or characters per line. In this case, it is usually desirable to let the user specify the number of lines or characters-per-line, either with the **wm geometry** command or by interactively resizing the window. In the case of text, and in other interesting cases also, only discrete sizes of the window make sense, such as integral numbers of lines and characters-per-line; arbitrary pixel sizes are not useful.

Gridded geometry management provides support for this kind of application. Tk (and the window manager) assume that there is a grid of some sort within the application and that the application should be resized in terms of *grid units* rather than pixels. Gridded geometry management is typically invoked by turning on the **setGrid** option for a widget; it can also be invoked with the **wm grid** command or by calling Tk_SetGrid. In each of these approaches the particular widget (or sometimes code in the application as a whole) specifies the relationship between integral grid sizes for the window and pixel sizes. To return to non-gridded geometry management, invoke **wm grid** with empty argument strings.

When gridded geometry management is enabled then all the dimensions specified in **wm minsize**, **wm maxsize**, and **wm geometry** commands are treated as grid units rather than pixel units. Interactive resizing is also carried out in even numbers of grid units rather than pixels.

Bugs

Most existing window managers appear to have bugs that affect the operation of the **wm** command. For example, some changes will not take effect if the window is already active: the window will have to be withdrawn and de-iconified in order to make the change happen.

Examples

A fixed-size window that says that it is fixed-size too:

```
toplevel .fixed
wm title     .fixed "Fixed-size Window"
wm resizable .fixed 0 0
```

A simple dialog-like window, centred on the screen:

```
# Create and arrange the dialog contents.
toplevel .msg
label  .msg.l  -text "This is a very simple dialog demo."
button .msg.ok -text OK -default active -command {destroy .msg}
pack .msg.ok -side bottom -fill x
pack .msg.l  -expand 1    -fill both

# Now set the widget up as a centred dialog.

# But first, we need the geometry managers to finish setting
# up the interior of the dialog, for which we need to run the
```

```
# event loop with the widget hidden completely...
wm withdraw .msg
update
set x [expr {([winfo screenwidth .] - [winfo width .msg]) / 2}]
set y [expr {([winfo screenheight .] - [winfo height .msg]) / 2}]
wm geometry  .msg +$x+$y
wm transient .msg .
wm title     .msg "Dialog demo"
wm deiconify .msg
```

Copyright (c) 1991-1994 The Regents of the University of California.
Copyright (c) 1994-1996 Sun Microsystems, Inc.

Ttk

ttk_button

Keywords: ttk_button, widget, button, default, command

Name

ttk::button - Widget that issues a command when pressed

Synopsis

ttk::button *pathName* ?*options*?

Description

A **ttk::button** widget displays a textual label and/or image, and evaluates a command when pressed.

Standard options

-class, **-compound**, **-cursor**, **-image**, **-state**, **-style**, **-takefocus**, **-text**, **-textvariable**, **-underline**, **-width**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-command (command/Command)
A script to evaluate when the widget is invoked.

-default (default/Default)
May be set to one of **normal**, **active**, or **disabled**. In a dialog box, one button may be designated the "default" button (meaning, roughly, "the one that gets invoked when the user presses "). **active** indicates that this is currently the default button; **normal** means that it may become the default button, and **disabled** means that it is not defaultable. The default is **normal**.

Depending on the theme, the default button may be displayed with an extra highlight ring, or with a different border color.

Widget command

In addition to the standard **cget**, **configure**, **identify**, **instate**, and **state** commands, buttons support the following additional widget commands:

***pathName* invoke**
Invokes the command associated with the button.

Standard styles

Ttk::button widgets support the **Toolbutton** style in all standard themes, which is useful for creating widgets for toolbars.

Styling options

The class name for a **ttk::button** is **TButton**.

Dynamic states: **active**, **disabled**, **pressed**, **readonly**.

TButton styling options configurable with ttk::style are:

-anchor *anchor*

-background *color*

-bordercolor *color*

-compound *compound*

-darkcolor *color*

-foreground *color*

-font *font*

-highlightcolor *color*

-highlightthickness *amount*

-lightcolor *color*

-padding *padding*

-relief *relief*

-shiftrelief *amount*
-shiftrelief specifies how far the button contents are shifted down and right in the *pressed* state. This action provides additional skeumorphic feedback.

-width *amount*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2004 Joe English

ttk_checkbutton

Keywords: ttk_checkbutton, widget, button, toggle, check, option

Name

ttk::checkbutton - On/off widget

Synopsis

ttk::checkbutton *pathName* ?*options*?

Description

A **ttk::checkbutton** widget is used to show or change a setting. It has two states, selected and deselected. The state of the checkbutton may be linked to a Tcl variable.

Standard options

-class, **-compound**, **-cursor**, **-image**, **-state**, **-style**, **-takefocus**, **-text**, **-textvariable**, **-underline**, **-width**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-command (command/Command)
A Tcl script to execute whenever the widget is invoked.

-offvalue (offValue/OffValue)
The value to store in the associated **-variable** when the widget is deselected. Defaults to **0**.

-onvalue (onValue/OnValue)
The value to store in the associated **-variable** when the widget is selected. Defaults to **1**.

-variable (variable/Variable)
The name of a global variable whose value is linked to the widget. Defaults to the widget pathname if not specified.

Widget command

In addition to the standard **cget**, **configure**, **identify**, **instate**, and **state** commands, checkbuttons support the following additional widget commands:

***pathname* invoke**
Toggles between the selected and deselected states and evaluates the associated **-command**. If the widget is currently selected, sets the **-variable** to the **-offvalue** and deselects the widget; otherwise, sets the **-variable** to the **-onvalue** Returns the result of the **-command**.

Widget states

The widget does not respond to user input if the **disabled** state is set. The widget sets the **selected** state whenever the linked **-variable** is set to the widget's **-onvalue**, and clears it otherwise. The widget sets the **alternate** state whenever the linked **-variable** is unset. (The **alternate** state may be used to indicate a "tri-state" or "indeterminate" selection.)

Standard styles

Ttk::checkbutton widgets support the **Toolbutton** style in all standard themes, which is useful for creating widgets for toolbars.

Styling options

The class name for a **ttk::checkbutton** is **TCheckbutton**.

Dynamic states: **active**, **alternate**, **disabled**, **pressed**, **selected**, **readonly**.

TCheckbutton styling options configurable with ttk::style are:

-background *color*

-compound *compound*

-foreground *color*

-indicatorbackground *color*

-indicatorcolor *color*

-indicatormargin *padding*

-indicatorrelief *relief*

-padding *padding*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2004 Joe English

ttk_combobox

Keywords: ttk_combobox, choice, entry, list box, text box, widget

Name

ttk::combobox - text field with popdown selection list

Synopsis

ttk::combobox *pathName* ?*options*?

Description

A **ttk::combobox** combines a text field with a pop-down list of values; the user may select the value of the text field from among the values in the list.

Standard options

-class, **-cursor**, **-takefocus**, **-style**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-exportselection (exportSelection/ExportSelection)
Boolean value. If set, the widget selection is linked to the X selection.

-justify (justify/Justify)
Specifies how the text is aligned within the widget. Must be one of **left**, **center**, or **right**.

-height (height/Height)
Specifies the height of the pop-down listbox, in rows.

-postcommand (postCommand/PostCommand)
A Tcl script to evaluate immediately before displaying the listbox. The **-postcommand** script may specify the **-values** to display.

-state (state/State)
One of **normal**, **readonly**, or **disabled**. In the **readonly** state, the value may not be edited directly, and the user can only select one of the **-values** from the dropdown list. In the **normal** state, the text field is directly editable. In the **disabled** state, no interaction is

possible.

-textvariable (textVariable/TextVariable)
Specifies the name of a global variable whose value is linked to the widget value. Whenever the variable changes value the widget value is updated, and vice versa.

-values (values/Values)
Specifies the list of values to display in the drop-down listbox.

-width (width/Width)
Specifies an integer value indicating the desired width of the entry window, in average-size characters of the widget's font.

Widget command

The following subcommands are possible for combobox widgets:

***pathName* current ?*newIndex*?**
If *newIndex* is supplied, sets the combobox value to the element at position *newIndex* in the list of **-values** (in addition to integers, the **end** index is supported and indicates the last element of the list). Otherwise, returns the index of the current value in the list of **-values** or **-1** if the current value does not appear in the list.

***pathName* get**
Returns the current value of the combobox.

pathName* set *value
Sets the value of the combobox to *value*.

The combobox widget also supports the following ttk::entry widget subcommands (see *ttk::entry(n)* for details):

bbox delete icursor
index insert selection
xview

The combobox widget also supports the following generic ttk::widget widget subcommands (see *ttk::widget(n)* for details):

cget configure identify
instate state

Virtual events

The combobox widget generates a **<>** virtual event when the user selects an element from the list of values. If the selection action unposts the listbox, this event is delivered after the listbox is unposted.

Styling options

The class name for a **ttk::combobox** is **TCombobox**. The **ttk::combobox** uses the entry and listbox widgets internally. The listbox frame has a class name of **ComboboxPopdownFrame**.

Dynamic states: **disabled**, **focus**, **pressed**, **readonly**.

TCombobox styling options configurable with ttk::style are:

-arrowcolor *color*

-arrowsize *amount*

-background *color*

-bordercolor *color*

-darkcolor *color*

-focusfill *color*

-foreground *color*

-fieldbackground ***color***
Can only be changed when using non-native and non-graphical themes.

-insertcolor *color*

-insertwidth *amount*

-lightcolor *color*

-padding *padding*

-postoffset *padding*

-selectbackground ***color***
Text entry select background.

-selectforeground ***color***
Text entry select foreground.

The **ttk::combobox** popdown listbox cannot be configured using ttk::style nor via the widget **configure** command. The listbox can be configured using the option database.

option add *TCombobox*Listbox.background *color*

option add *TCombobox*Listbox.font *font*

option add *TCombobox*Listbox.foreground *color*

option add *TCombobox*Listbox.selectBackground *color*

option add *TCombobox*Listbox.selectForeground *color*

To configure a specific listbox (subject to future change):

```
set popdown [ttk::combobox::PopdownWindow .mycombobox]
$popdown.f.l configure -font font
```

ComboboxPopdownFrame styling options configurable with ttk::style are:

-borderwidth *amount*

-relief *relief*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2004 Joe English

ttk_entry

Keywords: ttk_entry, entry, widget, text field

Name

ttk::entry - Editable text field widget

Synopsis

ttk::entry *pathName* ?*options*?

Description

An **ttk::entry** widget displays a one-line text string and allows that string to be edited by the user. The value of the string may be linked to a Tcl variable with the **-textvariable** option. Entry widgets support horizontal scrolling with the standard **-xscrollcommand** option and **xview** widget command.

Standard options

-class, **-cursor**, **-font**, **-foreground**, **-style**, **-takefocus**, **-xscrollcommand**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-exportselection (exportSelection/ExportSelection)
A boolean value specifying whether or not a selection in the widget should be linked to the X selection. If the selection is exported, then selecting in the widget deselects the current X selection, selecting outside the widget deselects any widget selection, and the widget will respond to selection retrieval requests when it has a selection.

-invalidcommand (invalidCommand/InvalidCommand)
A script template to evaluate whenever the **-validatecommand** returns 0. See Validation below for more information.

-justify (justify/Justify)
Specifies how the text is aligned within the entry widget. One of **left**, **center**, or **right**.

-show (show/Show)
If this option is specified, then the true contents of the entry are not displayed in the window. Instead, each character in the entry's value will be displayed as the first character in the value of this option, such as "*" or a bullet. This is useful, for example, if the entry is to be used to enter a password. If characters in the entry are selected and copied elsewhere, the information copied will be what is displayed, not the true contents of the entry.

-state (state/State)
Compatibility option; see *ttk::widget(n)* for details. Specifies one of three states for the entry, **normal**, **disabled**, or **readonly**. See Widget states, below.

-textvariable (textVariable/Variable)
Specifies the name of a global variable whose value is linked to the entry widget's contents. Whenever the variable changes value, the widget's contents are updated, and vice versa.

-validate (validate/Validate)
Specifies the mode in which validation should operate: **none**, **focus**, **focusin**, **focusout**, **key**, or **all**. Default is **none**, meaning that validation is disabled. See Validation below.

-validatecommand (validateCommand/ValidateCommand)
A script template to evaluate whenever validation is triggered. If set to the empty string (the default), validation is disabled. The script must return a boolean value. See Validation below.

-width (width/Width)
Specifies an integer value indicating the desired width of the entry window, in average-size characters of the widget's font.

Notes

A portion of the entry may be selected as described below. If an entry is exporting its selection (see the **-exportselection** option), then it will observe the standard X11 protocols for handling the selection; entry selections are available as type **STRING**. Entries also observe the standard Tk rules for dealing with the input focus. When an entry has the input focus it displays an *insert cursor* to indicate where new characters will be inserted.

Entries are capable of displaying strings that are too long to fit entirely within the widget's window. In this case, only a portion of the string will be displayed; commands described below may be used to change the view in the window. Entries use the standard **-xscrollcommand** mechanism for interacting with scrollbars (see the description of the **-xscrollcommand** option for details).

Indices

Many of the entry widget commands take one or more indices as arguments. An index specifies a particular character in the entry's string, in any of the following ways:

number
: Specifies the character as a numerical index, where 0 corresponds to the first character in the string.

@*number*
: In this form, *number* is treated as an x-coordinate in the entry's window; the character spanning that x-coordinate is used. For example, "**@0**" indicates the left-most character in the window.

end
: Indicates the character just after the last one in the entry's string. This is equivalent to specifying a numerical index equal to the length of the entry's string.

insert
: Indicates the character adjacent to and immediately following the insert cursor.

sel.first
: Indicates the first character in the selection. It is an error to use this form if the selection is not in the entry window.

sel.last
: Indicates the character just after the last one in the selection. It is an error to use this form if the selection is not in the entry window.

Abbreviations may be used for any of the forms above, e.g. "**e**" or "**sel.l**". In general, out-of-range indices are automatically rounded to the nearest legal value.

Widget command

The following subcommands are possible for entry widgets:

pathName* bbox *index
: Returns a list of four numbers describing the bounding box of the character given by *index*. The first two elements of the list give the x and y coordinates of the upper-left corner of the screen area covered by the character (in pixels relative to the widget) and the last two elements give the width and height of the character, in pixels. The bounding box may refer to a region outside the visible area of the window.

***pathName* delete *first* ?*last*?**
Delete one or more elements of the entry. *First* is the index of the first character to delete, and *last* is the index of the character just after the last one to delete. If *last* is not specified it defaults to *first*+1, i.e. a single character is deleted. This command returns the empty string.

***pathName* get**
Returns the entry's string.

pathName* icursor *index
Arrange for the insert cursor to be displayed just before the character given by *index*. Returns the empty string.

pathName* index *index
Returns the numerical index corresponding to *index*.

pathName* insert *index string
Insert *string* just before the character indicated by *index*. Returns the empty string.

pathName* selection *option arg
This command is used to adjust the selection within an entry. It has several forms, depending on *option*:

***pathName* selection clear**
Clear the selection if it is currently in this widget. If the selection is not in this widget then the command has no effect. Returns the empty string.

***pathName* selection present**
Returns 1 if there is are characters selected in the entry, 0 if nothing is selected.

pathName* selection range *start end
Sets the selection to include the characters starting with the one indexed by *start* and ending with the one just before *end*. If *end* refers to the same character as *start* or an earlier one, then the entry's selection is cleared.

***pathName* validate**
Force revalidation, independent of the conditions specified by the **-validate** option. Returns 0 if validation fails, 1 if it succeeds. Sets or clears the **invalid** state accordingly. See Validation below for more details.

The entry widget also supports the following generic ttk::widget widget subcommands (see *ttk::widget(n)* for details):

cget configure identify
instate state xview

Validation

The **-validate**, **-validatecommand**, and **-invalidcommand** options are used to enable entry widget validation.

Validation modes

There are two main validation modes: *prevalidation*, in which the **-validatecommand** is evaluated prior to each edit and the return value is used to determine whether to accept or reject the change; and *revalidation*, in which the **-validatecommand** is evaluated to determine whether the current value is valid.

The **-validate** option determines when validation occurs; it may be set to any of the following

values:

none
Default. This means validation will only occur when specifically requested by the **validate** widget command.

key
The entry will be prevalidated prior to each edit (specifically, whenever the **insert** or **delete** widget commands are called). If prevalidation fails, the edit is rejected.

focus
The entry is revalidated when the entry receives or loses focus.

focusin
The entry is revalidated when the entry receives focus.

focusout
The entry is revalidated when the entry loses focus.

all
Validation is performed for all above conditions.

The **-invalidcommand** is evaluated whenever the **-validatecommand** returns a false value.

The **-validatecommand** and **-invalidcommand** may modify the entry widget's value via the widget **insert** or **delete** commands, or by setting the linked **-textvariable**. If either does so during prevalidation, then the edit is rejected regardless of the value returned by the **-validatecommand**.

If **-validatecommand** is empty (the default), validation always succeeds.

Validation script substitutions

It is possible to perform percent substitutions on the **-validatecommand** and **-invalidcommand**, just as in a bind script. The following substitutions are recognized:

%d
Type of action: 1 for **insert** prevalidation, 0 for **delete** prevalidation, or -1 for revalidation.

%i
Index of character string to be inserted/deleted, if any, otherwise -1.

%P
In prevalidation, the new value of the entry if the edit is accepted. In revalidation, the current value of the entry.

%s
The current value of entry prior to editing.

%S
The text string being inserted/deleted, if any, {} otherwise.

%v
The current value of the **-validate** option.

%V
The validation condition that triggered the callback (**key**, **focusin**, **focusout**, or **forced**).

%W
The name of the entry widget.

Differences from Tk entry widget validation

The standard Tk entry widget automatically disables validation (by setting **-validate** to **none**) if the **-validatecommand** or **-invalidcommand** modifies the entry's value. The Tk themed entry widget only disables validation if one of the validation scripts raises an error, or if **-validatecommand** does not return a valid boolean value. (Thus, it is not necessary to re-enable validation after modifying the entry value in a validation script).

In addition, the standard entry widget invokes validation whenever the linked **-textvariable** is modified; the Tk themed entry widget does not.

Default bindings

The entry widget's default bindings enable the following behavior. In the descriptions below, "word" refers to a contiguous group of letters, digits, or "_" characters, or any single character other than these.

- Clicking mouse button 1 positions the insert cursor just before the character underneath the mouse cursor, sets the input focus to this widget, and clears any selection in the widget. Dragging with mouse button 1 down strokes out a selection between the insert cursor and the character under the mouse.
- Double-clicking with mouse button 1 selects the word under the mouse and positions the insert cursor at the end of the word. Dragging after a double click strokes out a selection consisting of whole words.
- Triple-clicking with mouse button 1 selects all of the text in the entry and positions the insert cursor at the end of the line.
- The ends of the selection can be adjusted by dragging with mouse button 1 while the Shift key is down. If the button is double-clicked before dragging then the selection will be adjusted in units of whole words.
- Clicking mouse button 1 with the Control key down will position the insert cursor in the entry without affecting the selection.
- If any normal printing characters are typed in an entry, they are inserted at the point of the insert cursor.
- The view in the entry can be adjusted by dragging with the middle mouse button (button 2, or button 3 in TkAqua). If the middle mouse button is clicked without moving the mouse, the selection is copied into the entry at the position of the mouse cursor.
- If the mouse is dragged out of the entry on the left or right sides while button 1 is pressed, the entry will automatically scroll to make more text visible (if there is more text off-screen on the side where the mouse left the window).
- The Left and Right keys move the insert cursor one character to the left or right; they also clear any selection in the entry. If Left or Right is typed with the Shift key down, then the insertion cursor moves and the selection is extended to include the new character. Control-Left and Control-Right move the insert cursor by words, and Control-Shift-Left and Control-Shift-Right move the insert cursor by words and also extend the selection. Control-b and Control-f behave the same as Left and Right, respectively.
- The Home key and Control-a move the insert cursor to the beginning of the entry and clear any selection in the entry. Shift-Home moves the insert cursor to the beginning of the entry and extends the selection to that point.
- The End key and Control-e move the insert cursor to the end of the entry and clear any selection in the entry. Shift-End moves the cursor to the end and extends the selection to that

point.
- Control-/ selects all the text in the entry.
- Control-\ clears any selection in the entry.
- The standard Tk <>, <>, <>, and <> virtual events operate on the selection in the expected manner.
- The Delete key deletes the selection, if there is one in the entry. If there is no selection, it deletes the character to the right of the insert cursor.
- The BackSpace key and Control-h delete the selection, if there is one in the entry. If there is no selection, it deletes the character to the left of the insert cursor.
- Control-d deletes the character to the right of the insert cursor.
- Control-k deletes all the characters to the right of the insertion cursor.

Widget states

In the **disabled** state, the entry cannot be edited and the text cannot be selected. In the **readonly** state, no insert cursor is displayed and the entry cannot be edited (specifically: the **insert** and **delete** commands have no effect). The **disabled** state is the same as **readonly**, and in addition text cannot be selected.

Note that changes to the linked **-textvariable** will still be reflected in the entry, even if it is disabled or readonly.

Typically, the text is "grayed-out" in the **disabled** state, and a different background is used in the **readonly** state.

The entry widget sets the **invalid** state if revalidation fails, and clears it whenever validation succeeds.

Styling options

The class name for a **ttk::entry** is **TEntry**.

Dynamic states: **disabled**, **focus**, **readonly**.

TEntry styling options configurable with ttk::style are:

-background ***color***
: For backwards compatibility, when using the aqua theme (for macOS), this option behaves as an alias for the **-fieldbackground** provided that no value is specified for **-fieldbackground**. Otherwise it is ignored.

-bordercolor *color*

-darkcolor *color*

-fieldbackground ***color***
: Some themes use a graphical background and their field background colors cannot be changed.

-foreground *color*

-insertcolor *color*

-insertwidth *amount*

-lightcolor *color*

-padding *padding*

-relief *relief*

-selectbackground *color*

-selectborderwidth *amount*

-selectforeground *color*

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 1998-2000 Scriptics Corporation.
Copyright (c) 2004 Joe English

ttk_frame

Keywords: ttk_frame, widget, frame, container

Name

ttk::frame - Simple container widget

Synopsis

ttk::frame *pathName* ?*options*?

Description

A **ttk::frame** widget is a container, used to group other widgets together.

Standard options

-class, **-cursor**, **-padding**, **-style**, **-takefocus**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-borderwidth (borderWidth/BorderWidth)
: The desired width of the widget border. Defaults to 0. May be ignored depending on the theme used.

-relief (relief/Relief)
: One of the standard Tk border styles: **flat**, **groove**, **raised**, **ridge**, **solid**, or **sunken**. Defaults to **flat**.

-width (width/Width)
: If specified, the widget's requested width in pixels.

-height (height/Height)
: If specified, the widget's requested height in pixels.

Widget command

Supports the standard widget commands **configure**, **cget**, **identify**, **instate**, and **state**; see *ttk::widget(n)*.

Notes

Note that if the pack, grid, or other geometry managers are used to manage the children of the frame, by the GM's requested size will normally take precedence over the frame widget's **-width** and **-height** options. pack propagate and grid propagate can be used to change this.

Styling options

The class name for a **ttk::frame** is **TFrame**.

TFrame styling options configurable with ttk::style are:

-background *color*

-relief *relief*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Bindings

When a new **ttk::frame** is created, it has no default event bindings; **ttk::frame**s are not intended to be interactive.

Copyright (c) 2005 Joe English

ttk_image

Keywords: ttk_image, style, theme, appearance, pixmap theme, image

Name

ttk_image - Define an element based on an image

Synopsis

ttk::style element create *name* **image** *imageSpec* ?*options*?

Description

The *image* element factory creates a new element in the current theme whose visual appearance is determined by Tk images. *imageSpec* is a list of one or more elements. The first element is the default image name. The rest of the list is a sequence of *statespec / value* pairs specifying other images to use when the element is in a particular state or combination of states.

Options

Valid *options* are:

-border *padding*
: *padding* is a list of up to four integers, specifying the left, top, right, and bottom borders, respectively. If fewer than four elements are specified, *bottom* defaults to *top*, *right* defaults to *left*, and *top* defaults to *left*. In other words, a list of three numbers specify the left, vertical, and right border; a list of two numbers specify the horizontal and the vertical border; a single number specifies the same border all the way around the element. See Image stretching, below.

-height *height*
: Specifies a minimum height for the element. If less than zero, the base image's height is used as a default.

-padding *padding*
: Specifies the element's interior padding. The padding is a list of up to four length specifications *left top right bottom*. If fewer than four elements are specified, *bottom* defaults

to *top*, *right* defaults to *left*, and *top* defaults to *left*. In other words, a list of three numbers specify the left, vertical, and right padding; a list of two numbers specify the horizontal and the vertical padding; a single number specifies the same padding all the way around the widget. Defaults to **-border** if not specified.

-sticky *spec*
Specifies how the image is placed within the final parcel. *spec* contains zero or more characters "n", "s", "w", or "e".

-width *width*
Specifies a minimum width for the element. If less than zero, the base image's width is used as a default.

Image stretching

If the element's allocated parcel is larger than the image, the image will be placed in the parcel based on the **-sticky** option. If the image needs to stretch horizontally (i.e., **-sticky ew**) or vertically (**-sticky ns**), subregions of the image are replicated to fill the parcel based on the **-border** option. The **-border** divides the image into 9 regions: four fixed corners, top and left edges (which may be tiled horizontally), left and right edges (which may be tiled vertically), and the central area (which may be tiled in both directions).

An image element that is not meant to claim any space (for example when used as a background image) should use **-width 0** and **-height 0**.

Example

```
set img1 [image create photo -file button.png]
set img2 [image create photo -file button-pressed.png]
set img3 [image create photo -file button-active.png]
ttk::style element create Button.button image \
    [list $img1  pressed $img2  active $img3] \
    -border {2 4} -sticky we
```

Copyright (c) 2004 Joe English

ttk_intro

Keywords: ttk_intro

Name

ttk::intro - Introduction to the Tk theme engine

Overview

The Tk themed widget set is based on a revised and enhanced version of TIP #48 (https://tip.tcl-lang.org/48) specified style engine. The main concepts are described below. The basic idea is to separate, to the extent possible, the code implementing a widget's behavior from the code implementing its appearance. Widget class bindings are primarily responsible for maintaining the widget state and invoking callbacks; all aspects of the widget's appearance are controlled by the style of the widget (i.e. the style of the elements of the widget).

Themes

A *theme* is a collection of elements and styles that determine the look and feel of the widget set. Themes can be used to:

- isolate platform differences (X11 vs. classic Windows vs. XP vs. Aqua ...)
- adapt to display limitations (low-color, grayscale, monochrome, tiny screens)
- accessibility (high contrast, large type)
- application suite branding
- blend in with the rest of the desktop (Gnome, KDE, Java)
- and, of course: eye candy.

Elements

An *element* displays an individual part of a widget. For example, a vertical scrollbar widget contains **uparrow**, **downarrow**, **trough** and **slider** elements.

Element names use a recursive dotted notation. For example, **uparrow** identifies a generic arrow element, and **Scrollbar.uparrow** and **Combobox.uparrow** identify widget-specific elements. When looking for an element, the style engine looks for the specific name first, and if an element of that name is not found it looks for generic elements by stripping off successive leading components of the element name.

Like widgets, elements have *options* which specify what to display and how to display it. For example, the **text** element (which displays a text string) has **-text**, **-font**, **-foreground**, **-background**, **-underline**, and **-width** options. The value of an element option is taken from (in precedence order):

- an option of the same name and type in the widget containing the element;
- a dynamic setting specified by **style map** and the current state;
- the default setting specified by **style configure**; or
- the element's built-in default value for the option.

Layouts

A *layout* specifies which elements make up a widget and how they are arranged. The layout engine uses a simplified version of the pack algorithm: starting with an initial cavity equal to the size of the widget, elements are allocated a parcel within the cavity along the side specified by the **-side** option, and placed within the parcel according to the **-sticky** option. For example, the layout for a horizontal scrollbar is:

```
ttk::style layout Horizontal.TScrollbar {
    Scrollbar.trough -children {
    Scrollbar.leftarrow -side left -sticky w
    Scrollbar.rightarrow -side right -sticky e
    Scrollbar.thumb -sticky ew
    }
}
```

By default, the layout for a widget is the same as its class name. Some widgets may override this (for example, the ttk::scrollbar widget chooses different layouts based on the **-orient** option).

States

In standard Tk, many widgets have a **-state** option which (in most cases) is either **normal** or **disabled**. Some widgets support additional states, such as the entry widget which has a **readonly** state and the various flavors of buttons which have **active** state.

The themed Tk widgets generalizes this idea: every widget has a bitmap of independent state flags. Widget state flags include **active**, **disabled**, **pressed**, **focus**, etc., (see *ttk::widget(n)* for the full list of state flags).

Instead of a **-state** option, every widget now has a **state** widget command which is used to set or query the state. A *state specification* is a list of symbolic state names indicating which bits are set, each optionally prefixed with an exclamation point indicating that the bit is cleared instead.

For example, the class bindings for the ttk::button widget are:

```
bind TButton <Enter>        { %W state active }
bind TButton <Leave>        { %W state !active }
bind TButton <ButtonPress-1>    { %W state pressed }
bind TButton <Button1-Leave>    { %W state !pressed }
bind TButton <Button1-Enter>    { %W state pressed }
bind TButton <ButtonRelease-1>  \
    { %W instate {pressed} { %W state !pressed ; %W invoke } }
```

This specifies that the widget becomes **active** when the pointer enters the widget, and inactive when it leaves. Similarly it becomes **pressed** when the mouse button is pressed, and**!pressed** on the ButtonRelease event. In addition, the button unpresses if pointer is dragged outside the widget while Button-1 is held down, and represses if it's dragged back in. Finally, when the mouse button is released, the widget's **-command** is invoked, but only if the button is currently in the **pressed** state. (The actual bindings are a little more complicated than the above, but not by much).

Styles

Each widget is associated with a *style*, which specifies values for element options. Style names use a recursive dotted notation like layouts and elements; by default, widgets use the class name to look up a style in the current theme. For example:

```
ttk::style configure TButton \
    -background #d9d9d9 \
    -foreground black \
    -relief raised \
    ;
```

Many elements are displayed differently depending on the widget state. For example, buttons have a different background when they are active, a different foreground when disabled, and a different relief when pressed. The **style map** command specifies dynamic option settings for a particular style:

```
ttk::style map TButton \
    -background [list disabled #d9d9d9  active #ececec] \
    -foreground [list disabled #a3a3a3] \
    -relief [list {pressed !disabled} sunken] \
    ;
```

Copyright (c) 2004 Joe English

ttk_label

Keywords: ttk_label

Name

ttk::label - Display a text string and/or image

Synopsis

ttk::label *pathName* ?*options*?

Description

A **ttk::label** widget displays a textual label and/or image. The label may be linked to a Tcl variable to automatically change the displayed text.

Standard options

-class, **-compound**, **-cursor**, **-image**, **-padding**, **-state**, **-style**, **-takefocus**, **-text**, **-textvariable**, **-underline**, **-width**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-anchor (anchor/Anchor)
: Specifies how the information in the widget is positioned relative to the inner margins. Legal values are **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, and **center**. See also **-justify**.

-background (frameColor/FrameColor)
: The widget's background color. If unspecified, the theme default is used.

-font (font/Font)
: Font to use for label text.

-foreground (textColor/TextColor)
: The widget's foreground color. If unspecified, the theme default is used.

-justify (justify/Justify)
: If there are multiple lines of text, specifies how the lines are laid out relative to one another. One of **left**, **center**, or **right**. See also **-anchor**.

-relief (relief/Relief)
: Specifies the 3-D effect desired for the widget border. Valid values are **flat**, **groove**, **raised**, **ridge**, **solid**, and **sunken**.

-wraplength (wrapLength/WrapLength)
: Specifies the maximum line length (in pixels). If this option is less than or equal to zero, then automatic wrapping is not performed; otherwise the text is split into lines such that no line is longer than the specified value.

Widget command

Supports the standard widget commands **configure**, **cget**, **identify**, **instate**, and **state**; see *ttk::widget(n)*.

Styling options

The class name for a **ttk::label** is **TLabel**.

Dynamic states: **disabled**, **readonly**.

TLabel styling options configurable with ttk::style are:

-background *color*

-compound *compound*

-foreground *color*

-font *font*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2004 Joe English

ttk_labelframe

Keywords: ttk_labelframe, widget, frame, container, label, groupbox

Name

ttk::labelframe - Container widget with optional label

Synopsis

ttk::labelframe *pathName* ?*options*?

Description

A **ttk::labelframe** widget is a container used to group other widgets together. It has an optional label, which may be a plain text string or another widget.

Standard options

-class, **-cursor**, **-padding**, **-style**, **-takefocus**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-height (height/Height)
: If specified, the widget's requested height in pixels. (See *ttk::frame(n)* for further notes on **-width** and **-height**).

-labelanchor (labelAnchor/LabelAnchor)
: Specifies where to place the label. Allowed values are (clockwise from the top upper left corner): **nw**, **n**, **ne**, **en**, **e**, **es**, **se**, **s**,**sw**, **ws**, **w** and **wn**. The default value is theme-dependent.

-labelwidget (labelWidget/LabelWidget)
: The name of a widget to use for the label. If set, overrides the **-text** option. The **-labelwidget** must be a child of the labelframe widget or one of the labelframe's ancestors, and must belong to the same top-level widget as the labelframe.

-text (text/Text)
: Specifies the text of the label.

-underline (underline/Underline)
: If set, specifies the integer index (0-based) of a character to underline in the text string. The

underlined character is used for mnemonic activation. Mnemonic activation for a **ttk::labelframe** sets the keyboard focus to the first child of the **ttk::labelframe** widget.

-width (width/Width)
If specified, the widget's requested width in pixels.

Widget command
Supports the standard widget commands **configure**, **cget**, **identify**, **instate**, and **state**; see *ttk::widget(n)*.

Styling options
The class name for a **ttk::labelframe** is **TLabelframe**. The text label has a class of **TLabelframe.Label**.

Dynamic states: **disabled**, **readonly**.

TLabelframe styling options configurable with ttk::style are:

-background *color*

-bordercolor *color*

-borderwidth *amount*

-darkcolor *color*

-labelmargins *amount*

-labeloutside *boolean*

-lightcolor *color*

-relief *relief*

TLabelframe.Label styling options configurable with ttk::style are:

-background *color*

-font *font*

-foreground *color*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2005 Joe English

ttk_menubutton
Keywords: ttk_menubutton, widget, button, menu

Name
ttk::menubutton - Widget that pops down a menu when pressed

Synopsis
ttk::menubutton *pathName* ?*options*?

Description
A **ttk::menubutton** widget displays a textual label and/or image, and displays a menu when pressed.

Standard options

-class, **-compound**, **-cursor**, **-image**, **-state**, **-style**, **-takefocus**, **-text**, **-textvariable**, **-underline**, **-width**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-direction (direction/Direction)
Specifies where the menu is to be popped up relative to the menubutton. One of: **above**, **below**, **left**, **right**, or **flush**. The default is **below**. **flush** pops the menu up directly over the menubutton.

-menu (menu/**Menu)**
Specifies the path name of the menu associated with the menubutton. To be on the safe side, the menu ought to be a direct child of the menubutton.

Widget command

Menubutton widgets support the standard **cget**, **configure**, **identify**, **instate**, and **state** methods. No other widget methods are used.

Standard styles

Ttk::menubutton widgets support the **Toolbutton** style in all standard themes, which is useful for creating widgets for toolbars.

Styling options

The class name for a **ttk::menubutton** is **TMenubutton**.

Dynamic states: **active**, **disabled**, **readonly**.

TMenubutton styling options configurable with ttk::style are:

-arrowsize *amount*

-background *color*

-compound *compound*

-foreground *color*

-font *font*

-padding *padding*

-relief *relief*

-width *amount*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2004 Joe English

ttk_notebook

Keywords: ttk_notebook, pane, tab

Name

ttk::notebook - Multi-paned container widget

Synopsis

ttk::notebook *pathname* ?*options...*?
pathname **add** *window* ?*options...*?
pathname **insert** *index window* ?*options...*?

Description

A **ttk::notebook** widget manages a collection of windows and displays a single one at a time. Each content window is associated with a *tab*, which the user may select to change the currently-displayed window.

Standard options

-class, **-cursor**, **-takefocus**, **-style**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-height (height/Height)
If present and greater than zero, specifies the desired height of the pane area (not including internal padding or tabs). Otherwise, the maximum height of all panes is used.

-padding (padding/Padding)
Specifies the amount of extra space to add around the outside of the notebook. The padding is a list of up to four length specifications *left top right bottom*. If fewer than four elements are specified, *bottom* defaults to *top*, *right* defaults to *left*, and *top* defaults to *left*. In other words, a list of three numbers specify the left, vertical, and right padding; a list of two numbers specify the horizontal and the vertical padding; a single number specifies the same padding all the way around the widget.

-width (width/Width)
If present and greater than zero, specifies the desired width of the pane area (not including internal padding). Otherwise, the maximum width of all panes is used.

Tab options

The following options may be specified for individual notebook panes:

-state (state/State)
Either **normal**, **disabled** or **hidden**. If **disabled**, then the tab is not selectable. If **hidden**, then the tab is not shown.

-sticky (sticky/Sticky)
Specifies how the content window is positioned within the pane area. Value is a string containing zero or more of the characters **n, s, e,** or **w**. Each letter refers to a side (north, south, east, or west) that the content window will "stick" to, as per the grid geometry manager.

-padding (padding/Padding)
Specifies the amount of extra space to add between the notebook and this pane. Syntax is the same as for the widget **-padding** option.

-text (text/Text)
Specifies a string to be displayed in the tab.

-image (image/Image)

Specifies an image to display in the tab. See *ttk_widget(n)* for details.

-compound (compound/Compound)
Specifies how to display the image relative to the text, in the case both **-text** and **-image** are present. See *label(n)* for legal values.

-underline (underline/Underline)
Specifies the integer index (0-based) of a character to underline in the text string. The underlined character is used for mnemonic activation if **ttk::notebook::enableTraversal** is called.

Tab identifiers

The *tabid* argument to the following commands may take any of the following forms:

- An integer between zero and the number of tabs;
- The name of a content window;
- A positional specification of the form "@*x*,*y*", which identifies the tab
- The literal string "**current**", which identifies the currently-selected tab; or:
- The literal string "**end**", which returns the number of tabs (only valid for "*pathname* **index**").

Widget command

***pathname* add *window* ?*options...*?**
Adds a new tab to the notebook. See Tab options for the list of available *options*. If *window* is currently managed by the notebook but hidden, it is restored to its previous position.

***pathname* configure ?*options*?**
See *ttk::widget(n)*.

pathname* cget *option
See *ttk::widget(n)*.

pathname* forget *tabid
Removes the tab specified by *tabid*, unmaps and unmanages the associated window.

pathname* hide *tabid
Hides the tab specified by *tabid*. The tab will not be displayed, but the associated window remains managed by the notebook and its configuration remembered. Hidden tabs may be restored with the **add** command.

pathname* identify *component x y
Returns the name of the element under the point given by *x* and *y*, or the empty string if no component is present at that location. The following subcommands are supported:

pathname* identify element *x y
Returns the name of the element at the specified location.

pathname* identify tab *x y
Returns the index of the tab at the specified location.

pathname* index *tabid
Returns the numeric index of the tab specified by *tabid*, or the total number of tabs if *tabid* is the string "**end**".

pathname* insert *pos subwindow options...
Inserts a pane at the specified position. *pos* is either the string **end**, an integer index, or the name of a managed subwindow. If *subwindow* is already managed by the notebook, moves it

to the specified position. See Tab options for the list of available options.

***pathname* instate *statespec* ?*script...*?**
See *ttk::widget(n)*.

***pathname* select ?*tabid*?**
Selects the specified tab. The associated content window will be displayed, and the previously-selected window (if different) is unmapped. If *tabid* is omitted, returns the widget name of the currently selected pane.

***pathname* state ?*statespec*?**
See *ttk::widget(n)*.

pathname* tab *tabid* ?*-option* ?*value ...
Query or modify the options of the specific tab. If no *-option* is specified, returns a dictionary of the tab option values. If one *-option* is specified, returns the value of that *option*. Otherwise, sets the *-option*s to the corresponding *value*s. See Tab options for the available options.

***pathname* tabs**
Returns the list of windows managed by the notebook, in the index order of their associated tabs.

Keyboard traversal

To enable keyboard traversal for a toplevel window containing a notebook widget *$nb*, call:

```
ttk::notebook::enableTraversal $nb
```

This will extend the bindings for the toplevel window containing the notebook as follows:

- **Control-Tab** selects the tab following the currently selected one.
- **Control-Shift-Tab** selects the tab preceding the currently selected one.
- **Alt-***K*, where *K* is the mnemonic (underlined) character of any tab, will select that tab.

Multiple notebooks in a single toplevel may be enabled for traversal, including nested notebooks. However, notebook traversal only works properly if all panes are direct children of the notebook.

Virtual events

The notebook widget generates a **<>** virtual event after a new tab is selected.

Example

```
pack [ttk::notebook .nb]
.nb add [frame .nb.f1] -text "First tab"
.nb add [frame .nb.f2] -text "Second tab"
.nb select .nb.f2
ttk::notebook::enableTraversal .nb
```

Styling options

The class name for a **ttk::notebook** is **TNotebook**. The tab has a class name of **TNotebook.Tab**

Dynamic states: **active**, **disabled**, **selected**.

TNotebook styling options configurable with ttk::style are:

-background *color*

-bordercolor *color*

-darkcolor *color*

-foreground *color*

-lightcolor *color*

-padding *padding*

-tabmargins *padding*

-tabposition ***position***
Specifies the position of the tab row or column as a string of length 1 or 2. The first character indicates the side as **n**, **s**, **w**, or **e**, while the second character (if present) is the sticky bit (specified as **w**, **e**, **n**, or **s**) within the tab position. The default position is **n** for the **aqua** theme and **nw** for all the other built-in themes.

-tabplacement ***placement***
Specifies the placement (anchor) of the tabs within the tab row or column as a string of length 1 or 2. The first character indicates the side as **w**, **e**, **n**, or **s**, while the second character (if present) is the sticky bit (specified as **s**, **n**, **e**, or **w**) within the tab placement. For example, the tab placement **ws** means: attach the west (left) side of each tab to the previous one and align the south (bottom) edges with each other. Similarly, the tab placement **se** specifies to attach the south (bottom) side of each tab to the previous one and align the east (right) edges with each other. The default placement depends on the first character of the tab position: **ws** for **n***, **wn** for **s***, **ne** for **w***, and **nw** for **e***.

TNotebook.Tab styling options configurable with ttk::style are:

-background *color*

-bordercolor *color*

-compound *compound*

-expand ***padding***
Defines how much the tab grows in size. Usually used with the **selected** dynamic state. **-tabmargins** should be set appropriately so that there is room for the tab growth. For example, the Ttk library file **vistaTheme.tcl** contains the lines

```
ttk::style configure TNotebook -tabmargins {2 2 2 0}
ttk::style map TNotebook.Tab -expand {selected {2 2 2 2}}
```

which are valid for the default value **nw** of the **-tabposition** style option. For a **ttk::notebook** style **nbStyle** defined by

```
set nbStyle SW.TNotebook
ttk::style configure $nbStyle -tabposition sw
```

you will have to adapt the above settings as follows:

```
ttk::style configure $nbStyle -tabmargins {2 0 2 2}
ttk::style map $nbStyle.Tab -expand {selected {2 2 2 2}}
```

The easiest way to do this is to invoke the library procedure **ttk::configureNotebookStyle** with **$nbStyle** as argument, after setting the style's **-tabposition** option.

-font *font*

-foreground *color*

-padding ***padding***
Some themes use a different *padding* for the selected tab. For example, the Ttk library file **clamTheme.tcl** contains the lines

```
ttk::style configure TNotebook.Tab -padding {6 2 6 2}
ttk::style map TNotebook.Tab -padding {selected {6 4 6 2}}
```

which are valid for the default value **nw** of the **-tabposition** style option. For a **ttk::notebook** style having a different tab position you will have to adapt the above settings accordingly. Again, the easiest way to do this is to invoke the library procedure **ttk::configureNotebookStyle** with the style name as argument, after setting the style's **-tabposition** option.

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2004 Joe English

ttk_panedwindow

Keywords: ttk_panedwindow

Name

ttk::panedwindow - Multi-pane container window

Synopsis

ttk::panedwindow *pathname* ?*options*?
pathname **add** *window* ?*options...*?
pathname **insert** *index window* ?*options...*?

Description

A **ttk::panedwindow** widget displays a number of subwindows, stacked either vertically or horizontally. The user may adjust the relative sizes of the subwindows by dragging the sash between panes.

Standard options

-class, **-cursor**, **-takefocus**, **-style**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-orient (orient/Orient)

Specifies the orientation of the window. If **vertical**, subpanes are stacked top-to-bottom; if **horizontal**, subpanes are stacked left-to-right.

-width (width/Width)
If present and greater than zero, specifies the desired width of the widget in pixels. Otherwise, the requested width is determined by the width of the managed windows.

-height (height/Height)
If present and greater than zero, specifies the desired height of the widget in pixels. Otherwise, the requested height is determined by the height of the managed windows.

Pane options

The following options may be specified for each pane:

-weight (weight/Weight)
An integer specifying the relative stretchability of the pane. When the paned window is resized, the extra space is added or subtracted to each pane proportionally to its **-weight**.

Widget command

Supports the standard **configure**, **cget**, **state**, and **instate** commands; see *ttk::widget(n)* for details. Additional commands:

pathname* add *subwindow options...
Adds a new pane to the window. See Pane options for the list of available options.

pathname* forget *pane
Removes the specified subpane from the widget. *pane* is either an integer index or the name of a managed subwindow.

pathname* identify *component x y
Returns the name of the element under the point given by *x* and *y*, or the empty string if no component is present at that location. If *component* is omitted, it defaults to **sash**. The following subcommands are supported:

pathname* identify element *x y
Returns the name of the element at the specified location.

pathname* identify sash *x y
Returns the index of the sash at the specified location.

pathname* insert *pos subwindow options...
Inserts a pane at the specified position. *pos* is either the string **end**, an integer index, or the name of a managed subwindow. If *subwindow* is already managed by the paned window, moves it to the specified position. See Pane options for the list of available options.

pathname* pane *pane -option ?value ?-option value...
Query or modify the options of the specified *pane*, where *pane* is either an integer index or the name of a managed subwindow. If no *-option* is specified, returns a dictionary of the pane option values. If one *-option* is specified, returns the value of that *option*. Otherwise, sets the *-option*s to the corresponding *value*s.

***pathname* panes**
Returns the list of all windows managed by the widget, in the index order of their associated panes.

pathname* sashpos *index ?newpos?

If *newpos* is specified, sets the position of sash number *index*. May adjust the positions of adjacent sashes to ensure that positions are monotonically increasing. Sash positions are further constrained to be between 0 and the total size of the widget. Returns the new position of sash number *index*.

The panedwindow widget also supports the following generic ttk::widget widget subcommands (see *ttk::widget(n)* for details):

cget configure
instate state

Virtual events

The panedwindow widget generates an **<>** virtual event on LeaveNotify/NotifyInferior events.

Styling options

The class name for a **ttk::panedwindow** is **TPanedwindow**. The sash has a class name of **Sash**.

TPanedwindow styling options configurable with ttk::style are:

-background *color*

Sash styling options configurable with ttk::style are:

-background *color*

-bordercolor *color*

-gripcount *count*

-handlepad *amount*

-handlesize *amount*

-lightcolor *color*

-sashpad *amount*

-sashrelief *relief*

-sashthickness *amount*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2005 Joe English

ttk_progressbar

Keywords: ttk_progressbar

Name

ttk::progressbar - Provide progress feedback

Synopsis

ttk::progressbar *pathName* ?*options*?

Description

A **ttk::progressbar** widget shows the status of a long-running operation. They can operate in

two modes: *determinate* mode shows the amount completed relative to the total amount of work to be done, and *indeterminate* mode provides an animated display to let the user know that something is happening.

Standard options

-class, **-cursor**, **-takefocus**, **-style**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-length (length/Length)
Specifies the length of the long axis of the progress bar (width if horizontal, height if vertical).

-maximum (maximum/Maximum)
A floating point number specifying the maximum **-value**. Defaults to 100.

-mode (mode/Mode)
One of **determinate** or **indeterminate**.

-orient (orient/Orient)
One of **horizontal** or **vertical**. Specifies the orientation of the progress bar.

-phase (phase/Phase)
Read-only option. The widget periodically increments the value of this option whenever the **-value** is greater than 0 and, in *determinate* mode, less than **-maximum**. This option may be used by the current theme to provide additional animation effects.

-value (value/Value)
The current value of the progress bar. In *determinate* mode, this represents the amount of work completed. In *indeterminate* mode, it is interpreted modulo **-maximum**; that is, the progress bar completes one "cycle" when the **-value** increases by **-maximum**. If **-variable** is set to an existing variable, specifying **-value** has no effect (the variable value takes precedence).

-variable (variable/Variable)
The name of a global Tcl variable which is linked to the **-value**. If specified to an existing variable, the **-value** of the progress bar is automatically set to the value of the variable whenever the latter is modified.

Widget command

pathName* cget *option
Returns the current value of the specified *option*; see *ttk::widget(n)*.

***pathName* configure ?*option*? ?*value option value ...*?**
Modify or query widget options; see *ttk::widget(n)*.

pathName* identify *x y
Returns the name of the element at position *x*, *y*. See *ttk::widget(n)*.

***pathName* instate *statespec* ?*script*?**
Test the widget state; see *ttk::widget(n)*.

***pathName* start ?*interval*?**
Begin autoincrement mode: schedules a recurring timer event that calls **step** every *interval* milliseconds. If omitted, *interval* defaults to 50 milliseconds (20 steps/second).

***pathName* state ?*stateSpec*?**

Modify or query the widget state; see *ttk::widget(n)*.

***pathName* step ?*amount*?**
Increments the **-value** by *amount*. *amount* defaults to 1.0 if omitted.

***pathName* stop**
Stop autoincrement mode: cancels any recurring timer event initiated by *pathName* **start**.

Styling options

The class name for a **ttk::progressbar** is **TProgressbar**.

TProgressbar styling options configurable with ttk::style are:

-background *color*

-bordercolor *color*

-darkcolor *color*

-lightcolor *color*

-maxphase
For the aqua theme.

-period
For the aqua theme.

-troughcolor *color*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2005 Joe English

ttk_radiobutton

Keywords: ttk_radiobutton, widget, button, option

Name

ttk::radiobutton - Mutually exclusive option widget

Synopsis

ttk::radiobutton *pathName* ?*options*?

Description

ttk::radiobutton widgets are used in groups to show or change a set of mutually-exclusive options. Radiobuttons are linked to a Tcl variable, and have an associated value; when a radiobutton is clicked, it sets the variable to its associated value.

Standard options

-class, **-compound**, **-cursor**, **-image**, **-state**, **-style**, **-takefocus**, **-text**, **-textvariable**, **-underline**, **-width**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-command (command/Command)

A Tcl script to evaluate whenever the widget is invoked.

-value (Value/Value)
The value to store in the associated **-variable** when the widget is selected.

-variable (variable/Variable)
The name of a global variable whose value is linked to the widget. Default value is **::selectedButton**.

Widget command

In addition to the standard **cget**, **configure**, **identify**, **instate**, and **state** commands, radiobuttons support the following additional widget commands:

***pathname* invoke**
Sets the **-variable** to the **-value**, selects the widget, and evaluates the associated **-command**. Returns the result of the **-command**, or the empty string if no **-command** is specified.

Widget states

The widget does not respond to user input if the **disabled** state is set. The widget sets the **selected** state whenever the linked **-variable** is set to the widget's **-value**, and clears it otherwise. The widget sets the **alternate** state whenever the linked **-variable** is unset. (The **alternate** state may be used to indicate a "tri-state" or "indeterminate" selection.)

Standard styles

Ttk::radiobutton widgets support the **Toolbutton** style in all standard themes, which is useful for creating widgets for toolbars.

Styling options

The class name for a **ttk::radiobutton** is **TRadiobutton**.

Dynamic states: **active**, **alternate**, **disabled**, **pressed**, **readonly**, **selected**.

TRadiobutton styling options configurable with ttk::style are:

-background *color*

-compound *compound*

-foreground *color*

-indicatorbackground *color*

-indicatorcolor *color*

-indicatormargin *padding*

-indicatorrelief *relief*

-padding *padding*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2004 Joe English

ttk_scale

Keywords: ttk_scale, scale, slider, trough, widget

Name
ttk::scale - Create and manipulate a scale widget

Synopsis
ttk::scale *pathName* ?*options...*?

Description
A **ttk::scale** widget is typically used to control the numeric value of a linked variable that varies uniformly over some range. A scale displays a *slider* that can be moved along over a *trough*, with the relative position of the slider over the trough indicating the value of the variable.

Standard options
-class, **-cursor**, **-style**, **-takefocus**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options
-command (command/Command)
Specifies the prefix of a Tcl command to invoke whenever the scale's value is changed via a widget command. The actual command consists of this option followed by a space and a real number indicating the new value of the scale.

-from (from/From)
A real value corresponding to the left or top end of the scale.

-length (length/Length)
Specifies the desired long dimension of the scale in screen units (i.e. any of the forms acceptable to Tk_GetPixels). For vertical scales this is the scale's height; for horizontal scales it is the scale's width.

-orient (orient/Orient)
Specifies which orientation whether the widget should be laid out horizontally or vertically. Must be either **horizontal** or **vertical** or an abbreviation of one of these.

-to (to/To)
Specifies a real value corresponding to the right or bottom end of the scale. This value may be either less than or greater than the **-from** option.

-value (value/Value)
Specifies the current floating-point value of the variable. If **-variable** is set to an existing variable, specifying **-value** has no effect (the variable value takes precedence).

-variable (variable/Variable)
Specifies the name of a global variable to link to the scale. Whenever the value of the variable changes, the scale will update to reflect this value. Whenever the scale is manipulated interactively, the variable will be modified to reflect the scale's new value.

Widget command
pathName* cget *option
Returns the current value of the specified *option*; see *ttk::widget(n)*.

***pathName* configure ?*option*? ?*value option value ...*?**

Modify or query widget options; see *ttk::widget(n)*.

***pathName* get ?*x y*?**
Get the current value of the **-value** option, or the value corresponding to the coordinates *x,y* if they are specified. *X* and *y* are pixel coordinates relative to the scale widget origin.

pathName* identify *x y
Returns the name of the element at position *x*, *y*. See *ttk::widget(n)*.

***pathName* instate *statespec* ?*script*?**
Test the widget state; see *ttk::widget(n)*.

pathName* set *value
Set the value of the widget (i.e. the **-value** option) to *value*. The value will be clipped to the range given by the **-from** and **-to** options. Note that setting the linked variable (i.e. the variable named in the **-variable** option) does not cause such clipping.

***pathName* state ?*stateSpec*?**
Modify or query the widget state; see *ttk::widget(n)*.

Internal commands

***pathName* coords ?*value*?**
Get the coordinates corresponding to *value*, or the coordinates corresponding to the current value of the **-value** option if *value* is omitted.

Styling options

The class name for a **ttk::scale** is **TScale**.

Dynamic states: **active**.

TScale styling options configurable with ttk::style are:

-background *color*

-borderwidth *amount*

-darkcolor *color*

-groovewidth *amount*

-lightcolor *color*

-sliderwidth *amount*

-troughcolor *color*

-troughrelief *relief*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2008 Donal Fellows

ttk_scrollbar

Keywords: ttk_scrollbar, scrollbar, widget

Name

ttk::scrollbar - Control the viewport of a scrollable widget

Synopsis

ttk::scrollbar *pathName* ?*options...*?

Description

ttk::scrollbar widgets are typically linked to an associated window that displays a document of some sort, such as a file being edited or a drawing. A scrollbar displays a *thumb* in the middle portion of the scrollbar, whose position and size provides information about the portion of the document visible in the associated window. The thumb may be dragged by the user to control the visible region. Depending on the theme, two or more arrow buttons may also be present; these are used to scroll the visible region in discrete units.

Standard options

-class, **-cursor**, **-style**, **-takefocus**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-command (command/Command)

A Tcl script prefix to evaluate to change the view in the widget associated with the scrollbar. Additional arguments are appended to the value of this option, as described in Scrolling commands below, whenever the user requests a view change by manipulating the scrollbar.

This option typically consists of a two-element list, containing the name of a scrollable widget followed by either **xview** (for horizontal scrollbars) or **yview** (for vertical scrollbars).

-orient (orient/Orient)

One of **horizontal** or **vertical**. Specifies the orientation of the scrollbar.

Widget command

pathName* cget *option

Returns the current value of the specified *option*; see *ttk::widget(n)*.

***pathName* configure ?*option*? ?*value option value ...*?**

Modify or query widget options; see *ttk::widget(n)*.

***pathName* get**

Returns the scrollbar settings in the form of a list whose elements are the arguments to the most recent **set** widget command.

pathName* identify *x y

Returns the name of the element at position *x*, *y*. See *ttk::widget(n)*.

***pathName* instate *statespec* ?*script*?**

Test the widget state; see *ttk::widget(n)*.

pathName* set *first last

This command is normally invoked by the scrollbar's associated widget from an **-xscrollcommand** or **-yscrollcommand** callback. Specifies the visible range to be displayed. *first* and *last* are real fractions between 0 and 1.

***pathName* state ?*stateSpec*?**

Modify or query the widget state; see *ttk::widget(n)*.

Internal commands

The following widget commands are used internally by the **TScrollbar** widget class bindings.

pathName* delta *deltaX deltaY

Returns a real number indicating the fractional change in the scrollbar setting that corresponds to a given change in thumb position. For example, if the scrollbar is horizontal, the result indicates how much the scrollbar setting must change to move the thumb *deltaX* pixels to the right (*deltaY* is ignored in this case). If the scrollbar is vertical, the result indicates how much the scrollbar setting must change to move the thumb *deltaY* pixels down. The arguments and the result may be zero or negative.

pathName* fraction *x y

Returns a real number between 0 and 1 indicating where the point given by *x* and *y* lies in the trough area of the scrollbar, where 0.0 corresponds to the top or left of the trough and 1.0 corresponds to the bottom or right. *X* and *y* are pixel coordinates relative to the scrollbar widget. If *x* and *y* refer to a point outside the trough, the closest point in the trough is used.

Scrolling commands

When the user interacts with the scrollbar, for example by dragging the thumb, the scrollbar notifies the associated widget that it must change its view. The scrollbar makes the notification by evaluating a Tcl command generated from the scrollbar's **-command** option. The command may take any of the following forms. In each case, *prefix* is the contents of the **-command** option, which usually has a form like **.t yview**

prefix* moveto *fraction

Fraction is a real number between 0 and 1. The widget should adjust its view so that the point given by *fraction* appears at the beginning of the widget. If *fraction* is 0 it refers to the beginning of the document. 1.0 refers to the end of the document, 0.333 refers to a point one-third of the way through the document, and so on.

***prefix* scroll *number* units**

The widget should adjust its view by *number* units. The units are defined in whatever way makes sense for the widget, such as characters or lines in a text widget. *Number* is either 1, which means one unit should scroll off the top or left of the window, or -1, which means that one unit should scroll off the bottom or right of the window.

***prefix* scroll *number* pages**

The widget should adjust its view by *number* pages. It is up to the widget to define the meaning of a page; typically it is slightly less than what fits in the window, so that there is a slight overlap between the old and new views. *Number* is either 1, which means the next page should become visible, or -1, which means that the previous page should become visible.

Widget states

The scrollbar automatically sets the **disabled** state bit. when the entire range is visible (range is 0.0 to 1.0), and clears it otherwise. It also sets the **active** and **pressed** state flags of individual elements, based on the position and state of the mouse pointer.

Example

```
set f [frame .f]
ttk::scrollbar $f.hsb -orient horizontal -command [list $f.t xview]
ttk::scrollbar $f.vsb -orient vertical -command [list $f.t yview]
text $f.t -xscrollcommand [list $f.hsb set] -yscrollcommand [list
    $f.vsb set]
grid $f.t -row 0 -column 0 -sticky nsew
grid $f.vsb -row 0 -column 1 -sticky nsew
grid $f.hsb -row 1 -column 0 -sticky nsew
grid columnconfigure $f 0 -weight 1
grid rowconfigure $f 0 -weight 1
pack $f
```

Styling options

The class name for a **ttk::scrollbar** is **TScrollbar**.

Dynamic states: **active**, **disabled**.

TScrollbar (or more specifically **Vertical.TScrollbar** and **Horizontal.TScrollbar**) styling options that are configurable with ttk::style are:

-arrowcolor *color*

-arrowsize *amount*

-background *color*

-bordercolor *color*

-darkcolor *color* (color of the dark part of the 3D relief)

-foreground *color*

-gripcount *count* (number of lines on the thumb)

-lightcolor *color* (color of the light part of the 3D relief)

-troughcolor *color*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 1994-1996 Sun Microsystems, Inc.
Copyright (c) 2004 Joe English

ttk_separator

Keywords: ttk_separator, widget, separator

Name

ttk::separator - Separator bar

Synopsis

ttk::separator *pathName* ?*options*?

Description

A **ttk::separator** widget displays a horizontal or vertical separator bar.

Standard options
-class, **-cursor**, **-style**, **-takefocus**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options
-orient (orient/Orient)

One of **horizontal** or **vertical**. Specifies the orientation of the separator.

Widget command
Separator widgets support the standard **cget**, **configure**, **identify**, **instate**, and **state** methods. No other widget methods are used.

Styling options
The class name for a **ttk::separator** is **TSeparator**.

TSeparator styling options configurable with ttk::style are:

-background *color*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2004 Joe English

ttk_sizegrip
Keywords: ttk_sizegrip, widget, sizegrip, grow box

Name
ttk::sizegrip - Bottom-right corner resize widget

Synopsis
ttk::sizegrip *pathName* ?*options*?

Description
A **ttk::sizegrip** widget (also known as a *grow box*) allows the user to resize the containing toplevel window by pressing and dragging the grip.

Standard options
-class, **-cursor**, **-style**, **-takefocus**

See the ttk_widget manual entry for details on the standard options.

Widget command
Sizegrip widgets support the standard **cget**, **configure**, **identify**, **instate**, and **state** methods. No other widget methods are used.

Platform-specific notes
On macOS, toplevel windows automatically include a built-in size grip by default. Adding a **ttk::sizegrip** there is harmless, since the built-in grip will just mask the widget.

Examples

Using pack:

```
pack [ttk::frame $top.statusbar] -side bottom -fill x
pack [ttk::sizegrip $top.statusbar.grip] -side right -anchor se
```

Using grid:

```
grid [ttk::sizegrip $top.statusbar.grip] \
    -row $lastRow -column $lastColumn -sticky se
# ... optional: add vertical scrollbar in $lastColumn,
# ... optional: add horizontal scrollbar in $lastRow
```

Bugs

If the containing toplevel's position was specified relative to the right or bottom of the screen (e.g., "**wm geometry ...** *w***x***h***-***x***-***y*" instead of "**wm geometry ...** *w***x***h***+***x***+***y*"), the sizegrip widget will not resize the window.

ttk::sizegrip widgets only support "southeast" resizing.

Styling options

The class name for a **ttk::sizegrip** is **TSizegrip**.

TSizegrip styling options configurable with ttk::style are:

-background *color*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2006 Joe English

ttk_spinbox

Keywords: ttk_spinbox, entry, spinbox, widget, text field

Name

ttk::spinbox - Selecting text field widget

Synopsis

ttk::spinbox *pathName* ?*options*?

Description

A **ttk::spinbox** widget is a ttk::entry widget with built-in up and down buttons that are used to either modify a numeric value or to select among a set of values. The widget implements all the features of the ttk::entry widget including support of the **-textvariable** option to link the value displayed by the widget to a Tcl variable.

Standard options

-class, **-cursor**, **-state**, **-style**, **-takefocus**, **-xscrollcommand**

See the ttk_widget manual entry for details on the standard options.

-validate, **-validatecommand**

See the ttk_entry manual entry for details on the standard options.

Widget-specific options

-command (command/Command)
Specifies a Tcl command to be invoked whenever a spinbutton is invoked.

-format (format/Format)
Specifies an alternate format to use when setting the string value when using the **-from** and **-to** range. This must be a format specifier of the form **%.f**, as it will format a floating-point number.

-from (from/From)
A floating-point value specifying the lowest value for the spinbox. This is used in conjunction with **-to** and **-increment** to set a numerical range.

-increment (increment/Increment)
A floating-point value specifying the change in value to be applied each time one of the widget spin buttons is pressed. The up button applies a positive increment, the down button applies a negative increment.

-to (to/To)
A floating-point value specifying the highest permissible value for the widget. See also **-from** and **-increment**. range.

-values (values/Values)
This must be a Tcl list of values. If this option is set then this will override any range set using the **-from**, **-to** and **-increment** options. The widget will instead use the values specified beginning with the first value.

-wrap (wrap/Wrap)
Must be a proper boolean value. If on, the spinbox will wrap around the values of data in the widget.

Indices

See the ttk::entry manual for information about indexing characters.

Validation

See the ttk::entry manual for information about using the **-validate** and **-validatecommand** options.

Widget command

The following subcommands are possible for spinbox widgets in addition to the commands described for the ttk::entry widget:

***pathName* get**
Returns the spinbox's current value.

pathName* set *value
Set the spinbox string to *value*. If a **-format** option has been configured then this format will be applied. If formatting fails or is not set or the **-values** option has been used then the value is set directly.

Virtual events

The spinbox widget generates a **<>** virtual event when the user presses, and a **<>** virtual event when the user presses.

Styling options

The class name for a **ttk::spinbox** is **TSpinbox**.

Dynamic states: **active**, **disabled**, **focus**, **readonly**.

TSpinbox styling options configurable with ttk::style are:

-arrowcolor *color*

-arrowsize *amount*

-background ***color***
: For backwards compatibility, when using the aqua theme (for macOS), this option behaves as an alias for the **-fieldbackground** provided that no value is specified for **-fieldbackground**. Otherwise it is ignored.

-bordercolor *color*

-darkcolor *color*

-fieldbackground *color*

-foreground *color*

-insertcolor *color*

-insertwidth *amount*

-lightcolor *color*

-padding *padding*

-selectbackground *color*

-selectforeground *color*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2008 Pat Thoyts

ttk_style

Keywords: ttk_style, style, theme, appearance

Name

ttk::style - Manipulate style database

Synopsis

ttk::style *option* ?*args*?

Notes

See also the Tcl'2004 conference presentation, available at
https://tktable.sourceforge.net/tile/tile-tcl2004.pdf

Definitions

Each widget is assigned a *style*, which specifies the set of elements making up the widget and

how they are arranged, along with dynamic and default settings for element options. By default, the style name is the same as the widget's class; this may be overridden by the **-style** option.

A *theme* is a collection of elements and styles which controls the overall look and feel of an application.

Description

The **ttk::style** command takes the following arguments:

ttk::style configure *style* ?*-option* ?*value option value...*? ?
Sets the default value of the specified option(s) in *style*.

ttk::style element *args*
see below ...

ttk::style element create *elementName type* ?*args...*?
Creates a new element in the current theme of type *type*. The only cross-platform built-in element type is *image* (see ttk_image(n)) but themes may define other element types (see **Ttk_RegisterElementFactory**). On suitable versions of Windows an element factory is registered to create Windows theme elements (see ttk_vsapi(n)).

ttk::style element names
Returns the list of elements defined in the current theme.

ttk::style element options *element*
Returns the list of *element*'s options.

ttk::style layout *style* ?*layoutSpec*?
Define the widget layout for style *style*. See Layouts below for the format of *layoutSpec*. If *layoutSpec* is omitted, return the layout specification for style *style*.

ttk::style lookup *style -option* ?*state* ?*default*??
Returns the value specified for *-option* in style *style* in state *state*, using the standard lookup rules for element options. *state* is a list of state names; if omitted, it defaults to all bits off (the "normal" state). If the *default* argument is present, it is used as a fallback value in case no specification for *-option* is found.

ttk::style map *style* ?*-option* { *statespec value...* }?
Sets dynamic values of the specified option(s) in *style*. Each *statespec / value* pair is examined in order; the value corresponding to the first matching *statespec* is used.

ttk::style theme *args*
see below ...

ttk::style theme create *themeName* ?-parent *basedon*? ?-settings *script...* ?
Creates a new theme. It is an error if *themeName* already exists. If **-parent** is specified, the new theme will inherit styles, elements, and layouts from the parent theme *basedon*. If **-settings** is present, *script* is evaluated in the context of the new theme as per **ttk::style theme settings**.

ttk::style theme names
Returns a list of all known themes.

ttk::style theme settings *themeName script*
Temporarily sets the current theme to *themeName*, evaluate *script*, then restore the previous theme. Typically *script* simply defines styles and elements, though arbitrary Tcl code may appear.

ttk::style theme use ?*themeName*?
Without an argument the result is the name of the current theme. Otherwise this command sets the current theme to *themeName*, and refreshes all widgets.

Layouts

A *layout* specifies a list of elements, each followed by one or more options specifying how to arrange the element. The layout mechanism uses a simplified version of the pack geometry manager: given an initial cavity, each element is allocated a parcel. Then the parcel actually used by the element is adjusted within the allocated parcel. Valid options are:

-children { *sublayout...* }
Specifies a list of elements to place inside the element.

-expand *boolean*
Specifies whether the allocated parcel is the entire cavity. If so, simultaneous specification of **-side** is ignored. Defaults to 0.

-side *side*
Specifies which side of the cavity to place the element; one of **left**, **right**, **top**, or **bottom**. For instance, **-side top** allocates the parcel along the top of the cavity having width and height respectively the width of the cavity and the height of the element. If omitted, the allocated parcel is the entire cavity (same effect as **-expand** 1).

-sticky [*nswe*]
Specifies the actual parcel position and size inside the allocated parcel. If specified as an empty string then the actual parcel is centered in the allocated parcel. Default is **nswe**.

For example:

```
ttk::style layout Horizontal.TScrollbar {
    Scrollbar.trough -children {
        Scrollbar.leftarrow -side left
        Scrollbar.rightarrow -side right
        Horizontal.Scrollbar.thumb -side left -sticky ew
    }
}
```

Copyright (c) 2004 Joe English

ttk_treeview

Keywords: ttk_treeview

Name

ttk::treeview - hierarchical multicolumn data display widget

Synopsis

ttk::treeview *pathname* ?*options*?

Description

The **ttk::treeview** widget displays a hierarchical collection of items. Each item has a textual label, an optional image, and an optional list of data values. The data values are displayed in

successive columns after the tree label.

The order in which data values are displayed may be controlled by setting the **-displaycolumns** widget option. The tree widget can also display column headings. Columns may be accessed by number or by symbolic names listed in the **-columns** widget option; see Column identifiers.

Each item is identified by a unique name. The widget will generate item IDs if they are not supplied by the caller. There is a distinguished root item, named **{}**. The root item itself is not displayed; its children appear at the top level of the hierarchy.

Each item also has a list of *tags*, which can be used to associate event bindings with individual items and control the appearance of the item.

Treeview widgets support horizontal and vertical scrolling with the standard **-**[**xy**] **scrollcommand** options and [**xy**]**view** widget commands.

Standard options

-class, **-cursor**, **-takefocus**, **-style**, **-xscrollcommand**, **-yscrollcommand**, **-padding**

See the ttk_widget manual entry for details on the standard options.

Widget-specific options

-columns (columns/Columns)
A list of column identifiers, specifying the number of columns and their names.

-displaycolumns (displayColumns/DisplayColumns)
A list of column identifiers (either symbolic names or integer indices) specifying which data columns are displayed and the order in which they appear, or the string **#all**. If set to **#all** (the default), all columns are shown in the order given.

-height (height/Height)
Specifies the number of rows which should be visible. Note: the requested width is determined from the sum of the column widths.

-selectmode (selectMode/SelectMode)
Controls how the built-in class bindings manage the selection. One of **extended**, **browse**, or **none**.

If set to **extended** (the default), multiple items may be selected. If **browse**, only a single item will be selected at a time. If **none**, the selection will not be changed.

Note that application code and tag bindings can set the selection however they wish, regardless of the value of **-selectmode**.

-show (show/Show)
A list containing zero or more of the following values, specifying which elements of the tree to display.

tree
Display tree labels in column #0.

headings
Display the heading row.

The default is **tree headings**, i.e., show all elements.

NOTE: Column #0 always refers to the tree column, even if **-show tree** is not specified.

Widget command

***pathname* bbox *item* ?*column*?**

Returns the bounding box (relative to the treeview widget's window) of the specified *item* in the form *x y width height*. If *column* is specified, returns the bounding box of that cell. If the *item* is not visible (i.e., if it is a descendant of a closed item or is scrolled offscreen), returns the empty list. If the element is partially visible, the result gives the full area of the element, including any parts that are not visible.

pathname* cget *option

Returns the current value of the specified *option*; see *ttk::widget(n)*.

***pathname* children *item* ?*newchildren*?**

If *newchildren* is not specified, returns the list of children belonging to *item*.

If *newchildren* is specified, replaces *item*'s child list with *newchildren*. Items in the old child list not present in the new child list are detached from the tree. None of the items in *newchildren* may be an ancestor of *item*.

***pathname* column *column* ?*-option* ?*value -option value...*?**

Query or modify the options for the specified *column*. If no *-option* is specified, returns a dictionary of option/value pairs. If a single *-option* is specified, returns the value of that option. Otherwise, the options are updated with the specified values. The following options may be set on each column:

-id *name*

The column name. This is a read-only option. For example, [*$pathname* **column #***n* **-id**] returns the data column associated with display column *#n*.

-anchor *anchor*

Specifies how the text in this column should be aligned with respect to the cell. *Anchor* is one of **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, or **center**.

-minwidth *minwidth*

The minimum width of the column in pixels. The treeview widget will not make the column any smaller than **-minwidth** when the widget is resized or the user drags a column separator. Default is 20 pixels.

-stretch *boolean*

Specifies whether or not the column width should be adjusted when the widget is resized or the user drags a column separator. *Boolean* may have any of the forms accepted by Tcl_GetBoolean. By default columns are stretchable.

-width *width*

The width of the column in pixels. Default is 200 pixels. The specified column width may be changed by Tk in order to honor **-stretch** and/or **-minwidth**, or when the widget is resized or the user drags a column separator.

Use *pathname column #0* to configure the tree column.

***pathname* configure ?*option*? ?*value option value ...*?**

Modify or query widget options; see *ttk::widget(n)*.

pathname* delete *itemList

Deletes each of the items in *itemList* and all of their descendants. The root item may not be deleted. See also: **detach**.

pathname* detach *itemList

Unlinks all of the specified items in *itemList* from the tree. The items and all of their descendants are still present and may be reinserted at another point in the tree with the **move** operation, but will not be displayed until that is done. The root item may not be detached. See also: **delete**.

pathname* exists *item

Returns 1 if the specified *item* is present in the tree, 0 otherwise.

***pathname* focus ?*item*?**

If *item* is specified, sets the focus item to *item*. Otherwise, returns the current focus item, or **{}** if there is none.

***pathname* heading *column* ?*-option* ?*value -option value...*?**

Query or modify the heading options for the specified *column*. Valid options are:

-text *text*

The text to display in the column heading.

-image *imageName*

Specifies an image to display to the right of the column heading.

-anchor *anchor*

Specifies how the heading text should be aligned. One of the standard Tk anchor values.

-command *script*

A script to evaluate when the heading label is pressed.

Use *pathname heading #0* to configure the tree column heading.

pathname* identify *component x y

Returns a description of the specified *component* under the point given by *x* and *y*, or the empty string if no such *component* is present at that position. The values *x* and *y* may have any of the forms acceptable to Tk_GetPixels. The following subcommands are supported:

pathname* identify region *x y

Returns one of:

heading

Tree heading area; use [**pathname identify column** *x y*] to determine the heading number.

separator

Space between two column headings; [**pathname identify column** *x y*] will return the display column identifier of the heading to left of the separator.

tree

The tree area.

cell

A data cell.

pathname **identify item** *x y* Returns the item ID of the item at position *y*.

pathname* identify column *x y

Returns the display column identifier of the cell at position *x*. The tree column has ID **#0**.

pathname* identify element *x y

The element at position *x,y*.

pathname* identify row *x y
Obsolescent synonym for *pathname* **identify item**.

See Column identifiers for a discussion of display columns and data columns.

pathname* index *item
Returns the integer index of *item* within its parent's list of children.

pathname* insert *parent index* ?-id *id*? *options...
Creates a new item. *parent* is the item ID of the parent item, or the empty string **{}** to create a new top-level item. *index* is an integer, or the value **end**, specifying where in the list of *parent*'s children to insert the new item. If *index* is less than or equal to zero, the new node is inserted at the beginning; if *index* is greater than or equal to the current number of children, it is inserted at the end. If **-id** is specified, it is used as the item identifier; *id* must not already exist in the tree. Otherwise, a new unique identifier is generated.

pathname **insert** returns the item identifier of the newly created item. See Item options for the list of available options.

***pathname* instate *statespec* ?*script*?**
Test the widget state; see *ttk::widget(n)*.

***pathname* item *item* ?*-option* ?*value -option value...*?**
Query or modify the options for the specified *item*. If no *-option* is specified, returns a dictionary of option/value pairs. If a single *-option* is specified, returns the value of that option. Otherwise, the item's options are updated with the specified values. See Item options for the list of available options.

pathname* move *item parent index
Moves *item* to position *index* in *parent*'s list of children. It is illegal to move an item under one of its descendants.

If *index* is less than or equal to zero, *item* is moved to the beginning; if greater than or equal to the number of children, it is moved to the end.

pathname* next *item
Returns the identifier of *item*'s next sibling, or **{}** if *item* is the last child of its parent.

pathname* parent *item
Returns the ID of the parent of *item*, or **{}** if *item* is at the top level of the hierarchy.

pathname* prev *item
Returns the identifier of *item*'s previous sibling, or **{}** if *item* is the first child of its parent.

pathname* see *item
Ensure that *item* is visible: sets all of *item*'s ancestors to **-open true**, and scrolls the widget if necessary so that *item* is within the visible portion of the tree.

***pathname* selection ?*selop itemList*?**
If *selop* is not specified, returns the list of selected items. Otherwise, *selop* is one of the following:

pathname* selection set *itemList
itemList becomes the new selection.

pathname* selection add *itemList

Add *itemList* to the selection

pathname* selection remove *itemList
Remove *itemList* from the selection

pathname* selection toggle *itemList
Toggle the selection state of each item in *itemList*.

***pathname* set *item* ?*column*? ?*value*?**
With one argument, returns a dictionary of column/value pairs for the specified *item*. With two arguments, returns the current value of the specified *column*. With three arguments, sets the value of column *column* in item *item* to the specified *value*. See also Column identifiers.

***pathname* state ?*stateSpec*?**
Modify or query the widget state; see *ttk::widget(n)*.

pathName* tag *args...
see below ...

***pathName* tag bind *tagName* ?*sequence*? ?*script*?**
Add a Tk binding script for the event sequence *sequence* to the tag *tagName*. When an X event is delivered to an item, binding scripts for each of the item's **-tags** are evaluated in order as per *bindtags(n)*.

,, and virtual events are sent to the focus item.,, and events are sent to the item under the mouse pointer. No other event types are supported.

The binding *script* undergoes **%**-substitutions before evaluation; see **bind(n)** for details.

***pathName* tag configure *tagName* ?*option*? ?*value option value...*?**
Query or modify the options for the specified *tagName*. If one or more *option/value* pairs are specified, sets the value of those options for the specified tag. If a single *option* is specified, returns the value of that option (or the empty string if the option has not been specified for *tagName*). With no additional arguments, returns a dictionary of the option settings for *tagName*. See Tag options for the list of available options.

***pathName* tag has *tagName* ?*item*?**
If *item* is specified, returns 1 or 0 depending on whether the specified item has the named tag. Otherwise, returns a list of all items which have the specified tag.

***pathName* tag names**
Returns a list of all tags used by the widget.

pathName* tag add *tag items
Adds the specified *tag* to each of the listed *items*. If *tag* is already present for a particular item, then the **-tags** for that item are unchanged.

***pathName* tag remove *tag* ?*items*?**
Removes the specified *tag* from each of the listed *items*. If *items* is omitted, removes *tag* from each item in the tree. If *tag* is not present for a particular item, then the **-tags** for that item are unchanged.

The treeview widget also supports the following generic ttk::widget widget subcommands (see *ttk::widget(n)* for details):

xview yview

Item options

The following item options may be specified for items in the **insert** and **item** widget commands.

-text (text/Text)
The textual label to display for the item.

-image (image**/Image)**
A Tk image, displayed to the left of the label.

-values (values/Values)
The list of values associated with the item.

Each item should have the same number of values as the **-columns** widget option. If there are fewer values than columns, the remaining values are assumed empty. If there are more values than columns, the extra values are ignored.

-open (open/Open)
A boolean value indicating whether the item's children should be displayed (**-open true**) or hidden (**-open false**).

-tags (tags/Tags)
A list of tags associated with this item.

Tag options

The following options may be specified on tags:

-foreground
Specifies the text foreground color.

-background
Specifies the cell or item background color.

-font
Specifies the font to use when drawing text.

-image
Specifies the item image, in case the item's **-image** option is empty.

Tag priority is decided by the creation order: tags created first receive higher priority.

Column identifiers

Column identifiers take any of the following forms:

- A symbolic name from the list of **-columns**.
- An integer *n*, specifying the *n*th data column.
- A string of the form **#***n*, where *n* is an integer, specifying the *n*th display column.

NOTE: Item **-values** may be displayed in a different order than the order in which they are stored.

NOTE: Column #0 always refers to the tree column, even if **-show tree** is not specified.

A *data column number* is an index into an item's **-values** list; a *display column number* is the column number in the tree where the values are displayed. Tree labels are displayed in column #0. If **-displaycolumns** is not set, then data column *n* is displayed in display column **#***n+1*. Again, **column #0 always refers to the tree column**.

Virtual events

The treeview widget generates the following virtual events.

<<TreeviewSelect>>
Generated whenever the selection changes. It might also be generated when selection is affected but not actually changed. Further, multiple selection changes could happen before events can be processed leading to multiple events with the same visible selection.

<<TreeviewOpen>>
Generated just before setting the focus item to **-open true**.

<<TreeviewClose>>
Generated just after setting the focus item to **-open false**.

The **focus** and **selection** widget commands can be used to determine the affected item or items.

Styling options

The class name for a **ttk::treeview** is **Treeview**. The treeview header class name is **Heading**. The treeview item class name is **Item**. The treeview cell class name is **Cell**.

Dynamic states: **disabled**, **selected**.

Treeview styling options configurable with ttk::style are:

-background *color*

-fieldbackground *color*

-font *font*

-foreground *color*

-indent ***amount***
Specifies how far items are indented from their parents. Defaults to 20 pixels. The value may have any of the forms acceptable to Tk_GetPixels.

-rowheight ***amount***
This is the standard height for an item. Defaults to 20 pixels. The value may have any of the forms acceptable to Tk_GetPixels. If **-rowheight** is not set by the style, it is set by measuring an item and a cell layout with the style's settings. This thus picks up the font and any focus ring or padding from the theme's layout. The **-rowheight** may need to be set to make sure that a row is large enough to contain any images.

To adjust the **-rowheight** for the Treeview style, use the following code after tk scaling has been applied. Note that even if you do not support or change tk scaling in your program, your users may have it set in their .wishrc.

```
ttk::style configure Treeview \
     -rowheight [expr {[font metrics font -linespace] + 2}]
```

Heading styling options configurable with ttk::style are:

-background *color*

-font *font*

-relief *relief*

Item styling options configurable with ttk::style are:

-foreground *color*

-indicatormargins *padding*

-indicatorsize *amount*

-padding *padding*

Cell styling options configurable with ttk::style are:

-padding *padding*

Some options are only available for specific themes.

See the ttk::style manual page for information on how to configure ttk styles.

Copyright (c) 2004 Joe English

ttk_vsapi

Keywords: ttk_vsapi, style, theme, appearance, windows

Name

ttk_vsapi - Define a Microsoft Visual Styles element

Synopsis

ttk::style element create *name* **vsapi** *className partId* ?*stateMap*? ?*options*?

Description

The **vsapi** element factory creates a new element in the current theme whose visual appearance is drawn using the Microsoft Visual Styles API which is used in ttk for the MS-Windows native styles. This factory permits any of the Visual Styles parts to be declared as Ttk elements that can then be included in a style layout to modify the appearance of Ttk widgets.

className and *partId* are required parameters and specify the Visual Styles class and part as given in the Microsoft documentation. The *stateMap* may be provided to map Ttk states to Visual Styles API states (see State map).

Options

Valid *options* are:

-padding *padding*
: Specify the element's interior padding. *padding* is a list of up to four integers specifying the left, top, right and bottom padding quantities respectively. If fewer than four elements are specified, *bottom* defaults to *top*, *right* defaults to *left*, and *top* defaults to *left*. In other words, a list of three numbers specify the left, vertical, and right padding; a list of two numbers specify the horizontal and the vertical padding; a single number specifies the same padding all the way around the widget. This option may not be mixed with any other options.

-margins *padding*
: Specifies the elements exterior padding. *padding* is a list of up to four integers specifying the left, top, right and bottom padding quantities respectively. This option may not be mixed with any other options.

-width *width*
Specifies the height for the element. If this option is set then the Visual Styles API will not be queried for the recommended size or the part. If this option is set then **-height** should also be set. The **-width** and **-height** options cannot be mixed with the **-padding** or **-margins** options.

-height *height*
Specifies the height of the element. See the comments for **-width**.

State map

The *stateMap* parameter is a list of ttk states and the corresponding Visual Styles API state value. This permits the element appearance to respond to changes in the widget state such as becoming active or being pressed. The list should be as described for the ttk::style map command but note that the last pair in the list should be the default state and is typically an empty list and 1. Unfortunately all the Visual Styles parts have different state values and these must be looked up either in the Microsoft documentation or more likely in the header files. The original header to use was *tmschema.h*, but in more recent versions of the Windows Development Kit this is *vssym32.h*.

If no *stateMap* parameter is given there is an implicit default map of {{} 1}

Example

Create a correctly themed close button by changing the layout of a ttk::button(n). This uses the WINDOW part WP_SMALLCLOSEBUTTON and as documented the states CBS_DISABLED, CBS_HOT, CBS_NORMAL and CBS_PUSHED are mapped from ttk states.

```
ttk::style element create smallclose vsapi WINDOW 19 \
    {disabled 4 pressed 3 active 2 {} 1}
ttk::style layout CloseButton {CloseButton.smallclose -sticky news}
pack [ttk::button .close -style CloseButton]
```

Change the appearance of a ttk::checkbutton(n) to use the Explorer pin part EBP_HEADERPIN.

```
ttk::style element create pin vsapi EXPLORERBAR 3 {
    {pressed !selected} 3
    {active !selected} 2
    {pressed selected} 6
    {active selected} 5
    {selected} 4
    {} 1
}
ttk::style layout Explorer.Pin {Explorer.Pin.pin -sticky news}
pack [ttk::checkbutton .pin -style Explorer.Pin]
```

Copyright (c) 2008 Pat Thoyts

ttk_widget

Keywords: ttk_widget, state, configure, option

Name

ttk::widget - Standard options and commands supported by Tk themed widgets

Description

This manual describes common widget options and commands.

Standard options

The following options are supported by all Tk themed widgets:

-class (undefined/undefined)
Specifies the window class. The class is used when querying the option database for the window's other options, to determine the default bindtags for the window, and to select the widget's default layout and style. This is a read-only option: it may only be specified when the window is created, and may not be changed with the **configure** widget command.

-cursor (cursor/Cursor)
Specifies the mouse cursor to be used for the widget. See Tk_GetCursor and *cursors(n)* in the Tk reference manual for the legal values. If set to the empty string (the default), the cursor is inherited from the parent widget.

-takefocus (takeFocus/TakeFocus)
Determines whether the window accepts the focus during keyboard traversal. Either **0**, **1**, a command prefix (to which the widget path is appended, and which should return **0** or **1**), or the empty string. See *options(n)* in the Tk reference manual for the full description.

-style (style/Style)
May be used to specify a custom widget style.

Scrollable widget options

The following options are supported by widgets that are controllable by a scrollbar. See *scrollbar(n)* for more information

-xscrollcommand (xScrollCommand/ScrollCommand)
A command prefix, used to communicate with horizontal scrollbars. When the view in the widget's window changes, the widget will generate a Tcl command by concatenating the scroll command and two numbers. Each of the numbers is a fraction between 0 and 1 indicating a position in the document; 0 indicates the beginning, and 1 indicates the end. The first fraction indicates the first information in the widget that is visible in the window, and the second fraction indicates the information just after the last portion that is visible.

Typically the **-xscrollcommand** option consists of the path name of a scrollbar widget followed by "set", e.g. ".x.scrollbar set". This will cause the scrollbar to be updated whenever the view in the window changes.

If this option is set to the empty string (the default), then no command will be executed.

-yscrollcommand (yScrollCommand/ScrollCommand)
A command prefix, used to communicate with vertical scrollbars. See the description of **-xscrollcommand** above for details.

Label options

The following options are supported by labels, buttons, and other button-like widgets:

-compound (compound/Compound)
Specifies how to display the image relative to the text, in the case both **-text** and **-image** are present. If set to the empty string (the default), the rules described in the "Elements" section of *ttk::intro(n)* explain which value is actually used. The other valid values are:

text
Display text only.

image
Display image only.

center
Display text centered on top of image.

top
see below ...

bottom
see below ...

left
see below ...

right
Display image above, below, left of, or right of the text, respectively.

none
Display the image if present, otherwise the text.

-font (font/Font)
Font to use for the text displayed by the widget.

-foreground (textColor/TextColor)
The widget's foreground color. If unspecified, the theme default is used.

-image (image/Image)
Specifies an image to display. This is a list of 1 or more elements. The first element is the default image name. The rest of the list is a sequence of *statespec / value* pairs as per **style map**, specifying different images to use when the widget is in a particular state or combination of states. All images in the list should have the same size.

-padding (padding/Padding)
Specifies the internal padding for the widget. The padding is a list of up to four length specifications *left top right bottom*. If fewer than four elements are specified, *bottom* defaults to *top*, *right* defaults to *left*, and *top* defaults to *left*. In other words, a list of three numbers specify the left, vertical, and right padding; a list of two numbers specify the horizontal and the vertical padding; a single number specifies the same padding all the way around the widget.

-text (text/Text)
Specifies a text string to be displayed inside the widget (unless overridden by **-textvariable** for the widgets supporting this option).

-textvariable (textVariable/Variable)
Specifies the name of a global variable whose value will be used in place of the **-text** resource.

-underline (underline/Underline)
If set, specifies the integer index (0-based) of a character to underline in the text string. The

underlined character is used for mnemonic activation.

-width (width/Width)
If greater than zero, specifies how much space, in character widths, to allocate for the text label. If less than zero, specifies a minimum width. If zero or unspecified, the natural width of the text label is used.

Compatibility options

This option is only available for themed widgets that have "corresponding" traditional Tk widgets.

-state (state/State)
May be set to **normal** or **disabled** to control the **disabled** state bit. This is a write-only option: setting it changes the widget state, but the **state** widget command does not affect the **-state** option.

Commands

pathName* cget *option
Returns the current value of the configuration option given by *option*.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. If *option* is specified with no *value*, then the command returns a list describing the named option: the elements of the list are the option name, database name, database class, default value, and current value. If no *option* is specified, returns a list describing all of the available options for *pathName*.

pathName* identify element *x y
Returns the name of the element under the point given by *x* and *y*, or an empty string if the point does not lie within any element. *x* and *y* are pixel coordinates relative to the widget. Some widgets accept other **identify** subcommands described in these widgets documentation.

***pathName* instate *statespec* ?*script*?**
Test the widget's state. If *script* is not specified, returns 1 if the widget state matches *statespec* and 0 otherwise. If *script* is specified, equivalent to

```
if {[pathName instate stateSpec]} script
```

***pathName* state ?*stateSpec*?**
Modify or inquire widget state. If *stateSpec* is present, sets the widget state: for each flag in *stateSpec*, sets the corresponding flag or clears it if prefixed by an exclamation point. Returns a new state spec indicating which flags were changed:

```
set changes [pathName state spec]
pathName state $changes
```

will restore *pathName* to the original state. If *stateSpec* is not specified, returns a list of the currently-enabled state flags.

pathName* xview *args

This command is used to query and change the horizontal position of the content in the widget's window. It can take any of the following forms:

***pathName* xview**
Returns a list containing two elements. Each element is a real fraction between 0 and 1; together they describe the horizontal span that is visible in the window. For example, if the first element is .2 and the second element is .6, 20% of the widget's content is off-screen to the left, the middle 40% is visible in the window, and 40% of the content is off-screen to the right. These are the same values passed to scrollbars via the **-xscrollcommand** option.

pathName* xview *index
Adjusts the view in the window so that the content given by *index* is displayed at the left edge of the window.

pathName* xview moveto *fraction
Adjusts the view in the window so that the character *fraction* of the way through the content appears at the left edge of the window. *Fraction* must be a fraction between 0 and 1.

pathName* xview scroll *number what
This command shifts the view in the window left or right according to *number* and *what*. *Number* must be an integer. *What* must be either **units** or **pages**. If *what* is **units**, the view adjusts left or right by *number* average-width characters on the display; if it is **pages** then the view adjusts by *number* screenfuls. If *number* is negative then characters farther to the left become visible; if it is positive then characters farther to the right become visible.

pathName* yview *args
This command is used to query and change the vertical position of the content in the widget's window. It can take any of the following forms:

***pathName* yview**
Returns a list containing two elements. Each element is a real fraction between 0 and 1; together they describe the vertical span that is visible in the window. For example, if the first element is .2 and the second element is .6, 20% of the widget's content is off-screen to the top, the middle 40% is visible in the window, and 40% of the content is off-screen to the bottom. These are the same values passed to scrollbars via the **-yscrollcommand** option.

pathName* yview *index
Adjusts the view in the window so that the content given by *index* is displayed at the top edge of the window.

pathName* yview moveto *fraction
Adjusts the view in the window so that the item *fraction* of the way through the content appears at the top edge of the window. *Fraction* must be a fraction between 0 and 1.

pathName* yview scroll *number what
This command shifts the view in the window up or down according to *number* and *what*. *Number* must be an integer. *What* must be either **units** or **pages**. If *what* is **units**, the view adjusts up or down by *number* average-width characters on the display; if it is **pages** then the view adjusts by *number* screenfuls. If *number* is negative then items farther to the top become visible; if it is positive then items farther to the bottom become visible.

Widget states

The widget state is a bitmap of independent state flags. Widget state flags include:

active
The mouse cursor is over the widget and pressing a mouse button will cause some action to occur. (aka "prelight" (Gnome), "hot" (Windows), "hover").

disabled
Widget is disabled under program control (aka "unavailable", "inactive").

focus
Widget has keyboard focus.

pressed
Widget is being pressed (aka "armed" in Motif).

selected
"On", "true", or "current" for things like checkbuttons and radiobuttons.

background
Windows and the Mac have a notion of an "active" or foreground window. The **background** state is set for widgets in a background window, and cleared for those in the foreground window.

readonly
Widget should not allow user modification.

alternate
A widget-specific alternate display format. For example, used for checkbuttons and radiobuttons in the "tristate" or "mixed" state, and for buttons with **-default active**.

invalid
The widget's value is invalid. (Potential uses: scale widget value out of bounds, entry widget value failed validation.)

hover
The mouse cursor is within the widget. This is similar to the **active** state; it is used in some themes for widgets that provide distinct visual feedback for the active widget in addition to the active element within the widget.

user1-user3
Freely usable for other purposes

A *state specification* or *stateSpec* is a list of state names, optionally prefixed with an exclamation point (!) indicating that the bit is off.

Examples

```
set b [ttk::button .b]

# Disable the widget:
$b state disabled

# Invoke the widget only if it is currently pressed and enabled:
$b instate {pressed !disabled} { .b invoke }

# Reenable widget:
$b state !disabled
```

Copyright (c) 2004 Joe English

License

These pages were converted from the original manual pages to Markdown (TIP 700 format). The text is unchanged; only markup without a place in the format (links without a target, layout images) may be lost. For parts under the Mozilla Public License 1.1 (tDOM) this is the description of modifications required by section 3.3.

This collection is not an official publication of the projects. Each part is under the terms of its original, which follow. The copyright notices of the original pages stay with each page.

Tcl license terms

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, ActiveState Corporation and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as

defined in Clause 252.227-7014 (b) (3) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Tk license terms

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, ActiveState Corporation, Apple Inc. and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (b) (3) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Index

A

abort 32, 98
about dialog 667
absolute file name 122
access mode 239
access position 300, 347
add 151
after 10
alias 161, 291, 556
anonymous function 13
antialiasing 667
API 272, 361
appearance 709, 735, 745
append 12, 175, 239, 438
Apple event 667
application 71, 322, 465, 622, 681
application name 666
apply 13
architecture 254, 255
argument 13, 257, 322, 681
arithmetic 98, 340
array 14
aspect ratio 684
assign 175
asynchronous I/O 120, 304, 369, 374
atom 676
attributes 113
auto-exec 176
auto-load 176, 249, 251
auto-loading 291, 556
auto_mkindex 291, 556

B

background error 17, 671
backslash 7
backslash substitution 317
base class 237
beep 386
bell 386
bgerror 17
binary 18, 106
binary code 187, 366
binary search 193
bind 304, 386, 508

binding 386, 397, 475, 508
bindtags 397
bisque 589
bitmap 398, 466
blocking 35, 64, 105, 106, 110, 120, 124, 131, 271
boolean 98, 125, 150, 379
break 32, 287
buffer 124
buffering 106
busy 400
button 403, 696, 697, 715, 725
byte array 106
bytecode 340

C

call 234, 301, 321
cancel 10
canvas 406
carriage return 106
case 32
case conversion 310
catch 33, 287
cd 35
cell 501
chan 35
channel 35, 64, 88, 110, 120, 124, 131, 259, 271, 272, 304, 347, 361
character 310
check 697
checkbutton 432
child interpreter 161, 291, 556
children 676
choice 699
chooseColor 436
chooseDirectory 437
class 48, 73, 237, 577, 676
cleanup 363
clear 438, 619
clipboard 438
clock 50
clone 67
close 64
color 436, 440, 589, 595
color selection 436
colors 440
command 7, 152, 205, 221, 286, 355, 696
command substitution 317

comment 7
compare 98, 310
compiler 340
compress 380
concat 65
concatenate 65, 90
conditional 150
configure 746
connection 304
console 460
container 602, 708, 714
content 602
context 370, 371
continue 66, 287
conversion specifier 127, 296
copy 67
copy files 113
coroutine 68
cpu architecture 254, 255
create 80, 239
ctype 310
current directory 122
cursor 462
cursors 462

D

database 575
date 50
dde 71, 622
debug 209
decompress 380
default 696
define 73, 475
definition 73
deflate 380
deiconify 684
delay 10
delete 286
delete files 113
destroy 465
dialog 436, 437, 466, 491
dict 80
dictionary 80
directory 113, 437
domain name 304
duplication 67
dynamic library 187

E

element 174, 175, 182, 183, 184, 185, 190, 191, 192, 196, 198
element names 14
else 150
encoding 87, 106, 271
end of file 35, 88, 110, 131, 271
end of line 106, 110, 131, 271
ensemble 221, 272, 361
entry 467, 624, 699, 701, 733
environment 340, 672
eof 88
equal 310
error 33, 89, 287, 340, 347, 363, 364
eval 90
evaluate 90
event 136, 369, 374, 386, 397, 475, 508
event handler 120
events 35, 483
exception 33, 89, 287, 347, 363
exec 91
execute 91
exist 132
exit 98
exported 221
expr 98
expression 98, 205

F

false 150
fblocked 105
fconfigure 106
fcopy 110
file 113, 132, 239, 250, 300, 308
file selection dialog 496
fileevent 120
filename 122
filter 80, 106
final 363
flush 124, 369
flushing 106
focus 483, 485
focus model 684
focusNext 485
font 486, 491
font chooser 491
font panel 491
font selection 491

fontchooser 491
for 125
foreach 126, 186
format 18, 127, 438, 619
frame 371, 494, 708, 714
fuzzy comparison 98

G

generator 68
geometry 676, 684
geometry management 590
geometry manager 501, 583, 585, 602
getOpenFile 496
gets 131
glob 132
global 135, 371, 373
grab 499
grid 501, 684
group 684
groupbox 714
grow box 732
gzip 380

H

half-close 64
handle 475
handler 369, 619
height 507, 602, 676
history 136
host 304
http 137

I

i18n 211
ICCCM 619
icon 667, 684
iconify 684
identifier 676
idle 369
idle callback 10
if 150
image 398, 507, 595, 709
incr 151
increment 151
increments 684
index 182, 190, 196, 249, 251, 310
inflate 380
info 152

information 152, 676
input 35
insert 183
integer value 310
interactive 460
internal 221
internationalization 211
internet 137
interp 161
interpreter 152, 322, 460, 681
interpreters 676
introspection 152, 301
iterate 80
iteration 66, 125, 126, 186

J

join 65, 174

K

keyboard 483
keyboard events 400, 499
keyboard traversal 485, 568
keysym 508
keysyms 508

L

l10n 211
label 544, 714
labelframe 546
lambda 13
lappend 175
lassign 175
length 185
level 152, 370, 371
library 176
lindex 182
line 131
linear search 193
linemode 106
linsert 183
list 65, 126, 174, 175, 182, 183, 184, 185, 186, 190, 191, 192, 193, 196, 198, 309
list box 699
listbox 548
llength 185
lmap 186
load 187, 291, 556
loadTk 556

localization 211
location 501, 583, 585, 602
lookup 80
loop 32, 66, 125, 126, 186, 379
lower 557
lrange 190
lrepeat 190
lreplace 191
lreverse 192
lsearch 193
lset 196
lsort 198

M

map 80, 186
mapped 676
match 32, 193, 261, 279, 284, 310, 318
mathfunc 201
mathop 205
memory 209
menu 557, 568, 605, 715
menu bar 568
menubar 568
menubutton 569
message 211, 571
message box 573
messageBox 573
metaclass 48
method 73, 220, 234
method chain 234
method visibility 220
modal 466
modules 351
move files 113
msgcat 211
multiple 175
my 220

N

name 71, 113, 577, 622
namespace 135, 152, 221, 286, 370, 371, 373
network address 304
newline 106, 259
next 234
non-blocking 35, 131, 239
non-existent command 364
nonblocking 64, 105, 106, 110, 120, 124, 271

NSImage 667

O

object 48, 67, 73, 152, 220, 237, 301
obscure 557, 610
offset 35
open 239
operating system 254, 255
operator 205
option 440, 462, 575, 697, 725, 746
option menu 577
optionMenu 577
options 577
order 198
output 35, 124, 259
output channels 460
own 619

P

pack 501, 585
pack-old 583
package 245, 249, 251, 351
packagens 249
packer 583, 585
palette 589
pane 716
panedwindow 590
parcel 583, 585
parent 676
parent interpreter 161, 291, 556
parse 296
parsing 279
path name 676
pattern 132, 193, 279, 284, 310
performance measurement 349
permissions 239
photo 595
pid 250
pipeline 91, 239, 250
pixmap theme 709
pkgMkIndex 251
place 602
platform 106, 254, 255
platform-specific 437
platform_shell 255
pointer events 400, 499
popup 605

portability 122
position 684
POSIX 340
post 568
precision 340
prefix 256, 272, 361
priority 575
private method 220
proc 257
procedure 13, 135, 152, 257, 287, 371, 373
process 98, 239
process identifier 250
prompt 322, 681
propagation 501, 585
public method 220
puts 259
pwd 260

Q

quoting 184, 284, 317

R

radiobutton 606
raise 610
range 190
re_syntax 261
read 110, 131, 271, 303, 355
readable 35, 120
record 136
recursion 321
redirection 91
refchan 272
reflection 272
regexp 279
registry 281
regsub 284
regular expression 32, 193, 261, 279, 284, 318
relative file name 122
remote execution 71, 622
remove 368
rename 286, 355
rename files 113
replace 191, 196
reporting 17, 671
resource management 363
result 287
retrieve 575

return 287
reverse 192, 310
ring 386
root class 237
rubber sheet 602

S

safe 291
safe interpreter 161, 187, 291, 366, 556
scale 611, 727
scan 18, 296
screen 676
script 7, 90, 120, 308, 348, 349
script file 322, 681
scrollbar 614, 728
search 14, 193
security 622
security policy 137
seek 35, 300
seeking 347
selection 437, 438, 619
self 301
send 622, 666
separator 174, 731
serial 239
set 175, 196, 303
shared library 187, 366
shell 322, 681
size 501, 583, 585, 684
sizegrip 732
sleep 10
slider 611, 727
slot 73
socket 137, 304
sort 198
source 291, 308, 556
spinbox 624, 733
split 309
splitting 279
sprintf 127
stack frame 370
stacking order 557, 610
standard option 577
stat 113
state 746
stdio 35
string 127, 193, 261, 279, 309, 310

style 709, 735, 745
sublist 190
subprocess 91, 340
subst 317
substitution 7, 127, 284, 317
switch 318, 577

T

tab 716
table lookup 256
tag 397
tail recursion 321
tailcall 321
target 619
Tcl 7
tclsh 322
tcltest 324
tclvars 340
tcp 304
tell 35, 347
test 324, 379
test harness 324
test suite 324
text 211, 632, 672
text box 699
text field 701, 733
theme 709, 735, 745
throw 347
time 10, 50, 348, 349
timerate 349
title 684
tk 666
tk_mac 667
tkerror 671
tkvars 632, 672
tkwait 673
tm 351
toggle 697
toolkit 681
top-level 483, 485
top-level window 684
toplevel 674, 681
trace 355
transchan 361
transformation 361
translation 106, 110, 211, 271
trough 611, 727

true 150
try 363
ttk_button 696
ttk_checkbutton 697
ttk_combobox 699
ttk_entry 701
ttk_frame 708
ttk_image 709
ttk_intro 710
ttk_label 713
ttk_labelframe 714
ttk_menubutton 715
ttk_notebook 716
ttk_panedwindow 721
ttk_progressbar 723
ttk_radiobutton 725
ttk_scale 727
ttk_scrollbar 728
ttk_separator 731
ttk_sizegrip 732
ttk_spinbox 733
ttk_style 735
ttk_treeview 737
ttk_vsapi 745
ttk_widget 746
type 438, 619
types of images 507

U

unicode 87
units 684
unknown 176, 364
unload 366
unloading 366
unset 355, 368
update 80, 369
uplevel 370
upvar 371
user 113, 340

V

value 151
variable 7, 12, 135, 151, 152, 175, 221, 303, 355, 368, 370, 371, 373, 374, 673
variable substitution 317
variables 340, 672
version 245, 249, 251, 672

virtual event 475
virtual root 676
visibility 673
volume-relative file name 122
vwait 374

W

wait 374, 673
while 379
whitespace 176
widget 403, 406, 432, 467, 494, 544, 546, 548, 557, 569, 571, 590, 606, 611, 614, 624, 632, 674, 696, 697, 699, 701, 708, 714, 715, 725, 727, 728, 731, 732, 733
width 507, 602, 676
window 400, 460, 465, 499, 673, 676
window manager 483, 684
windows 745
winfo 676
wish 681
wm 684
word 176, 310
working directory 35, 260
writable 35
writable. 120
write 259, 303, 355
www 137

Z

zlib 380